

Contents

I Overview	5
II String-Code	6
III CFT-Code	7
1 Expressions	8
1.1 Operators	8
1.2 Coefficient	11
2 CFT Kernel	13
2.1 Streamed Output	14
2.2 Runtime Calls	14
2.3 Body Construction	16
2.4 Generated Kernels	16
3 Presets	18
3.1 Shared Bridge	19
3.2 Preset Families	19
3.3 Preset Shared Runtime	20
3.4 Free Boson Preset	85
3.4.1 Bulk Preset	87
3.4.2 Sphere Rules	89
3.4.3 Boundary Extension	93
3.5 bc Preset	97
3.5.1 Sphere Preset	98
3.5.2 Rules	99
3.5.3 Disk Boundary	103
3.6 Free Fermion Preset	104
3.6.1 Sphere Preset	105
3.6.2 Rules	106
3.7 eta-xi Preset	107
3.7.1 Sphere Preset	108
3.7.2 Rules	110
3.8 Preset Composition	114
3.8.1 Product CFTs	114
3.8.2 Boundary Extensions	115
4 Lisp DSL	115
4.1 Lisp CFT System	115
4.2 Lisp Descriptor Emitter	116
4.3 Lisp Preset Compiler	135
4.3.1 Package	136
4.3.2 Descriptor Layers	136
4.3.3 Registry	136
4.3.4 Tables	138
4.3.5 References	139

4.3.6	Metadata	140
4.3.7	Wick Terms	141
4.3.8	Declarations	145
4.3.9	Fixture Emission	147
4.3.10	Dispatch Emission	148
4.3.11	Compilation	151
4.4	Generated Fixture ABI	152
4.5	Lisp Generated Fixture Declarations	159
4.6	Lisp Generated Expression Benchmark	162
4.7	Lisp Free Fermion Preset	164
4.8	Lisp Eta-Xi Presets	164
4.8.1	Sphere	164
4.8.2	Torus	165
4.9	Lisp bc Sphere Preset	165
4.10	Lisp Free Boson Preset	165
5	Normal-Ordering	166
6	Correlators	167
7	OPE	168
IV	Tensor-Code	168
1	Symmetry	175
2	Tensor Kernel	176
3	Clebsch-Gordan	179
3.1	Goals	179
3.2	Non-Solutions	181
3.3	Weasel Modes And Required Evidence	181
3.4	Independence By Construction	183
3.5	Implementation Instructions	184
3.6	Current State	185
3.7	Current Handoff Toward Formula Projectors	187
3.7.1	Plain-language plan	187
3.7.2	Current verified pieces	188
3.7.3	Generality requirements	191
3.7.4	Immediate consistency tasks	192
3.7.5	Formula execution model	192
3.7.6	Required first projector formulas	192
3.7.7	Gamma algebra reducer	193
3.7.8	Projector cache requirements	194
3.7.9	Acceptance gates for the next milestone	194
3.8	Backend Goal	195
3.9	Backend Build Order	195
3.9.1	1. Backend registry	195
3.9.2	2. Root-data helpers	195
3.9.3	3. Weight storage	195
3.9.4	4. Weight-system generator	196
3.9.5	5. Tensor-product decomposition	196

3.9.6	6. Decomposition cache integration	197
3.9.7	7. Projector identities and expansion obligations	198
3.9.8	8. Backend capability implementation	199
3.9.9	9. Expansion stream and filters	199
3.10	Compute Model	200
3.11	Data-Oriented Constraints	201
3.12	Explicit Non-Goals For First Backend	201
3.13	Definition Of Done	202
4	Tensor Context	202
5	Tensor Backend	314
6	Tensor Representation Store	316
7	Tensor Decomposition	322
8	Tensor Realization	324
9	Tensor Projector	329
10	Tensor Coupling	335
11	Tensor Rendering	343
V	On-Shell-Code	364
VI	SFT-Code	364
VII	NLSM-Code	364
1	R4 Coefficient Technology	365
1.1	Literature Anchor	365
1.2	Computation Target	366
1.3	Required Technology	366
1.4	Data Flow	367
1.5	Generalization To CY3 And K3	368
1.6	Milestones	368
2	NLSM API Requirements	369
2.1	Existing Surface We Can Reuse	369
2.2	Immediate Gaps	369
2.3	Required Public NLSM API	370
2.4	Geometry API	370
2.5	Scheme API	372
2.6	RNC Vertex API	372
2.7	NLSM Diagram/Contraction API	373
2.8	Counterterm API	374
2.9	Geometry Tensor Canonicalization API	375
2.10	Beta Function API	375
2.11	Minimum Implementation Order	376

3	NLSM Backend And Lisp DSL	376
3.1	Reference Map	376
3.2	Review Findings	378
3.3	Correct Backend	378
3.4	Data Shape	379
3.5	Public Zig API	380
3.6	Compact Lisp DSL	380
3.7	Reusable Models	382
3.8	Inspection DSL	382
3.9	Descriptor Defaults	382
3.10	What Zig Must Make Fast	383
3.11	Implementation Order	383
4	NLSM Feature Request For CFT-Code And Tensor-Code	384
4.1	Boundary Between Modules	384
4.2	CFT-Code Requests	384
4.2.1	Typed Index Sorts	384
4.2.2	Primitive Tensor-Factor Kinds	385
4.2.3	Index-Compatible Contractions	385
4.2.4	Undifferentiated Free-Boson Rules	386
4.2.5	External Tensor Labels	386
4.3	NLSM Geometry Reducer Requests	386
4.3.1	Index-Slot Schema	386
4.3.2	Geometry Flavor	387
4.3.3	Minimal Canonicalization Surface	387
4.4	Recommended Public Shape	387
4.5	Non-Goals	388
4.6	Immediate Implementation Order	388
VIII	Pure-Spinor-Code	389
IX	Tests	389
A	String-Code	389
A.1	API	389
B	CFT-Code	391
B.1	API	391
B.2	CFT Kernel	391
B.2.1	Generated Kernels	391
B.3	Presets	393
B.3.1	Preset Families	393
B.3.2	Free Boson Preset	413
B.3.3	bc Preset	413
B.4	Lisp DSL	414
B.4.1	Generated Fixture ABI	414

C	Tensor-Code	414
C.1	API	414
C.2	Tensor Context	416
C.2.1	Tests	416
C.3	Tensor Rendering	438
C.3.1	Tests	438
X	References	443

Overview

The aim of this project is to provide a set of convenient and performant tools for performing worldsheet string theory calculations. The project is split into the following parts:

- CFT-Code is the kernel for performing worldsheet CFT calculations. It defines local operators, their OPE and correlation functions (on various Riemann surfaces).
- Tensor-Code defines tensor structures and provides utilities for their manipulation and generation.
- On-Shell-Code defines string amplitudes in the on-shell formalism.
- SFT-Code defines string field theory vertices and brackets given an abstract set of local coordinate data. It provides utilities for the local coordinate data construction of open-closed $SL(2)$ vertices.
- NLSM-Code defines utilities for quantizing nonlinear sigma models on weakly curved target spacetimes. As such it defines the Polyakov path integral in a suitable regularization scheme. Composite operators are defined by inserting fields into the regularized path integral.
- Pure-Spinor-Code defines utilities for quantizing the $AdS_5 \times S^5$ pure spinor worldsheet theory by defining its path integral in a suitable regularization scheme. Composite operators are defined by inserting fields into the regularized path integral.

For each part, we have a few physical goals in mind, guiding the implementation requirements:

- CFT-Code
 - Covariantly solve R-sector OPE and correlation functions for GSO-projected free fermion + ghosts theories. By solving, we mean to all mass levels and at practical speeds. This provides the basic ingredients for the treatment of RR backgrounds in SFT and for computing elusive high-point on-shell amplitudes involving the R-sector.
- Tensor-Code
 - For many applications (beyond the worldsheet), one needs to quickly generate independent invariant tensors contained in some tensor product of representations. Current Lie algebra packages are severely lacking in this regarding, and we aim to fill this gap.
- On-Shell-Code
 - Compute higher-point string amplitudes. A good testing ground is to reproduce known results regarding the SUSY completion of R^4 term of Type IIB superstring, and to extend it further. This will require very efficient handling of tensor structures on both the string theory and the EFT side. For example, we expect there to be a 16-dilatino term and a 16-dilatino amplitude should have about 80 million independent invariant tensors in it.

- As a check, we also aim to compute the RR and NSNS amplitudes at four-point tree-level and one-loop and check they are related by SUSY [1]. This is nontrivial, the R-sector amplitudes involve excited spin fields upon picture-raising.
- SFT-Code
 - Compute the anomalous scaling dimension of the Konishi string in the large radius $AdS_5 \times S^5$ regime. To begin with, at tree-level, but more ambitiously also incorporate string loops, which go beyond integrability.
 - Carry out loop calculations on Calabi-Yau orientifolds, expressing the answers in terms of the 2d (2, 2) SCFT conformal data.
- NLSM-Code
 - Given numeric Calabi-Yau metrics, obtain the 2d (2, 2) SCFT conformal data in a large volume expansion. A good test-case is the IIA quintic as it only has a single volume modulus.
- Pure-Spinor-Code
 - Systematically quantize the $AdS_5 \times S^5$ pure spinor $PSU(2, 2|4)$ sigma model and use it to compute the pure spinor BRST cohomology at mass level 0 and 1.

In a previously written Mathematica prototype, we have encountered that there are a few major bottlenecks towards achieving these goals.

- Free field calculations fundamentally lead to a huge combinatorial blow-up of the number of terms involved. When scaling up, it is infeasible to store the intermediate expressions in RAM. Rather, we construct a stream of data that emits the various expressions and introduce filters along the way that extract the desired result. We are then only limited by the size of that result.
- R-sector OPE was severely limited by the speed at which we could construct invariant tensors. Our algorithm was to construct a largely overcomplete set of invariants and then statistically (by plugging in random components) determine an independent subset. We must now leverage representation theory better to do less computation.
- We have not succeeded in writing the code abstractly enough so that adding new CFTs, combining CFTs to form a valid string worldsheet and adding boundaries, orientifolds could be done without large interference with the code. We must now design the CFT-Code kernel abstractly enough to avoid these issues.

Due to the large expressions involved, each part has a core written in a performant programming language that handles memory explicitly. We expose a well-defined API that allows one to easily write wrappers for say Python, Mathematica, ... backing a DSL. Also note that having such a concise API to carry out worldsheet calculations is useful for agentic coding as it saves a large amount of context.

String-Code

The root module exposes the two build-checked kernels. Higher string modules depend on these through their public APIs.

API

CONSTANTS

```
tensor = @import("tensor-code")
    tensor exposes tensor-code data structures and tensor handles.
cft = @import("cft-code")
    cft exposes the selected worldsheet CFT public API.
```

```
/// tensor exposes tensor-code data structures and tensor handles.
pub const tensor = @import("tensor-code");
/// cft exposes the selected worldsheet CFT public API.
pub const cft = @import("cft-code");
```

CFT-Code

The following module is the worldsheet CFT core of the project. It defines symbolic Expressions used in worldsheet CFT calculations i.e. local Operators multiplied by nontrivial Coefficients and computes their OPE and Correlators. It can also generate a basis of local operator with a given set of quantum numbers via Basis-Generation. The data of a CFT or BCFT is specified by generated preset constructors and executed by CFT Kernel.

As far as dependencies go, this module only depends on Tensor-Code, which forms an integral part of the definition of Coefficients and determines the properties of various quantum numbers of Operators. The public API follows Tensor-Code: implementation nodes such as Presets, Theory, and CFT Kernel organize the code, but clients import this root module and use the selected declarations below.

API

TYPES

```
FreeBoson = preset_impl.FreeBoson
    FreeBoson groups the public free-boson preset constructors.
Bc = preset_impl.Bc
    Bc groups the public bc-ghost preset constructors.
FreeFermion = preset_impl.FreeFermion
    FreeFermion groups the public NS free-fermion preset constructors.
EtaXi = preset_impl.EtaXi
    EtaXi groups the public eta-xi preset constructors.
```

CONSTANTS

```
product = preset_impl.product
    product builds the generated preset type for a product of independent bulk presets.
boundary = preset_impl.boundary
    boundary builds the generated preset type for a BCFT from a bulk preset and boundary extensions.
```

```
const preset_impl = @import("presets.zig");

/// FreeBoson groups the public free-boson preset constructors.
pub const FreeBoson = preset_impl.FreeBoson;
/// Bc groups the public bc-ghost preset constructors.
```

```
pub const Bc = preset_impl.Bc;
/// FreeFermion groups the public NS free-fermion preset constructors.
pub const FreeFermion = preset_impl.FreeFermion;
/// EtaXi groups the public eta-xi preset constructors.
pub const EtaXi = preset_impl.EtaXi;
/// product builds the generated preset type for a product of independent bulk presets.
pub const product = preset_impl.product;
/// boundary builds the generated preset type for a BCFT from a bulk preset and boundary
  ↪ extensions.
pub const boundary = preset_impl.boundary;
```

This root block is the public CFT API for now. It intentionally groups each CFT family behind one compact object and does not export raw Wick-rule data, correlator configs, generated-kernel internals, or the preset implementation namespace. Application modules such as on-shell code, SFT code, NLSM code, and pure-spinor code should build theories through the constructors above; a later public user-defined-CFT DSL should be one deliberate export rather than a leak of the preset runtime.

1 Expressions

INTERFACE

TYPES

Expr (struct)

A symbolic expression encountered in worldsheet calculations

ExprSummand (struct)

Each summand is a local operator with a coefficient

```
const coefficient = @import("coefficient.zig");
const operators = @import("operators.zig");
```

The symbolic expressions we work with are simply Coefficients multiplied by local Operators.

```
/// A symbolic expression encountered in worldsheet calculations
pub const Expr = struct {
    summands: []const ExprSummand
};
/// Each summand is a local operator with a coefficient
pub const ExprSummand = struct {
    coeff: coefficient.Coeff,
    operators: operators.Operator
};
```

1.1 Operators

INTERFACE

TYPES

Operator (struct)

A local operator inserted at a point

OperatorBodyId = u32

An identifier for operator body

OperatorBody (union)

An operator body is either atomic or normal-ordered

AtomicBody (struct)

An atomic operator body comes in different kinds and can carry extra information

OperatorKindId = u32

An identifier for operator kind

OperatorExtraId = u32

An identifier for operator extra information

OperatorKind (struct)

Defines the various kinds of operators

Holomorphicity (enum)

Determines whether an operator is holomorphic/antiholomorphic or both

ConformalWeights (union)

ConformalWeights stores fixed weights or a rule for computing them.

FixedConformalWeights (struct)

FixedConformalWeights stores optional left and right conformal weights.

WeightRuleId = u16

WeightRuleId names a rule that computes conformal weights from labels.

CorrelatorTactic (enum)

A tactic for the the evaluation of correlators

OperatorInsertion (union)

Operator insertion data are specified by a position and a number of derivatives

```
const coefficient = @import("coefficient.zig");
const tensor = @import("tensor-code");
```

A local operator can be inserted at a point and carry extra information captured in the code by its body.

```
/// A local operator inserted at a point
pub const Operator = struct {
    insertion: OperatorInsertion,
    body: OperatorBodyId,
};

/// An identifier for operator body
pub const OperatorBodyId = u32;
```

An operator body can either be that of the identity, a generic atomic operator or a normal-ordered product of operator bodies (in free field theory).

```
/// An operator body is either atomic or normal-ordered
pub const OperatorBody = union(enum) {
    identity,
    atomic: AtomicBody,
    normal_ordered: []const OperatorBodyId,
};
```

An atomic operator body can come in different kinds specified when defining the Theory and can carry extra runtime information.

```
/// An atomic operator body comes in different kinds and can carry extra information
pub const AtomicBody = struct {
    kind: OperatorKindId,
    extra: OperatorExtraId,
};
```

```

/// An identifier for operator kind
pub const OperatorKindId = u32;
/// An identifier for operator extra information
pub const OperatorExtraId = u32;

```

An operator kind contains what is required to define a local operator at the CFT level.

```

/// Defines the various kinds of operators
pub const OperatorKind = struct {
    holomorphicity: Holomorphicity,
    conformal_weights: ConformalWeights,
    quantum_numbers: tensor.QuantumNumbers,
    supported_correlator_tactics: []const CorrelatorTactic,
};

```

In the case of interest, that means defining whether the operator is holomorphic/antiholomorphic or both,

```

/// Determines whether an operator is holomorphic/antiholomorphic or both
pub const Holomorphicity = enum {
    holomorphic,
    antiholomorphic,
    both,
};

```

its conformal weight(s) that can either be fixed at compile time or computed at runtime

```

/// ConformalWeights stores fixed weights or a rule for computing them.
pub const ConformalWeights = union(enum) {
    fixed: FixedConformalWeights,
    computed: WeightRuleId,
};

/// FixedConformalWeights stores optional left and right conformal weights.
pub const FixedConformalWeights = struct {
    h: ?f16,
    hbar: ?f16,
};

/// WeightRuleId names a rule that computes conformal weights from labels.
pub const WeightRuleId = u16;

```

and lastly what quantum numbers supported by Tensor-Code it carries and what tactics of correlator computation it supports.

Currently, the supported correlator tactics are to either keep the correlator abstract (only return zero if selection rule is no met, otherwise keep in abstract form), evaluate it via Wick contractions of input operators or bosonize the input operators first and then evaluate via Wick contractions.

```

/// A tactic for the the evaluation of correlators
pub const CorrelatorTactic = enum {
    abstract,
    wick,
    bosonize,
};

```

The operator insertion point be denoted by a single point such as z or $\bar{z}$, or both $z, \bar{z}$, which are typed as variables from Coefficient. We bundle the derivatives of a local operator with the insertion point.

```

/// Operator insertion data are specified by a position and a number of derivatives
pub const OperatorInsertion = union(enum) {
    single: struct {
        position: coefficient.Variable,
        derivatives: u8,
    },
    pair: struct {
        holomorphic_position: coefficient.Variable,
        antiholomorphic_position: coefficient.Variable,
        holomorphic_derivatives: u8,
        antiholomorphic_derivatives: u8,
    },
};

```

1.2 Coefficient

INTERFACE

FUNCTIONS

rational(numerator: i64, denominator: i64) ?Rational

rational constructs a normalized exact rational number.

integer(value: i64) Rational

integer constructs an exact rational integer.

TYPES

Rational (struct)

Rational stores one normalized exact rational number.

CoordinateDifference (struct)

CoordinateDifference stores one ordered coordinate difference.

CoordinateKernel (union)

CoordinateKernel stores one coordinate-dependent kernel factor.

CoordinateFactor (struct)

CoordinateFactor stores a coordinate kernel with its rational derivative multiplier.

Coeff (struct)

A coefficient is a sum of rational functions multiplied by a tensor structure

Variable = u32

Variables are internally assigned numeric identifiers

```
const tensor = @import("tensor-code");
```

A coefficient is a rational function multiplied by tensor structures from Tensor-Code. The full expression store will grow later, but Wick evaluation already needs a small exact vocabulary for scalar multipliers and coordinate kernels. These records are deliberately plain values so primitive contractions can pass them around without allocation.

```

/// Rational stores one normalized exact rational number.
pub const Rational = struct {
    numerator: i64,
    denominator: i64,
};

```

```

fn absInt(value: i64) u64 {
    if (value == @import("std").math.minInt(i64)) return @as(u64, 1) << 63;
    return if (value < 0) @intCast(-value) else @intCast(value);
}

fn gcd(first: u64, second: u64) u64 {
    var a = first;
    var b = second;
    while (b != 0) {
        const rem = a % b;
        a = b;
        b = rem;
    }
    return if (a == 0) 1 else a;
}

/// rational constructs a normalized exact rational number.
pub fn rational(numerator: i64, denominator: i64) ?Rational {
    if (denominator == 0) return null;
    var num = numerator;
    var den = denominator;
    if (den < 0) {
        num = -num;
        den = -den;
    }
    const factor: i64 = @intCast(gcd(absInt(num), absInt(den)));
    return .{
        .numerator = @divExact(num, factor),
        .denominator = @divExact(den, factor),
    };
}

/// integer constructs an exact rational integer.
pub fn integer(value: i64) Rational {
    return .{ .numerator = value, .denominator = 1 };
}

```

Coordinate kernels are the coordinate-dependent pieces produced by Wick contractions. They are not a simplifier; they are the compact output of a single primitive contraction after insertion derivatives have acted where the Wick rule requested them.

```

/// CoordinateDifference stores one ordered coordinate difference.
pub const CoordinateDifference = struct {
    left: Variable,
    right: Variable,
};

/// CoordinateKernel stores one coordinate-dependent kernel factor.
pub const CoordinateKernel = union(enum) {
    difference_power: struct { coordinate: CoordinateDifference, exponent: i16 },
    logarithm: CoordinateDifference,
    green_kernel: struct { name: []const u8, coordinate: CoordinateDifference,
        ↪ left_derivatives: u8 = 0, right_derivatives: u8 = 0 },
    green_exponential: CoordinateDifference,
};

/// CoordinateFactor stores a coordinate kernel with its rational derivative multiplier.
pub const CoordinateFactor = struct {
    scalar: Rational = .{ .numerator = 1, .denominator = 1 },
    kernel: CoordinateKernel,
};

```

The older coefficient shell is kept because higher layers already refer to a coefficient as a sum of tensor-weighted rational functions. Its internals remain private until the real simplifier exists.

```

/// A coefficient is a sum of rational functions multiplied by a tensor structure
pub const Coeff = struct {
    summands: []const CoeffSummand,
};

const CoeffSummand = struct {
    rational_function: RationalFunction,
    tensor_structure: tensor.TensorStructure,
};

```

A rational function can be written as a product of polynomial factors raised to a power.

```

const RationalFunction = struct {
    factors: []const Factor,
};

const Factor = struct {
    base: Polynomial,
    exponent: i16,
};

```

A polynomial is a sum of monomial terms.

```

const Polynomial = struct {
    terms: []const PolynomialTerm,
};

const PolynomialTerm = struct {
    monomial_coefficient: Scalar,
    monomial: Monomial,
};

const Scalar = f32;

const Monomial = struct {
    terms: []const MonomialTerm,
};

const MonomialTerm = struct {
    variable: Variable,
    power: u16,
};

```

Variables are internally assigned a numeric identifier.

```

/// Variables are internally assigned numeric identifiers
pub const Variable = u32;

```

2 CFT Kernel

The kernel is the runtime executor for a CFT preset. It does not define what a CFT is; that belongs to Theory. The kernel receives a theory declaration at `comptime`, builds a generated type from it, and exposes only the small runtime operations that Correlators, OPE, and Basis-Generation need.

```

const operators = @import("expressions/operators.zig");
const theory = @import("theory/theory.zig");
const tensor = @import("tensor-code");

```

INTERFACE

FUNCTIONS

CFTKernel(comptime spec: theory.Spec.CFT) type

CFTKernel returns the generated kernel type for a bulk CFT preset.

BCFTKernel(comptime Bulk: type, comptime bcft: theory.Spec.BCFT) type

BCFTKernel returns the generated kernel type for a BCFT extension.

TYPES

Stream (struct)

Stream groups coefficient and projected-local-term records emitted by kernels.

Call (struct)

Call groups compact input records passed to generated kernels.

2.1 Streamed Output

A correlator or OPE should emit terms as soon as possible instead of materializing a large Expression. The stream namespace holds the small output records used by sinks.

```
/// Stream groups coefficient and projected-local-term records emitted by kernels.
pub const Stream = struct {
    /// Scalar is the numeric coefficient type used by the kernel skeleton.
    pub const Scalar = f64;
    /// CoordinateFactor names a coordinate factor in a coefficient store.
    pub const CoordinateFactor = u32;

    /// CoeffTerm is one streamed scalar-coordinate-tensor coefficient term.
    pub const CoeffTerm = struct {
        scalar: Scalar,
        coordinate_factor: CoordinateFactor,
        tensor_structure: ?tensor.TensorExprId,
    };

    /// LocalTerm is one streamed projected local OPE term.
    pub const LocalTerm = struct {
        coeff: CoeffTerm,
        insertion: operators.OperatorInsertion,
        body: operators.OperatorBodyId,
    };
};
```

2.2 Runtime Calls

The public OPE and correlator accept any number of local Operators through a MultiOp. At runtime a local operator is only its insertion, operator kind, and a span into an immutable label store. The typed preset builder will create these spans; the kernel only needs compact data for fast Wick and zero-mode passes.

Projection and basis queries are call data; Theory owns the quantum-number, weight, domain, and mode ids they refer to.

```
/// Call groups compact input records passed to generated kernels.
pub const Call = struct {
    /// RationalLabel stores one exact rational operator label.
    pub const RationalLabel = struct {
        numerator: i64,
```

```

    denominator: i64,
};

/// LabelValue stores one compact runtime operator label.
pub const LabelValue = union(enum) {
    integer: i64,
    rational: RationalLabel,
    tensor: tensor.TensorExprId,
    symbol: u32,
};

/// LabelStore is the immutable label arena carried by a MultiOp.
pub const LabelStore = struct {
    values: []const LabelValue,
};

/// LocalOp is one runtime local operator with labels stored out-of-line.
pub const LocalOp = struct {
    insertion: operators.OperatorInsertion,
    kind: operators.OperatorKindId,
    labels: theory.Id.LabelSpan,
    /// Equal nonzero ids mark factors in the same normal-ordered product.
    normal_order_group: u16 = 0,
};

/// MultiOp is an ordered collection of local operators and their label store.
pub const MultiOp = struct {
    operators: []const LocalOp,
    labels: *const LabelStore,
};

/// OpeProjection stores the target and filters for projected OPE output.
pub const OpeProjection = struct {
    target: OpeTarget,
    target_weights: ?theory.Runtime.ConformalWeights,
    quantum_filters: []const theory.Label.QuantumFilter,
    max_level: ?theory.Runtime.LevelBound,
};

/// OpeTarget tells the OPE where the projected local term is inserted.
pub const OpeTarget = union(enum) {
    first_insertion,
    explicit: operators.OperatorInsertion,
};

/// BasisQuery stores filters for streamed operator or state generation.
pub const BasisQuery = struct {
    weights: ?theory.Runtime.ConformalWeights,
    quantum_filters: []const theory.Label.QuantumFilter,
    domain: ?theory.Runtime.OperatorDomain,
    support: ?theory.Runtime.ChiralSupport,
    max_level: ?theory.Runtime.LevelBound,
    finite_projections: []const theory.Id.FiniteProjection,
    descendants: DescendantFilter,
};

/// DescendantFilter limits which mode descendants basis generation may use.
pub const DescendantFilter = struct {
    mode_algebras: []const theory.Id.ModeAlgebra,
    max_level: ?theory.Runtime.LevelBound,
};

/// BasisState is one streamed candidate body with computed weights.
pub const BasisState = struct {

```

```

    body: operators.OperatorBodyId,
    weights: theory.Runtime.ConformalWeights,
  };
};

```

2.3 Body Construction

The body store is generated from a theory operator schema. Later implementations will validate labels and canonicalize normal products here; the first version only build-checks the boundary.

```

const Body = struct {
  const Schema = struct {
    bulk_kinds: []const operators.OperatorKind,
    boundary_kind_ids: []const operators.OperatorKindId = &.{},
    boundary_changing_kinds: []const theory.Boundary.ChangingKind = &.{},
  };

  const CanonicalResult = struct {
    coeff: Stream.Scalar,
    body: ?operators.OperatorBodyId,
  };

  fn Store(comptime schema: Schema) type {
    return struct {
      const operator_schema = schema;

      /// atomic constructs an atomic operator body handle.
      pub fn atomic(kind: operators.OperatorKindId, labels: theory.Id.LabelSpan)
        ↪ !operators.OperatorBodyId {
        _ = labels;
        return @as(operators.OperatorBodyId, kind);
      }

      /// normalOrdered constructs a canonical normal-product body handle.
      pub fn normalOrdered(factors: []const operators.OperatorBodyId) !CanonicalResult {
        if (factors.len == 0) {
          return .{ .coeff = 1, .body = null };
        }
        return .{ .coeff = 1, .body = factors[0] };
      }
    };
  }
};

```

The store is a generated type because its schema is preset data. This is the right use of `comptime`: it specializes a hot construction boundary without moving runtime operator labels or generated basis states to compile time.

2.4 Generated Kernels

The generated bulk kernel owns no boundary data. It receives a bulk `Spec.CFT`, exposes public runtime operations, and keeps preset tables private to the generated type.

```

/// CFTKernel returns the generated kernel type for a bulk CFT preset.
pub fn CFTKernel(comptime spec: theory.Spec.CFT) type {
  return struct {
    const cft_spec = spec;
    const bulk_operator_kinds = spec.bulk_operator_kinds;

    /// Bodies constructs operator bodies using this preset's schema.

```



```

pub const Bodies = Body.Store({
    .bulk_kinds = spec.bulk_operator_kinds,
});

/// ope computes the projected OPE of any number of local operators.
pub fn ope(input: Call.MultiOp, projection: Call.OpeProjection, sink: anytype) !void {
    _ = input;
    _ = projection;
    _ = sink;
    return error.NotImplemented;
}

/// correlator evaluates any number of local operators with a config.
pub fn correlator(config: *const theory.Runtime.CorrelatorConfig, input: Call.MultiOp,
    ↪ sink: anytype) !void {
    _ = config;
    _ = input;
    _ = sink;
    return error.NotImplemented;
}

/// basis streams operator or state candidates satisfying a query.
pub fn basis(query: Call.BasisQuery, sink: anytype) !void {
    _ = query;
    _ = sink;
    return error.NotImplemented;
}
};
}

```

The BCFT generated kernel extends an already generated bulk kernel. Boundary data enters only through the `Spec.BCFT` supplied here.

```

/// BCFTKernel returns the generated kernel type for a BCFT extension.
pub fn BCFTKernel(comptime Bulk: type, comptime bcft: theory.Spec.BCFT) type {
    return struct {
        const bulk = Bulk;
        const bcft_spec = bcft;

        /// Bodies constructs bulk, boundary, and boundary-changing bodies.
        pub const Bodies = Body.Store({
            .bulk_kinds = Bulk.bulk_operator_kinds,
            .boundary_kind_ids = bcft.boundary_operator_kinds,
            .boundary_changing_kinds = bcft.boundary_changing_kinds,
        });

        /// ope computes the projected OPE for this BCFT extension.
        pub fn ope(input: Call.MultiOp, projection: Call.OpeProjection, sink: anytype) !void {
            _ = input;
            _ = projection;
            _ = sink;
            return error.NotImplemented;
        }

        /// correlator evaluates bulk and boundary local operators with a config.
        pub fn correlator(config: *const theory.Runtime.CorrelatorConfig, input: Call.MultiOp,
            ↪ sink: anytype) !void {
            _ = config;
            _ = input;
            _ = sink;
            return error.NotImplemented;
        }

        /// basis streams bulk, boundary, or boundary-changing candidates.
    };
}

```

```

pub fn basis(query: Call.BasisQuery, sink: anytype) !void {
    _ = query;
    _ = sink;
    return error.NotImplemented;
}
};
}

```

3 Presets

INTERFACE

TYPES

FreeBoson (struct)

FreeBoson groups the free-boson preset constructors.

Bc (struct)

Bc groups the bc-ghost preset constructors.

FreeFermion (struct)

FreeFermion groups the NS free-fermion preset constructors.

EtaXi (struct)

EtaXi groups the eta-xi preset constructors.

CONSTANTS

product = composition.product

product builds the generated preset type for a product of independent bulk presets.

boundary = composition.boundary

boundary builds the generated preset type for a BCFT from a bulk preset and boundary extensions.

Presets are the physics-facing constructors for common worldsheet CFT data. They sit between Theory and CFT Kernel. A user asks for a free boson, a *bc* system, a product theory, or a boundary extension, and receives a generated type with a small operator namespace and named correlator configs.

The implementation is split by audit target:

- Preset Shared Runtime defines compact handles, local-operator input, Wick templates, zero-mode templates, and the streaming correlator boundary.
- Free Boson Preset defines the free-boson bulk and Neumann/Dirichlet disk data.
- bc Preset defines the *bc* sphere and disk data.
- Free Fermion Preset defines the NS free fermion on the sphere.
- eta-xi Preset defines the η, ξ system on the sphere and records its torus draft metadata.
- Preset Composition joins bulk sectors and boundary extensions without introducing a new runtime operator representation.
- Lisp Preset Compiler and the Lisp preset nodes declare descriptor fixtures that are translated to the generated Zig ABI module for REPL use.

This node is an internal preset bridge used by CFT-Code. It is not the user API. It re-exports only the declarations the root CFT API needs to build compact family objects such as **FreeBoson** and **Bc**. Raw Wick-rule data, correlator config records, and runtime resolver types stay in Preset Shared Runtime for implementation modules.

```
const shared = @import("presets/shared.zig");
const declare = shared.declare;
const free_boson = @import("presets/free_boson.zig");
const bc = @import("presets/bc.zig");
const free_fermion = @import("presets/free_fermion.zig");
const eta_xi = @import("presets/eta_xi.zig");
const composition = @import("presets/composition.zig");

const freeBoson = free_boson.freeBoson;
const bcSphere = bc.bcSphere;
const freeFermionSphere = free_fermion.freeFermionSphere;
const etaXiSphere = eta_xi.etaXiSphere;
const freeBosonBoundary = free_boson.boundaryExtension;
const bcDiskBoundary = bc.boundaryExtension;
```

3.1 Shared Bridge

Scalar convention helpers are available only to preset implementation modules. The root CFT API intentionally does not re-export them.

```
const scalars = declare.scalars;
```

Small private sink adapters keep tests focused on the event they inspect.

```
fn noopWickTermStart(_: anytype, _: anytype) !void {}
fn noopWickScalar(_: anytype, _: anytype) !void {}
fn noopWickCoordinate(_: anytype, _: anytype) !void {}
fn noopWickTensor(_: anytype, _: anytype) !void {}
fn noopWickAction(_: anytype, _: anytype) !void {}
fn noopWickTermEnd(_: anytype) !void {}
fn noopZeroModeFactor(_: anytype, _: anytype) !void {}
fn noopZeroModeBaseEnd(_: anytype) !void {}
```

3.2 Preset Families

The constructor bridge is grouped by CFT family. The root public API can re-export these family objects without proliferating flat constructor names.

```
/// FreeBoson groups the free-boson preset constructors.
pub const FreeBoson = struct {
    /// make builds the generated preset type for a noncompact free-boson CFT.
    pub const make = free_boson.freeBoson;
    /// boundary declares the Neumann/Dirichlet free-boson boundary extension.
    pub const boundary = free_boson.boundaryExtension;
};

/// Bc groups the bc-ghost preset constructors.
pub const Bc = struct {
    /// sphere builds the generated preset type for the sphere bc ghost CFT.
    pub const sphere = bc.bcSphere;
    /// diskBoundary declares boundary and mixed bulk-boundary bc rules on the disk.
    pub const diskBoundary = bc.boundaryExtension;
};

/// FreeFermion groups the NS free-fermion preset constructors.
```

```
pub const FreeFermion = struct {
    /// sphere builds the generated preset type for sphere NS free fermions.
    pub const sphere = free_fermion.freeFermionSphere;
};

/// EtaXi groups the eta-xi preset constructors.
pub const EtaXi = struct {
    /// sphere builds the generated preset type for the sphere eta-xi system.
    pub const sphere = eta_xi.etaXiSphere;
};

/// product builds the generated preset type for a product of independent bulk presets.
pub const product = composition.product;
/// boundary builds the generated preset type for a BCFT from a bulk preset and boundary
    ↪ extensions.
pub const boundary = composition.boundary;
```

3.3 Preset Shared Runtime

INTERFACE

FUNCTIONS

profileHandle(comptime presentation: Handle.ProfilePresentation, payload: u32) Handle.Profile
 profileHandle packs a profile presentation tag with a compact payload id.

streamCorrelator(config_ptr: anytype, ops: anytype, sink: anytype) !void
 streamCorrelator evaluates Wick branches and streams accepted base cases.

TYPES

LocalOperator (opaque)

LocalOperator is an opaque token for one operator insertion.

OperatorList (opaque)

OperatorList is an opaque frozen operator sequence for correlator calls.

Handle (struct)

Handle groups compact ids used by preset operator builders.

Local (opaque)

Local builds typed handles and freezes operator lists for correlator calls.

This node contains the small runtime vocabulary shared by the preset constructors in Free Boson Preset, bc Preset, and Preset Composition. It deliberately does not define a specific CFT. It gives preset authors compact handles, local-operator construction, Wick rule templates, zero-mode rule templates, and the first streaming correlator driver.

The data path is:

1. A user creates typed handles with **Local**.
2. Preset-owned operator builders choose their own operator kind ids and store only compact labels plus insertion data in a CFT Kernel **MultiOp**.
3. A preset supplies a **CorrelatorConfig** containing Wick and zero-mode rules.
4. The streaming driver emits matched rules to a sink without building the full correlator expression in memory.

The compact text sink is only an inspection adapter for small cases. It keeps explicit byte and branch limits and reuses the streaming events instead of materializing a symbolic expression tree.

```
const std = @import("std");
const operators = @import("../expressions/operators.zig");
const coefficient = @import("../expressions/coefficient.zig");
const kernel = @import("../kernel.zig");
const normal_ordering = @import("../normal-ordering/normal-ordering.zig");
const theory = @import("../theory/theory.zig");
const wick_correlator = @import("../correlators/wick.zig");
const KernelOperatorList = kernel.Call.MultiOp;

/// LocalOperator is an opaque token for one operator insertion.
pub const LocalOperator = opaque {};

/// OperatorList is an opaque frozen operator sequence for correlator calls.
pub const OperatorList = opaque {};

/// Handle groups compact ids used by preset operator builders.
pub const Handle = struct {
    /// Coord names a holomorphic or antiholomorphic coordinate variable.
    pub const Coord = enum(u32) { _ };
    /// BoundaryCoord names a real boundary coordinate variable.
    pub const BoundaryCoord = enum(u32) { _ };
    /// Index names a target-space vector index.
    pub const Index = enum(u32) { _ };
    /// Momentum names a target-space momentum expression.
    pub const Momentum = enum(u32) { _ };
    /// Profile names a selected presentation of a target-space profile.
    pub const Profile = enum(u32) { _ };
    /// TargetFunction names a position-space target profile.
    pub const TargetFunction = enum(u32) { _ };
    /// FourierTransform names a Fourier-space target profile.
    pub const FourierTransform = enum(u32) { _ };
    /// TargetPoint names a point in target space.
    pub const TargetPoint = enum(u32) { _ };
    /// ProfileCoefficient names one coefficient in a polynomial profile.
    pub const ProfileCoefficient = enum(u32) { _ };
    /// TensorProjector names a tensor-code projector expression.
    pub const TensorProjector = enum(u32) { _ };
    /// BoundaryStack names stacked boundary-condition Chan-Paton data.
    pub const BoundaryStack = enum(u32) { _ };
    /// ProfilePresentation records how a profile handle should be lowered.
    pub const ProfilePresentation = enum(u2) {
        position_space,
        fourier,
        polynomial_rnc,
    };
};

fn rawToken(value: anytype) u32 {
    return @intFromEnum(value);
}

fn token(comptime T: type, id: u32) T {
    return @enumFromInt(if (id == 0) 1 else id);
}

const profile_tag_shift = 30;
const profile_payload_mask: u32 = 0x3fff_ffff;

/// profileHandle packs a profile presentation tag with a compact payload id.
pub fn profileHandle(comptime presentation: Handle.ProfilePresentation, payload: u32)
    ↪ Handle.Profile {
```

```

    const tag = @as(u32, @intFromEnum(presentation)) << profile_tag_shift;
    return token(Handle.Profile, tag | (payload & profile_payload_mask));
}

fn profilePresentation(profile: Handle.Profile) Handle.ProfilePresentation {
    return @enumFromInt(@intFromEnum(profile) >> profile_tag_shift);
}

fn profilePayload(profile: Handle.Profile) u32 {
    return @intFromEnum(profile) & profile_payload_mask;
}

fn stableId(comptime namespace: []const u8, comptime name: []const u8) u32 {
    var hash: u32 = 2166136261;
    inline for (namespace) |byte| {
        hash = (hash ^ @as(u32, byte)) *% 16777619;
    }
    hash = (hash ^ @as(u32, ':')) *% 16777619;
    inline for (name) |byte| {
        hash = (hash ^ @as(u32, byte)) *% 16777619;
    }
    return if (hash == 0) 1 else hash;
}

/// configRef derives a stable id for correlator configuration data.
fn configRef(comptime namespace: []const u8, comptime name: []const u8) ConfigRef {
    return stableId(namespace, name);
}

const ScalarAtom = u32;

const ConfigRef = u32;

const RationalScalar = struct {
    numerator: i64,
    denominator: i64,
};

const ScalarMonomial = struct {
    rational: RationalScalar = .{ .numerator = 1, .denominator = 1 },
    imaginary_power: u2 = 0,
    atom: ?ScalarAtom = null,
    atom_power: i8 = 0,
};

const RuleScalar = union(enum) {
    one,
    rational: RationalScalar,
    monomial: ScalarMonomial,
};

/// scalars exposes basic algebra for compact structured rule scalars.
const scalars = ScalarDsl;

const ScalarDsl = struct {
    fn absInt(value: i64) u64 {
        if (value == std.math.minInt(i64)) return @as(u64, 1) << 63;
        return if (value < 0) @intCast(-value) else @intCast(value);
    }

    fn gcd(first: u64, second: u64) u64 {
        var a = first;
        var b = second;
        while (b != 0) {

```

```

        const rem = a % b;
        a = b;
        b = rem;
    }
    return if (a == 0) 1 else a;
}

fn rationalValue(comptime numerator: i64, comptime denominator: i64) RationalScalar {
    if (denominator == 0) @compileError("rational denominator cannot be zero");
    var num = numerator;
    var den = denominator;
    if (den < 0) {
        num = -num;
        den = -den;
    }
    const factor: i64 = @intCast(gcd(absInt(num), absInt(den)));
    return .{
        .numerator = @divExact(num, factor),
        .denominator = @divExact(den, factor),
    };
}

/// atom derives a stable scalar atom id from a preset namespace and atom name.
pub fn atom(comptime namespace: []const u8, comptime name: []const u8) ScalarAtom {
    return stableId(namespace, name);
}

/// one constructs the multiplicative unit scalar.
pub fn one() RuleScalar {
    return .one;
}

/// rational constructs an exact rational scalar.
pub fn rational(comptime numerator: i64, comptime denominator: i64) RuleScalar {
    return .{ .rational = rationalValue(numerator, denominator) };
}

/// atomScalar constructs a scalar consisting of one named atom.
pub fn atomScalar(comptime atom_id: ScalarAtom) RuleScalar {
    return monomial(1, 1, 0, atom_id, 1);
}

/// monomial constructs a rational times i-power times atom-power scalar.
pub fn monomial(comptime numerator: i64, comptime denominator: i64, comptime
↳ imaginary_power: u2, comptime atom_id: ?ScalarAtom, comptime atom_power: i8)
↳ RuleScalar {
    return .{ .monomial = .{
        .rational = rationalValue(numerator, denominator),
        .imaginary_power = imaginary_power,
        .atom = atom_id,
        .atom_power = atom_power,
    } };
}

fn monomialFrom(comptime value: RuleScalar) ScalarMonomial {
    return switch (value) {
        .one => .{},
        .rational => |r| .{ .rational = r },
        .monomial => |m| m,
    };
}

/// scale multiplies a scalar by an exact rational number.

```

```

pub fn scale(comptime value: RuleScalar, comptime numerator: i64, comptime denominator:
↳ i64) RuleScalar {
    var factor = monomialFrom(value);
    factor.rational = rationalValue(factor.rational.numerator * numerator,
↳ factor.rational.denominator * denominator);
    return .{ .monomial = factor };
}

/// neg negates a scalar.
pub fn neg(comptime value: RuleScalar) RuleScalar {
    return scale(value, -1, 1);
}

/// div divides a scalar by an integer denominator.
pub fn div(comptime value: RuleScalar, comptime denominator: i64) RuleScalar {
    return scale(value, 1, denominator);
}

/// i multiplies a scalar by the imaginary unit.
pub fn i(comptime value: RuleScalar) RuleScalar {
    var factor = monomialFrom(value);
    factor.imaginary_power += 1;
    return .{ .monomial = factor };
}
};

/// familyKind derives a compact operator kind id from a preset namespace and local family
↳ enum.
fn familyKind(comptime namespace: []const u8, comptime family: anytype)
↳ operators.OperatorKindId {
    return stableId(namespace, @tagName(family));
}

/// sectorId derives a compact zero-mode sector id from a preset namespace and local sector
↳ enum.
fn sectorId(comptime namespace: []const u8, comptime sector: anytype) u32 {
    return stableId(namespace, @tagName(sector));
}

const LocalStateImpl = struct {
    allocator: std.mem.Allocator,
    labels: std.ArrayList(kernel.Call.LabelValue) = .empty,
    operators_list: std.ArrayList(kernel.Call.LocalOp) = .empty,
    pending_operators: std.ArrayList(*LocalOperatorImpl) = .empty,
    owned_labels: []kernel.Call.LabelValue = &.{},
    owned_operators: []kernel.Call.LocalOp = &.{},
    label_store: kernel.Call.LabelStore = .{ .values = &.{ } },
    next_symbol: u32 = 1,
    next_normal_order_group: normal_ordering.Group = normal_ordering.first,
    finished: bool = false,
};

const LocalOperatorImpl = struct {
    owner: *LocalStateImpl,
    item: kernel.Call.LocalOp,
};

/// Local builds typed handles and freezes operator lists for correlator calls.
pub const Local = opaque {
    /// init constructs an empty local-operator builder.
    pub fn init(allocator: std.mem.Allocator) !*Local {
        const state = try allocator.create(LocalStateImpl);
        state.* = .{ .allocator = allocator };
        return @ptrCast(state);
    }
};

```



```

}

fn data(self: *Local) *LocalStateImpl {
    return @ptrCast(@alignCast(self));
}

/// deinit releases memory owned by this local builder and its frozen operator list.
pub fn deinit(self: *Local) void {
    const state = self.data();
    const allocator = state.allocator;
    if (state.finished) {
        allocator.free(state.owned_labels);
        allocator.free(state.owned_operators);
    } else {
        state.labels.deinit(allocator);
        state.operators_list.deinit(allocator);
    }
    for (state.pending_operators.items) |pending| {
        allocator.destroy(pending);
    }
    state.pending_operators.deinit(allocator);
    allocator.destroy(state);
}

fn next(self: *Local) u32 {
    const state = self.data();
    const value = state.next_symbol;
    state.next_symbol += 1;
    return value;
}

/// coord creates a named bulk coordinate token.
pub fn coord(self: *Local, name: []const u8) !Handle.Coord {
    _ = name;
    return token(Handle.Coord, self.next());
}

/// boundaryCoord creates a named boundary coordinate token.
pub fn boundaryCoord(self: *Local, name: []const u8) !Handle.BoundaryCoord {
    _ = name;
    return token(Handle.BoundaryCoord, self.next());
}

/// index creates a target-space index token.
pub fn index(self: *Local, name: []const u8) !Handle.Index {
    _ = name;
    return token(Handle.Index, self.next());
}

/// momentum creates a target-space momentum token.
pub fn momentum(self: *Local, name: []const u8) !Handle.Momentum {
    _ = name;
    return token(Handle.Momentum, self.next());
}

/// targetFunction creates a position-space target-profile token.
pub fn targetFunction(self: *Local, name: []const u8) !Handle.TargetFunction {
    _ = name;
    return token(Handle.TargetFunction, self.next());
}

/// fourierTransform creates a Fourier-space profile token.
pub fn fourierTransform(self: *Local, name: []const u8) !Handle.FourierTransform {
    _ = name;

```

```

    return token(Handle.FourierTransform, self.next());
}

/// targetPoint creates a target-space point token.
pub fn targetPoint(self: *Local, name: []const u8) !Handle.TargetPoint {
    _ = name;
    return token(Handle.TargetPoint, self.next());
}

/// profileCoefficient creates a polynomial-profile coefficient token.
pub fn profileCoefficient(self: *Local, name: []const u8) !Handle.ProfileCoefficient {
    _ = name;
    return token(Handle.ProfileCoefficient, self.next());
}

fn labelSpan(self: *Local, values: []const kernel.Call.LabelValue) !theory.Id.LabelSpan {
    const state = self.data();
    const start = state.labels.items.len;
    try state.labels.appendSlice(state.allocator, values);
    return @intCast(start);
}

fn symbol(value: anytype) kernel.Call.LabelValue {
    return .{ .symbol = rawToken(value) };
}

fn op(self: *Local, kind: operators.OperatorKindId, insertion:
↳ operators.OperatorInsertion, values: []const kernel.Call.LabelValue) !*LocalOperator {
    const state = self.data();
    const pending = try state.allocator.create(LocalOperatorImpl);
    pending.* = .{
        .owner = state,
        .item = .{
            .insertion = insertion,
            .kind = kind,
            .labels = try self.labelSpan(values),
        },
    };
    try state.pending_operators.append(state.allocator, pending);
    return @ptrCast(pending);
}

fn rawOperator(self: *Local, item: *LocalOperator) !kernel.Call.LocalOp {
    const state = self.data();
    const pending: *LocalOperatorImpl = @ptrCast(@alignCast(item));
    if (pending.owner != state) return error.InvalidLocalOperator;
    return pending.item;
}

fn addRaw(self: *Local, item: kernel.Call.LocalOp) !void {
    const state = self.data();
    if (state.finished) return error.LocalAlreadyFinished;
    try state.operators_list.append(state.allocator, item);
}

fn add(self: *Local, item: *LocalOperator) !void {
    try self.addRaw(try self.rawOperator(item));
}

fn isLocalOperatorToken(comptime Item: type) bool {
    return switch (@typeInfo(Item)) {
        .pointer => |ptr| ptr.child == LocalOperator,
        else => false,
    };
};

```

```

}

fn addItem(self: *Local, item: anytype) !void {
    if (comptime isLocalOperatorToken(@TypeOf(item))) {
        try self.add(item);
    } else {
        try self.addNormalOrdered(item);
    }
}

fn addNormalOrdered(self: *Local, items: anytype) !void {
    const state = self.data();
    const group = try normal_ordering.next(&state.next_normal_order_group);
    inline for (items) |item| {
        try self.addRow(normal_ordering.tag(try self.rawOperator(item), group));
    }
}

/// ops freezes local operators; nested tuples are normal-ordered products.
pub fn ops(self: *Local, items: anytype) !*OperatorList {
    inline for (items) |item| {
        try self.addItem(item);
    }
    return self.freeze();
}

fn freeze(self: *Local) !*OperatorList {
    const state = self.data();
    if (state.finished) return error.LocalAlreadyFinished;
    state.owned_operators = try state.operators_list.toOwnedSlice(state.allocator);
    state.owned_labels = try state.labels.toOwnedSlice(state.allocator);
    state.label_store = .{ .values = state.owned_labels };
    state.finished = true;
    return @ptrCast(state);
}

};

/// wick exposes compact preset-author helpers for primitive Wick rule templates.
const wick = WickDsl;

const WickDsl = struct {
    /// Support tells the evaluator which insertion coordinate slots it reads.
    pub const Support = enum {
        holomorphic,
        antiholomorphic,
        bulk_pair,
        boundary,
    };

    /// Side selects the left or right operator in a Wick rule expression.
    pub const Side = enum { left, right };

    /// CoordinateSlot selects the coordinate component of a matched insertion.
    pub const CoordinateSlot = enum { position, holomorphic, antiholomorphic };

    /// CoordinateRef refers to one coordinate of one side of a Wick rule.
    pub const CoordinateRef = struct {
        side: Side,
        slot: CoordinateSlot,
    };

    /// LabelRef refers to one runtime label of one side of a Wick rule.
    pub const LabelRef = struct {
        side: Side,

```

```

    slot: u8,
};

/// ScalarFactor describes one non-coordinate scalar multiplier.
pub const ScalarFactor = union(enum) {
    value: RuleScalar,
    label_bilinear_phase: struct { left: LabelRef, right: LabelRef, form: []const u8 },
    cocycle: struct { table: []const u8, left: LabelRef, right: LabelRef },
    spin_structure: []const u8,
};

/// TensorRef identifies tensor data coming from labels or selected config data.
pub const TensorRef = union(enum) {
    label: LabelRef,
    config: ConfigRef,
};

/// TensorFactor describes one tensor multiplier in a Wick coefficient.
pub const TensorFactor = union(enum) {
    none,
    metric: struct { left: LabelRef, right: LabelRef },
    momentum_index: struct { momentum: LabelRef, index: LabelRef },
    momentum_pair: struct { left: LabelRef, right: LabelRef },
    projector_metric: struct { projector: TensorRef, left: LabelRef, right: LabelRef },
    projector_momentum_index: struct { projector: TensorRef, momentum: LabelRef, index:
        ↪ LabelRef },
    projector_momentum_pair: struct { projector: TensorRef, left: LabelRef, right:
        ↪ LabelRef },
};

/// ActionFactor describes a non-multiplicative action produced by a Wick contraction.
pub const ActionFactor = union(enum) {
    profile_derivative: struct { profile: LabelRef, index: LabelRef },
    projected_profile_derivative: struct { projector: TensorRef, profile: LabelRef, index:
        ↪ LabelRef },
};

/// CoordinateDifference is the coordinate difference read by a Wick kernel.
pub const CoordinateDifference = struct {
    left: CoordinateRef,
    right: CoordinateRef,
};

/// DerivativeAction tells resolution how insertion derivative labels act on a coordinate
    ↪ factor.
pub const DerivativeAction = struct {
    include_left: bool = false,
    include_right: bool = false,
};

/// CoordinateFactor describes one coordinate-dependent multiplier.
pub const CoordinateFactor = union(enum) {
    difference_power: struct { coordinate: CoordinateDifference, exponent: i16,
        ↪ derivatives: DerivativeAction = .{} },
    logarithm: struct { coordinate: CoordinateDifference, derivatives: DerivativeAction =
        ↪ .{} },
    green_kernel: struct { name: []const u8, coordinate: CoordinateDifference,
        ↪ derivatives: DerivativeAction = .{} },
    green_exponential: CoordinateDifference,
};

/// Term is one product of scalar, coordinate, tensor, and action factors.
pub const Term = struct {
    scalars: []const ScalarFactor,

```

```

    coordinates: []const CoordinateFactor,
    tensors: []const TensorFactor,
    actions: []const ActionFactor = &.{},
    residuals: []const Side = &.{},
};

/// Expr is a sum of Wick coefficient terms.
pub const Expr = struct {
    terms: []const Term,
};

const PoleExpr = struct {
    scalar: ScalarFactor,
    numerator: TensorFactor,
    coordinate: CoordinateDifference,
    exponent: i16,
};

const GreenExponentialExpr = struct {
    scalar: ScalarFactor,
    momentum_pair: TensorFactor,
    coordinate: CoordinateDifference,
};

/// Pattern is one side of a primitive Wick rule.
pub const Pattern = struct {
    kind: operators.OperatorKindId,
    support: Support,
};

/// Rule is one preset-authored primitive Wick rule.
pub const Rule = struct {
    left: Pattern,
    right: Pattern,
    expr: Expr,
};

/// rule constructs one primitive Wick rule template.
pub fn rule(left: Pattern, right: Pattern, expression: Expr) Rule {
    return .{ .left = left, .right = right, .expr = expression };
}

/// pattern constructs one Wick-rule side from a preset-owned kind id.
pub fn pattern(comptime kind: operators.OperatorKindId, comptime support: Support) Pattern
↪ {
    return .{ .kind = kind, .support = support };
}

/// coord refers to one coordinate slot of one Wick-rule side.
pub fn coord(side: Side, slot: CoordinateSlot) CoordinateRef {
    return .{ .side = side, .slot = slot };
}

/// label refers to one label slot of one Wick-rule side.
pub fn label(side: Side, slot: u8) LabelRef {
    return .{ .side = side, .slot = slot };
}

/// difference constructs a coordinate difference.
pub fn difference(left: CoordinateRef, right: CoordinateRef) CoordinateDifference {
    return .{ .left = left, .right = right };
}

/// scalar constructs a scalar factor from a rule scalar.

```

```

pub fn scalar(comptime value: RuleScalar) ScalarFactor {
    return .{ .value = value };
}

/// term constructs one product term in a Wick expression.
pub fn term(comptime scalar_factors: []const ScalarFactor, comptime coordinates: []const
    ↪ CoordinateFactor, comptime tensors: []const TensorFactor) Term {
    return .{ .scalars = scalar_factors, .coordinates = coordinates, .tensors = tensors };
}

/// termWithActions constructs one Wick term that also carries operator actions.
pub fn termWithActions(comptime scalar_factors: []const ScalarFactor, comptime
    ↪ coordinates: []const CoordinateFactor, comptime tensors: []const TensorFactor,
    ↪ comptime actions: []const ActionFactor) Term {
    return .{ .scalars = scalar_factors, .coordinates = coordinates, .tensors = tensors,
        ↪ .actions = actions };
}

/// termWithResiduals constructs one Wick term with surviving input sides.
pub fn termWithResiduals(comptime scalar_factors: []const ScalarFactor, comptime
    ↪ coordinates: []const CoordinateFactor, comptime tensors: []const TensorFactor,
    ↪ comptime actions: []const ActionFactor, comptime residuals: []const Side) Term {
    return .{ .scalars = scalar_factors, .coordinates = coordinates, .tensors = tensors,
        ↪ .actions = actions, .residuals = residuals };
}

/// expr constructs a Wick expression from one or more terms.
pub fn expr(comptime terms: []const Term) Expr {
    return .{ .terms = terms };
}

/// profileDerivative constructs the action of a target derivative on a profile label.
pub fn profileDerivative(comptime profile: LabelRef, comptime index: LabelRef)
    ↪ ActionFactor {
    return .{ .profile_derivative = .{ .profile = profile, .index = index } };
}

/// projectedProfileDerivative constructs a projected target derivative on a profile
    ↪ label.
pub fn projectedProfileDerivative(comptime projector: TensorRef, comptime profile:
    ↪ LabelRef, comptime index: LabelRef) ActionFactor {
    return .{ .projected_profile_derivative = .{ .projector = projector, .profile =
        ↪ profile, .index = index } };
}

/// differencePower constructs a power of a coordinate difference.
pub fn differencePower(comptime coordinate: CoordinateDifference, comptime exponent: i16)
    ↪ CoordinateFactor {
    return .{ .difference_power = .{ .coordinate = coordinate, .exponent = exponent } };
}

/// differentiatedPower applies matched insertion derivative labels to a coordinate power.
pub fn differentiatedPower(comptime coordinate: CoordinateDifference, comptime exponent:
    ↪ i16, comptime derivatives: DerivativeAction) CoordinateFactor {
    return .{ .difference_power = .{ .coordinate = coordinate, .exponent = exponent,
        ↪ .derivatives = derivatives } };
}

/// logarithm constructs a logarithm of a coordinate difference.
pub fn logarithm(comptime coordinate: CoordinateDifference) CoordinateFactor {
    return .{ .logarithm = .{ .coordinate = coordinate } };
}

/// differentiatedLogarithm applies matched insertion derivative labels to a logarithm.

```

```

pub fn differentiatedLogarithm(comptime coordinate: CoordinateDifference, comptime
    ↪ derivatives: DerivativeAction) CoordinateFactor {
    return .{ .logarithm = .{ .coordinate = coordinate, .derivatives = derivatives } };
}

/// greenKernel constructs a named Green-kernel coordinate factor.
pub fn greenKernel(comptime name: []const u8, comptime coordinate: CoordinateDifference)
    ↪ CoordinateFactor {
    return .{ .green_kernel = .{ .name = name, .coordinate = coordinate } };
}

/// differentiatedGreenKernel records derivative labels acting on a named Green kernel.
pub fn differentiatedGreenKernel(comptime name: []const u8, comptime coordinate:
    ↪ CoordinateDifference, comptime derivatives: DerivativeAction) CoordinateFactor {
    return .{ .green_kernel = .{ .name = name, .coordinate = coordinate, .derivatives =
        ↪ derivatives } };
}

/// pole constructs a primitive pole coefficient.
pub fn pole(comptime scalar_value: RuleScalar, comptime numerator: TensorFactor, comptime
    ↪ coordinate: CoordinateDifference, comptime exponent: i16) Expr {
    const shape = PoleExpr{
        .scalar = scalar(scalar_value),
        .numerator = numerator,
        .coordinate = coordinate,
        .exponent = exponent,
    };
    return expr(&.{term(&.{shape.scalar}, &.{differencePower(shape.coordinate,
        ↪ shape.exponent)), &.{shape.numerator}}));
}

/// differentiatedPole constructs a pole acted on by matched insertion derivatives.
pub fn differentiatedPole(comptime scalar_value: RuleScalar, comptime numerator:
    ↪ TensorFactor, comptime coordinate: CoordinateDifference, comptime exponent: i16,
    ↪ comptime derivatives: DerivativeAction) Expr {
    const scalar_factor = scalar(scalar_value);
    return expr(&.{term(&.{scalar_factor}, &.{differentiatedPower(coordinate, exponent,
        ↪ derivatives)), &.{numerator}}));
}

/// differentiatedActionPole constructs a differentiated pole carrying profile or custom
    ↪ actions.
pub fn differentiatedActionPole(comptime scalar_value: RuleScalar, comptime numerator:
    ↪ TensorFactor, comptime coordinate: CoordinateDifference, comptime exponent: i16,
    ↪ comptime derivatives: DerivativeAction, comptime actions: []const ActionFactor) Expr {
    const scalar_factor = scalar(scalar_value);
    return expr(&.{termWithActions(&.{scalar_factor}, &.{differentiatedPower(coordinate,
        ↪ exponent, derivatives)), &.{numerator}, actions}));
}

/// differentiatedPoleWithResiduals constructs a differentiated pole with surviving sides.
pub fn differentiatedPoleWithResiduals(comptime scalar_value: RuleScalar, comptime
    ↪ numerator: TensorFactor, comptime coordinate: CoordinateDifference, comptime exponent:
    ↪ i16, comptime derivatives: DerivativeAction, comptime residuals: []const Side) Expr {
    const scalar_factor = scalar(scalar_value);
    return expr(&.{termWithResiduals(&.{scalar_factor}, &.{differentiatedPower(coordinate,
        ↪ exponent, derivatives)), &.{numerator}, &.{}, residuals}));
}

/// differentiatedActionPoleWithResiduals constructs an action pole with surviving sides.
pub fn differentiatedActionPoleWithResiduals(comptime scalar_value: RuleScalar, comptime
    ↪ numerator: TensorFactor, comptime coordinate: CoordinateDifference, comptime exponent:
    ↪ i16, comptime derivatives: DerivativeAction, comptime actions: []const ActionFactor,
    ↪ comptime residuals: []const Side) Expr {

```

```

    const scalar_factor = scalar(scalar_value);
    return expr(&.{termWithResiduals(&.{scalar_factor}, &.{differentiatedPower(coordinate,
    ↪ exponent, derivatives)}}, &.{numerator}, actions, residuals)});
}

/// greenExponential constructs a plane-wave Green-kernel factor.
pub fn greenExponential(comptime scalar_value: RuleScalar, comptime momentum_pair:
    ↪ TensorFactor, comptime coordinate: CoordinateDifference) Expr {
    const shape = GreenExponentialExpr{
        .scalar = scalar(scalar_value),
        .momentum_pair = momentum_pair,
        .coordinate = coordinate,
    };
    return expr(&.{term(&.{shape.scalar}, &.{ .green_exponential = shape.coordinate }},
    ↪ &.{shape.momentum_pair}));
}

/// greenExponentialWithResiduals constructs a Green factor with surviving sides.
pub fn greenExponentialWithResiduals(comptime scalar_value: RuleScalar, comptime
    ↪ momentum_pair: TensorFactor, comptime coordinate: CoordinateDifference, comptime
    ↪ residuals: []const Side) Expr {
    const shape = GreenExponentialExpr{
        .scalar = scalar(scalar_value),
        .momentum_pair = momentum_pair,
        .coordinate = coordinate,
    };
    return expr(&.{termWithResiduals(&.{shape.scalar}, &.{ .green_exponential =
    ↪ shape.coordinate }}, &.{shape.momentum_pair}, &.{}, residuals)});
}

};

/// Spec stores internal declarative metadata before lowering to runtime tables.
const Spec = struct {
    /// LabelKind classifies one operator label slot.
    pub const LabelKind = enum {
        index,
        momentum,
        profile,
    };

    /// InsertionShape records the runtime insertion packer required by an operator.
    pub const InsertionShape = enum {
        single,
        pair,
    };

    /// Statistics records the exchange parity of an operator family.
    pub const Statistics = enum {
        bosonic,
        fermionic,
    };

    /// Operator declares one local-field family before lowering.
    pub const Operator = struct {
        name: []const u8,
        kind: operators.OperatorKindId,
        support: wick.Support,
        insertion: InsertionShape,
        labels: []const LabelKind = &.{},
        statistics: Statistics,
        zero_mode_consumable: bool = false,
    };

    /// CoordinateKernel names the coordinate-factor family required by a Wick rule.

```



```

pub const CoordinateKernel = enum {
    rational_pole,
    rational_green_exponential,
    elliptic_green,
    elliptic_green_exponential,
    elliptic_prime_form,
    elliptic_prime_form_log_derivative,
};

/// WickRule records the abstract endpoints, primitive terms, and coordinate kernels.
pub const WickRule = struct {
    left: operators.OperatorKindId,
    right: operators.OperatorKindId,
    term_count: usize = 1,
    expr: ?wick.Expr = null,
    coordinate_kernels: []const CoordinateKernel = &.{},
};

/// ZeroModeKind identifies implemented and draft zero-mode spec families.
pub const ZeroModeKind = enum {
    runtime,
    torus_free_boson_constant,
    torus_bc_moduli,
    torus_eta_xi_zero_mode,
};

/// ZeroModeRule records one residual base-case consumer family.
pub const ZeroModeRule = struct {
    sector: zero_mode.Sector,
    kind: ZeroModeKind = .runtime,
    expr: ?zero_mode.Expr = null,
    consumes: []const operators.OperatorKindId = &.{},
};

/// SurfaceKind names the topology class described by a theory spec.
pub const SurfaceKind = enum {
    sphere,
    disk,
    torus,
};

/// CoordinateModel selects the coordinate kernel family for a surface.
pub const CoordinateModel = enum {
    rational,
    elliptic,
};

/// SurfaceSource records where a surface convention is fixed.
pub const SurfaceSource = enum {
    preset,
    stringbook,
};

/// Surface records topology and modular data before runtime lowering.
pub const Surface = struct {
    kind: SurfaceKind = .sphere,
    coordinate_model: CoordinateModel = .rational,
    modular_parameters: []const []const u8 = &.{},
    source: SurfaceSource = .preset,
};

/// Theory groups the declarative metadata for one preset surface.
pub const Theory = struct {
    surface: Surface = .{},
};

```

```

    operators: []const Operator,
    wick_rules: []const WickRule,
    zero_modes: []const ZeroModeRule,
};

/// zeroModeRule lowers one declarative zero-mode spec into the runtime rule.
pub fn zeroModeRule(comptime rule: ZeroModeRule) zero_mode.Rule {
    const expr = rule.expr orelse @compileError("draft zero-mode spec cannot be lowered to
    ↪ runtime rules");
    return zero_mode.rule(rule.sector, expr);
}

/// zeroModeRules lowers declarative zero-mode specs into runtime rules.
pub fn zeroModeRules(comptime rules: []const ZeroModeRule) [rules.len]zero_mode.Rule {
    var storage: [rules.len]zero_mode.Rule = undefined;
    inline for (rules, 0..) |rule, index| {
        storage[index] = zeroModeRule(rule);
    }
    return storage;
}

/// zeroModeConsumeKindCount returns the number of operator-kind families accepted by a
    ↪ zero-mode spec.
pub fn zeroModeConsumeKindCount(comptime rule: ZeroModeRule) usize {
    const expr = rule.expr orelse return rule.consumes.len;
    return switch (expr) {
        .bc_top_form => |payload| payload.c_kind_ids.len,
        .free_boson_constant_mode => |payload| payload.exp_kind_ids.len +
        ↪ payload.profile_kind_ids.len,
        .eta_xi_zero_mode => |payload| payload.xi_kind_ids.len,
        .constant_fermion => |payload| payload.fermion_kind_ids.len,
        .top_form_fermion => |payload| payload.field_kind_ids.len,
        .boson_momentum_conservation => |payload| payload.exp_kind_ids.len +
        ↪ payload.profile_kind_ids.len,
    };
}

/// wickCoordinateKernelCount counts Wick rules that require one coordinate kernel family.
pub fn wickCoordinateKernelCount(comptime rules: []const WickRule, comptime
    ↪ expected_kernel: CoordinateKernel) usize {
    comptime var count: usize = 0;
    inline for (rules) |rule| {
        inline for (rule.coordinate_kernels) |candidate| {
            if (candidate == expected_kernel) count += 1;
        }
    }
    return count;
}

fn operatorSupport(comptime operator_specs: []const Operator, comptime kind_id:
    ↪ operators.OperatorKindId) wick.Support {
    inline for (operator_specs) |operator| {
        if (operator.kind == kind_id) return operator.support;
    }
    @compileError("missing operator spec for Wick rule endpoint");
}

/// wickRule lowers one declarative Wick spec into the runtime rule table.
pub fn wickRule(comptime rule: WickRule, comptime operator_specs: []const Operator)
    ↪ wick.Rule {
    const expression = rule.expr orelse @compileError("draft Wick spec cannot be lowered
    ↪ to runtime rules");
    return wick.rule(
        wick.pattern(rule.left, operatorSupport(operator_specs, rule.left)),

```

```

        wick.pattern(rule.right, operatorSupport(operator_specs, rule.right)),
        expression,
    );
}

/// wickRules lowers declarative Wick specs into the runtime rule table.
pub fn wickRules(comptime rules: []const WickRule, comptime operator_specs: []const
↳ Operator) [rules.len]wick.Rule {
    var storage: [rules.len]wick.Rule = undefined;
    inline for (rules, 0..) |rule, index| {
        storage[index] = wickRule(rule, operator_specs);
    }
    return storage;
}

/// fermionKindCount counts operators that contribute exchange signs.
pub fn fermionKindCount(comptime operator_specs: []const Operator) usize {
    comptime var count: usize = 0;
    inline for (operator_specs) |operator| {
        if (operator.statistics == .fermionic) count += 1;
    }
    return count;
}

/// fermionKinds lowers operator statistics into runtime fermion kind ids.
pub fn fermionKinds(comptime operator_specs: []const Operator)
↳ [fermionKindCount(operator_specs)]operators.OperatorKindId {
    var storage: [fermionKindCount(operator_specs)]operators.OperatorKindId = undefined;
    comptime var index: usize = 0;
    inline for (operator_specs) |operator| {
        if (operator.statistics == .fermionic) {
            storage[index] = operator.kind;
            index += 1;
        }
    }
    return storage;
}
};

fn coordVariable(coord: Handle.Coord) u32 {
    return rawToken(coord);
}

fn singleInsertion(z: Handle.Coord, derivatives: u8) operators.OperatorInsertion {
    return .{ .single = .{
        .position = coordVariable(z),
        .derivatives = derivatives,
    } };
}

fn pairInsertion(z: Handle.Coord, zbar: Handle.Coord) operators.OperatorInsertion {
    return .{ .pair = .{
        .holomorphic_position = coordVariable(z),
        .antiholomorphic_position = coordVariable(zbar),
        .holomorphic_derivatives = 0,
        .antiholomorphic_derivatives = 0,
    } };
}

fn labelTupleCount(comptime Labels: type) comptime_int {
    const info = @typeInfo(Labels);
    if (info != .@"struct" or !info.@"struct".is_tuple) @compileError("operator labels must be
↳ passed as a tuple");
    return info.@"struct".fields.len;
}

```

```

}

fn fillSymbolLabels(comptime count: usize, out: *[count]kernel.Call.LabelValue, labels:
↳ anytype) void {
    inline for (0..count) |index| {
        out[index] = Local.symbol(labels[index]);
    }
}

fn buildKindOperator(local: *Local, kind_id: operators.OperatorKindId, insertion:
↳ operators.OperatorInsertion, labels: anytype) !*LocalOperator {
    const count = comptime labelTupleCount(@TypeOf(labels));
    var values: [count]kernel.Call.LabelValue = undefined;
    fillSymbolLabels(count, &values, labels);
    return local.op(kind_id, insertion, values[0..]);
}

fn assertOperatorShape(comptime operator: Spec.Operator, comptime insertion:
↳ Spec.InsertionShape, comptime label_count: usize) void {
    if (operator.insertion != insertion) @compileError("operator builder insertion shape does
↳ not match spec");
    if (operator.labels.len != label_count) @compileError("operator builder label count does
↳ not match spec");
}

fn boundaryCoord(y: Handle.BoundaryCoord) Handle.Coord {
    return token(Handle.Coord, rawToken(y));
}

fn buildSingleOperator(local: *Local, comptime operator: Spec.Operator, z: Handle.Coord,
↳ derivatives: u8, labels: anytype) !*LocalOperator {
    comptime assertOperatorShape(operator, .single, labelTupleCount(@TypeOf(labels)));
    return buildKindOperator(local, operator.kind, singleInsertion(z, derivatives), labels);
}

fn buildPairOperator(local: *Local, comptime operator: Spec.Operator, z: Handle.Coord, zbar:
↳ Handle.Coord, labels: anytype) !*LocalOperator {
    comptime assertOperatorShape(operator, .pair, labelTupleCount(@TypeOf(labels)));
    return buildKindOperator(local, operator.kind, pairInsertion(z, zbar), labels);
}

fn localBuilder(local: anytype) *Local {
    const Input = @TypeOf(local);
    return switch (@TypeInfo(Input)) {
        .pointer => |ptr| if (ptr.child == Local)
            local
        else if (ptr.child == *Local)
            local.*
        else
            @compileError("operator builders require a local builder"),
        else => @compileError("operator builders require a local builder"),
    };
}

/// operatorBuilder generates insertion and label packing from one operator schema.
fn operatorBuilder(comptime operator: Spec.Operator) type {
    return struct {
        /// single builds a single-coordinate local operator from this schema.
        pub fn single(local: anytype, z: Handle.Coord, derivatives: u8, labels: anytype)
↳ !*LocalOperator {
            return buildSingleOperator(localBuilder(local), operator, z, derivatives, labels);
        }
    }
}

```

```

    /// boundarySingle builds a boundary single-coordinate local operator from this
    ↪ schema.
    pub fn boundarySingle(local: anytype, y: Handle.BoundaryCoord, derivatives: u8,
    ↪ labels: anytype) !*LocalOperator {
        return buildSingleOperator(localBuilder(local), operator, boundaryCoord(y),
        ↪ derivatives, labels);
    }

    /// pair builds a bulk-pair local operator from this schema.
    pub fn pair(local: anytype, z: Handle.Coord, zbar: Handle.Coord, labels: anytype)
    ↪ !*LocalOperator {
        return buildPairOperator(localBuilder(local), operator, z, zbar, labels);
    }
};

}

/// zero_mode exposes compact preset-author helpers for zero-mode rules.
const zero_mode = ZeroModeDsl;

const ZeroModeDsl = struct {
    /// Sector names a preset-owned zero-mode sector.
    pub const Sector = u32;

    /// BcSupport selects the finite c-zero-mode basis used by the bc evaluator.
    pub const BcSupport = enum {
        sphere_holomorphic,
        sphere_antiholomorphic,
        disk_doubled,
    };

    /// BcTopForm declares one top-form ghost zero-mode saturation rule.
    pub const BcTopForm = struct {
        support: BcSupport,
        c_kind_ids: []const operators.OperatorKindId,
        normalization: RuleScalar = .one,
    };

    /// EtaXiSupport selects the chiral xi constant mode saturated by a surface.
    pub const EtaXiSupport = enum {
        sphere_holomorphic,
        sphere_antiholomorphic,
        torus_holomorphic,
        torus_antiholomorphic,
    };

    /// EtaXiZeroMode declares one xi constant-mode saturation rule.
    pub const EtaXiZeroMode = struct {
        support: EtaXiSupport,
        xi_kind_ids: []const operators.OperatorKindId,
        normalization: RuleScalar = .one,
    };

    /// ConstantFermion declares one generic constant fermion saturation rule.
    pub const ConstantFermion = struct {
        support: EtaXiSupport,
        fermion_kind_ids: []const operators.OperatorKindId,
        normalization: RuleScalar = .one,
    };

    /// TopFormFermion declares one generic top-form fermion saturation rule.
    pub const TopFormFermion = struct {
        support: BcSupport,
        field_kind_ids: []const operators.OperatorKindId,
        normalization: RuleScalar = .one,
    };
};

```

```

};

/// RankSource records where a power of 2pi gets its exponent.
pub const RankSource = union(enum) {
    none,
    literal: u16,
    target_dimension: ConfigRef,
    projector_rank: ConfigRef,
};

/// Normalization records the CFT normalization attached to a zero-mode rule.
pub const Normalization = struct {
    scalar: RuleScalar = .one,
    two_pi_power: RankSource = .none,
};

/// FreeBosonConstantMode declares the free-boson constant-mode base case.
pub const FreeBosonConstantMode = struct {
    integration_projector: ?ConfigRef = null,
    fixed_projector: ?ConfigRef = null,
    fixed_position: ?ConfigRef = null,
    exp_kind_ids: []const operators.OperatorKindId,
    profile_kind_ids: []const operators.OperatorKindId,
    normalization: Normalization = .{},
};

/// Expr selects the zero-mode evaluator and payload.
pub const Expr = union(enum) {
    bc_top_form: BcTopForm,
    free_boson_constant_mode: FreeBosonConstantMode,
    eta_xi_zero_mode: EtaXiZeroMode,
    constant_fermion: ConstantFermion,
    top_form_fermion: TopFormFermion,
    boson_momentum_conservation: FreeBosonConstantMode,
};

/// Rule is one preset-authored zero-mode rule.
pub const Rule = struct {
    sector: Sector,
    expr: Expr,
};

/// rule constructs a zero-mode rule.
pub fn rule(sector: Sector, expr: Expr) Rule {
    return .{ .sector = sector, .expr = expr };
}

};

const PairLookupEntry = struct {
    key: u64,
    rule_index: u32,
    reversed: bool,
};

const BranchBudget = enum {
    standard,
};

const BranchLimits = struct {
    max_operators: usize,
    max_zero_mode_residuals: usize,
    max_zero_mode_items: usize,
    max_terms: usize,
    max_scalars: usize,
};

```

```

    max_coordinates: usize,
    max_tensors: usize,
    max_actions: usize,
    max_residuals: usize,
    max_cached_terms: usize,
    max_cached_scalars: usize,
    max_cached_coordinates: usize,
    max_cached_tensors: usize,
    max_cached_actions: usize,
    max_cached_residuals: usize,
};

const CorrelatorConfigInput = struct {
    wick_rules: []const wick.Rule,
    zero_modes: []const zero_mode.Rule,
    config_entries: []const ConfigEntry = &.{},
    fermion_kinds: []const operators.OperatorKindId = &.{},
    branch_budget: BranchBudget = .standard,
};

const CorrelatorConfigData = struct {
    wick_rules: []const wick.Rule,
    pair_lookup: []const PairLookupEntry,
    zero_modes: []const zero_mode.Rule,
    config_entries: []const ConfigEntry,
    fermion_kinds: []const operators.OperatorKindId,
};

/// correlatorConfig creates an opaque public handle for lowered correlator data.
fn correlatorConfig(comptime input: CorrelatorConfigInput) type {
    return struct {
        const branch_limits = branchLimits(input);
        const pair_lookup_storage = pairLookupStorage(input.wick_rules);
        const impl = CorrelatorConfigData{
            .wick_rules = input.wick_rules,
            .pair_lookup = &pair_lookup_storage,
            .zero_modes = input.zero_modes,
            .config_entries = input.config_entries,
            .fermion_kinds = input.fermion_kinds,
        };
    };
}

fn configData(config_ptr: anytype) *const CorrelatorConfigData {
    const Ptr = @TypeOf(config_ptr);
    const ptr_info = @typeInfo(Ptr);
    if (ptr_info != .pointer) @compileError("correlator config must be passed by pointer");
    const Config = ptr_info.pointer.child;
    if (!@hasDecl(Config, "impl")) @compileError("invalid correlator config handle");
    return &Config.impl;
}

const ConfigValue = union(enum) {
    tensor_projector: Handle.TensorProjector,
    target_point: Handle.TargetPoint,
    target_dimension: u16,
    boundary_stack: Handle.BoundaryStack,
};

const ConfigEntry = struct {
    id: ConfigRef,
    value: ConfigValue,
};

```

```

/// config exposes typed constructors for private config payload entries.
const ConfigDsl = struct {
    /// tensorProjector binds a tensor-projector handle to a config reference.
    pub fn tensorProjector(id: ConfigRef, value: Handle.TensorProjector) ConfigEntry {
        return .{ .id = id, .value = .{ .tensor_projector = value } };
    }

    /// targetPoint binds a target-space point handle to a config reference.
    pub fn targetPoint(id: ConfigRef, value: Handle.TargetPoint) ConfigEntry {
        return .{ .id = id, .value = .{ .target_point = value } };
    }

    /// targetDimension binds a target-space dimension to a config reference.
    pub fn targetDimension(id: ConfigRef, value: u16) ConfigEntry {
        return .{ .id = id, .value = .{ .target_dimension = value } };
    }

    /// boundaryStack binds Chan-Paton boundary-stack data to a config reference.
    pub fn boundaryStack(id: ConfigRef, value: Handle.BoundaryStack) ConfigEntry {
        return .{ .id = id, .value = .{ .boundary_stack = value } };
    }
};

/// configEntries packs typed config entries without exposing their carrier type.
fn configEntries(comptime entries: anytype)
→ [@TypeInfo(@TypeOf(entries)).@"struct".fields.len]ConfigEntry {
    var storage: [@TypeInfo(@TypeOf(entries)).@"struct".fields.len]ConfigEntry = undefined;
    inline for (entries, 0..) |entry, index| {
        storage[index] = entry;
    }
    return storage;
}

const BoundaryExtensionInput = struct {
    op: type,
    wick_rules: []const wick.Rule,
    zero_modes: []const zero_mode.Rule,
    config_entries: []const ConfigEntry = &.{},
    fermion_kinds: []const operators.OperatorKindId = &.{},
};

const BoundaryExtensionData = struct {
    wick_rules: []const wick.Rule,
    zero_modes: []const zero_mode.Rule,
    config_entries: []const ConfigEntry,
    fermion_kinds: []const operators.OperatorKindId,
};

const LoweredSlice = enum {
    wick_rules,
    zero_modes,
    config_entries,
    fermion_kinds,
};

fn correlatorSliceCount(config_ptr: anytype, comptime slice: LoweredSlice) usize {
    const data = configData(config_ptr);
    return switch (slice) {
        .wick_rules => data.wick_rules.len,
        .zero_modes => data.zero_modes.len,
        .config_entries => data.config_entries.len,
        .fermion_kinds => data.fermion_kinds.len,
    };
}

```



```

fn appendCorrelatorSlice(config_ptr: anytype, comptime slice: LoweredSlice, storage: anytype,
↳ index: *usize) void {
    const data = configData(config_ptr);
    switch (slice) {
        .wick_rules => for (data.wick_rules) |item| {
            storage[index.*] = item;
            index.* += 1;
        },
        .zero_modes => for (data.zero_modes) |item| {
            storage[index.*] = item;
            index.* += 1;
        },
        .config_entries => for (data.config_entries) |item| {
            storage[index.*] = item;
            index.* += 1;
        },
        .fermion_kinds => for (data.fermion_kinds) |item| {
            storage[index.*] = item;
            index.* += 1;
        },
    }
}

/// BoundaryExtension creates an opaque public handle for boundary preset data.
fn BoundaryExtension(comptime input: BoundaryExtensionInput) type {
    return struct {
        /// op exposes boundary-local operator builders for this extension.
        pub const op = input.op;
        const impl = BoundaryExtensionData{
            .wick_rules = input.wick_rules,
            .zero_modes = input.zero_modes,
            .config_entries = input.config_entries,
            .fermion_kinds = input.fermion_kinds,
        };
    };
}

fn boundaryExtensionData(comptime extension: anytype) BoundaryExtensionData {
    const Extension = @TypeOf(extension);
    if (!@hasDecl(Extension, "impl")) @compileError("invalid boundary extension handle");
    return Extension.impl;
}

fn boundaryExtensionSliceCount(comptime extension: anytype, comptime slice: LoweredSlice)
↳ usize {
    const data = boundaryExtensionData(extension);
    return switch (slice) {
        .wick_rules => data.wick_rules.len,
        .zero_modes => data.zero_modes.len,
        .config_entries => data.config_entries.len,
        .fermion_kinds => data.fermion_kinds.len,
    };
}

fn appendBoundaryExtensionSlice(comptime extension: anytype, comptime slice: LoweredSlice,
↳ storage: anytype, index: *usize) void {
    const data = boundaryExtensionData(extension);
    switch (slice) {
        .wick_rules => for (data.wick_rules) |item| {
            storage[index.*] = item;
            index.* += 1;
        },
        .zero_modes => for (data.zero_modes) |item| {

```

```

        storage[index.*] = item;
        index.* += 1;
    },
    .config_entries => for (data.config_entries) |item| {
        storage[index.*] = item;
        index.* += 1;
    },
    .fermion_kinds => for (data.fermion_kinds) |item| {
        storage[index.*] = item;
        index.* += 1;
    },
}
}

fn factorHasSphereConfig(comptime Factor: type) bool {
    return @hasDecl(Factor, "config") and @hasDecl(Factor.config, "sphere");
}

fn sphereConfigSliceCount(comptime factors: anytype, comptime slice: LoweredSlice) usize {
    var count: usize = 0;
    inline for (factors) |Factor| {
        if (factorHasSphereConfig(Factor)) count +=
            ↪ correlatorSliceCount(&Factor.config.sphere, slice);
    }
    return count;
}

fn appendSphereConfigSlice(comptime factors: anytype, comptime slice: LoweredSlice, storage:
↪ anytype) void {
    var index: usize = 0;
    inline for (factors) |Factor| {
        if (factorHasSphereConfig(Factor)) {
            appendCorrelatorSlice(&Factor.config.sphere, slice, storage, &index);
        }
    }
}

fn mergedSphereWickStorage(comptime factors: anytype) [sphereConfigSliceCount(factors,
↪ .wick_rules)]wick.Rule {
    var storage: [sphereConfigSliceCount(factors, .wick_rules)]wick.Rule = undefined;
    appendSphereConfigSlice(factors, .wick_rules, storage[0..]);
    return storage;
}

fn mergedSphereZeroStorage(comptime factors: anytype) [sphereConfigSliceCount(factors,
↪ .zero_modes)]zero_mode.Rule {
    var storage: [sphereConfigSliceCount(factors, .zero_modes)]zero_mode.Rule = undefined;
    appendSphereConfigSlice(factors, .zero_modes, storage[0..]);
    return storage;
}

fn mergedSphereConfigStorage(comptime factors: anytype) [sphereConfigSliceCount(factors,
↪ .config_entries)]ConfigEntry {
    var storage: [sphereConfigSliceCount(factors, .config_entries)]ConfigEntry = undefined;
    appendSphereConfigSlice(factors, .config_entries, storage[0..]);
    return storage;
}

fn mergedSphereFermionStorage(comptime factors: anytype) [sphereConfigSliceCount(factors,
↪ .fermion_kinds)]operators.OperatorKindId {
    var storage: [sphereConfigSliceCount(factors, .fermion_kinds)]operators.OperatorKindId =
        ↪ undefined;
    appendSphereConfigSlice(factors, .fermion_kinds, storage[0..]);
    return storage;
}

```

```

}

/// mergeSphereConfigs builds one opaque sphere config from independent factors.
fn mergeSphereConfigs(comptime factors: anytype) type {
    return struct {
        const wick_storage = mergedSphereWickStorage(factors);
        const zero_storage = mergedSphereZeroStorage(factors);
        const config_storage = mergedSphereConfigStorage(factors);
        const fermion_storage = mergedSphereFermionStorage(factors);

        /// config is the merged sphere correlator config handle.
        pub const config = correlatorConfig(.{
            .wick_rules = &wick_storage,
            .zero_modes = &zero_storage,
            .config_entries = &config_storage,
            .fermion_kinds = &fermion_storage,
        }){};
    };
}

fn boundaryExtensionSliceTotal(comptime extensions: anytype, comptime slice: LoweredSlice)
    ↪ usize {
    var count: usize = 0;
    inline for (extensions) |extension| {
        count += boundaryExtensionSliceCount(extension, slice);
    }
    return count;
}

fn boundaryConfigSliceCount(comptime Bulk: type, comptime extensions: anytype, comptime slice:
    ↪ LoweredSlice) usize {
    var count: usize = 0;
    if (factorHasSphereConfig(Bulk)) count += correlatorSliceCount(&Bulk.config.sphere,
        ↪ slice);
    inline for (extensions) |extension| {
        count += boundaryExtensionSliceCount(extension, slice);
    }
    return count;
}

fn appendBoundaryExtensionSlices(comptime extensions: anytype, comptime slice: LoweredSlice,
    ↪ storage: anytype) void {
    var index: usize = 0;
    inline for (extensions) |extension| {
        appendBoundaryExtensionSlice(extension, slice, storage, &index);
    }
}

fn appendBoundaryConfigSlices(comptime Bulk: type, comptime extensions: anytype, comptime
    ↪ slice: LoweredSlice, storage: anytype) void {
    var index: usize = 0;
    if (factorHasSphereConfig(Bulk)) {
        appendCorrelatorSlice(&Bulk.config.sphere, slice, storage, &index);
    }
    inline for (extensions) |extension| {
        appendBoundaryExtensionSlice(extension, slice, storage, &index);
    }
}

fn mergedBoundaryWickStorage(comptime extensions: anytype)
    ↪ [boundaryExtensionSliceTotal(extensions, .wick_rules)]wick.Rule {
    var storage: [boundaryExtensionSliceTotal(extensions, .wick_rules)]wick.Rule = undefined;
    appendBoundaryExtensionSlices(extensions, .wick_rules, storage[0..]);
    return storage;
}

```

```

}

fn mergedBoundaryZeroStorage(comptime extensions: anytype)
↳ [boundaryExtensionSliceTotal(extensions, .zero_modes)]zero_mode.Rule {
    var storage: [boundaryExtensionSliceTotal(extensions, .zero_modes)]zero_mode.Rule =
        ↳ undefined;
    appendBoundaryExtensionSlices(extensions, .zero_modes, storage[0..]);
    return storage;
}

fn mergedBoundaryConfigStorage(comptime Bulk: type, comptime extensions: anytype)
↳ [boundaryConfigSliceCount(Bulk, extensions, .config_entries)]ConfigEntry {
    var storage: [boundaryConfigSliceCount(Bulk, extensions, .config_entries)]ConfigEntry =
        ↳ undefined;
    appendBoundaryConfigSlices(Bulk, extensions, .config_entries, storage[0..]);
    return storage;
}

fn mergedBoundaryFermionStorage(comptime Bulk: type, comptime extensions: anytype)
↳ [boundaryConfigSliceCount(Bulk, extensions, .fermion_kinds)]operators.OperatorKindId {
    var storage: [boundaryConfigSliceCount(Bulk, extensions,
        ↳ .fermion_kinds)]operators.OperatorKindId = undefined;
    appendBoundaryConfigSlices(Bulk, extensions, .fermion_kinds, storage[0..]);
    return storage;
}

/// mergeBoundaryConfig builds one opaque disk config from bulk data and extensions.
fn mergeBoundaryConfig(comptime Bulk: type, comptime extensions: anytype) type {
    return struct {
        const wick_storage = mergedBoundaryWickStorage(extensions);
        const zero_storage = mergedBoundaryZeroStorage(extensions);
        const config_storage = mergedBoundaryConfigStorage(Bulk, extensions);
        const fermion_storage = mergedBoundaryFermionStorage(Bulk, extensions);

        /// config is the merged disk correlator config handle.
        pub const config = correlatorConfig({
            .wick_rules = &wick_storage,
            .zero_modes = &zero_storage,
            .config_entries = &config_storage,
            .fermion_kinds = &fermion_storage,
        }){};
    };
}

const DeclareSpec = Spec;
const DeclareWick = wick;
const DeclareZeroMode = zero_mode;
const DeclareScalars = scalars;
const DeclareConfig = ConfigDsl;
const declareConfigRef = configRef;
const declareFamilyKind = familyKind;
const declareSectorId = sectorId;
const declareOperatorBuilder = operatorBuilder;
const declareCorrelatorConfig = correlatorConfig;
const declareBoundaryExtension = BoundaryExtension;
const declareConfigEntries = configEntries;
const declareMergeSphereConfigs = mergeSphereConfigs;
const declareMergeBoundaryConfig = mergeBoundaryConfig;

/// declare exposes preset declaration lowering without leaking individual internals.
pub const declare = struct {
    /// Spec stores declarative metadata before lowering to runtime tables.
    pub const Spec = DeclareSpec;
    /// wick exposes compact preset-author helpers for Wick rules.

```

```

pub const wick = DeclareWick;
/// zero_mode exposes compact preset-author helpers for zero-mode rules.
pub const zero_mode = DeclareZeroMode;
/// scalars exposes exact scalar helpers for preset rule coefficients.
pub const scalars = DeclareScalars;
/// config exposes typed constructors for private config payload entries.
pub const config = DeclareConfig;

/// configRef derives a compact config id from a namespace-local name.
pub const configRef = declareConfigRef;
/// familyKind derives a compact operator kind id from a preset family tag.
pub const familyKind = declareFamilyKind;
/// sectorId derives a compact zero-mode sector id from a preset sector tag.
pub const sectorId = declareSectorId;
/// operatorBuilder generates insertion and label packing from one operator schema.
pub const operatorBuilder = declareOperatorBuilder;
/// correlatorConfig creates an opaque public handle for lowered correlator data.
pub const correlatorConfig = declareCorrelatorConfig;
/// BoundaryExtension creates an opaque public handle for boundary preset data.
pub const BoundaryExtension = declareBoundaryExtension;
/// configEntries packs typed config entries without exposing their carrier type.
pub const configEntries = declareConfigEntries;
/// mergeSphereConfigs builds one opaque sphere config from independent factors.
pub const mergeSphereConfigs = declareMergeSphereConfigs;
/// mergeBoundaryConfig builds one opaque disk config from bulk data and extensions.
pub const mergeBoundaryConfig = declareMergeBoundaryConfig;
};

fn wickRuleKey(left_kind: operators.OperatorKindId, right_kind: operators.OperatorKindId) u64
↪ {
    return (@as(u64, left_kind) << 32) | @as(u64, right_kind);
}

fn pairLookupEntryCount(comptime rules: []const wick.Rule) usize {
    var count: usize = 0;
    for (rules) |rule| {
        count += 1;
        if (rule.left.kind != rule.right.kind) count += 1;
    }
    return count;
}

fn sortPairLookup(entries: []PairLookupEntry) void {
    var index: usize = 1;
    while (index < entries.len) : (index += 1) {
        const value = entries[index];
        var hole = index;
        while (hole > 0 and entries[hole - 1].key > value.key) : (hole -= 1) {
            entries[hole] = entries[hole - 1];
        }
        entries[hole] = value;
    }
}

fn pairLookupStorage(comptime rules: []const wick.Rule)
↪ [pairLookupEntryCount(rules)]PairLookupEntry {
    var entries: [pairLookupEntryCount(rules)]PairLookupEntry = undefined;
    var entry_index: usize = 0;

    for (rules, 0..) |rule, rule_index| {
        if (rule_index > std.math.maxInt(u32)) @compileError("too many Wick rules for compact
↪ rule index");
        entries[entry_index] = .{
            .key = wickRuleKey(rule.left.kind, rule.right.kind),

```

```

        .rule_index = @intCast(rule_index),
        .reversed = false,
    };
    entry_index += 1;

    if (rule.left.kind != rule.right.kind) {
        entries[entry_index] = .{
            .key = wickRuleKey(rule.right.kind, rule.left.kind),
            .rule_index = @intCast(rule_index),
            .reversed = true,
        };
        entry_index += 1;
    }
}

sortPairLookup(entries[0..]);
return entries;
}

const Correlator = struct {
    /// WickPair is one oriented primitive Wick contraction between two concrete local
    ⇨ operators.
    const WickPair = struct {
        rule: *const wick.Rule,
        config: *const CorrelatorConfigData,
        left: kernel.Call.LocalOp,
        right: kernel.Call.LocalOp,
        labels: *const kernel.Call.LabelStore,
        reversed: bool,

        fn sideOp(self: WickPair, side: wick.Side) kernel.Call.LocalOp {
            return switch (side) {
                .left => if (self.reversed) self.right else self.left,
                .right => if (self.reversed) self.left else self.right,
            };
        }

        fn coordinate(self: WickPair, coordinate_ref: wick.CoordinateRef)
        ⇨ ?coefficient.Variable {
            const op = self.sideOp(coordinate_ref.side);
            return switch (op.insertion) {
                .single => |single| switch (coordinate_ref.slot) {
                    .position, .holomorphic => single.position,
                    .antiholomorphic => null,
                },
                .pair => |pair| switch (coordinate_ref.slot) {
                    .position, .holomorphic => pair.holomorphic_position,
                    .antiholomorphic => pair.antiholomorphic_position,
                },
            };
        }

        fn derivativeCount(self: WickPair, coordinate_ref: wick.CoordinateRef) ?u8 {
            const op = self.sideOp(coordinate_ref.side);
            return switch (op.insertion) {
                .single => |single| switch (coordinate_ref.slot) {
                    .position, .holomorphic => single.derivatives,
                    .antiholomorphic => null,
                },
                .pair => |pair| switch (coordinate_ref.slot) {
                    .position, .holomorphic => pair.holomorphic_derivatives,
                    .antiholomorphic => pair.antiholomorphic_derivatives,
                },
            };
        }
    };
};

```

```

    }

    fn label(self: WickPair, label_ref: wick.LabelRef) ?kernel.Call.LabelValue {
        const op = self.sideOp(label_ref.side);
        const index: usize = @as(usize, @intCast(op.labels)) + label_ref.slot;
        if (index >= self.labels.values.len) return null;
        return self.labels.values[index];
    }

    fn configValue(self: WickPair, config_ref: ConfigRef) ?ConfigValue {
        for (self.config.config_entries) |entry| {
            if (entry.id == config_ref) return entry.value;
        }
        return null;
    }
};

/// WickResidualRef points to one surviving original input occurrence.
const WickResidualRef = struct {
    input_index: usize,
    operator: kernel.Call.LocalOp,
};

/// ResolvedCoordinateRef is one concrete coordinate variable with its derivative count.
const ResolvedCoordinateRef = struct {
    variable: coefficient.Variable,
    derivatives: u8,
};

/// ResolvedCoordinateDifference is a concrete coordinate difference read by a Wick
↪ factor.
const ResolvedCoordinateDifference = struct {
    left: ResolvedCoordinateRef,
    right: ResolvedCoordinateRef,
};

/// ResolvedDerivativeAction records the derivative counts requested by one coordinate
↪ factor.
const ResolvedDerivativeAction = struct {
    left: ?u8 = null,
    right: ?u8 = null,
};

/// ResolvedTensorRef is tensor data resolved either from an operator label or config
↪ entry.
const ResolvedTensorRef = union(enum) {
    label: kernel.Call.LabelValue,
    config: ConfigValue,
};

/// ResolvedScalarFactor is a scalar factor with all label references plugged in.
const ResolvedScalarFactor = union(enum) {
    value: RuleScalar,
    label_bilinear_phase: struct { left: kernel.Call.LabelValue, right:
        ↪ kernel.Call.LabelValue, form: []const u8 },
    cocycle: struct { table: []const u8, left: kernel.Call.LabelValue, right:
        ↪ kernel.Call.LabelValue },
    spin_structure: []const u8,
};

/// ResolvedTensorFactor is a tensor multiplier with concrete labels and config values.
const ResolvedTensorFactor = union(enum) {
    none,
    metric: struct { left: kernel.Call.LabelValue, right: kernel.Call.LabelValue },
};

```

```

momentum_index: struct { momentum: kernel.Call.LabelValue, index:
    ↪ kernel.Call.LabelValue },
momentum_pair: struct { left: kernel.Call.LabelValue, right: kernel.Call.LabelValue },
projector_metric: struct { projector: ResolvedTensorRef, left: kernel.Call.LabelValue,
    ↪ right: kernel.Call.LabelValue },
projector_momentum_index: struct { projector: ResolvedTensorRef, momentum:
    ↪ kernel.Call.LabelValue, index: kernel.Call.LabelValue },
projector_momentum_pair: struct { projector: ResolvedTensorRef, left:
    ↪ kernel.Call.LabelValue, right: kernel.Call.LabelValue },
};

/// ResolvedActionFactor is a Wick action with concrete profile, index, and projector
    ↪ data.
const ResolvedActionFactor = union(enum) {
    profile_derivative: struct { profile: kernel.Call.LabelValue, index:
        ↪ kernel.Call.LabelValue },
    projected_profile_derivative: struct { projector: ResolvedTensorRef, profile:
        ↪ kernel.Call.LabelValue, index: kernel.Call.LabelValue },
};

/// ResolvedCoordinateFactor is a coordinate kernel with concrete coordinates and
    ↪ derivative data.
const ResolvedCoordinateFactor = union(enum) {
    difference_power: struct { coordinate: ResolvedCoordinateDifference, exponent: i16,
        ↪ derivatives: ResolvedDerivativeAction },
    logarithm: struct { coordinate: ResolvedCoordinateDifference, derivatives:
        ↪ ResolvedDerivativeAction },
    green_kernel: struct { name: []const u8, coordinate: ResolvedCoordinateDifference,
        ↪ derivatives: ResolvedDerivativeAction },
    green_exponential: ResolvedCoordinateDifference,
};

/// ResolvedTerm is one Wick term whose factors can be resolved without allocation.
const ResolvedTerm = struct {
    pair: WickPair,
    source: *const wick.Term,

    fn scalarCount(self: ResolvedTerm) usize {
        return self.source.scalars.len;
    }

    fn coordinateCount(self: ResolvedTerm) usize {
        return self.source.coordinates.len;
    }

    fn tensorCount(self: ResolvedTerm) usize {
        return self.source.tensors.len;
    }

    fn actionCount(self: ResolvedTerm) usize {
        return self.source.actions.len;
    }

    fn residualCount(self: ResolvedTerm) usize {
        return self.source.residuals.len;
    }

    fn scalar(self: ResolvedTerm, index: usize) ?ResolvedScalarFactor {
        if (index >= self.source.scalars.len) return null;
        return resolveScalarFactor(self.pair, self.source.scalars[index]);
    }

    fn coordinate(self: ResolvedTerm, index: usize) ?ResolvedCoordinateFactor {
        if (index >= self.source.coordinates.len) return null;

```



```

        return resolveCoordinateFactor(self.pair, self.source.coordinates[index]);
    }

    fn tensor(self: ResolvedTerm, index: usize) ?ResolvedTensorFactor {
        if (index >= self.source.tensors.len) return null;
        return resolveTensorFactor(self.pair, self.source.tensors[index]);
    }

    fn action(self: ResolvedTerm, index: usize) ?ResolvedActionFactor {
        if (index >= self.source.actions.len) return null;
        return resolveActionFactor(self.pair, self.source.actions[index]);
    }

    fn residual(self: ResolvedTerm, index: usize) ?wick.Side {
        if (index >= self.source.residuals.len) return null;
        return self.source.residuals[index];
    }
};

/// ResolvedWickPair gives allocation-free access to concrete Wick terms for one pair.
const ResolvedWickPair = struct {
    pair: WickPair,

    fn termCount(self: ResolvedWickPair) usize {
        return self.pair.rule.expr.terms.len;
    }

    fn term(self: ResolvedWickPair, index: usize) ?ResolvedTerm {
        if (index >= self.pair.rule.expr.terms.len) return null;
        return .{ .pair = self.pair, .source = &self.pair.rule.expr.terms[index] };
    }
};

/// PrimitiveWickTerm is one complete primitive Wick contribution without later
↔ simplification.
const PrimitiveWickTerm = struct {
    resolved: ResolvedTerm,

    fn scalarCount(self: PrimitiveWickTerm) usize {
        return self.resolved.scalarCount();
    }

    fn coordinateCount(self: PrimitiveWickTerm) usize {
        return self.resolved.coordinateCount();
    }

    fn tensorCount(self: PrimitiveWickTerm) usize {
        return self.resolved.tensorCount();
    }

    fn actionCount(self: PrimitiveWickTerm) usize {
        return self.resolved.actionCount();
    }

    fn residualCount(self: PrimitiveWickTerm) usize {
        return self.resolved.residualCount();
    }

    fn scalar(self: PrimitiveWickTerm, index: usize) ?ResolvedScalarFactor {
        return self.resolved.scalar(index);
    }

    fn coordinate(self: PrimitiveWickTerm, index: usize) ?coefficient.CoordinateFactor {
        const factor = self.resolved.coordinate(index) orelse return null;

```

```

        return evaluateCoordinateFactor(factor);
    }

    fn tensor(self: PrimitiveWickTerm, index: usize) ?ResolvedTensorFactor {
        return self.resolved.tensor(index);
    }

    fn action(self: PrimitiveWickTerm, index: usize) ?ResolvedActionFactor {
        return self.resolved.action(index);
    }

    fn residual(self: PrimitiveWickTerm, index: usize) ?wick.Side {
        return self.resolved.residual(index);
    }
};

/// PrimitiveWickPair gives allocation-free complete terms for one primitive Wick pair.
const PrimitiveWickPair = struct {
    pair: WickPair,

    fn termCount(self: PrimitiveWickPair) usize {
        return self.pair.rule.expr.terms.len;
    }

    fn term(self: PrimitiveWickPair, index: usize) ?PrimitiveWickTerm {
        const resolved = (ResolvedWickPair{ .pair = self.pair }).term(index) orelse return
        ↪ null;
        return .{ .resolved = resolved };
    }
};

fn resolveCoordinateRef(pair: WickPair, coordinate_ref: wick.CoordinateRef)
    ↪ ?ResolvedCoordinateRef {
    return .{
        .variable = pair.coordinate(coordinate_ref) orelse return null,
        .derivatives = pair.derivativeCount(coordinate_ref) orelse return null,
    };
}

fn resolveCoordinateDifference(pair: WickPair, difference: wick.CoordinateDifference)
    ↪ ?ResolvedCoordinateDifference {
    return .{
        .left = resolveCoordinateRef(pair, difference.left) orelse return null,
        .right = resolveCoordinateRef(pair, difference.right) orelse return null,
    };
}

fn resolveDerivativeAction(pair: WickPair, coordinate: wick.CoordinateDifference, action:
    ↪ wick.DerivativeAction) ?ResolvedDerivativeAction {
    return .{
        .left = if (action.include_left) pair.derivativeCount(coordinate.left) orelse
        ↪ return null else null,
        .right = if (action.include_right) pair.derivativeCount(coordinate.right) orelse
        ↪ return null else null,
    };
}

fn resolveTensorRef(pair: WickPair, tensor_ref: wick.TensorRef) ?ResolvedTensorRef {
    return switch (tensor_ref) {
        .label => |label_ref| .{ .label = pair.label(label_ref) orelse return null },
        .config => |config_ref| .{ .config = pair.configValue(config_ref) orelse return
        ↪ null },
    };
}

```

```

fn resolveScalarFactor(pair: WickPair, factor: wick.ScalarFactor) ?ResolvedScalarFactor {
  return switch (factor) {
    .value => |value| .{ .value = value },
    .label_bilinear_phase => |phase| .{ .label_bilinear_phase = .{
      .left = pair.label(phase.left) orelse return null,
      .right = pair.label(phase.right) orelse return null,
      .form = phase.form,
    } },
    .cocycle => |cocycle| .{ .cocycle = .{
      .table = cocycle.table,
      .left = pair.label(cocycle.left) orelse return null,
      .right = pair.label(cocycle.right) orelse return null,
    } },
    .spin_structure => |spin_structure| .{ .spin_structure = spin_structure },
  };
}

fn resolveTensorFactor(pair: WickPair, factor: wick.TensorFactor) ?ResolvedTensorFactor {
  return switch (factor) {
    .none => .none,
    .metric => |metric| .{ .metric = .{
      .left = pair.label(metric.left) orelse return null,
      .right = pair.label(metric.right) orelse return null,
    } },
    .momentum_index => |item| .{ .momentum_index = .{
      .momentum = pair.label(item.momentum) orelse return null,
      .index = pair.label(item.index) orelse return null,
    } },
    .momentum_pair => |item| .{ .momentum_pair = .{
      .left = pair.label(item.left) orelse return null,
      .right = pair.label(item.right) orelse return null,
    } },
    .projector_metric => |item| .{ .projector_metric = .{
      .projector = resolveTensorRef(pair, item.projector) orelse return null,
      .left = pair.label(item.left) orelse return null,
      .right = pair.label(item.right) orelse return null,
    } },
    .projector_momentum_index => |item| .{ .projector_momentum_index = .{
      .projector = resolveTensorRef(pair, item.projector) orelse return null,
      .momentum = pair.label(item.momentum) orelse return null,
      .index = pair.label(item.index) orelse return null,
    } },
    .projector_momentum_pair => |item| .{ .projector_momentum_pair = .{
      .projector = resolveTensorRef(pair, item.projector) orelse return null,
      .left = pair.label(item.left) orelse return null,
      .right = pair.label(item.right) orelse return null,
    } },
  };
}

fn resolveActionFactor(pair: WickPair, factor: wick.ActionFactor) ?ResolvedActionFactor {
  return switch (factor) {
    .profile_derivative => |item| .{ .profile_derivative = .{
      .profile = pair.label(item.profile) orelse return null,
      .index = pair.label(item.index) orelse return null,
    } },
    .projected_profile_derivative => |item| .{ .projected_profile_derivative = .{
      .projector = resolveTensorRef(pair, item.projector) orelse return null,
      .profile = pair.label(item.profile) orelse return null,
      .index = pair.label(item.index) orelse return null,
    } },
  };
}

```

```

fn resolveCoordinateFactor(pair: WickPair, factor: wick.CoordinateFactor)
↳ ?ResolvedCoordinateFactor {
    return switch (factor) {
        .difference_power => |item| .{ .difference_power = .{
            .coordinate = resolveCoordinateDifference(pair, item.coordinate) orelse return
            ↳ null,
            .exponent = item.exponent,
            .derivatives = resolveDerivativeAction(pair, item.coordinate,
            ↳ item.derivatives) orelse return null,
        } },
        .logarithm => |item| .{ .logarithm = .{
            .coordinate = resolveCoordinateDifference(pair, item.coordinate) orelse return
            ↳ null,
            .derivatives = resolveDerivativeAction(pair, item.coordinate,
            ↳ item.derivatives) orelse return null,
        } },
        .green_kernel => |item| .{ .green_kernel = .{
            .name = item.name,
            .coordinate = resolveCoordinateDifference(pair, item.coordinate) orelse return
            ↳ null,
            .derivatives = resolveDerivativeAction(pair, item.coordinate,
            ↳ item.derivatives) orelse return null,
        } },
        .green_exponential => |coordinate| .{ .green_exponential =
        ↳ resolveCoordinateDifference(pair, coordinate) orelse return null },
    };
}

fn coordinateDifference(coordinate: ResolvedCoordinateDifference)
↳ coefficient.CoordinateDifference {
    return .{
        .left = coordinate.left.variable,
        .right = coordinate.right.variable,
    };
}

fn selectedDerivativeCount(action: ResolvedDerivativeAction) u16 {
    const left: u16 = if (action.left) |count| count else 0;
    const right: u16 = if (action.right) |count| count else 0;
    return left + right;
}

fn selectedRightDerivativeCount(action: ResolvedDerivativeAction) u16 {
    return if (action.right) |count| count else 0;
}

fn checkedMulInt(left: i64, right: i64) ?i64 {
    const result = @mulWithOverflow(left, right);
    if (result[1] != 0) return null;
    return result[0];
}

fn checkedAddExponent(exponent: i16, derivative_count: u16) ?i16 {
    const next = @as(i32, exponent) - @as(i32, derivative_count);
    if (next < std.math.minInt(i16) or next > std.math.maxInt(i16)) return null;
    return @intCast(next);
}

fn fallingPower(exponent: i16, derivative_count: u16) ?i64 {
    var value: i64 = 1;
    var current: i64 = exponent;
    var index: u16 = 0;
    while (index < derivative_count) : (index += 1) {

```

```

        value = checkedMulInt(value, current) orelse return null;
        current -= 1;
    }
    return value;
}

fn factorial(value: u16) ?i64 {
    var result: i64 = 1;
    var current: u16 = 2;
    while (current <= value) : (current += 1) {
        result = checkedMulInt(result, current) orelse return null;
    }
    return result;
}

fn signFromParity(power: u16) i64 {
    return if ((power & 1) == 0) 1 else -1;
}

fn evaluateDifferencePower(item: anytype) ?coefficient.CoordinateFactor {
    const derivative_count = selectedDerivativeCount(item.derivatives);
    const right_count = selectedRightDerivativeCount(item.derivatives);
    const falling = fallingPower(item.exponent, derivative_count) orelse return null;
    const signed = checkedMulInt(signFromParity(right_count), falling) orelse return null;
    return .{
        .scalar = coefficient.rational(signed, 1) orelse return null,
        .kernel = .{ .difference_power = .{
            .coordinate = coordinateDifference(item.coordinate),
            .exponent = checkedAddExponent(item.exponent, derivative_count) orelse return
                ↪ null,
        } },
    };
}

fn evaluateLogarithm(item: anytype) ?coefficient.CoordinateFactor {
    const derivative_count = selectedDerivativeCount(item.derivatives);
    if (derivative_count == 0) {
        return .{
            .scalar = coefficient.integer(1),
            .kernel = .{ .logarithm = coordinateDifference(item.coordinate) },
        };
    }

    const right_count = selectedRightDerivativeCount(item.derivatives);
    const base = factorial(derivative_count - 1) orelse return null;
    const signed = checkedMulInt(signFromParity(right_count + derivative_count - 1), base)
        ↪ orelse return null;
    return .{
        .scalar = coefficient.rational(signed, 1) orelse return null,
        .kernel = .{ .difference_power = .{
            .coordinate = coordinateDifference(item.coordinate),
            .exponent = -@as(i16, @intCast(derivative_count)),
        } },
    };
}

fn evaluateGreenKernel(item: anytype) coefficient.CoordinateFactor {
    return .{
        .scalar = coefficient.integer(1),
        .kernel = .{ .green_kernel = .{
            .name = item.name,
            .coordinate = coordinateDifference(item.coordinate),
            .left_derivatives = if (item.derivatives.left) |count| count else 0,
            .right_derivatives = if (item.derivatives.right) |count| count else 0,
        } },
    };
}

```

```

    } },
  };
}

fn evaluateCoordinateFactor(factor: ResolvedCoordinateFactor)
↳ ?coefficient.CoordinateFactor {
  return switch (factor) {
    .difference_power => |item| evaluateDifferencePower(item),
    .logarithm => |item| evaluateLogarithm(item),
    .green_kernel => |item| evaluateGreenKernel(item),
    .green_exponential => |coordinate| .{
      .scalar = coefficient.integer(1),
      .kernel = .{ .green_exponential = coordinateDifference(coordinate) },
    },
  };
}

const WickTermEvent = struct {
  left_index: usize,
  right_index: usize,
  term_index: usize,
};

const MatchedWickTerm = struct {
  pair: WickPair,
  rule_index: usize,
  term_index: usize,
};

const ZeroModeRuntime = struct {
  const ResidualCursor = struct {
    ops: kernel.Call.MultiOp,
    indices: ?[]const usize,

    fn len(self: ResidualCursor) usize {
      return if (self.indices) |indices| indices.len else self.ops.operators.len;
    }

    fn opIndex(self: ResidualCursor, residual_index: usize) usize {
      return if (self.indices) |indices| indices[residual_index] else residual_index;
    }

    fn op(self: ResidualCursor, residual_index: usize) kernel.Call.LocalOp {
      return self.ops.operators[self.opIndex(residual_index)];
    }
  };

  const CJet = struct {
    coordinate: coefficient.Variable,
    derivative_order: u8,
  };

  const MomentumDelta = struct {
    projector: ?ConfigValue,
    momenta: []const kernel.Call.LabelValue,
    two_pi_power: u16,
    scalar: RuleScalar,
  };

  const DirichletPhase = struct {
    projector: ConfigValue,
    position: ConfigValue,
    momenta: []const kernel.Call.LabelValue,
  };
};

```

```

};

const ProfileFactor = struct {
    profile: kernel.Call.LabelValue,
    coordinate: coefficient.Variable,
};

const ZeroModeFactor = union(enum) {
    bc_top_form: struct {
        support: zero_mode.BcSupport,
        normalization: RuleScalar,
        jets: [3]CJet,
    },
    eta_xi_zero_mode: struct {
        support: zero_mode.EtaXiSupport,
        normalization: RuleScalar,
        xi: CJet,
    },
    momentum_delta: MomentumDelta,
    dirichlet_phase: DirichletPhase,
    profile_fourier_integral: ProfileFactor,
    profile_polynomial_integral: ProfileFactor,
    profile_gaussian_differential: ProfileFactor,
};

const TextBudget = enum {
    small,
    medium,
};

/// text exposes bounded result-inspection sinks.
pub const text = struct {
    /// compact returns a writer-backed sink for small symbolic correlators.
    pub fn compact(writer: anytype, comptime budget: TextBudget)
        ↪ CompactTextSink(@TypeOf(writer), budget) {
        return .{ .writer = writer };
    }
};

fn textByteLimit(comptime budget: TextBudget) usize {
    return switch (budget) {
        .small => 4096,
        .medium => 65536,
    };
}

fn textBranchLimit(comptime budget: TextBudget) usize {
    return switch (budget) {
        .small => 32,
        .medium => 1024,
    };
}

fn CompactTextSink(comptime Writer: type, comptime budget: TextBudget) type {
    return struct {
        const Self = @This();

        writer: Writer,
        bytes_written: usize = 0,
        branch_count: usize = 0,
        branch_open: bool = false,
        branch_has_factor: bool = false,
        term_open: bool = false,
    };
}

```

```

term_has_factor: bool = false,
pending_sign: i8 = 1,

fn writeAll(self: *Self, bytes: []const u8) !void {
    if (self.bytes_written + bytes.len > textByteLimit(budget)) return
        ↪ error.RenderLimitExceeded;
    try self.writer.writeAll(bytes);
    self.bytes_written += bytes.len;
}

fn writeFmt(self: *Self, comptime fmt: []const u8, args: anytype) !void {
    var buffer: [256]u8 = undefined;
    const rendered = std.fmt.bufPrint(&buffer, fmt, args) catch return
        ↪ error.RenderLimitExceeded;
    try self.writeAll(rendered);
}

fn beginBranch(self: *Self) !void {
    if (self.branch_open) return;
    if (self.branch_count == textBranchLimit(budget)) return
        ↪ error.RenderLimitExceeded;
    if (self.branch_count != 0) {
        try self.writeAll(if (self.pending_sign < 0) " - " else " + ");
    } else if (self.pending_sign < 0) {
        try self.writeAll("-");
    }
    self.branch_open = true;
    self.branch_has_factor = false;
    self.pending_sign = 1;
}

fn factorSeparator(self: *Self) !void {
    try self.beginBranch();
    if (self.term_open) {
        if (self.term_has_factor) try self.writeAll("*");
        self.term_has_factor = true;
        return;
    }
    if (self.branch_has_factor) try self.writeAll("*");
    self.branch_has_factor = true;
}

fn writeRational(self: *Self, value: RationalScalar) !void {
    if (value.denominator == 1) return self.writeFmt("{ }", .{value.numerator});
    return self.writeFmt("{ }/{ }", .{ value.numerator, value.denominator });
}

fn writeRuleScalar(self: *Self, value: RuleScalar) !void {
    switch (value) {
        .one => try self.writeAll("1"),
        .rational => |rational| try self.writeRational(rational),
        .monomial => |monomial| {
            try self.writeRational(monomial.rational);
            if (monomial.imaginary_power != 0) try self.writeFmt("*i^{ }",
                ↪ .{monomial.imaginary_power});
            if (monomial.atom) |atom| try self.writeFmt("*s^{ }^{}", .{ atom,
                ↪ monomial.atom_power });
        },
    }
}

fn writeLabel(self: *Self, value: kernel.Call.LabelValue) !void {
    switch (value) {
        .integer => |item| try self.writeFmt("{ }", .{item}),
    }
}

```



```

        .rational => |item| try self.writeFmt("{} / {}", .{ item.numerator,
        ↪ item.denominator }),
        .tensor => |item| try self.writeFmt("T{}", .{item}),
        .symbol => |item| try self.writeFmt("q{}", .{item}),
    }
}

fn writeConfig(self: *Self, value: ConfigValue) !void {
    switch (value) {
        .tensor_projector => |item| try self.writeFmt("P{}", .{@intFromEnum(item)}),
        .target_point => |item| try self.writeFmt("x{}", .{@intFromEnum(item)}),
        .target_dimension => |item| try self.writeFmt("D{}", .{item}),
        .boundary_stack => |item| try self.writeFmt("CP{}", .{@intFromEnum(item)}),
    }
}

fn writeTensorRef(self: *Self, value: Correlator.ResolvedTensorRef) !void {
    switch (value) {
        .label => |item| try self.writeLabel(item),
        .config => |item| try self.writeConfig(item),
    }
}

fn writeCoordinateDifference(self: *Self, value: coefficient.CoordinateDifference)
    ↪ !void {
    try self.writeFmt("z{}-z{}", .{ value.left, value.right });
}

pub fn emitWickBranchSign(self: *Self, sign: i8) !void {
    if (sign < 0) self.pending_sign = -1;
}

pub fn emitWickBranchTerm(self: *Self, branch: anytype) !void {
    const Proxy = struct {
        renderer: *Self,

        pub fn emitWickTermStart(proxy: *@This(), event: Correlator.WickTermEvent)
            ↪ !void {
            try proxy.renderer.emitWickTermStart(event);
        }

        pub fn emitWickScalar(proxy: *@This(), factor:
            ↪ Correlator.ResolvedScalarFactor) !void {
            try proxy.renderer.emitWickScalar(factor);
        }

        pub fn emitWickCoordinate(proxy: *@This(), factor:
            ↪ coefficient.CoordinateFactor) !void {
            try proxy.renderer.emitWickCoordinate(factor);
        }

        pub fn emitWickTensor(proxy: *@This(), factor:
            ↪ Correlator.ResolvedTensorFactor) !void {
            try proxy.renderer.emitWickTensor(factor);
        }

        pub fn emitWickAction(proxy: *@This(), factor:
            ↪ Correlator.ResolvedActionFactor) !void {
            try proxy.renderer.emitWickAction(factor);
        }

        pub fn emitWickResidualOperator(proxy: *@This(), residual:
            ↪ Correlator.WickResidualRef) !void {
            try proxy.renderer.emitWickResidualOperator(residual);
        }
    }
}

```

```

    }

    pub fn emitWickTermEnd(proxy: *@This()) !void {
        try proxy.renderer.emitWickTermEnd();
    }
};

var proxy = Proxy{ .renderer = self };
try branch.emit(&proxy);
}

pub fn emitWickTermStart(self: *Self, _: Correlator.WickTermEvent) !void {
    try self.factorSeparator();
    try self.writeAll("(");
    self.term_open = true;
    self.term_has_factor = false;
}

pub fn emitWickScalar(self: *Self, factor: Correlator.ResolvedScalarFactor) !void {
    try self.factorSeparator();
    switch (factor) {
        .value => |value| try self.writeRuleScalar(value),
        .label_bilinear_phase => |item| {
            try self.writeFmt("phase[{}]"(.format(item.form)));
            try self.writeLabel(item.left);
            try self.writeAll(",");
            try self.writeLabel(item.right);
            try self.writeAll(")");
        },
        .cocycle => |item| {
            try self.writeFmt("cocycle[{}]"(.format(item.table)));
            try self.writeLabel(item.left);
            try self.writeAll(",");
            try self.writeLabel(item.right);
            try self.writeAll(")");
        },
        .spin_structure => |name| try self.writeFmt("spin[{}]"(.format(name))),
    }
}

pub fn emitWickCoordinate(self: *Self, factor: coefficient.CoordinateFactor) !void {
    try self.factorSeparator();
    if (factor.scalar.numerator != 1 or factor.scalar.denominator != 1) {
        try self.writeFmt("{} / {}*"(.format(factor.scalar.numerator,
        ↪ factor.scalar.denominator)));
    }
    switch (factor.kernel) {
        .difference_power => |item| {
            try self.writeCoordinateDifference(item.coordinate);
            try self.writeFmt("^{}"(.format(item.exponent)));
        },
        .logarithm => |coordinate| {
            try self.writeAll("log");
            try self.writeCoordinateDifference(coordinate);
        },
        .green_kernel => |item| {
            try self.writeFmt("G[{}; {}, {}]"(.format(item.name, item.left_derivatives,
            ↪ item.right_derivatives)));
            try self.writeCoordinateDifference(item.coordinate);
        },
        .green_exponential => |coordinate| {
            try self.writeAll("expG");
            try self.writeCoordinateDifference(coordinate);
        },
    }
}

```

```

    }
}

pub fn emitWickTensor(self: *Self, factor: Correlator.ResolvedTensorFactor) !void {
    try self.factorSeparator();
    switch (factor) {
        .none => try self.writeAll("1"),
        .metric => |item| {
            try self.writeAll("eta(");
            try self.writeLabel(item.left);
            try self.writeAll(",");
            try self.writeLabel(item.right);
            try self.writeAll(")");
        },
        .momentum_index => |item| {
            try self.writeAll("p(");
            try self.writeLabel(item.momentum);
            try self.writeAll("; ");
            try self.writeLabel(item.index);
            try self.writeAll(")");
        },
        .momentum_pair => |item| {
            try self.writeAll("p(");
            try self.writeLabel(item.left);
            try self.writeAll(").p(");
            try self.writeLabel(item.right);
            try self.writeAll(")");
        },
        .projector_metric => |item| {
            try self.writeAll("eta[");
            try self.writeTensorRef(item.projector);
            try self.writeAll("]");
            try self.writeLabel(item.left);
            try self.writeAll(",");
            try self.writeLabel(item.right);
            try self.writeAll(")");
        },
        .projector_momentum_index => |item| {
            try self.writeAll("p[");
            try self.writeTensorRef(item.projector);
            try self.writeAll("]");
            try self.writeLabel(item.momentum);
            try self.writeAll("; ");
            try self.writeLabel(item.index);
            try self.writeAll(")");
        },
        .projector_momentum_pair => |item| {
            try self.writeAll("p[");
            try self.writeTensorRef(item.projector);
            try self.writeAll("]");
            try self.writeLabel(item.left);
            try self.writeAll(",");
            try self.writeLabel(item.right);
            try self.writeAll(")");
        },
    },
}

pub fn emitWickAction(self: *Self, factor: Correlator.ResolvedActionFactor) !void {
    try self.factorSeparator();
    switch (factor) {
        .profile_derivative => |item| {
            try self.writeAll("dProfile(");
            try self.writeLabel(item.profile);

```

```

        try self.writeAll(",");
        try self.writeLabel(item.index);
        try self.writeAll(")");
    },
    .projected_profile_derivative => |item| {
        try self.writeAll("dProfile[");
        try self.writeTensorRef(item.projector);
        try self.writeAll("]");
        try self.writeLabel(item.profile);
        try self.writeAll(",");
        try self.writeLabel(item.index);
        try self.writeAll(")");
    },
}

pub fn emitWickResidualOperator(self: *Self, residual: Correlator.WickResidualRef)
↪ !void {
    try self.factorSeparator();
    try self.writeFmt("op{", .{residual.input_index});
}

pub fn emitWickTermEnd(self: *Self) !void {
    if (!self.term_has_factor) try self.writeAll("1");
    try self.writeAll(")");
    self.term_open = false;
    self.branch_has_factor = true;
}

pub fn emitZeroModeFactor(self: *Self, factor: ZeroModeRuntime.ZeroModeFactor) !void {
    try self.factorSeparator();
    switch (factor) {
        .bc_top_form => |item| try self.writeFmt("bcTop[{s}]",
            ↪ .{@tagName(item.support)}),
        .eta_xi_zero_mode => |item| try self.writeFmt("etaXiZero[{s};z{"]", .{
            ↪ @tagName(item.support), item.xi.coordinate }),
        .momentum_delta => |item| try self.writeFmt("deltaP[{"]",
            ↪ .{item.momenta.len}),
        .dirichlet_phase => |item| {
            try self.writeAll("dirichletPhase[");
            try self.writeConfig(item.projector);
            try self.writeAll(",");
            try self.writeConfig(item.position);
            try self.writeAll("]");
        },
        .profile_fourier_integral => |item| {
            try self.writeAll("profileFourier(");
            try self.writeLabel(item.profile);
            try self.writeFmt(";z{"]", .{item.coordinate});
        },
        .profile_polynomial_integral => |item| {
            try self.writeAll("profilePolynomial(");
            try self.writeLabel(item.profile);
            try self.writeFmt(";z{"]", .{item.coordinate});
        },
        .profile_gaussian_differential => |item| {
            try self.writeAll("profileGaussian(");
            try self.writeLabel(item.profile);
            try self.writeFmt(";z{"]", .{item.coordinate});
        },
    }
}

pub fn emitZeroModeBaseEnd(self: *Self) !void {

```

```

        try self.beginBranch();
        if (!self.branch_has_factor) try self.writeAll("1");
        self.branch_open = false;
        self.branch_has_factor = false;
        self.branch_count += 1;
    }
};
}

fn residualConsumed(mask: u128, index: usize) bool {
    return (mask & (@as(u128, 1) << @intCast(index))) != 0;
}

fn markResidualConsumed(mask: *u128, index: usize) void {
    mask.* |= @as(u128, 1) << @intCast(index);
}

fn kindIn(kind: operators.OperatorKindId, candidates: []const operators.OperatorKindId) bool {
    for (candidates) |candidate| {
        if (candidate == kind) return true;
    }
    return false;
}

fn localPosition(op: kernel.Call.LocalOp) ?coefficient.Variable {
    return switch (op.insertion) {
        .single => |single| single.position,
        .pair => |pair| pair.holomorphic_position,
    };
}

fn localDerivativeOrder(op: kernel.Call.LocalOp) u8 {
    return switch (op.insertion) {
        .single => |single| single.derivatives,
        .pair => |pair| pair.holomorphic_derivatives,
    };
}

fn firstLabel(ops: kernel.Call.MultiOp, op: kernel.Call.LocalOp) ?kernel.Call.LabelValue {
    const index: usize = @intCast(op.labels);
    if (index >= ops.labels.values.len) return null;
    return ops.labels.values[index];
}

fn configValue(config_ptr: *const CorrelatorConfigData, config_ref: ConfigRef) ?ConfigValue {
    for (config_ptr.config_entries) |entry| {
        if (entry.id == config_ref) return entry.value;
    }
    return null;
}

fn projectorRank(value: ConfigValue) ?u16 {
    return switch (value) {
        .tensor_projector => |projector| @intCast(@intFromEnum(projector) >> 24),
        else => null,
    };
}

fn rankValue(config_ptr: *const CorrelatorConfigData, source: zero_mode.RankSource) ?u16 {
    return switch (source) {
        .none => 0,
        .literal => |value| value,
        .target_dimension => |config_ref| switch (configValue(config_ptr, config_ref) orelse
            ↪ return null) {

```

```

        .target_dimension => |dimension| dimension,
        else => null,
    },
    .projector_rank => |config_ref| projectorRank(configValue(config_ptr, config_ref)
    ↪ orelse return null),
};
}

fn hasRankSource(source: zero_mode.RankSource) bool {
    return switch (source) {
        .none => false,
        else => true,
    };
}

fn emitBcTopForm(rule: zero_mode.BcTopForm, residual: ZeroModeRuntime.ResidualCursor,
    ↪ consumed: *u128, sink: anytype) !void {
    var jets: [3]ZeroModeRuntime.CJet = undefined;
    var residual_ids: [3]usize = undefined;
    var jet_count: usize = 0;
    var overflow = false;

    var residual_index: usize = 0;
    while (residual_index < residual.len()) : (residual_index += 1) {
        if (residualConsumed(consumed.*, residual_index)) continue;
        const op = residual.op(residual_index);
        if (!kindIn(op.kind, rule.c_kind_ids)) continue;
        if (jet_count >= jets.len) {
            overflow = true;
            continue;
        }
        jets[jet_count] = .{
            .coordinate = localPosition(op) orelse return error.InvalidZeroModeOperator,
            .derivative_order = localDerivativeOrder(op),
        };
        residual_ids[jet_count] = residual_index;
        jet_count += 1;
    }

    if (overflow or jet_count != 3) return;
    for (residual_ids) |residual_id| {
        markResidualConsumed(consumed, residual_id);
    }
    try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{ .bc_top_form = .{
        .support = rule.support,
        .normalization = rule.normalization,
        .jets = jets,
    } });
}

fn acceptBcTopForm(rule: zero_mode.BcTopForm, residual: ZeroModeRuntime.ResidualCursor,
    ↪ consumed: *u128) !void {
    var residual_ids: [3]usize = undefined;
    var jet_count: usize = 0;
    var overflow = false;

    var residual_index: usize = 0;
    while (residual_index < residual.len()) : (residual_index += 1) {
        if (residualConsumed(consumed.*, residual_index)) continue;
        const op = residual.op(residual_index);
        if (!kindIn(op.kind, rule.c_kind_ids)) continue;
        if (localPosition(op) == null) return error.InvalidZeroModeOperator;
        if (jet_count >= residual_ids.len) {
            overflow = true;

```

```

        continue;
    }
    residual_ids[jet_count] = residual_index;
    jet_count += 1;
}

if (overflow or jet_count != 3) return;
for (residual_ids) |residual_id| {
    markResidualConsumed(consumed, residual_id);
}
}

fn emitEtaXiZeroMode(rule: zero_mode.EtaXiZeroMode, residual: ZeroModeRuntime.ResidualCursor,
↳ consumed: *u128, sink: anytype) !void {
    var xi: ZeroModeRuntime.CJet = undefined;
    var residual_id: usize = 0;
    var found = false;
    var overflow = false;

    var residual_index: usize = 0;
    while (residual_index < residual.len()) : (residual_index += 1) {
        if (residualConsumed(consumed.*, residual_index)) continue;
        const op = residual.op(residual_index);
        if (!kindIn(op.kind, rule.xi_kind_ids)) continue;
        if (localDerivativeOrder(op) != 0 or found) {
            overflow = true;
            continue;
        }
        xi = .{
            .coordinate = localPosition(op) orelse return error.InvalidZeroModeOperator,
            .derivative_order = 0,
        };
        residual_id = residual_index;
        found = true;
    }

    if (overflow or !found) return;
    markResidualConsumed(consumed, residual_id);
    try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{ .eta_xi_zero_mode = .{
        .support = rule.support,
        .normalization = rule.normalization,
        .xi = xi,
    } });
}

fn acceptEtaXiZeroMode(rule: zero_mode.EtaXiZeroMode, residual:
↳ ZeroModeRuntime.ResidualCursor, consumed: *u128) !void {
    var residual_id: usize = 0;
    var found = false;
    var overflow = false;

    var residual_index: usize = 0;
    while (residual_index < residual.len()) : (residual_index += 1) {
        if (residualConsumed(consumed.*, residual_index)) continue;
        const op = residual.op(residual_index);
        if (!kindIn(op.kind, rule.xi_kind_ids)) continue;
        if (localPosition(op) == null) return error.InvalidZeroModeOperator;
        if (localDerivativeOrder(op) != 0 or found) {
            overflow = true;
            continue;
        }
        residual_id = residual_index;
        found = true;
    }
}

```

```

    if (overflow or !found) return;
    markResidualConsumed(consumed, residual_id);
}

fn emitFreeBosonConstantMode(comptime limits: BranchLimits, config_ptr: *const
↳ CorrelatorConfigData, rule: zero_mode.FreeBosonConstantMode, residual:
↳ ZeroModeRuntime.ResidualCursor, consumed: *u128, sink: anytype) !void {
    var momenta: [limits.max_zero_mode_items]kernel.Call.LabelValue = undefined;
    var momentum_count: usize = 0;

    var residual_index: usize = 0;
    while (residual_index < residual.len()) : (residual_index += 1) {
        if (residualConsumed(consumed.*, residual_index)) continue;
        const op = residual.op(residual_index);
        if (kindIn(op.kind, rule.exp_kind_ids)) {
            if (momentum_count >= momenta.len) return error.TooManyZeroModeItems;
            momenta[momentum_count] = firstLabel(residual.ops, op) orelse return
↳ error.InvalidZeroModeOperator;
            momentum_count += 1;
            markResidualConsumed(consumed, residual_index);
            continue;
        }

        if (kindIn(op.kind, rule.profile_kind_ids)) {
            const label = firstLabel(residual.ops, op) orelse return
↳ error.InvalidZeroModeOperator;
            const profile = switch (label) {
                .symbol => |value| @as(Handle.Profile, @enumFromInt(value)),
                else => return error.InvalidZeroModeOperator,
            };
            const factor = ZeroModeRuntime.ProfileFactor{
                .profile = label,
                .coordinate = localPosition(op) orelse return error.InvalidZeroModeOperator,
            };
            switch (profilePresentation(profile)) {
                .position_space => try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{
↳ .profile_gaussian_differential = factor }),
                .fourier => try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{
↳ .profile_fourier_integral = factor }),
                .polynomial_rnc => try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{
↳ .profile_polynomial_integral = factor }),
            }
            markResidualConsumed(consumed, residual_index);
        }
    }

    if (momentum_count != 0 or hasRankSource(rule.normalization.two_pi_power)) {
        const projector = if (rule.integration_projector) |config_ref| configValue(config_ptr,
↳ config_ref) orelse return error.InvalidZeroModeConfig else null;
        try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{ .momentum_delta = .{
            .projector = projector,
            .momenta = momenta[0..momentum_count],
            .two_pi_power = rankValue(config_ptr, rule.normalization.two_pi_power) orelse
↳ return error.InvalidZeroModeConfig,
            .scalar = rule.normalization.scalar,
        } });
    }

    if (rule.fixed_projector) |projector_ref| {
        if (momentum_count == 0) return;
        const position_ref = rule.fixed_position orelse return error.InvalidZeroModeConfig;
        try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{ .dirichlet_phase = .{

```



```

        .projector = configValue(config_ptr, projector_ref) orelse return
        ↪ error.InvalidZeroModeConfig,
        .position = configValue(config_ptr, position_ref) orelse return
        ↪ error.InvalidZeroModeConfig,
        .momenta = momenta[0..momentum_count],
    } });
}

fn acceptFreeBosonConstantMode(comptime limits: BranchLimits, config_ptr: *const
↪ CorrelatorConfigData, rule: zero_mode.FreeBosonConstantMode, residual:
↪ ZeroModeRuntime.ResidualCursor, consumed: *u128) !void {
    var momentum_count: usize = 0;

    var residual_index: usize = 0;
    while (residual_index < residual.len()) : (residual_index += 1) {
        if (residualConsumed(consumed.*, residual_index)) continue;
        const op = residual.op(residual_index);
        if (kindIn(op.kind, rule.exp_kind_ids)) {
            if (momentum_count >= limits.max_zero_mode_items) return
            ↪ error.TooManyZeroModeItems;
            _ = firstLabel(residual.ops, op) orelse return error.InvalidZeroModeOperator;
            momentum_count += 1;
            markResidualConsumed(consumed, residual_index);
            continue;
        }

        if (kindIn(op.kind, rule.profile_kind_ids)) {
            const label = firstLabel(residual.ops, op) orelse return
            ↪ error.InvalidZeroModeOperator;
            const profile = switch (label) {
                .symbol => |value| @as(Handle.Profile, @enumFromInt(value)),
                else => return error.InvalidZeroModeOperator,
            };
            _ = localPosition(op) orelse return error.InvalidZeroModeOperator;
            _ = profilePresentation(profile);
            markResidualConsumed(consumed, residual_index);
        }
    }

    if (momentum_count != 0 or hasRankSource(rule.normalization.two_pi_power)) {
        if (rule.integration_projector) |config_ref| _ = configValue(config_ptr, config_ref)
        ↪ orelse return error.InvalidZeroModeConfig;
        _ = rankValue(config_ptr, rule.normalization.two_pi_power) orelse return
        ↪ error.InvalidZeroModeConfig;
    }

    if (rule.fixed_projector) |projector_ref| {
        if (momentum_count == 0) return;
        const position_ref = rule.fixed_position orelse return error.InvalidZeroModeConfig;
        _ = configValue(config_ptr, projector_ref) orelse return error.InvalidZeroModeConfig;
        _ = configValue(config_ptr, position_ref) orelse return error.InvalidZeroModeConfig;
    }
}

fn allResidualsConsumed(residual: ZeroModeRuntime.ResidualCursor, consumed: u128) bool {
    var residual_index: usize = 0;
    while (residual_index < residual.len()) : (residual_index += 1) {
        if (!residualConsumed(consumed, residual_index)) return false;
    }
    return true;
}

```

```

fn emitEmptyFreeBosonConstantMode(config_ptr: *const CorrelatorConfigData, rule:
↳ zero_mode.FreeBosonConstantMode, sink: anytype) !void {
    if (hasRankSource(rule.normalization.two_pi_power)) {
        const empty_momenta: [0]kernel.Call.LabelValue = .{};
        const projector = if (rule.integration_projector) |config_ref| configValue(config_ptr,
↳ config_ref) orelse return error.InvalidZeroModeConfig else null;
        try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{ .momentum_delta = .{
            .projector = projector,
            .momenta = empty_momenta[0..],
            .two_pi_power = rankValue(config_ptr, rule.normalization.two_pi_power) orelse
↳ return error.InvalidZeroModeConfig,
            .scalar = rule.normalization.scalar,
        } });
    }
}

fn BcTopFormProcedure(comptime rule: zero_mode.BcTopForm) type {
    return struct {
        fn canConsumeKind(kind: operators.OperatorKindId) bool {
            return kindIn(kind, rule.c_kind_ids);
        }

        fn accept(comptime limits: BranchLimits, config_ptr: *const CorrelatorConfigData,
↳ residual: ZeroModeRuntime.ResidualCursor, consumed: *u128) !void {
            _ = limits;
            _ = config_ptr;
            return acceptBcTopForm(rule, residual, consumed);
        }

        fn emit(comptime limits: BranchLimits, config_ptr: *const CorrelatorConfigData,
↳ residual: ZeroModeRuntime.ResidualCursor, consumed: *u128, sink: anytype) !void {
            _ = limits;
            _ = config_ptr;
            return emitBcTopForm(rule, residual, consumed, sink);
        }

        fn emitEmpty(config_ptr: *const CorrelatorConfigData, sink: anytype) !void {
            _ = config_ptr;
            _ = sink;
        }
    };
}

fn EtaXiZeroModeProcedure(comptime rule: zero_mode.EtaXiZeroMode) type {
    return struct {
        fn canConsumeKind(kind: operators.OperatorKindId) bool {
            return kindIn(kind, rule.xi_kind_ids);
        }

        fn accept(comptime limits: BranchLimits, config_ptr: *const CorrelatorConfigData,
↳ residual: ZeroModeRuntime.ResidualCursor, consumed: *u128) !void {
            _ = limits;
            _ = config_ptr;
            return acceptEtaXiZeroMode(rule, residual, consumed);
        }

        fn emit(comptime limits: BranchLimits, config_ptr: *const CorrelatorConfigData,
↳ residual: ZeroModeRuntime.ResidualCursor, consumed: *u128, sink: anytype) !void {
            _ = limits;
            _ = config_ptr;
            return emitEtaXiZeroMode(rule, residual, consumed, sink);
        }

        fn emitEmpty(config_ptr: *const CorrelatorConfigData, sink: anytype) !void {

```

```

        _ = config_ptr;
        _ = sink;
    }
};
}

fn ConstantFermionProcedure(comptime rule: zero_mode.ConstantFermion) type {
    const adapted = zero_mode.EtaXiZeroMode{
        .support = rule.support,
        .xi_kind_ids = rule.fermion_kind_ids,
        .normalization = rule.normalization,
    };
    return EtaXiZeroModeProcedure(adapted);
}

fn TopFormFermionProcedure(comptime rule: zero_mode.TopFormFermion) type {
    const adapted = zero_mode.BcTopForm{
        .support = rule.support,
        .c_kind_ids = rule.field_kind_ids,
        .normalization = rule.normalization,
    };
    return BcTopFormProcedure(adapted);
}

fn FreeBosonConstantModeProcedure(comptime rule: zero_mode.FreeBosonConstantMode) type {
    return struct {
        fn canConsumeKind(kind: operators.OperatorKindId) bool {
            return kindIn(kind, rule.exp_kind_ids) or kindIn(kind, rule.profile_kind_ids);
        }

        fn accept(comptime limits: BranchLimits, config_ptr: *const CorrelatorConfigData,
            ↪ residual: ZeroModeRuntime.ResidualCursor, consumed: *u128) !void {
            return acceptFreeBosonConstantMode(limits, config_ptr, rule, residual, consumed);
        }

        fn emit(comptime limits: BranchLimits, config_ptr: *const CorrelatorConfigData,
            ↪ residual: ZeroModeRuntime.ResidualCursor, consumed: *u128, sink: anytype) !void {
            return emitFreeBosonConstantMode(limits, config_ptr, rule, residual, consumed,
                ↪ sink);
        }

        fn emitEmpty(config_ptr: *const CorrelatorConfigData, sink: anytype) !void {
            return emitEmptyFreeBosonConstantMode(config_ptr, rule, sink);
        }
    };
}

fn ZeroModeProcedure(comptime expr: zero_mode.Expr) type {
    return switch (expr) {
        .bc_top_form => |rule| BcTopFormProcedure(rule),
        .free_boson_constant_mode => |rule| FreeBosonConstantModeProcedure(rule),
        .eta_xi_zero_mode => |rule| EtaXiZeroModeProcedure(rule),
        .constant_fermion => |rule| ConstantFermionProcedure(rule),
        .top_form_fermion => |rule| TopFormFermionProcedure(rule),
        .boson_momentum_conservation => |rule| FreeBosonConstantModeProcedure(rule),
    };
}

fn generatedZeroModeCanConsumeKind(comptime ConfigHandle: type, kind:
    ↪ operators.OperatorKindId) bool {
    inline for (ConfigHandle.impl.zero_modes) |rule| {
        if (ZeroModeProcedure(rule.expr).canConsumeKind(kind)) return true;
    }
    return false;
}

```

```

}

fn generatedZeroModeCursorSucceeds(comptime ConfigHandle: type, config_ptr: *const
↳ CorrelatorConfigData, residual: ZeroModeRuntime.ResidualCursor) !bool {
    const limits = ConfigHandle.branch_limits;
    if (residual.len() > limits.max_zero_mode_residuals) return
    ↳ error.TooManyZeroModeResiduals;

    var consumed: u128 = 0;
    inline for (ConfigHandle.impl.zero_modes) |rule| {
        try ZeroModeProcedure(rule.expr).accept(limits, config_ptr, residual, &consumed);
    }

    return allResidualsConsumed(residual, consumed);
}

fn generatedZeroModeBaseCaseSucceeds(comptime ConfigHandle: type, config_ptr: *const
↳ CorrelatorConfigData, ops: kernel.Call.MultiOp, residual_indices: []const usize) !bool {
    if (residual_indices.len == 0) return true;
    const residual = ZeroModeRuntime.ResidualCursor{ .ops = ops, .indices = residual_indices
    ↳ };
    return generatedZeroModeCursorSucceeds(ConfigHandle, config_ptr, residual);
}

fn generatedEmptyZeroModeBaseCase(comptime ConfigHandle: type, config_ptr: *const
↳ CorrelatorConfigData, sink: anytype) !void {
    inline for (ConfigHandle.impl.zero_modes) |rule| {
        try ZeroModeProcedure(rule.expr).emitEmpty(config_ptr, sink);
    }
    try sink.emitZeroModeBaseEnd();
}

fn generatedTrustedZeroModeBaseCase(comptime ConfigHandle: type, config_ptr: *const
↳ CorrelatorConfigData, ops: kernel.Call.MultiOp, residual_indices: []const usize, sink:
↳ anytype) !void {
    const limits = ConfigHandle.branch_limits;
    if (residual_indices.len > limits.max_zero_mode_residuals) return
    ↳ error.TooManyZeroModeResiduals;
    if (residual_indices.len == 0) {
        try generatedEmptyZeroModeBaseCase(ConfigHandle, config_ptr, sink);
        return;
    }

    var consumed: u128 = 0;
    const residual = ZeroModeRuntime.ResidualCursor{ .ops = ops, .indices = residual_indices
    ↳ };
    // The branch walker has already dry-run validated these residuals.
    inline for (ConfigHandle.impl.zero_modes) |rule| {
        try ZeroModeProcedure(rule.expr).emit(limits, config_ptr, residual, &consumed, sink);
    }
    if (!allResidualsConsumed(residual, consumed)) return error.InvalidWickBranch;
    try sink.emitZeroModeBaseEnd();
}

/// generatedZeroModeBaseCase consumes residual operators with configured zero-mode
↳ procedures.
fn generatedZeroModeBaseCase(comptime ConfigHandle: type, config_ptr: *const
↳ CorrelatorConfigData, ops: kernel.Call.MultiOp, residual_indices: ?[]const usize, sink:
↳ anytype) !bool {
    const limits = ConfigHandle.branch_limits;
    const residual = ZeroModeRuntime.ResidualCursor{ .ops = ops, .indices = residual_indices
    ↳ };
    if (residual.len() > limits.max_zero_mode_residuals) return
    ↳ error.TooManyZeroModeResiduals;

```

```

    if (residual.len() == 0) {
        try generatedEmptyZeroModeBaseCase(ConfigHandle, config_ptr, sink);
        return true;
    }
    if (!try generatedZeroModeCursorSucceeds(ConfigHandle, config_ptr, residual)) return
    ↪ false;

    var consumed: u128 = 0;
    inline for (ConfigHandle.impl.zero_modes) |rule| {
        try ZeroModeProcedure(rule.expr).emit(limits, config_ptr, residual, &consumed, sink);
    }

    if (!allResidualsConsumed(residual, consumed)) return false;
    try sink.emitZeroModeBaseEnd();
    return true;
}

fn ruleMatches(rule: wick.Rule, left: kernel.Call.LocalOp, right: kernel.Call.LocalOp) ?bool {
    if (rule.left.kind == left.kind and rule.right.kind == right.kind) {
        return false;
    }
    if (rule.left.kind == right.kind and rule.right.kind == left.kind) {
        return true;
    }
    return null;
}

fn matchRule(config_ptr: *const CorrelatorConfigData, rule: *const wick.Rule, left:
    ↪ kernel.Call.LocalOp, right: kernel.Call.LocalOp, labels: *const kernel.Call.LabelStore)
    ↪ ?Correlator.WickPair {
    const reversed = ruleMatches(rule.*, left, right) orelse return null;
    return .{
        .rule = rule,
        .config = config_ptr,
        .left = left,
        .right = right,
        .labels = labels,
        .reversed = reversed,
    };
}

fn lowerBoundPairLookup(entries: []const PairLookupEntry, key: u64) usize {
    var lo: usize = 0;
    var hi: usize = entries.len;
    while (lo < hi) {
        const mid = lo + ((hi - lo) / 2);
        if (entries[mid].key < key) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }
    return lo;
}

fn emitPrimitiveWickTerm(pair: Correlator.WickPair, left_index: usize, right_index: usize,
    ↪ term_index: usize, sink: anytype) !void {
    const primitive = (Correlator.PrimitiveWickPair{ .pair = pair }).term(term_index) orelse
    ↪ return error.InvalidWickTerm;

    const event = Correlator.WickTermEvent{
        .left_index = left_index,
        .right_index = right_index,
        .term_index = term_index,
    }

```

```

};
try sink.emitWickTermStart(event);

var scalar_index: usize = 0;
while (scalar_index < primitive.scalarCount()) : (scalar_index += 1) {
    try sink.emitWickScalar(primitive.scalar(scalar_index) orelse return
        ↪ error.InvalidWickFactor);
}

var coordinate_index: usize = 0;
while (coordinate_index < primitive.coordinateCount()) : (coordinate_index += 1) {
    try sink.emitWickCoordinate(primitive.coordinate(coordinate_index) orelse return
        ↪ error.InvalidWickFactor);
}

var tensor_index: usize = 0;
while (tensor_index < primitive.tensorCount()) : (tensor_index += 1) {
    try sink.emitWickTensor(primitive.tensor(tensor_index) orelse return
        ↪ error.InvalidWickFactor);
}

var action_index: usize = 0;
while (action_index < primitive.actionCount()) : (action_index += 1) {
    try sink.emitWickAction(primitive.action(action_index) orelse return
        ↪ error.InvalidWickFactor);
}

var residual_index: usize = 0;
while (residual_index < primitive.residualCount()) : (residual_index += 1) {
    try emitWickResidualOperator(pair, left_index, right_index,
        ↪ primitive.residual(residual_index) orelse return error.InvalidWickFactor, sink);
}

try sink.emitWickTermEnd();
}

fn wickPairTermPayloadFrom(rule_index: usize, term_index: usize, reversed: bool) !u64 {
    if (rule_index > std.math.maxInt(u32)) return error.TooManyWickRules;
    if (term_index > std.math.maxInt(u16)) return error.TooManyWickTerms;
    return @as(u64, @intCast(rule_index)) |
        (@as(u64, @intCast(term_index)) << 32) |
        (@as(u64, @intFromBool(reversed)) << 48);
}

fn wickPayloadRuleIndex(payload: u64) usize {
    return @intCast(payload & 0xffff_ffff);
}

fn wickPayloadTermIndex(payload: u64) usize {
    return @intCast((payload >> 32) & 0xffff);
}

fn wickPayloadReversed(payload: u64) bool {
    return ((payload >> 48) & 1) != 0;
}

fn actualSideIndex(pair: Correlator.WickPair, left_index: usize, right_index: usize, side:
    ↪ wick.Side) usize {
    return switch (side) {
        .left => if (pair.reversed) right_index else left_index,
        .right => if (pair.reversed) left_index else right_index,
    };
}

```

```

fn emitWickResidualOperator(pair: Correlator.WickPair, left_index: usize, right_index: usize,
↳ side: wick.Side, sink: anytype) !void {
    const SinkPtr = @TypeOf(sink);
    const sink_info = @TypeInfo(SinkPtr);
    if (sink_info != .pointer) return;
    const Sink = sink_info.pointer.child;
    if (!@hasDecl(Sink, "emitWickResidualOperator")) return;

    const input_index = actualSideIndex(pair, left_index, right_index, side);
    const operator = switch (side) {
        .left => pair.sideOp(.left),
        .right => pair.sideOp(.right),
    };
    try sink.emitWickResidualOperator(Correlator.WickResidualRef{
        .input_index = input_index,
        .operator = operator,
    });
}

const cache_pair_direct_width = 16;
const cache_pair_hash_slots = cache_pair_direct_width * cache_pair_direct_width;
const no_cached_pair = std.math.maxInt(u16);

fn branchBudgetOperatorCount(comptime budget: BranchBudget) usize {
    return switch (budget) {
        .standard => 128,
    };
}

fn branchBudgetCacheTerms(comptime budget: BranchBudget) usize {
    return switch (budget) {
        .standard => 66,
    };
}

fn maxTermFactors(comptime rules: []const wick.Rule, comptime selector: enum { scalars,
↳ coordinates, tensors, actions, residuals }) usize {
    var result: usize = 0;
    for (rules) |rule| {
        for (rule.expr.terms) |term| {
            const count = switch (selector) {
                .scalars => term.scalars.len,
                .coordinates => term.coordinates.len,
                .tensors => term.tensors.len,
                .actions => term.actions.len,
                .residuals => term.residuals.len,
            };
            if (count > result) result = count;
        }
    }
    return result;
}

fn atLeastOne(value: usize) usize {
    return if (value == 0) 1 else value;
}

fn branchLimits(comptime input: CorrelatorConfigInput) BranchLimits {
    const max_operators = branchBudgetOperatorCount(input.branch_budget);
    if (max_operators > 128) @compileError("zero-mode consumption mask supports at most 128
↳ residual operators");
    const max_terms = max_operators / 2;
    const max_cached_terms = branchBudgetCacheTerms(input.branch_budget);
    const scalar_terms = atLeastOne(maxTermFactors(input.wick_rules, .scalars));
}

```

```

const coordinate_terms = atLeastOne(maxTermFactors(input.wick_rules, .coordinates));
const tensor_terms = atLeastOne(maxTermFactors(input.wick_rules, .tensors));
const action_terms = atLeastOne(maxTermFactors(input.wick_rules, .actions));
const residual_terms = atLeastOne(maxTermFactors(input.wick_rules, .residuals));

return .{
    .max_operators = max_operators,
    .max_zero_mode_residuals = max_operators,
    .max_zero_mode_items = max_operators,
    .max_terms = max_terms,
    .max_scalars = scalar_terms * max_terms,
    .max_coordinates = coordinate_terms * max_terms,
    .max_tensors = tensor_terms * max_terms,
    .max_actions = action_terms * max_terms,
    .max_residuals = residual_terms * max_terms,
    .max_cached_terms = max_cached_terms,
    .max_cached_scalars = scalar_terms * max_cached_terms,
    .max_cached_coordinates = coordinate_terms * max_cached_terms,
    .max_cached_tensors = tensor_terms * max_cached_terms,
    .max_cached_actions = action_terms * max_cached_terms,
    .max_cached_residuals = residual_terms * max_cached_terms,
};
}

const AccumulatedWickTerm = struct {
    event: Correlator.WickTermEvent,
    cache_index: u16 = no_cached_pair,
    scalar_start: u16,
    scalar_count: u16 = 0,
    coordinate_start: u16,
    coordinate_count: u16 = 0,
    tensor_start: u16,
    tensor_count: u16 = 0,
    action_start: u16,
    action_count: u16 = 0,
    residual_start: u16,
    residual_count: u16 = 0,
};

const CachedPairKey = struct {
    left: u8,
    right: u8,
    payload: u64,
};

fn AccumulatedBranch(comptime limits: BranchLimits) type {
    return struct {
        const Self = @This();

        const Snapshot = struct {
            term_count: u16,
            scalar_count: u16,
            coordinate_count: u16,
            tensor_count: u16,
            action_count: u16,
            residual_count: u16,
            logical_scalar_count: u16,
            logical_coordinate_count: u16,
            logical_tensor_count: u16,
            logical_action_count: u16,
            logical_residual_count: u16,
        };

        terms: [limits.max_terms] AccumulatedWickTerm = undefined,
    };
}

```



```

term_count: u16 = 0,
scalars: [limits.max_scalars]Correlator.ResolvedScalarFactor = undefined,
scalar_count: u16 = 0,
coordinates: [limits.max_coordinates]coefficient.CoordinateFactor = undefined,
coordinate_count: u16 = 0,
tensors: [limits.max_tensors]Correlator.ResolvedTensorFactor = undefined,
tensor_count: u16 = 0,
actions: [limits.max_actions]Correlator.ResolvedActionFactor = undefined,
action_count: u16 = 0,
residuals: [limits.max_residuals]Correlator.WickResidualRef = undefined,
residual_count: u16 = 0,
cache_keys: [limits.max_cached_terms]CachedPairKey = undefined,
cache_pair_index: [cache_pair_hash_slots]u16 = [_]u16{no_cached_pair} **
↳ cache_pair_hash_slots,
cache_terms: [limits.max_cached_terms]AccumulatedWickTerm = undefined,
cache_count: u16 = 0,
cache_scalars: [limits.max_cached_scalars]Correlator.ResolvedScalarFactor = undefined,
cache_scalar_count: u16 = 0,
cache_coordinates: [limits.max_cached_coordinates]coefficient.CoordinateFactor =
↳ undefined,
cache_coordinate_count: u16 = 0,
cache_tensors: [limits.max_cached_tensors]Correlator.ResolvedTensorFactor = undefined,
cache_tensor_count: u16 = 0,
cache_actions: [limits.max_cached_actions]Correlator.ResolvedActionFactor = undefined,
cache_action_count: u16 = 0,
cache_residuals: [limits.max_cached_residuals]Correlator.WickResidualRef = undefined,
cache_residual_count: u16 = 0,
logical_scalar_count: u16 = 0,
logical_coordinate_count: u16 = 0,
logical_tensor_count: u16 = 0,
logical_action_count: u16 = 0,
logical_residual_count: u16 = 0,

/// init constructs an empty branch factor accumulator.
pub fn init() Self {
    return .{};
}

/// snapshot records current array lengths for backtracking.
pub fn snapshot(self: *const Self) Snapshot {
    return .{
        .term_count = self.term_count,
        .scalar_count = self.scalar_count,
        .coordinate_count = self.coordinate_count,
        .tensor_count = self.tensor_count,
        .action_count = self.action_count,
        .residual_count = self.residual_count,
        .logical_scalar_count = self.logical_scalar_count,
        .logical_coordinate_count = self.logical_coordinate_count,
        .logical_tensor_count = self.logical_tensor_count,
        .logical_action_count = self.logical_action_count,
        .logical_residual_count = self.logical_residual_count,
    };
}

/// restore truncates accumulated factors to a previous snapshot.
pub fn restore(self: *Self, saved: Snapshot) void {
    self.term_count = saved.term_count;
    self.scalar_count = saved.scalar_count;
    self.coordinate_count = saved.coordinate_count;
    self.tensor_count = saved.tensor_count;
    self.action_count = saved.action_count;
    self.residual_count = saved.residual_count;
    self.logical_scalar_count = saved.logical_scalar_count;

```

```

        self.logical_coordinate_count = saved.logical_coordinate_count;
        self.logical_tensor_count = saved.logical_tensor_count;
        self.logical_action_count = saved.logical_action_count;
        self.logical_residual_count = saved.logical_residual_count;
    }

    fn currentTerm(self: *Self) *AccumulatedWickTerm {
        return &self.terms[self.term_count - 1];
    }

    fn cachePairSlot(left: usize, right: usize) usize {
        if (left < cache_pair_direct_width and right < cache_pair_direct_width) {
            return left * cache_pair_direct_width + right;
        }
        return ((left * 131) ^ right) & (cache_pair_hash_slots - 1);
    }

    fn findCache(self: *const Self, left: usize, right: usize, payload: u64) ?u16 {
        const direct = self.cache_pair_index[cachePairSlot(left, right)];
        if (direct != no_cached_pair) {
            const key = self.cache_keys[direct];
            if (key.left == left and key.right == right and key.payload == payload) return
                ↪ direct;
        }

        var index: u16 = 0;
        while (index < self.cache_count) : (index += 1) {
            const key = self.cache_keys[index];
            if (key.left == left and key.right == right and key.payload == payload) return
                ↪ index;
        }
        return null;
    }

    fn appendCachedTerm(self: *Self, cache_index: u16) !void {
        if (self.term_count == self.terms.len) return error.TooManyAccumulatedWickTerms;
        const cached = self.cache_terms[cache_index];
        self.terms[self.term_count] = cached;
        self.terms[self.term_count].cache_index = cache_index;
        self.term_count += 1;
        self.logical_scalar_count += cached.scalar_count;
        self.logical_coordinate_count += cached.coordinate_count;
        self.logical_tensor_count += cached.tensor_count;
        self.logical_action_count += cached.action_count;
        self.logical_residual_count += cached.residual_count;
    }

    /// primitiveTermCount returns the number of primitive pair terms in this branch.
    pub fn primitiveTermCount(self: *const Self) usize {
        return self.term_count;
    }

    /// scalarFactorCount returns the number of scalar factors in this branch.
    pub fn scalarFactorCount(self: *const Self) usize {
        return self.logical_scalar_count;
    }

    /// coordinateFactorCount returns the number of coordinate factors in this branch.
    pub fn coordinateFactorCount(self: *const Self) usize {
        return self.logical_coordinate_count;
    }

    /// tensorFactorCount returns the number of tensor factors in this branch.
    pub fn tensorFactorCount(self: *const Self) usize {

```

```

        return self.logical_tensor_count;
    }

    /// actionFactorCount returns the number of action factors in this branch.
    pub fn actionFactorCount(self: *const Self) usize {
        return self.logical_action_count;
    }

    /// residualFactorCount returns the number of residual operator refs in this branch.
    pub fn residualFactorCount(self: *const Self) usize {
        return self.logical_residual_count;
    }

    /// pushPairTerm appends one cached or freshly resolved primitive pair term.
    pub fn pushPairTerm(branch: *Self, config_ptr: *const CorrelatorConfigData, ops:
↳ kernel.Call.MultiOp, left_index: usize, right_index: usize, payload: u64) !void {
        if (branch.findCache(left_index, right_index, payload)) |cache_index| {
            try branch.appendCachedTerm(cache_index);
            return;
        }

        if (branch.cache_count == branch.cache_terms.len) {
            try emitWickPairTermPayloadData(config_ptr, ops, left_index, right_index,
↳ payload, branch);
            return;
        }

        const cache_index = branch.cache_count;
        const CacheSink = struct {
            branch: *Self,
            cache_index: u16,

            fn current(self: *@This()) *AccumulatedWickTerm {
                return &self.branch.cache_terms[self.cache_index];
            }
        }

        pub fn emitWickTermStart(self: *@This(), event: Correlator.WickTermEvent)
↳ !void {
            self.branch.cache_terms[self.cache_index] = .{
                .event = event,
                .scalar_start = self.branch.cache_scalar_count,
                .coordinate_start = self.branch.cache_coordinate_count,
                .tensor_start = self.branch.cache_tensor_count,
                .action_start = self.branch.cache_action_count,
                .residual_start = self.branch.cache_residual_count,
            };
        }

        pub fn emitWickScalar(self: *@This(), factor: Correlator.ResolvedScalarFactor)
↳ !void {
            if (self.branch.cache_scalar_count == self.branch.cache_scalars.len)
↳ return error.TooManyCachedScalars;
            self.branch.cache_scalars[self.branch.cache_scalar_count] = factor;
            self.branch.cache_scalar_count += 1;
            self.current().scalar_count += 1;
        }

        pub fn emitWickCoordinate(self: *@This(), factor:
↳ coefficient.CoordinateFactor) !void {
            if (self.branch.cache_coordinate_count ==
↳ self.branch.cache_coordinates.len) return
↳ error.TooManyCachedCoordinates;
            self.branch.cache_coordinates[self.branch.cache_coordinate_count] =
↳ factor;
        }

```

```

        self.branch.cache_coordinate_count += 1;
        self.current().coordinate_count += 1;
    }

    pub fn emitWickTensor(self: *@This(), factor: Correlator.ResolvedTensorFactor)
    ↪ !void {
        if (self.branch.cache_tensor_count == self.branch.cache_tensors.len)
            ↪ return error.TooManyCachedTensors;
        self.branch.cache_tensors[self.branch.cache_tensor_count] = factor;
        self.branch.cache_tensor_count += 1;
        self.current().tensor_count += 1;
    }

    pub fn emitWickAction(self: *@This(), factor: Correlator.ResolvedActionFactor)
    ↪ !void {
        if (self.branch.cache_action_count == self.branch.cache_actions.len)
            ↪ return error.TooManyCachedActions;
        self.branch.cache_actions[self.branch.cache_action_count] = factor;
        self.branch.cache_action_count += 1;
        self.current().action_count += 1;
    }

    pub fn emitWickResidualOperator(self: *@This(), residual:
    ↪ Correlator.WickResidualRef) !void {
        if (self.branch.cache_residual_count == self.branch.cache_residuals.len)
            ↪ return error.TooManyCachedResiduals;
        self.branch.cache_residuals[self.branch.cache_residual_count] = residual;
        self.branch.cache_residual_count += 1;
        self.current().residual_count += 1;
    }

    pub fn emitWickTermEnd(_: *@This()) !void {}
};

var cache_sink = CacheSink{ .branch = branch, .cache_index = cache_index };
try emitWickPairTermPayloadData(config_ptr, ops, left_index, right_index, payload,
    ↪ &cache_sink);
branch.cache_keys[cache_index] = .{ .left = @intCast(left_index), .right =
    ↪ @intCast(right_index), .payload = payload };
const direct_slot = cachePairSlot(left_index, right_index);
if (branch.cache_pair_index[direct_slot] == no_cached_pair)
    ↪ branch.cache_pair_index[direct_slot] = cache_index;
branch.cache_count += 1;
try branch.appendCachedTerm(cache_index);
}

/// emitWickTermStart appends one primitive term boundary.
pub fn emitWickTermStart(self: *Self, event: Correlator.WickTermEvent) !void {
    if (self.term_count == self.terms.len) return error.TooManyAccumulatedWickTerms;
    self.terms[self.term_count] = .{
        .event = event,
        .scalar_start = self.scalar_count,
        .coordinate_start = self.coordinate_count,
        .tensor_start = self.tensor_count,
        .action_start = self.action_count,
        .residual_start = self.residual_count,
    };
    self.term_count += 1;
}

/// emitWickScalar appends one scalar factor.
pub fn emitWickScalar(self: *Self, factor: Correlator.ResolvedScalarFactor) !void {
    if (self.scalar_count == self.scalars.len) return error.TooManyAccumulatedScalars;
    self.scalars[self.scalar_count] = factor;
}

```

```

        self.scalar_count += 1;
        self.currentTerm().scalar_count += 1;
        self.logical_scalar_count += 1;
    }

    /// emitWickCoordinate appends one coordinate factor.
    pub fn emitWickCoordinate(self: *Self, factor: coefficient.CoordinateFactor) !void {
        if (self.coordinate_count == self.coordinates.len) return
            ↪ error.TooManyAccumulatedCoordinates;
        self.coordinates[self.coordinate_count] = factor;
        self.coordinate_count += 1;
        self.currentTerm().coordinate_count += 1;
        self.logical_coordinate_count += 1;
    }

    /// emitWickTensor appends one tensor factor.
    pub fn emitWickTensor(self: *Self, factor: Correlator.ResolvedTensorFactor) !void {
        if (self.tensor_count == self.tensors.len) return error.TooManyAccumulatedTensors;
        self.tensors[self.tensor_count] = factor;
        self.tensor_count += 1;
        self.currentTerm().tensor_count += 1;
        self.logical_tensor_count += 1;
    }

    /// emitWickAction appends one action factor.
    pub fn emitWickAction(self: *Self, factor: Correlator.ResolvedActionFactor) !void {
        if (self.action_count == self.actions.len) return error.TooManyAccumulatedActions;
        self.actions[self.action_count] = factor;
        self.action_count += 1;
        self.currentTerm().action_count += 1;
        self.logical_action_count += 1;
    }

    /// emitWickResidualOperator appends one residual operator reference.
    pub fn emitWickResidualOperator(self: *Self, residual: Correlator.WickResidualRef)
        ↪ !void {
        if (self.residual_count == self.residuals.len) return
            ↪ error.TooManyAccumulatedResiduals;
        self.residuals[self.residual_count] = residual;
        self.residual_count += 1;
        self.currentTerm().residual_count += 1;
        self.logical_residual_count += 1;
    }

    /// emitWickTermEnd accepts the primitive term boundary.
    pub fn emitWickTermEnd(_: *Self) !void {}

    /// emit streams all accumulated primitive terms to the final sink.
    pub fn emit(self: *const Self, sink: anytype) !void {
        const SinkPtr = @TypeOf(sink);
        const sink_info = @TypeInfo(SinkPtr);
        if (sink_info == .pointer and @hasDecl(sink_info.pointer.child,
            ↪ "emitWickBranchTerm")) {
            try sink.emitWickBranchTerm(self);
            return;
        }

        const emit_residuals = sink_info == .pointer and @hasDecl(sink_info.pointer.child,
            ↪ "emitWickResidualOperator");
        const replay_actions = self.logical_action_count != 0;
        const replay_residuals = emit_residuals and self.logical_residual_count != 0;

        var term_index: usize = 0;
        while (term_index < self.term_count) : (term_index += 1) {

```

```

const term = self.terms[term_index];
const source_term = if (term.cache_index == no_cached_pair) term else
↳ self.cache_terms[term.cache_index];
try sink.emitWickTermStart(term.event);

var scalar_index: usize = 0;
while (scalar_index < term.scalar_count) : (scalar_index += 1) {
    if (term.cache_index == no_cached_pair) {
        try sink.emitWickScalar(self.scalars[source_term.scalar_start +
↳ scalar_index]);
    } else {
        try sink.emitWickScalar(self.cache_scalars[source_term.scalar_start +
↳ scalar_index]);
    }
}

var coordinate_index: usize = 0;
while (coordinate_index < term.coordinate_count) : (coordinate_index += 1) {
    if (term.cache_index == no_cached_pair) {
        try
↳ sink.emitWickCoordinate(self.coordinates[source_term.coordinate_start
↳ + coordinate_index]);
    } else {
        try
↳ sink.emitWickCoordinate(self.cache_coordinates[source_term.coordinate_start
↳ + coordinate_index]);
    }
}

var tensor_index: usize = 0;
while (tensor_index < term.tensor_count) : (tensor_index += 1) {
    if (term.cache_index == no_cached_pair) {
        try sink.emitWickTensor(self.tensors[source_term.tensor_start +
↳ tensor_index]);
    } else {
        try sink.emitWickTensor(self.cache_tensors[source_term.tensor_start +
↳ tensor_index]);
    }
}

if (replay_actions) {
    var action_index: usize = 0;
    while (action_index < term.action_count) : (action_index += 1) {
        if (term.cache_index == no_cached_pair) {
            try sink.emitWickAction(self.actions[source_term.action_start +
↳ action_index]);
        } else {
            try
↳ sink.emitWickAction(self.cache_actions[source_term.action_start
↳ + action_index]);
        }
    }
}

if (replay_residuals) {
    var residual_index: usize = 0;
    while (residual_index < term.residual_count) : (residual_index += 1) {
        if (term.cache_index == no_cached_pair) {
            try
↳ sink.emitWickResidualOperator(self.residuals[source_term.residual_start
↳ + residual_index]);
        } else {

```

```

        try
        ↪ sink.emitWickResidualOperator(self.cache_residuals[source_term.residual_start
        ↪ + residual_index]);
    }
}

    }
}

    try sink.emitWickTermEnd();
}
}
};
}

fn matchIndexedWickPairTerm(config_ptr: *const CorrelatorConfigData, ops: kernel.Call.MultiOp,
↪ left_index: usize, right_index: usize, left: kernel.Call.LocalOp, right:
↪ kernel.Call.LocalOp, term_ordinal: usize) !?Correlator.MatchedWickTerm {
    const key = wickRuleKey(left.kind, right.kind);
    var entry_index = lowerBoundPairLookup(config_ptr.pair_lookup, key);
    var remaining = term_ordinal;

    while (entry_index < config_ptr.pair_lookup.len and
    ↪ config_ptr.pair_lookup[entry_index].key == key) : (entry_index += 1) {
        const entry = config_ptr.pair_lookup[entry_index];
        const rule_index: usize = @intCast(entry.rule_index);
        if (rule_index >= config_ptr.wick_rules.len) return error.InvalidPairLookup;
        const rule = &config_ptr.wick_rules[rule_index];
        if (remaining < rule.expr.terms.len) {
            return .{ .pair = .{
                .rule = rule,
                .config = config_ptr,
                .left = left,
                .right = right,
                .labels = ops.labels,
                .reversed = entry.reversed,
            }, .rule_index = rule_index, .term_index = remaining };
        }
        remaining -= rule.expr.terms.len;
    }

    _ = left_index;
    _ = right_index;
    return null;
}

fn matchScannedWickPairTerm(config_ptr: *const CorrelatorConfigData, ops: kernel.Call.MultiOp,
↪ left_index: usize, right_index: usize, left: kernel.Call.LocalOp, right:
↪ kernel.Call.LocalOp, term_ordinal: usize) ?Correlator.MatchedWickTerm {
    var remaining = term_ordinal;

    for (config_ptr.wick_rules, 0..) |*rule, rule_index| {
        if (matchRule(config_ptr, rule, left, right, ops.labels)) |pair| {
            if (remaining < pair.rule.expr.terms.len) {
                return .{ .pair = pair, .rule_index = rule_index, .term_index = remaining };
            }
            remaining -= pair.rule.expr.terms.len;
        }
    }

    _ = left_index;
    _ = right_index;
    return null;
}

```

```

fn matchWickPairTerm(config_ptr: *const CorrelatorConfigData, ops: kernel.Call.MultiOp,
↳ left_index: usize, right_index: usize, term_ordinal: usize) !?Correlator.MatchedWickTerm {
    if (left_index >= ops.operators.len or right_index >= ops.operators.len or left_index ==
↳ right_index) return error.InvalidOperatorIndex;

    const left = ops.operators[left_index];
    const right = ops.operators[right_index];
    if (config_ptr.pair_lookup.len != 0) {
        return matchIndexedWickPairTerm(config_ptr, ops, left_index, right_index, left, right,
↳ term_ordinal);
    }

    return matchScannedWickPairTerm(config_ptr, ops, left_index, right_index, left, right,
↳ term_ordinal);
}

fn emitIndexedWickPairTerms(config_ptr: *const CorrelatorConfigData, ops: kernel.Call.MultiOp,
↳ left_index: usize, right_index: usize, left: kernel.Call.LocalOp, right:
↳ kernel.Call.LocalOp, sink: anytype) !usize {
    const key = wickRuleKey(left.kind, right.kind);
    var entry_index = lowerBoundPairLookup(config_ptr.pair_lookup, key);
    var emitted: usize = 0;

    while (entry_index < config_ptr.pair_lookup.len and
↳ config_ptr.pair_lookup[entry_index].key == key) : (entry_index += 1) {
        const entry = config_ptr.pair_lookup[entry_index];
        const rule_index: usize = @intCast(entry.rule_index);
        if (rule_index >= config_ptr.wick_rules.len) return error.InvalidPairLookup;
        const rule = &config_ptr.wick_rules[rule_index];
        const pair = Correlator.WickPair{
            .rule = rule,
            .config = config_ptr,
            .left = left,
            .right = right,
            .labels = ops.labels,
            .reversed = entry.reversed,
        };

        var term_index: usize = 0;
        while (term_index < pair.rule.expr.terms.len) : (term_index += 1) {
            try emitPrimitiveWickTerm(pair, left_index, right_index, term_index, sink);
            emitted += 1;
        }
    }

    return emitted;
}

fn emitScannedWickPairTerms(config_ptr: *const CorrelatorConfigData, ops: kernel.Call.MultiOp,
↳ left_index: usize, right_index: usize, left: kernel.Call.LocalOp, right:
↳ kernel.Call.LocalOp, sink: anytype) !usize {
    var emitted: usize = 0;

    for (config_ptr.wick_rules) |*rule| {
        if (matchRule(config_ptr, rule, left, right, ops.labels)) |pair| {
            var term_index: usize = 0;
            while (term_index < pair.rule.expr.terms.len) : (term_index += 1) {
                try emitPrimitiveWickTerm(pair, left_index, right_index, term_index, sink);
                emitted += 1;
            }
        }
    }

    return emitted;
}

```



```

}

fn countIndexedWickPairTerms(config_ptr: *const CorrelatorConfigData, left:
↳ kernel.Call.LocalOp, right: kernel.Call.LocalOp) !usize {
    const key = wickRuleKey(left.kind, right.kind);
    var entry_index = lowerBoundPairLookup(config_ptr.pair_lookup, key);
    var count: usize = 0;

    while (entry_index < config_ptr.pair_lookup.len and
↳ config_ptr.pair_lookup[entry_index].key == key) : (entry_index += 1) {
        const rule_index: usize = @intCast(config_ptr.pair_lookup[entry_index].rule_index);
        if (rule_index >= config_ptr.wick_rules.len) return error.InvalidPairLookup;
        count += config_ptr.wick_rules[rule_index].expr.terms.len;
    }

    return count;
}

fn countScannedWickPairTerms(config_ptr: *const CorrelatorConfigData, left:
↳ kernel.Call.LocalOp, right: kernel.Call.LocalOp) usize {
    var count: usize = 0;
    for (config_ptr.wick_rules) |rule| {
        if (ruleMatches(rule, left, right) != null) count += rule.expr.terms.len;
    }
    return count;
}

/// countWickPairTerms counts primitive terms for two input positions without resolving
↳ factors.
fn countWickPairTermsData(config_ptr: *const CorrelatorConfigData, ops: kernel.Call.MultiOp,
↳ left_index: usize, right_index: usize) !usize {
    if (left_index >= ops.operators.len or right_index >= ops.operators.len or left_index ==
↳ right_index) return error.InvalidOperatorIndex;

    const left = ops.operators[left_index];
    const right = ops.operators[right_index];
    if (config_ptr.pair_lookup.len != 0) {
        return countIndexedWickPairTerms(config_ptr, left, right);
    }

    return countScannedWickPairTerms(config_ptr, left, right);
}

fn emitWickPairTermsData(config_ptr: *const CorrelatorConfigData, ops: kernel.Call.MultiOp,
↳ left_index: usize, right_index: usize, sink: anytype) !usize {
    if (left_index >= ops.operators.len or right_index >= ops.operators.len or left_index ==
↳ right_index) return error.InvalidOperatorIndex;

    const left = ops.operators[left_index];
    const right = ops.operators[right_index];
    if (config_ptr.pair_lookup.len != 0) {
        return emitIndexedWickPairTerms(config_ptr, ops, left_index, right_index, left, right,
↳ sink);
    }

    return emitScannedWickPairTerms(config_ptr, ops, left_index, right_index, left, right,
↳ sink);
}

fn emitWickPairTermData(config_ptr: *const CorrelatorConfigData, ops: kernel.Call.MultiOp,
↳ left_index: usize, right_index: usize, term_ordinal: usize, sink: anytype) !void {
    const matched = try matchWickPairTerm(config_ptr, ops, left_index, right_index,
↳ term_ordinal) orelse return error.InvalidWickTerm;

```

```

    try emitPrimitiveWickTerm(matched.pair, left_index, right_index, matched.term_index,
        ↪ sink);
}

fn wickPairTermPayloadData(config_ptr: *const CorrelatorConfigData, ops: kernel.Call.MultiOp,
    ↪ left_index: usize, right_index: usize, term_ordinal: usize) !u64 {
    const matched = try matchWickPairTerm(config_ptr, ops, left_index, right_index,
        ↪ term_ordinal) orelse return error.InvalidWickTerm;
    return wickPairTermPayloadFrom(matched.rule_index, matched.term_index,
        ↪ matched.pair.reversed);
}

fn wickPairTermInfoData(config_ptr: *const CorrelatorConfigData, ops: kernel.Call.MultiOp,
    ↪ left_index: usize, right_index: usize, term_ordinal: usize, residuals: *[2]usize, payload:
    ↪ *u64, residual_count: *usize) !void {
    const matched = try matchWickPairTerm(config_ptr, ops, left_index, right_index,
        ↪ term_ordinal) orelse return error.InvalidWickTerm;
    const source = matched.pair.rule.expr.terms[matched.term_index].residuals;
    if (source.len > residuals.len) return error.TooManyWickTermResiduals;
    for (source, 0..) |side, index| {
        residuals[index] = actualSideIndex(matched.pair, left_index, right_index, side);
    }
    payload.* = try wickPairTermPayloadFrom(matched.rule_index, matched.term_index,
        ↪ matched.pair.reversed);
    residual_count.* = source.len;
}

fn emitWickPairTermPayloadData(config_ptr: *const CorrelatorConfigData, ops:
    ↪ kernel.Call.MultiOp, left_index: usize, right_index: usize, payload: u64, sink: anytype)
    ↪ !void {
    const rule_index = wickPayloadRuleIndex(payload);
    if (rule_index >= config_ptr.wick_rules.len) return error.InvalidWickTerm;
    const pair = Correlator.WickPair{
        .rule = &config_ptr.wick_rules[rule_index],
        .config = config_ptr,
        .left = ops.operators[left_index],
        .right = ops.operators[right_index],
        .labels = ops.labels,
        .reversed = wickPayloadReversed(payload),
    };
    try emitPrimitiveWickTerm(pair, left_index, right_index, wickPayloadTermIndex(payload),
        ↪ sink);
}

fn wickPairTermResidualsData(config_ptr: *const CorrelatorConfigData, ops:
    ↪ kernel.Call.MultiOp, left_index: usize, right_index: usize, term_ordinal: usize,
    ↪ residuals: *[2]usize) !usize {
    const matched = try matchWickPairTerm(config_ptr, ops, left_index, right_index,
        ↪ term_ordinal) orelse return error.InvalidWickTerm;
    const source = matched.pair.rule.expr.terms[matched.term_index].residuals;
    if (source.len > residuals.len) return error.TooManyWickTermResiduals;
    for (source, 0..) |side, index| {
        residuals[index] = actualSideIndex(matched.pair, left_index, right_index, side);
    }
    return source.len;
}

fn RuntimeWickTactic(comptime ConfigHandle: type) type {
    return struct {
        /// Config is the shared preset runtime correlator config.
        pub const Config = CorrelatorConfigData;
        const traversal_operator_count = ConfigHandle.branch_limits.max_operators;
        /// BranchState accumulates resolved Wick factors along one DFS branch.
        pub const BranchState = AccumulatedBranch(ConfigHandle.branch_limits);
    };
}

```

```

/// emitPairTerms adapts original occurrence pairs to the primitive Wick emitter.
pub fn emitPairTerms(config_ptr: *const Config, ops: kernel.Call.MultiOp, left_index:
↳ usize, right_index: usize, sink: anytype) !usize {
    return emitWickPairTermsData(config_ptr, ops, left_index, right_index, sink);
}

/// emitPairTerm emits one selected primitive Wick term.
pub fn emitPairTerm(config_ptr: *const Config, ops: kernel.Call.MultiOp, left_index:
↳ usize, right_index: usize, term_ordinal: usize, sink: anytype) !void {
    return emitWickPairTermData(config_ptr, ops, left_index, right_index,
↳ term_ordinal, sink);
}

/// pairTermCount counts primitive pair terms without emitting factors.
pub fn pairTermCount(config_ptr: *const Config, ops: kernel.Call.MultiOp, left_index:
↳ usize, right_index: usize) !usize {
    return countWickPairTermsData(config_ptr, ops, left_index, right_index);
}

/// pairTermResiduals returns original residual occurrences for one primitive term.
pub fn pairTermResiduals(config_ptr: *const Config, ops: kernel.Call.MultiOp,
↳ left_index: usize, right_index: usize, term_ordinal: usize, residuals: *[2]usize)
↳ !usize {
    return wickPairTermResidualsData(config_ptr, ops, left_index, right_index,
↳ term_ordinal, residuals);
}

/// pairTermPayload returns cached rule payload for a primitive pair term.
pub fn pairTermPayload(config_ptr: *const Config, ops: kernel.Call.MultiOp,
↳ left_index: usize, right_index: usize, term_ordinal: usize) !u64 {
    return wickPairTermPayloadData(config_ptr, ops, left_index, right_index,
↳ term_ordinal);
}

/// pairTermInfo returns residual occurrences and cached replay payload from one pair
↳ match.
pub fn pairTermInfo(config_ptr: *const Config, ops: kernel.Call.MultiOp, left_index:
↳ usize, right_index: usize, term_ordinal: usize, residuals: *[2]usize, payload:
↳ *u64, residual_count: *usize) !void {
    return wickPairTermInfoData(config_ptr, ops, left_index, right_index,
↳ term_ordinal, residuals, payload, residual_count);
}

/// emitPairTermPayload replays one primitive term without matching rules again.
pub fn emitPairTermPayload(config_ptr: *const Config, ops: kernel.Call.MultiOp,
↳ left_index: usize, right_index: usize, payload: u64, sink: anytype) !void {
    return emitWickPairTermPayloadData(config_ptr, ops, left_index, right_index,
↳ payload, sink);
}

/// pushBranchPairTerm appends one primitive pair term to a branch accumulator.
pub fn pushBranchPairTerm(config_ptr: *const Config, ops: kernel.Call.MultiOp,
↳ left_index: usize, right_index: usize, payload: u64, branch: *BranchState) !void {
    return branch.pushPairTerm(config_ptr, ops, left_index, right_index, payload);
}

/// emitAccumulatedBranch streams cached pair factors followed by the zero-mode base
↳ case.
pub fn emitAccumulatedBranch(config_ptr: *const Config, ops: kernel.Call.MultiOp,
↳ branch: *const BranchState, sign: i8, residual_indices: []const usize, sink:
↳ anytype) !void {
    if (sign < 0) {
        try emitBranchSign(config_ptr, sign, sink);
    }
}

```

```

    }
    try branch.emit(sink);
    try generatedTrustedZeroModeBaseCase(ConfigHandle, config_ptr, ops,
    ↪ residual_indices, sink);
}

/// emitFullContractionBranch streams a full Wick branch with the empty base case.
pub fn emitFullContractionBranch(config_ptr: *const Config, ops: kernel.Call.MultiOp,
    ↪ branch: *const BranchState, sign: i8, sink: anytype) !void {
    if (sign < 0) {
        try emitBranchSign(config_ptr, sign, sink);
    }
    try branch.emit(sink);
    _ = ops;
    try generatedEmptyZeroModeBaseCase(ConfigHandle, config_ptr, sink);
}

/// operatorParity returns true for configured fermionic operator kinds.
pub fn operatorParity(config_ptr: *const Config, ops: kernel.Call.MultiOp, index:
    ↪ usize) bool {
    const kind = ops.operators[index].kind;
    return kindIn(kind, config_ptr.fermion_kinds);
}

/// operatorCanRemainInZeroMode returns true when a residual can be consumed by zero
    ↪ modes.
pub fn operatorCanRemainInZeroMode(config_ptr: *const Config, ops:
    ↪ kernel.Call.MultiOp, index: usize) bool {
    _ = config_ptr;
    return generatedZeroModeCanConsumeKind(ConfigHandle, ops.operators[index].kind);
}

/// emitBranchSign forwards an accumulated fermion sign when the sink accepts it.
pub fn emitBranchSign(_: *const Config, sign: i8, sink: anytype) !void {
    const SinkPtr = @TypeOf(sink);
    const sink_info = @TypeInfo(SinkPtr);
    if (sink_info != .pointer) return;
    const Sink = sink_info.pointer.child;
    if (@hasDecl(Sink, "emitWickBranchSign")) {
        try sink.emitWickBranchSign(sign);
    }
}

/// emitZeroModeBaseCase adapts residual original occurrences to the zero-mode
    ↪ evaluator.
pub fn emitZeroModeBaseCase(config_ptr: *const Config, ops: kernel.Call.MultiOp,
    ↪ residual_indices: []const usize, sink: anytype) !bool {
    return generatedZeroModeBaseCase(ConfigHandle, config_ptr, ops, residual_indices,
    ↪ sink);
}

/// zeroModeBaseCaseSucceeds tests base-case viability without emitting factors.
pub fn zeroModeBaseCaseSucceeds(config_ptr: *const Config, ops: kernel.Call.MultiOp,
    ↪ residual_indices: []const usize) !bool {
    return generatedZeroModeBaseCaseSucceeds(ConfigHandle, config_ptr, ops,
    ↪ residual_indices);
}
};
}

fn operatorListData(ops: anytype) KernelOperatorList {
    const Ops = @TypeOf(ops);
    if (Ops == KernelOperatorList) return ops;
    return switch (@TypeInfo(Ops)) {

```

```

        .pointer => |ptr| if (ptr.child == OperatorList) blk: {
            const state: *const LocalStateImpl = @ptrCast(@alignCast(ops));
            break :blk .{ .operators = state.owned_operators, .labels = &state.label_store };
        } else {
            @compileError("correlator operators must come from local.ops");
        },
        else => @compileError("correlator operators must come from local.ops"),
    };
}

/// streamCorrelator evaluates Wick branches and streams accepted base cases.
pub fn streamCorrelator(config_ptr: anytype, ops: anytype, sink: anytype) !void {
    const Ptr = @TypeOf(config_ptr);
    const ptr_info = @typeInfo(Ptr);
    if (ptr_info != .pointer) @compileError("correlator config must be passed by pointer");
    const ConfigHandle = ptr_info.pointer.child;
    const data = configData(config_ptr);
    return wick_correlator.wickCorrelator(RuntimeWickTactic(ConfigHandle), data,
        ↪ operatorListData(ops), sink);
}

```

3.4 Free Boson Preset

INTERFACE

FUNCTIONS

`freeBoson(comptime cfg: FreeBosonConfig) type`

`freeBoson` builds the generated preset type for a noncompact free-boson CFT.

`boundaryExtension(comptime cfg: FreeBosonBoundaryConfig) FreeBosonBoundaryExtension(cfg)`

`boundaryExtension` declares the Neumann/Dirichlet free-boson boundary extension.

This node defines the free-boson preset on the sphere and its Neumann/Dirichlet boundary extension. It uses the compact vocabulary from Preset Shared Runtime and is exported through Presets.

The preset does not perform tensor algebra. A D -dimensional target space only fixes which $SO(D)$ -type labels are expected; later coefficient evaluation uses Tensor-Code to evaluate metrics, momenta, and projectors.

```

const std = @import("std");
const shared = @import("shared.zig");
const declare = shared.declare;

const operators = @import("../expressions/operators.zig");
const Handle = shared.Handle;
const Local = shared.Local;
const Builder = *shared.Local;
const Operator = *shared.LocalOperator;
const Spec = declare.Spec;
const wick = declare.wick;
const zero_mode = declare.zero_mode;

```

The preset owns the free-boson operator vocabulary. The enum below is physics data; Preset Shared Runtime only receives the compact kind ids derived from it. The base separates this preset's ids from other implemented presets, while individual ids still come from enum order rather than handwritten numbers.

```

const namespace = "free_boson";
const scalars = declare.scalars;
const RuleScalar = @TypeOf(scalars.one());
const alpha_prime_atom = scalars.atom(namespace, "alpha_prime");
const k_disk_atom = scalars.atom(namespace, "K_D2");
const neumann_projector_ref = declare.configRef(namespace, "neumann_projector");
const dirichlet_projector_ref = declare.configRef(namespace, "dirichlet_projector");
const bulk_boundary_projector_ref = declare.configRef(namespace, "bulk_boundary_projector");
const dirichlet_position_ref = declare.configRef(namespace, "dirichlet_position");
const chan_paton_ref = declare.configRef(namespace, "chan_paton");
const target_dimension_ref = declare.configRef(namespace, "target_dimension");

const Family = enum {
    x,
    d_x,
    d_xt,
    exp_x,
    profile_x,
    d_x_boundary,
    exp_x_boundary,
    profile_x_boundary,
};

const ZeroSector = enum {
    bulk,
    boundary,
    torus,
};

fn kind(comptime family: Family) operators.OperatorKindId {
    return declare.familyKind(namespace, family);
}

fn zeroSector(comptime sector: ZeroSector) zero_mode.Sector {
    return declare.sectorId(namespace, sector);
}

fn rawToken(value: anytype) u32 {
    return @intFromEnum(value);
}

fn token(comptime T: type, id: u32) T {
    return @enumFromInt(if (id == 0) 1 else id);
}

fn stableNameId(comptime name: []const u8) u32 {
    var hash: u32 = 2166136261;
    inline for (name) |byte| {
        hash = (hash ^ @as(u32, byte)) *% 16777619;
    }
    return if (hash == 0) 1 else hash;
}

fn stableSubspaceId(comptime dimensions: []const u16) u32 {
    var hash: u32 = 2166136261;
    inline for (dimensions) |dimension| {
        hash = (hash ^ @as(u32, dimension & 0xff)) *% 16777619;
        hash = (hash ^ @as(u32, dimension >> 8)) *% 16777619;
    }
    return if (hash == 0) 1 else hash;
}

fn projectorId(comptime rank: u8, hash: u32) u32 {
    return (@as(u32, rank) << 24) | (hash & 0x00ff_ffff);
}

```

```

}

fn polynomialProfileTerm(term: anytype) struct { coefficient: Handle.ProfileCoefficient,
↪ power: u16 } {
    const Term = @TypeOf(term);
    const info = @TypeInfo(Term);
    if (info != .@"struct" or !info.@"struct".is_tuple or info.@"struct".fields.len != 2) {
        @compileError("polynomial profile terms must be .{ coefficient, power } tuples");
    }
    return .{ .coefficient = term[0], .power = term[1] };
}

fn polynomialProfileId(point: Handle.TargetPoint, terms: anytype) u32 {
    var hash = rawToken(point) ^ 0x9e37_79b9;
    inline for (terms) |raw_term| {
        const term = polynomialProfileTerm(raw_term);
        hash = (hash ^ rawToken(term.coefficient)) *% 16777619;
        hash = (hash ^ @as(u32, term.power)) *% 16777619;
    }
    return if (hash == 0) 1 else hash;
}

const TargetSpace = struct {
    /// subspace names a target-space projector onto a coordinate subspace.
    pub fn subspace(comptime dimensions: []const u16) Handle.TensorProjector {
        return token(Handle.TensorProjector, projectId(@intCast(dimensions.len),
        ↪ stableSubspaceId(dimensions)));
    }

    /// complement names the complementary target-space projector.
    pub fn complement(comptime dimensions: []const u16) Handle.TensorProjector {
        return token(Handle.TensorProjector, projectId(0, stableSubspaceId(dimensions) ^
        ↪ 0xa5a5_a5a5));
    }

    /// point names a target-space point used by boundary conditions.
    pub fn point(comptime name: []const u8) Handle.TargetPoint {
        return token(Handle.TargetPoint, stableNameId(name));
    }

    /// boundaryStack names Chan-Paton data for a stacked boundary condition.
    pub fn boundaryStack(comptime name: []const u8) Handle.BoundaryStack {
        return token(Handle.BoundaryStack, stableNameId(name));
    }
};

```

3.4.1 Bulk Preset

The default normalization is the stringbook free-field normalization

$$T^X = -\frac{1}{\alpha'} \partial X^\mu \partial X_\mu$$

[2]. The rule scalar remains symbolic here because different applications set α' and ∂X normalizations differently.

```

const FreeBosonConfig = struct {
    dimension: u16,
};

/// freeBoson builds the generated preset type for a noncompact free-boson CFT.
pub fn freeBoson(comptime cfg: FreeBosonConfig) type {
    return struct {
        /// op exposes free-boson local operator builders.
    };
}

```

```

pub const op = FreeBosonOp;
/// profile exposes profile-presentation builders.
pub const profile = ProfileOp;
/// target exposes deterministic target-space tokens for preset data.
pub const target = TargetSpace;
/// config exposes named correlator configs for this preset.
pub const config = FreeBosonCorrelatorConfig(cfg);
/// text exposes bounded result-inspection sinks.
pub const text = shared.text;
/// local constructs a label-preserving local-operator builder.
pub fn local(allocator: std.mem.Allocator) !Builder {
    return shared.Local.init(allocator);
}

/// correlator streams rule matches for free-boson insertions.
pub fn correlator(config_ptr: anytype, ops: anytype, sink: anytype) !void {
    return shared.streamCorrelator(config_ptr, ops, sink);
}
};
}

```

The derivative meaning is $dX(\mu, n, z) = \partial^{n+1} X^\mu(z)$. The explicit X body is kept because NLSM expansions and position-space profiles produce logarithmic Green kernels rather than only plane-wave contractions.

```

const FreeBosonOp = struct {
    /// X builds an explicit bulk free-boson field insertion.
    pub fn X(local: anytype, mu: Handle.Index, z: Handle.Coord, zbar: Handle.Coord) !Operator
    ⇨ {
        return declare.operatorBuilder(sphereOperator(.x)).pair(local, z, zbar, .{mu});
    }

    /// dX builds a holomorphic derivative field insertion.
    pub fn dX(local: anytype, mu: Handle.Index, n: u8, z: Handle.Coord) !Operator {
        return declare.operatorBuilder(sphereOperator(.d_x)).single(local, z, n, .{mu});
    }

    /// dXt builds an antiholomorphic derivative field insertion.
    pub fn dXt(local: anytype, mu: Handle.Index, n: u8, zbar: Handle.Coord) !Operator {
        return declare.operatorBuilder(sphereOperator(.d_xt)).single(local, zbar, n, .{mu});
    }

    /// expX builds a normal-ordered plane-wave insertion.
    pub fn expX(local: anytype, k: Handle.Momentum, z: Handle.Coord, zbar: Handle.Coord)
    ⇨ !Operator {
        return declare.operatorBuilder(sphereOperator(.exp_x)).pair(local, z, zbar, .{k});
    }

    /// profile builds a target-space profile insertion.
    pub fn profile(local: anytype, f: Handle.Profile, z: Handle.Coord, zbar: Handle.Coord)
    ⇨ !Operator {
        return declare.operatorBuilder(sphereOperator(.profile_x)).pair(local, z, zbar, .{f});
    }

    /// profileVector builds a vector-valued target-space profile insertion.
    pub fn profileVector(local: anytype, f: Handle.Profile, nu: Handle.Index, z: Handle.Coord,
    ⇨ zbar: Handle.Coord) !Operator {
        return declare.operatorBuilder(profile_vector_operator).pair(local, z, zbar, .{ f, nu
        ⇨ });
    }
};

```

Profiles are handles to a chosen presentation. Fourier profiles resolve to plane waves and momentum conservation can partially evaluate the inverse Fourier transform. Position-space

profiles keep the zero-mode integral explicit. Polynomial profiles support Riemann-normal-coordinate expansions.

```
const ProfileOp = struct {
  /// positionSpace selects the exact residual position-space profile presentation.
  pub fn positionSpace(f: Handle.TargetFunction) Handle.Profile {
    return shared.profileHandle(.position_space, rawToken(f));
  }

  /// fourier selects the Fourier-space profile presentation.
  pub fn fourier(f_hat: Handle.FourierTransform) Handle.Profile {
    return shared.profileHandle(.fourier, rawToken(f_hat));
  }

  /// polynomialRnc selects a finite polynomial profile presentation.
  pub fn polynomialRnc(point: Handle.TargetPoint, terms: anytype) Handle.Profile {
    return shared.profileHandle(.polynomial_rnc, polynomialProfileId(point, terms));
  }
};
```

3.4.2 Sphere Rules

On the sphere the primitive free-boson rules include same-chirality $\partial X \partial X$ contractions, ∂X contractions with plane waves and target-space profiles, and the plane-wave Green-kernel factor. Labels supply indices, momenta, and profile handles at runtime.

```
fn holomorphicDifference() wick.CoordinateDifference {
  return wick.difference(wick.coord(.left, .position), wick.coord(.right, .position));
}

fn antiholomorphicDifference() wick.CoordinateDifference {
  return wick.difference(wick.coord(.left, .position), wick.coord(.right, .position));
}

fn bulkHolomorphicDifference() wick.CoordinateDifference {
  return wick.difference(wick.coord(.left, .holomorphic), wick.coord(.right, .holomorphic));
}

fn bulkAntiholomorphicDifference() wick.CoordinateDifference {
  return wick.difference(wick.coord(.left, .antiholomorphic), wick.coord(.right,
    ↪ .antiholomorphic));
}

fn bulkToBoundaryHolomorphicDifference() wick.CoordinateDifference {
  return wick.difference(wick.coord(.left, .holomorphic), wick.coord(.right, .position));
}

fn bulkToBoundaryAntiholomorphicDifference() wick.CoordinateDifference {
  return wick.difference(wick.coord(.left, .antiholomorphic), wick.coord(.right,
    ↪ .position));
}

fn freeBosonDxDx(comptime alpha: RuleScalar) wick.Expr {
  return wick.differentiatedPole(scalars.div(scalars.neg(alpha), 2), .{ .metric = .{
    .left = wick.label(.left, 0),
    .right = wick.label(.right, 0),
  } }, holomorphicDifference(), -2, .{ .include_left = true, .include_right = true });
}

fn freeBosonDxDxtDxt(comptime alpha: RuleScalar) wick.Expr {
  return wick.differentiatedPole(scalars.div(scalars.neg(alpha), 2), .{ .metric = .{
    .left = wick.label(.left, 0),
    .right = wick.label(.right, 0),
  } }, antiholomorphicDifference(), -2, .{ .include_left = true, .include_right = true });
}
```

```

    } }, antiholomorphicDifference(), -2, .{ .include_left = true, .include_right = true });
}

fn freeBosonDxExpX(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPoleWithResiduals(scalars.div(scalars.neg(scalars.i(alpha)), 2),
    ↪ .{ .momentum_index = .{
        .momentum = wick.label(.right, 0),
        .index = wick.label(.left, 0),
    } }, holomorphicDifference(), -1, .{ .include_left = true }, &.{.right});
}

fn freeBosonDxProfile(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedActionPoleWithResiduals(scalars.div(scalars.neg(alpha), 2),
    ↪ .none, holomorphicDifference(), -1, .{ .include_left = true }, &.{
        wick.profileDerivative(wick.label(.right, 0), wick.label(.left, 0)),
    }, &.{.right});
}

fn freeBosonDxtExpX(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPoleWithResiduals(scalars.div(scalars.neg(scalars.i(alpha)), 2),
    ↪ .{ .momentum_index = .{
        .momentum = wick.label(.right, 0),
        .index = wick.label(.left, 0),
    } }, antiholomorphicDifference(), -1, .{ .include_left = true }, &.{.right});
}

fn freeBosonDxtProfile(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedActionPoleWithResiduals(scalars.div(scalars.neg(alpha), 2),
    ↪ .none, antiholomorphicDifference(), -1, .{ .include_left = true }, &.{
        wick.profileDerivative(wick.label(.right, 0), wick.label(.left, 0)),
    }, &.{.right});
}

fn freeBosonExpXExpX(comptime alpha: RuleScalar) wick.Expr {
    return wick.expr(&.{wick.termWithResiduals(&.{wick.scalar(scalars.div(alpha, 2))}, &.{
        .{ .green_exponential = bulkHolomorphicDifference() },
        .{ .green_exponential = bulkAntiholomorphicDifference() },
    }, &.{.momentum_pair = .{
        .left = wick.label(.left, 0),
        .right = wick.label(.right, 0),
    } }}, &.{}, &.{ .left, .right }));
}

fn sphereZeroSpec(comptime normalization: RuleScalar) [1]Spec.ZeroModeRule {
    return [1]Spec.ZeroModeRule{
        .{ .sector = zeroSector(.bulk), .expr = .{ .free_boson_constant_mode = .{
            .exp_kind_ids = &.{kind(.exp_x)},
            .profile_kind_ids = &.{kind(.profile_x)},
            .normalization = .{
                .scalar = normalization,
                .two_pi_power = .{ .target_dimension = target_dimension_ref },
            },
        } },
    };
}

fn sphereZeroStorage() [1]zero_mode.Rule {
    const spec = sphereZeroSpec(.one);
    return Spec.zeroModeRules(&spec);
}

const sphere_operator_spec = [1]Spec.Operator{
    .{ .name = "X", .kind = kind(.x), .support = .bulk_pair, .insertion = .pair, .labels =
    ↪ &.{.index}, .statistics = .bosonic },

```

```

    .{ .name = "dX", .kind = kind(.d_x), .support = .holomorphic, .insertion = .single,
    ↪ .labels = &.{.index}, .statistics = .bosonic },
    .{ .name = "dXt", .kind = kind(.d_xt), .support = .antiholomorphic, .insertion = .single,
    ↪ .labels = &.{.index}, .statistics = .bosonic },
    .{ .name = "expX", .kind = kind(.exp_x), .support = .bulk_pair, .insertion = .pair,
    ↪ .labels = &.{.momentum}, .statistics = .bosonic, .zero_mode_consumable = true },
    .{ .name = "profile", .kind = kind(.profile_x), .support = .bulk_pair, .insertion = .pair,
    ↪ .labels = &.{.profile}, .statistics = .bosonic, .zero_mode_consumable = true },
};

fn sphereWickSpec(comptime alpha: RuleScalar) [7]Spec.WickRule {
    return []Spec.WickRule{
        .{ .left = kind(.d_x), .right = kind(.d_x), .expr = freeBosonDxDx(alpha) },
        .{ .left = kind(.d_xt), .right = kind(.d_xt), .expr = freeBosonDxtDxt(alpha) },
        .{ .left = kind(.d_x), .right = kind(.exp_x), .expr = freeBosonDxExpX(alpha) },
        .{ .left = kind(.d_xt), .right = kind(.exp_x), .expr = freeBosonDxtExpX(alpha) },
        .{ .left = kind(.d_x), .right = kind(.profile_x), .expr = freeBosonDxProfile(alpha) },
        .{ .left = kind(.d_xt), .right = kind(.profile_x), .expr = freeBosonDxtProfile(alpha)
        ↪ },
        .{ .left = kind(.exp_x), .right = kind(.exp_x), .expr = freeBosonExpXExpX(alpha) },
    };
}

const sphere_wick_spec = sphereWickSpec(.one);

const torus_wick_spec = []Spec.WickRule{
    .{ .left = kind(.d_x), .right = kind(.d_x), .coordinate_kernels = &.{.elliptic_green} },
    .{ .left = kind(.d_xt), .right = kind(.d_xt), .coordinate_kernels = &.{.elliptic_green} },
    .{ .left = kind(.d_x), .right = kind(.exp_x), .coordinate_kernels = &.{.elliptic_green} },
    .{ .left = kind(.d_xt), .right = kind(.exp_x), .coordinate_kernels = &.{.elliptic_green}
    ↪ },
    .{ .left = kind(.d_x), .right = kind(.profile_x), .coordinate_kernels =
    ↪ &.{.elliptic_green} },
    .{ .left = kind(.d_xt), .right = kind(.profile_x), .coordinate_kernels =
    ↪ &.{.elliptic_green} },
    .{ .left = kind(.exp_x), .right = kind(.exp_x), .coordinate_kernels =
    ↪ &.{.elliptic_green_exponential} },
};

const sphere_zero_spec = sphereZeroSpec(.one);

```

The torus draft follows the genus-one convention in the stringbook: the coordinate obeys $z \sim z + 2\pi \sim z + 2\pi\tau$, the nonzero modes use the torus Green function built from $\theta_1(z|\tau)$, and the constant target-space mode gives the momentum-conservation factor in the exponential correlator [2]. The metadata below records only that zero-mode sector and its consumed operator families; it is not lowered into a runtime config until the torus evaluator is implemented.

```

const torus_zero_spec = []Spec.ZeroModeRule{
    .{
        .sector = zeroSector(.torus),
        .kind = .torus_free_boson_constant,
        .consumes = &.{ kind(.exp_x), kind(.profile_x) },
    },
};

const sphere_spec = Spec.Theory{
    .operators = &sphere_operator_spec,
    .wick_rules = &sphere_wick_spec,
    .zero_modes = &sphere_zero_spec,
};

const torus_draft_spec = Spec.Theory{
    .surface = .{

```

```

        .kind = .torus,
        .coordinate_model = .elliptic,
        .modular_parameters = &.{ "tau" },
        .source = .stringbook,
    },
    .operators = &sphere_operator_spec,
    .wick_rules = &torus_wick_spec,
    .zero_modes = &torus_zero_spec,
};

const boundary_operator_spec = [_]Spec.Operator{
    .{ .name = "dXBoundary", .kind = kind(.d_x_boundary), .support = .boundary, .insertion =
    ↪ .single, .labels = &.{ .index }, .statistics = .bosonic },
    .{ .name = "expXBoundary", .kind = kind(.exp_x_boundary), .support = .boundary, .insertion
    ↪ = .single, .labels = &.{ .momentum }, .statistics = .bosonic, .zero_mode_consumable =
    ↪ true },
    .{ .name = "profileBoundary", .kind = kind(.profile_x_boundary), .support = .boundary,
    ↪ .insertion = .single, .labels = &.{ .profile }, .statistics = .bosonic,
    ↪ .zero_mode_consumable = true },
};

const disk_operator_spec = sphere_operator_spec ++ boundary_operator_spec;

const profile_vector_operator = Spec.Operator{ .name = "profileVector", .kind =
    ↪ kind(.profile_x), .support = .bulk_pair, .insertion = .pair, .labels = &.{ .profile,
    ↪ .index }, .statistics = .bosonic, .zero_mode_consumable = true };
const boundary_profile_vector_operator = Spec.Operator{ .name = "profileBoundaryVector", .kind
    ↪ = kind(.profile_x_boundary), .support = .boundary, .insertion = .single, .labels = &.{
    ↪ .profile, .index }, .statistics = .bosonic, .zero_mode_consumable = true };

fn sphereOperator(comptime family: Family) Spec.Operator {
    const kind_id = kind(family);
    inline for (sphere_operator_spec) |operator| {
        if (operator.kind == kind_id) return operator;
    }
    @compileError("missing free-boson sphere operator spec");
}

fn boundaryOperator(comptime family: Family) Spec.Operator {
    const kind_id = kind(family);
    inline for (boundary_operator_spec) |operator| {
        if (operator.kind == kind_id) return operator;
    }
    @compileError("missing free-boson boundary operator spec");
}

fn assertSphereSpec(comptime wick_count: usize, comptime zero_count: usize) void {
    if (sphere_spec.wick_rules.len != wick_count) @compileError("free-boson sphere spec Wick
    ↪ count does not match lowered rules");
    if (sphere_spec.zero_modes.len != zero_count) @compileError("free-boson sphere spec
    ↪ zero-mode count does not match lowered rules");
    if (Spec.zeroModeConsumeKindCount(sphere_spec.zero_modes[0]) != 2)
    ↪ @compileError("free-boson sphere spec zero-mode coverage is incomplete");
}

fn assertTorusDraftSpec() void {
    if (torus_draft_spec.surface.kind != .torus) @compileError("free-boson torus draft spec
    ↪ surface kind is incomplete");
    if (torus_draft_spec.surface.coordinate_model != .elliptic) @compileError("free-boson
    ↪ torus draft spec coordinate model is incomplete");
    if (torus_draft_spec.surface.modular_parameters.len != 1) @compileError("free-boson torus
    ↪ draft spec modular metadata is incomplete");
    if (torus_draft_spec.surface.source != .stringbook) @compileError("free-boson torus draft
    ↪ spec source convention is incomplete");
}

```

```

if (torus_draft_spec.operators.len != sphere_spec.operators.len) @compileError("free-boson
↳ torus draft spec operator metadata is incomplete");
if (torus_draft_spec.wick_rules.len != sphere_spec.wick_rules.len)
↳ @compileError("free-boson torus draft spec Wick metadata is incomplete");
if (Spec.wickCoordinateKernelCount(torus_draft_spec.wick_rules, .elliptic_green) != 6)
↳ @compileError("free-boson torus draft spec Green-kernel metadata is incomplete");
if (Spec.wickCoordinateKernelCount(torus_draft_spec.wick_rules,
↳ .elliptic_green_exponential) != 1) @compileError("free-boson torus draft spec
↳ exponential Green-kernel metadata is incomplete");
if (Spec.wickCoordinateKernelCount(torus_draft_spec.wick_rules, .rational_pole) != 0)
↳ @compileError("free-boson torus draft spec still uses rational pole metadata");
if (torus_draft_spec.zero_modes.len != 1) @compileError("free-boson torus draft spec
↳ zero-mode metadata is incomplete");
if (Spec.zeroModeConsumeKindCount(torus_draft_spec.zero_modes[0]) != 2)
↳ @compileError("free-boson torus draft spec zero-mode coverage is incomplete");
}

fn FreeBosonRules(comptime cfg: FreeBosonConfig) type {
    _ = cfg;
    const alpha = scalars.atomScalar(alpha_prime_atom);
    return struct {
        const sphere_wick_runtime_spec = sphereWickSpec(alpha);
        const sphere_wick_storage = Spec.wickRules(&sphere_wick_runtime_spec,
↳ &sphere_operator_spec);

        const sphere_wick: []const wick.Rule = &sphere_wick_storage;
    };
}

fn FreeBosonCorrelatorConfig(comptime cfg: FreeBosonConfig) type {
    const rules = FreeBosonRules(cfg);
    return struct {
        const sphere_zero_storage = sphereZeroStorage();
        comptime {
            assertSphereSpec(rules.sphere_wick.len, sphere_zero_storage.len);
            assertTorusDraftSpec();
        }

        /// sphere selects free-boson sphere Wick and zero-mode rules.
        pub const sphere = declare.correlatorConfig(.{
            .wick_rules = rules.sphere_wick,
            .zero_modes = &sphere_zero_storage,
            .config_entries = &declare.configEntries(.{
                declare.config.targetDimension(target_dimension_ref, cfg.dimension),
            }),
        }){};
    };
}

```

3.4.3 Boundary Extension

The boundary extension is separate from the bulk free boson. It adds Neumann/Dirichlet data, optional Chan-Paton stack data, boundary-local operators, mixed bulk-boundary rules, and disk zero-mode rules. The doubling choice is already reflected in these rule templates before runtime.

```

const FreeBosonBoundaryConfig = struct {
    neumann: Handle.TensorProjector,
    dirichlet: Handle.TensorProjector,
    dirichlet_position: Handle.TargetPoint,
    chan_paton: ?Handle.BoundaryStack = null,
};

const FreeBosonBoundaryOp = struct {

```

```

/// dXBoundary builds stringbook's holomorphic boundary limit of dX.
pub fn dXBoundary(local: anytype, mu: Handle.Index, n: u8, y: Handle.BoundaryCoord)
  ↪ !Operator {
  ↪ return declare.operatorBuilder(boundaryOperator(.d_x_boundary)).boundarySingle(local,
  ↪ y, n, .{mu});
}

/// expXBoundary builds a Neumann boundary plane-wave insertion.
pub fn expXBoundary(local: anytype, k: Handle.Momentum, y: Handle.BoundaryCoord) !Operator
  ↪ {
  ↪ return
  ↪ declare.operatorBuilder(boundaryOperator(.exp_x_boundary)).boundarySingle(local,
  ↪ y, 0, .{k});
}

/// profileBoundary builds a boundary profile insertion.
pub fn profileBoundary(local: anytype, f: Handle.Profile, y: Handle.BoundaryCoord)
  ↪ !Operator {
  ↪ return
  ↪ declare.operatorBuilder(boundaryOperator(.profile_x_boundary)).boundarySingle(local,
  ↪ y, 0, .{f});
}

/// profileBoundaryVector builds a vector-valued boundary profile insertion.
pub fn profileBoundaryVector(local: anytype, f: Handle.Profile, nu: Handle.Index, y:
  ↪ Handle.BoundaryCoord) !Operator {
  ↪ return declare.operatorBuilder(boundary_profile_vector_operator).boundarySingle(local,
  ↪ y, 0, .{ f, nu });
}
};

fn FreeBosonBoundaryExtension(comptime cfg: FreeBosonBoundaryConfig) type {
  const data = FreeBosonBoundaryData(cfg);
  return declare.BoundaryExtension(.{
    .op = FreeBosonBoundaryOp,
    .wick_rules = data.disk_wick,
    .zero_modes = data.disk_zero_modes,
    .config_entries = data.config_entries,
  });
}

/// boundaryExtension declares the Neumann/Dirichlet free-boson boundary extension.
pub fn boundaryExtension(comptime cfg: FreeBosonBoundaryConfig)
  ↪ FreeBosonBoundaryExtension(cfg) {
  ↪ return .{};
}

```

The boundary rules use projectors named by the correlator config. These are not fake runtime labels: they point to configuration data such as the Neumann projector and the bulk-boundary doubling projector.

```

fn boundaryFreeBosonDxDx(comptime alpha: RuleScalar) wick.Expr {
  return wick.differentiatedPole(scalars.neg(alpha), .{ .projector_metric = .{
    .projector = .{ .config = neumann_projector_ref },
    .left = wick.label(.left, 0),
    .right = wick.label(.right, 0),
  } }, holomorphicDifference(), -2, .{ .include_left = true, .include_right = true });
}

fn boundaryFreeBosonDxExpX(comptime alpha: RuleScalar) wick.Expr {
  return wick.differentiatedPoleWithResiduals(scalars.neg(scalars.i(alpha)), .{
    ↪ .projector_momentum_index = .{
    ↪ .projector = .{ .config = neumann_projector_ref },

```

```

        .momentum = wick.label(.right, 0),
        .index = wick.label(.left, 0),
    } }, holomorphicDifference(), -1, .{ .include_left = true }, &.{.right});
}

fn boundaryFreeBosonDxProfile(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedActionPoleWithResiduals(scalars.neg(alpha), .none,
        ↪ holomorphicDifference(), -1, .{ .include_left = true }, &.{
            wick.projectedProfileDerivative(.{ .config = neumann_projector_ref },
                ↪ wick.label(.right, 0), wick.label(.left, 0)),
            }, &.{.right});
}

fn boundaryFreeBosonExpXExpX(comptime alpha: RuleScalar) wick.Expr {
    return wick.greenExponentialWithResiduals(alpha, .{ .projector_momentum_pair = .{
        .projector = .{ .config = neumann_projector_ref },
        .left = wick.label(.left, 0),
        .right = wick.label(.right, 0),
    } }, holomorphicDifference(), &.{ .left, .right });
}

fn mixedFreeBosonDxDxBoundary(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPole(scalars.div(scalars.neg(alpha), 2), .{ .projector_metric =
        ↪ .{
            .projector = .{ .config = bulk_boundary_projector_ref },
            .left = wick.label(.left, 0),
            .right = wick.label(.right, 0),
        } }, bulkToBoundaryHolomorphicDifference(), -2, .{ .include_left = true, .include_right =
        ↪ true });
}

fn mixedFreeBosonDxtDxBoundary(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPole(scalars.div(scalars.neg(alpha), 2), .{ .projector_metric =
        ↪ .{
            .projector = .{ .config = bulk_boundary_projector_ref },
            .left = wick.label(.left, 0),
            .right = wick.label(.right, 0),
        } }, bulkToBoundaryAntiholomorphicDifference(), -2, .{ .include_left = true,
        ↪ .include_right = true });
}

fn mixedFreeBosonDxExpXBoundary(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPoleWithResiduals(scalars.div(scalars.neg(scalars.i(alpha)), 2),
        ↪ .{ .projector_momentum_index = .{
            .projector = .{ .config = bulk_boundary_projector_ref },
            .momentum = wick.label(.right, 0),
            .index = wick.label(.left, 0),
        } }, bulkToBoundaryHolomorphicDifference(), -1, .{ .include_left = true }, &.{.right});
}

fn mixedFreeBosonDxProfileBoundary(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedActionPoleWithResiduals(scalars.div(scalars.neg(alpha), 2),
        ↪ .none, bulkToBoundaryHolomorphicDifference(), -1, .{ .include_left = true }, &.{
            wick.projectedProfileDerivative(.{ .config = bulk_boundary_projector_ref },
                ↪ wick.label(.right, 0), wick.label(.left, 0)),
            }, &.{.right});
}

fn mixedFreeBosonDxtExpXBoundary(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPoleWithResiduals(scalars.div(scalars.neg(scalars.i(alpha)), 2),
        ↪ .{ .projector_momentum_index = .{
            .projector = .{ .config = bulk_boundary_projector_ref },
            .momentum = wick.label(.right, 0),
            .index = wick.label(.left, 0),
        } }, bulkToBoundaryHolomorphicDifference(), -1, .{ .include_left = true }, &.{.right});
}

```



```

    } }, bulkToBoundaryAntiholomorphicDifference(), -1, .{ .include_left = true },
    ↪ &.{.right});
}

fn mixedFreeBosonDxtProfileBoundary(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedActionPoleWithResiduals(scalars.div(scalars.neg(alpha), 2),
    ↪ .none, bulkToBoundaryAntiholomorphicDifference(), -1, .{ .include_left = true }, &.{
        wick.projectedProfileDerivative(.{ .config = bulk_boundary_projector_ref },
        ↪ wick.label(.right, 0), wick.label(.left, 0)),
    }, &.{.right});
}

fn mixedFreeBosonExpXExpXBoundary(comptime alpha: RuleScalar) wick.Expr {
    return wick.greenExponentialWithResiduals(scalars.div(alpha, 2), .{
    ↪ .projector_momentum_pair = .{
        .projector = .{ .config = bulk_boundary_projector_ref },
        .left = wick.label(.left, 0),
        .right = wick.label(.right, 0),
    } }, bulkToBoundaryHolomorphicDifference(), &.{ .left, .right });
}

fn diskWickSpec(comptime alpha: RuleScalar) [11]Spec.WickRule {
    return [_]Spec.WickRule{
        .{ .left = kind(.d_x_boundary), .right = kind(.d_x_boundary), .expr =
        ↪ boundaryFreeBosonDxDx(alpha) },
        .{ .left = kind(.d_x_boundary), .right = kind(.exp_x_boundary), .expr =
        ↪ boundaryFreeBosonDxExpX(alpha) },
        .{ .left = kind(.d_x_boundary), .right = kind(.profile_x_boundary), .expr =
        ↪ boundaryFreeBosonDxProfile(alpha) },
        .{ .left = kind(.exp_x_boundary), .right = kind(.exp_x_boundary), .expr =
        ↪ boundaryFreeBosonExpXExpX(alpha) },
        .{ .left = kind(.d_x), .right = kind(.d_x_boundary), .expr =
        ↪ mixedFreeBosonDxDxBoundary(alpha) },
        .{ .left = kind(.d_xt), .right = kind(.d_x_boundary), .expr =
        ↪ mixedFreeBosonDxtDxBoundary(alpha) },
        .{ .left = kind(.d_x), .right = kind(.exp_x_boundary), .expr =
        ↪ mixedFreeBosonDxExpXBoundary(alpha) },
        .{ .left = kind(.d_xt), .right = kind(.exp_x_boundary), .expr =
        ↪ mixedFreeBosonDxtExpXBoundary(alpha) },
        .{ .left = kind(.d_x), .right = kind(.profile_x_boundary), .expr =
        ↪ mixedFreeBosonDxProfileBoundary(alpha) },
        .{ .left = kind(.d_xt), .right = kind(.profile_x_boundary), .expr =
        ↪ mixedFreeBosonDxtProfileBoundary(alpha) },
        .{ .left = kind(.exp_x), .right = kind(.exp_x_boundary), .expr =
        ↪ mixedFreeBosonExpXExpXBoundary(alpha) },
    };
}

fn diskZeroSpec(comptime normalization: RuleScalar) [1]Spec.ZeroModeRule {
    return [_]Spec.ZeroModeRule{
        .{ .sector = zeroSector(.boundary), .expr = .{ .free_boson_constant_mode = .{
            .integration_projector = neumann_projector_ref,
            .fixed_projector = dirichlet_projector_ref,
            .fixed_position = dirichlet_position_ref,
            .exp_kind_ids = &.{ kind(.exp_x), kind(.exp_x_boundary) },
            .profile_kind_ids = &.{ kind(.profile_x), kind(.profile_x_boundary) },
            .normalization = .{
                .scalar = normalization,
                .two_pi_power = .{ .projector_rank = neumann_projector_ref },
            },
        } },
    };
}

```



```

fn diskZeroStorage() [1]zero_mode.Rule {
    const spec = diskZeroSpec(scalars.atomScalar(k_disk_atom));
    return Spec.zeroModeRules(&spec);
}

fn FreeBosonBoundaryData(comptime cfg: FreeBosonBoundaryConfig) type {
    const alpha = scalars.atomScalar(alpha_prime_atom);
    return struct {
        const disk_wick_runtime_spec = diskWickSpec(alpha);
        const disk_wick_storage = Spec.wickRules(&disk_wick_runtime_spec,
            ↪ &disk_operator_spec);
        const disk_zero_storage = diskZeroStorage();
        const config_storage = if (cfg.chan_paton) |stack| declare.configEntries(.{
            declare.config.tensorProjector(neumann_projector_ref, cfg.neumann),
            declare.config.tensorProjector(dirichlet_projector_ref, cfg.dirichlet),
            declare.config.tensorProjector(bulk_boundary_projector_ref, cfg.neumann),
            declare.config.targetPoint(dirichlet_position_ref, cfg.dirichlet_position),
            declare.config.boundaryStack(chan_paton_ref, stack),
        }) else declare.configEntries(.{
            declare.config.tensorProjector(neumann_projector_ref, cfg.neumann),
            declare.config.tensorProjector(dirichlet_projector_ref, cfg.dirichlet),
            declare.config.tensorProjector(bulk_boundary_projector_ref, cfg.neumann),
            declare.config.targetPoint(dirichlet_position_ref, cfg.dirichlet_position),
        });

        const disk_wick: []const wick.Rule = &disk_wick_storage;
        const disk_zero_modes: []const zero_mode.Rule = &disk_zero_storage;
        const config_entries = &config_storage;
    };
}

```

3.5 bc Preset

INTERFACE

FUNCTIONS

bcSphere(comptime cfg: BcSphereConfig) type

bcSphere builds the generated preset type for the sphere bc ghost CFT.

boundaryExtension(comptime cfg: BcDiskConfig) BcBoundaryExtension(cfg)

boundaryExtension declares boundary and mixed bulk-boundary bc rules on the disk.

This node defines the *bc* ghost preset on the sphere and its disk boundary extension. It is independent of the matter CFT and uses the shared operator/Wick vocabulary from Preset Shared Runtime.

The local OPE is

$$b(z)c(0) \sim \frac{1}{z},$$

with weights $h_b = 2$, $h_c = -1$. The sphere top form requires three *c*-ghost insertions in each chiral copy [2].

```

const std = @import("std");
const shared = @import("shared.zig");
const declare = shared.declare;

const operators = @import("../expressions/operators.zig");
const Handle = shared.Handle;
const Local = shared.Local;
const Builder = *shared.Local;

```

```

const Operator = *shared.LocalOperator;
const Spec = declare.Spec;
const wick = declare.wick;
const zero_mode = declare.zero_mode;

```

The ghost preset owns the b, c operator vocabulary. It uses a separate id block from the free boson and derives each concrete kind id from the enum tag, so this file names physics families without maintaining numeric ids per operator.

```

const namespace = "bc";

const Family = enum {
    b,
    c,
    bt,
    ct,
    b_boundary,
    c_boundary,
};

const ZeroSector = enum {
    sphere,
    disk,
    torus,
};

fn kind(comptime family: Family) operators.OperatorKindId {
    return declare.familyKind(namespace, family);
}

fn zeroSector(comptime sector: ZeroSector) zero_mode.Sector {
    return declare.sectorId(namespace, sector);
}

```

3.5.1 Sphere Preset

The sphere preset exposes a holomorphic copy and, by default, an antiholomorphic copy. The config flag changes both the builder namespace and the emitted rule slice at compile time.

```

const BcSphereConfig = struct {
    include_antiholomorphic_copy: bool = true,
};

/// bcSphere builds the generated preset type for the sphere bc ghost CFT.
pub fn bcSphere(comptime cfg: BcSphereConfig) type {
    return struct {
        /// op exposes bc ghost local operator builders.
        pub const op = BcSphereOp(cfg.include_antiholomorphic_copy);
        /// config exposes named correlator configs for this preset.
        pub const config = BcSphereCorrelatorConfig(cfg);
        /// text exposes bounded result-inspection sinks.
        pub const text = shared.text;
        /// local constructs a label-preserving local-operator builder.
        pub fn local(allocator: std.mem.Allocator) !Builder {
            return shared.Local.init(allocator);
        }

        /// correlator streams rule matches for bc insertions.
        pub fn correlator(config_ptr: anytype, ops: anytype, sink: anytype) !void {
            return shared.streamCorrelator(config_ptr, ops, sink);
        }
    };
}

```

```
}
```

Derivative labels mean $\partial^n b$, $\partial^n c$, and their antiholomorphic copies. No ghost-number book-keeping is performed here; that is part of theory data and later basis or correlator evaluation.

```
fn BcSphereOp(comptime include_antiholomorphic_copy: bool) type {
  const Holomorphic = struct {
    /// b builds a holomorphic b-ghost insertion.
    pub fn b(local: anytype, n: u8, z: Handle.Coord) !Operator {
      return declare.operatorBuilder(bcOperator(.b)).single(local, z, n, .{});
    }

    /// c builds a holomorphic c-ghost insertion.
    pub fn c(local: anytype, n: u8, z: Handle.Coord) !Operator {
      return declare.operatorBuilder(bcOperator(.c)).single(local, z, n, .{});
    }
  };

  if (!include_antiholomorphic_copy) return Holomorphic;

  return struct {
    /// b builds a holomorphic b-ghost insertion.
    pub const b = Holomorphic.b;
    /// c builds a holomorphic c-ghost insertion.
    pub const c = Holomorphic.c;

    /// bt builds an antiholomorphic b-ghost insertion.
    pub fn bt(local: anytype, n: u8, zbar: Handle.Coord) !Operator {
      return declare.operatorBuilder(bcOperator(.bt)).single(local, zbar, n, .{});
    }

    /// ct builds an antiholomorphic c-ghost insertion.
    pub fn ct(local: anytype, n: u8, zbar: Handle.Coord) !Operator {
      return declare.operatorBuilder(bcOperator(.ct)).single(local, zbar, n, .{});
    }
  };
}
```

3.5.2 Rules

The *bc* Wick rules have no tensor numerator. Their zero-mode rule records the top-form saturation separately from pairwise contractions.

```
fn holomorphicDifference() wick.CoordinateDifference {
  return wick.difference(wick.coord(.left, .position), wick.coord(.right, .position));
}

fn antiholomorphicDifference() wick.CoordinateDifference {
  return wick.difference(wick.coord(.left, .position), wick.coord(.right, .position));
}

fn bcBC() wick.Expr {
  return wick.differentiatedPole(.one, .none, holomorphicDifference(), -1, .{ .include_left
    ⇨ = true, .include_right = true });
}

fn bcBtCt() wick.Expr {
  return wick.differentiatedPole(.one, .none, antiholomorphicDifference(), -1, .{
    ⇨ .include_left = true, .include_right = true });
}

fn sphereZeroSpec(comptime cfg: BcSphereConfig) if (cfg.include_antiholomorphic_copy)
⇨ [2]Spec.ZeroModeRule else [1]Spec.ZeroModeRule {
```

```

const holomorphic = Spec.ZeroModeRule{ .sector = zeroSector(.sphere), .expr = .{
  ↪ .bc_top_form = .{
    .support = .sphere_holomorphic,
    .c_kind_ids = &.{kind(.c)},
    .normalization = .one,
  } } };
if (!cfg.include_antiholomorphic_copy) return [_]Spec.ZeroModeRule{holomorphic};
return [_]Spec.ZeroModeRule{
  holomorphic,
  .{ .sector = zeroSector(.sphere), .expr = .{ .bc_top_form = .{
    .support = .sphere_antiholomorphic,
    .c_kind_ids = &.{kind(.ct)},
    .normalization = .one,
  } } },
};
}

fn sphereZeroStorage(comptime cfg: BcSphereConfig) if (cfg.include_antiholomorphic_copy)
↪ [2]zero_mode.Rule else [1]zero_mode.Rule {
  const spec = sphereZeroSpec(cfg);
  return Spec.zeroModeRules(&spec);
}

const sphere_holomorphic_operator_spec = [_]Spec.Operator{
  .{ .name = "b", .kind = kind(.b), .support = .holomorphic, .insertion = .single,
    ↪ .statistics = .fermionic },
  .{ .name = "c", .kind = kind(.c), .support = .holomorphic, .insertion = .single,
    ↪ .statistics = .fermionic, .zero_mode_consumable = true },
};

const sphere_full_operator_spec = sphere_holomorphic_operator_spec ++ [_]Spec.Operator{
  .{ .name = "bt", .kind = kind(.bt), .support = .antiholomorphic, .insertion = .single,
    ↪ .statistics = .fermionic },
  .{ .name = "ct", .kind = kind(.ct), .support = .antiholomorphic, .insertion = .single,
    ↪ .statistics = .fermionic, .zero_mode_consumable = true },
};

const sphere_holomorphic_fermion_storage =
↪ Spec.fermionKinds(&sphere_holomorphic_operator_spec);

const sphere_full_fermion_storage = Spec.fermionKinds(&sphere_full_operator_spec);

const sphere_holomorphic_wick_spec = [_]Spec.WickRule{
  .{ .left = kind(.b), .right = kind(.c), .expr = bcBC() },
};

const sphere_full_wick_spec = sphere_holomorphic_wick_spec ++ [_]Spec.WickRule{
  .{ .left = kind(.bt), .right = kind(.ct), .expr = bcBtCt() },
};

const sphere_holomorphic_wick_storage = Spec.wickRules(&sphere_holomorphic_wick_spec,
↪ &sphere_holomorphic_operator_spec);

const sphere_full_wick_storage = Spec.wickRules(&sphere_full_wick_spec,
↪ &sphere_full_operator_spec);

const torus_wick_spec = [_]Spec.WickRule{
  .{ .left = kind(.b), .right = kind(.c), .coordinate_kernels = &.{elliptic_prime_form} },
  .{ .left = kind(.bt), .right = kind(.ct), .coordinate_kernels = &.{elliptic_prime_form}
    ↪ },
};

const sphere_holomorphic_zero_spec = sphereZeroSpec(.{ .include_antiholomorphic_copy = false
↪ });

```

```
const sphere_full_zero_spec = sphereZeroSpec({});
```

For the genus-one amplitude the stringbook fixes one integrated b insertion for the torus modulus and one residual c zero mode in each chiral copy, after one puncture is fixed by torus translations [2]. The draft metadata records those holomorphic and antiholomorphic b, c families without lowering them into the current sphere/disk top-form runtime evaluator.

```
const torus_zero_spec = [_]Spec.ZeroModeRule{
  .{
    .sector = zeroSector(.torus),
    .kind = .torus_bc_moduli,
    .consumes = &.{ kind(.b), kind(.c), kind(.bt), kind(.ct) },
  },
};

fn bcSphereSpec(comptime include_antiholomorphic_copy: bool) Spec.Theory {
  return .{
    .operators = if (include_antiholomorphic_copy) &sphere_full_operator_spec else
      ↪ &sphere_holomorphic_operator_spec,
    .wick_rules = if (include_antiholomorphic_copy) &sphere_full_wick_spec else
      ↪ &sphere_holomorphic_wick_spec,
    .zero_modes = if (include_antiholomorphic_copy) &sphere_full_zero_spec else
      ↪ &sphere_holomorphic_zero_spec,
  };
}

const torus_draft_spec = Spec.Theory{
  .surface = .{
    .kind = .torus,
    .coordinate_model = .elliptic,
    .modular_parameters = &.{ "tau" },
    .source = .stringbook,
  },
  .operators = &sphere_full_operator_spec,
  .wick_rules = &torus_wick_spec,
  .zero_modes = &torus_zero_spec,
};

const disk_boundary_operator_spec = [_]Spec.Operator{
  .{ .name = "bBoundary", .kind = kind(.b_boundary), .support = .boundary, .insertion =
    ↪ .single, .statistics = .fermionic },
  .{ .name = "cBoundary", .kind = kind(.c_boundary), .support = .boundary, .insertion =
    ↪ .single, .statistics = .fermionic, .zero_mode_consumable = true },
};

const disk_full_operator_spec = sphere_full_operator_spec ++ disk_boundary_operator_spec;

fn bcOperator(comptime family: Family) Spec.Operator {
  const kind_id = kind(family);
  inline for (sphere_full_operator_spec) |operator| {
    if (operator.kind == kind_id) return operator;
  }
  @compileError("missing bc operator spec");
}

fn bcBoundaryOperator(comptime family: Family) Spec.Operator {
  const kind_id = kind(family);
  inline for (disk_boundary_operator_spec) |operator| {
    if (operator.kind == kind_id) return operator;
  }
  @compileError("missing bc boundary operator spec");
}
```

```

fn assertSphereSpec(comptime include_antiholomorphic_copy: bool, comptime wick_count: usize,
    ↪ comptime zero_count: usize, comptime fermion_count: usize) void {
    const spec = bcSphereSpec(include_antiholomorphic_copy);
    if (spec.wick_rules.len != wick_count) @compileError("bc sphere spec Wick count does not
    ↪ match lowered rules");
    if (spec.zero_modes.len != zero_count) @compileError("bc sphere spec zero-mode count does
    ↪ not match lowered rules");
    if (spec.operators.len != fermion_count) @compileError("bc sphere spec fermion coverage
    ↪ does not match lowered rules");
}

fn assertTorusDraftSpec() void {
    if (torus_draft_spec.surface.kind != .torus) @compileError("bc torus draft spec surface
    ↪ kind is incomplete");
    if (torus_draft_spec.surface.coordinate_model != .elliptic) @compileError("bc torus draft
    ↪ spec coordinate model is incomplete");
    if (torus_draft_spec.surface.modular_parameters.len != 1) @compileError("bc torus draft
    ↪ spec modular metadata is incomplete");
    if (torus_draft_spec.surface.source != .stringbook) @compileError("bc torus draft spec
    ↪ source convention is incomplete");
    if (torus_draft_spec.operators.len != sphere_full_operator_spec.len) @compileError("bc
    ↪ torus draft spec operator metadata is incomplete");
    if (torus_draft_spec.wick_rules.len != sphere_full_wick_spec.len) @compileError("bc torus
    ↪ draft spec Wick metadata is incomplete");
    if (Spec.wickCoordinateKernelCount(torus_draft_spec.wick_rules, .elliptic_prime_form) !=
    ↪ 2) @compileError("bc torus draft spec prime-form metadata is incomplete");
    if (Spec.wickCoordinateKernelCount(torus_draft_spec.wick_rules, .rational_pole) != 0)
    ↪ @compileError("bc torus draft spec still uses rational pole metadata");
    if (torus_draft_spec.zero_modes.len != 1) @compileError("bc torus draft spec zero-mode
    ↪ metadata is incomplete");
    if (Spec.zeroModeConsumeKindCount(torus_draft_spec.zero_modes[0]) != 4) @compileError("bc
    ↪ torus draft spec zero-mode coverage is incomplete");
}

fn BcSphereRules(comptime include_antiholomorphic_copy: bool) type {
    const selected_wick = if (include_antiholomorphic_copy) sphere_full_wick_storage else
    ↪ sphere_holomorphic_wick_storage;
    const selected_fermions = if (include_antiholomorphic_copy) sphere_full_fermion_storage
    ↪ else sphere_holomorphic_fermion_storage;
    return struct {
        const sphere_wick: []const wick.Rule = &selected_wick;
        const sphere_fermion_kinds: []const operators.OperatorKindId = &selected_fermions;
    };
}

fn BcSphereCorrelatorConfig(comptime cfg: BcSphereConfig) type {
    const rules = BcSphereRules(cfg.include_antiholomorphic_copy);
    return struct {
        const sphere_zero_storage = sphereZeroStorage(cfg);
        comptime {
            assertSphereSpec(cfg.include_antiholomorphic_copy, rules.sphere_wick.len,
            ↪ sphere_zero_storage.len, rules.sphere_fermion_kinds.len);
            assertTorusDraftSpec();
        }

        /// sphere selects bc sphere Wick and zero-mode rules.
        pub const sphere = declare.correlatorConfig({
            .wick_rules = rules.sphere_wick,
            .zero_modes = &sphere_zero_storage,
            .fermion_kinds = rules.sphere_fermion_kinds,
        }){};
    };
}

```

3.5.3 Disk Boundary

The disk extension adds boundary-local b, c operators and mixed bulk-boundary contractions. It is an extension chosen by a BCFT composition, not a modification of the bulk bc preset.

```
const BcDiskConfig = struct {
    include_mixed_bulk_boundary: bool = true,
};

const BcDiskBoundaryOp = struct {
    /// bBoundary builds a boundary b-ghost insertion.
    pub fn bBoundary(local: anytype, n: u8, y: Handle.BoundaryCoord) !Operator {
        return declare.operatorBuilder(bcBoundaryOperator(.b_boundary)).boundarySingle(local,
            ↪ y, n, .{});
    }

    /// cBoundary builds a boundary c-ghost insertion.
    pub fn cBoundary(local: anytype, n: u8, y: Handle.BoundaryCoord) !Operator {
        return declare.operatorBuilder(bcBoundaryOperator(.c_boundary)).boundarySingle(local,
            ↪ y, n, .{});
    }
};

fn BcBoundaryExtension(comptime cfg: BcDiskConfig) type {
    const data = BcDiskData(cfg);
    return declare.BoundaryExtension(.{
        .op = BcDiskBoundaryOp,
        .wick_rules = if (cfg.include_mixed_bulk_boundary) &disk_full_wick_storage else
            ↪ &disk_boundary_wick_storage,
        .zero_modes = data.disk_zero_modes,
        .fermion_kinds = data.disk_fermion_kinds,
    });
}

/// boundaryExtension declares boundary and mixed bulk-boundary bc rules on the disk.
pub fn boundaryExtension(comptime cfg: BcDiskConfig) BcBoundaryExtension(cfg) {
    return .{};
}

const disk_boundary_wick_spec = [_]Spec.WickRule{
    .{ .left = kind(.b_boundary), .right = kind(.c_boundary), .expr = bcBC() },
};

const disk_full_wick_spec = disk_boundary_wick_spec ++ [_]Spec.WickRule{
    .{ .left = kind(.b), .right = kind(.c_boundary), .expr = bcBC() },
    .{ .left = kind(.c), .right = kind(.b_boundary), .expr = bcBC() },
    .{ .left = kind(.bt), .right = kind(.c_boundary), .expr = bcBtCt() },
    .{ .left = kind(.ct), .right = kind(.b_boundary), .expr = bcBtCt() },
};

const disk_boundary_wick_storage = Spec.wickRules(&disk_boundary_wick_spec,
    ↪ &disk_boundary_operator_spec);

const disk_full_wick_storage = Spec.wickRules(&disk_full_wick_spec, &disk_full_operator_spec);

const disk_boundary_fermion_storage = Spec.fermionKinds(&disk_boundary_operator_spec);

const disk_full_fermion_storage = Spec.fermionKinds(&disk_full_operator_spec);

fn diskZeroSpec() [1]Spec.ZeroModeRule {
    return [_]Spec.ZeroModeRule{
        .{ .sector = zeroSector(.disk), .expr = .{ .bc_top_form = .{
            .support = .disk_doubled,
            .c_kind_ids = &.{ kind(.c), kind(.ct), kind(.c_boundary) },
        }
    }
};
```

```

        .normalization = .one,
    } } },
};
}

fn diskZeroStorage(comptime cfg: BcDiskConfig) [1]zero_mode.Rule {
    _ = cfg;
    const spec = diskZeroSpec();
    return Spec.zeroModeRules(&spec);
}

fn BcDiskData(comptime cfg: BcDiskConfig) type {
    const selected_fermions = if (cfg.include_mixed_bulk_boundary) disk_full_fermion_storage
    ⇨ else disk_boundary_fermion_storage;
    return struct {
        const disk_zero_storage = diskZeroStorage(cfg);

        const disk_zero_modes: []const zero_mode.Rule = &disk_zero_storage;
        const disk_fermion_kinds: []const operators.OperatorKindId = &selected_fermions;
    };
}

```

3.6 Free Fermion Preset

INTERFACE

FUNCTIONS

freeFermionSphere(comptime cfg: FreeFermionSphereConfig) type
 freeFermionSphere builds the generated preset type for sphere NS free fermions.

This node defines the NS-sector free worldsheet fermion on the sphere. It uses the shared preset lowering path from Preset Shared Runtime and deliberately does not define a torus surface. Loop-level free-fermion data must later be combined with the ϕ system rather than frozen here in isolation.

The stringbook convention is

$$\psi^\mu(z)\psi^\nu(w) \sim \frac{\eta^{\mu\nu}}{z-w}.$$

Derivative labels mean $\partial^n \psi^\mu$. The correlator engine receives only the lowered fermionic kind ids and pole rule; all theory names remain in this preset.

```

const std = @import("std");
const shared = @import("shared.zig");
const declare = shared.declare;

const operators = @import("../expressions/operators.zig");
const Handle = shared.Handle;
const Builder = *shared.Local;
const Operator = *shared.LocalOperator;
const Spec = declare.Spec;
const wick = declare.wick;

```

The preset owns the two chiral operator families. A compile-time flag can keep only the holomorphic NS sector when a chiral computation does not need the antiholomorphic copy.


```

const namespace = "free_fermion";

const Family = enum {
    psi,
    psit,
};

fn kind(comptime family: Family) operators.OperatorKindId {
    return declare.familyKind(namespace, family);
}

```

3.6.1 Sphere Preset

The public surface mirrors the other presets: users get an operator namespace, an opaque sphere config, the bounded text adapter, a local builder, and one streaming correlator entry point.

```

const FreeFermionSphereConfig = struct {
    dimension: u16,
    include_antiholomorphic_copy: bool = true,
};

/// freeFermionSphere builds the generated preset type for sphere NS free fermions.
pub fn freeFermionSphere(comptime cfg: FreeFermionSphereConfig) type {
    if (cfg.dimension == 0) @compileError("free-fermion target dimension must be nonzero");
    return struct {
        /// op exposes NS free-fermion local operator builders.
        pub const op = FreeFermionSphereOp(cfg.include_antiholomorphic_copy);
        /// config exposes named correlator configs for this preset.
        pub const config = FreeFermionCorrelatorConfig(cfg);
        /// text exposes bounded result-inspection sinks.
        pub const text = shared.text;
        /// local constructs a label-preserving local-operator builder.
        pub fn local(allocator: std.mem.Allocator) !Builder {
            return shared.Local.init(allocator);
        }

        /// correlator streams rule matches for NS free-fermion insertions.
        pub fn correlator(config_ptr: anytype, ops: anytype, sink: anytype) !void {
            return shared.streamCorrelator(config_ptr, ops, sink);
        }
    };
}

```

The builders pack one target-space vector index and one coordinate. They do not expose the compact operator kind ids used by the engine.

```

fn FreeFermionSphereOp(comptime include_antiholomorphic_copy: bool) type {
    const Holomorphic = struct {
        /// psi builds a holomorphic NS free-fermion insertion.
        pub fn psi(local: anytype, mu: Handle.Index, n: u8, z: Handle.Coord) !Operator {
            return declare.operatorBuilder(fermionOperator(.psi)).single(local, z, n, .{mu});
        }
    };

    if (!include_antiholomorphic_copy) return Holomorphic;

    return struct {
        /// psi builds a holomorphic NS free-fermion insertion.
        pub const psi = Holomorphic.psi;

        /// psit builds an antiholomorphic NS free-fermion insertion.
        pub fn psit(local: anytype, mu: Handle.Index, n: u8, zbar: Handle.Coord) !Operator {

```

```

        return declare.operatorBuilder(fermionOperator(.psit)).single(local, zbar, n,
        ↪ .{mu});
    }
};
}

```

3.6.2 Rules

The sphere rule is a metric numerator times a simple rational pole. Fermionic statistics are lowered as data so the generic branch walker supplies the Wick permutation sign for higher-point correlators.

```

fn holomorphicDifference() wick.CoordinateDifference {
    return wick.difference(wick.coord(.left, .position), wick.coord(.right, .position));
}

fn antiholomorphicDifference() wick.CoordinateDifference {
    return wick.difference(wick.coord(.left, .position), wick.coord(.right, .position));
}

fn psiPsi() wick.Expr {
    return wick.differentiatedPole(.one, .{ .metric = .{
        .left = wick.label(.left, 0),
        .right = wick.label(.right, 0),
    } }, holomorphicDifference(), -1, .{ .include_left = true, .include_right = true });
}

fn psitPsit() wick.Expr {
    return wick.differentiatedPole(.one, .{ .metric = .{
        .left = wick.label(.left, 0),
        .right = wick.label(.right, 0),
    } }, antiholomorphicDifference(), -1, .{ .include_left = true, .include_right = true });
}

const sphere_holomorphic_operator_spec = []Spec.Operator{
    .{ .name = "psi", .kind = kind(.psi), .support = .holomorphic, .insertion = .single,
    ↪ .labels = &.{.index}, .statistics = .fermionic },
};

const sphere_full_operator_spec = sphere_holomorphic_operator_spec ++ []Spec.Operator{
    .{ .name = "psit", .kind = kind(.psit), .support = .antiholomorphic, .insertion = .single,
    ↪ .labels = &.{.index}, .statistics = .fermionic },
};

const sphere_holomorphic_wick_spec = []Spec.WickRule{
    .{ .left = kind(.psi), .right = kind(.psi), .expr = psiPsi() },
};

const sphere_full_wick_spec = sphere_holomorphic_wick_spec ++ []Spec.WickRule{
    .{ .left = kind(.psit), .right = kind(.psit), .expr = psitPsit() },
};

const sphere_holomorphic_wick_storage = Spec.wickRules(&sphere_holomorphic_wick_spec,
    ↪ &sphere_holomorphic_operator_spec);
const sphere_full_wick_storage = Spec.wickRules(&sphere_full_wick_spec,
    ↪ &sphere_full_operator_spec);

const sphere_holomorphic_fermion_storage =
    ↪ Spec.fermionKinds(&sphere_holomorphic_operator_spec);
const sphere_full_fermion_storage = Spec.fermionKinds(&sphere_full_operator_spec);

const sphere_holomorphic_zero_spec = []Spec.ZeroModeRule{};
const sphere_full_zero_spec = []Spec.ZeroModeRule{};

```

```

fn sphereSpec(comptime include_antiholomorphic_copy: bool) Spec.Theory {
    return .{
        .operators = if (include_antiholomorphic_copy) &sphere_full_operator_spec else
            ↪ &sphere_holomorphic_operator_spec,
        .wick_rules = if (include_antiholomorphic_copy) &sphere_full_wick_spec else
            ↪ &sphere_holomorphic_wick_spec,
        .zero_modes = if (include_antiholomorphic_copy) &sphere_full_zero_spec else
            ↪ &sphere_holomorphic_zero_spec,
    };
}

fn fermionOperator(comptime family: Family) Spec.Operator {
    const kind_id = kind(family);
    inline for (sphere_full_operator_spec) |operator| {
        if (operator.kind == kind_id) return operator;
    }
    @compileError("missing free-fermion operator spec");
}

fn assertSphereSpec(comptime include_antiholomorphic_copy: bool, comptime wick_count: usize,
    ↪ comptime zero_count: usize, comptime fermion_count: usize) void {
    const spec = sphereSpec(include_antiholomorphic_copy);
    if (spec.wick_rules.len != wick_count) @compileError("free-fermion sphere spec Wick count
        ↪ does not match lowered rules");
    if (spec.zero_modes.len != zero_count) @compileError("free-fermion sphere spec zero-mode
        ↪ count does not match lowered rules");
    if (spec.operators.len != fermion_count) @compileError("free-fermion sphere spec fermion
        ↪ coverage does not match lowered rules");
}

fn FreeFermionRules(comptime include_antiholomorphic_copy: bool) type {
    const selected_wick = if (include_antiholomorphic_copy) sphere_full_wick_storage else
        ↪ sphere_holomorphic_wick_storage;
    const selected_fermions = if (include_antiholomorphic_copy) sphere_full_fermion_storage
        ↪ else sphere_holomorphic_fermion_storage;
    return struct {
        const sphere_wick: []const wick.Rule = &selected_wick;
        const sphere_fermion_kinds: []const operators.OperatorKindId = &selected_fermions;
    };
}

fn FreeFermionCorrelatorConfig(comptime cfg: FreeFermionSphereConfig) type {
    const rules = FreeFermionRules(cfg.include_antiholomorphic_copy);
    return struct {
        comptime {
            assertSphereSpec(cfg.include_antiholomorphic_copy, rules.sphere_wick.len, 0,
                ↪ rules.sphere_fermion_kinds.len);
        }

        /// sphere selects NS free-fermion sphere Wick rules.
        pub const sphere = declare.correlatorConfig(.{
            .wick_rules = rules.sphere_wick,
            .zero_modes = &.{},
            .fermion_kinds = rules.sphere_fermion_kinds,
        }){};
    };
}

```

3.7 eta-xi Preset

`etaXiSphere(comptime cfg: EtaXiSphereConfig) type`

`etaXiSphere` builds the generated preset type for the sphere eta-xi system.

This node defines the fermionic η, ξ system on the sphere and records its torus draft metadata. It lowers through Preset Shared Runtime so the branch walker supplies the Grassmann signs and the zero-mode base case remains a compact residual consumer.

The stringbook conventions are $h_\eta = 1$, $h_\xi = 0$, and

$$\eta(z)\xi(w) \sim \frac{1}{z-w}.$$

On higher genus surfaces, stringbook formula (??) gives the full η, ξ, ϕ correlator as prime-form products and theta-function ratios. For the pure torus η, ξ sector, expanding the extra ξ zero mode leaves the pair kernel $\partial \log E$, the genus-one third-kind differential with residue 1. The torus runtime therefore lowers that log-derivative kernel, not the prime form itself.

```
const std = @import("std");
const shared = @import("shared.zig");
const declare = shared.declare;

const operators = @import("../expressions/operators.zig");
const Handle = shared.Handle;
const Builder = *shared.Local;
const Operator = *shared.LocalOperator;
const Spec = declare.Spec;
const wick = declare.wick;
const zero_mode = declare.zero_mode;
```

The preset owns only the four chiral field families and its own zero-mode sectors.

```
const namespace = "eta_xi";

const Family = enum {
    eta,
    xi,
    etat,
    xit,
};

const ZeroSector = enum {
    sphere,
    torus,
};

fn kind(comptime family: Family) operators.OperatorKindId {
    return declare.familyKind(namespace, family);
}

fn zeroSector(comptime sector: ZeroSector) zero_mode.Sector {
    return declare.sectorId(namespace, sector);
}
```

3.7.1 Sphere Preset

The public preset exposes only builders, named configs, text sinks, and the streaming correlator entry point. Internal kind ids and rule tables stay inside the generated config.

```

const EtaXiSphereConfig = struct {
    include_antiholomorphic_copy: bool = true,
};

/// etaXiSphere builds the generated preset type for the sphere eta-xi system.
pub fn etaXiSphere(comptime cfg: EtaXiSphereConfig) type {
    return struct {
        /// op exposes eta-xi local operator builders.
        pub const op = EtaXiSphereOp(cfg.include_antiholomorphic_copy);
        /// config exposes named correlator configs for this preset.
        pub const config = EtaXiSphereCorrelatorConfig(cfg);
        /// text exposes bounded result-inspection sinks.
        pub const text = shared.text;
        /// local constructs a label-preserving local-operator builder.
        pub fn local(allocator: std.mem.Allocator) !Builder {
            return shared.Local.init(allocator);
        }

        /// correlator streams rule matches for eta-xi insertions.
        pub fn correlator(config_ptr: anytype, ops: anytype, sink: anytype) !void {
            return shared.streamCorrelator(config_ptr, ops, sink);
        }
    };
}

```

Derivative labels mean $\partial^n \eta$ and $\partial^n \xi$. Only the undifferentiated ξ can saturate the constant zero mode.

```

fn EtaXiSphereOp(comptime include_antiholomorphic_copy: bool) type {
    const Holomorphic = struct {
        /// eta builds a holomorphic eta insertion.
        pub fn eta(local: anytype, n: u8, z: Handle.Coord) !Operator {
            return declare.operatorBuilder(etaXiOperator(.eta)).single(local, z, n, .{});
        }

        /// xi builds a holomorphic xi insertion.
        pub fn xi(local: anytype, n: u8, z: Handle.Coord) !Operator {
            return declare.operatorBuilder(etaXiOperator(.xi)).single(local, z, n, .{});
        }
    };

    if (!include_antiholomorphic_copy) return Holomorphic;

    return struct {
        /// eta builds a holomorphic eta insertion.
        pub const eta = Holomorphic.eta;
        /// xi builds a holomorphic xi insertion.
        pub const xi = Holomorphic.xi;

        /// etat builds an antiholomorphic eta insertion.
        pub fn etat(local: anytype, n: u8, zbar: Handle.Coord) !Operator {
            return declare.operatorBuilder(etaXiOperator(.etat)).single(local, zbar, n, .{});
        }

        /// xit builds an antiholomorphic xi insertion.
        pub fn xit(local: anytype, n: u8, zbar: Handle.Coord) !Operator {
            return declare.operatorBuilder(etaXiOperator(.xit)).single(local, zbar, n, .{});
        }
    };
}

```

3.7.2 Rules

The Wick rule is the same first-order pole structure used by the generic differentiated-pole emitter. Fermionic statistics are lowered as kind-id data.

```
fn holomorphicDifference() wick.CoordinateDifference {
    return wick.difference(wick.coord(.left, .position), wick.coord(.right, .position));
}

fn antiholomorphicDifference() wick.CoordinateDifference {
    return wick.difference(wick.coord(.left, .position), wick.coord(.right, .position));
}

fn etaXi() wick.Expr {
    return wick.differentiatedPole(.one, .none, holomorphicDifference(), -1, .{ .include_left
    ↪ = true, .include_right = true });
}

fn etatXit() wick.Expr {
    return wick.differentiatedPole(.one, .none, antiholomorphicDifference(), -1, .{
    ↪ .include_left = true, .include_right = true });
}

fn etaXiTorus() wick.Expr {
    return wick.expr(&.{wick.term(&.{wick.scalar(.one)}),
    ↪ &.{wick.differentiatedGreenKernel("elliptic_prime_form_log_derivative",
    ↪ holomorphicDifference(), .{ .include_left = true, .include_right = true })},
    ↪ &.{.none}}));
}

fn etatXitTorus() wick.Expr {
    return wick.expr(&.{wick.term(&.{wick.scalar(.one)}),
    ↪ &.{wick.differentiatedGreenKernel("elliptic_prime_form_log_derivative",
    ↪ antiholomorphicDifference(), .{ .include_left = true, .include_right = true })},
    ↪ &.{.none}}));
}
```

The sphere zero-mode rules consume exactly one undifferentiated ξ per selected chiral copy.

```
fn sphereZeroSpec(comptime cfg: EtaXiSphereConfig) if (cfg.include_antiholomorphic_copy)
↪ [2]Spec.ZeroModeRule else [1]Spec.ZeroModeRule {
    const holomorphic = Spec.ZeroModeRule{ .sector = zeroSector(.sphere), .expr = .{
    ↪ .eta_xi_zero_mode = .{
        .support = .sphere_holomorphic,
        .xi_kind_ids = &.{kind(.xi)},
        .normalization = .one,
    } } };
    if (!cfg.include_antiholomorphic_copy) return [_]Spec.ZeroModeRule{holomorphic};
    return [_]Spec.ZeroModeRule{
        holomorphic,
        .{ .sector = zeroSector(.sphere), .expr = .{ .eta_xi_zero_mode = .{
            .support = .sphere_antiholomorphic,
            .xi_kind_ids = &.{kind(.xit)},
            .normalization = .one,
        } } },
    };
}

fn sphereZeroStorage(comptime cfg: EtaXiSphereConfig) if (cfg.include_antiholomorphic_copy)
↪ [2]zero_mode.Rule else [1]zero_mode.Rule {
    const spec = sphereZeroSpec(cfg);
    return Spec.zeroModeRules(&spec);
}
```

```

fn torusZeroSpec(comptime cfg: EtaXiSphereConfig) if (cfg.include_antiholomorphic_copy)
↪ [2]Spec.ZeroModeRule else [1]Spec.ZeroModeRule {
    const holomorphic = Spec.ZeroModeRule{ .sector = zeroSector(.torus), .expr = .{
        ↪ .eta_xi_zero_mode = .{
            .support = .torus_holomorphic,
            .xi_kind_ids = &.{kind(.xi)},
            .normalization = .one,
        } } };
    if (!cfg.include_antiholomorphic_copy) return [_]Spec.ZeroModeRule{holomorphic};
    return [_]Spec.ZeroModeRule{
        holomorphic,
        .{ .sector = zeroSector(.torus), .expr = .{ .eta_xi_zero_mode = .{
            .support = .torus_antiholomorphic,
            .xi_kind_ids = &.{kind(.xit)},
            .normalization = .one,
        } } },
    };
}

fn torusZeroStorage(comptime cfg: EtaXiSphereConfig) if (cfg.include_antiholomorphic_copy)
↪ [2]zero_mode.Rule else [1]zero_mode.Rule {
    const spec = torusZeroSpec(cfg);
    return Spec.zeroModeRules(&spec);
}

```

The torus rules record the elliptic prime-form log derivative and the extra xi insertion required by stringbook.

```

const sphere_holomorphic_operator_spec = [_]Spec.Operator{
    .{ .name = "eta", .kind = kind(.eta), .support = .holomorphic, .insertion = .single,
        ↪ .statistics = .fermionic },
    .{ .name = "xi", .kind = kind(.xi), .support = .holomorphic, .insertion = .single,
        ↪ .statistics = .fermionic, .zero_mode_consumable = true },
};

const sphere_full_operator_spec = sphere_holomorphic_operator_spec ++ [_]Spec.Operator{
    .{ .name = "etat", .kind = kind(.etat), .support = .antiholomorphic, .insertion = .single,
        ↪ .statistics = .fermionic },
    .{ .name = "xit", .kind = kind(.xit), .support = .antiholomorphic, .insertion = .single,
        ↪ .statistics = .fermionic, .zero_mode_consumable = true },
};

const sphere_holomorphic_wick_spec = [_]Spec.WickRule{
    .{ .left = kind(.eta), .right = kind(.xi), .expr = etaXi() },
};

const sphere_full_wick_spec = sphere_holomorphic_wick_spec ++ [_]Spec.WickRule{
    .{ .left = kind(.etat), .right = kind(.xit), .expr = etatXit() },
};

const sphere_holomorphic_wick_storage = Spec.wickRules(&sphere_holomorphic_wick_spec,
↪ &sphere_holomorphic_operator_spec);
const sphere_full_wick_storage = Spec.wickRules(&sphere_full_wick_spec,
↪ &sphere_full_operator_spec);

const sphere_holomorphic_fermion_storage =
↪ Spec.fermionKinds(&sphere_holomorphic_operator_spec);
const sphere_full_fermion_storage = Spec.fermionKinds(&sphere_full_operator_spec);

const sphere_holomorphic_zero_spec = sphereZeroSpec(.{ .include_antiholomorphic_copy = false
↪ });
const sphere_full_zero_spec = sphereZeroSpec(.{});

const torus_holomorphic_wick_spec = [_]Spec.WickRule{

```

```

    .{ .left = kind(.eta), .right = kind(.xi), .expr = etaXiTorus(), .coordinate_kernels =
    ↪ &.{elliptic_prime_form_log_derivative} },
};

const torus_full_wick_spec = torus_holomorphic_wick_spec ++ [_]Spec.WickRule{
    .{ .left = kind(.etat), .right = kind(.xit), .expr = etatXitTorus(), .coordinate_kernels =
    ↪ &.{elliptic_prime_form_log_derivative} },
};

const torus_holomorphic_wick_storage = Spec.wickRules(&torus_holomorphic_wick_spec,
    ↪ &sphere_holomorphic_operator_spec);
const torus_full_wick_storage = Spec.wickRules(&torus_full_wick_spec,
    ↪ &sphere_full_operator_spec);

const torus_holomorphic_zero_spec = torusZeroSpec(.{ .include_antiholomorphic_copy = false });
const torus_full_zero_spec = torusZeroSpec(.{});

```

The asserts pin both the sphere runtime and the torus metadata so later elliptic work cannot silently drop theory data.

```

fn etaXiSphereSpec(comptime include_antiholomorphic_copy: bool) Spec.Theory {
    return .{
        .operators = if (include_antiholomorphic_copy) &sphere_full_operator_spec else
        ↪ &sphere_holomorphic_operator_spec,
        .wick_rules = if (include_antiholomorphic_copy) &sphere_full_wick_spec else
        ↪ &sphere_holomorphic_wick_spec,
        .zero_modes = if (include_antiholomorphic_copy) &sphere_full_zero_spec else
        ↪ &sphere_holomorphic_zero_spec,
    };
}

const torus_draft_spec = Spec.Theory{
    .surface = .{
        .kind = .torus,
        .coordinate_model = .elliptic,
        .modular_parameters = &.{ "tau" },
        .source = .stringbook,
    },
    .operators = &sphere_full_operator_spec,
    .wick_rules = &torus_full_wick_spec,
    .zero_modes = &torus_full_zero_spec,
};

fn etaXiOperator(comptime family: Family) Spec.Operator {
    const kind_id = kind(family);
    inline for (sphere_full_operator_spec) |operator| {
        if (operator.kind == kind_id) return operator;
    }
    @compileError("missing eta-xi operator spec");
}

fn assertSphereSpec(comptime include_antiholomorphic_copy: bool, comptime wick_count: usize,
    ↪ comptime zero_count: usize, comptime fermion_count: usize) void {
    const spec = etaXiSphereSpec(include_antiholomorphic_copy);
    if (spec.wick_rules.len != wick_count) @compileError("eta-xi sphere spec Wick count does
    ↪ not match lowered rules");
    if (spec.zero_modes.len != zero_count) @compileError("eta-xi sphere spec zero-mode count
    ↪ does not match lowered rules");
    if (spec.operators.len != fermion_count) @compileError("eta-xi sphere spec fermion
    ↪ coverage does not match lowered rules");
}

fn assertTorusDraftSpec() void {

```



```

if (torus_draft_spec.surface.kind != .torus) @compileError("eta-xi torus draft spec
↳ surface kind is incomplete");
if (torus_draft_spec.surface.coordinate_model != .elliptic) @compileError("eta-xi torus
↳ draft spec coordinate model is incomplete");
if (torus_draft_spec.surface.modular_parameters.len != 1) @compileError("eta-xi torus
↳ draft spec modular metadata is incomplete");
if (torus_draft_spec.surface.source != .stringbook) @compileError("eta-xi torus draft spec
↳ source convention is incomplete");
if (torus_draft_spec.operators.len != sphere_full_operator_spec.len) @compileError("eta-xi
↳ torus draft spec operator metadata is incomplete");
if (torus_draft_spec.wick_rules.len != sphere_full_wick_spec.len) @compileError("eta-xi
↳ torus draft spec Wick metadata is incomplete");
if (Spec.wickCoordinateKernelCount(torus_draft_spec.wick_rules,
↳ .elliptic_prime_form_log_derivative) != 2) @compileError("eta-xi torus draft spec
↳ prime-form log-derivative metadata is incomplete");
if (Spec.wickCoordinateKernelCount(torus_draft_spec.wick_rules, .elliptic_prime_form) !=
↳ 0) @compileError("eta-xi torus draft spec uses the prime form instead of its log
↳ derivative");
if (Spec.wickCoordinateKernelCount(torus_draft_spec.wick_rules, .rational_pole) != 0)
↳ @compileError("eta-xi torus draft spec still uses rational pole metadata");
if (torus_draft_spec.zero_modes.len != 2) @compileError("eta-xi torus draft spec zero-mode
↳ metadata is incomplete");
if (Spec.zeroModeConsumeKindCount(torus_draft_spec.zero_modes[0]) != 1)
↳ @compileError("eta-xi torus draft spec holomorphic zero-mode coverage is incomplete");
if (Spec.zeroModeConsumeKindCount(torus_draft_spec.zero_modes[1]) != 1)
↳ @compileError("eta-xi torus draft spec antiholomorphic zero-mode coverage is
↳ incomplete");
}

fn EtaXiSphereRules(comptime include_antiholomorphic_copy: bool) type {
    const selected_wick = if (include_antiholomorphic_copy) sphere_full_wick_storage else
↳ sphere_holomorphic_wick_storage;
    const selected_torus_wick = if (include_antiholomorphic_copy) torus_full_wick_storage else
↳ torus_holomorphic_wick_storage;
    const selected_fermions = if (include_antiholomorphic_copy) sphere_full_fermion_storage
↳ else sphere_holomorphic_fermion_storage;
    return struct {
        const sphere_wick: []const wick.Rule = &selected_wick;
        const torus_wick: []const wick.Rule = &selected_torus_wick;
        const sphere_fermion_kinds: []const operators.OperatorKindId = &selected_fermions;
    };
}

fn EtaXiSphereCorrelatorConfig(comptime cfg: EtaXiSphereConfig) type {
    const rules = EtaXiSphereRules(cfg.include_antiholomorphic_copy);
    return struct {
        const sphere_zero_storage = sphereZeroStorage(cfg);
        const torus_zero_storage = torusZeroStorage(cfg);
        comptime {
            assertSphereSpec(cfg.include_antiholomorphic_copy, rules.sphere_wick.len,
↳ sphere_zero_storage.len, rules.sphere_fermion_kinds.len);
            assertTorusDraftSpec();
        }

        /// sphere selects eta-xi sphere Wick and xi zero-mode rules.
        pub const sphere = declare.correlatorConfig({
            .wick_rules = rules.sphere_wick,
            .zero_modes = &sphere_zero_storage,
            .fermion_kinds = rules.sphere_fermion_kinds,
        }){};

        /// torus selects eta-xi torus prime-form log-derivative Wick and xi zero-mode rules.
        pub const torus = declare.correlatorConfig({
            .wick_rules = rules.torus_wick,

```

```

        .zero_modes = &torus_zero_storage,
        .fermion_kinds = rules.sphere_fermion_kinds,
    }){};
};
}

```

3.8 Preset Composition

INTERFACE

FUNCTIONS

`product(comptime factors: anytype) type`

product builds the generated preset type for a product of independent bulk presets.

`boundary(comptime Bulk: type, comptime extensions: anytype) type`

boundary builds the generated preset type for a BCFT from a bulk preset and boundary extensions.

Composition turns bulk presets such as Free Boson Preset and bc Preset into products and BCFTs. It uses the shared runtime vocabulary from Preset Shared Runtime.

The important invariant is that tensor products do not allocate a new runtime operator representation. Product quantum numbers remain lists at theory level; runtime local operators are still just insertion data, a body kind, and a label span. Composition asks the shared lowering path to merge opaque configs, so it does not name config-entry records or slice append helpers.

```

const std = @import("std");
const shared = @import("shared.zig");
const declare = shared.declare;

const Builder = *shared.Local;

```

3.8.1 Product CFTs

A product preset forms its sphere config from the generated sphere config of each factor. It deliberately does not synthesize preset-named operator namespaces. Future friendly aliases belong in the user DSL layer, not in the lowering path.

```

fn ProductCorrelatorConfig(comptime factors: anytype) type {
    const merged = declare.mergeSphereConfigs(factors);
    return struct {
        /// sphere selects composed product Wick and zero-mode rules.
        pub const sphere = merged.config;
    };
}

/// product builds the generated preset type for a product of independent bulk presets.
pub fn product(comptime factors: anytype) type {
    return struct {
        /// op is intentionally empty; use the factor builders passed to product.
        pub const op = struct {};
        /// config exposes named correlator configs for this product preset.
        pub const config = ProductCorrelatorConfig(factors);
        /// text exposes bounded result-inspection sinks.
        pub const text = shared.text;
        /// local constructs a label-preserving local-operator builder.
        pub fn local(allocator: std.mem.Allocator) !Builder {
            return shared.Local.init(allocator);
        }
    };
}

```

```

    /// correlator streams rule matches for product-theory insertions.
    pub fn correlator(config_ptr: anytype, ops: anytype, sink: anytype) !void {
        return shared.streamCorrelator(config_ptr, ops, sink);
    }
};
}

```

3.8.2 Boundary Extensions

A BCFT is built from a bulk preset plus explicit boundary extensions. The bulk preset remains boundary-free; boundary data contributes new operator families, mixed Wick rules, zero-mode caveats, and optional Chan-Paton stack data. Boundary-local builders remain on the extension handles passed to `boundary`, so composition does not inspect preset names to create aliases.

The disk config is formed by the shared merge helper. Composition keeps only the physics-facing bulk/extension relationship and does not see packed config entries, slice selectors, or append procedures.

```

fn BoundaryCorrelatorConfig(comptime Bulk: type, comptime extensions: anytype) type {
    const merged = declare.mergeBoundaryConfig(Bulk, extensions);
    return struct {
        /// disk selects composed boundary Wick and zero-mode rules.
        pub const disk = merged.config;
    };
}

fn BoundaryOp(comptime Bulk: type, comptime extensions: anytype) type {
    _ = extensions;
    return struct {
        /// bulk exposes the bulk operator namespaces.
        pub const bulk = Bulk.op;
    };
}

/// boundary builds the generated preset type for a BCFT from a bulk preset and boundary
↪ extensions.
pub fn boundary(comptime Bulk: type, comptime extensions: anytype) type {
    return struct {
        /// op exposes bulk and boundary operator builders.
        pub const op = BoundaryOp(Bulk, extensions);
        /// config exposes named correlator configs for this BCFT.
        pub const config = BoundaryCorrelatorConfig(Bulk, extensions);
        /// text exposes bounded result-inspection sinks.
        pub const text = shared.text;
        /// local constructs a label-preserving local-operator builder.
        pub fn local(allocator: std.mem.Allocator) !Builder {
            return shared.Local.init(allocator);
        }

        /// correlator streams rule matches for BCFT insertions.
        pub fn correlator(config_ptr: anytype, ops: anytype, sink: anytype) !void {
            return shared.streamCorrelator(config_ptr, ops, sink);
        }
    };
}

```

4 Lisp DSL

4.1 Lisp CFT System

The Lisp system loads descriptor emission first, then preset declarations, then the REPL fixture helpers that insert fields through the generated C ABI.

```
(defsystem "string-code-cft"
  :description "Lisp CFT presets, descriptor emission, and generated C ABI runtime helpers."
  :serial t
  :depends-on ("uiop")
  :components ((:file "string-code-cft-descriptor")
               (:file "string-code-cft-presets")
               (:module "presets"
                :serial t
                :components
                ((:file "free-fermion-10")
                 (:file "eta-xi")
                 (:file "bc-sphere")
                 (:file "free-boson-10"))))
               (:file "string-code-cft-fixtures"))))
```

4.2 Lisp Descriptor Emitter

This node is the Lisp side of the descriptor boundary Generated Descriptor Boundary. It has three jobs: store normalized row data, print deterministic Zig descriptor tables, and wrap the generated C ABI for the REPL. The descriptor rows are compile-time data; runtime expression construction only touches the C ABI records returned by Zig.

```
(defpackage #:string-code.cft.descriptor
  (:use #:cl)
  (:export
   #:make-theory
   #:add-symbol
   #:add-parameter
   #:add-quantum-number
   #:add-field-quantum-number
   #:add-surface
   #:add-field
   #:add-metadata
   #:add-wick-rule
   #:add-zero-mode
   #:make-wick-term
   #:emit-zig-descriptor
   #:write-build-manifest
   #:load-generated-library
   #:make-runtime-context
   #:destroy-runtime-context
   #:set-runtime-constant
   #:intern-runtime-symbol
   #:insert-runtime-field
   #:normal-order-runtime-field
   #:freeze-runtime-operators
   #:run-correlator
   #:count-correlator
   #:correlator-expression
   #:collect-correlator))

(in-package #:string-code.cft.descriptor)
```

The IR is plain row data. Constructors append rows in declaration order and the emitter sorts id-bearing tables before printing. Quantum-number schema rows are separate from field assignments so the next basis-generator pass can scan charges and irreps without reading Wick rules.

Symbols are stored once per theory. Every DSL symbol is normalized to a small descriptor string at this boundary, so generated Zig never sees Lisp packages.

```

(defstruct (theory (:constructor %make-theory))
  name
  hash
  (symbols (make-array 0 :adjustable t :fill-pointer 0))
  (parameters nil)
  (quantum-numbers nil)
  (field-quantum-numbers nil)
  (surfaces nil)
  (fields nil)
  (metadata nil)
  (wick-rules nil)
  (zero-modes nil)
  (result-symbols nil))

(defstruct parameter id symbol role)
(defstruct quantum-number id symbol kind group-symbol)
(defstruct field-quantum-number field quantum-number value)
(defstruct surface id kind coordinate-model modular-parameter)
(defstruct label-schema id role symbol)
(defstruct field id symbol insertion labels statistics zero-mode weight anti-weight)
(defstruct wick-term scalars coordinates tensors actions residuals)
(defstruct wick-rule surface left right terms)
(defstruct zero-mode surface kind consumes normalization two-pi-power)
(defstruct (runtime-context (:constructor %make-runtime-context))
  handle
  theory-id
  constants
  (scalar-atom-names (make-hash-table))
  scalar-monomials
  (symbols (make-array 0 :adjustable t :fill-pointer 0)))

(defun descriptor-symbol-name (value)
  (etypecase value
    (string value)
    (symbol (string-downcase (string value)))))

(defun add-symbol (theory name)
  (let ((symbols (theory-symbols theory))
        (text (descriptor-symbol-name name)))
    (or (loop for index below (length symbols)
              when (string= (aref symbols index) text)
                return index)
        (vector-push-extend text symbols))))

(defun make-theory (name hash)
  (let ((theory (%make-theory :name name :hash hash)))
    (add-symbol theory name)
    theory))

(defun add-parameter (theory symbol role)
  (let ((id (length (theory-parameters theory))))
    (push (make-parameter :id id :symbol (add-symbol theory symbol) :role role)
          (theory-parameters theory))
    id))

(defun add-quantum-number (theory symbol kind &key group)
  (let ((id (length (theory-quantum-numbers theory))))
    (push (make-quantum-number
            :id id
            :symbol (add-symbol theory (descriptor-symbol-name symbol))
            :kind kind
            :group-symbol (when group (add-symbol theory (descriptor-symbol-name group))))
          (theory-quantum-numbers theory))
    id))

```

```

(defun add-field-quantum-number (theory field quantum-number value)
  (push (make-field-quantum-number
        :field field
        :quantum-number quantum-number
        :value value)
        (theory-field-quantum-numbers theory)))

(defun add-surface (theory kind coordinate-model &key modular-parameter)
  (let ((id (length (theory-surfaces theory))))
    (push (make-surface :id id :kind kind :coordinate-model coordinate-model
                        :modular-parameter modular-parameter)
          (theory-surfaces theory))
    id))

```

Field labels infer roles from declaration tags unless the caller passes a full label row. This is intentionally small and deterministic.

```

(defun role-of-label (label)
  (etypecase label
    (label-schema (label-schema-role label))
    (cons (second label))
    (symbol :vector-index)))

(defun label-name (label)
  (etypecase label
    (label-schema (label-schema-symbol label))
    (cons (first label))
    (symbol label)))

(defun make-label-rows (theory labels)
  (loop for label in labels
        for id from 0
        collect (make-label-schema
                  :id id
                  :role (role-of-label label)
                  :symbol (add-symbol theory (string-downcase (string (label-name label)))))))

(defun add-field (theory symbol insertion labels statistics &key zero-mode weight anti-weight)
  (let ((id (length (theory-fields theory))))
    (push (make-field :id id
                      :symbol (add-symbol theory symbol)
                      :insertion insertion
                      :labels (make-label-rows theory labels)
                      :statistics statistics
                      :zero-mode zero-mode
                      :weight weight
                      :anti-weight anti-weight)
          (theory-fields theory))
    id))

```

```

(defun metadata-symbol-id (theory value)
  (etypecase value
    (integer value)
    (string (add-symbol theory value))
    (symbol (add-symbol theory (string-downcase (string value))))))

(declare (ftype function add-metadata))

(defun normalized-metadata-form (theory form)
  (etypecase form
    (integer form)
    (cons
     (case (first form)

```

```

(:rational form)
(:parameter form)
(:field-label form)
(:add (list :add
            (add-metadata theory (second form))
            (add-metadata theory (third form))))
(:mul (list :mul
            (add-metadata theory (second form))
            (add-metadata theory (third form))))
(:bilinear (list :bilinear
                 (add-metadata theory (second form))
                 (add-metadata theory (third form))
                 (metadata-symbol-id theory (fourth form))))
(otherwise (error "Unknown metadata form ~S." form))))))

(defun metadata-row-id (theory row)
  (let ((count (length (theory-metadata theory))))
    (loop for item in (theory-metadata theory)
          for index from 0
          when (equal item row)
            return (- count index 1))))

(defun add-metadata (theory form)
  (let ((row (normalized-metadata-form theory form)))
    (if (integerp row)
        row
        (or (metadata-row-id theory row)
            (let ((id (length (theory-metadata theory))))
              (push row (theory-metadata theory))
              id)))))

(defun add-wick-rule (theory surface left right terms)
  (push (make-wick-rule :surface surface :left left :right right :terms terms)
        (theory-wick-rules theory)))

(defun add-zero-mode (theory surface kind consumes &key (normalization :one) (two-pi-power 0))
  (push (make-zero-mode :surface surface :kind kind :consumes consumes
                        :normalization normalization :two-pi-power two-pi-power)
        (theory-zero-modes theory)))

(defun by-id (items reader)
  (sort (copy-list items) #'< :key reader))

```

The emitter prints only primitive descriptor data. It does not branch on theory or field names; all semantic choices have already been lowered into dense ids and compact tagged rows.

```

(defun zig-keyword (value)
  (substitute #\_ #- (string-downcase (string value))))

(defun emit-id-list (stream ids)
  (format stream "&.{")
  (loop for id in ids for first = t then nil
        do (format stream "~:[, ~;~]~D" first id))
  (format stream "}"))

(defun emit-symbols (stream theory var)
  (format stream "const ~A = [] []const u8{ " var)
  (loop for index below (length (theory-symbols theory))
        for first = t then nil
        do (format stream "~:[, ~;~]~S" first (aref (theory-symbols theory) index)))
  (format stream " };~%"))

(defun emit-label (stream label)
  (format stream ".{ .id = ~D, .role = ~A, .symbol = ~D }"))

```

```

(label-schema-id label)
(zig-keyword (label-schema-role label))
(label-schema-symbol label)))

```

```

(defun emit-field (stream field)
  (format stream ".{ .id = ~D, .symbol = ~D, .insertion = ~A, .labels = &.{ "
    (field-id field) (field-symbol field) (zig-keyword (field-insertion field)))
  (loop for label in (field-labels field) for first = t then nil
    do (progn
      (unless first (format stream ", "))
      (emit-label stream label)))
  (format stream "}, .statistics = ~A"
    (zig-keyword (field-statistics field)))
  (when (field-zero-mode field)
    (format stream ", .zero_mode_consumable = true"))
  (when (field-weight field)
    (format stream ", .weight = ~D" (field-weight field)))
  (when (field-anti-weight field)
    (format stream ", .anti_weight = ~D" (field-anti-weight field)))
  (format stream " }"))

(defun emit-surface (stream surface)
  (format stream ".{ .id = ~D, .kind = ~A, .coordinate_model = ~A"
    (surface-id surface)
    (zig-keyword (surface-kind surface))
    (zig-keyword (surface-coordinate-model surface)))
  (when (surface-modular-parameter surface)
    (format stream ", .modular_parameter = ~D" (surface-modular-parameter surface)))
  (format stream " }"))

(defun emit-parameter (stream parameter)
  (format stream ".{ .id = ~D, .symbol = ~D, .role = ~A }"
    (parameter-id parameter)
    (parameter-symbol parameter)
    (zig-keyword (parameter-role parameter))))

(defun emit-quantum-number (stream number)
  (format stream ".{ .id = ~D, .symbol = ~D, .kind = ~A"
    (quantum-number-id number)
    (quantum-number-symbol number)
    (zig-keyword (quantum-number-kind number)))
  (when (quantum-number-group-symbol number)
    (format stream ", .group_symbol = ~D" (quantum-number-group-symbol number)))
  (format stream " }"))

(defun emit-rational-row (stream numerator denominator)
  (format stream ".{ .numerator = ~D, .denominator = ~D }" numerator denominator))

(defun emit-quantum-value (stream value)
  (etypecase value
    (integer
      (format stream ".{ .integer = ~D }" value))
    (rational
      (format stream ".{ .rational = "
        (emit-rational-row stream (numerator value) (denominator value))
        (format stream " }"))
      (cons
        (ecase (first value)
          (:symbol
            (format stream ".{ .symbol = ~D }" (second value)))
          (:rational
            (format stream ".{ .rational = "
              (emit-rational-row stream (second value) (third value))
              (format stream " }"))))))))

```



```

(defun emit-field-quantum-number (stream item)
  (format stream "{ .field = ~D, .quantum_number = ~D, .value = "
    (field-quantum-number-field item)
    (field-quantum-number-quantum-number item))
  (emit-quantum-value stream (field-quantum-number-value item))
  (format stream " }"))

(defun emit-metadata (stream form)
  (case (first form)
    (:rational
     (format stream "{ .rational = "
       (emit-rational-row stream (second form) (third form))
       (format stream " }"))
    (:parameter
     (format stream "{ .parameter = ~D }" (second form)))
    (:field-label
     (format stream "{ .field_label = { .field = ~D, .label = ~D } }"
       (second form) (third form)))
    (:add
     (format stream "{ .add = { .left = ~D, .right = ~D } }"
       (second form) (third form)))
    (:mul
     (format stream "{ .mul = { .left = ~D, .right = ~D } }"
       (second form) (third form)))
    (:bilinear
     (format stream "{ .bilinear = { .left = ~D, .right = ~D, .form_symbol = ~D } }"
       (second form) (third form) (fourth form)))
    (otherwise (error "Unknown metadata form ~S." form))))

(defun side-name (value)
  (ecase value
    (:left "left")
    (:right "right")))

(defun coordinate-slot-name (value)
  (ecase value
    (:position "position")
    (:holomorphic "holomorphic")
    (:antiholomorphic "antiholomorphic")))

(defun emit-coordinate-ref (stream value)
  (etypecase value
    (symbol
     (format stream "{ .side = ~A, .slot = .position }" (side-name value)))
    (cons
     (format stream "{ .side = ~A, .slot = ~A }"
       (side-name (first value))
       (coordinate-slot-name (second value))))))

(defun emit-label-ref (stream value)
  (format stream "{ .side = ~A, .slot = ~D }"
    (side-name (first value)) (second value))

(defun emit-zig-list (stream values printer)
  (format stream "&{")
  (loop for value in values for first = t then nil
    do (progn
      (unless first (format stream ", "))
      (funcall printer stream value)))
  (format stream "}"))

(defun emit-scalar-factor (stream form)
  (etypecase form

```

```

(symbol
  (ecase form
    (:one (format stream ".one"))))
(cons
  (ecase (first form)
    (:rational
      (format stream ".{ .rational = ")
      (emit-rational-row stream (second form) (third form))
      (format stream " }"))
    (:parameter
      (format stream ".{ .parameter = ~D }" (second form)))
    (:neg-parameter-half
      (format stream ".{ .neg_parameter_half = ~D }" (second form)))
    (:neg-i-parameter-half
      (format stream ".{ .neg_i_parameter_half = ~D }" (second form)))
    (:parameter-half
      (format stream ".{ .parameter_half = ~D }" (second form))))))

(defun derive-left-p (value)
  (and (consp value) (member :derive-left value)))

(defun derive-right-p (value)
  (and (consp value) (member :derive-right value)))

(defun emit-coordinate-factor (theory stream form)
  (ecase (first form)
    (:difference-power
      (format stream ".{ .difference_power = .{ .left = ")
      (emit-coordinate-ref stream (second form))
      (format stream ", .right = ")
      (emit-coordinate-ref stream (third form))
      (format stream ", .exponent = ~D" (fourth form))
      (when (derive-left-p form)
        (format stream ", .derive_left = true"))
      (when (derive-right-p form)
        (format stream ", .derive_right = true"))
      (format stream " } }"))
    (:named-kernel
      (format stream ".{ .named_kernel = .{ .symbol = ~D, .left = "
        (metadata-symbol-id theory (second form)))
      (emit-coordinate-ref stream (third form))
      (format stream ", .right = ")
      (emit-coordinate-ref stream (fourth form))
      (when (derive-left-p form)
        (format stream ", .derive_left = true"))
      (when (derive-right-p form)
        (format stream ", .derive_right = true"))
      (format stream " } }"))
    (:green-exponential
      (format stream ".{ .green_exponential = .{ .left = ")
      (emit-coordinate-ref stream (second form))
      (format stream ", .right = ")
      (emit-coordinate-ref stream (third form))
      (format stream " } }"))))

(defun emit-tensor-factor (stream form)
  (ecase (first form)
    (:metric
      (format stream ".{ .metric = .{ .left = ")
      (emit-label-ref stream (second form))
      (format stream ", .right = ")
      (emit-label-ref stream (third form))
      (format stream " } }"))
    (:momentum-index

```

```

(format stream "{ .momentum_index = { .momentum = "
(emit-label-ref stream (second form))
(format stream ", .index = ")
(emit-label-ref stream (third form))
(format stream " } }"))
(:momentum-pair
(format stream "{ .momentum_pair = { .left = "
(emit-label-ref stream (second form))
(format stream ", .right = ")
(emit-label-ref stream (third form))
(format stream " } }"))))

(defun emit-side (stream side)
  (format stream "~A" (side-name side)))

(defun emit-wick-term (theory stream term)
  (format stream "{ .scalars = "
  (emit-zig-list stream (or (wick-term-scalars term) '(:one)) #'emit-scalar-factor)
  (format stream ", .coordinates = "
  (emit-zig-list stream (wick-term-coordinates term)
    (lambda (out form) (emit-coordinate-factor theory out form)))
  (format stream ", .tensors = "
  (emit-zig-list stream (wick-term-tensors term) #'emit-tensor-factor)
  (format stream ", .residuals = "
  (emit-zig-list stream (wick-term-residuals term) #'emit-side)
  (format stream " }"))

(defun emit-wick-rule (stream rule term-var)
  (format stream "{ .surface = ~D, .left = ~D, .right = ~D, .terms = &~A }"
    (wick-rule-surface rule)
    (wick-rule-left rule)
    (wick-rule-right rule)
    term-var))

(defun emit-zero-mode (stream rule)
  (format stream "{ .surface = ~D, .kind = .~A, .consumes = "
    (zero-mode-surface rule)
    (zig-keyword (zero-mode-kind rule)))
  (emit-id-list stream (zero-mode-consumes rule))
  (unless (eq (zero-mode-normalization rule) :one)
    (format stream ", .normalization = "
    (emit-scalar-factor stream (zero-mode-normalization rule))))
  (unless (zerop (zero-mode-two-pi-power rule))
    (format stream ", .two_pi_power = ~D" (zero-mode-two-pi-power rule)))
  (format stream " }"))

(defun emit-row-array (stream type var rows printer)
  (format stream "const ~A = [_]d.~A{~%" var type)
  (dolist (row rows)
    (format stream " "
    (funcall printer stream row)
    (format stream ",~%"))
  (format stream "};~%"))

(defun prepare-coordinate-symbols (theory form)
  (when (eq (first form) :named-kernel)
    (metadata-symbol-id theory (second form))))

(defun prepare-emitter-symbols (theory)
  (dolist (rule (theory-wick-rules theory))
    (dolist (term (wick-rule-terms rule))
      (dolist (factor (wick-term-coordinates term))
        (prepare-coordinate-symbols theory factor)))))

```

```

(defun emit-wick-term-arrays (stream theory rules prefix)
  (loop for rule in rules
        for index from 0
        for var = (format nil "~A_terms_~D" prefix index)
        collect var
        do (emit-row-array stream "WickTerm" var (wick-rule-terms rule)
                          (lambda (out term) (emit-wick-term theory out term))))))

(defun emit-zig-descriptor (theory stream &key (descriptor-name "generated_descriptor")
  → (prefix "generated"))
  (prepare-emitter-symbols theory)
  (let ((symbols-var (format nil "~A_symbols" prefix))
        (parameters-var (format nil "~A_parameters" prefix))
        (quantum-numbers-var (format nil "~A_quantum_numbers" prefix))
        (field-quantum-numbers-var (format nil "~A_field_quantum_numbers" prefix))
        (surfaces-var (format nil "~A_surfaces" prefix))
        (fields-var (format nil "~A_fields" prefix))
        (metadata-var (format nil "~A_metadata" prefix))
        (wick-var (format nil "~A_wick" prefix))
        (zero-modes-var (format nil "~A_zero_modes" prefix)))
    (emit-symbols stream theory symbols-var)
    (emit-row-array stream "Parameter" parameters-var
                      (by-id (theory-parameters theory) #'parameter-id)
                      #'emit-parameter)
    (emit-row-array stream "QuantumNumber" quantum-numbers-var
                      (by-id (theory-quantum-numbers theory) #'quantum-number-id)
                      #'emit-quantum-number)
    (emit-row-array stream "FieldQuantumNumber" field-quantum-numbers-var
                      (reverse (theory-field-quantum-numbers theory))
                      #'emit-field-quantum-number)
    (emit-row-array stream "Surface" surfaces-var
                      (by-id (theory-surfaces theory) #'surface-id)
                      #'emit-surface)
    (emit-row-array stream "Field" fields-var
                      (by-id (theory-fields theory) #'field-id)
                      #'emit-field)
    (emit-row-array stream "MetadataExpr" metadata-var
                      (reverse (theory-metadata theory))
                      #'emit-metadata)
    (let* ((rules (reverse (theory-wick-rules theory)))
           (term-vars (emit-wick-term-arrays stream theory rules prefix)))
      (format stream "const ~A = [_]d.WickRule{~%" wick-var)
      (loop for rule in rules
            for term-var in term-vars
            do (progn
                (format stream "    ")
                (emit-wick-rule stream rule term-var)
                (format stream ",~%"))
              (format stream "};~%"))
      (emit-row-array stream "ZeroModeRule" zero-modes-var
                      (reverse (theory-zero-modes theory))
                      #'emit-zero-mode)
      (format stream "pub const ~A = d.Descriptor{~%" descriptor-name)
      (format stream "    .theory_symbol = 0,~%")
      (format stream "    .theory_hash = ~D,~%" (theory-hash theory))
      (format stream "    .symbols = &~A,~%" symbols-var)
      (format stream "    .parameters = &~A,~%" parameters-var)
      (format stream "    .quantum_numbers = &~A,~%" quantum-numbers-var)
      (format stream "    .field_quantum_numbers = &~A,~%" field-quantum-numbers-var)
      (format stream "    .surfaces = &~A,~%" surfaces-var)
      (format stream "    .fields = &~A,~%" fields-var)
      (format stream "    .metadata = &~A,~%" metadata-var)
      (format stream "    .wick_rules = &~A,~%" wick-var)
      (format stream "    .zero_modes = &~A,~%" zero-modes-var)

```

```
(format stream "}~%")))
```

The manifest is deliberately small: source hash, generated path, and library path. Runtime calls require a loaded library and otherwise fail before any expression object is built. CFFI definitions are installed lazily because a user can inspect presets without loading CFFI or a generated shared object.

```
(defun write-build-manifest (path &key source-hash zig-path library-path)
  (with-open-file (stream path :direction :output)
    (format stream "(:source-hash ~S :zig-path ~S :library-path ~S)~%"
      source-hash zig-path library-path)))

(defvar *abi-bound* nil)
(defvar *abi-callback-pointer* nil)
(defvar *abi-chunk-callback-pointer* nil)
(defvar *abi-sinks* (make-hash-table))
(defvar *next-abi-sink-id* 0)
(defvar *event-size* 0)
(defvar *factor-size* 0)
(defvar *name-size* 0)
(defvar *name-ptr-offset* 0)
(defvar *name-len-offset* 0)
(defvar *term-size* 0)
(defparameter *event-kinds*
  #(:sum-term-begin :sum-term-end :wick-term-begin :wick-term-end
    :scalar :coordinate :tensor :zero-mode :residual-operator))

(declare
  (ftype function
    set-mem-aref* mem-ref* foreign-slot-value* null-pointer-p*
    pointer-address* inc-pointer* foreign-type-size* event-pointer-at
    name-pointer-at term-pointer-at term-expression
    %abi-last-error %abi-context-create
    %abi-context-destroy %abi-scalar-atom-name
    %abi-symbol-intern %abi-field-insert
    %abi-normal-ordering %abi-operator-list-freeze
    %abi-correlator-count %abi-correlator-run
    %abi-correlator-run-buffered %abi-correlator-expression-records
    %abi-expression-buffer-free))

(declare
  (ftype function
    factor-name-token-by-id factor-coordinate-expression-values
    factor-tensor-expression-values factor-scalar-expression-values
    factor-zero-mode-expression-values factor-expression-values
    term-product-expression))

(defun cffi-package ()
  (or (find-package '#:cffi)
      (error "CFFI is required to use generated libraries.")))

(defun cffi-symbol (name)
  (intern name (cffi-package)))

(defun foreign-string-to-lisp* (&rest args)
  (apply (symbol-function (cffi-symbol "FOREIGN-STRING-TO-LISP")) args))

(defun foreign-alloc* (&rest args)
  (apply (symbol-function (cffi-symbol "FOREIGN-ALLOC")) args))

(defun foreign-free* (pointer)
  (funcall (symbol-function (cffi-symbol "FOREIGN-FREE")) pointer))
```

```

(defun event-from-pointer (pointer)
  (let* ((type '(:struct generated-event))
        (kind (foreign-slot-value* pointer type 'kind))
        (name-pointer (foreign-slot-value* pointer type 'name-ptr))
        (name-length (foreign-slot-value* pointer type 'name-len))
        (name (and (> name-length 0)
                   (not (null-pointer-p* name-pointer))
                   (foreign-string-to-lisp* name-pointer :count name-length))))
    (list :kind (aref *event-kinds* kind)
          :a (foreign-slot-value* pointer type 'a)
          :b (foreign-slot-value* pointer type 'b)
          :c (foreign-slot-value* pointer type 'c)
          :d (foreign-slot-value* pointer type 'd)
          :name name)))

(defun ensure-abi-bindings ()
  (unless *abi-bound*
    (let ((defcstruct (cffi-symbol "DEFCSTRUCT"))
          (defcfun (cffi-symbol "DEFCFUN"))
          (defcallback (cffi-symbol "DEFCALLBACK"))
          (callback (cffi-symbol "CALLBACK"))
          (inc-pointer (cffi-symbol "INC-POINTER"))
          (foreign-type-size (cffi-symbol "FOREIGN-TYPE-SIZE"))
          (foreign-slot-offset (cffi-symbol "FOREIGN-SLOT-OFFSET"))
          (with-foreign-slots (cffi-symbol "WITH-FOREIGN-SLOTS"))
          (mem-aref (cffi-symbol "MEM-AREF"))
          (mem-ref (cffi-symbol "MEM-REF"))
          (foreign-slot-value (cffi-symbol "FOREIGN-SLOT-VALUE"))
          (null-pointer-p (cffi-symbol "NULL-POINTER-P"))
          (pointer-address (cffi-symbol "POINTER-ADDRESS")))
      (eval `(,defcstruct generated-event
              (kind :uint8)
              (a :uint32)
              (b :uint32)
              (c :uint32)
              (d :int32)
              (name-ptr :pointer)
              (name-len :size))))
      (eval `(,defcstruct generated-factor
              (kind :uint8)
              (a :uint32)
              (b :uint32)
              (c :uint32)
              (d :int32)
              (name-id :uint32))))
      (eval `(,defcstruct generated-name
              (ptr :pointer)
              (len :size))))
      (eval `(,defcstruct generated-term
              (first-factor :size)
              (factor-count :size))))
      (eval `(defun set-mem-aref* (pointer type index value)
              (setf (,mem-aref pointer type index) value)))
      (eval `(defun mem-ref* (pointer type)
              (,mem-ref pointer type)))
      (eval `(defun foreign-slot-value* (pointer type slot)
              (,foreign-slot-value pointer type slot)))
      (eval `(defun null-pointer-p* (pointer)
              (,null-pointer-p pointer)))
      (eval `(defun pointer-address* (pointer)
              (,pointer-address pointer)))
      (eval `(defun inc-pointer* (pointer offset)
              (,inc-pointer pointer offset)))
      (eval `(defun foreign-type-size* (type)

```

```

        (,foreign-type-size type)))
(eval `(defun event-pointer-at (events index)
        (,inc-pointer events (* index *event-size*))))
(eval `(defun name-pointer-at (names index)
        (,inc-pointer names (* index *name-size*))))
(eval `(defun term-pointer-at (terms index)
        (,inc-pointer terms (* index *term-size*))))
(setf *event-size* (eval `(,foreign-type-size '(:struct generated-event)))
      *factor-size* (eval `(,foreign-type-size '(:struct generated-factor)))
      *name-size* (eval `(,foreign-type-size '(:struct generated-name)))
      *name-ptr-offset* (eval `(,foreign-slot-offset '(:struct generated-name) 'ptr))
      *name-len-offset* (eval `(,foreign-slot-offset '(:struct generated-name) 'len))
      *term-size* (eval `(,foreign-type-size '(:struct generated-term))))
(eval `(defun term-expression (builder factors names term)
        (declare (optimize (speed 3) (safety 1) (debug 0)))
        (,with-foreign-slots ((first-factor factor-count) term (:struct
        ↪ generated-term))
          (let ((items nil)
                (scalars nil)
                (greens nil)
                (momentum-pairs nil)
                (others nil))
            (loop for index from first-factor below (+ first-factor factor-count)
                  for factor = (,inc-pointer factors (* index *factor-size*))
                  do (,with-foreign-slots ((kind a b c d name-id) factor (:struct
        ↪ generated-factor))
                    (let ((expr (factor-expression-values builder names kind a b c d
        ↪ name-id)))
                      (when (and expr (not (eql expr 1)))
                        (push expr items)
                        (cond
                          ((= kind 4) (push expr scalars))
                          ((and (= kind 5) (exp-green-expression-p expr))
                           (push expr greens))
                          ((and (= kind 6) (momentum-pair-expression-p expr))
                           (push expr momentum-pairs))
                          (t (push expr others)))))))
              (term-product-expression items scalars greens momentum-pairs others))))))
(eval `(,defcfun ("sc_generated_last_error" %abi-last-error) :pointer))
(eval `(,defcfun ("sc_generated_context_create" %abi-context-create) :pointer
        (theory-id :uint32)))
(eval `(,defcfun ("sc_generated_context_destroy" %abi-context-destroy) :void
        (context :pointer)))
(eval `(,defcfun ("sc_generated_scalar_atom_name" %abi-scalar-atom-name) :int
        (theory-id :uint32)
        (atom :uint32)
        (out-name :pointer)
        (out-name-len :pointer)))
(eval `(,defcfun ("sc_generated_symbol_intern" %abi-symbol-intern) :int
        (context :pointer)
        (name :string)
        (name-len :size)
        (out-symbol :pointer)))
(eval `(,defcfun ("sc_generated_field_insert" %abi-field-insert) :int
        (context :pointer)
        (field-id :uint16)
        (coordinates :pointer)
        (coordinate-count :size)
        (labels :pointer)
        (label-count :size)))
(eval `(,defcfun ("sc_generated_normal_ordering" %abi-normal-ordering) :int
        (context :pointer)
        (field-count :size)))
(eval `(,defcfun ("sc_generated_operator_list_freeze" %abi-operator-list-freeze) :int

```

```

        (context :pointer)))
(eval `(,defcfun ("sc_generated_correlator_count" %abi-correlator-count) :int
        (context :pointer)
        (out-count :pointer)))
(eval `(,defcfun ("sc_generated_correlator_run" %abi-correlator-run) :int
        (context :pointer)
        (payload :pointer)
        (callback :pointer)))
(eval `(,defcfun ("sc_generated_correlator_run_buffered" %abi-correlator-run-buffered)
  ↪ :int
        (context :pointer)
        (payload :pointer)
        (callback :pointer)))
(eval `(,defcfun ("sc_generated_correlator_expression_records"
  ↪ %abi-correlator-expression-records) :int
        (context :pointer)
        (out-terms :pointer)
        (out-term-count :pointer)
        (out-factors :pointer)
        (out-factor-count :pointer)
        (out-names :pointer)
        (out-name-count :pointer)))
(eval `(,defcfun ("sc_generated_expression_buffer_free" %abi-expression-buffer-free)
  ↪ :void
        (terms :pointer)
        (term-count :size)
        (factors :pointer)
        (factor-count :size)
        (names :pointer)
        (name-count :size)))
(eval `(,defcallback %abi-event-callback :int
        ((payload :pointer) (event :pointer))
        (let ((sink (gethash (mem-ref* payload :uint64) *abi-sinks*)))
          (when sink
            (funcall sink event))
          0)))
(eval `(,defcallback %abi-event-chunk-callback :int
        ((payload :pointer) (events :pointer) (event-count :size))
        (let ((sink (gethash (mem-ref* payload :uint64) *abi-sinks*)))
          (when sink
            (loop for index below event-count
                  do (funcall sink (event-pointer-at events index))))
          0)))
(setf *abi-callback-pointer* (eval `(,callback %abi-event-callback)))
(setf *abi-chunk-callback-pointer* (eval `(,callback %abi-event-chunk-callback)))
(setf *abi-bound* t)))

(defun load-generated-library (path)
  (funcall (symbol-function (cffi-symbol "LOAD-FOREIGN-LIBRARY")) path)
  (ensure-abi-bindings)
  path)

(defun check-abi (status)
  (unless (zerop status)
    (let ((message (%abi-last-error)))
      (error "Generated ABI call failed: ~A"
             (foreign-string-to-lisp* message)))))

(defun keyword-name (name)
  (intern (string-upcase name) '#:keyword))

(defun constant-name (name)
  (etypecase name
    (string (keyword-name name))))

```



```

(symbol (keyword-name (string name))))))

(defun normalize-runtime-constants (constants)
  (let ((table (make-hash-table)))
    (dolist (item constants table)
      (etypecase item
        (cons
         (setf (gethash (constant-name (car item)) table) (cdr item)))
        (symbol
         (setf (gethash (constant-name item) table) (constant-name item)))))))

(defun set-runtime-constant (context name value)
  "Set one symbolic scalar constant for later expression construction."
  (setf (gethash (constant-name name) (runtime-context-constants context)) value
        (runtime-context-scalar-monomials context) nil)
  value)

(defun runtime-constant-value (context name)
  (multiple-value-bind (value present)
    (gethash name (runtime-context-constants context))
    (if present value name)))

(defun make-runtime-context (&key library theory-id constants)
  (unless theory-id
    (error "A generated theory id is required. "))
  (when library
    (load-generated-library library))
  (ensure-abi-bindings)
  (let ((handle (%abi-context-create theory-id)))
    (when (null-pointer-p* handle)
      (check-abi -1))
    (%make-runtime-context :handle handle :theory-id theory-id
                          :constants (normalize-runtime-constants constants))))

(defun destroy-runtime-context (context)
  (when (runtime-context-handle context)
    (%abi-context-destroy (runtime-context-handle context))
    (setf (runtime-context-handle context) nil)))

(defun runtime-handle (context)
  (or (runtime-context-handle context)
      (error "No generated runtime context is active. ")))

(defun remember-runtime-symbol (context id name)
  (let ((symbols (runtime-context-symbols context)))
    (loop while (< (length symbols) id)
      do (vector-push-extend nil symbols))
    (setf (aref symbols (1- id)) (keyword-name name))
    id))

(defun runtime-symbol-name (context id)
  (let ((symbols (runtime-context-symbols context)))
    (if (and (> id 0) (<= id (length symbols)))
        (or (aref symbols (1- id)) `(:symbol ,id))
        `(:symbol ,id))))

(defun intern-runtime-symbol (context name)
  (let ((out (foreign-alloc* :uint32)))
    (unwind-protect
      (progn
        (check-abi
         (%abi-symbol-intern (runtime-handle context) name (length name) out))
        (remember-runtime-symbol context (mem-ref* out :uint32) name))
      (foreign-free* out))))

```

Runtime insertion is intentionally shallow. Lisp interns coordinate and label names once per context, passes only dense ids through C, and lets Zig own the operator-list storage.

```
(defun copy-uint32-values (values)
  (let* ((count (length values))
        (pointer (foreign-alloc* :uint32 :count (max 1 count))))
    (loop for value in values
          for index from 0
          do (set-mem-aref* pointer :uint32 index value))
    (values pointer count)))

(defun insert-runtime-field (context field-id coordinates labels)
  (multiple-value-bind (coord-pointer coord-count) (copy-uint32-values coordinates)
    (multiple-value-bind (label-pointer label-count) (copy-uint32-values labels)
      (unwind-protect
        (check-abi
         (%abi-field-insert (runtime-handle context) field-id coord-pointer coord-count
                           ↪ label-pointer label-count))
        (foreign-free* label-pointer)
        (foreign-free* coord-pointer))))
  field-id)

(defun normal-order-runtime-field (context count)
  (check-abi
   (%abi-normal-ordering (runtime-handle context) count))
  count)

(defun freeze-runtime-operators (context)
  (check-abi
   (%abi-operator-list-freeze (runtime-handle context)))
  :generated)

(defun count-correlator (context frozen)
  (declare (ignore frozen))
  (let ((out (foreign-alloc* :size)))
    (unwind-protect
      (progn
        (check-abi
         (%abi-correlator-count (runtime-handle context) out))
        (mem-ref* out :size))
      (foreign-free* out))))

(defun run-correlator-raw (context sink)
  (let* ((sink-id (prog1 *next-abi-sink-id*
                     (incf *next-abi-sink-id*)))
        (payload (foreign-alloc* :uint64)))
    (setf (gethash sink-id *abi-sinks*) sink)
    (set-mem-aref* payload :uint64 0 sink-id)
    (unwind-protect
      (check-abi
       (%abi-correlator-run (runtime-handle context) payload *abi-callback-pointer*))
      (remhash sink-id *abi-sinks*)
      (foreign-free* payload))))

(defun run-correlator-buffered (context sink)
  (let* ((sink-id (prog1 *next-abi-sink-id*
                     (incf *next-abi-sink-id*)))
        (payload (foreign-alloc* :uint64)))
    (setf (gethash sink-id *abi-sinks*) sink)
    (set-mem-aref* payload :uint64 0 sink-id)
    (unwind-protect
      (check-abi
       (%abi-correlator-run-buffered (runtime-handle context) payload
                                     ↪ *abi-chunk-callback-pointer*)))
```

```

    (remhash sink-id *abi-sinks*)
    (foreign-free* payload))))

(defun run-correlator (context frozen sink)
  (declare (ignore frozen))
  (run-correlator-raw context
    (lambda (event)
      (funcall sink (event-from-pointer event)))))

(defstruct (expression-builder (:constructor make-expression-builder (context)))
  context
  terms)

(declare
  (inline name-ptr name-len name-token product-expression sum-expression))

(defun name-ptr (name)
  (mem-ref* (inc-pointer* name *name-ptr-offset*) :pointer))

(defun name-len (name)
  (mem-ref* (inc-pointer* name *name-len-offset*) :size))

(defun name-token (name)
  (let ((pointer (name-ptr name))
        (length (name-len name)))
    (when (and (> length 0) (not (null-pointer-p* pointer)))
      (keyword-name (foreign-string-to-lisp* pointer :count length)))))

(defun factor-name-token-by-id (names id)
  (and (> id 0) (aref names (1- id))))

(defun product-expression (factors)
  (cond
    ((null factors) 1)
    ((null (cdr factors)) (car factors))
    (t (cons '* factors))))

(defun sum-expression (terms)
  (cond
    ((null terms) 0)
    ((null (cdr terms)) (car terms))
    (t (cons '+ terms))))

(defun exp-green-expression-p (expr)
  (and (consp expr) (eq (first expr) :exp-green)))

(defun momentum-pair-expression-p (expr)
  (and (consp expr) (eq (first expr) :momentum-pair)))

(defun explicit-green-expression (green exponent)
  `(expt (- ,(second green) ,(third green)) ,exponent))

(defun product-factor-list (expr)
  (if (and (consp expr) (eq (first expr) '*))
      (rest expr)
      (list expr)))

(defun momentum-sum-expression (momenta)
  (cond
    ((null momenta) 0)
    ((null (cdr momenta)) (car momenta))
    (t (cons '+ momenta))))

(defun momentum-delta-expression (header momenta)

```

```

(let ((power (second header))
      (sum (momentum-sum-expression momenta)))
  (product-expression
    (remove 1
      (list (if (zerop power) 1 `(expt (* 2 :pi) ,power))
            `(:momentum-delta ,power ,sum))))))

(defun rewrite-zero-mode-factors (items)
  (let ((rest nil)
        (momenta nil)
        (header nil)
        (has-delta nil))
    (dolist (item items)
      (when (and (consp item) (eq (first item) :momentum-delta-header))
        (setf has-delta t)))
    (dolist (item items)
      (cond
        ((and (consp item) (eq (first item) :momentum-delta-header))
         (setf header item))
        ((and (consp item) (eq (first item) :momentum-delta-momentum))
         (push (second item) momenta))
        ((and has-delta (consp item) (eq (first item) :residual))
         nil)
        (t (push item rest))))
    (if header
      (append (nreverse rest)
              (list (momentum-delta-expression header (nreverse momenta))))
      items)))

(defun term-product-expression (items scalars greens momentum-pairs others)
  (if (and greens momentum-pairs)
    (let ((exponent (product-expression
                     (append
                      (loop for scalar in (nreverse scalars)
                        append (product-factor-list scalar))
                      (nreverse momentum-pairs)))))
      (product-expression
        (append
         (mapcar (lambda (green)
                   (explicit-green-expression green exponent))
                  (nreverse greens))
         (rewrite-zero-mode-factors (nreverse others))))))
    (product-expression (rewrite-zero-mode-factors (nreverse items)))))

(defun factor-coordinate-expression-values (builder names a b d name-id)
  (let* ((context (expression-builder-context builder))
        (left (runtime-symbol-name context a))
        (right (runtime-symbol-name context b))
        (name (factor-name-token-by-id names name-id)))
    (cond
      ((null name) `(expt (- ,left ,right) ,d))
      ((eq name :exp-green) `(:exp-green ,left ,right))
      (t `((name ,left ,right)))))

(defun factor-tensor-expression-values (builder names a b name-id)
  (let* ((context (expression-builder-context builder))
        (left (runtime-symbol-name context a))
        (right (runtime-symbol-name context b))
        (name (or (factor-name-token-by-id names name-id) :tensor)))
    `((name ,left ,right)))

(defun scalar-atom-expression (context atom)
  (when (> atom 0)
    (let ((name

```

```

(multiple-value-bind (cached present)
  (gethash atom (runtime-context-scalar-atom-names context))
  (if present
    cached
    (let ((out-name (foreign-alloc* :pointer))
          (out-name-len (foreign-alloc* :size)))
      (unwind-protect
        (let ((status (%abi-scalar-atom-name
                      (runtime-context-theory-id context)
                      atom out-name out-name-len)))
          (if (zerop status)
              (let* ((pointer (mem-ref* out-name :pointer))
                     (length (mem-ref* out-name-len :size))
                     (token (keyword-name
                              (foreign-string-to-lisp*
                               pointer :count length))))
                (setf (gethash atom (runtime-context-scalar-atom-names
                                   ↪ context))
                      token))
                (setf (gethash atom (runtime-context-scalar-atom-names
                                   ↪ context))
                      `(:scalar-atom ,atom))))
              (foreign-free* out-name-len)
              (foreign-free* out-name))))))
    (if (keywordp name)
        (runtime-constant-value context name)
        name))))

(defun rational-expression (numerator denominator)
  (cond
    ((= numerator denominator) 1)
    ((= denominator 1) numerator)
    (t `(/ ,numerator ,denominator))))

(defun scalar-atom-power-expression (atom power)
  (cond
    ((or (null atom) (= power 0)) 1)
    ((= power 1) atom)
    (t `(expt ,atom ,power))))

(defun signed-byte8 (value)
  (if (> value 127) (- value 256) value))

(defun scalar-imaginary-factors (power)
  (ecase power
    (0 nil)
    (1 '(:i))
    (2 '(-1))
    (3 '(-1 :i))))

```

The expression folder consumes compact term/factor/name arrays from Zig. It builds the user-facing S-expression only after the expansion stream has already been accepted, and it keeps scalar monomial decoding cached by atom/flags.

```

(defun make-scalar-monomial-expression (context atom denominator flags numerator)
  (let* ((imaginary-power (logand flags 3))
         (atom-power (signed-byte8 (ldb (byte 8 8) flags)))
         (atom-expr (scalar-atom-expression context atom))
         (coefficient (if (and (numberp atom-expr) (= atom-power 1))
                          (/ (* numerator atom-expr) denominator)
                          (rational-expression numerator denominator)))
         (atom-factor (unless (and (numberp atom-expr) (= atom-power 1))
                          (scalar-atom-power-expression atom-expr atom-power))))
    (product-expression

```

```

(remove-if (lambda (factor) (or (null factor) (eql factor 1)))
  (append
    (list coefficient)
    (scalar-imaginary-factors imaginary-power)
    (list atom-factor))))))

(defun cached-scalar-monomial-expression (context atom denominator flags numerator)
  (loop for row in (runtime-context-scalar-monomials context)
    when (and (= atom (aref row 0))
              (= denominator (aref row 1))
              (= flags (aref row 2))
              (= numerator (aref row 3)))
      return (aref row 4)
    finally
      (let ((expr (make-scalar-monomial-expression
                     context atom denominator flags numerator)))
        (push (vector atom denominator flags numerator expr)
              (runtime-context-scalar-monomials context))
        (return expr))))

(defun scalar-monomial-expression (builder atom denominator flags numerator)
  (let ((context (expression-builder-context builder)))
    (cached-scalar-monomial-expression context atom denominator flags numerator)))

(defun factor-scalar-expression-values (builder names a b c d name-id)
  (let ((sign d))
    (cond
      ((and (= sign -1) (= name-id 0)) -1)
      (t
       (let ((name (factor-name-token-by-id names name-id)))
         (cond
           ((eq name :scalar-monomial)
            (scalar-monomial-expression builder a b c d))
           (name
            `(.name
              ,(runtime-symbol-name (expression-builder-context builder) a)
              ,(runtime-symbol-name (expression-builder-context builder) b)))
           (t 1)))))))

(defun factor-zero-mode-expression-values (builder names a b name-id)
  (let ((name (factor-name-token-by-id names name-id)))
    (cond
      ((eq name :momentum-delta)
       `(:momentum-delta-header ,a ,b))
      ((eq name :momentum-delta-momentum)
       `(:momentum-delta-momentum
         ,(runtime-symbol-name (expression-builder-context builder) a)))
      ((eq name :eta-xi-zero-mode)
       `(:eta-xi-zero-mode ,(runtime-symbol-name (expression-builder-context builder) a)))
      ((eq name :bc-top-form)
       '(:bc-top-form))
      (name `(.name))
      (t '(:zero-mode)))))

(defun factor-expression-values (builder names kind a b c d name-id)
  (case kind
    (4 (factor-scalar-expression-values builder names a b c d name-id))
    (5 (factor-coordinate-expression-values builder names a b d name-id))
    (6 (factor-tensor-expression-values builder names a b name-id))
    (7 (factor-zero-mode-expression-values builder names a b name-id))
    (8 `(:residual ,a))
    (otherwise 1)))

(defun expression-name-vector (names count)

```

```

(let ((tokens (make-array count)))
  (loop for index below count
        do (setf (aref tokens index)
                  (name-token (name-pointer-at names index))))
  tokens))

(defun fold-expression-records (context)
  (let ((out-terms (foreign-alloc* :pointer))
        (out-term-count (foreign-alloc* :size))
        (out-factors (foreign-alloc* :pointer))
        (out-factor-count (foreign-alloc* :size))
        (out-names (foreign-alloc* :pointer))
        (out-name-count (foreign-alloc* :size))
        (builder (make-expression-builder context)))
    (unwind-protect
      (progn
        (check-abi
         (%abi-correlator-expression-records
          (runtime-handle context)
          out-terms out-term-count out-factors out-factor-count out-names out-name-count))
        (let ((terms (mem-ref* out-terms :pointer))
              (term-count (mem-ref* out-term-count :size))
              (factors (mem-ref* out-factors :pointer))
              (factor-count (mem-ref* out-factor-count :size))
              (names (mem-ref* out-names :pointer))
              (name-count (mem-ref* out-name-count :size)))
          (unwind-protect
            (let ((name-tokens (expression-name-vector names name-count)))
              (loop for index below term-count
                    do (push (term-expression builder factors name-tokens
                                              (term-pointer-at terms index)
                                              (expression-builder-terms builder))))
              (%abi-expression-buffer-free terms term-count factors factor-count names
              ↪ name-count)))
            (sum-expression (nreverse (expression-builder-terms builder))))
          (foreign-free* out-name-count)
          (foreign-free* out-names)
          (foreign-free* out-factor-count)
          (foreign-free* out-factors)
          (foreign-free* out-term-count)
          (foreign-free* out-terms))))
    (defun correlator-expression (context frozen)
      (declare (ignore frozen))
      (fold-expression-records context))

    (defun collect-correlator (context frozen &key (limit 1024))
      (let ((events nil)
            (count 0))
        (run-correlator context frozen
          (lambda (event)
            (when (>= count limit)
              (error "Result collection limit reached."))
            (incf count)
            (push event events))))
        (nreverse events)))

```

4.3 Lisp Preset Compiler

This node lowers compact Lisp CFT presets to descriptor rows and emits the Zig fixture and dispatch modules imported by the generated C ABI. It does not evaluate correlators and it does not own REPL insertion helpers.

4.3.1 Package

The public surface is deliberately small: define presets, inspect one lowered theory, emit generated Zig sources, or compile the fixed generated ABI library.

```
(eval-when (:compile-toplevel :load-toplevel :execute)
  (require :asdf))

(defpackage #:string-code.cft.presets
  (:use #:cl #:string-code.cft.descriptor)
  (:export
   #:define-cft-preset
   #:preset-theory
   #:preset-theory-id-for
   #:write-generated-fixtures
   #:write-generated-sources
   #:compile-preset-library
   #:free-fermion-10
   #:eta-xi-sphere
   #:eta-xi-torus
   #:bc-sphere
   #:free-boson-10))

(in-package #:string-code.cft.presets)
```

4.3.2 Descriptor Layers

The preset path has four layers, and only the first one is user-facing:

- The preset file declares CFT data: parameters, surfaces, fields, Wick contractions, and zero modes.
- The Lisp descriptor rows are compact normalized data for Zig: symbols, surfaces, fields, metadata, Wick terms, and zero-mode rows.
- The generated Zig fixture needs internal names: table prefixes, descriptor constants, generated theory type names, and stable theory ids.
- The C ABI receives a theory id at runtime so the REPL can select a compiled generated theory.

The generated names and ids are compiler data. They are kept here, not in preset files.

4.3.3 Registry

The registry stores compile-time identity for the generated ABI. The builder is a function so users can inspect a fresh descriptor before compiling.

```
(defstruct preset name builder theory-name hash theory-id)
(defstruct field-shape id insertion labels)

(defparameter *presets* (make-hash-table))
(defparameter *next-theory-id* 1)
```

Preset names generate theory names, type names, descriptor prefixes, and theory ids. Numeric suffixes such as -10 are treated as dimension labels and are not part of generated Zig type names. Surface suffixes are kept only when a family has more than one generated surface.

```
(defun preset-options-form-p (form)
  (and (consp form) (keywordp (first form))))
```



```
(defun split-name-parts (name)
  (let ((text (string-downcase (string name)))
        (start 0)
        (parts nil))
    (loop for index from 0 below (length text)
          when (char= (aref text index) #\-)
            do (progn
                 (when (< start index)
                   (push (subseq text start index) parts))
                 (setf start (1+ index))))
    (when (< start (length text))
      (push (subseq text start) parts))
    (nreverse parts)))
```

```
(defun numeric-name-part-p (part)
  (and (> (length part) 0)
       (loop for char across part always (digit-char-p char))))

(defun surface-name-part-p (part)
  (member part '("sphere" "torus" "disk") :test #'string=))

(defun drop-trailing-numeric-parts (parts)
  (loop while (and parts (numeric-name-part-p (car (last parts))))
        do (setf parts (butlast parts)))
  parts)

(defun drop-trailing-surface-part (parts)
  (if (and parts (surface-name-part-p (car (last parts))))
      (butlast parts)
      parts))
```

```
(defun join-name-parts (parts separator)
  (with-output-to-string (stream)
    (loop for part in parts
          for first = t then nil
          do (progn
               (unless first
                 (write-string separator stream))
               (write-string part stream)))))

(defun generated-theory-name (name)
  (join-name-parts (drop-trailing-surface-part (split-name-parts name)) "-"))
```

Theory ids are assigned in preset load order. Hashes are generated from the canonical theory name and declaration forms.

```
(defun next-theory-id-form ()
  (prog1 *next-theory-id*
    (incf *next-theory-id*)))

(defun hash-byte (hash byte)
  (logand #xffffffff (* #x01000193 (logxor hash byte))))

(defun generated-theory-hash (name forms)
  (let ((hash #x811c9dc5)
        (text (with-output-to-string (stream)
                  (prin1 name stream)
                  (prin1 forms stream)))))
    (loop for char across text
          do (setf hash (hash-byte hash (char-code char))))
  hash))
```

The macro accepts optional metadata for experiments, but built-in preset files do not need ABI headers.

```
(defmacro define-cft-preset (name &body body)
  "Define NAME as a descriptor preset from CFT declaration forms."
  (let* ((user-options (if (preset-options-form-p (first body)) (first body) nil))
        (forms (if user-options (rest body) body))
        (theory-name (or (getf user-options :name)
                          (generated-theory-name name)))
        (hash (or (getf user-options :hash)
                   (generated-theory-hash theory-name forms)))
        (theory-id (or (getf user-options :theory-id)
                        (next-theory-id-form))))
    `(progn
      (defun ,name ()
        (build-cft-preset ,theory-name ,hash ',forms))
      (setf (gethash ',name *presets*)
            (make-preset :name ',name
                        :builder #' ,name
                        :theory-name ,theory-name
                        :hash ,hash
                        :theory-id ,theory-id))
      ',name)))
```

4.3.4 Tables

Preset declarations resolve names to compact descriptor ids. Fields and labels use normalized string keys so case differences in Lisp symbols do not leak into the descriptor.

```
(defun table-get (table key label)
  (multiple-value-bind (value found) (gethash key table)
    (unless found
      (error "Unknown ~A ~S." label key))
    value))

(defun option-value (items key &optional default)
  (let ((tail (member key items)))
    (if tail (second tail) default)))
```

```
(defun field-key (name)
  (string-downcase (string name)))

(defun label-key (field label)
  (list (field-key field) (string-downcase (string label))))
```

Field labels must be visible before weight metadata is parsed, because weights may refer to the field being declared.

```
(defun register-field-shape (fields labels field labels-spec)
  (let ((field-name (field-key field))
        (field-id (hash-table-count fields)))
    (setf (gethash field-name fields)
          (make-field-shape :id field-id :insertion nil :labels labels-spec))
    (loop for label in labels-spec
          for slot from 0
          do (setf (gethash (label-key field (first label)) labels) slot))
    field-id))

(defun set-field-shape-insertion (fields field insertion)
  (setf (field-shape-insertion (table-get fields (field-key field) "field"))
        insertion))
```

The current surface is the last declared surface. Single-surface presets can therefore write Wick and zero-mode rules without repeating `sphere` or `torus`.

```

(defun current-surface (surfaces)
  (or (gethash :current surfaces)
      (error "No current surface. Declare a SURFACE before surface-less WICK or ZERO-MODE
  ↪ forms.")))

(defun surface-form-id (surfaces value)
  (if (and (symbolp value) (gethash value surfaces))
      (gethash value surfaces)
      (current-surface surfaces)))

(defun starts-with-surface-p (surfaces items)
  (and (symbolp (first items)) (gethash (first items) surfaces)))

```

Field declarations have ergonomic forms and lower to the same descriptor shapes. **chiral-field** means one coordinate; **bulk-field** means holomorphic and antiholomorphic coordinates. A string symbol is optional and only needed when printed case matters, as for **dX**.

```

(defun field-declaration-parts (form insertion)
  (let* ((id (second form))
        (tail (cddr form))
        (symbol (if (stringp (first tail))
                     (prog1 (first tail) (setf tail (rest tail)))
                     (string-downcase (string id)))))
    (labels (if (and tail (consp (first tail)) (not (keywordp (first tail))))
                (prog1 (first tail) (setf tail (rest tail)))
                nil))
      (statistics (first tail))
      (options (rest tail)))
    (unless (member statistics '(:bosonic :fermionic))
      (error "Field ~S has invalid statistics ~S." id statistics))
    (values id symbol insertion labels statistics options)))

(defun declare-field
  (theory parameters quantum-numbers fields labels
   id symbol insertion label-spec statistics options)
  (let* ((field-id (register-field-shape fields labels id label-spec))
        (weight-form (option-value options :weight))
        (anti-weight-form (option-value options :anti-weight))
        (weight-id (when weight-form
                      (parse-metadata-id theory parameters fields labels weight-form)))
        (anti-weight-id (cond
                          ((null anti-weight-form) nil)
                          ((equal anti-weight-form weight-form) weight-id)
                          (t (parse-metadata-id theory parameters fields labels
                                                  anti-weight-form)))))
    (set-field-shape-insertion fields id insertion)
    (unless (= field-id
               (add-field theory symbol insertion label-spec statistics
                          :zero-mode (option-value options :zero-mode)
                          :weight weight-id
                          :anti-weight anti-weight-id))
      (error "Field table drift while declaring ~S." id))
    (parse-field-quantum-numbers
     theory quantum-numbers field-id
     (option-value options :quantum-numbers nil))))

```

4.3.5 References

Reference parsers convert preset names into descriptor ids. Integer ids are accepted only as an escape hatch for generated code.

```

(defun parse-parameter-ref (parameters value)
  (etypecase value
    (integer value)
    (symbol (table-get parameters value "parameter"))))

(defun parse-quantum-number-ref (quantum-numbers value)
  (etypecase value
    (integer value)
    (symbol (table-get quantum-numbers value "quantum number"))))

(defun parse-quantum-number-kind (kind)
  (ecase kind
    ((:u1 :u1-charge u1 u1-charge) :u1-charge)
    ((:ade-irrep ade-irrep) :ade-irrep)))

```

Quantum-number values are deliberately scalar. U(1) charges lower to exact integers or rationals; representation-valued numbers lower to symbol ids. The Zig descriptor validator checks that the value kind matches the declared quantum-number kind.

```

(defun parse-field-ref (fields value)
  (etypecase value
    (integer value)
    (symbol (field-shape-id (table-get fields (field-key value) "field")))))

(defun parse-quantum-value (theory value)
  (etypecase value
    (integer value)
    (rational value)
    (string `(:symbol ,(add-symbol theory value)))
    (symbol `(:symbol ,(add-symbol theory (string-downcase (string value)))))
    (cons
     (ecase (first value)
       ((:rational rational)
        `(:rational ,(second value) ,(third value))))))

(defun parse-field-quantum-numbers (theory quantum-numbers field-id specs)
  (dolist (spec specs)
    (destructuring-bind (name value) spec
      (add-field-quantum-number theory
                                field-id
                                (parse-quantum-number-ref quantum-numbers name)
                                (parse-quantum-value theory value)))))

(defun parse-label-ref (labels field value)
  (etypecase value
    (integer value)
    (symbol (table-get labels (label-key field value) "field label"))))

```

4.3.6 Metadata

Metadata forms describe field weights and similar declaration-time expressions. They lower to the compact descriptor metadata language.

```

(defun parse-metadata-form (theory parameters fields labels form)
  (etypecase form
    (integer form)
    (cons
     (ecase (first form)
       ((:rational rational)
        `(:rational ,(second form) ,(third form)))
       ((:parameter parameter)
        `(:parameter ,(parse-parameter-ref parameters (second form))))
       ((:field-label field-label)

```

```

`(:field-label ,(parse-field-ref fields (second form))
               ,(parse-label-ref labels (second form) (third form)))
((:add add +)
 `(:add ,(parse-metadata-id theory parameters fields labels (second form))
        ,(parse-metadata-id theory parameters fields labels (third form))))
((:mul mul *)
 `(:mul ,(parse-metadata-id theory parameters fields labels (second form))
        ,(parse-metadata-id theory parameters fields labels (third form))))
((:bilinear bilinear dot)
 `(:bilinear ,(parse-metadata-id theory parameters fields labels (second form))
              ,(parse-metadata-id theory parameters fields labels (third form))
              ,(fourth form))))))

```

The descriptor builder interns duplicate metadata rows, so repeated symbols such as k in $k \cdot k$ do not allocate duplicate descriptor nodes.

```

(defun parse-metadata-id (theory parameters fields labels form)
  (add-metadata theory (parse-metadata-form theory parameters fields labels form)))

```

4.3.7 Wick Terms

Scalar factors are user-facing products of exact rationals, i , and named parameters. The parser lowers common products to the compact descriptor tags used by Zig.

```

(defun i-symbol-p (value)
  (and (symbolp value) (string-equal value 'i)))

(defun rational-factor (value)
  (cond
   ((integerp value) value)
   ((rationalp value) value)
   ((and (consp value) (member (first value) '(:rational rational)))
    (/ (second value) (third value)))
   (t nil)))

```

The product parser is intentionally narrow. Unsupported scalar forms fail at preset compile time instead of leaking into the generated descriptor.

```

(defun scalar-product-factors (form)
  (if (and (consp form) (member (first form) '(* :mul mul)))
      (rest form)
      (list form)))

(defun scalar-factor-product (parameters form)
  (let ((coefficient 1)
        (imaginary-power 0)
        (parameter nil))
    (dolist (factor (scalar-product-factors form))
      (let ((rational (rational-factor factor)))
        (cond
         (rational
          (setf coefficient (* coefficient rational)))
         ((i-symbol-p factor)
          (setf imaginary-power (mod (1+ imaginary-power) 4)))
         ((and (consp factor) (member (first factor) '(:parameter parameter)))
          (when parameter
            (error "Scalar factor has more than one parameter: ~S." form))
          (setf parameter (parse-parameter-ref parameters (second factor))))
         ((symbolp factor)
          (when parameter
            (error "Scalar factor has more than one parameter: ~S." form))
          (setf parameter (parse-parameter-ref parameters factor)))))))

```

```
(t
  (error "Unsupported scalar factor ~S in ~S." factor form))))
(values coefficient imaginary-power parameter))
```

The final lowering preserves the compact descriptor format without exposing it to preset authors.

```
(defun parse-scalar-factor (parameters form)
  (if (eq form :one)
      :one
      (multiple-value-bind (coefficient imaginary-power parameter)
        (scalar-factor-product parameters form)
        (cond
         ((and (null parameter) (zerop imaginary-power) (= coefficient 1))
          :one)
         ((and (null parameter) (zerop imaginary-power))
          `(:rational ,(numerator coefficient) ,(denominator coefficient)))
         ((and parameter (zerop imaginary-power) (= coefficient 1))
          `(:parameter ,parameter))
         ((and parameter (zerop imaginary-power) (= coefficient 1/2))
          `(:parameter-half ,parameter))
         ((and parameter (zerop imaginary-power) (= coefficient -1/2))
          `(:neg-parameter-half ,parameter))
         ((and parameter (= imaginary-power 1) (= coefficient -1/2))
          `(:neg-i-parameter-half ,parameter))
         (t
          (error "Scalar factor cannot be lowered to the current compact descriptor: ~S."
                 form)))))))
```

Coordinate factors are already descriptor-shaped. The parser only normalizes operator names and preserves coordinate slots.

```
(defun parse-coordinate-factor (form)
  (ecase (first form)
    ((:difference-power difference-power)
     `(:difference-power ,(second form)
                          ,(third form)
                          ,(fourth form)
                          ,@(cddddr form)))
    ((:named-kernel named-kernel)
     `(:named-kernel ,(second form)
                     ,(third form)
                     ,(fourth form)
                     ,@(cddddr form)))
    ((:green-exponential green-exponential)
     `(:green-exponential ,(second form)
                           ,(third form)))))
```

Tensor factors need the current Wick pair because (:left mu) and (:right k) refer to labels on different fields.

```
(defun parse-side-label-ref (labels left-field right-field ref)
  (destructuring-bind (side value) ref
    (ecase side
      (:left `(:left ,(parse-label-ref labels left-field value)))
      (:right `(:right ,(parse-label-ref labels right-field value))))))
```

```
(defun parse-tensor-factor (labels left-field right-field form)
  (ecase (first form)
    ((:metric metric)
     `(:metric ,(parse-side-label-ref labels left-field right-field (second form))
               ,(parse-side-label-ref labels left-field right-field (third form))))
    ((:momentum-index momentum-index)
```

```

`(:momentum-index ,(parse-side-label-ref labels left-field right-field (second form))
                     ,(parse-side-label-ref labels left-field right-field (third form))))
((:momentum-pair momentum-pair)
 `(:momentum-pair ,(parse-side-label-ref labels left-field right-field (second form))
                    ,(parse-side-label-ref labels left-field right-field (third form)))))

```

The high-level Wick syntax binds field-call arguments to endpoint-local label and coordinate refs. Single fields have one position coordinate; pair fields use holomorphic and antiholomorphic slots.

```

(defun insertion-coordinate-slots (insertion)
  (ecase insertion
    (:single '(:position))
    (:pair '(:holomorphic :antiholomorphic))))

(defun coordinate-ref-form (side slot)
  (if (eq slot :position)
      side
      (list side slot)))

```

```

(defun bind-field-call (fields side call label-env coordinate-env)
  (destructuring-bind (field &rest args) call
    (let* ((shape (table-get fields (field-key field) "field"))
           (label-count (length (field-shape-labels shape)))
           (slots (insertion-coordinate-slots (field-shape-insertion shape)))
           (labels (subseq args 0 label-count))
           (coordinates (subseq args label-count)))
      (unless (= (length coordinates) (length slots))
        (error "Field call ~S has ~D coordinate arguments; expected ~D."
               call (length coordinates) (length slots)))
      (loop for label in labels
            for slot from 0
            do (setf (gethash label label-env) (list side slot)))
      (loop for coordinate in coordinates
            for slot in slots
            do (setf (gethash coordinate coordinate-env)
                     (coordinate-ref-form side slot)))
      field)))

```

Expression lowering is intentionally small: products, coordinate differences, named coordinate kernels, Green exponentials, and the supported tensor structures.

```

(defun env-ref (env name label)
  (multiple-value-bind (value found) (gethash name env)
    (unless found
      (error "Unknown ~A variable ~S." label name))
    value))

(defun coordinate-ref-position-p (ref)
  (symbolp ref))

(defun difference-form-p (form)
  (and (consp form) (member (first form) '(:difference difference))))

```

```

(defun lower-difference-power (coordinate-env difference exponent)
  (unless (and (difference-form-p difference)
               (= (length difference) 3))
    (error "Expected coordinate difference, got ~S." difference))
  (let ((left (env-ref coordinate-env (second difference) "coordinate"))
        (right (env-ref coordinate-env (third difference) "coordinate")))
    `(:difference-power ,left ,right ,exponent
      ,@(when (coordinate-ref-position-p left) '(:derive-left))
      ,@(when (coordinate-ref-position-p right) '(:derive-right)))))

```

```

(defun named-coordinate-kernel-p (form coordinate-env)
  (and (consp form)
       (= (length form) 3)
       (symbolp (first form))
       (gethash (second form) coordinate-env)
       (gethash (third form) coordinate-env)))

(defun parse-expression-coordinate (coordinate-env form)
  (cond
   ((and (consp form) (member (first form) '(/ :div div))
        (= (length form) 3)
        (eql (second form) 1))
    (lower-difference-power coordinate-env (third form) -1))
   ((and (consp form) (member (first form) '(pow exp))
        (= (length form) 3))
    (lower-difference-power coordinate-env (second form) (third form)))
   ((and (consp form) (member (first form) '(green-exp green-exponential))
        (= (length form) 3))
    `(:green-exponential ,(env-ref coordinate-env (second form) "coordinate")
      ,(env-ref coordinate-env (third form) "coordinate")))
   ((named-coordinate-kernel-p form coordinate-env)
    `(:named-kernel ,(first form)
                    ,(env-ref coordinate-env (second form) "coordinate")
                    ,(env-ref coordinate-env (third form) "coordinate")
                    :derive-left))
   (t nil)))

```

```

(defun parse-expression-tensor (label-env form)
  (when (consp form)
    (case (first form)
      ((metric)
       `(:metric ,(env-ref label-env (second form) "label")
                 ,(env-ref label-env (third form) "label"))))
      ((momentum-index)
       `(:momentum-index ,(env-ref label-env (second form) "label")
                          ,(env-ref label-env (third form) "label"))))
      ((momentum-pair)
       `(:momentum-pair ,(env-ref label-env (second form) "label")
                        ,(env-ref label-env (third form) "label"))))))

```

The expression term collector classifies product factors and leaves all scalar factors together so $(* -1/2 i \alpha\text{-prime})$ lowers as one scalar.

```

(defun expression-product-factors (form)
  (if (and (consp form) (member (first form) '(* :mul mul)))
      (loop for factor in (rest form) append (expression-product-factors factor))
      (list form)))

(defun parse-expression-term (parameters label-env coordinate-env expression residuals)
  (let ((scalar-factors nil)
        (coordinates nil)
        (tensors nil))
    (dolist (factor (expression-product-factors expression))
      (let ((coordinate (parse-expression-coordinate coordinate-env factor))
            (tensor (parse-expression-tensor label-env factor)))
        (cond
         (coordinate (push coordinate coordinates))
         (tensor (push tensor tensors))
         (t (push factor scalar-factors)))))
    (make-wick-term
     :scalars (list (parse-scalar-factor
                     parameters
                     (if scalar-factors
                         `(* ,@(reverse scalar-factors))

```



```

      :one)))
:coordinates (reverse coordinates)
:tensors (reverse tensors)
:residuals residuals)))

```

The descriptor-shaped `term` form remains as an explicit escape hatch for generated or debugging code.

```

(defun parse-term (parameters labels left-field right-field form)
  (unless (eq (first form) 'term)
    (error "Expected TERM form, got ~S." form))
  (let ((body (rest form)))
    (make-wick-term
     :scalars (mapcar (lambda (item) (parse-scalar-factor parameters item))
                      (option-value body :scalars '(:one)))
     :coordinates (mapcar #'parse-coordinate-factor
                          (option-value body :coordinates nil))
     :tensors (mapcar (lambda (item)
                        (parse-tensor-factor labels left-field right-field item))
                      (option-value body :tensors nil))
     :residuals (option-value body :residuals nil))))

```

Both high-level and descriptor-shaped Wick forms flow through this helper. The high-level form binds labels and coordinates from the two field calls and then classifies the product factors in the expression.

```

(defun add-wick-form
  (theory parameters fields labels surface left body)
  (if (and body (consp (first body)) (not (eq (caar body) 'term)))
      (let* ((right (first body))
              (expression (second body))
              (options (cddr body))
              (label-env (make-hash-table))
              (coordinate-env (make-hash-table))
              (left-field (bind-field-call fields :left left label-env coordinate-env))
              (right-field (bind-field-call fields :right right label-env coordinate-env)))
        (add-wick-rule
         theory
         surface
         (parse-field-ref fields left-field)
         (parse-field-ref fields right-field)
         (list (parse-expression-term parameters label-env coordinate-env
                                       expression
                                       (option-value options :residuals nil)))))
      (destructuring-bind (left-field right-field) left
        (add-wick-rule
         theory
         surface
         (parse-field-ref fields left-field)
         (parse-field-ref fields right-field)
         (mapcar (lambda (term)
                    (parse-term parameters labels left-field right-field term))
                  body)))))

```

4.3.8 Declarations

Declaration forms append descriptor rows in source order. Quantum-number schema rows are separate from field quantum-number assignments so the later basis generator can scan them without touching Wick data.

```

(defun apply-preset-form
  (theory parameters quantum-numbers surfaces fields labels form)
  (ecase (first form)
    (parameter
      (setf (gethash (second form) parameters)
            (add-parameter theory (third form) (or (fourth form) :scalar-parameter))))
    (quantum-number
      (setf (gethash (second form) quantum-numbers)
            (add-quantum-number theory
                                (or (option-value (cddddr form) :symbol)
                                    (string-downcase (string (second form))))
                                (parse-quantum-number-kind (third form))
                                :group (option-value (cddddr form) :group))))
    (surface
      (let ((surface-id
              (add-surface theory (third form) (fourth form)
                            :modular-parameter
                            (when (option-value (cddddr form) :modular-parameter)
                              (parse-parameter-ref
                               parameters
                               (option-value (cddddr form) :modular-parameter))))))
        (setf (gethash (second form) surfaces) surface-id
              (gethash :current surfaces) surface-id)))
    (field
      (destructuring-bind (id symbol insertion label-spec statistics &rest options) (rest form))
      (declare-field theory parameters quantum-numbers fields labels
                     id symbol insertion label-spec statistics options)))
    (chiral-field
      (multiple-value-bind (id symbol insertion label-spec statistics options)
        (field-declaration-parts form :single)
        (declare-field theory parameters quantum-numbers fields labels
                       id symbol insertion label-spec statistics options)))
    (bulk-field
      (multiple-value-bind (id symbol insertion label-spec statistics options)
        (field-declaration-parts form :pair)
        (declare-field theory parameters quantum-numbers fields labels
                       id symbol insertion label-spec statistics options))))

(wick
  (let* ((items (rest form))
         (has-surface (starts-with-surface-p surfaces items))
         (surface (surface-form-id surfaces (first items)))
         (left (if has-surface (second items) (first items)))
         (body (if has-surface (cddr items) (rest items))))
    (add-wick-form theory parameters fields labels surface left body)))

(zero-mode
  (let* ((items (rest form))
         (has-surface (starts-with-surface-p surfaces items))
         (surface (surface-form-id surfaces (first items)))
         (body (if has-surface (rest items) items)))
    (destructuring-bind (kind consumes &rest options) body
      (add-zero-mode theory
                     surface
                     kind
                     (mapcar (lambda (field) (parse-field-ref fields field)) consumes)
                     :normalization (option-value options :normalization :one)
                     :two-pi-power (option-value options :two-pi-power 0))))))

```

Building a preset uses only small name tables and returns the descriptor theory.

```

(defun build-cft-preset (name hash forms)
  (let ((theory (make-theory name hash))
        (parameters (make-hash-table))
        (quantum-numbers (make-hash-table))

```

```

(surfaces (make-hash-table))
(fields (make-hash-table :test #'equal))
(labels (make-hash-table :test #'equal)))
(dolist (form forms theory)
  (apply-preset-form
   theory parameters quantum-numbers surfaces fields labels form))))

```

4.3.9 Fixture Emission

Presets are emitted in generated theory-id order. The ids are assigned when the preset files load through ASDF.

```

(defun sorted-presets ()
  (sort (loop for preset being the hash-values of *presets*
             collect preset)
        #'< :key #'preset-theory-id))

```

```

(defun preset-name-parts (preset)
  (drop-trailing-numeric-parts (split-name-parts (preset-name preset))))

(defun preset-family-parts (preset)
  (drop-trailing-surface-part (preset-name-parts preset)))

(defun same-family-p (left right)
  (equal (preset-family-parts left) (preset-family-parts right)))

(defun preset-family-count (preset presets)
  (loop for item in presets count (same-family-p preset item)))

```

Generated Zig names are derived from the preset set. A family with one surface gets a short type such as Bc; a family with multiple surfaces keeps the surface suffix, such as EtaXiSphere.

```

(defun generated-name-parts (preset presets)
  (if (> (preset-family-count preset presets) 1)
      (preset-name-parts preset)
      (preset-family-parts preset)))

(defun generated-prefix (preset presets)
  (join-name-parts (generated-name-parts preset presets) "_"))

(defun generated-descriptor-name (preset presets)
  (format nil "~A_descriptor" (generated-prefix preset presets)))

(defun capitalized-name-part (part)
  (string-capitalize part))

(defun generated-type-name (preset presets)
  (with-output-to-string (stream)
    (dolist (part (generated-name-parts preset presets))
      (write-string (capitalized-name-part part) stream))))

```

```

(defun preset-key (name)
  (etypecase name
    (symbol
     (or (find-symbol (symbol-name name) (find-package '#:string-code.cft.presets))
         name))
    (string
     (find-symbol (string-upcase name) (find-package '#:string-code.cft.presets)))))

(defun registered-preset (name)
  (table-get *presets* (preset-key name) "preset"))

```

```
(defun preset-theory (name)
  "Build the descriptor theory declared by preset NAME."
  (funcall (preset-builder (registered-preset name))))

(defun preset-theory-id-for (name)
  "Return the generated ABI theory id for preset NAME."
  (preset-theory-id (registered-preset name)))
```

The prologue is the only Zig syntax that is not descriptor data.

```
(defun emit-fixture-prologue (stream)
  (format stream "const descriptor = @import(\"descriptor.zig\");~%~%")
  (format stream "const d = descriptor;~%~%")
  (format stream "fn cref(side: d.Side) d.CoordinateRef {~%")
  (format stream "    return .{ .side = side, .slot = .position };~%")
  (format stream "}~%~%")
  (format stream "fn bulk(side: d.Side, slot: d.CoordinateSlot) d.CoordinateRef {~%")
  (format stream "    return .{ .side = side, .slot = slot };~%")
  (format stream "}~%~%")
  (format stream "fn lref(side: d.Side, slot: d.Id) d.LabelRef {~%")
  (format stream "    return .{ .side = side, .slot = slot };~%")
  (format stream "}~%~%"))
```

Each preset becomes one descriptor plus one generated theory type.

```
(defun emit-fixture-type (stream preset presets)
  (emit-zig-descriptor (funcall (preset-builder preset)) stream
    :prefix (generated-prefix preset presets)
    :descriptor-name (generated-descriptor-name preset presets))
  (format stream "~%/// ~A is the lowered generated preset.~%"
    (generated-type-name preset presets))
  (format stream "pub const ~A = d.GeneratedTheory(~A);~%~%"
    (generated-type-name preset presets)
    (generated-descriptor-name preset presets)))
```

The generated fixture self-test checks descriptor validity without allocating correlator expressions.

```
(defun emit-fixture-self-test (stream presets)
  (format stream "/// selfTest validates that generated preset descriptors are loadable.~%"
    (format stream "pub fn selfTest() !void {~%"
      (dolist (preset presets)
        (format stream "    try d.validateDescriptor(~A);~%"
          (generated-descriptor-name preset presets))))
    (format stream "}~%"))
```

```
(defun write-generated-fixtures
  (&key (path "src/cft-code/correlators/generated_fixtures.zig"))
  "Write the Zig fixture module from registered Lisp presets."
  (with-open-file (stream path :direction :output :if-exists :supersede
    :if-does-not-exist :create)
    (let ((presets (sorted-presets)))
      (emit-fixture-prologue stream)
      (dolist (preset presets)
        (emit-fixture-type stream preset presets))
      (emit-fixture-self-test stream presets)))
  path)
```

4.3.10 Dispatch Emission

The dispatch module is generated with the fixtures. It owns `TheoryId`, the context union, and every switch over generated theory types.

```

(defun emit-dispatch-prologue (stream)
  (format stream "const std = @import(\"std\");~%")
  (format stream "const fixtures = @import(\"generated_fixtures.zig\");~%")
  (format stream "const kernel = @import(\"../kernel.zig\");~%~%")
  (format stream "const allocator = std.heap.c_allocator;~%~%"))

(defun emit-theory-id (stream presets)
  (format stream "/// TheoryId selects one generated fixture through the generic ABI.~%")
  (format stream "pub const TheoryId = enum(u32) {~%")
  (dolist (preset presets)
    (format stream "    ~A = ~D,~%"
      (generated-prefix preset presets)
      (preset-theory-id preset)))
  (format stream "};~%~%")
  (format stream "pub const first_theory_id = TheoryId.~A;~%~%"
    (generated-prefix (first presets) presets)))

```

```

(defun emit-context-fields (stream presets)
  (format stream "pub const ContextTag = union(TheoryId) {~%")
  (dolist (preset presets)
    (format stream "    ~A: *fixtures.~A.Context,~%"
      (generated-prefix preset presets)
      (generated-type-name preset presets)))
  (format stream "~~%"))

```

The context methods are direct switches. They do not allocate and expose only the operations the C ABI needs.

```

(defun emit-create-method (stream presets)
  (format stream "    pub fn create(id: TheoryId) !ContextTag {~%")
  (format stream "        return switch (id) {~%")
  (dolist (preset presets)
    (format stream "        ~A => .{ ~A = try fixtures.~A.contextCreate(allocator) },~%"
      (generated-prefix preset presets)
      (generated-prefix preset presets)
      (generated-type-name preset presets)))
  (format stream "    };~%")
  (format stream "    }~%~%"))

(defun emit-destroy-method (stream presets)
  (format stream "    pub fn destroy(self: ContextTag) void {~%")
  (format stream "        switch (self) {~%")
  (dolist (preset presets)
    (format stream "        ~A => |inner| fixtures.~A.contextDestroy(inner),~%"
      (generated-prefix preset presets)
      (generated-type-name preset presets)))
  (format stream "    };~%")
  (format stream "    }~%~%"))

```

```

(defun emit-symbol-intern-method (stream presets)
  (format stream "    pub fn symbolIntern(self: ContextTag, name: []const u8) !u32 {~%")
  (format stream "        return switch (self) {~%")
  (dolist (preset presets)
    (format stream "        ~A => |inner| fixtures.~A.symbolIntern(inner, name),~%"
      (generated-prefix preset presets)
      (generated-type-name preset presets)))
  (format stream "    };~%")
  (format stream "    }~%~%"))

(defun emit-field-insert-method (stream presets)
  (format stream "    pub fn fieldInsert(self: ContextTag, field_id: u16, coords: []const u32,")
  (format stream "        labels: []const u32) !void {~%")
  (format stream "        switch (self) {~%")

```

```

(dolist (preset presets)
  (format stream "          .~A => |inner| try fixtures.~A.fieldInsert(inner, field_id,
    ↪ coords, labels),~%"
    (generated-prefix preset presets)
    (generated-type-name preset presets)))
(format stream "      }~%"
(format stream "    }~%~%"))

```

```

(defun emit-normal-ordering-method (stream presets)
  (format stream "    pub fn normalOrdering(self: ContextTag, field_count: usize) !void {~%"
  (format stream "        switch (self) {~%"
  (dolist (preset presets)
    (format stream "          .~A => |inner| try fixtures.~A.normalOrdering(inner,
      ↪ field_count),~%"
      (generated-prefix preset presets)
      (generated-type-name preset presets)))
  (format stream "      }~%"
  (format stream "    }~%~%"))

```

```

(defun emit-freeze-method (stream presets)
  (format stream "    pub fn freeze(self: ContextTag) !kernel.Call.MultiOp {~%"
  (format stream "        return switch (self) {~%"
  (dolist (preset presets)
    (format stream "          .~A => |inner| fixtures.~A.operatorListFreeze(inner),~%"
      (generated-prefix preset presets)
      (generated-type-name preset presets)))
  (format stream "      };~%"
  (format stream "    }~%~%"))

```

```

(defun emit-count-method (stream presets)
  (format stream "    pub fn count(self: ContextTag, ops: kernel.Call.MultiOp) !usize {~%"
  (format stream "        return switch (self) {~%"
  (dolist (preset presets)
    (format stream "          .~A => fixtures.~A.correlatorCount(ops),~%"
      (generated-prefix preset presets)
      (generated-type-name preset presets)))
  (format stream "      };~%"
  (format stream "    }~%~%"))

```

```

(defun emit-run-method (stream presets)
  (format stream "    pub fn run(self: ContextTag, ops: kernel.Call.MultiOp, state: anytype,
    ↪ thunk: anytype) !void {~%"
  (format stream "        switch (self) {~%"
  (dolist (preset presets)
    (format stream "          .~A => try fixtures.~A.correlatorRun(ops, state, thunk),~%"
      (generated-prefix preset presets)
      (generated-type-name preset presets)))
  (format stream "      }~%"
  (format stream "    }~%~%"))

```

```

(defun emit-context-end (stream)
  (format stream "};~%~%"))

```

The remaining dispatch functions decode raw ids and expose generated theory hashes to Lisp.

```

(defun emit-scalar-atom-name-function (stream presets)
  (format stream "pub fn scalarAtomParameterName(id: TheoryId, atom: u32) ?[]const u8 {~%"
  (format stream "    return switch (id) {~%"
  (dolist (preset presets)
    (format stream "      .~A => fixtures.~A.scalarAtomParameterName(atom),~%"
      (generated-prefix preset presets)
      (generated-type-name preset presets)))
  (format stream "    };~%"
  (format stream "  }~%~%"))

```

```
(defun emit-theory-id-function (stream presets)
  (format stream "pub fn theoryId(raw: u32) !TheoryId {~%}"
    (format stream "    return switch (raw) {~%}"
      (dolist (preset presets)
        (format stream "        ~D => .~A,~%"
          (preset-theory-id preset)
          (generated-prefix preset presets)))
        (format stream "    else => error.UnknownTheory,~%")
      (format stream "    };~%")
      (format stream "};~%")))
```

```
(defun emit-theory-hash-function (stream presets)
  (format stream "pub fn theoryHash(id: TheoryId) u32 {~%}"
    (format stream "    return switch (id) {~%}"
      (dolist (preset presets)
        (format stream "        .~A => fixtures.~A.theoryHash(),~%"
          (generated-prefix preset presets)
          (generated-type-name preset presets)))
        (format stream "    };~%")
      (format stream "};~%")))
```

```
(defun write-generated-dispatch
  (&key (path "src/cft-code/correlators/generated_dispatch.zig"))
  "Write the generated Zig dispatcher for registered presets."
  (let ((presets (sorted-presets)))
    (with-open-file (stream path :direction :output :if-exists :supersede
      :if-does-not-exist :create)
      (emit-dispatch-prologue stream)
      (emit-theory-id stream presets)
      (emit-context-fields stream presets)
      (emit-create-method stream presets)
      (emit-destroy-method stream presets)
      (emit-symbol-intern-method stream presets)
      (emit-field-insert-method stream presets)
      (emit-normal-ordering-method stream presets)
      (emit-freeze-method stream presets)
      (emit-count-method stream presets)
      (emit-run-method stream presets)
      (emit-context-end stream)
      (emit-scalar-atom-name-function stream presets)
      (emit-theory-id-function stream presets)
      (emit-theory-hash-function stream presets)))
    path)
```

4.3.11 Compilation

The compile helper writes the generated fixture and dispatch sources imported by `src/cft-code/generated_abi.zig`.

```
(defun write-generated-sources ()
  "Write all generated Zig modules derived from registered Lisp presets."
  (write-generated-fixtures)
  (write-generated-dispatch))

(defun compile-preset-library
  (&key
    (library-path "zig-out/lib/libstring_code_cft_generated.so")
    (global-cache-dir "/tmp/zig-global-cache")
    (zig "zig"))
  "Emit registered presets and compile the generated C ABI shared library."
  (write-generated-sources)
  (uiop:run-program
    (list zig "build-lib" "-OReleaseFast"
```

```

    "--dep" "tensor-code"
    "-Mroot=src/cft-code/generated_abi.zig"
    "-Mtensor-code=src/tensor-code/tensor-code.zig"
    "-lc" "--name" "string_code_cft_generated"
    "-dynamic" "--global-cache-dir" global-cache-dir
    "-femit-bin" library-path)
:output *standard-output*
:error-output *error-output*)
library-path)

```

4.4 Generated Fixture ABI

This node exposes the smallest C ABI for the descriptor fixtures emitted by Lisp Preset Compiler. The exported entrypoints are theory-generic: a theory id selects the generated descriptor instance, and field ids distinguish runtime insertions.

```

const std = @import("std");
const descriptor = @import("correlators/descriptor.zig");
const dispatch = @import("correlators/generated_dispatch.zig");
const kernel = @import("kernel.zig");

const allocator = std.heap.c_allocator;
const TheoryId = dispatch.TheoryId;
const ContextTag = dispatch.ContextTag;

/// Context is an opaque generated-theory runtime handle for the C ABI.
const Context = struct {
    inner: ContextTag,
    frozen: ?kernel.Call.MultiOp = null,
};

```

The ABI event is a stable POD record. String names are borrowed for the duration of the callback and copied by Lisp only when it collects events.

```

/// Event is one stable C ABI result event.
pub const Event = extern struct {
    kind: u8,
    a: u32,
    b: u32,
    c: u32,
    d: i32,
    name_ptr: ?[*]const u8,
    name_len: usize,
};

/// ExpressionTerm is one product in the expanded symbolic sum.
pub const ExpressionTerm = extern struct {
    first_factor: usize,
    factor_count: usize,
};

/// ExpressionFactor is one nontrivial symbolic product factor.
pub const ExpressionFactor = extern struct {
    kind: u8,
    a: u32,
    b: u32,
    c: u32,
    d: i32,
    name_id: u32,
};

/// ExpressionName is one borrowed symbolic name used by expression factors.

```



```

pub const ExpressionName = extern struct {
    ptr: ?[*]const u8,
    len: usize,
};

/// EventCallback receives one streamed result event.
pub const EventCallback = *const fn (?*anyopaque, *const Event) callconv(.c) c_int;

/// EventChunkCallback receives a bounded chunk of streamed result events.
pub const EventChunkCallback = *const fn (?*anyopaque, [*]const Event, usize) callconv(.c)
    ↪ c_int;

const StreamState = struct {
    payload: ?*anyopaque,
    callback: EventCallback,
};

const event_chunk_capacity = 256;

const BufferedStreamState = struct {
    payload: ?*anyopaque,
    callback: EventChunkCallback,
    events: [event_chunk_capacity]Event = undefined,
    len: usize = 0,

    fn flush(self: *@This()) !void {
        if (self.len == 0) return;
        if (self.callback(self.payload, &self.events, self.len) != 0) return
            ↪ error.CallbackFailed;
        self.len = 0;
    }

    fn push(self: *@This(), event: Event) !void {
        self.events[self.len] = event;
        self.len += 1;
        if (self.len == self.events.len) try self.flush();
    }
};

fn abiEvent(event: descriptor.ResultEvent) Event {
    const name_ptr: ?[*]const u8 = if (event.name) |name| name.ptr else null;
    const name_len: usize = if (event.name) |name| name.len else 0;
    return .{
        .kind = @intFromEnum(event.kind),
        .a = event.a,
        .b = event.b,
        .c = event.c,
        .d = event.d,
        .name_ptr = name_ptr,
        .name_len = name_len,
    };
}

fn streamThunk(payload: *anyopaque, event: descriptor.ResultEvent) !void {
    const state: *StreamState = @ptrCast(@alignCast(payload));
    const stable_event = abiEvent(event);
    if (state.callback(state.payload, &stable_event) != 0) return error.CallbackFailed;
}

fn bufferedStreamThunk(payload: *anyopaque, event: descriptor.ResultEvent) !void {
    const state: *BufferedStreamState = @ptrCast(@alignCast(payload));
    try state.push(abiEvent(event));
}

```

Expression records are the fast REPL path. Zig still streams the correlator once, but this

state stores only term ranges, nontrivial factors, and interned borrowed names. Lisp builds cons expressions from these compact rows after the kernel returns.

```
const NameKey = struct {
    ptr: usize,
    len: usize,
};

const ExpressionRecordState = struct {
    terms: std.ArrayList(ExpressionTerm) = .empty,
    factors: std.ArrayList(ExpressionFactor) = .empty,
    names: std.ArrayList(ExpressionName) = .empty,
    name_ids: std.AutoHashMap(NameKey, u32) = std.AutoHashMap(NameKey, u32).init(allocator),
    term_start: usize = 0,

    fn deinit(self: *@This()) void {
        self.terms.deinit(allocator);
        self.factors.deinit(allocator);
        self.names.deinit(allocator);
        self.name_ids.deinit();
    }

    fn nameId(self: *@This(), maybe_name: ?[]const u8) !u32 {
        const name = maybe_name orelse return 0;
        const key = NameKey{ .ptr = @intFromPtr(name.ptr), .len = name.len };
        if (self.name_ids.get(key)) |id| return id;
        const id: u32 = @intCast(self.names.items.len + 1);
        try self.names.append(allocator, .{ .ptr = name.ptr, .len = name.len });
        try self.name_ids.put(key, id);
        return id;
    }

    fn finishTerm(self: *@This()) !void {
        try self.terms.append(allocator, .{
            .first_factor = self.term_start,
            .factor_count = self.factors.items.len - self.term_start,
        });
        self.term_start = self.factors.items.len;
    }

    fn pushFactor(self: *@This(), event: descriptor.ResultEvent) !void {
        if (event.kind == .scalar and event.d == 0 and event.name == null) return;
        try self.factors.append(allocator, .{
            .kind = @intFromEnum(event.kind),
            .a = event.a,
            .b = event.b,
            .c = event.c,
            .d = event.d,
            .name_id = try self.nameId(event.name),
        });
    }
};

fn expressionRecordThunk(payload: *anyopaque, event: descriptor.ResultEvent) !void {
    const state: *ExpressionRecordState = @ptrCast(@alignCast(payload));
    switch (event.kind) {
        .sum_term_begin => state.term_start = state.factors.items.len,
        .sum_term_end => try state.finishTerm(),
        .wick_term_begin, .wick_term_end => {},
        .scalar, .coordinate, .tensor, .zero_mode, .residual_operator => try
            ↪ state.pushFactor(event),
    }
}
```

Errors are reported by a process-local short buffer. The ABI does not allocate errors or throw across the C boundary.

```
var last_error_storage: [160]u8 = []u8{0} ** 160;

fn clearError() void {
    last_error_storage[0] = 0;
}

fn setErrorName(name: []const u8) c_int {
    const len = @min(name.len, last_error_storage.len - 1);
    @memcpy(last_error_storage[0..len], name[0..len]);
    last_error_storage[len] = 0;
    return -1;
}

fn setError(err: anyerror) c_int {
    return setErrorName(@errorName(err));
}

/// sc_generated_last_error returns the last ABI error string.
export fn sc_generated_last_error() [*:0]const u8 {
    return @ptrCast(&last_error_storage);
}
```

Context creation and destruction dispatch on generated theory ids. The entrypoints are generic; no theory-specific exported functions are needed.

```
/// sc_generated_descriptor_abi_version returns the descriptor ABI version.
export fn sc_generated_descriptor_abi_version() u32 {
    return descriptor.descriptor_abi_version;
}

/// sc_generated_theory_hash returns the generated theory hash for a theory id.
export fn sc_generated_theory_hash(raw_theory: u32) u32 {
    clearError();
    const id = dispatch.theoryId(raw_theory) catch |err| {
        _ = setError(err);
        return 0;
    };
    return dispatch.theoryHash(id);
}

/// sc_generated_scalar_atom_name returns a descriptor parameter name for an atom id.
export fn sc_generated_scalar_atom_name(raw_theory: u32, atom: u32, out_ptr: ???[*]const u8,
    ↪ out_len: ?*usize) c_int {
    clearError();
    const id = dispatch.theoryId(raw_theory) catch |err| return setError(err);
    const ptr = out_ptr orelse return setErrorName("NullOutput");
    const len = out_len orelse return setErrorName("NullOutput");
    const name = dispatch.scalarAtomParameterName(id, atom) orelse return
    ↪ setErrorName("UnknownScalarAtom");
    ptr.* = name.ptr;
    len.* = name.len;
    return 0;
}

/// sc_generated_context_create allocates a context for a generated theory id.
export fn sc_generated_context_create(raw_theory: u32) ?*Context {
    clearError();
    const id = dispatch.theoryId(raw_theory) catch |err| {
        _ = setError(err);
        return null;
    };
};
```

```

const ctx = allocator.create(Context) catch |err| {
    _ = setError(err);
    return null;
};
ctx.* = .{ .inner = ContextTag.create(id) catch |err| {
    allocator.destroy(ctx);
    _ = setError(err);
    return null;
} };
return ctx;
}

/// sc_generated_context_destroy releases a generated context.
export fn sc_generated_context_destroy(ctx: ?*Context) void {
    const handle = ctx orelse return;
    handle.inner.destroy();
    allocator.destroy(handle);
}

```

The mutation entrypoints operate on compact ids and slices supplied by Lisp.

```

fn byteSlice(ptr: ?[*]const u8, len: usize) ![]const u8 {
    const data = ptr orelse return error.NullPointer;
    return data[0..len];
}

fn u32Slice(ptr: ?[*]const u32, len: usize) ![]const u32 {
    if (len == 0) return &.{};
    const data = ptr orelse return error.NullPointer;
    return data[0..len];
}

/// sc_generated_symbol_intern interns a runtime symbol in one context.
export fn sc_generated_symbol_intern(ctx: ?*Context, name_ptr: ?[*]const u8, name_len: usize,
↪ out_symbol: ?*u32) c_int {
    clearError();
    const handle = ctx orelse return setErrorName("NullContext");
    const out = out_symbol orelse return setErrorName("NullOutput");
    const name = byteSlice(name_ptr, name_len) catch |err| return setError(err);
    out.* = handle.inner.symbolIntern(name) catch |err| return setError(err);
    return 0;
}

/// sc_generated_field_insert appends one generated field occurrence.
export fn sc_generated_field_insert(ctx: ?*Context, field_id: u16, coords_ptr: ?[*]const u32,
↪ coord_len: usize, labels_ptr: ?[*]const u32, label_len: usize) c_int {
    clearError();
    const handle = ctx orelse return setErrorName("NullContext");
    const coords = u32Slice(coords_ptr, coord_len) catch |err| return setError(err);
    const labels = u32Slice(labels_ptr, label_len) catch |err| return setError(err);
    handle.inner.fieldInsert(field_id, coords, labels) catch |err| return setError(err);
    handle.frozen = null;
    return 0;
}

/// sc_generated_normal_ordering tags the last count fields as one normal product.
export fn sc_generated_normal_ordering(ctx: ?*Context, count: usize) c_int {
    clearError();
    const handle = ctx orelse return setErrorName("NullContext");
    handle.inner.normalOrdering(count) catch |err| return setError(err);
    handle.frozen = null;
    return 0;
}

```

```

/// sc_generated_operator_list_freeze freezes the current operator list.
export fn sc_generated_operator_list_freeze(ctx: ?*Context) c_int {
    clearError();
    const handle = ctx orelse return setErrorName("NullContext");
    handle.frozen = handle.inner.freeze() catch |err| return setError(err);
    return 0;
}

```

Count and stream run against the last frozen operator list. Count uses the descriptor count sink and does not allocate Lisp expression data.

```

fn frozen(handle: *Context) !kernel.Call.MultiOp {
    return handle.frozen orelse error.OperatorListNotFrozen;
}

/// sc_generated_correlator_count returns the number of accepted branches.
export fn sc_generated_correlator_count(ctx: ?*Context, out_count: ?*usize) c_int {
    clearError();
    const handle = ctx orelse return setErrorName("NullContext");
    const out = out_count orelse return setErrorName("NullOutput");
    const ops = frozen(handle) catch |err| return setError(err);
    out.* = handle.inner.count(ops) catch |err| return setError(err);
    return 0;
}

/// sc_generated_correlator_run streams result events to a callback.
export fn sc_generated_correlator_run(ctx: ?*Context, payload: ?*anyopaque, callback:
    → ?EventCallback) c_int {
    clearError();
    const handle = ctx orelse return setErrorName("NullContext");
    const cb = callback orelse return setErrorName("NullCallback");
    const ops = frozen(handle) catch |err| return setError(err);
    var state = StreamState{ .payload = payload, .callback = cb };
    handle.inner.run(ops, &state, streamThunk) catch |err| return setError(err);
    return 0;
}

/// sc_generated_correlator_run_buffered streams bounded event chunks.
export fn sc_generated_correlator_run_buffered(ctx: ?*Context, payload: ?*anyopaque, callback:
    → ?EventChunkCallback) c_int {
    clearError();
    const handle = ctx orelse return setErrorName("NullContext");
    const cb = callback orelse return setErrorName("NullCallback");
    const ops = frozen(handle) catch |err| return setError(err);
    var state = BufferedStreamState{ .payload = payload, .callback = cb };
    handle.inner.run(ops, &state, bufferedStreamThunk) catch |err| return setError(err);
    state.flush() catch |err| return setError(err);
    return 0;
}

```

The expression-record export owns the temporary arrays until Lisp has copied or folded them. On every allocation failure it frees earlier arrays before returning an ABI error.

```

/// sc_generated_correlator_expression_records returns compact expression terms and factors.
export fn sc_generated_correlator_expression_records(
    ctx: ?*Context,
    out_terms: ?*[*]ExpressionTerm,
    out_term_count: ?*usize,
    out_factors: ?*[*]ExpressionFactor,
    out_factor_count: ?*usize,
    out_names: ?*[*]ExpressionName,
    out_name_count: ?*usize,
) c_int {

```

```

clearError();
const handle = ctx orelse return setErrorName("NullContext");
const terms_out = out_terms orelse return setErrorName("NullOutput");
const term_count_out = out_term_count orelse return setErrorName("NullOutput");
const factors_out = out_factors orelse return setErrorName("NullOutput");
const factor_count_out = out_factor_count orelse return setErrorName("NullOutput");
const names_out = out_names orelse return setErrorName("NullOutput");
const name_count_out = out_name_count orelse return setErrorName("NullOutput");
const ops = frozen(handle) catch |err| return setError(err);
var state = ExpressionRecordState{};
defer state.deinit();
handle.inner.run(ops, &state, expressionRecordThunk) catch |err| return setError(err);
const factors = state.factors.toOwnedSlice(allocator) catch |err| return setError(err);
const terms = state.terms.toOwnedSlice(allocator) catch |err| {
    allocator.free(factors);
    return setError(err);
};
const names = state.names.toOwnedSlice(allocator) catch |err| {
    allocator.free(terms);
    allocator.free(factors);
    return setError(err);
};
terms_out.* = terms.ptr;
term_count_out.* = terms.len;
factors_out.* = factors.ptr;
factor_count_out.* = factors.len;
names_out.* = names.ptr;
name_count_out.* = names.len;
return 0;
}

/// sc_generated_expression_buffer_free releases compact expression buffers.
export fn sc_generated_expression_buffer_free(
    terms: ?[*]ExpressionTerm,
    term_count: usize,
    factors: ?[*]ExpressionFactor,
    factor_count: usize,
    names: ?[*]ExpressionName,
    name_count: usize,
) void {
    if (terms) |ptr| allocator.free(ptr[0..term_count]);
    if (factors) |ptr| allocator.free(ptr[0..factor_count]);
    if (names) |ptr| allocator.free(ptr[0..name_count]);
}

```

The tests exercise the ABI on one tensor/coordinate path and one count path.

```

const AbiRecorder = struct {
    count: usize = 0,
    saw_tensor: bool = false,
    saw_coordinate: bool = false,

    fn push(payload: ?*anyopaque, event_ptr: *const Event) callconv(.c) c_int {
        const self: *AbiRecorder = @ptrCast(@alignCast(payload.?));
        const event = event_ptr.*;
        self.count += 1;
        if (event.kind == @intFromEnum(descriptor.ResultEventKind.tensor) and event.a == 1 and
            ↪ event.b == 2) {
            self.saw_tensor = true;
        }
        if (event.kind == @intFromEnum(descriptor.ResultEventKind.coordinate) and event.a == 3
            ↪ and event.b == 4) {
            self.saw_coordinate = true;
        }
    }
}

```

```

        return 0;
    }
};

/// selfTest runs the generated C ABI fixture invariants.
pub fn selfTest() !void {
    const ctx = sc_generated_context_create(@intFromEnum(dispatch.first_theory_id)) orelse
        ↪ return error.ContextCreateFailed;
    defer sc_generated_context_destroy(ctx);

    var mu: u32 = 0;
    var nu: u32 = 0;
    var z: u32 = 0;
    var w: u32 = 0;
    try std.testing.expectEqual(@as(c_int, 0), sc_generated_symbol_intern(ctx, "mu".ptr, 2,
        ↪ &mu));
    try std.testing.expectEqual(@as(c_int, 0), sc_generated_symbol_intern(ctx, "nu".ptr, 2,
        ↪ &nu));
    try std.testing.expectEqual(@as(c_int, 0), sc_generated_symbol_intern(ctx, "z".ptr, 1,
        ↪ &z));
    try std.testing.expectEqual(@as(c_int, 0), sc_generated_symbol_intern(ctx, "w".ptr, 1,
        ↪ &w));

    try std.testing.expectEqual(@as(c_int, 0), sc_generated_field_insert(ctx, 0, &.{z}, 1,
        ↪ &.{mu}, 1));
    try std.testing.expectEqual(@as(c_int, 0), sc_generated_field_insert(ctx, 0, &.{w}, 1,
        ↪ &.{nu}, 1));
    try std.testing.expectEqual(@as(c_int, 0), sc_generated_operator_list_freeze(ctx));

    var count: usize = 0;
    try std.testing.expectEqual(@as(c_int, 0), sc_generated_correlator_count(ctx, &count));
    try std.testing.expectEqual(@as(usize, 1), count);

    var recorder = AbiRecorder{};
    try std.testing.expectEqual(@as(c_int, 0), sc_generated_correlator_run(ctx, &recorder,
        ↪ AbiRecorder.push));
    try std.testing.expect(recorder.saw_tensor);
    try std.testing.expect(recorder.saw_coordinate);
}

```

4.5 Lisp Generated Fixture Declarations

This node records the minimal Lisp-owned declarations for the generated fixtures. They use the generic emitter from Lisp Descriptor Emitter and add one REPL macro that owns runtime lifetime and returns the symbolic expression directly.

```

(defpackage #:string-code.cft.fixtures
  (:use #:cl #:string-code.cft.descriptor)
  (:export
    #:free-fermion-10
    #:eta-xi-sphere
    #:eta-xi-torus
    #:bc-sphere
    #:free-boson-10
    #:make-free-fermion-runtime
    #:psi
    #:make-eta-xi-sphere-runtime
    #:make-eta-xi-torus-runtime
    #:eta
    #:xi
    #:make-bc-runtime

```

```

#:b-field
#:c-field
#:make-free-boson-runtime
#:dX
#:expX
#:with-correlator-context
#:correlator
#:corr
#:ops
#:R))

(in-package #:string-code.cft.fixtures)

(defun runtime-symbol-id (context value)
  (etypecase value
    (integer value)
    (string (intern-runtime-symbol context value))
    (symbol (intern-runtime-symbol context (string-downcase (string value))))))

```

The REPL macro layer rewrites user field forms into explicit runtime insertions. `ops` just groups forms; `R` groups forms and then asks Zig to normal-order the number of fields in that group. No symbolic expression is built before the operator list is frozen.

```

(eval-when (:compile-toplevel :load-toplevel :execute)
  (defun fixture-call-symbol (name)
    (if (symbolp name)
        (multiple-value-bind (symbol status)
          (find-symbol (symbol-name name) "STRING-CODE.CFT.FIXTURES")
          (if status symbol name))
        name))

  (defun literal-symbol-form (value)
    (if (symbolp value) `,value value))

  (defun fixture-form-name (form)
    (and (consp form) (symbolp (first form)) (first form)))

  (defun field-form-p (form)
    (and (consp form)
         (not (member (fixture-form-name form) '(ops R)
                       :test #'string-equal))))

  (defun field-call-form (context form)
    (unless (consp form)
      (error "Correlator field form must be a list, got ~S." form))
    `',(fixture-call-symbol (first form)) ,context
    ,@(mapcar #'literal-symbol-form (rest form))))

  (declare (ftype function correlator-body-forms correlator-form-forms))

  (defun insertion-count (form)
    (cond
      ((field-form-p form) 1)
      ((and (consp form) (string-equal (fixture-form-name form) 'ops))
       (loop for item in (rest form) sum (insertion-count item)))
      ((and (consp form) (string-equal (fixture-form-name form) 'R))
       (loop for item in (rest form) sum (insertion-count item)))
      (t (error "Unknown correlator form ~S." form))))

  (defun correlator-form-forms (context form)
    (cond
      ((field-form-p form) (list (field-call-form context form)))
      ((and (consp form) (string-equal (fixture-form-name form) 'ops))
       (correlator-body-forms context (rest form)))

```



```

    ((and (consp form) (string-equal (fixture-form-name form) 'R))
      (append (correlator-body-forms context (rest form))
        `((normal-order-runtime-field ,context ,(insertion-count form)))))
    (t (error "Unknown correlator form ~S." form))))

(defun correlator-body-forms (context forms)
  (loop for form in forms append (correlator-form-forms context form)))

(defmacro with-correlator-context ((context (maker &rest runtime-args)) &body body)
  "Bind CONTEXT to a generated runtime and release it after BODY."
  `(let ((,context ,(fixture-call-symbol maker) ,@runtime-args))
    (unwind-protect
      (progn ,@body)
      (destroy-runtime-context ,context))))

(defmacro correlator ((maker &rest runtime-args) &body fields)
  "Evaluate fixture field forms and return the symbolic correlator expression."
  (let ((context (gensym "CONTEXT-"))
        (frozen (gensym "FROZEN-")))
    `(with-correlator-context (,context ,(fixture-call-symbol maker) ,@runtime-args)
      ,@(correlator-body-forms context fields)
      (let ((frozen (freeze-runtime-operators ,context)))
        (correlator-expression ,context ,frozen)))))

(defmacro corr ((maker &rest runtime-args) &body fields)
  "Alias for CORRELATOR."
  `(correlator (,maker ,@runtime-args) ,@fields))

```

Preset accessors return fresh descriptors for inspection. Runtime constructors create C ABI contexts for evaluation.

```

(defun free-fermion-10 ()
  (string-code.cft.preset:free-fermion-10))

(defun eta-xi-sphere ()
  (string-code.cft.preset:eta-xi-sphere))

```

```

(defun eta-xi-torus ()
  (string-code.cft.preset:eta-xi-torus))

(defun bc-sphere ()
  (string-code.cft.preset:bc-sphere))

```

```

(defun free-boson-10 ()
  (string-code.cft.preset:free-boson-10))

```

The user-facing runtime constructors resolve generated theory ids through the preset registry. The numeric ids stay compiler-owned.

```

(defun make-preset-runtime (name library &key constants)
  (make-runtime-context
    :library library
    :theory-id (string-code.cft.preset:presets:presets-theory-id-for name)
    :constants constants))

(defun make-free-fermion-runtime (&key library)
  (make-preset-runtime 'free-fermion-10 library))

(defun psi (context mu z)
  (insert-runtime-field context 0
    (list (runtime-symbol-id context z))
    (list (runtime-symbol-id context mu))))

```

```

(defun make-eta-xi-sphere-runtime (&key library)
  (make-preset-runtime 'eta-xi-sphere library))

(defun make-eta-xi-torus-runtime (&key library)
  (make-preset-runtime 'eta-xi-torus library))

(defun eta (context z)
  (insert-runtime-field context 0
    (list (runtime-symbol-id context z))
    nil))

(defun xi (context z)
  (insert-runtime-field context 1
    (list (runtime-symbol-id context z))
    nil))

(defun make-bc-runtime (&key library)
  (make-preset-runtime 'bc-sphere library))

(defun b-field (context z)
  (insert-runtime-field context 0
    (list (runtime-symbol-id context z))
    nil))

(defun c-field (context z)
  (insert-runtime-field context 1
    (list (runtime-symbol-id context z))
    nil))

(defun free-boson-constants (alpha-prime constants)
  (acons :alpha-prime alpha-prime constants))

(defun make-free-boson-runtime (&key library (alpha-prime :alpha-prime) constants)
  (make-preset-runtime 'free-boson-10 library
    :constants (free-boson-constants alpha-prime constants)))

(defun dX (context mu z)
  (insert-runtime-field context 0
    (list (runtime-symbol-id context z)
      (list (runtime-symbol-id context mu)))))

(defun expX (context k z z-bar)
  (insert-runtime-field context 1
    (list (runtime-symbol-id context z)
      (runtime-symbol-id context z-bar))
    (list (runtime-symbol-id context k)))))

```

4.6 Lisp Generated Expression Benchmark

This benchmark times the Lisp expression folder over compact generated ABI term/factor records. The comparison baseline is the generated C ABI event stream, not the count-only path, because expanded expression output must still read every emitted factor. Build the generated shared library with `-Doptimize=ReleaseFast` before timing.

```

(ql:quickload :cffi :silent t)
(require :asdf)
(push #p"lisp/" asdf:*central-registry*)
(asdf:load-system "string-code-cft")

(defparameter *generated-library* "zig-out/lib/libstring_code_cft_generated.so")
(defparameter *sample-count* 11)

```

```

(defstruct bench-case name maker fill abi-run-ns)

(defun now-ns ()
  (multiple-value-bind (sec usec) (sb-ext:get-time-of-day)
    (+ (* sec 1000000000) (* usec 1000))))

(defun median (values)
  (let ((copy (sort (copy-seq values) #'<)))
    (aref copy (floor (length copy) 2))))

(defun sample-ns (thunk)
  (let ((samples (make-array *sample-count*)))
    (dotimes (index *sample-count*)
      (sb-ext:gc :full nil)
      (let ((start (now-ns)))
        (funcall thunk)
        (setf (aref samples index) (- (now-ns) start))))
    (median samples)))

(defun expression-term-count (expr)
  (if (and (consp expr) (eq (first expr) '+))
      (length (rest expr))
      1))

(defun fill-psi (count)
  (lambda (context)
    (dotimes (index count)
      (string-code.cft.fixtures:psi context
        (format nil "mu~D" index)
        (format nil "z~D" index)))))

(defun fill-dx (count)
  (lambda (context)
    (dotimes (index count)
      (string-code.cft.fixtures:dX context
        (format nil "mu~D" index)
        (format nil "z~D" index)))))

(defun fill-exp (count)
  (lambda (context)
    (dotimes (index count)
      (string-code.cft.fixtures:expX context
        (format nil "k~D" index)
        (format nil "z~D" index)
        (format nil "zb~D" index)))))

(defun run-case (case)
  (let ((context (funcall (bench-case-maker case) :library *generated-library*)))
    (unwind-protect
      (progn
        (funcall (bench-case-fill case) context)
        (let* ((frozen (string-code.cft.descriptor:freeze-runtime-operators context))
              (count (string-code.cft.descriptor:count-correlator context frozen))
              (expr-ns (sample-ns
                        (lambda ()
                          (expression-term-count
                           (string-code.cft.descriptor:correlator-expression context
                             ↪ frozen)))))
                (ratio (/ (float expr-ns) (bench-case-abi-run-ns case))))
          (format t "~A count=~D expr-ns=~D abi-run-ns=~D ratio=~.2F~%"
                  (bench-case-name case) count expr-ns
                  (bench-case-abi-run-ns case) ratio)))
      (string-code.cft.descriptor:destroy-runtime-context context))))

```

```

(defparameter *cases*
  (list
    (make-bench-case :name "psi8"
      :maker #'string-code.cft.fixtures:make-free-fermion-runtime
      :fill (fill-psi 8)
      :abi-run-ns 136715)
    (make-bench-case :name "psi10"
      :maker #'string-code.cft.fixtures:make-free-fermion-runtime
      :fill (fill-psi 10)
      :abi-run-ns 1056899)
    (make-bench-case :name "dx8"
      :maker #'string-code.cft.fixtures:make-free-boson-runtime
      :fill (fill-dx 8)
      :abi-run-ns 125244)
    (make-bench-case :name "exp4"
      :maker #'string-code.cft.fixtures:make-free-boson-runtime
      :fill (fill-exp 4)
      :abi-run-ns 8268)))

(dolist (case *cases*)
  (run-case case))

```

4.7 Lisp Free Fermion Preset

This preset declares the indexed sphere free fermion. The only Wick datum is the $\eta_{\mu\nu}/(z-w)$ contraction.

```

(in-package #:string-code.cft.presets)

(define-cft-preset free-fermion-10
  (surface sphere :sphere :rational)
  (quantum-number spin10 :ade-irrep :group d5)
  (chiral-field psi ((mu :vector-index)) :fermionic
    :quantum-numbers ((spin10 vector)))
  (wick (psi mu z) (psi nu w)
    (* (metric mu nu) (/ 1 (- z w)))))

```

4.8 Lisp Eta-Xi Presets

The η, ξ presets share fields but use different surface kernels. The sphere rule uses the rational pole; the torus rule names the prime-form derivative kernel.

```

(in-package #:string-code.cft.presets)

```

4.8.1 Sphere

```

(define-cft-preset eta-xi-sphere
  (surface sphere :sphere :rational)
  (quantum-number ghost-number :u1)
  (chiral-field eta :fermionic
    :quantum-numbers ((ghost-number 1)))
  (chiral-field xi :fermionic
    :zero-mode t
    :quantum-numbers ((ghost-number -1)))
  (wick (eta z) (xi w)
    (/ 1 (- z w)))
  (zero-mode :constant-fermion (xi)))

```

4.8.2 Torus

```
(define-cft-preset eta-xi-torus
  (parameter tau "tau" :modular-parameter)
  (surface torus :torus :elliptic :modular-parameter tau)
  (quantum-number ghost-number :u1)
  (chiral-field eta :fermionic
    :quantum-numbers ((ghost-number 1)))
  (chiral-field xi :fermionic
    :zero-mode t
    :quantum-numbers ((ghost-number -1)))
  (wick (eta z) (xi w)
    (prime-log-d z w))
  (zero-mode :constant-fermion (xi)))
```

4.9 Lisp bc Sphere Preset

The sphere *bc* preset declares a single $b(z)c(w)$ rational-pole contraction and the c top-form zero mode.

```
(in-package #:string-code.cft.presets)
```

```
(define-cft-preset bc-sphere
  (surface sphere :sphere :rational)
  (quantum-number ghost-number :u1)
  (chiral-field b :fermionic
    :quantum-numbers ((ghost-number -1)))
  (chiral-field c :fermionic
    :zero-mode t
    :quantum-numbers ((ghost-number 1)))
  (wick (b z) (c w)
    (/ 1 (- z w)))
  (zero-mode :top-form-fermion (c)))
```

4.10 Lisp Free Boson Preset

This preset declares the ten-dimensional free boson on the sphere. The scalar parameter `alpha-prime` is symbolic in the descriptor and can be assigned a runtime convention value by the Lisp REPL layer.

```
(in-package #:string-code.cft.presets)
```

The field declarations keep label-dependent conformal weights in descriptor metadata. The $k \cdot k$ expression references the `expX` momentum label by name, not by a numeric slot.

```
(define-cft-preset free-boson-10
  (parameter alpha-prime "alpha-prime" :scalar-parameter)
  (surface sphere :sphere :rational)
  (quantum-number spin10 :ade-irrep :group d5)
  (chiral-field dX "dX" ((mu :vector-index)) :bosonic
    :quantum-numbers ((spin10 vector)))
  (bulk-field expX "expX" ((k :momentum)) :bosonic
    :zero-mode t
    :weight (* (rational 1 4)
      (* (parameter alpha-prime)
        (dot (field-label expX k)
          (field-label expX k)
          eta)))
    :anti-weight (* (rational 1 4)
```

```

      (* (parameter alpha-prime)
         (dot (field-label expX k)
              (field-label expX k)
              eta))))

```

The $dXdX$ contraction carries the metric tensor and the second-order rational pole.

```

(wick (dX mu z) (dX nu w)
  (* -1/2 alpha-prime
    (metric mu nu)
    (pow (- z w) -2)))

```

The dXe^{ikX} contraction leaves the plane wave as a residual operator, which keeps the expansion tree fixed for the streaming kernel.

```

(wick (dX mu z) (expX k w wb)
  (* -1/2 i alpha-prime
    (momentum-index k mu)
    (/ 1 (- z w)))
:residuals (:right))

```

The exponential-exponential rule emits holomorphic and antiholomorphic Green factors with the shared momentum-pair tensor.

```

(wick (expX p z zb) (expX k w wb)
  (* 1/2 alpha-prime
    (momentum-pair p k)
    (green-exp z w)
    (green-exp zb wb))
:residuals (:left :right))

```

The sphere zero-mode factor is the stringbook momentum-conservation normalization for ten noncompact bosons.

```

(zero-mode :boson-momentum-conservation (expX) :two-pi-power 10))

```

5 Normal-Ordering

This module owns the generic normal-ordering group policy. The group id stays on `kernel.Call.LocalOp` because Wick traversal needs it in the hot pair loop; this module only allocates, tags, and compares those compact ids.

```

const kernel = @import("../kernel.zig");

/// Group is the compact id stored on local operators in one normal product.
pub const Group = u16;

/// none marks an operator that is not inside a normal-ordered product.
pub const none: Group = 0;

/// first is the first valid context-local normal-ordering group id.
pub const first: Group = 1;

```

Group ids are context-local. Overflow wraps to `none`, and the next request fails instead of silently producing an untagged normal product.

```

/// next returns a fresh normal-ordering group and advances the counter.
pub fn next(counter: *Group) !Group {
  if (counter.* == none) return error.TooManyNormalOrderedProducts;
  const group = counter.*;
  counter.* += 1;
}

```

```

    return group;
}

/// tag returns OP marked as part of GROUP.
pub fn tag(op: kernel.Call.LocalOp, group: Group) kernel.Call.LocalOp {
    var tagged = op;
    tagged.normal_order_group = group;
    return tagged;
}

```

The Wick kernel treats equal nonzero group ids as a forbidden contraction pair. Bulk tagging is used by both generated descriptor contexts and hand-written Zig preset builders.

```

/// same reports whether two operators are in the same nonempty normal product.
pub fn same(left: kernel.Call.LocalOp, right: kernel.Call.LocalOp) bool {
    const group = left.normal_order_group;
    return group != none and group == right.normal_order_group;
}

/// tagLast marks the last COUNT operators as one normal-ordered product.
pub fn tagLast(ops: []kernel.Call.LocalOp, count: usize, counter: *Group) !void {
    if (count == 0 or count > ops.len) return error.InvalidNormalOrderGroup;
    const group = try next(counter);
    const start = ops.len - count;
    for (ops[start..]) |*op| {
        op.* = tag(op.*, group);
    }
}

```

6 Correlators

This module defines the notion of a correlator, which takes symbolic Expressions and computes their correlator (a Coefficient) via several possible tactics inferred from the definition of the Theory involved.

The base tactics are

- Abstract: the correlator remains unevaluated, only returns zero if selection rules as inferred through Tensor-Code from the quantum numbers of the inputs (they must intertwine to a singlet modulo possible anomalies, which can vary from one Riemann surface to another i.e. ghost number adding up to 3 on the sphere for bc system) are not met.
- Wick: the correlator is evaluated by Wick contractions until a base case is reached. Both the value of the Wick propagator and what the base case is can depend on the Riemann surface, but the underlying abstract algorithm remains the same. What we mean by base case is that the correlator dependence on zero-modes cannot quite be inferred by a Wick contraction logic (e.g. why the base case $\langle ccc \rangle$ is not zero for c-ghosts of bc system on a sphere, note that e.g. the base case $\langle c \partial c c \rangle$ follows). The zero mode data is specified by the user, just as the Wick contractions are surface-by-surface, yet the underlying algorithm remains fixed.
- Bosonization: the correlator is evaluated by writing out all tensor structures of Tensor-Code that can appear in the result and matching their coefficients by plugging in particular values of group-theoretic indices to both sides of the correlator, evaluating the tensor structures numerically and evaluating the correlator via bosonization rules and the Wick tactic on the bosonized form of the operators.

7 OPE

This module defines the notion of an OPE, which takes symbolic Expressions and computes their OPE (again Expressions) via several possible tactics inferred from the definition of the Theory involved.

Since the OPE is a local expression, we evaluate it via the definition of the theory on the Riemann sphere. When the user defines a non-degenerate pairing (for a unitary CFT, can be the usual BPZ pairing) on the basis of Operators, we can evaluate the OPE by writing out all Operators that can appear in the OPE (we know the quantum numbers of the inputs so we know the quantum numbers of the output, being in their tensor product) via Basis-Generation and compute their coefficients by combining the pairing with Correlators. This way, we can immediately transfer optimizations of Correlators and Basis-Generation to an optimized OPE.

Tensor-Code

API

TYPES

`SimpleLieAlgebra = symmetry.SimpleLieAlgebra`

SimpleLieAlgebra stores the simple family and rank.

`LieFamily = symmetry.LieFamily`

LieFamily names the supported simple Lie families.

`QuantumNumbers = symmetry.QuantumNumbers`

QuantumNumbers is the tensor-product list of representation ids.

`RepresentationId = symmetry.RepresentationId`

RepresentationId names a representation in a representation store.

`TensorStructure = symmetry.TensorStructure`

TensorStructure stores simple built-in tensor structures.

`AlgebraHandle = store.AlgebraHandle`

AlgebraHandle names a registered Lie algebra in a tensor context.

`IrrepHandle = store.IrrepHandle`

IrrepHandle names an abstract irreducible representation.

`RealizationHandle = realization.RealizationHandle`

RealizationHandle names a concrete index realization of an irrep.

`RealizationChannel = realization.RealizationChannel`

RealizationChannel selects one irrep copy inside a realization product.

`BasisHandle = coupling.BasisHandle`

BasisHandle names a generated invariant basis.

`InvariantHandle = coupling.InvariantHandle`

InvariantHandle names one invariant inside a basis.

`BasisAudit = coupling.BasisAudit`

BasisAudit stores compact evidence for one generated path basis.

`FormulaCoverageAudit = context.FormulaCoverageAudit`

FormulaCoverageAudit stores formula-readiness counters for a generated basis.

`ProjectorDescriptor = projector.ProjectorDescriptor`

ProjectorDescriptor exposes read-only metadata for a projector id.

`ProjectorRole = projector.ProjectorRole`

ProjectorRole records what mathematical object a projector selects.

`ProductChannelKey = projector.ProductChannelKey`
 ProductChannelKey identifies one target channel inside a binary product.

`ProjectorDerivation = projector.ProjectorDerivation`
 ProjectorDerivation stores the construction route and expansion state.

`ProjectorFormulaKind = projector.ProjectorFormulaKind`
 ProjectorFormulaKind records the executable local formula route.

`ProjectorFormulaAudit = projector.ProjectorFormulaAudit`
 ProjectorFormulaAudit records symbolic checks passed by a formula route.

`Context = context.Context`
 Context is an opaque handle to tensor-code stores and caches.

`ContextOptions = context.ContextOptions`
 ContextOptions configures algebra and rendering defaults.

`AlgebraConventions = root_data.AlgebraConventions`
 AlgebraConventions selects configurable exact-algebra normalizations.

`AlgebraSpec = store.AlgebraSpec`
 AlgebraSpec describes a requested algebra before it is interned.

`IrrepSpec = store.IrrepSpec`
 IrrepSpec describes a requested representation before it is interned.

`ExternalLeg = realization.ExternalLeg`
 ExternalLeg describes a public invariant leg with named free indices.

`RealizationSpec = realization.RealizationSpec`
 RealizationSpec describes a primitive or projected external index model.

`InvariantBasisRequest = coupling.InvariantBasisRequest`
 InvariantBasisRequest is the CFT-facing request for invariant tensors.

`TensorExpansionTerm = coupling.TensorExpansionTerm`
 TensorExpansionTerm stores a CFT coefficient attached to one invariant id.

`RenderOptions = rendering.RenderOptions`
 RenderOptions controls streamed invariant rendering.

`EvalOptions = rendering.EvalOptions`
 EvalOptions controls component evaluation of one invariant.

`SymbolicTerm = rendering.SymbolicTerm`
 SymbolicTerm stores one streamed coefficient times symbolic atoms.

`SymbolicAtom = rendering.SymbolicAtom`
 SymbolicAtom stores one symbolic invariant contraction atom.

`SymbolicAtomTag = rendering.SymbolicAtomTag`
 SymbolicAtomTag names one symbolic atom variant.

`AtomLoweringAudit = rendering.AtomLoweringAudit`
 AtomLoweringAudit records how far streamed atoms have been lowered.

`AtomLoweringAuditSink = rendering.AtomLoweringAuditSink`
 AtomLoweringAuditSink accumulates atom-lowering counters from terms.

`GammaMatrix = rendering.GammaMatrix`
 GammaMatrix stores one explicit spinor-vector Clifford atom.

`GammaForm = rendering.GammaForm`
 GammaForm stores one explicit spinor bilinear coupled to a form irrep.

`GammaAction = rendering.GammaAction`
 GammaAction stores an antisymmetric gamma form acting on a spinor.

`CliffordProduct = rendering.CliffordProduct`
 CliffordProduct stores one reduced gamma-product term.

`CartanProduct = rendering.CartanProduct`
 CartanProduct stores a compact highest-weight product projection.

`TensorSpinorProjection = rendering.TensorSpinorProjection`
 TensorSpinorProjection stores a compact form-valued spinor-tower channel.

`TensorFormProjection = rendering.TensorFormProjection`
 TensorFormProjection stores a compact tensor-spinor terminal channel.

`GammaTrace = rendering.GammaTrace`
 GammaTrace stores the gamma-trace part of a vector-spinor projector.

`SpinorPair = rendering.SpinorPair`
 SpinorPair stores one explicit invariant spinor bilinear atom.

`VectorSpinorIdentity = rendering.VectorSpinorIdentity`
 VectorSpinorIdentity stores the identity on a vector-spinor slot.

`StructureConstant = rendering.StructureConstant`
 StructureConstant stores one backend-specific invariant tensor atom.

`RationalValue = rendering.RationalValue`
 RationalValue is a decoded small exact rational coefficient.

`GammaBlock = rendering.GammaBlock`
 GammaBlock stores one internal antisymmetrized gamma factor.

`CliffordProductTerm = rendering.CliffordProductTerm`
 CliffordProductTerm stores one grade term in a gamma-block product.

`CliffordProductExpansion = rendering.CliffordProductExpansion`
 CliffordProductExpansion stores the small finite grade expansion.

`CliffordVectorSource = rendering.CliffordVectorSource`
 CliffordVectorSource identifies which input gamma block owns a vector.

`CliffordMetricContraction = rendering.CliffordMetricContraction`
 CliffordMetricContraction stores one metric between input gamma slots.

`CliffordFreeVector = rendering.CliffordFreeVector`
 CliffordFreeVector stores one uncontracted vector slot in output order.

`CliffordProductMetricFactor = rendering.CliffordProductMetricFactor`
 CliffordProductMetricFactor names one metric in a reduced gamma product.

`CliffordProductFreeVectorFactor = rendering.CliffordProductFreeVectorFactor`
 CliffordProductFreeVectorFactor maps one input slot into the output gamma.

`CliffordProductFactor = rendering.CliffordProductFactor`
 CliffordProductFactor stores one explicit factor of a reduced product atom.

`CliffordProductFactorRecord = rendering.CliffordProductFactorRecord`
 CliffordProductFactorRecord stores one streamed factor with product context.

`CliffordProductFactorScan = rendering.CliffordProductFactorScan`
 CliffordProductFactorScan summarizes and validates lowered reducer factors.

`CliffordProductFactorAuditSink = rendering.CliffordProductFactorAuditSink`
 CliffordProductFactorAuditSink accumulates factor scans from rendered terms.

`CliffordContractionPlan = rendering.CliffordContractionPlan`
 CliffordContractionPlan stores metric pairs and residual gamma slots.

`GammaChirality = rendering.GammaChirality`
 GammaChirality classifies chiral spinor conventions for gamma renderers.

`NamedOperatorKind = rendering.NamedOperatorKind`

NamedOperatorKind classifies compact backend-rendered operators.

`FilterDecision = rendering.FilterDecision`

FilterDecision is the tri-state result for streamed expansion filters.

`ExpansionFilter = rendering.ExpansionFilter`

ExpansionFilter decides whether streamed terms should continue or emit.

`ExpansionAudit = rendering.ExpansionAudit`

ExpansionAudit stores compact counters from streamed expansion.

`TensorExprId = kernel.TensorExprId`

TensorExprId names a tensor expression stored by tensor-code.

CONSTANTS

`gammaChiralityTag = rendering.gammaChiralityTag`

gammaChiralityTag returns a compact chirality tag for a spinor Dynkin label.

`orthogonalDimension = rendering.orthogonalDimension`

orthogonalDimension returns the vector dimension for B/D algebras.

`gammaDualityValid = rendering.gammaDualityValid`

gammaDualityValid checks whether a gamma block duality tag matches its grade.

`gammaProductExpansion = rendering.gammaProductExpansion`

gammaProductExpansion returns the grade expansion of two gamma blocks.

`gammaProductContractionPlan = rendering.gammaProductContractionPlan`

gammaProductContractionPlan returns metrics and free slots for one product term.

`cliffordProductTerm = rendering.cliffordProductTerm`

cliffordProductTerm decodes the grade term stored in a streamed atom.

`cliffordProductContractionPlan = rendering.cliffordProductContractionPlan`

cliffordProductContractionPlan reconstructs the metric/free-slot plan.

`cliffordProductFactorCount = rendering.cliffordProductFactorCount`

cliffordProductFactorCount returns the number of explicit factors in an atom.

`cliffordProductFactorAt = rendering.cliffordProductFactorAt`

cliffordProductFactorAt returns one metric or residual vector factor.

`streamCliffordProductFactors = rendering.streamCliffordProductFactors`

streamCliffordProductFactors emits factors from compact or lowered reducers.

`scanCliffordProductFactors = rendering.scanCliffordProductFactors`

scanCliffordProductFactors validates and counts compact or lowered factors.

`appendLoweredCliffordProductAtoms = rendering.appendLoweredCliffordProductAtoms`

appendLoweredCliffordProductAtoms replaces reducer atoms with factor atoms.

`rationalValue = rendering.rationalValue`

rationalValue decodes an exact rational coefficient.

This module handles finite-dimensional representation data for the symmetry algebras used by the other modules. For a Lie algebra $\mathfrak{g}$, the central problem is to construct bases of

$$\mathrm{Hom}_{\mathfrak{g}}(T, R_1 \otimes \cdots \otimes R_k),$$

usually with $T = \mathbf{1}$. These bases are Clebsch-Gordan data and invariant tensors. Concrete index notation, such as gamma matrices, Young symmetrizers, metrics, epsilons, or structure constants, is a rendering layer over this representation-theoretic basis.

The public module exposes named declarations directly. Source files such as Symmetry and Tensor Kernel organize implementation and prose, but clients should import this node and use

the individual public declarations below. The caller-facing object is `Context`: it registers algebras, irreps, and realizations, then returns compact handles for invariant bases and rendered/evaluated output streams.

```
const std = @import("std");
const context = @import("context.zig");
const coupling = @import("coupling.zig");
const kernel = @import("kernel.zig");
const projector = @import("projector.zig");
const realization = @import("realization.zig");
const rendering = @import("rendering.zig");
const root_data = @import("root-data.zig");
const store = @import("representation-store.zig");
const symmetry = @import("symmetry.zig");

/// SimpleLieAlgebra stores the simple family and rank.
pub const SimpleLieAlgebra = symmetry.SimpleLieAlgebra;
/// LieFamily names the supported simple Lie families.
pub const LieFamily = symmetry.LieFamily;
/// QuantumNumbers is the tensor-product list of representation ids.
pub const QuantumNumbers = symmetry.QuantumNumbers;
/// RepresentationId names a representation in a representation store.
pub const RepresentationId = symmetry.RepresentationId;
/// TensorStructure stores simple built-in tensor structures.
pub const TensorStructure = symmetry.TensorStructure;

/// AlgebraHandle names a registered Lie algebra in a tensor context.
pub const AlgebraHandle = store.AlgebraHandle;
/// IrrepHandle names an abstract irreducible representation.
pub const IrrepHandle = store.IrrepHandle;
/// RealizationHandle names a concrete index realization of an irrep.
pub const RealizationHandle = realization.RealizationHandle;
/// RealizationChannel selects one irrep copy inside a realization product.
pub const RealizationChannel = realization.RealizationChannel;
/// BasisHandle names a generated invariant basis.
pub const BasisHandle = coupling.BasisHandle;
/// InvariantHandle names one invariant inside a basis.
pub const InvariantHandle = coupling.InvariantHandle;
/// BasisAudit stores compact evidence for one generated path basis.
pub const BasisAudit = coupling.BasisAudit;
/// FormulaCoverageAudit stores formula-readiness counters for a generated basis.
pub const FormulaCoverageAudit = context.FormulaCoverageAudit;
/// ProjectorDescriptor exposes read-only metadata for a projector id.
pub const ProjectorDescriptor = projector.ProjectorDescriptor;
/// ProjectorRole records what mathematical object a projector selects.
pub const ProjectorRole = projector.ProjectorRole;
/// ProductChannelKey identifies one target channel inside a binary product.
pub const ProductChannelKey = projector.ProductChannelKey;
/// ProjectorDerivation stores the construction route and expansion state.
pub const ProjectorDerivation = projector.ProjectorDerivation;
/// ProjectorFormulaKind records the executable local formula route.
pub const ProjectorFormulaKind = projector.ProjectorFormulaKind;
/// ProjectorFormulaAudit records symbolic checks passed by a formula route.
pub const ProjectorFormulaAudit = projector.ProjectorFormulaAudit;
/// Context is an opaque handle to tensor-code stores and caches.
pub const Context = context.Context;
/// ContextOptions configures algebra and rendering defaults.
pub const ContextOptions = context.ContextOptions;
/// AlgebraConventions selects configurable exact-algebra normalizations.
pub const AlgebraConventions = root_data.AlgebraConventions;
/// AlgebraSpec describes a requested algebra before it is interned.
pub const AlgebraSpec = store.AlgebraSpec;
/// IrrepSpec describes a requested representation before it is interned.
pub const IrrepSpec = store.IrrepSpec;
```

```

/// ExternalLeg describes a public invariant leg with named free indices.
pub const ExternalLeg = realization.ExternalLeg;
/// RealizationSpec describes a primitive or projected external index model.
pub const RealizationSpec = realization.RealizationSpec;
/// InvariantBasisRequest is the CFT-facing request for invariant tensors.
pub const InvariantBasisRequest = coupling.InvariantBasisRequest;
/// TensorExpansionTerm stores a CFT coefficient attached to one invariant id.
pub const TensorExpansionTerm = coupling.TensorExpansionTerm;
/// RenderOptions controls streamed invariant rendering.
pub const RenderOptions = rendering.RenderOptions;
/// EvalOptions controls component evaluation of one invariant.
pub const EvalOptions = rendering.EvalOptions;
/// SymbolicTerm stores one streamed coefficient times symbolic atoms.
pub const SymbolicTerm = rendering.SymbolicTerm;
/// SymbolicAtom stores one symbolic invariant contraction atom.
pub const SymbolicAtom = rendering.SymbolicAtom;
/// SymbolicAtomTag names one symbolic atom variant.
pub const SymbolicAtomTag = rendering.SymbolicAtomTag;
/// AtomLoweringAudit records how far streamed atoms have been lowered.
pub const AtomLoweringAudit = rendering.AtomLoweringAudit;
/// AtomLoweringAuditSink accumulates atom-lowering counters from terms.
pub const AtomLoweringAuditSink = rendering.AtomLoweringAuditSink;
/// GammaMatrix stores one explicit spinor-vector Clifford atom.
pub const GammaMatrix = rendering.GammaMatrix;
/// GammaForm stores one explicit spinor bilinear coupled to a form irrep.
pub const GammaForm = rendering.GammaForm;
/// GammaAction stores an antisymmetric gamma form acting on a spinor.
pub const GammaAction = rendering.GammaAction;
/// CliffordProduct stores one reduced gamma-product term.
pub const CliffordProduct = rendering.CliffordProduct;
/// CartanProduct stores a compact highest-weight product projection.
pub const CartanProduct = rendering.CartanProduct;
/// TensorSpinorProjection stores a compact form-valued spinor-tower channel.
pub const TensorSpinorProjection = rendering.TensorSpinorProjection;
/// TensorFormProjection stores a compact tensor-spinor terminal channel.
pub const TensorFormProjection = rendering.TensorFormProjection;
/// GammaTrace stores the gamma-trace part of a vector-spinor projector.
pub const GammaTrace = rendering.GammaTrace;
/// SpinorPair stores one explicit invariant spinor bilinear atom.
pub const SpinorPair = rendering.SpinorPair;
/// VectorSpinorIdentity stores the identity on a vector-spinor slot.
pub const VectorSpinorIdentity = rendering.VectorSpinorIdentity;
/// StructureConstant stores one backend-specific invariant tensor atom.
pub const StructureConstant = rendering.StructureConstant;
/// RationalValue is a decoded small exact rational coefficient.
pub const RationalValue = rendering.RationalValue;
/// GammaBlock stores one internal antisymmetrized gamma factor.
pub const GammaBlock = rendering.GammaBlock;
/// CliffordProductTerm stores one grade term in a gamma-block product.
pub const CliffordProductTerm = rendering.CliffordProductTerm;
/// CliffordProductExpansion stores the small finite grade expansion.
pub const CliffordProductExpansion = rendering.CliffordProductExpansion;
/// CliffordVectorSource identifies which input gamma block owns a vector.
pub const CliffordVectorSource = rendering.CliffordVectorSource;
/// CliffordMetricContraction stores one metric between input gamma slots.
pub const CliffordMetricContraction = rendering.CliffordMetricContraction;
/// CliffordFreeVector stores one uncontracted vector slot in output order.
pub const CliffordFreeVector = rendering.CliffordFreeVector;
/// CliffordProductMetricFactor names one metric in a reduced gamma product.
pub const CliffordProductMetricFactor = rendering.CliffordProductMetricFactor;
/// CliffordProductFreeVectorFactor maps one input slot into the output gamma.
pub const CliffordProductFreeVectorFactor = rendering.CliffordProductFreeVectorFactor;
/// CliffordProductFactor stores one explicit factor of a reduced product atom.
pub const CliffordProductFactor = rendering.CliffordProductFactor;

```

```

/// CliffordProductFactorRecord stores one streamed factor with product context.
pub const CliffordProductFactorRecord = rendering.CliffordProductFactorRecord;
/// CliffordProductFactorScan summarizes and validates lowered reducer factors.
pub const CliffordProductFactorScan = rendering.CliffordProductFactorScan;
/// CliffordProductFactorAuditSink accumulates factor scans from rendered terms.
pub const CliffordProductFactorAuditSink = rendering.CliffordProductFactorAuditSink;
/// CliffordContractionPlan stores metric pairs and residual gamma slots.
pub const CliffordContractionPlan = rendering.CliffordContractionPlan;
/// GammaChirality classifies chiral spinor conventions for gamma renderers.
pub const GammaChirality = rendering.GammaChirality;
/// NamedOperatorKind classifies compact backend-rendered operators.
pub const NamedOperatorKind = rendering.NamedOperatorKind;
/// FilterDecision is the tri-state result for streamed expansion filters.
pub const FilterDecision = rendering.FilterDecision;
/// ExpansionFilter decides whether streamed terms should continue or emit.
pub const ExpansionFilter = rendering.ExpansionFilter;
/// ExpansionAudit stores compact counters from streamed expansion.
pub const ExpansionAudit = rendering.ExpansionAudit;

/// TensorExprId names a tensor expression stored by tensor-code.
pub const TensorExprId = kernel.TensorExprId;

/// gammaChiralityTag returns a compact chirality tag for a spinor Dynkin label.
pub const gammaChiralityTag = rendering.gammaChiralityTag;
/// orthogonalDimension returns the vector dimension for B/D algebras.
pub const orthogonalDimension = rendering.orthogonalDimension;
/// gammaDualityValid checks whether a gamma block duality tag matches its grade.
pub const gammaDualityValid = rendering.gammaDualityValid;
/// gammaProductExpansion returns the grade expansion of two gamma blocks.
pub const gammaProductExpansion = rendering.gammaProductExpansion;
/// gammaProductContractionPlan returns metrics and free slots for one product term.
pub const gammaProductContractionPlan = rendering.gammaProductContractionPlan;
/// cliffordProductTerm decodes the grade term stored in a streamed atom.
pub const cliffordProductTerm = rendering.cliffordProductTerm;
/// cliffordProductContractionPlan reconstructs the metric/free-slot plan.
pub const cliffordProductContractionPlan = rendering.cliffordProductContractionPlan;
/// cliffordProductFactorCount returns the number of explicit factors in an atom.
pub const cliffordProductFactorCount = rendering.cliffordProductFactorCount;
/// cliffordProductFactorAt returns one metric or residual vector factor.
pub const cliffordProductFactorAt = rendering.cliffordProductFactorAt;
/// streamCliffordProductFactors emits factors from compact or lowered reducers.
pub const streamCliffordProductFactors = rendering.streamCliffordProductFactors;
/// scanCliffordProductFactors validates and counts compact or lowered factors.
pub const scanCliffordProductFactors = rendering.scanCliffordProductFactors;
/// appendLoweredCliffordProductAtoms replaces reducer atoms with factor atoms.
pub const appendLoweredCliffordProductAtoms = rendering.appendLoweredCliffordProductAtoms;
/// rationalValue decodes an exact rational coefficient.
pub const rationalValue = rendering.rationalValue;

const RootCountingSink = struct {
    term_count: u32 = 0,
    epsilon_count: u32 = 0,
    structure_constant_count: u32 = 0,
    projector_operator_count: u32 = 0,

    pub fn emitTerm(self: *RootCountingSink, term: SymbolicTerm) !void {
        self.term_count += 1;
        for (term.atoms) |atom| {
            switch (atom) {
                .epsilon => self.epsilon_count += 1,
                .structure_constant => self.structure_constant_count += 1,
                .named_operator => |operator| {

```

```

        if (operator.kind == .projector) self.projector_operator_count += 1;
    },
    else => {},
}
}
}
};

```

1 Symmetry

INTERFACE

TYPES

SimpleLieAlgebra (struct)

SimpleLieAlgebra stores the simple family and rank.

LieFamily (enum)

LieFamily names the supported simple Lie families.

QuantumNumbers = []const RepresentationId

QuantumNumbers is the tensor-product list of representation ids.

SymmetryId = u8

SymmetryId names a symmetry in a symmetry store.

RepresentationId = u8

RepresentationId names a representation in a representation store.

U1ChargeRational (struct)

U1ChargeRational stores a rational U(1) charge.

TensorIndexId = u32

TensorIndexId names a tensor index in a tensor expression.

TensorStructure (union)

TensorStructure stores simple built-in tensor structures.

A symmetry is labeled by its name and a Lie Algebra, which can be u(1) or a simple Lie algebra.

```

/// Symmetry labels a named symmetry algebra.
const Symmetry = struct {
    name: []const u8,
    algebra: LieAlgebra,
};

/// LieAlgebra distinguishes U(1) from simple Lie algebras.
const LieAlgebra = union(enum) {
    u1,
    simple: SimpleLieAlgebra,
};

/// SimpleLieAlgebra stores the simple family and rank.
pub const SimpleLieAlgebra = struct {
    family: LieFamily,
    rank: u8,
};

/// LieFamily names the supported simple Lie families.
pub const LieFamily = enum {
    a,
    b,
    c,

```

```

    d,
    e6,
    e7,
    e8,
    f4,
    g2,
};

```

A quantum number is labeled by the symmetry algebra and its representation.

```

/// QuantumNumbers is the tensor-product list of representation ids.
pub const QuantumNumbers = []const RepresentationId;

/// SymmetryId names a symmetry in a symmetry store.
pub const SymmetryId = u8;
/// RepresentationId names a representation in a representation store.
pub const RepresentationId = u8;

/// Representation stores a symmetry id and its label.
const Representation = struct {
    symmetry: SymmetryId,
    label: RepresentationLabel,
};

/// RepresentationLabel stores either a rational charge or Dynkin labels.
const RepresentationLabel = union(enum) {
    u1_charge: U1ChargeRational,
    dynkin: []const i8,
};

/// U1ChargeRational stores a rational U(1) charge.
pub const U1ChargeRational = struct {
    numerator: i8,
    denominator: i8,
};

```

An abstract tensor index carries a label and a particular representation.

```

/// TensorIndex stores a representation id and concrete index label.
const TensorIndex = struct {
    representation: RepresentationId,
    label: u32,
};

/// TensorIndexId names a tensor index in a tensor expression.
pub const TensorIndexId = u32;

```

An example of a concrete tensor structures - should only appear for SO-type groups, todo.

```

/// TensorStructure stores simple built-in tensor structures.
pub const TensorStructure = union(enum) {
    none,
    kronecker_delta: struct {
        left: TensorIndexId,
        right: TensorIndexId,
    },
};

```

2 Tensor Kernel

Tensor-code supplies compact handles for tensor structures that appear in CFT coefficients. The expensive work is generation and selection of invariant tensors, so those procedures should stream

candidates rather than building a large expression tree inside the kernel.

An invariant source represents basis elements in

$$\text{Hom}_{\mathfrak{g}}(T, \otimes_i R_i)$$

by handles. Rendering into a concrete notation is backend-dependent: A_n may render through Young symmetrizers and ϵ/δ tensors, D_n through metric, epsilon, and spinorial gamma data, and exceptional algebras through their available primitive invariant tensors.

```
const coupling = @import("coupling.zig");
const realization = @import("realization.zig");
const store = @import("representation-store.zig");
const symmetry = @import("symmetry.zig");
```

INTERFACE

TYPES

TensorExprId = u32

TensorExprId names a tensor expression stored by tensor-code.

The basic ids are deliberately small. They name entries in stores owned by a preset, a tensor generator, or a client.

```
/// TensorExprId names a tensor expression stored by tensor-code.
pub const TensorExprId = u32;
/// TensorTermId names one term inside a tensor expression store.
const TensorTermId = u32;
/// BasisPathId names one compact coupling path in an invariant basis.
const BasisPathId = u32;
/// RenderedTermId names one streamed symbolic tensor term.
const RenderedTermId = u32;
/// ReductionTermId names one coefficient in a target invariant basis.
const ReductionTermId = u32;
/// InvariantFamilyId names a family of invariant tensor candidates.
const InvariantFamilyId = u16;
/// AlgebraSchemaId names a Lie or superalgebra schema.
const AlgebraSchemaId = u16;
/// GradingId names a grading attached to a tensor algebra schema.
const GradingId = u16;
```

Invariant generation should be pull-based. A caller can feed the source to an independent-basis selector without materializing every candidate first.

```
/// TensorExprSource streams tensor expression ids from a generator.
const TensorExprSource = struct {
    next_fn: *const fn (*anyopaque) ?TensorExprId,
    state: *anyopaque,

    /// next returns the next generated tensor expression id.
    pub fn next(self: *TensorExprSource) ?TensorExprId {
        return self.next_fn(self.state);
    }
};
```

Basis paths and rendered terms are streamed separately. This lets CFT code attach invariant ids to rational functions without asking tensor-code to expand concrete tensor expressions during ordinary simplification.

```

/// BasisPathSource streams invariant ids from a generated basis.
const BasisPathSource = struct {
    next_fn: *const fn (*anyopaque) ?coupling.InvariantHandle,
    state: *anyopaque,

    /// next returns the next invariant in the basis.
    pub fn next(self: *BasisPathSource) ?coupling.InvariantHandle {
        return self.next_fn(self.state);
    }
};

/// RenderedTermSource streams symbolic terms for one invariant rendering.
const RenderedTermSource = struct {
    next_fn: *const fn (*anyopaque) ?RenderedTermId,
    state: *anyopaque,

    /// next returns the next rendered symbolic term.
    pub fn next(self: *RenderedTermSource) ?RenderedTermId {
        return self.next_fn(self.state);
    }
};

/// ReductionTermSource streams coefficients in a chosen invariant basis.
const ReductionTermSource = struct {
    next_fn: *const fn (*anyopaque) ?ReductionTermId,
    state: *anyopaque,

    /// next returns the next reduction coefficient term.
    pub fn next(self: *ReductionTermSource) ?ReductionTermId {
        return self.next_fn(self.state);
    }
};

```

An invariant family says which algebra and external representations define the candidate space. The actual candidates are produced by a generator.

```

/// InvariantFamily declares the tensor-invariant problem to generate.
const InvariantFamily = struct {
    name: []const u8,
    algebra: AlgebraSchemaId,
    external_representations: []const symmetry.RepresentationId,
};

```

The new invariant request keeps external leg names and realizations. The old family type remains for compatibility with early kernel sketches.

```

/// InvariantRequest records a named public invariant-basis request.
const InvariantRequest = struct {
    name: []const u8,
    algebra: store.AlgebraHandle,
    external_legs: []const realization.ExternalLeg,
    target: coupling.TargetIrrep = .singlet,
    tree_policy: coupling.TreePolicy = .auto,
    basis_policy: coupling.BasisPolicy = .default,
};

```

The algebra schema stores only compact references to core tensors such as a metric or structure constants. This is enough for CFT coefficient code to refer to a symmetry algebra without owning its full tensor simplifier.

```

/// AlgebraSchema records the compact tensor data of a symmetry algebra.
const AlgebraSchema = struct {
    name: []const u8,

```

```

symmetry: symmetry.SymmetryId,
grading: ?GradingId = null,
metric: ?TensorExprId = null,
structure_constants: ?TensorExprId = null,
};

```

3 Clebsch-Gordan

This node is the handoff for implementing the tensor-code backend that will eventually generate Clebsch-Gordan data and invariant tensor bases. The first backend milestone is not explicit CG coefficients. It is a reliable, generic, cached decomposition engine:

$$R \otimes S = \bigoplus_T m_T T.$$

The CG generator will consume these decompositions to enumerate coupling paths to a target representation, usually the singlet **1**.

3.1 Goals

The implementation must follow AGENTS.md:

- write simple explicit procedural Zig;
- design lean data structures first;
- avoid large intermediate symbolic expressions;
- keep data flows parallelizable;
- expose the bare minimum public API;
- keep the CFT-facing API in Tensor-Code;
- keep internal backend/decomposition/projector details out of the public root unless a caller actually needs them;
- document the implementation in org-roam nodes and keep prose concise.

The mathematical and product requirements are:

- support A_n for $SU(N)$;
- support B_n, D_n for $SO(N)$ and $Spin(N)$;
- support E_6, E_7, E_8 , with E_8 a key target;
- do not overfocus the architecture on $SO(10)$, even though $Spin(10)$ is the main stress test;
- identify irreps by Dynkin labels, not dimensions;
- treat multiplicities as first-class channel copies;
- distinguish abstract irreps from concrete index realizations;
- support constrained realizations such as $144 \subset 10 \otimes 16$ without carrying the full ambient tensor product through every internal step;
- represent large invariant bases as compact coupling paths, not dense tensor components;

- render explicit gamma/projector formulas only on demand and by streaming terms to a sink;
- keep named projector paths distinct from fully expanded projector formulas.

The goals are concrete:

1. Abstract representation theory must be correct first. Irreps are Dynkin labels; decompositions have explicit multiplicities; channels are not identified by dimension.
2. Large invariant bases must be stored as compact coupling paths. A basis element is a recipe: external legs, intermediate channels, and local projector/CG kernel ids.
3. Projectors in returned results must not be anonymous tokens. A projector id may be the compact storage form, but it must carry role, convention, and derivation metadata. Until a formula renderer exists, expanded rendering must fail with a typed missing-formula error.
4. Expansion must be streamed. Fully expandable does not mean eagerly expanded. It means the renderer can enumerate the terms without inventing missing identities or asking the user for hidden Fierz relations.
5. The runnable target is not merely obtaining 73744 abstract basis expressions. The runnable target is also being able to expand each selected basis element through a caller-supplied filter, and for the resulting stream to be consumed at high throughput by CFT/correlator code.
6. Filters are part of the execution model. They may discard terms by index support, operator kind, realization leg, component assignment, symmetry, coefficient class, or target downstream contraction. The renderer must apply filters while streaming, not after materializing the full expansion. Partial filtering must be sound: a partial product may be rejected only when no completion can survive. Otherwise the filter must return **undecided** and the expansion continues.
7. Coupling-tree policy is part of the basis convention. Build one canonical tree first, and certify independence for that tree. Multiple trees are different bases unless a basis-change or reduction map is explicitly constructed.
8. Fierz and projector identities must be derived from the backend construction: channel decomposition, idempotent splitting, highest-weight solvers, Casimir separation where valid, and explicit residual linear algebra only when necessary. They are not user-supplied tables.
9. Compute estimates must include projector expansion cost. Counting coupling paths alone is insufficient if each path contains a projector whose expansion is large.

The motivating stress target remains:

$$\dim \text{Hom}_{\text{Spin}(10)}(\mathbf{1}, 16^{12}) = 73744.$$

Those 73744 objects should be records describing coupling paths and local projector ids. They should not be expanded into component tensors during basis enumeration. However, every projector id in those paths must be traceable to a stored expansion procedure. When the caller asks for concrete notation, rendering should stream terms built from metrics, epsilons, named operators, gamma-like backend operators, or projector blocks. The stream must be filterable as it is produced. A successful implementation is one where a downstream consumer can iterate through selected expanded terms without waiting for the full expansion of all 73744 invariants to be allocated.

3.2 Non-Solutions

The following are explicitly not acceptable:

- Returning only abstract representation-theory counts. Counts are useful for feasibility, but CFT code needs actual invariant handles and eventually streamable tensor expressions.
- Returning opaque named projectors with no derivation metadata. A compact projector id is acceptable as an intermediate named form only if expanded rendering remains a typed missing-formula error until a real formula route is implemented.
- Returning compact coupling paths with no efficient expansion path. A list of 73744 abstract expressions is not enough.
- Expanding all 73744 basis elements into memory before filtering. Filters must be pushed into the streaming expansion loop.
- Treating rendering as a slow debug printer. Rendering is part of the runnable computational path and must feed downstream correlator code at a fast pace.
- Treating $144 \subset 10 \otimes 16$ as $10 \otimes 16$ throughout all internal coupling steps and projecting only at the very end. The realization projector belongs at the boundary.
- Relying on hand-entered Fierz identities. Identities must follow from the algebraic construction and verified projector/idempotent relations.
- Building dense tensors or dense component spaces such as 16^{12} .
- Designing an $SO(10)$ -only system. $Spin(10)$ is a stress test, not the architecture.
- Making gamma matrices part of the generic public ADE API. Gamma-like objects are backend-rendered named operators or internal $SO/Spin$ atoms.
- Optimizing by hiding mathematical ambiguity. Multiplicity copies, realization channels, basis conventions, and signature/rendering conventions must be explicit.
- Rejecting a partial expanded term because the current prefix does not yet match a filter. Prefix filters must distinguish **reject** from **undecided**.
- Concatenating paths from several coupling trees and calling the union a larger independent basis. This is allowed only after constructing and applying a basis-change or reduction map.

3.3 Weasel Modes And Required Evidence

An implementation is not accepted because it can print plausible data. It must produce the required records and expose enough evidence that the records are real. The following failure modes are common and must be rejected during review:

- Claiming a "fast structure generator" that only returns the count of invariants. Required evidence: stored basis handles with inspectable path records, one record per basis element.
- Returning a list of abstract labels without local channel data. Required evidence: each path stores the coupling tree, intermediate channel handles, multiplicity-copy ids, and local projector/CG kernel ids.

- Returning projectors as names such as `P_144` with no expansion route. Required evidence: every projector id reports its derivation kind and an expansion source, or returns a typed `NotExpandable` error before it appears in a returned basis.
- Using an overcomplete gamma-monomial list and calling it a basis. Required evidence: basis construction happens in representation/multiplicity space before rendering; gamma terms are a rendering of already-independent basis elements.
- Running a dense component-space linear solve and hiding the cost behind a small output. Required evidence: logs/counters for maximum allocated weight multiset size, path count, projector count, and no allocations proportional to 16^{12} or another full tensor component space.
- Filtering after materializing all expanded terms. Required evidence: filters are called during expansion, and a reject-all filter allocates no large output buffer.
- Failing to distinguish multiplicity copies. Required evidence: decomposition terms include multiplicity, and local kernels/projectors carry the selected copy.
- Treating projected realizations as ambient products until the final output. Required evidence: external legs using constrained realizations carry a boundary projector once, and internal coupling uses the target irrep.
- Claiming independence from successful numerical spot checks only. Required evidence: independence follows from the decomposition path construction, and numerical/rendered checks are secondary audits.
- Hardcoding known decompositions for the acceptance tests. Required evidence: tests include at least one generated decomposition not present as a literal special case, and the backend exposes counters showing which generic procedure produced it.

Every major stage must produce a compact audit record:

- algebra and convention ids;
- input irreps;
- number of weights generated;
- total weight multiplicity;
- product multiset size;
- decomposition channel count;
- sum of channel dimensions times multiplicities;
- number of basis paths;
- number of distinct projector ids;
- maximum known projector expansion size, if available;
- filter accept/reject/undecided counts during streamed expansion.

These audits are mandatory internal milestone evidence, not optional debug output. They are not user-facing API by default, but each backend milestone must produce enough audit data to prove that it is doing the promised computation.

3.4 Independence By Construction

The basis must be independent before rendering. The construction should be:

1. For each binary product $R \otimes S$, decompose into irreps:

$$R \otimes S = \bigoplus_T M_{RS}^T \otimes T.$$

Here M_{RS}^T is the finite multiplicity space. If $m_T = \dim M_{RS}^T$, choose a basis vector e_i for each copy $i = 0, \dots, m_T - 1$.

2. A local CG kernel is an intertwiner selecting one basis vector in M_{RS}^T . Distinct multiplicity-copy ids are distinct basis vectors in that multiplicity space.
3. For a fixed canonical coupling tree, a global path is a product of local choices: at every internal node, choose the output channel and multiplicity-copy basis vector.
4. The set of all paths ending in the requested target T , usually $\mathbf{1}$, is a basis of the multiplicity space

$$\text{Hom}_{\mathfrak{g}}(T, R_1 \otimes \dots \otimes R_n)$$

for that tree and basis convention.

Independence follows for that fixed tree because the decomposition is a direct sum of irreducible channels and explicit multiplicity spaces. We are not selecting linearly independent gamma expressions from an overcomplete list. We are selecting basis vectors in multiplicity spaces and composing intertwiners along one tree convention.

Different tree policies give different coordinate systems on the same multiplicity space. They must not be concatenated. If the implementation later supports several trees, each tree must separately produce the target multiplicity. The combined count across trees is not an independence certificate; it is redundant data until a basis-change or reduction map is available.

Rendered formulas may look dependent because gamma matrices, epsilons, metrics, and projector blocks satisfy identities. Those identities are part of the rendering backend. They must not be used to define the abstract basis. Rendering must be a faithful expansion of the already-selected path basis in a chosen convention. If a renderer cannot prove or verify faithfulness for a convention, it must report a typed error or mark the expansion as an audit target; it must not silently collapse or add basis elements.

Required independence checks:

- The number of emitted basis paths equals the target multiplicity computed by dynamic programming over decompositions.
- For every binary decomposition, the sum

$$\sum_T m_T \dim T$$

equals $\dim R \dim S$.

- Local projector records for a product channel must satisfy the abstract projector laws in their construction record: idempotency and channel separation. When explicit formulas exist, audits should verify $P_i P_j = \delta_{ij} P_i$ in the chosen representation of the local product.
- Boundary realization projectors must satisfy the same rule: e.g. $144 \subset 10 \otimes 16$ is represented by an idempotent whose image has dimension 144, and the complementary channel is accounted for.

- Rendered term streams are audits of the basis, not the source of the basis. Component evaluation or filtered numerical checks can catch renderer bugs but cannot replace the multiplicity-space construction.

3.5 Implementation Instructions

Proceed in this order. Do not skip ahead to rendered gamma formulas or high-level invariant counts.

1. Implement the backend registry privately in **Context**. It must select a backend by algebra family and return typed unsupported-capability errors.
2. Implement flat weight storage and a weight-system cache. Store signed weight coordinates in one buffer and weight records as offsets. Add audit counters for weight count and total multiplicity.
3. Implement the generic highest-weight weight-system generator. Validate it on known dimensions before any tensor-product code is accepted.
4. Implement tensor-product character multiplication and subtraction. The result must be flat product terms with multiplicities. Store the result in Tensor Decomposition.
5. Add decomposition acceptance tests: $SU(2) : 2 \otimes 2 = 1 \oplus 3$, $Spin(10) : 10 \otimes 16 = 16 \oplus 144$, $Spin(10) : 16 \otimes 16 = 10 \oplus 120 \oplus 126$, and $E_8 : 248 \otimes 248$ with the known channels.
6. Implement dynamic-programming path counting to a target using cached binary decompositions. This is still counting, but it must use the same channel records that path enumeration will use.
7. Implement path enumeration for one canonical coupling tree. Each emitted path must store its local channel choices and multiplicity-copy ids. The path count must equal the dynamic count. Do not enumerate several trees until basis-change or reduction maps exist. The public request path must at least verify that $R \otimes R^*$ contains one singlet for representative A , D , E_6 , E_7 , and E_8 irreps. It must also verify representative multi-leg invariants such as $A_2 : 3^3$, $D_5 : 10^2$, $E_6 : 27^3$, and $E_8 : 248^3$.
8. Implement projector identity creation. Each projector id must record its role, convention, derivation kind, and expansion obligation.
9. Implement a minimal expansion interface that can walk one path and call a sink with streamed symbolic terms. Named projector chains may be emitted as compact symbolic atoms; expanded projector requests must return typed **NotImplemented** for missing local formulas rather than successful formula placeholders.
10. Implement filters inside the expansion loop. The filter decision is tri-state: **accept**, **reject**, or **undecided**. A partial term can be rejected only when rejection is final under all completions. Prove with a reject-all filter test that no large output buffer is allocated.
11. Only after the generic path and projector infrastructure is correct, add specialized SO/Spin gamma/projector renderers, SU tableau/Gelfand-Tsetlin optimizations, or E-type special renderers.

At every step, prefer the smallest working data structure. Add a new abstraction only when it removes real duplication or separates a hot data path. Do not add public exports for backend internals unless CFT code has a concrete call site that needs them.

3.6 Current State

The public API is rooted at `Tensor-Code`. Public callers create a context, register an algebra, register irreps, optionally register concrete realizations, and request an invariant basis:

```
var ctx = try tensor.Context.init(allocator);
defer ctx.deinit();

const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
const leg = tensor.ExternalLeg.primitive(spinor, &.{
    .init(.spinor, "a"),
});
const basis = try ctx.invariantBasis(.{
    .algebra = so10,
    .external_legs = &.{leg},
});
```

Conventions are configurable at context construction:

```
var ctx = try tensor.Context.initWithOptions(allocator, .{
    .algebra_conventions = .{
        .quadratic_casimir_scale = .{ .numerator = 1, .denominator = 1 },
    },
    .default_signature = .complex,
});
defer ctx.deinit();
```

The current substrate is:

- Tensor Root Data validates supported simple algebras, stores Cartan data, computes positive roots, dual Dynkin labels, Weyl dimensions, and quadratic Casimirs.
- Tensor Representation Store interns algebras and irreps. It computes metadata from Dynkin labels and provides explicit `dualIrrep` lookup without recursively closing registration under duals.
- Tensor Realization records concrete external index shapes. Projected realizations store a selected channel, so a constrained leg such as $144 \subset 10 \otimes 16$ is a first-class boundary realization.
- Tensor Decomposition owns the binary product decomposition cache. It stores flat product terms and has a hash index keyed by the pair of irrep handles.
- Tensor Projector owns projector identities: product-channel projectors, boundary-realization projectors, and basis-change projectors.
- Tensor Backend defines the internal backend capability contract.
- Tensor Rendering defines streamed symbolic terms. `SymbolicTerm` carries borrowed slices valid only until the sink returns. The public root exposes the compact explicit atom payloads needed by current SO/Spin renderers. Named operators carry a compact kind tag, so filters can distinguish projector, gamma, metric, epsilon, structure-constant, and custom backend operators without knowing their family-specific formula payload. Projector operator ids can be resolved through the context to read-only descriptors containing their role, convention, and derivation metadata. A built-in operator-kind filter can reject terms containing a selected operator kind before they reach the sink; explicit `gamma_matrix` and `gamma_form` atoms are treated as gamma-kind contractions, while `metric_pair` and `spinor_pair` atoms are treated as metric-kind contractions.

Build-checked metadata cases currently include:

- B_4 : vector 9, spinor 16;
- D_5 : vector 10, spinor 16, vector-spinor 144;
- E_6 : fundamental 27;
- E_7 : fundamental 56;
- E_8 : adjoint 248.

The implemented backend path now includes:

- generic highest-weight weight systems for A, B, D, E_6, E_7, E_8 ;
- cached binary product decompositions;
- dynamic path counting and canonical path enumeration;
- public basis count and compact path audits;
- named projector-chain rendering through a streamed filter boundary;
- projected realization boundary projectors emitted once at the external leg boundary, so constrained realizations such as $144 \subset 10 \otimes 16$ do not leak their ambient product into internal coupling steps;
- an orthogonal vector-square singlet renderer that emits a streamed metric atom for B_n, D_n vector pairs;
- an orthogonal spinor-pair singlet renderer that emits a streamed `spinor_pair` atom for B_n, D_n spinor bilinears. The atom records the source product-channel projector, orthogonal dimension, and chirality tags;
- a basic D_5 spinor-spinor-vector singlet renderer that emits a streamed `gamma_matrix` atom for $16 \otimes 16 \rightarrow 10$. The atom carries the source projector id so callers can resolve the $16 \otimes 16 \rightarrow 10$ channel descriptor through the context, and stores the orthogonal dimension, gamma rank, chirality, and duality tags needed by Clifford-aware consumers. The two D_5 chiral spinors 16 and $16'$ receive distinct chirality tags through rendering-level metadata helpers;
- a D_5 spinor-spinor-form renderer that emits a streamed `gamma_form` atom for the $16 \otimes 16 \rightarrow 120, 126$ and $16' \otimes 16' \rightarrow 120, 126'$ channels. The external form leg is the dual channel when the middle form is complex, so the path remains the abstract multiplicity-space path while the rendered atom records rank, chirality, and self-duality metadata.

The following are still intentionally not implemented:

- general local CG/projector formula construction beyond the current orthogonal metric, spinor-pair, and gamma primitives;
- explicit boundary-realization projector formulas beyond compact named boundary atoms;
- full Clifford/gamma algebra expansion beyond the current rank-1, rank-3, and middle-form gamma atoms;
- non-special projector term expansion execution;
- formula-level projector expansion caches;
- `Context.evaluateInvariant`.

3.7 Current Handoff Toward Formula Projectors

The current code can find the abstract invariant basis for the target benchmark. In particular, the $Spin(10)$ 16^{12} path basis is generated as 73744 compact coupling paths, not as dense 16^{12} components. The remaining work is not another counting backend. The remaining work is to make projector ids act on streamed symbolic terms, so selected basis elements can be expanded through gamma, metric, spinor-pair, epsilon, and projector formulas without materializing a full expression tree.

$Spin(10)$ and 16^{12} are stress tests and acceptance examples, not the scope boundary. The solver must remain a general representation-theory engine for arbitrary products of registered A, D, E irreps, with B, D SO/Spin formula renderers isolated as backend-specific expansion routes. Do not let gamma-matrix needs distort the generic ADE decomposition, path enumeration, or projector identity model.

3.7.1 Plain-language plan

The plan is to keep the solver general first. Irreps are Dynkin labels, products are decomposed by the generic backend, and invariant bases are compact coupling paths. $Spin(10)$ and $16^{12} = 73744$ are stress tests for this pipeline, not special cases that define the architecture.

The implemented part already finds invariant bases without dense tensors:

- generic decomposition works for A, B, D, E_6, E_7, E_8 ;
- invariant bases are counted and stored as compact paths;
- $Spin(10)$ 16^{12} produces 73744 paths;
- every local channel has a projector id with metadata;
- named projector chains can be streamed through filters;
- basic SO/Spin atoms can already be emitted for metrics, spinor pairings, and low-rank gamma/form couplings.

The remaining work is projector formula execution. A projector id must become a small streamed procedure, not a name and not a large expression tree. The procedure acts on the current term frontier, emits a few output terms, applies filters early, and reuses scratch buffers. This follows the project rules: simple procedural code, lean data structures, no large intermediate symbolic expressions, and parallelizable data flow.

The next concrete target is the $144 \subset 10 \otimes 16$ boundary projector. It is the gamma-traceless vector-spinor projector:

$$10 \otimes 16 = 16' \oplus 144.$$

It should stream the identity vector-spinor term minus the normalized gamma-trace term. The checks are symbolic projector laws: gamma-tracelessness, idempotency, and separation from the $16'$ channel. These checks must not use dense $10 \cdot 16$ component matrices.

After that, the same formula interface should handle product-channel projectors built from spinor bilinears and gamma/form atoms. This requires a Clifford reducer for gamma products, metric contractions, chirality rules, and middle-form duality. The reducer is an SO/Spin backend formula tool; it must not change the generic ADE basis construction.

To keep the design honest, add a non-SO formula route after the first SO/Spin formulas work, for example a small A_2 or E_6 invariant. This ensures the formula layer remains a general projector-expansion interface rather than a hidden $Spin(10)$ -only subsystem.

The current renderer has three different levels:

- compact names: a product-channel or boundary-realization projector is emitted as a `named_operator` atom with a resolvable projector id;
- primitive formulas: special low-rank orthogonal channels emit explicit `metric_pair`, `spinor_pair`, `gamma_matrix`, or `gamma_form` atoms;
- projector formulas: the $144 \subset 10 \otimes 16$ boundary formula is implemented as a two-term stream. Other expanded projector requests must still fail with a typed error unless a filter rejects the term before expansion is needed.

The next implementation goal is therefore precise:

1. every projector id that can appear in a returned basis must resolve to a formula route or a typed missing-formula state;
2. formula routes must be executable as streaming term transformations;
3. the output of a local formula may include gamma atoms, and later projectors must be able to act on those atoms without first expanding the entire invariant into components;
4. each formula route must carry enough audit data to verify idempotency, channel separation, and normalization in the chosen convention.

3.7.2 Current verified pieces

The following pieces are already implemented and build-checked:

- ADE/B/D generic highest-weight decomposition for A, B, D, E_6, E_7, E_8 ;
- cached flat weight systems and cached binary product decompositions;
- path counting and canonical path enumeration through the same product records;
- $Spin(10) : 16^{12}$ benchmark count 73744;
- local product projector ids for every coupling step;
- projected external realizations, including $144 \subset 10 \otimes 16$, as boundary data rather than internal ambient products;
- named projector-chain streaming with a tri-state filter;
- explicit orthogonal primitive atoms for vector metrics, spinor pairings, $16 \ 16 \ 10$, $16' \ 16' \ 10$, $16 \ 16 \ 120$, $16' \ 16' \ 120$, $16 \ 16 \ 126$, and $16' \ 16' \ 126'$. The corresponding local product-channel projector ids now report executable formula kinds. Spinor pair and spinor-form channel audits are computed from Clifford grade, chirality, orthogonal dimension, and middle-form duality, so a wrong chirality or invalid self-duality input is blocked rather than receiving a generic primitive pass. Two-step gamma paths now flow through a left-fold local formula walker, streaming the first gamma/form atom and the later metric or form-pair projector atom separately, so filters and downstream code can see local formula composition rather than only a collapsed whole-path atom;
- an orthogonal form-spinor local formula route that streams a `gamma_action` atom. This covers the first previously missing $Spin(10) \ 16^{12}$ local channel shape $126 \otimes 16 \rightarrow 672$ without introducing dense component spaces;

- an orthogonal spinor-tower Cartan-product route that streams a **cartan_product** atom for multiplicity-zero B/D channels whose Dynkin labels add on one spinor node. This covers the next $Spin(10)$ channel $672 \otimes 16 \rightarrow 2772$ without broadening the route to unrelated $SU(2)$ symmetric products;
- an orthogonal spinor-tower tensor-spinor route that streams a **tensor_spinor_projection** atom for B/D channels with one ordinary form node and one lower residual spinor-tower power. This covers the two remaining $2772 \otimes 16$ channels $2772 \otimes 16 \rightarrow 29568$ and $2772 \otimes 16 \rightarrow 5280$;
- a recursive tensor-spinor tower route that reuses the same atom with a compact ordinary-form bitmask. It covers $29568 \otimes 16 \rightarrow 48114$, where the output keeps the existing rank-3 form node, adds a rank-1 form node, and lowers the spinor-tower power;
- a tensor-spinor mask-shift route for channels that lower tower power and shift exactly one ordinary form rank upward. It covers $48114 \otimes 16 \rightarrow 25200$, moving the rank set from $\{1, 3\}$ to $\{2, 3\}$;
- a terminal tensor-spinor route that streams a **tensor_form_projection** atom when tower power one contracts with a spinor into a non-spinor tensor-form channel. It covers $25200 \otimes 16 \rightarrow 4125$, represented by a compact nibble-packed ordinary-form profile;
- a tensor-form spinor route that streams a **tensor_spinor_projection** atom when an ordinary tensor-form profile acts on a spinor and removes one form count into a tower-power-one tensor-spinor. It covers $4125 \otimes 16 \rightarrow 1200$;
- a tensor-spinor middle-form route that streams a **tensor_form_projection** atom with middle-form rank and duality metadata. It covers $1200 \otimes 16 \rightarrow 126'$, the D-type middle-form channel represented on the spinor Dynkin nodes;
- an exact tensor-spinor form-terminal route that streams a **tensor_form_projection** atom when tower power one contracts with a spinor into an ordinary tensor-form with the same form profile. It covers $1200 \otimes 16 \rightarrow 120$;
- D-type ordinary form parsing now treats a common value on both spinor nodes as a rank $n - 1$ form count, while residual value on one spinor node remains the chiral spinor tower. This lets the same compact profile machinery handle D_5 rank-4 forms such as $[0, 0, 0, 1, 1]$;
- tensor-form spinor shift routes transform compact ordinary-form profiles before producing a tower-power-one tensor-spinor, including remove-then-shift upward, remove-then-shift downward, one-rank shift down, two-rank shift down, mixed shift down, and all-ranks shift down cases;
- D-type rank-form tower routes remove the rank $n - 1$ form from a tensor-spinor and increase tower power, including the terminal pure-tower output case;
- a D-type rank-wrap shift route moves one rank-1 form count to the rank $n - 1$ form position while lowering tower power;
- a tensor-spinor form-power route increments an existing form-rank count while lowering tower power;
- additional tensor-spinor terminal and tower routes now cover D-type opposite-chirality rank shifts, rank splits into rank $r - 1$ plus rank $n - 1$, terminal rank splits, terminal form addition, form-tower rank-down moves, form-preserving tower growth, opposite-chirality tower lowering with rank shift, opposite-chirality form-add terminal channels,

opposite-chirality terminal all-ranks shift, same-chirality remove-then-shift tower growth, same-chirality all-ranks shift tower growth, and pair-to-middle profile merges. These routes are expressed as transformations of compact form profiles, not as D_5 -specific channel tables;

- a streamed orthogonal vector-spinor boundary formula for projected $144 \subset 10 \otimes 16$ realizations. Expanded rendering emits the identity vector-spinor term and the normalized gamma-trace subtraction as separate terms after the compact-prefix filter boundary.
- non-SO formula routes for $A_2 : 3^3$, rendered as an epsilon atom, $E_6 : 27^3$, rendered as a backend-specific structure-constant atom, and a generic simple-adjoint cubic route exercised by $E_8 : 248^3$. In these cases the two local product-channel projectors are marked as verified backend-specific structure formulas.
- a generic product-identity atom for multiplicity-zero singlet products, so $1 \otimes R \rightarrow R$ and $R \otimes 1 \rightarrow R$ channels are streamed as explicit identity steps instead of named missing projectors.
- additional compact-profile routes for the remaining D_5 spinor-tower and tensor-spinor shapes in the 16^{12} benchmark: pure opposite-spinor tower lowering, opposite-spinor tower-to-form channels, tensor-form spinor preservation, arbitrary downward form shifts, opposite terminal neighbor splits, opposite terminal add-shift channels, opposite tower-lowering form-add and rank-split channels, all-rank shifts, and form-tower remove-shift channels.
- a public formula coverage audit for generated bases. After the current identity, form-spinor, spinor-tower, tensor-form, and tensor-spinor profile routes, the 16^{12} benchmark records 811184 expandable local steps out of 811184, 73744 fully expandable paths out of 73744, and no missing local projector channel without allocating expanded tensors. The same public API is exercised on the non-SO $A_2 : 3^3$ epsilon route, $E_6 : 27^3$ structure route, and $E_8 : 248^3$ adjoint structure route, so downstream callers can gate expanded streaming without relying on private tests.
- basis-level streamed rendering over compact paths. The 16^{12} benchmark now renders all 73744 expanded basis elements through the streaming API, emitting 73744 terms and 811184 formula atoms without named projector placeholders or dense tensor-component storage. The basis renderer reuses scratch atom buffers across paths, so the full-basis stream does not allocate fresh formula-term arrays for each invariant.
- a formula-program cache keyed by projector id and stored projector convention. It records hit/miss counters separately from named one-atom projector caches, while the emitted symbolic terms are still rebuilt into caller-owned scratch buffers for each streamed path.
- render requests whose signature does not match the context projector convention now fail before touching projector caches, so the current convention-keyed formula cache cannot be reused under a silently different signature.
- the first compact Clifford reducer primitive. `gammaProductExpansion` takes two antisymmetrized gamma-block ranks plus duality tags and returns the finite grade expansion $|p - q|, |p - q| + 2, \dots, p + q$ as a fixed small buffer of rank, contraction-count, coefficient, and output-duality records. `gammaProductContractionPlan` turns each grade term into canonical cross-block metric contractions and residual gamma vector slots. Both helpers check invalid ranks and middle-form duality without dense component spaces. Adjacent streamed gamma/form atoms followed by a `gamma_action` now append compact `clifford_product` reducer atoms, so the full 16^{12} expanded stream records 387804 gamma-product reductions in addition to the local projector formula atoms. The public rendering helpers `cliffordProductTerm` and `cliffordProductContractionPlan` decode these

streamed atoms back into their grade term and canonical metric/free-slot plan, checking that the stored counts are consistent. The indexed helpers `cliffordProductFactorCount` and `cliffordProductFactorAt` expose the same reducer atom as an allocation-free sequence of explicit metric factors and residual output-vector slots. `streamCliffordProductFactors` walks a borrowed `SymbolicTerm` and emits those factors through a caller sink. It accepts both compact `clifford_product` atoms and already-lowered `clifford_product_factor` atoms, so lower-level formula or evaluation code can consume either render mode without allocating an intermediate expression. `scanCliffordProductFactors` provides the corresponding validated scan: compact products and lowered products produce the same counts, while malformed lowered factor order or inconsistent per-product metadata is rejected before evaluator code consumes it. The scan object can also accumulate many accepted terms through `recordTerm`, giving evaluator sinks a small reusable state object for streaming basis-level factor audits. `CliffordProductFactorAuditSink` wraps that scan behind the ordinary `emitTerm` sink interface, so context rendering can be audited without a custom test-only sink. Renderers can now opt into this path with `RenderOptions.stream_clifford_product_factors`; the normal compact term is still emitted first, and `ExpansionAudit.clifford_product_factors` records how many lower-level factor records were sent to the sink. The stronger `RenderOptions.lower_clifford_product_atoms` mode rewrites each compact `clifford_product` atom into `clifford_product_factor` atoms in the emitted `SymbolicTerm` itself, using renderer scratch storage and preserving the reducer coefficient, output grade, duality, operator ids, and orthogonal dimension on each factor record. The 16^{12} benchmark now streams all 73744 terms through this lowered mode as well: emitted terms contain zero compact `clifford_product` atoms and more lowered factor atoms than the compact reducer count, while still using the same per-path scratch flow. The lowered benchmark also checks the exact atom accounting identity `lowered atoms = compact atoms - compact reducers + lowered factors` and verifies that lowered factor records carry source operator ids, *Spin*(10) dimension, and nonzero reducer coefficients.

These facts are necessary but not sufficient for the final goal. They prove that the multiplicity-space basis exists and that the concrete *Spin*(10) 16^{12} basis audit has full local formula coverage and can be streamed end to end in expanded symbolic-atom form. They do not yet prove that arbitrary ADE product-channel projectors can be expanded into a faithful formula stream.

3.7.3 Generality requirements

All next steps must preserve these boundaries:

- the public invariant-basis API remains Dynkin-label based and family-generic;
- A_n, D_n, E_6, E_7, E_8 decomposition and path enumeration must not depend on *Spin*(10) dimensions, gamma atoms, or hand-entered channel tables;
- SO/Spin Clifford formulas are backend-rendered formulas attached to projector ids, not the definition of the abstract basis;
- adding a formula for $10 \otimes 16 \rightarrow 144$ must not special-case basis generation for 16^{12} ;
- future *SU*(*N*) tableau/Gelfand-Tsetlin renderers and exceptional renderers must fit the same projector-formula interface;
- acceptance tests should keep one large stress case such as 16^{12} , but must also include non-*SO*(10) ADE products so regressions in the generic solver are visible.

3.7.4 Immediate consistency tasks

Before adding more formulas, finish these small consistency tasks:

- **BasisAudit** counts both local coupling projectors and boundary realization projectors. Formula coverage audits now also count boundary formula obligations, so a basis with only projected legs cannot be marked fully expandable while its boundary formula is still missing.
- The streamed expansion audit reports how many boundary projector atoms and local projector atoms were encountered, separately from filter accept/reject/undecided counts.
- Named boundary projectors should keep their current behavior: named mode may emit them, while expanded mode must fail with a typed missing-formula error unless the boundary formula has been implemented for that realization.

3.7.5 Formula execution model

Do not implement projector formulas as eager expression builders. A projector formula is a small procedure that transforms a borrowed streamed term frontier. The procedure should:

- read only the current compact term slice and the local projector descriptor;
- append a small number of output terms directly to the next frontier or sink;
- reuse scratch buffers owned by the renderer;
- call the filter as soon as a partial term is soundly rejectable;
- cache the formula program and any invariant convention data by projector id and rendering convention.

The formula IR should be lean. A useful first shape is:

```
const ProjectorFormulaKind = enum {  
    identity,  
    orthogonal_vector_metric,  
    orthogonal_spinor_pair,  
    orthogonal_gamma_trace,  
    orthogonal_gamma_traceless,  
    orthogonal_spinor_form_channel,  
    named_only,  
};
```

The executable formula should be a switch over this kind, not a tree of heap allocated symbolic expressions. Each case emits one or more **SymbolicAtom** records into scratch storage and immediately applies the active filter.

3.7.6 Required first projector formulas

The first nontrivial formula target is implemented for the SO/Spin acceptance formula: the boundary projector for $144 \subset 10 \otimes 16$. This is the gamma-traceless vector-spinor. It exercises projected realizations and gamma algebra through the general projector-formula stream, not a one-off render path. The ambient tensor $V \otimes S$ decomposes as

$$10 \otimes 16 = 16' \oplus 144.$$

In a fixed Clifford convention, the spinor-channel projector is the gamma-trace part

$$P_{16'} \sim \frac{1}{N} \gamma_m \gamma^n,$$

where $N = \dim V = 10$. The 144 projector is the complement

$$P_{144} = I_{V \otimes S} - P_{16'}.$$

Implementation requirements for this formula:

- stored as a boundary-realization projector formula for projected 144 legs;
- rendered as a two-term streamed formula: identity vector-spinor term minus gamma-trace term;
- normalization encoded with the same rational-id mechanism used for streamed coefficients;
- compute the formula audit from the orthogonal dimension N , using $\Gamma^2 = N\Gamma$ for the gamma-trace channel and rejecting zero denominator inputs;
- verify gamma-tracelessness $\gamma^m P_{144,m} = 0$, idempotency $P_{144}^2 = P_{144}$, and separation from the $16'$ gamma-trace channel with symbolic Clifford identities, not dense $10 \cdot 16$ matrices.

The second formula target is the SO/Spin product-channel projector family generated by spinor bilinears:

- $16 \otimes 16 \rightarrow 10, 120, 126$;
- $16' \otimes 16' \rightarrow 10, 120, 126'$;
- $16 \otimes 16' \rightarrow 1$ and higher even-form channels once they are needed by a selected path.

For these channels, the compact gamma atoms already record enough metadata: orthogonal dimension, rank, chirality, and duality. What is missing is a formula route that can compose these atoms with later projectors and reduce gamma products using Clifford identities. This should be implemented as a gamma-algebra reducer over streamed atoms, not as component matrices.

The first non-SO formula routes now use the same interface for $A_2 : 3^3$, rendered as an epsilon atom, $E_6 : 27^3$, rendered as a compact backend-specific structure atom, and simple adjoint cubics such as $E_8 : 248^3$. These use backend-specific structure formula metadata so the layer does not become a hidden *Spin*(10)-only subsystem.

3.7.7 Gamma algebra reducer

Projector formulas that act on output gammas require a reducer for products of antisymmetrized gamma blocks. The reducer must support:

- contraction of vector metrics through gamma atoms;
- multiplication of rank- p and rank- q antisymmetrized gammas into the finite sum of ranks $|p - q|, |p - q| + 2, \dots, p + q$, with convention-controlled signs and rational coefficients. The current reducer computes this compact grade plan, combinatorial coefficient, cross-block metric pair plan, and residual gamma-slot order in fixed buffers;
- Hodge duality for middle forms in D_n , including the D_5 126 and $126'$ duality tags;
- chirality selection rules, so invalid gamma strings are rejected or reduced to zero before reaching the sink;

- canonical ordering of gamma atoms so repeated reductions hit the formula cache.

Do not hide these as handwritten Fierz tables. The reducer should implement the Clifford algebra rules once and derive the channel projectors from the stored product-channel metadata. Hand-entered coefficients are acceptable only as convention constants that are then checked by idempotency and channel separation tests.

3.7.8 Projector cache requirements

The projector store caches named one-atom expansions and formula-program routes. Formula-program cache entries are keyed by:

- projector id;
- render mode;
- signature convention;
- normalization id;
- gamma convention id if this becomes separate from signature.

The cached value is not an expanded global invariant. It is the compact local formula route stored on the projector id; term atoms are still written into scratch storage for the currently streamed path. Global expansion still streams one selected path at a time. The current renderer rejects non-default signatures until projector ids can be regenerated under the requested convention, rather than reusing a cache entry with the wrong signature.

3.7.9 Acceptance gates for the next milestone

The next milestone is reached only when all of the following are true:

- named rendering still works for 16^{12} without expanding dense tensors;
- expanded rendering of a projected 144 boundary leg emits the gamma-trace complement formula instead of a named placeholder;
- a reject-all filter still stops before executing the boundary formula;
- the 144 boundary projector reports `expandable_terms` only after its idempotency and gamma-traceless audits pass;
- product-channel projector formulas for the already-rendered gamma atoms can compose with later local projectors in the stream;
- tests verify projector laws in symbolic form from $N = \dim V$ for $10 \otimes 16 = 16' \oplus 144$, including rejected zero-dimension audit input, and for at least one spinor-square channel;
- at least one non- $SO(10)$ ADE invariant still exercises the same generic path-counting, path-enumeration, and named-projector streaming path;
- no test or implementation allocates arrays proportional to 16^{12} or to a dense tensor-component space.

3.8 Backend Goal

The first backend should be a generic highest-weight backend. It should be a correctness backend and cache builder, not yet a family-specific optimized renderer. Its first job is to answer:

1. is this algebra supported?
2. what is the weight system of this irrep?
3. what is the decomposition of $R \otimes S$?
4. what stable projector identity corresponds to one product channel?

The backend must not allocate dense tensor-product component spaces. It may construct compact weight multisets and flat cache records.

3.9 Backend Build Order

3.9.1 1. Backend registry

Add an internal backend registry owned by the private `Context` implementation. It maps `LieFamily` to a backend record and selects a backend from an algebra family. It should not be exported from Tensor-Code.

Start with one backend implementation:

- file: `generic-highest-weight-backend.zig`;
- supports $A_n, B_n, D_n, E_6, E_7, E_8$;
- returns typed missing-capability errors for unimplemented operations.

The registry should make no CFT-facing API changes. CFT code still asks the context for invariant bases.

3.9.2 2. Root-data helpers

Use Tensor Root Data as the source of algebra conventions. Do not duplicate Cartan matrices or exceptional diagrams in the backend.

The backend will need internal helpers for:

- rank;
- Cartan entries;
- simple-root reflection on weights;
- dominant-weight test;
- highest-weight validation;
- exact rational inner products where Freudenthal or Casimir formulas need them.

Keep these helper APIs internal unless a later caller clearly needs them.

3.9.3 3. Weight storage

Weights are not public API. Store them in flat buffers:

```
const WeightRef = struct {
    dynkin_offset: u32,
};

const WeightMultiplicity = struct {
    weight: WeightRef,
    multiplicity: u32,
};
```

The actual Dynkin-coordinate arrays should live in one flat signed buffer, probably `i32` once tensor products become large enough. Signed coordinates are necessary because intermediate weights and reflected weights need not be dominant.

Store rank once on the weight-system span or get it from the algebra. Do not store rank in every weight record. Do not allocate one heap object per weight.

3.9.4 4. Weight-system generator

Implement a cached weight-system generator for one highest-weight irrep. Freudenthal multiplicity recursion is the generic first choice because it works uniformly for the supported simple families.

Cache key:

```
const WeightSystemKey = struct {
    algebra: store.AlgebraHandle,
    irrep: store.IrrepHandle,
};
```

Cache value:

```
const WeightSystem = struct {
    weights_offset: u32,
    weights_len: u32,
    rank: u8,
    dynkin_coords_offset: u32,
    dynkin_coords_len: u32,
};
```

Acceptance checks:

- D_5 spinor has total weight multiplicity 16;
- D_5 vector has total weight multiplicity 10;
- B_4 spinor has total weight multiplicity 16;
- E_8 adjoint has total weight multiplicity 248.

3.9.5 5. Tensor-product decomposition

Use character subtraction first. It is generic and easier to audit than a family-specific closed formula.

Algorithm:

1. get cached weight systems for R and S ;
2. add all pairs of weights to build the weight multiset for $R \otimes S$;
3. find a maximal dominant weight still present, using one deterministic dominance-order rule;
4. its remaining multiplicity is the multiplicity of the next channel;

5. subtract that many copies of the cached character of that highest weight;
6. repeat until the multiset is empty.

The dominance selection rule must be explicit and stable. A practical first rule is:

1. scan only weights with positive residual multiplicity;
2. discard non-dominant weights;
3. choose a dominant weight not strictly below another remaining dominant weight in the root partial order;
4. break ties by lexicographic Dynkin labels.

After every subtraction, no residual multiplicity may become negative. A negative residual is a backend bug, not a recoverable ambiguity. The selected channel multiplicity is the residual multiplicity of the selected highest weight before subtraction.

The audit invariant after each subtraction is:

$$\chi_{\text{remaining}} + \sum_i m_i \chi_{T_i} = \chi_{R\chi S}.$$

The final residual character must be zero. This catches lost multiplicities, wrong dominant-weight selection, and incorrect subtraction signs.

This is not the final fastest algorithm. It is the first trusted backend. It will become the oracle against which specialized *SO/Spin*, *SU*, and exceptional backends are checked.

Acceptance checks:

- $SU(2) : 2 \otimes 2 = 1 \oplus 3$;
- $Spin(10) : 10 \otimes 16 = 16 \oplus 144$;
- $Spin(10) : 16 \otimes 16 = 10 \oplus 120 \oplus 126$;
- $E_8 : 248 \otimes 248$ matches the known decomposition, including multiplicities and dimensions.

3.9.6 6. Decomposition cache integration

The backend writes decompositions into Tensor Decomposition. The cache stores:

- product key R, S ;
- flat product terms (T, m_T) ;
- stable handle for repeated requests.

The future internal context method should look like:

```
fn decomposeProduct(ctx: *Context, left: store.IrrepHandle, right: store.IrrepHandle)
  ↪ !decomposition.ProductDecompositionHandle
```

Do not expose this method publicly unless CFT code needs it.

Invariant-basis generation must distinguish true zero multiplicity from a failed computation. Unsupported tree policies, unresolved registered realizations, and backend decomposition failures are typed errors. They must not be stored as successful empty bases, because that would make missing implementation look like a mathematically valid invariant count.

3.9.7 7. Projector identities and expansion obligations

For each product term, the context creates a product-channel projector identity in Tensor Projector. The first decomposition backend does not expand projector formulas immediately. A successful basis may therefore contain projector ids that are renderable as compact names only, but expanded rendering must fail before emitting placeholder terms. Each projector record still needs enough metadata to attach a later formula route.

The projector identity records:

- left irrep;
- right irrep;
- target channel;
- convention id;
- derivation kind;
- expansion source, such as Casimir polynomial, idempotent split, highest-weight solver output, or backend-specific verified construction.

Do not store expandability as a boolean. The backend needs enough state to distinguish several useful stages:

```
const ExpansionStatus = enum {  
    not_requested,  
    expandable_named,  
    expandable_blocks,  
    expandable_terms,  
    blocked_missing_formula,  
    blocked_unverified_identity,  
};
```

`expandable_named` is only a rendering mode. It is not proof that the projector has a formula. A projector appearing in a successful invariant basis may stop at this stage while formula renderers are missing. Requests for `expanded_blocks` or `expanded_terms` must check the status and return a typed error before successful placeholder emission.

Acceptance checks:

- repeated requests return the same projector id;
- multiplicity copies are explicit;
- no symbolic projector expansion is allocated during decomposition;
- every returned projector reports an `ExpansionStatus` in the requested convention;
- blocked projector statuses are typed errors before expanded rendering succeeds, not silently named placeholders;
- a reject-all filter can stop a non-empty expanded render before missing local formulas are requested.

3.9.8 8. Backend capability implementation

Update Tensor Backend only as needed to support the generic backend. The expected internal procedures are:

- `validateAlgebra;`
- `irrepMetadata`, if a backend-specific metadata path is needed;
- `weightSystem;`
- `decomposeProduct;`
- `localProjectorId`.

Backend projector formula rendering remains `NotImplemented`. Context-level named path rendering is allowed to emit compact projector atoms.

3.9.9 9. Expansion stream and filters

Before explicit projector formulas are implemented, define the execution boundary they must satisfy. The expansion API should be a streaming pipeline:

```
fn renderInvariantExpanded(  
    ctx: *Context,  
    basis: coupling.BasisHandle,  
    invariant: coupling.InvariantHandle,  
    options: rendering.RenderOptions,  
    filter: ExpansionFilter,  
    sink: anytype,  
) !void
```

The exact public shape may differ, but the semantics should not:

```
const FilterDecision = enum {  
    accept,  
    reject,  
    undecided,  
};
```

- the renderer walks the compact coupling path;
- local projector expansions are pulled lazily;
- each local expansion term is combined into the current partial term;
- the filter is applied as early as possible to borrowed scratch terms;
- accepted terms are emitted immediately to the sink;
- rejected terms are not stored;
- undecided partial terms may continue, but the implementation must track their frontier size;
- repeated local projector expansions are cached by projector id and convention when doing so reduces work.

Partial terms can be rejected only when the predicate is final and sound under all later factors. Otherwise the filter must return **undecided**. This is part of correctness, not just an optimization detail.

Filter callbacks and sinks must not retain borrowed slices from streamed terms. If a caller needs to keep a term, it must copy it into caller-owned storage. This keeps the renderer free to reuse scratch buffers and prevents a hidden materialized expansion from growing behind the streaming interface.

The filter is not a post-processing pass over a giant vector. It is an execution hook that keeps the streamed expansion small enough to be useful.

Possible filter predicates:

- term contains or avoids a given external index block;
- term uses only selected symbolic operator kinds;
- term is compatible with a component assignment;
- term can contribute to a downstream contraction;
- term has a coefficient/operator class accepted by the target CFT routine.

Acceptance checks:

- expanding one invariant can stream terms without allocating the full expansion;
- expanding many invariants can reuse local projector expansions;
- a filter that rejects all terms performs no large output allocation;
- a filter that accepts a sparse subset scales with the accepted frontier plus unavoidable local projector traversal, not with a materialized full basis.

3.10 Compute Model

The first backend has four different compute scales:

1. Decomposition compute: weight-system generation, tensor-product weight multisets, and character subtraction.
2. Path enumeration compute: dynamic programming over cached binary decompositions to enumerate coupling paths to a target.
3. Expansion compute: streaming the local projector/CG expansions appearing in selected paths.
4. Filter compute: deciding early which partial or complete expansion terms can be discarded before they reach downstream CFT code.

The first milestone implements the first scale and prepares records for the third and fourth. It must not pretend expansion is free. A 73744-path result can be small as stored paths while still being expensive to render if each path contains large expandable projectors. Therefore every feasibility estimate must track:

- number of basis paths;
- number of distinct local projector ids used by those paths;

- maximum and total expected expansion terms per projector when known;
- whether expansion can be shared across repeated projector ids;
- whether expansion is requested as named, block-expanded, or term-expanded;
- expected filter rejection rate;
- whether filters can be applied to partial products or only complete terms;
- number of undecided partial terms retained at each expansion layer;
- sink throughput and backpressure behavior.

3.11 Data-Oriented Constraints

The expensive objects are weight systems, tensor-product weight multisets, path records, and projector expansions. Therefore:

- cache weight systems by irrep handle;
- store weights in flat buffers;
- sort or hash compact weight keys, not heap-allocated slices;
- store product decompositions as flat terms;
- keep projector ids as small records;
- cache projector expansions by projector id and convention when expansion is requested repeatedly;
- push filters into expansion loops;
- emit accepted terms immediately;
- stream rendered terms into sinks;
- never build dense component tensors.

The 16^{12} target should produce coupling paths, not components. A single dense tensor in 16^{12} has 16^{12} components, while the desired answer has 73744 basis paths. The algorithm must scale with the number of channels and paths, not the full component space.

3.12 Explicit Non-Goals For First Backend

Do not implement the following in the first backend:

- gamma-matrix rendering;
- bulk explicit projector expansion execution;
- explicit CG coefficient tables;
- higher-Casimir projector splitting;
- sparse nullspace solvers;
- parallel execution;

- $SU(N)$ Gelfand-Tsetlin or tableau optimized CGs;
- $SO/Spin$ Clifford-specialized renderers;
- E_8 adjoint-specialized renderers.

Those belong after binary decomposition is correct, cached, and tested. The data model must still be designed so these operations can be added without changing the meaning of existing projector ids.

3.13 Definition Of Done

The backend handoff is complete when:

- `zig build --global-cache-dir /tmp/zig-global-cache` passes;
- all acceptance decompositions above pass;
- weight systems are cached and flat;
- product decompositions are cached by key;
- no dense tensor component arrays are allocated;
- `Context.invariantBasis` can internally request product decompositions;
- projector records created by the context have explicit expansion obligations;
- named invariant paths can be streamed through a filter without materializing their full term list;
- repeated named local projector expansions are cached by projector id;
- expanded invariant requests stop with typed formula errors until local projector renderers exist;
- repeated formula-level projector expansions can be cached and reused once specialized renderers exist;
- a downstream sink can consume accepted terms incrementally;
- evaluate remains explicitly unimplemented until projector formulas and component term streaming are ready.

4 Tensor Context

The context module is the procedural boundary of tensor-code. Public callers get an opaque state pointer and methods; the stores remain in a private implementation record so callers cannot bypass validation. Local formula routes delegate reusable explicit projector programs to Projector Constructor.

```
const std = @import("std");
const backend = @import("backend.zig");
const coupling = @import("coupling.zig");
const decomposition = @import("decomposition.zig");
const generic_backend = @import("generic-highest-weight-backend.zig");
const projector = @import("projector.zig");
const projector_constructor = @import("projector-constructor.zig");
const realization = @import("realization.zig");
```

```

const rendering = @import("rendering.zig");
const root_data = @import("root-data.zig");
const store = @import("representation-store.zig");
const symmetry = @import("symmetry.zig");

/// ContextId names an initialized tensor-code context or preset.
pub const ContextId = struct {
    value: u32,

    /// init constructs a context id from a store index.
    pub fn init(value: u32) ContextId {
        return .{ .value = value };
    }
};

/// ContextOptions configures algebra and rendering defaults.
pub const ContextOptions = struct {
    algebra_conventions: root_data.AlgebraConventions = .{},
    default_signature: rendering.SignatureConvention = .complex,
};

/// FormulaCoverageAudit stores formula-readiness counters for a generated basis.
pub const FormulaCoverageAudit = struct {
    path_count: u32 = 0,
    local_step_count: u64 = 0,
    expandable_step_count: u64 = 0,
    missing_step_count: u64 = 0,
    boundary_projector_count: u32 = 0,
    boundary_step_count: u64 = 0,
    expandable_boundary_step_count: u64 = 0,
    missing_boundary_step_count: u64 = 0,
    fully_expandable_path_count: u32 = 0,
    distinct_projector_count: u32 = 0,
    distinct_boundary_projector_count: u32 = 0,
    first_missing_projector: ?projector.ProjectorId = null,
    first_missing_left: store.IrrepHandle = .{ .value = 0 },
    first_missing_right: store.IrrepHandle = .{ .value = 0 },
    first_missing_output: store.IrrepHandle = .{ .value = 0 },
    first_missing_boundary: ?realization.RealizationHandle = null,
};

/// Context is an opaque handle to tensor-code stores and caches.
pub const Context = struct {
    state: *anyopaque,
    allocator: std.mem.Allocator,

    /// init constructs an empty tensor-code context.
    pub fn init(allocator: std.mem.Allocator) !Context {
        return Context.initWithOptions(allocator, .{});
    }

    /// initWithOptions constructs a context with explicit conventions.
    pub fn initWithOptions(allocator: std.mem.Allocator, options: ContextOptions) !Context {
        const state = try allocator.create(Impl);
        errdefer allocator.destroy(state);
        state.* = try Impl.init(allocator, options);
        return .{
            .state = state,
            .allocator = allocator,
        };
    }

    /// deinit releases memory owned by this context.
    pub fn deinit(self: *Context) void {

```

```

    const state = self.impl();
    state.deinit();
    self.allocator.destroy(state);
    self.* = undefined;
}

/// registerAlgebra interns a Lie algebra and returns its handle.
pub fn registerAlgebra(self: *Context, spec: store.AlgebraSpec) !store.AlgebraHandle {
    switch (spec) {
        .simple => |simple| try self.impl().validateBackend(simple),
        .u1 => {},
    }
    return self.impl().representations.internAlgebra(spec);
}

/// registerIrrep interns an abstract irreducible representation.
pub fn registerIrrep(self: *Context, algebra: store.AlgebraHandle, spec: store.IrrepSpec)
↳ !store.IrrepHandle {
    return self.impl().representations.internIrrep(algebra, spec);
}

/// dualIrrep returns the registered dual representation.
pub fn dualIrrep(self: *Context, irrep: store.IrrepHandle) !store.IrrepHandle {
    return self.impl().representations.dualIrrep(irrep);
}

/// irrepMetadata returns derived metadata for a registered irrep.
pub fn irrepMetadata(self: Context, irrep: store.IrrepHandle) ?store.IrrepMetadata {
    return self.impl().representations.irrepMetadata(irrep);
}

/// registerRealization interns a concrete index realization of an irrep.
pub fn registerRealization(self: *Context, spec: realization.RealizationSpec)
↳ !realization.RealizationHandle {
    try self.validateRealizationSpec(spec);
    return self.impl().realizations.internRealization(spec);
}

/// invariantBasis generates or retrieves a compact invariant basis.
pub fn invariantBasis(self: *Context, request: coupling.InvariantBasisRequest)
↳ !coupling.BasisHandle {
    try self.validateInvariantBasisRequest(request);
    const path_count = try self.impl().countInvariantPaths(request);
    var paths = try self.impl().enumerateInvariantPaths(request);
    defer paths.deinit();
    if (path_count != paths.paths.items.len) return error.PathCountMismatch;
    const boundary_projector_count = try self.impl().countBoundaryProjectors(request);
    return self.impl().couplings.internBasisRequestWithPaths(request, path_count,
↳ paths.paths.items, paths.steps.items, boundary_projector_count);
}

/// basisInvariantCount returns the number of invariants in a generated basis.
pub fn basisInvariantCount(self: Context, basis: coupling.BasisHandle) ?u128 {
    return self.impl().couplings.basisPathCount(basis);
}

/// basisAudit returns compact generation counters for a basis.
pub fn basisAudit(self: Context, basis: coupling.BasisHandle) ?coupling.BasisAudit {
    return self.impl().couplings.basisAudit(basis);
}

/// basisFormulaCoverageAudit returns formula-readiness counters for a basis.
pub fn basisFormulaCoverageAudit(self: *Context, basis: coupling.BasisHandle)
↳ !FormulaCoverageAudit {

```

```

        return self.impl().basisFormulaCoverageAudit(basis);
    }

    /// projectorDescriptor resolves a streamed projector operator id.
    pub fn projectorDescriptor(self: Context, operator_id: u32) ?projector.ProjectorDescriptor
    ↪ {
        return self.impl().projectors.projectorDescriptor(operator_id);
    }

    /// renderInvariant streams one invariant in the requested convention.
    pub fn renderInvariant(self: *Context, basis: coupling.BasisHandle, invariant:
    ↪ coupling.InvariantHandle, options: rendering.RenderOptions, sink: anytype) !void {
        _ = try self.renderInvariantFiltered(basis, invariant, options,
        ↪ rendering.ExpansionFilter.acceptAll(), sink);
    }

    /// renderInvariantFiltered streams one invariant through a tri-state filter.
    pub fn renderInvariantFiltered(self: *Context, basis: coupling.BasisHandle, invariant:
    ↪ coupling.InvariantHandle, options: rendering.RenderOptions, filter:
    ↪ rendering.ExpansionFilter, sink: anytype) !rendering.ExpansionAudit {
        try self.validateRenderOptions(options);
        const path = self.impl().couplingPathForInvariant(basis, invariant) orelse return
        ↪ error.UnknownInvariant;
        const request = self.impl().couplings.basisRequest(basis) orelse return
        ↪ error.UnknownBasis;
        var scratch: ExpansionScratch = .{};
        defer scratch.deinit(self.allocator);
        return self.impl().streamPathExpansion(request, path, options, filter, sink,
        ↪ &scratch);
    }

    /// renderBasis streams every invariant in a generated basis.
    pub fn renderBasis(self: *Context, basis: coupling.BasisHandle, options:
    ↪ rendering.RenderOptions, sink: anytype) !void {
        _ = try self.renderBasisFiltered(basis, options,
        ↪ rendering.ExpansionFilter.acceptAll(), sink);
    }

    /// renderBasisFiltered streams every invariant in a basis through a filter.
    pub fn renderBasisFiltered(self: *Context, basis: coupling.BasisHandle, options:
    ↪ rendering.RenderOptions, filter: rendering.ExpansionFilter, sink: anytype)
    ↪ !rendering.ExpansionAudit {
        try self.validateRenderOptions(options);
        const paths = self.impl().couplings.basisPaths(basis) orelse return
        ↪ error.UnknownBasis;
        const request = self.impl().couplings.basisRequest(basis) orelse return
        ↪ error.UnknownBasis;
        var audit: rendering.ExpansionAudit = .{};
        var scratch: ExpansionScratch = .{};
        defer scratch.deinit(self.allocator);
        for (paths) |path| {
            const path_audit = try self.impl().streamPathExpansion(request, path, options,
            ↪ filter, sink, &scratch);
            audit.merge(path_audit);
        }
        return audit;
    }

    /// evaluateInvariant streams component-evaluation data for one invariant.
    pub fn evaluateInvariant(self: *Context, invariant: coupling.InvariantHandle, options:
    ↪ rendering.EvalOptions, sink: anytype) !void {
        _ = self;
        _ = invariant;
        _ = options;
    }

```

```

    _ = sink;
    return error.NotImplemented;
}

/// evaluateBasisInvariant streams lowered symbolic evaluation data for one basis
⇨ invariant.
pub fn evaluateBasisInvariant(self: *Context, basis: coupling.BasisHandle, invariant:
⇨ coupling.InvariantHandle, options: rendering.EvalOptions, sink: anytype)
⇨ !rendering.EvaluationAudit {
    try self.validateEvalOptions(options);
    const render_options = rendering.RenderOptions{
        .projectors = .expanded_terms,
        .signature = options.signature,
        .lower_clifford_product_atoms = true,
    };
    var adapter = EvaluationSinkAdapter(@TypeOf(sink)){ .inner = sink };
    const expansion_audit = try self.renderInvariantFiltered(basis, invariant,
⇨ render_options, rendering.ExpansionFilter.acceptAll(), &adapter);
    return .{
        .terms = expansion_audit.emitted,
        .rejected = expansion_audit.rejected,
        .lowered_clifford_factors = expansion_audit.clifford_product_factors,
    };
}

fn validateRealizationSpec(self: Context, spec: realization.RealizationSpec) !void {
    const target_algebra = self.impl().representations.irrepAlgebra(spec.channel.irrep)
⇨ or else return error.UnknownIrrep;
    for (spec.ambient) |ambient| {
        const ambient_algebra = self.impl().representations.irrepAlgebra(ambient) or else
⇨ return error.UnknownIrrep;
        if (ambient_algebra.value != target_algebra.value) return
⇨ error.MixedRealizationAlgebras;
    }
}

fn validateInvariantBasisRequest(self: Context, request: coupling.InvariantBasisRequest)
⇨ !void {
    if (!self.impl().representations.containsAlgebra(request.algebra)) return
⇨ error.UnknownAlgebra;
    for (request.external_legs) |leg| {
        const leg_algebra = try self.externalLegAlgebra(leg);
        if (leg_algebra) |algebra| {
            if (algebra.value != request.algebra.value) return
⇨ error.MixedInvariantAlgebras;
        }
    }
}

fn validateRenderOptions(self: Context, options: rendering.RenderOptions) !void {
    if (options.signature != self.impl().options.default_signature) return
⇨ error.UnsupportedRenderSignature;
    if (options.gamma_only and options.projectors != .expanded_terms) return
⇨ error.GammaOnlyRequiresExpandedTerms;
}

fn validateEvalOptions(self: Context, options: rendering.EvalOptions) !void {
    if (options.signature != self.impl().options.default_signature) return
⇨ error.UnsupportedEvalSignature;
}

fn externalLegAlgebra(self: Context, leg: realization.ExternalLeg) !?store.AlgebraHandle {
    return switch (leg.realization) {

```

```

        .primitive_irrep => |irrep| self.impl().representations.irrepAlgebra(irrep) orelse
        ↪ return error.UnknownIrrep,
        .handle => |handle| {
            const irrep = self.impl().realizations.targetIrrep(handle) orelse return
            ↪ error.UnknownRealization;
            return self.impl().representations.irrepAlgebra(irrep) orelse return
            ↪ error.UnknownIrrep;
        },
        .registered_name => null,
    };
}

fn externalLegIrrep(self: Context, leg: realization.ExternalLeg) !store.IrrepHandle {
    return self.impl().externalLegIrrep(leg);
}

fn impl(self: Context) *Impl {
    return @ptrCast(@alignCast(self.state));
}

};

const Impl = struct {
    id: ContextId,
    allocator: std.mem.Allocator,
    options: ContextOptions,
    generic_backend: *generic_backend.Backend,
    backends: BackendRegistry,
    representations: store.Store,
    realizations: realization.Store,
    decompositions: decomposition.Store,
    projectors: projector.Store,
    tensor_form_projection_programs: projector_constructor.TensorFormProjectionProgramCache,
    tensor_spinor_projection_programs:
    ↪ projector_constructor.TensorSpinorProjectionProgramCache,
    structural_projection_programs: projector_constructor.StructuralProjectorProgramCache,
    couplings: coupling.Store,

    fn init(allocator: std.mem.Allocator, options: ContextOptions) !Impl {
        const generic = try allocator.create(generic_backend.Backend);
        errdefer allocator.destroy(generic);
        generic.* = generic_backend.Backend.init(allocator, options.algebra_conventions);

        return .{
            .id = ContextId.init(0),
            .allocator = allocator,
            .options = options,
            .generic_backend = generic,
            .backends = BackendRegistry.init(generic),
            .representations = store.Store.init(allocator, options.algebra_conventions),
            .realizations = realization.Store.init(allocator),
            .decompositions = decomposition.Store.init(allocator),
            .projectors = projector.Store.init(allocator),
            .tensor_form_projection_programs =
            ↪ projector_constructor.TensorFormProjectionProgramCache.init(allocator),
            .tensor_spinor_projection_programs =
            ↪ projector_constructor.TensorSpinorProjectionProgramCache.init(allocator),
            .structural_projection_programs =
            ↪ projector_constructor.StructuralProjectorProgramCache.init(allocator),
            .couplings = coupling.Store.init(allocator),
        };
    }
}

fn deinit(self: *Impl) void {
    self.couplings.deinit();
}

```

```

    self.structural_projection_programs.deinit();
    self.tensor_spinor_projection_programs.deinit();
    self.tensor_form_projection_programs.deinit();
    self.projectors.deinit();
    self.decompositions.deinit();
    self.realizations.deinit();
    self.representations.deinit();
    self.generic_backend.deinit();
    self.allocator.destroy(self.generic_backend);
    self.* = undefined;
}

fn validateBackend(self: *Impl, simple: symmetry.SimpleLieAlgebra) !void {
    const record = self.backends.select(simple) orelse return
        ↪ error.UnsupportedAlgebraFamily;
    try record.capabilities.validate_algebra(record.state, simple);
}

fn decomposeProduct(self: *Impl, left: store.IrrepHandle, right: store.IrrepHandle)
    ↪ !decomposition.ProductDecompositionHandle {
    const algebra = self.representations.irrepAlgebra(left) orelse return
        ↪ error.UnknownIrrep;
    const right_algebra = self.representations.irrepAlgebra(right) orelse return
        ↪ error.UnknownIrrep;
    if (algebra.value != right_algebra.value) return error.MixedProductAlgebras;
    const simple = self.representations.simpleAlgebra(algebra) orelse return
        ↪ error.UnsupportedAlgebraFamily;
    const record = self.backends.select(simple) orelse return
        ↪ error.UnsupportedAlgebraFamily;
    return record.capabilities.decompose_product(record.state, &self.representations,
        ↪ &self.decompositions, .{
        .left = left,
        .right = right,
    });
}

fn countBoundaryProjectors(self: *Impl, request: coupling.InvariantBasisRequest) !u32 {
    var count: u32 = 0;
    for (request.external_legs) |leg| {
        const handle = switch (leg.realization) {
            .handle => |handle| handle,
            .primitive_irrep, .registered_name => continue,
        };
        const spec = self.realizations.realizationSpecFor(handle) orelse return
            ↪ error.UnknownRealization;
        if (spec.ambient.len != 0) count += 1;
    }
    return count;
}

fn couplingPathForInvariant(self: *Impl, basis: coupling.BasisHandle, invariant:
    ↪ coupling.InvariantHandle) ?coupling.CouplingPath {
    const paths = self.couplings.basisPaths(basis) orelse return null;
    const index: usize = @intCast(invariant.value);
    if (index >= paths.len) return null;
    return paths[index];
}

fn basisFormulaCoverageAudit(self: *Impl, basis: coupling.BasisHandle)
    ↪ !FormulaCoverageAudit {
    const paths = self.couplings.basisPaths(basis) orelse return error.UnknownBasis;
    const request = self.couplings.basisRequest(basis) orelse return error.UnknownBasis;
    var distinct = std.AutoHashMap(projector.ProjectorId, void).init(self.allocator);
    defer distinct.deinit();

```



```

var audit: FormulaCoverageAudit = .{
  .path_count = @intCast(paths.len),
};
const boundary_expandable = try self.auditBoundaryFormulaCoverage(request, &audit);
for (paths) |path| {
  const steps = self.couplings.pathSteps(path);
  var path_expandable = boundary_expandable;
  for (steps) |step| {
    audit.local_step_count += 1;
    try distinct.put(step.projector, {});
    const derivation = self.projectors.projectorDerivation(step.projector) orelse
    ↪ return error.UnknownProjector;
    if (derivation.status == .expandable_terms) {
      audit.expandable_step_count += 1;
    } else {
      audit.missing_step_count += 1;
      path_expandable = false;
      if (audit.first_missing_projector == null) {
        audit.first_missing_projector = step.projector;
        audit.first_missing_left = step.left;
        audit.first_missing_right = step.right;
        audit.first_missing_output = step.output;
      }
    }
  }
  if (path_expandable) audit.fully_expandable_path_count += 1;
}
audit.distinct_projector_count = @intCast(distinct.count());
return audit;
}

fn auditBoundaryFormulaCoverage(self: *Impl, request: coupling.InvariantBasisRequest,
↪ audit: *FormulaCoverageAudit) !bool {
  var distinct = std.AutoHashMap(projector.ProjectorId, void).init(self.allocator);
  defer distinct.deinit();

  var all_expandable = true;
  const path_multiplier: u64 = @intCast(audit.path_count);
  for (request.external_legs) |leg| {
    const handle = switch (leg.realization) {
      .handle => |handle| handle,
      .primitive_irrep, .registered_name => continue,
    };
    const spec = self.realizations.realizationSpecFor(handle) orelse return
    ↪ error.UnknownRealization;
    if (spec.ambient.len == 0) continue;

    const boundary_projector = try self.boundaryRealizationProjector(handle);
    try distinct.put(boundary_projector, {});
    audit.boundary_projector_count += 1;
    audit.boundary_step_count += path_multiplier;

    const derivation = self.projectors.projectorDerivation(boundary_projector) orelse
    ↪ return error.UnknownProjector;
    if (derivation.status == .expandable_terms) {
      audit.expandable_boundary_step_count += path_multiplier;
    } else {
      all_expandable = false;
      audit.missing_boundary_step_count += path_multiplier;
      if (audit.first_missing_projector == null) {
        audit.first_missing_projector = boundary_projector;
        audit.first_missing_boundary = handle;
      }
    }
  }
}

```

```

    }
}
audit.distinct_boundary_projector_count = @intCast(distinct.count());
return all_expandable;
}

fn streamPathExpansion(self: *Impl, request: coupling.InvariantBasisRequest, path:
↳ coupling.CouplingPath, options: rendering.RenderOptions, filter:
↳ rendering.ExpansionFilter, sink: anytype, scratch: *ExpansionScratch)
↳ !rendering.ExpansionAudit {
    const steps = self.couplings.pathSteps(path);
    scratch.atoms.clearRetainingCapacity();
    const boundary_projector_count = try
↳ self.appendBoundaryRealizationAtoms(&scratch.atoms, request);

    var audit: rendering.ExpansionAudit = .{
        .invariants = 1,
        .max_frontier = @max(1, @as(u32, @intCast(steps.len))),
    };
    audit.boundary_projector_atoms = boundary_projector_count;
    if (boundary_projector_count != 0 and filter.rejectsAll()) {
        audit.recordDecision(.reject);
        return audit;
    }

    if (options.projectors != .named) {
        if (boundary_projector_count == 0) {
            if (try self.streamLocalFormulaPathTerms(steps, options, filter, sink, &audit,
↳ scratch)) return audit;
        } else {
            try self.streamBoundaryFormulaTerms(request, steps, options, filter, sink,
↳ &audit, scratch);
            return audit;
        }
    }

    const explicit_formula = try self.appendExplicitPathAtoms(&scratch.atoms, steps);
    const local_projector_count = if (explicit_formula) 0 else try
↳ self.appendNamedProjectorAtoms(&scratch.atoms, steps);
    audit.local_projector_atoms = local_projector_count;
    const term: rendering.SymbolicTerm = .{
        .coefficient = rendering.rationalOne(),
        .atoms = scratch.atoms.items,
    };
    const decision = try filter.decide(term);
    audit.recordDecision(decision);
    switch (decision) {
        .reject => return audit,
        .accept, .undecided => {
            if (options.projectors == .named) {
                try emitRenderedTerm(self.allocator, options, sink, term, &audit,
↳ &scratch.lowered_atoms);
                return audit;
            }
            if (!explicit_formula and steps.len != 0) {
                try self.requirePathProjectorExpansion(steps, options.projectors);
                return error.ProjectorExpansionNotImplemented;
            }
            try emitRenderedTerm(self.allocator, options, sink, term, &audit,
↳ &scratch.lowered_atoms);
            return audit;
        },
    }
}
}

```

```

fn appendBoundaryRealizationAtoms(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), request: coupling.InvariantBasisRequest) !u32
↳ {
    var appended: u32 = 0;
    for (request.external_legs, 0..) |leg, leg_index| {
        const handle = switch (leg.realization) {
            .handle => |handle| handle,
            .primitive_irrep, .registered_name => continue,
        };
        const spec = self.realizations.realizationSpecFor(handle) orelse return
↳ error.UnknownRealization;
        if (spec.ambient.len == 0) continue;

        const operator_id = try self.boundaryRealizationProjector(handle);
        const first_index = externalIndexOffset(request, leg_index);
        try atoms.append(self allocator, .{ .named_operator = .{
            .kind = .projector,
            .operator_id = operator_id,
            .first_index = first_index,
            .index_len = @intCast(leg.indices.len),
        } });
        appended += 1;
    }
    return appended;
}

fn appendExplicitPathAtoms(self: *Impl, atoms: *std.ArrayList(rendering.SymbolicAtom),
↳ steps: []const coupling.CouplingStep) !bool {
    if (try self.appendSpecialExplicitPathAtoms(atoms, steps)) return true;
    return self.appendLocalFormulaPathAtoms(atoms, steps);
}

fn appendSpecialExplicitPathAtoms(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), steps: []const coupling.CouplingStep) !bool {
    if (steps.len == 2 and self.isA2FundamentalCubicStep(steps[0], steps[1])) {
        try self.ensurePathFormulaPrograms(steps);
        try atoms.append(self allocator, .{ .epsilon = .{ .block = 0 } });
        return true;
    }
    if (steps.len == 2 and self.isE6FundamentalCubicStep(steps[0], steps[1])) {
        try self.ensurePathFormulaPrograms(steps);
        try atoms.append(self allocator, .{ .structure_constant = .{
            .operator_id = steps[0].projector,
            .first = 0,
            .second = 1,
            .third = 2,
            .family = .e6,
            .rank = 6,
        } });
        return true;
    }
    if (steps.len == 2 and self.isSimpleAdjointCubicStep(steps[0], steps[1])) {
        const simple = self.stepSimpleAlgebra(steps[0]) orelse return
↳ error.UnsupportedAlgebraFamily;
        try self.ensurePathFormulaPrograms(steps);
        try atoms.append(self allocator, .{ .structure_constant = .{
            .operator_id = steps[0].projector,
            .first = 0,
            .second = 1,
            .third = 2,
            .family = simple.family,
            .rank = simple.rank,
        } });
    }
}

```

```

        return true;
    }
    return false;
}

fn ensurePathFormulaPrograms(self: *Impl, steps: []const coupling.CouplingStep) !void {
    for (steps) |step| {
        _ = try self.projectors.ensureFormulaProgram(step.projector);
    }
}

fn appendLocalFormulaPathAtoms(self: *Impl, atoms: *std.ArrayList(rendering.SymbolicAtom),
    ↪ steps: []const coupling.CouplingStep) !bool {
    if (steps.len == 0) return false;
    const scratch_base: u32 = @intCast(steps.len + 1);
    var previous_formula_atom: ?rendering.SymbolicAtom = null;
    for (steps, 0..) |step, step_index| {
        const left_index: rendering.IndexRef = if (step_index == 0) 0 else scratch_base +
            ↪ @as(u32, @intCast(step_index - 1));
        const right_index: rendering.IndexRef = @intCast(step_index + 1);
        const output_index: rendering.IndexRef = scratch_base + @as(u32,
            ↪ @intCast(step_index));
        const atom_index = atoms.items.len;
        if (!try self.appendLocalFormulaStepAtom(atoms, step, left_index, right_index,
            ↪ output_index)) return false;
        if (atoms.items.len == atom_index) return error.InvalidProjectorExpansion;
        const current_formula_atom = atoms.items[atom_index];
        if (previous_formula_atom |previous| {
            try appendCliffordProductAtoms(self.allocator, atoms, previous,
                ↪ current_formula_atom);
        }
        previous_formula_atom = current_formula_atom;
    }
    return true;
}

fn appendLocalFormulaStepAtom(self: *Impl, atoms: *std.ArrayList(rendering.SymbolicAtom),
    ↪ step: coupling.CouplingStep, left_index: rendering.IndexRef, right_index:
    ↪ rendering.IndexRef, output_index: rendering.IndexRef) !bool {
    const formula_kind = self.localProductFormulaKind(step);
    if (formula_kind == .named_only) return false;
    const cached_kind = try self.projectors.ensureFormulaProgram(step.projector);
    if (cached_kind != formula_kind) return error.InvalidProjectorExpansion;
    return self.appendLocalFormulaStepAtomForKind(atoms, step, left_index, right_index,
        ↪ output_index, cached_kind);
}

fn appendLocalFormulaStepAtomForKind(self: *Impl, atoms:
    ↪ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
    ↪ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef,
    ↪ formula_kind: projector.ProjectorFormulaKind) !bool {
    return switch (formula_kind) {
        .identity => {
            try atoms.append(self.allocator, .{ .identity_route = .{
                .operator_id = step.projector,
                .left = left_index,
                .right = right_index,
                .output = output_index,
            } });
            return true;
        },
        .orthogonal_vector_metric => {
            try atoms.append(self.allocator, .{ .metric_pair = .{
                .left = left_index,

```

```

        .right = right_index,
    } });
    return true;
},
.orthogonal_spinor_pair => {
    const simple = self.stepSimpleAlgebra(step) orelse return
    ↪ error.InvalidSpinorPairChannel;
    const left = self.representations.irrepDynkin(step.left) orelse return
    ↪ error.InvalidSpinorPairChannel;
    const right = self.representations.irrepDynkin(step.right) orelse return
    ↪ error.InvalidSpinorPairChannel;
    try atoms.append(self.allocator, .{ .spinor_pair = .{
        .operator_id = step.projector,
        .spinor_left = left_index,
        .spinor_right = right_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .chirality_left = @intFromEnum(rendering.gammaChiralityTag(simple, left)),
        .chirality_right = @intFromEnum(rendering.gammaChiralityTag(simple,
            ↪ right)),
    } });
    return true;
},
.orthogonal_spinor_form_channel => return
↪ self.appendOrthogonalSpinorFormStepAtom(atoms, step, left_index, right_index,
↪ output_index),
.orthogonal_form_pair => {
    try atoms.append(self.allocator, .{ .generalized_delta = .{
        .upper = left_index,
        .lower = right_index,
    } });
    return true;
},
.orthogonal_form_spinor_channel => return
↪ self.appendOrthogonalFormSpinorStepAtom(atoms, step, left_index, right_index,
↪ output_index),
.cartan_product_channel => return
↪ self.appendSpinorTowerCartanProductStepAtom(atoms, step, left_index,
↪ right_index, output_index),
.orthogonal_structural_projection => return
↪ self.appendLocalStructuralExpression(atoms, step, left_index, right_index,
↪ output_index),
.orthogonal_spinor_tower_form_channel => return
↪ self.appendOrthogonalSpinorTowerFormStepAtom(atoms, step, left_index,
↪ right_index, output_index),
.orthogonal_spinor_tower_middle_form_channel => return
↪ self.appendOrthogonalSpinorTowerMiddleFormStepAtom(atoms, step, left_index,
↪ right_index, output_index),
.orthogonal_spinor_tower_opposite_form_channel => return
↪ self.appendOrthogonalSpinorTowerOppositeFormStepAtom(atoms, step, left_index,
↪ right_index, output_index),
.orthogonal_spinor_tower_opposite_lower_channel => return
↪ self.appendOrthogonalSpinorTowerOppositeLowerStepAtom(atoms, step, left_index,
↪ right_index, output_index),
.orthogonal_tensor_spinor_tower_channel => return
↪ self.appendOrthogonalTensorSpinorTowerStepAtom(atoms, step, left_index,
↪ right_index, output_index),
.orthogonal_tensor_spinor_form_power_channel => return
↪ self.appendOrthogonalTensorSpinorFormPowerStepAtom(atoms, step, left_index,
↪ right_index, output_index),
.orthogonal_tensor_spinor_tower_raise_channel => return
↪ self.appendOrthogonalTensorSpinorTowerRaiseStepAtom(atoms, step, left_index,
↪ right_index, output_index),

```

```

.orthogonal_tensor_spinor_opposite_tower_lower_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeTowerLowerStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_opposite_shift_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeShiftStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_opposite_all_shift_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeAllShiftStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_opposite_form_add_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeFormAddStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_opposite_rank_split_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeRankSplitStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_shift_channel => return
↳ self.appendOrthogonalTensorSpinorShiftStepAtom(atoms, step, left_index,
↳ right_index, output_index),
.orthogonal_tensor_spinor_rank_wrap_shift_channel => return
↳ self.appendOrthogonalTensorSpinorRankWrapShiftStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_rank_split_channel => return
↳ self.appendOrthogonalTensorSpinorRankSplitStepAtom(atoms, step, left_index,
↳ right_index, output_index),
.orthogonal_tensor_spinor_terminal_channel => return
↳ self.appendOrthogonalTensorSpinorTerminalStepAtom(atoms, step, left_index,
↳ right_index, output_index),
.orthogonal_tensor_spinor_opposite_terminal_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeTerminalStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_opposite_form_add_terminal_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeFormAddTerminalStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_opposite_form_add_shift_terminal_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeFormAddShiftTerminalStepAtom(atoms,
↳ step, left_index, right_index, output_index),
.orthogonal_tensor_spinor_opposite_rank_split_terminal_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeRankSplitTerminalStepAtom(atoms,
↳ step, left_index, right_index, output_index),
.orthogonal_tensor_spinor_opposite_shift_terminal_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeShiftTerminalStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_rank_wrap_terminal_channel => return
↳ self.appendOrthogonalTensorSpinorRankWrapTerminalStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_rank_split_terminal_channel => return
↳ self.appendOrthogonalTensorSpinorRankSplitTerminalStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_form_terminal_channel => return
↳ self.appendOrthogonalTensorSpinorFormTerminalStepAtom(atoms, step, left_index,
↳ right_index, output_index),
.orthogonal_tensor_spinor_form_add_terminal_channel => return
↳ self.appendOrthogonalTensorSpinorFormAddTerminalStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_form_power_terminal_channel => return
↳ self.appendOrthogonalTensorSpinorFormPowerTerminalStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_form_tower_channel => return
↳ self.appendOrthogonalTensorSpinorFormTowerStepAtom(atoms, step, left_index,
↳ right_index, output_index),
.orthogonal_tensor_spinor_form_tower_terminal_channel => return
↳ self.appendOrthogonalTensorSpinorFormTowerTerminalStepAtom(atoms, step,
↳ left_index, right_index, output_index),

```

```

.orthogonal_tensor_spinor_form_tower_remove_shift_channel => return
↳ self.appendOrthogonalTensorSpinorFormTowerRemoveShiftStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_form_tower_shift_down_channel => return
↳ self.appendOrthogonalTensorSpinorFormTowerShiftDownStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_form_tower_all_shift_down_channel => return
↳ self.appendOrthogonalTensorSpinorFormTowerAllShiftDownStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_opposite_form_tower_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeFormTowerStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_opposite_form_tower_terminal_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeFormTowerTerminalStepAtom(atoms,
↳ step, left_index, right_index, output_index),
.orthogonal_tensor_spinor_opposite_form_tower_merge_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeFormTowerMergeStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_opposite_form_tower_remove_shift_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeFormTowerRemoveShiftStepAtom(atoms,
↳ step, left_index, right_index, output_index),
.orthogonal_tensor_spinor_opposite_form_tower_shift_down_channel => return
↳ self.appendOrthogonalTensorSpinorOppositeFormTowerShiftDownStepAtom(atoms,
↳ step, left_index, right_index, output_index),
.orthogonal_tensor_form_spinor_channel => return
↳ self.appendOrthogonalTensorFormSpinorStepAtom(atoms, step, left_index,
↳ right_index, output_index),
.orthogonal_tensor_form_spinor_preserve_channel => return
↳ self.appendOrthogonalTensorFormSpinorPreserveStepAtom(atoms, step, left_index,
↳ right_index, output_index),
.orthogonal_tensor_form_spinor_shift_channel => return
↳ self.appendOrthogonalTensorFormSpinorShiftStepAtom(atoms, step, left_index,
↳ right_index, output_index),
.orthogonal_tensor_form_spinor_remove_shift_down_channel => return
↳ self.appendOrthogonalTensorFormSpinorRemoveShiftDownStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_form_spinor_shift_down_channel => return
↳ self.appendOrthogonalTensorFormSpinorShiftDownStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_form_spinor_shift_down_two_channel => return
↳ self.appendOrthogonalTensorFormSpinorShiftDownTwoStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_form_spinor_mixed_shift_down_channel => return
↳ self.appendOrthogonalTensorFormSpinorMixedShiftDownStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_form_spinor_all_shift_down_channel => return
↳ self.appendOrthogonalTensorFormSpinorAllShiftDownStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_form_spinor_shift_down_any_channel => return
↳ self.appendOrthogonalTensorFormSpinorShiftDownAnyStepAtom(atoms, step,
↳ left_index, right_index, output_index),
.orthogonal_tensor_spinor_middle_form_channel => return
↳ self.appendOrthogonalTensorSpinorMiddleFormStepAtom(atoms, step, left_index,
↳ right_index, output_index),
else => false,
};
}

fn appendOrthogonalSpinorFormStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
  const simple = self.stepSimpleAlgebra(step) orelse return error.InvalidGammaChannel;

```



```

const left = self.representations.irrepDynkin(step.left) orelse return
↳ error.InvalidGammaChannel;
if (self.isOrthogonalSpinorSpinorVectorStep(step)) {
    try atoms.append(self.allocator, .{ .gamma_matrix = .{
        .operator_id = step.projector,
        .spinor_left = left_index,
        .spinor_right = right_index,
        .vector = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .rank = 1,
        .chirality = @intFromEnum(rendering.gammaChiralityTag(simple, left)),
        .duality = .none,
    } });
    return true;
}
const form_info = self.orthogonalSpinorSpinorFormStep(step) orelse return false;
try atoms.append(self.allocator, .{ .gamma_form = .{
    .operator_id = step.projector,
    .spinor_left = left_index,
    .spinor_right = right_index,
    .form = output_index,
    .orthogonal_dimension = rendering.orthogonalDimension(simple),
    .rank = form_info.rank,
    .chirality = @intFromEnum(rendering.gammaChiralityTag(simple, left)),
    .duality = form_info.duality,
} });
return true;
}

fn appendOrthogonalFormSpinorStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalFormSpinorStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidGammaActionChannel;
    const spinor = self.representations.irrepDynkin(step.right) orelse return
↳ error.InvalidGammaActionChannel;
    try atoms.append(self.allocator, .{ .gamma_action = .{
        .operator_id = step.projector,
        .form = left_index,
        .spinor_input = right_index,
        .spinor_output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .rank = info.rank,
        .chirality = @intFromEnum(rendering.gammaChiralityTag(simple, spinor)),
        .duality = info.duality,
    } });
    return true;
}

fn appendSpinorTowerCartanProductStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidCartanProductChannel;
    const left = self.representations.irrepDynkin(step.left) orelse return
↳ error.InvalidCartanProductChannel;
    const right = self.representations.irrepDynkin(step.right) orelse return
↳ error.InvalidCartanProductChannel;
    const output = self.representations.irrepDynkin(step.output) orelse return
↳ error.InvalidCartanProductChannel;
    const left_tower = spinorTowerInfo(simple, left) orelse return false;

```



```

const right_tower = spinorTowerInfo(simple, right) orelse return false;
const output_tower = spinorTowerInfo(simple, output) orelse return false;
if (left_tower.chirality != right_tower.chirality or left_tower.chirality !=
    ⇨ output_tower.chirality) return false;
if (output_tower.power != left_tower.power + right_tower.power) return false;
var left_slot: u16 = 0;
while (left_slot < left_tower.power) : (left_slot += 1) {
    try atoms.append(self.allocator, .{ .spinor_index_delta = .{
        .operator_id = step.projector,
        .source_tower = left_index,
        .output_tower = output_index,
        .source_slot = left_slot,
        .output_slot = left_slot,
        .chirality = @intFromEnum(output_tower.chirality),
    } });
}
var right_slot: u16 = 0;
while (right_slot < right_tower.power) : (right_slot += 1) {
    try atoms.append(self.allocator, .{ .spinor_index_delta = .{
        .operator_id = step.projector,
        .source_tower = right_index,
        .output_tower = output_index,
        .source_slot = right_slot,
        .output_slot = left_tower.power + right_slot,
        .chirality = @intFromEnum(output_tower.chirality),
    } });
}
return true;
}

fn appendLocalStructuralExpression(self: *Impl, atoms:
    ⇨ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
    ⇨ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
    ⇨ !bool {
    const term_count = try self.localStructuralTermCount(step, left_index, right_index,
        ⇨ output_index);
    if (term_count != 1) return false;
    _ = try self.appendLocalStructuralTerm(atoms, step, left_index, right_index,
        ⇨ output_index, 0);
    return true;
}

fn appendOrthogonalSpinorTowerFormStepAtom(self: *Impl, atoms:
    ⇨ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
    ⇨ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
    ⇨ !bool {
    const info = self.orthogonalSpinorTowerFormStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
    ⇨ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ⇨ error.InvalidTensorSpinorChannel,
        .form_rank = info.forms.first_rank,
        .form_count = info.forms.count,
        .form_mask = info.forms.mask,
        .form_profile = info.forms.profile,
        .tower_power = info.tower_power,
        .chirality = @intFromEnum(info.chirality),
        .duality = info.forms.duality,
    })
}

```

```

    });
    return true;
}

fn appendOrthogonalSpinorTowerMiddleFormStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalSpinorTowerMiddleFormStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_form_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .input_form_profile = 0,
        .input_form_mask = 0,
        .output_form_profile = 0,
        .output_form_count = 1,
        .output_form_rank = info.rank,
        .output_duality = info.duality,
        .chirality = 0,
    });
    return true;
}

fn appendOrthogonalSpinorTowerOppositeFormStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalSpinorTowerOppositeFormStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .form_rank = info.forms.first_rank,
        .form_count = info.forms.count,
        .form_mask = info.forms.mask,
        .form_profile = info.forms.profile,
        .tower_power = info.tower_power,
        .chirality = @intFromEnum(info.chirality),
        .duality = info.forms.duality,
    });
    return true;
}

fn appendOrthogonalSpinorTowerOppositeLowerStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalSpinorTowerOppositeLowerStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,

```

```

        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ⇨ error.InvalidTensorSpinorChannel,
        .form_rank = 0,
        .form_count = 0,
        .form_mask = 0,
        .form_profile = 0,
        .tower_power = info.power,
        .chirality = @intFromEnum(info.chirality),
    });
    return true;
}

fn appendOrthogonalTensorSpinorTowerStepAtom(self: *Impl, atoms:
⇨ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
⇨ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
⇨ !bool {
    const info = self.orthogonalTensorSpinorTowerStep(step) orelse return false;
    const input = self.orthogonalTensorSpinorLeftInfo(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
    ⇨ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ⇨ error.InvalidTensorSpinorChannel,
        .input_form_profile = input.forms.profile,
        .input_form_count = input.forms.total_power,
        .input_tower_power = input.tower_power,
        .form_rank = info.forms.first_rank,
        .form_count = info.forms.count,
        .form_mask = info.forms.mask,
        .form_profile = info.forms.profile,
        .tower_power = info.tower_power,
        .chirality = @intFromEnum(info.chirality),
        .duality = info.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorFormPowerStepAtom(self: *Impl, atoms:
⇨ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
⇨ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
⇨ !bool {
    const info = self.orthogonalTensorSpinorFormPowerStep(step) orelse return false;
    const input = self.orthogonalTensorSpinorLeftInfo(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
    ⇨ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ⇨ error.InvalidTensorSpinorChannel,
        .input_form_profile = input.forms.profile,
        .input_form_count = input.forms.total_power,
        .input_tower_power = input.tower_power,

```

```

        .form_rank = info.forms.first_rank,
        .form_count = info.forms.count,
        .form_mask = info.forms.mask,
        .form_profile = info.forms.profile,
        .tower_power = info.tower_power,
        .chirality = @intFromEnum(info.chirality),
        .duality = info.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorTowerRaiseStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorTowerRaiseStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorOppositeTowerLowerStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorOppositeTowerLowerStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,

```

```

        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorOppositeShiftStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorOppositeShiftStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorOppositeAllShiftStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorOppositeAllShiftStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

```

```

fn appendOrthogonalTensorSpinorOppositeFormAddStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorOppositeFormAddStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorOppositeRankSplitStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorOppositeRankSplitStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorShiftStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {

```

```

const info = self.orthogonalTensorSpinorShiftStep(step) orelse return false;
const input = self.orthogonalTensorSpinorLeftInfo(step) orelse return false;
const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
_ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
    .operator_id = step.projector,
    .left = left_index,
    .right = right_index,
    .output = output_index,
    .orthogonal_dimension = rendering.orthogonalDimension(simple),
    .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
    .input_form_profile = input.forms.profile,
    .input_form_count = input.forms.total_power,
    .input_tower_power = input.tower_power,
    .form_rank = info.forms.first_rank,
    .form_count = info.forms.count,
    .form_mask = info.forms.mask,
    .form_profile = info.forms.profile,
    .tower_power = info.tower_power,
    .chirality = @intFromEnum(info.chirality),
    .duality = info.forms.duality,
});
return true;
}

fn appendOrthogonalTensorSpinorRankWrapShiftStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
const info = self.orthogonalTensorSpinorRankWrapShiftStep(step) orelse return false;
const input = self.orthogonalTensorSpinorLeftInfo(step) orelse return false;
const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
_ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
    .operator_id = step.projector,
    .left = left_index,
    .right = right_index,
    .output = output_index,
    .orthogonal_dimension = rendering.orthogonalDimension(simple),
    .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
    .input_form_profile = input.forms.profile,
    .input_form_count = input.forms.total_power,
    .input_tower_power = input.tower_power,
    .form_rank = info.forms.first_rank,
    .form_count = info.forms.count,
    .form_mask = info.forms.mask,
    .form_profile = info.forms.profile,
    .tower_power = info.tower_power,
    .chirality = @intFromEnum(info.chirality),
    .duality = info.forms.duality,
});
return true;
}

fn appendOrthogonalTensorSpinorRankSplitStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
const info = self.orthogonalTensorSpinorRankSplitStep(step) orelse return false;
const input = self.orthogonalTensorSpinorLeftInfo(step) orelse return false;
const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;

```



```

    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ↪ error.InvalidTensorSpinorChannel,
        .input_form_profile = input.forms.profile,
        .input_form_count = input.forms.total_power,
        .input_tower_power = input.tower_power,
        .form_rank = info.forms.first_rank,
        .form_count = info.forms.count,
        .form_mask = info.forms.mask,
        .form_profile = info.forms.profile,
        .tower_power = info.tower_power,
        .chirality = @intFromEnum(info.chirality),
        .duality = info.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorTerminalStepAtom(self: *Impl, atoms:
    ↪ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
    ↪ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
    ↪ !bool {
    const info = self.orthogonalTensorSpinorTerminalStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
    ↪ error.InvalidTensorSpinorChannel;
    try appendTensorFormProjectionGammaFallback(&self.tensor_form_projection_programs,
    ↪ atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ↪ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_mask = info.input.forms.mask,
        .output_form_profile = info.output.profile,
        .output_form_count = info.output.total_power,
        .output_form_rank = info.output.first_rank,
        .output_duality = info.output.duality,
        .chirality = @intFromEnum(info.input.chirality),
    });
    return true;
}

fn appendOrthogonalTensorSpinorOppositeTerminalStepAtom(self: *Impl, atoms:
    ↪ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
    ↪ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
    ↪ !bool {
    const info = self.orthogonalTensorSpinorOppositeTerminalStep(step) orelse return
    ↪ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
    ↪ error.InvalidTensorSpinorChannel;
    try appendTensorFormProjectionGammaFallback(&self.tensor_form_projection_programs,
    ↪ atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),

```



```

        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ↪ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_mask = info.input.forms.mask,
        .output_form_profile = info.output.profile,
        .output_form_count = info.output.total_power,
        .output_form_rank = info.output.first_rank,
        .output_duality = info.output.duality,
        .chirality = @intFromEnum(info.input.chirality),
    });
    return true;
}

fn appendOrthogonalTensorSpinorOppositeFormAddTerminalStepAtom(self: *Impl, atoms:
    ↪ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
    ↪ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
    ↪ !bool {
    const info = self.orthogonalTensorSpinorOppositeFormAddTerminalStep(step) orelse
    ↪ return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
    ↪ error.InvalidTensorSpinorChannel;
    try appendTensorFormProjectionGammaFallback(&self.tensor_form_projection_programs,
    ↪ atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ↪ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_mask = info.input.forms.mask,
        .output_form_profile = info.output.profile,
        .output_form_count = info.output.total_power,
        .output_form_rank = info.output.first_rank,
        .output_duality = info.output.duality,
        .chirality = @intFromEnum(info.input.chirality),
    });
    return true;
}

fn appendOrthogonalTensorSpinorOppositeFormAddShiftTerminalStepAtom(self: *Impl, atoms:
    ↪ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
    ↪ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
    ↪ !bool {
    const info = self.orthogonalTensorSpinorOppositeFormAddShiftTerminalStep(step) orelse
    ↪ return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
    ↪ error.InvalidTensorSpinorChannel;
    try appendTensorFormProjectionGammaFallback(&self.tensor_form_projection_programs,
    ↪ atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ↪ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_mask = info.input.forms.mask,
        .output_form_profile = info.output.profile,
        .output_form_count = info.output.total_power,
        .output_form_rank = info.output.first_rank,
        .output_duality = info.output.duality,
    });
}

```

```

        .chirality = @intFromEnum(info.input.chirality),
    });
    return true;
}

fn appendOrthogonalTensorSpinorOppositeRankSplitTerminalStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorOppositeRankSplitTerminalStep(step) orelse
↳ return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    try appendTensorFormProjectionGammaFallback(&self.tensor_form_projection_programs,
↳ atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_mask = info.input.forms.mask,
        .output_form_profile = info.output.profile,
        .output_form_count = info.output.total_power,
        .output_form_rank = info.output.first_rank,
        .output_duality = info.output.duality,
        .chirality = @intFromEnum(info.input.chirality),
    });
    return true;
}

fn appendOrthogonalTensorSpinorOppositeShiftTerminalStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorOppositeShiftTerminalStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    try appendTensorFormProjectionGammaFallback(&self.tensor_form_projection_programs,
↳ atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_mask = info.input.forms.mask,
        .output_form_profile = info.output.profile,
        .output_form_count = info.output.total_power,
        .output_form_rank = info.output.first_rank,
        .output_duality = info.output.duality,
        .chirality = @intFromEnum(info.input.chirality),
    });
    return true;
}

```

```

fn appendOrthogonalTensorSpinorRankWrapTerminalStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorRankWrapTerminalStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    try appendTensorFormProjectionGammaFallback(&self.tensor_form_projection_programs,
↳ atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_mask = info.input.forms.mask,
        .output_form_profile = info.output.profile,
        .output_form_count = info.output.total_power,
        .output_form_rank = info.output.first_rank,
        .output_duality = info.output.duality,
        .chirality = @intFromEnum(info.input.chirality),
    });
    return true;
}

fn appendOrthogonalTensorSpinorRankSplitTerminalStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorRankSplitTerminalStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    try appendTensorFormProjectionGammaFallback(&self.tensor_form_projection_programs,
↳ atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_mask = info.input.forms.mask,
        .output_form_profile = info.output.profile,
        .output_form_count = info.output.total_power,
        .output_form_rank = info.output.first_rank,
        .output_duality = info.output.duality,
        .chirality = @intFromEnum(info.input.chirality),
    });
    return true;
}

fn appendOrthogonalTensorSpinorFormTerminalStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorFormTerminalStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;

```

```

const right = self.representations.irrepDynkin(step.right) orelse return
↳ error.InvalidTensorSpinorChannel;
try atoms.append(self.allocator, .{ .spinor_pair = .{
    .operator_id = step.projector,
    .spinor_left = tensorSpinorSpinorIndex(left_index),
    .spinor_right = right_index,
    .orthogonal_dimension = rendering.orthogonalDimension(simple),
    .chirality_left = @intFromEnum(info.input.chirality),
    .chirality_right = @intFromEnum(rendering.gammaChiralityTag(simple, right)),
} });
try atoms.append(self.allocator, .{ .generalized_delta = .{
    .upper = left_index,
    .lower = output_index,
} });
return true;
}

fn appendOrthogonalTensorSpinorFormAddTerminalStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
const info = self.orthogonalTensorSpinorFormAddTerminalStep(step) orelse return false;
const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
try appendTensorFormProjectionGammaFallback(&self.tensor_form_projection_programs,
↳ atoms, .{
    .operator_id = step.projector,
    .left = left_index,
    .right = right_index,
    .output = output_index,
    .orthogonal_dimension = rendering.orthogonalDimension(simple),
    .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
    .input_form_profile = info.input.forms.profile,
    .input_form_mask = info.input.forms.mask,
    .output_form_profile = info.output.profile,
    .output_form_count = info.output.total_power,
    .output_form_rank = info.output.first_rank,
    .output_duality = info.output.duality,
    .chirality = @intFromEnum(info.input.chirality),
});
return true;
}

fn appendOrthogonalTensorSpinorFormPowerTerminalStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
const info = self.orthogonalTensorSpinorFormPowerTerminalStep(step) orelse return
↳ false;
const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
try appendTensorFormProjectionGammaFallback(&self.tensor_form_projection_programs,
↳ atoms, .{
    .operator_id = step.projector,
    .left = left_index,
    .right = right_index,
    .output = output_index,
    .orthogonal_dimension = rendering.orthogonalDimension(simple),
    .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
    .input_form_profile = info.input.forms.profile,
    .input_form_mask = info.input.forms.mask,
    .output_form_profile = info.output.profile,

```

```

        .output_form_count = info.output.total_power,
        .output_form_rank = info.output.first_rank,
        .output_duality = info.output.duality,
        .chirality = @intFromEnum(info.input.chirality),
    });
    return true;
}

fn appendOrthogonalTensorSpinorFormTowerStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorFormTowerStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorFormTowerTerminalStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorFormTowerTerminalStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = 0,
        .form_count = 0,
        .form_mask = 0,
        .form_profile = 0,
        .tower_power = info.output.power,
        .chirality = @intFromEnum(info.output.chirality),
    });
    return true;
}

```

```

}

fn appendOrthogonalTensorSpinorFormTowerRemoveShiftStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorFormTowerRemoveShiftStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorFormTowerShiftDownStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorFormTowerShiftDownStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

```

```

fn appendOrthogonalTensorSpinorFormTowerAllShiftDownStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorFormTowerAllShiftDownStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorOppositeFormTowerStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorSpinorOppositeFormTowerStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorOppositeFormTowerTerminalStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {

```



```

const info = self.orthogonalTensorSpinorOppositeFormTowerTerminalStep(step) orelse
↳ return false;
const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
_ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
    .operator_id = step.projector,
    .left = left_index,
    .right = right_index,
    .output = output_index,
    .orthogonal_dimension = rendering.orthogonalDimension(simple),
    .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
    .input_form_profile = info.input.forms.profile,
    .input_form_count = info.input.forms.total_power,
    .input_tower_power = info.input.tower_power,
    .form_rank = 0,
    .form_count = 0,
    .form_mask = 0,
    .form_profile = 0,
    .tower_power = info.output.power,
    .chirality = @intFromEnum(info.output.chirality),
});
return true;
}

fn appendOrthogonalTensorSpinorOppositeFormTowerMergeStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
const info = self.orthogonalTensorSpinorOppositeFormTowerMergeStep(step) orelse return
↳ false;
const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
_ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
    .operator_id = step.projector,
    .left = left_index,
    .right = right_index,
    .output = output_index,
    .orthogonal_dimension = rendering.orthogonalDimension(simple),
    .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
    .input_form_profile = info.input.forms.profile,
    .input_form_count = info.input.forms.total_power,
    .input_tower_power = info.input.tower_power,
    .form_rank = info.output.forms.first_rank,
    .form_count = info.output.forms.count,
    .form_mask = info.output.forms.mask,
    .form_profile = info.output.forms.profile,
    .tower_power = info.output.tower_power,
    .chirality = @intFromEnum(info.output.chirality),
    .duality = info.output.forms.duality,
});
return true;
}

fn appendOrthogonalTensorSpinorOppositeFormTowerRemoveShiftStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
const info = self.orthogonalTensorSpinorOppositeFormTowerRemoveShiftStep(step) orelse
↳ return false;
const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
_ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{

```



```

        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ↪ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorOppositeFormTowerShiftDownStepAtom(self: *Impl, atoms:
    ↪ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
    ↪ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
    ↪ !bool {
    const info = self.orthogonalTensorSpinorOppositeFormTowerShiftDownStep(step) orelse
    ↪ return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
    ↪ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ↪ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_count = info.input.forms.total_power,
        .input_tower_power = info.input.tower_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorSpinorMiddleFormStepAtom(self: *Impl, atoms:
    ↪ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
    ↪ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
    ↪ !bool {
    const info = self.orthogonalTensorSpinorMiddleFormStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
    ↪ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_form_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),

```

```

        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ↪ error.InvalidTensorSpinorChannel,
        .input_form_profile = info.input.forms.profile,
        .input_form_mask = info.input.forms.mask,
        .output_form_profile = 0,
        .output_form_count = 1,
        .output_form_rank = info.output.rank,
        .output_duality = info.output.duality,
        .chirality = @intFromEnum(info.input.chirality),
    });
    return true;
}

fn appendOrthogonalTensorFormSpinorStepAtom(self: *Impl, atoms:
    ↪ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
    ↪ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
    ↪ !bool {
    const info = self.orthogonalTensorFormSpinorStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
    ↪ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ↪ error.InvalidTensorSpinorChannel,
        .left_has_spinor = false,
        .input_form_profile = info.input.profile,
        .input_form_count = info.input.total_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorFormSpinorPreserveStepAtom(self: *Impl, atoms:
    ↪ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
    ↪ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
    ↪ !bool {
    const info = self.orthogonalTensorFormSpinorPreserveStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
    ↪ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
        ↪ error.InvalidTensorSpinorChannel,
        .left_has_spinor = false,
        .input_form_profile = info.input.profile,
        .input_form_count = info.input.total_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,

```

```

        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorFormSpinorShiftStepAtom(self: *Impl, atoms:
    ↪ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
    ↪ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
    ↪ !bool {
    const spec = try self.orthogonalTensorFormSpinorShiftProjectionSpec(step, left_index,
    ↪ right_index, output_index) orelse return false;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, spec);
    return true;
}

fn orthogonalTensorFormSpinorShiftProjectionSpec(self: *Impl, step: coupling.CouplingStep,
    ↪ left_index: rendering.IndexRef, right_index: rendering.IndexRef, output_index:
    ↪ rendering.IndexRef) !?projector_constructor.TensorSpinorProjectionSpec {
    const info = self.orthogonalTensorFormSpinorShiftStep(step) orelse return null;
    const simple = self.stepSimpleAlgebra(step) orelse return
    ↪ error.InvalidTensorSpinorChannel;
    return .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
    ↪ error.InvalidTensorSpinorChannel,
        .left_has_spinor = false,
        .input_form_profile = info.input.profile,
        .input_form_count = info.input.total_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    };
}

fn appendOrthogonalTensorFormSpinorRemoveShiftDownStepAtom(self: *Impl, atoms:
    ↪ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
    ↪ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
    ↪ !bool {
    const info = self.orthogonalTensorFormSpinorRemoveShiftDownStep(step) orelse return
    ↪ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
    ↪ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
    ↪ error.InvalidTensorSpinorChannel,
        .left_has_spinor = false,
        .input_form_profile = info.input.profile,
        .input_form_count = info.input.total_power,
        .form_rank = info.output.forms.first_rank,

```

```

        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorFormSpinorShiftDownStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorFormSpinorShiftDownStep(step) orelse return false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .left_has_spinor = false,
        .input_form_profile = info.input.profile,
        .input_form_count = info.input.total_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorFormSpinorShiftDownTwoStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorFormSpinorShiftDownTwoStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .left_has_spinor = false,
        .input_form_profile = info.input.profile,
        .input_form_count = info.input.total_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
    });
}

```

```

        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorFormSpinorMixedShiftDownStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorFormSpinorMixedShiftDownStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .left_has_spinor = false,
        .input_form_profile = info.input.profile,
        .input_form_count = info.input.total_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn appendOrthogonalTensorFormSpinorAllShiftDownStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorFormSpinorAllShiftDownStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .left_has_spinor = false,
        .input_form_profile = info.input.profile,
        .input_form_count = info.input.total_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

```

```

fn appendOrthogonalTensorFormSpinorShiftDownAnyStepAtom(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !bool {
    const info = self.orthogonalTensorFormSpinorShiftDownAnyStep(step) orelse return
↳ false;
    const simple = self.stepSimpleAlgebra(step) orelse return
↳ error.InvalidTensorSpinorChannel;
    _ = try self.tensor_spinor_projection_programs.appendExpression(atoms, .{
        .operator_id = step.projector,
        .left = left_index,
        .right = right_index,
        .output = output_index,
        .orthogonal_dimension = rendering.orthogonalDimension(simple),
        .right_chirality = self.stepRightSpinorChirality(simple, step) orelse return
↳ error.InvalidTensorSpinorChannel,
        .left_has_spinor = false,
        .input_form_profile = info.input.profile,
        .input_form_count = info.input.total_power,
        .form_rank = info.output.forms.first_rank,
        .form_count = info.output.forms.count,
        .form_mask = info.output.forms.mask,
        .form_profile = info.output.forms.profile,
        .tower_power = info.output.tower_power,
        .chirality = @intFromEnum(info.output.chirality),
        .duality = info.output.forms.duality,
    });
    return true;
}

fn stepSimpleAlgebra(self: *Impl, step: coupling.CouplingStep) ?symmetry.SimpleLieAlgebra
↳ {
    const algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    return self.representations.simpleAlgebra(algebra);
}

fn orthogonalTensorSpinorLeftInfo(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorSpinorInfo {
    const simple = self.stepSimpleAlgebra(step) orelse return null;
    const left = self.representations.irrepDynkin(step.left) orelse return null;
    return orthogonalTensorSpinorInfo(simple, left);
}

fn stepRightSpinorChirality(self: *Impl, simple: symmetry.SimpleLieAlgebra, step:
↳ coupling.CouplingStep) ?u8 {
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    if (!isSpinorDynkin(simple, right)) return null;
    return @intFromEnum(rendering.gammaChiralityTag(simple, right));
}

fn countInvariantPaths(self: *Impl, request: coupling.InvariantBasisRequest) !u128 {
    switch (request.tree_policy) {
        .auto => {},
        .fixed => return error.FixedTreeCountingNotImplemented,
    }

    const target = try self.targetIrrep(request.algebra, request.target);
    if (request.external_legs.len == 0) {
        const singlet = try self.targetIrrep(request.algebra, .singlet);
        return if (target.value == singlet.value) 1 else 0;
    }

    const first = try self.externalLegIrrep(request.external_legs[0]);

```

```

    var memo = std.AutoHashMap(u64, u128).init(self.allocator);
    defer memo.deinit();
    return self.countContinuations(request, first, 1, target, &memo);
}

fn enumerateInvariantPaths(self: *Impl, request: coupling.InvariantBasisRequest)
↪ !PathEnumeration {
    switch (request.tree_policy) {
        .auto => {},
        .fixed => return error.FixedTreeCountingNotImplemented,
    }

    var out = try PathEnumeration.init(self.allocator);
    errdefer out.deinit();
    const target = try self.targetIrrep(request.algebra, request.target);
    if (request.external_legs.len == 0) {
        const singlet = try self.targetIrrep(request.algebra, .singlet);
        if (target.value == singlet.value) {
            try out.paths.append(self.allocator, .{ .step_offset = 0, .step_len = 0 });
        }
        return out;
    }

    var nodes: std.ArrayList(PartialPathNode) = .empty;
    defer nodes.deinit(self.allocator);
    var current: std.ArrayList(u32) = .empty;
    defer current.deinit(self.allocator);
    var next: std.ArrayList(u32) = .empty;
    defer next.deinit(self.allocator);
    var continuation_counts = std.AutoHashMap(u64, u128).init(self.allocator);
    defer continuation_counts.deinit();

    const first = try self.externalLegIrrep(request.external_legs[0]);
    try nodes.append(self.allocator, .{
        .output = first,
        .parent = 0,
        .has_parent = false,
        .step = undefined,
    });
    try current.append(self.allocator, 0);

    for (request.external_legs[1..], 1..) |leg, leg_index| {
        next.clearRetainingCapacity();
        const factor = try self.externalLegIrrep(leg);
        for (current.items) |node_index| {
            const node = nodes.items[node_index];
            const product = try self.decomposeProduct(node.output, factor);
            const terms = self.decompositions.productTerms(product) orelse return
            ↪ error.UnknownProductDecomposition;
            var terms_copy: std.ArrayList(decomposition.ProductTerm) = .empty;
            defer terms_copy.deinit(self.allocator);
            try terms_copy.appendSlice(self.allocator, terms);
            for (terms_copy.items) |term| {
                const suffix_count = try self.countContinuations(request, term.irrep,
                ↪ leg_index + 1, target, &continuation_counts);
                if (suffix_count == 0) continue;
                for (0..term.multiplicity) |copy| {
                    const local_projector = try self.localProductProjector(node.output,
                    ↪ factor, term.irrep, @intCast(copy));
                    const child_index: u32 = @intCast(nodes.items.len);
                    try nodes.append(self.allocator, .{
                        .output = term.irrep,
                        .parent = node_index,
                        .has_parent = true,

```

```

        .step = .{
            .left = node.output,
            .right = factor,
            .output = term.irrep,
            .multiplicity_copy = @intCast(copy),
            .projector = local_projector,
        },
    });
    try next.append(self.allocator, child_index);
}
}
}
current.clearRetainingCapacity();
try current.appendSlice(self.allocator, next.items);
}

var reverse_steps: std.ArrayList(coupling.CouplingStep) = .empty;
defer reverse_steps.deinit(self.allocator);
for (current.items) |node_index| {
    if (nodes.items[node_index].output.value != target.value) continue;
    reverse_steps.clearRetainingCapacity();
    var cursor = node_index;
    while (nodes.items[cursor].has_parent) {
        try reverse_steps.append(self.allocator, nodes.items[cursor].step);
        cursor = nodes.items[cursor].parent;
    }

    const step_offset: u32 = @intCast(out.steps.items.len);
    var i = reverse_steps.items.len;
    while (i > 0) {
        i -= 1;
        try out.steps.append(self.allocator, reverse_steps.items[i]);
    }
    try out.paths.append(self.allocator, .{
        .step_offset = step_offset,
        .step_len = @intCast(reverse_steps.items.len),
    });
}
return out;
}

fn countContinuations(self: *Impl, request: coupling.InvariantBasisRequest, current:
↳ store.IrrepHandle, next_index: usize, target: store.IrrepHandle, memo:
↳ *std.AutoHashMap(u64, u128)) !u128 {
    if (next_index == request.external_legs.len) {
        return if (current.value == target.value) 1 else 0;
    }

    const key = continuationKey(next_index, current);
    if (memo.get(key)) |cached| return cached;

    const factor = try self.externalLegIrrep(request.external_legs[next_index]);
    const product = try self.decomposeProduct(current, factor);
    const terms = self.decompositions.productTerms(product) orelse return
↳ error.UnknownProductDecomposition;
    var terms_copy: std.ArrayList(decomposition.ProductTerm) = .empty;
    defer terms_copy.deinit(self.allocator);
    try terms_copy.appendSlice(self.allocator, terms);

    var total: u128 = 0;
    for (terms_copy.items) |term| {
        const suffix_count = try self.countContinuations(request, term.irrep, next_index +
↳ 1, target, memo);
        if (suffix_count == 0) continue;

```



```

        const contribution = try std.math.mul(u128, suffix_count, term.multiplicity);
        total = try std.math.add(u128, total, contribution);
    }

    try memo.put(key, total);
    return total;
}

fn localProductProjector(self: *Impl, left: store.IrrepHandle, right: store.IrrepHandle,
↳ output: store.IrrepHandle, multiplicity_copy: u16) !projector.ProjectorId {
    const derivation = self.localProductProjectorDerivation(left, right, output,
↳ multiplicity_copy);
    return self.projectors.internProjector(.{
        .role = .{ .product_channel = .{
            .left = left,
            .right = right,
            .output = output,
            .multiplicity_copy = multiplicity_copy,
        } },
        .convention = .{
            .render_mode = .named,
            .signature = self.options.default_signature,
            .normalization_id = 0,
        },
        }, derivation);
}

fn localProductProjectorDerivation(self: *Impl, left: store.IrrepHandle, right:
↳ store.IrrepHandle, output: store.IrrepHandle, multiplicity_copy: u16)
↳ projector.ProjectorDerivation {
    const step: coupling.CouplingStep = .{
        .left = left,
        .right = right,
        .output = output,
        .multiplicity_copy = multiplicity_copy,
        .projector = 0,
    };
    const formula_kind = self.localProductFormulaKind(step);
    if (formula_kind == .named_only) {
        return .{
            .kind = .highest_weight_solver,
            .source = .highest_weight_solver,
            .status = .expandable_named,
        };
    }
    const formula_audit = self.localProductFormulaAudit(step, formula_kind);
    return .{
        .kind = .backend_specific_verified,
        .source = .backend_specific_verified,
        .status = if (formula_audit.verified()) .expandable_terms else
↳ .blocked_unverified_identity,
        .formula_kind = formula_kind,
        .formula_audit = formula_audit,
    };
}

fn localProductFormulaAudit(self: *Impl, step: coupling.CouplingStep, formula_kind:
↳ projector.ProjectorFormulaKind) projector.ProjectorFormulaAudit {
    return switch (formula_kind) {
        .orthogonal_spinor_pair => self.orthogonalSpinorPairFormulaAudit(step),
        .orthogonal_spinor_form_channel => self.orthogonalSpinorFormFormulaAudit(step),
        .orthogonal_form_spinor_channel => self.orthogonalFormSpinorFormulaAudit(step),
        .cartan_product_channel =>
↳ self.orthogonalSpinorTowerCartanProductFormulaAudit(step),
    }
}

```

```

        .identity,
        .orthogonal_vector_metric,
        .orthogonal_form_pair,
        .backend_specific_structure,
        => exactPrimitiveFormulaAudit(),
        else => if (isGramFormulaKind(formula_kind))
            ↪ self.localProductGramFormulaAudit(step, formula_kind) else .{},
    };
}

fn localProductGramFormulaAudit(self: *Impl, step: coupling.CouplingStep, formula_kind:
    ↪ projector.ProjectorFormulaKind) projector.ProjectorFormulaAudit {
    const term_count = self.localFormulaStepTermCount(step, formula_kind) catch return
        ↪ .{};
    if (term_count == 0) return .{};
    return exactGramProjectorFormulaAudit();
}

fn localProductFormulaKind(self: *Impl, step: coupling.CouplingStep)
    ↪ projector.ProjectorFormulaKind {
    if (self.isIdentityProductStep(step)) return .identity;
    if (self.isOrthogonalVectorMetricStep(step)) return .orthogonal_vector_metric;
    if (self.isOrthogonalSpinorPairSingletStep(step)) return .orthogonal_spinor_pair;
    if (self.isOrthogonalSpinorSpinorVectorStep(step)) return
        ↪ .orthogonal_spinor_form_channel;
    if (self.orthogonalSpinorSpinorFormStep(step) != null) return
        ↪ .orthogonal_spinor_form_channel;
    if (self.isOrthogonalFormPairSingletStep(step)) return .orthogonal_form_pair;
    if (self.orthogonalFormSpinorStep(step) != null) return
        ↪ .orthogonal_form_spinor_channel;
    if (self.isCartanProductStep(step)) return .cartan_product_channel;
    if (self.orthogonalStructuralStep(step, 0, 1, 2) |structural| {
        const spec = structuralProjectorSpecFromOrthogonalStep(structural);
        if ((self.structural_projection_programs.termCount(spec) catch 0) != 0) return
            ↪ .orthogonal_structural_projection;
    }
    if (self.orthogonalSpinorTowerFormStep(step) != null) return
        ↪ .orthogonal_spinor_tower_form_channel;
    if (self.orthogonalSpinorTowerMiddleFormStep(step) != null) return
        ↪ .orthogonal_spinor_tower_middle_form_channel;
    if (self.orthogonalSpinorTowerOppositeFormStep(step) != null) return
        ↪ .orthogonal_spinor_tower_opposite_form_channel;
    if (self.orthogonalSpinorTowerOppositeLowerStep(step) != null) return
        ↪ .orthogonal_spinor_tower_opposite_lower_channel;
    if (self.orthogonalTensorSpinorTowerStep(step) != null) return
        ↪ .orthogonal_tensor_spinor_tower_channel;
    if (self.orthogonalTensorSpinorFormPowerStep(step) != null) return
        ↪ .orthogonal_tensor_spinor_form_power_channel;
    if (self.orthogonalTensorSpinorTowerRaiseStep(step) != null) return
        ↪ .orthogonal_tensor_spinor_tower_raise_channel;
    if (self.orthogonalTensorSpinorOppositeTowerLowerStep(step) != null) return
        ↪ .orthogonal_tensor_spinor_opposite_tower_lower_channel;
    if (self.orthogonalTensorSpinorOppositeShiftStep(step) != null) return
        ↪ .orthogonal_tensor_spinor_opposite_shift_channel;
    if (self.orthogonalTensorSpinorOppositeAllShiftStep(step) != null) return
        ↪ .orthogonal_tensor_spinor_opposite_all_shift_channel;
    if (self.orthogonalTensorSpinorOppositeFormAddStep(step) != null) return
        ↪ .orthogonal_tensor_spinor_opposite_form_add_channel;
    if (self.orthogonalTensorSpinorOppositeRankSplitStep(step) != null) return
        ↪ .orthogonal_tensor_spinor_opposite_rank_split_channel;
    if (self.orthogonalTensorSpinorShiftStep(step) != null) return
        ↪ .orthogonal_tensor_spinor_shift_channel;
    if (self.orthogonalTensorSpinorRankWrapShiftStep(step) != null) return
        ↪ .orthogonal_tensor_spinor_rank_wrap_shift_channel;

```

```

if (self.orthogonalTensorSpinorRankSplitStep(step) != null) return
↳ .orthogonal_tensor_spinor_rank_split_channel;
if (self.orthogonalTensorSpinorTerminalStep(step) != null) return
↳ .orthogonal_tensor_spinor_terminal_channel;
if (self.orthogonalTensorSpinorOppositeTerminalStep(step) != null) return
↳ .orthogonal_tensor_spinor_opposite_terminal_channel;
if (self.orthogonalTensorSpinorOppositeFormAddTerminalStep(step) != null) return
↳ .orthogonal_tensor_spinor_opposite_form_add_terminal_channel;
if (self.orthogonalTensorSpinorOppositeRankSplitTerminalStep(step) != null) return
↳ .orthogonal_tensor_spinor_opposite_rank_split_terminal_channel;
if (self.orthogonalTensorSpinorOppositeFormAddShiftTerminalStep(step) != null) return
↳ .orthogonal_tensor_spinor_opposite_form_add_shift_terminal_channel;
if (self.orthogonalTensorSpinorOppositeShiftTerminalStep(step) != null) return
↳ .orthogonal_tensor_spinor_opposite_shift_terminal_channel;
if (self.orthogonalTensorSpinorRankWrapTerminalStep(step) != null) return
↳ .orthogonal_tensor_spinor_rank_wrap_terminal_channel;
if (self.orthogonalTensorSpinorRankSplitTerminalStep(step) != null) return
↳ .orthogonal_tensor_spinor_rank_split_terminal_channel;
if (self.orthogonalTensorSpinorFormTerminalStep(step) != null) return
↳ .orthogonal_tensor_spinor_form_terminal_channel;
if (self.orthogonalTensorSpinorFormPowerTerminalStep(step) != null) return
↳ .orthogonal_tensor_spinor_form_power_terminal_channel;
if (self.orthogonalTensorSpinorFormAddTerminalStep(step) != null) return
↳ .orthogonal_tensor_spinor_form_add_terminal_channel;
if (self.orthogonalTensorSpinorFormTowerStep(step) != null) return
↳ .orthogonal_tensor_spinor_form_tower_channel;
if (self.orthogonalTensorSpinorFormTowerTerminalStep(step) != null) return
↳ .orthogonal_tensor_spinor_form_tower_terminal_channel;
if (self.orthogonalTensorSpinorFormTowerRemoveShiftStep(step) != null) return
↳ .orthogonal_tensor_spinor_form_tower_remove_shift_channel;
if (self.orthogonalTensorSpinorFormTowerShiftDownStep(step) != null) return
↳ .orthogonal_tensor_spinor_form_tower_shift_down_channel;
if (self.orthogonalTensorSpinorFormTowerAllShiftDownStep(step) != null) return
↳ .orthogonal_tensor_spinor_form_tower_all_shift_down_channel;
if (self.orthogonalTensorSpinorOppositeFormTowerStep(step) != null) return
↳ .orthogonal_tensor_spinor_opposite_form_tower_channel;
if (self.orthogonalTensorSpinorOppositeFormTowerTerminalStep(step) != null) return
↳ .orthogonal_tensor_spinor_opposite_form_tower_terminal_channel;
if (self.orthogonalTensorSpinorOppositeFormTowerMergeStep(step) != null) return
↳ .orthogonal_tensor_spinor_opposite_form_tower_merge_channel;
if (self.orthogonalTensorSpinorOppositeFormTowerRemoveShiftStep(step) != null) return
↳ .orthogonal_tensor_spinor_opposite_form_tower_remove_shift_channel;
if (self.orthogonalTensorSpinorOppositeFormTowerShiftDownStep(step) != null) return
↳ .orthogonal_tensor_spinor_opposite_form_tower_shift_down_channel;
if (self.orthogonalTensorFormSpinorStep(step) != null) return
↳ .orthogonal_tensor_form_spinor_channel;
if (self.orthogonalTensorFormSpinorPreserveStep(step) != null) return
↳ .orthogonal_tensor_form_spinor_preserve_channel;
if (self.orthogonalTensorFormSpinorShiftStep(step) != null) return
↳ .orthogonal_tensor_form_spinor_shift_channel;
if (self.orthogonalTensorFormSpinorRemoveShiftDownStep(step) != null) return
↳ .orthogonal_tensor_form_spinor_remove_shift_down_channel;
if (self.orthogonalTensorFormSpinorShiftDownStep(step) != null) return
↳ .orthogonal_tensor_form_spinor_shift_down_channel;
if (self.orthogonalTensorFormSpinorAllShiftDownStep(step) != null) return
↳ .orthogonal_tensor_form_spinor_all_shift_down_channel;
if (self.orthogonalTensorFormSpinorShiftDownTwoStep(step) != null) return
↳ .orthogonal_tensor_form_spinor_shift_down_two_channel;
if (self.orthogonalTensorFormSpinorMixedShiftDownStep(step) != null) return
↳ .orthogonal_tensor_form_spinor_mixed_shift_down_channel;
if (self.orthogonalTensorFormSpinorShiftDownAnyStep(step) != null) return
↳ .orthogonal_tensor_form_spinor_shift_down_any_channel;
if (self.orthogonalTensorSpinorMiddleFormStep(step) != null) return
↳ .orthogonal_tensor_spinor_middle_form_channel;

```

```

    if (self.isA2FundamentalPairToAntiFundamentalStep(step)) return
    ↪ .backend_specific_structure;
    if (self.isA2AntiFundamentalFundamentalSingletStep(step)) return
    ↪ .backend_specific_structure;
    if (self.isE6FundamentalPairToDualFundamentalStep(step)) return
    ↪ .backend_specific_structure;
    if (self.isE6DualFundamentalFundamentalSingletStep(step)) return
    ↪ .backend_specific_structure;
    if (self.isSimpleAdjointAdjointStep(step)) return .backend_specific_structure;
    if (self.isSimpleAdjointPairSingletStep(step)) return .backend_specific_structure;
    return .named_only;
}

fn orthogonalStructuralStep(self: *Impl, step: coupling.CouplingStep, left_index:
↪ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↪ ?OrthogonalStructuralStep {
    const simple = self.stepSimpleAlgebra(step) orelse return null;
    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_descriptor = orthogonalIrrepDescriptor(simple, left);
    const right_descriptor = orthogonalIrrepDescriptor(simple, right);
    const output_descriptor = orthogonalIrrepDescriptor(simple, output);
    if (left_descriptor.kind == .unsupported or right_descriptor.kind == .unsupported or
    ↪ output_descriptor.kind == .unsupported) return null;
    if (left_descriptor.dimension != right_descriptor.dimension or
    ↪ left_descriptor.dimension != output_descriptor.dimension) return null;
    if (left_descriptor.kind == .scalar and right_descriptor.kind == .scalar and
    ↪ output_descriptor.kind == .scalar) return null;
    return .{
        .operator_id = step.projector,
        .dimension = left_descriptor.dimension,
        .left = .{ .irrep = left_descriptor, .index = left_index },
        .right = .{ .irrep = right_descriptor, .index = right_index },
        .output = .{ .irrep = output_descriptor, .index = output_index },
        .multiplicity_copy = step.multiplicity_copy,
    };
}

fn boundaryRealizationProjector(self: *Impl, handle: realization.RealizationHandle)
↪ !projector.ProjectorId {
    const has_formula = try self.hasOrthogonalVectorSpinorBoundaryFormula(handle);
    const formula_audit = if (has_formula) try
    ↪ self.orthogonalVectorSpinorBoundaryFormulaAudit(handle) else
    ↪ projector.ProjectorFormulaAudit{};
    const formula_verified = has_formula and formula_audit.verified() and
    ↪ formula_audit.gamma_traceless;
    return self.projectors.internProjector(.{
        .role = .{ .boundary_realization = handle },
        .convention = .{
            .render_mode = .named,
            .signature = self.options.default_signature,
            .normalization_id = 0,
        },
    }, .{
        .kind = if (formula_verified) .backend_specific_verified else
        ↪ .highest_weight_solver,
        .source = if (formula_verified) .backend_specific_verified else
        ↪ .highest_weight_solver,
        .status = if (formula_verified) .expandable_terms else if (has_formula)
        ↪ .blocked_unverified_identity else .expandable_named,
        .formula_kind = if (has_formula) .orthogonal_gamma_traceless else .named_only,
        .formula_audit = formula_audit,
    });
}

```

```

}

fn requirePathProjectorExpansion(self: *Impl, steps: []const coupling.CouplingStep, mode:
↳ rendering.ProjectorRenderMode) !void {
    for (steps) |step| {
        if (!self.projectors.canRenderMode(step.projector, mode)) return
        ↳ error.ProjectorExpansionNotImplemented;
    }
}

fn isOrthogonalVectorMetricStep(self: *Impl, step: coupling.CouplingStep) bool {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return false;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↳ false;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↳ false;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↳ output_algebra.value) return false;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return false;
    switch (simple.family) {
        .b, .d => {},
        else => return false,
    }

    const left = self.representations.irrepDynkin(step.left) orelse return false;
    const right = self.representations.irrepDynkin(step.right) orelse return false;
    const output = self.representations.irrepDynkin(step.output) orelse return false;
    return isVectorDynkin(left) and isVectorDynkin(right) and isZeroDynkin(output);
}

fn isIdentityProductStep(self: *Impl, step: coupling.CouplingStep) bool {
    if (step.multiplicity_copy != 0) return false;
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return false;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↳ false;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↳ false;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↳ output_algebra.value) return false;

    const left = self.representations.irrepDynkin(step.left) orelse return false;
    const right = self.representations.irrepDynkin(step.right) orelse return false;
    const output = self.representations.irrepDynkin(step.output) orelse return false;
    if (left.len != right.len or left.len != output.len) return false;
    if (isZeroDynkin(left) and std.mem.eql(i16, right, output)) return true;
    if (isZeroDynkin(right) and std.mem.eql(i16, left, output)) return true;
    return false;
}

fn isOrthogonalSpinorPairSingletStep(self: *Impl, step: coupling.CouplingStep) bool {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return false;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↳ false;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↳ false;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↳ output_algebra.value) return false;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return false;
    switch (simple.family) {
        .b, .d => {},
        else => return false,
    }

    const left = self.representations.irrepDynkin(step.left) orelse return false;

```

```

    const right = self.representations.irrepDynkin(step.right) orelse return false;
    const output = self.representations.irrepDynkin(step.output) orelse return false;
    return isSpinorDynkin(simple, left) and isSpinorDynkin(simple, right) and
    ↪ isZeroDynkin(output);
}

fn isOrthogonalSpinorSpinorVectorStep(self: *Impl, step: coupling.CouplingStep) bool {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return false;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ false;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ false;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return false;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return false;
    switch (simple.family) {
        .b, .d => {},
        else => return false,
    }

    const left = self.representations.irrepDynkin(step.left) orelse return false;
    const right = self.representations.irrepDynkin(step.right) orelse return false;
    const output = self.representations.irrepDynkin(step.output) orelse return false;
    return isSpinorDynkin(simple, left) and isSpinorDynkin(simple, right) and
    ↪ isVectorDynkin(output);
}

fn orthogonalSpinorPairFormulaAudit(self: *Impl, step: coupling.CouplingStep)
↪ projector.ProjectorFormulaAudit {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return .{};
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return .{};
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ .{};
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return .{};
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return .{};
    const left = self.representations.irrepDynkin(step.left) orelse return .{};
    const right = self.representations.irrepDynkin(step.right) orelse return .{};
    return orthogonalSpinorBilinearFormulaAudit(simple, left, right, 0, .none);
}

fn orthogonalSpinorFormFormulaAudit(self: *Impl, step: coupling.CouplingStep)
↪ projector.ProjectorFormulaAudit {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return .{};
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return .{};
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ .{};
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return .{};
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return .{};
    const left = self.representations.irrepDynkin(step.left) orelse return .{};
    const right = self.representations.irrepDynkin(step.right) orelse return .{};
    const output = self.representations.irrepDynkin(step.output) orelse return .{};
    var rank: u8 = 1;
    var duality: rendering.DualityTag = .none;
    if (!isVectorDynkin(output)) {
        const info = self.orthogonalSpinorSpinorFormStep(step) orelse return .{};
        rank = info.rank;
        duality = info.duality;
    }
    return orthogonalSpinorBilinearFormulaAudit(simple, left, right, rank, duality);
}

```



```

fn orthogonalFormSpinorFormulaAudit(self: *Impl, step: coupling.CouplingStep)
  ↪ projector.ProjectorFormulaAudit {
  const info = self.orthogonalFormSpinorStep(step) orelse return .{};
  const simple = self.stepSimpleAlgebra(step) orelse return .{};
  const right = self.representations.irrepDynkin(step.right) orelse return .{};
  const output = self.representations.irrepDynkin(step.output) orelse return .{};
  const input_chirality = rendering.gammaChiralityTag(simple, right);
  const output_chirality = rendering.gammaChiralityTag(simple, output);
  if (!orthogonalGammaGradeNormNonZero(simple, rendering.orthogonalDimension(simple),
    ↪ info.rank, info.duality)) return .{};
  const chirality_valid = switch (simple.family) {
    .b => input_chirality == .none and output_chirality == .none,
    .d => output_chirality == if (info.rank % 2 == 0) input_chirality else
    ↪ oppositeGammaChirality(input_chirality),
    else => false,
  };
  if (!chirality_valid) return .{};
  return exactPrimitiveFormulaAudit();
}

fn orthogonalSpinorTowerCartanProductFormulaAudit(self: *Impl, step:
  ↪ coupling.CouplingStep) projector.ProjectorFormulaAudit {
  const simple = self.stepSimpleAlgebra(step) orelse return .{};
  const left = self.representations.irrepDynkin(step.left) orelse return .{};
  const right = self.representations.irrepDynkin(step.right) orelse return .{};
  const output = self.representations.irrepDynkin(step.output) orelse return .{};
  const left_tower = spinorTowerInfo(simple, left) orelse return .{};
  const right_tower = spinorTowerInfo(simple, right) orelse return .{};
  const output_tower = spinorTowerInfo(simple, output) orelse return .{};
  if (left_tower.chirality != right_tower.chirality or left_tower.chirality !=
    ↪ output_tower.chirality) return .{};
  if (output_tower.power != left_tower.power + right_tower.power) return .{};
  return exactPrimitiveFormulaAudit();
}

fn orthogonalSpinorSpinorFormStep(self: *Impl, step: coupling.CouplingStep)
  ↪ ?OrthogonalFormInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
  if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
  const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
  switch (simple.family) {
    .b, .d => {},
    else => return null,
  }

  const left = self.representations.irrepDynkin(step.left) orelse return null;
  const right = self.representations.irrepDynkin(step.right) orelse return null;
  const output = self.representations.irrepDynkin(step.output) orelse return null;
  if (!isSpinorDynkin(simple, left) or !isSpinorDynkin(simple, right)) return null;
  const form_info = orthogonalFormInfo(simple, output) orelse return null;
  return if (form_info.rank > 1) form_info else null;
}

fn orthogonalFormSpinorStep(self: *Impl, step: coupling.CouplingStep) ?OrthogonalFormInfo
  ↪ {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;

```

```

const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ null;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
↳ output_algebra.value) return null;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
switch (simple.family) {
    .b, .d => {},
    else => return null,
}

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const form_info = orthogonalFormInfo(simple, left) orelse return null;
if (spinorTowerInfo(simple, left) != null and spinorTowerInfo(simple, output) != null)
↳ return null;
if (!isSpinorDynkin(simple, right) or !isSpinorDynkin(simple, output)) return null;
return form_info;
}

fn orthogonalSpinorTowerFormStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorSpinorInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
↳ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
↳ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    switch (simple.family) {
        .b, .d => {},
        else => return null,
    }

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_tower = spinorTowerInfo(simple, left) orelse return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) != left_tower.chirality) return null;
    if (left_tower.power < 2) return null;
    const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
    if (output_info.tower_power != left_tower.power - 1) return null;
    if (output_info.chirality != left_tower.chirality) return null;
    return output_info;
}

fn orthogonalSpinorTowerMiddleFormStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalFormInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
↳ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
↳ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_tower = spinorTowerInfo(simple, left) orelse return null;

```



```

    if (left_tower.power != 3) return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) == left_tower.chirality) return null;
    const output_info = orthogonalFormInfo(simple, output) orelse return null;
    if (output_info.duality == .none) return null;
    if (output_info.rank != rendering.orthogonalDimension(simple) / 2) return null;
    return output_info;
}

fn orthogonalSpinorTowerOppositeFormStep(self: *Impl, step: coupling.CouplingStep)
↪ ?OrthogonalTensorSpinorInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = spinorTowerInfo(simple, left) orelse return null;
    if (left_info.power < 2) return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
    const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
    if (output_info.chirality != left_info.chirality) return null;
    if (output_info.tower_power != left_info.power - 1) return null;
    if (output_info.forms.total_power != 1) return null;
    return output_info;
}

fn orthogonalSpinorTowerOppositeLowerStep(self: *Impl, step: coupling.CouplingStep)
↪ ?SpinorTowerInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = spinorTowerInfo(simple, left) orelse return null;
    if (left_info.power < 2) return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
    const output_info = spinorTowerInfo(simple, output) orelse return null;
    if (output_info.chirality != left_info.chirality) return null;
    if (output_info.power != left_info.power - 1) return null;
    return output_info;
}

fn orthogonalTensorSpinorTowerStep(self: *Impl, step: coupling.CouplingStep)
↪ ?OrthogonalTensorSpinorInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;

```

```

const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
↳ null;
const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ null;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
↳ output_algebra.value) return null;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
switch (simple.family) {
    .b, .d => {},
    else => return null,
}

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
if (left_info.tower_power < 2) return null;
const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
if (output_info.tower_power != left_info.tower_power - 1) return null;
if (output_info.chirality != left_info.chirality) return null;
if (output_info.forms.count != left_info.forms.count + 1) return null;
if ((output_info.forms.mask & left_info.forms.mask) != left_info.forms.mask) return
↳ null;
return output_info;
}

fn orthogonalTensorSpinorFormPowerStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorSpinorInfo {
const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
↳ null;
const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ null;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
↳ output_algebra.value) return null;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
switch (simple.family) {
    .b, .d => {},
    else => return null,
}

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
if (left_info.tower_power < 2) return null;
const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
if (output_info.tower_power != left_info.tower_power - 1) return null;
if (output_info.chirality != left_info.chirality) return null;
if (output_info.forms.count != left_info.forms.count) return null;
if (output_info.forms.total_power != left_info.forms.total_power + 1) return null;
if (!formProfileIncrementedExistingOnce(left_info.forms.profile,
↳ output_info.forms.profile)) return null;
return output_info;
}

fn orthogonalTensorSpinorTowerRaiseStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorSpinorTransitionInfo {
const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;

```

```

const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
↳ null;
const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ null;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
↳ output_algebra.value) return null;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
switch (simple.family) {
    .b, .d => {},
    else => return null,
}

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
if (output_info.chirality != left_info.chirality) return null;
if (output_info.tower_power != left_info.tower_power + 1) return null;
if (output_info.forms.profile != left_info.forms.profile) return null;
return .{
    .input = left_info,
    .output = output_info,
};
}

fn orthogonalTensorSpinorOppositeTowerLowerStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorSpinorTransitionInfo {
const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
↳ null;
const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ null;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
↳ output_algebra.value) return null;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
if (simple.family != .d) return null;

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (left_info.tower_power < 2) return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
if (output_info.chirality != left_info.chirality) return null;
if (output_info.tower_power != left_info.tower_power - 1) return null;
if (output_info.forms.profile != left_info.forms.profile) return null;
return .{
    .input = left_info,
    .output = output_info,
};
}

fn orthogonalTensorSpinorOppositeShiftStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorSpinorTransitionInfo {
const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
↳ null;
const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ null;

```

```

    if (left_algebra.value != right_algebra.value or left_algebra.value !=
        ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
    if (left_info.tower_power < 2) return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
    const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
    if (output_info.chirality != left_info.chirality) return null;
    if (output_info.tower_power != left_info.tower_power - 1) return null;
    if (output_info.forms.total_power != left_info.forms.total_power) return null;
    if (!formProfileShiftedByOnce(left_info.forms.profile, output_info.forms.profile, 2))
        ↪ return null;
    return .{
        .input = left_info,
        .output = output_info,
    };
}

fn orthogonalTensorSpinorOppositeAllShiftStep(self: *Impl, step: coupling.CouplingStep)
    ↪ ?OrthogonalTensorSpinorTransitionInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
        ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
        ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
        ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
    if (left_info.tower_power < 2) return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
    const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
    if (output_info.chirality != left_info.chirality) return null;
    if (output_info.tower_power != left_info.tower_power - 1) return null;
    if (output_info.forms.total_power != left_info.forms.total_power) return null;
    if (!formProfileShiftedAllUpOnce(left_info.forms.profile, output_info.forms.profile))
        ↪ return null;
    return .{
        .input = left_info,
        .output = output_info,
    };
}

fn orthogonalTensorSpinorOppositeFormAddStep(self: *Impl, step: coupling.CouplingStep)
    ↪ ?OrthogonalTensorSpinorTransitionInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
        ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
        ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
        ↪ output_algebra.value) return null;

```

```

const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
if (simple.family != .d) return null;

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (left_info.tower_power < 2) return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
if (output_info.chirality != left_info.chirality) return null;
if (output_info.tower_power != left_info.tower_power - 1) return null;
if (output_info.forms.total_power != left_info.forms.total_power + 1) return null;
if (!formProfileAddedOnce(left_info.forms.profile, output_info.forms.profile)) return
  ↪ null;
return .{
  .input = left_info,
  .output = output_info,
};
}

fn orthogonalTensorSpinorOppositeRankSplitStep(self: *Impl, step: coupling.CouplingStep)
  ↪ ?OrthogonalTensorSpinorTransitionInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
  if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
  const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
  if (simple.family != .d) return null;

  const left = self.representations.irrepDynkin(step.left) orelse return null;
  const right = self.representations.irrepDynkin(step.right) orelse return null;
  const output = self.representations.irrepDynkin(step.output) orelse return null;
  const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
  if (left_info.tower_power < 2) return null;
  if (!isSpinorDynkin(simple, right)) return null;
  if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
  const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
  if (output_info.chirality != left_info.chirality) return null;
  if (output_info.tower_power != left_info.tower_power - 1) return null;
  if (output_info.forms.total_power != left_info.forms.total_power + 1) return null;
  if (!formProfileSplitToNeighborsOnce(left_info.forms.profile,
    ↪ output_info.forms.profile)) return null;
  return .{
    .input = left_info,
    .output = output_info,
  };
}

fn orthogonalTensorSpinorShiftStep(self: *Impl, step: coupling.CouplingStep)
  ↪ ?OrthogonalTensorSpinorInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
  if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
  const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
  switch (simple.family) {

```

```

        .b, .d => {},
        else => return null,
    }

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
    if (left_info.tower_power < 2) return null;
    const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
    if (output_info.tower_power != left_info.tower_power - 1) return null;
    if (output_info.chirality != left_info.chirality) return null;
    if (output_info.forms.count != left_info.forms.count) return null;
    if (!formMaskShiftedUpOnce(left_info.forms.mask, output_info.forms.mask)) return null;
    return output_info;
}

fn orthogonalTensorSpinorRankWrapShiftStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorSpinorInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↳ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↳ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↳ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    switch (simple.family) {
        .b, .d => {},
        else => return null,
    }

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
    if (left_info.tower_power < 2) return null;
    const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
    if (output_info.tower_power != left_info.tower_power - 1) return null;
    if (output_info.chirality != left_info.chirality) return null;
    if (output_info.forms.count != left_info.forms.count) return null;
    if (!formProfileMovedOnce(left_info.forms.profile, output_info.forms.profile, 0,
    ↳ simple.rank - 2)) return null;
    return output_info;
}

fn orthogonalTensorSpinorRankSplitStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorSpinorInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↳ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↳ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↳ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    switch (simple.family) {
        .b, .d => {},
        else => return null,
    }
}

```



```

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
if (left_info.tower_power < 2) return null;
const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
if (output_info.tower_power != left_info.tower_power - 1) return null;
if (output_info.chirality != left_info.chirality) return null;
if (output_info.forms.total_power != left_info.forms.total_power + 1) return null;
if (!formProfileSplitDownToRankWrapOnce(left_info.forms.profile,
    ↪ output_info.forms.profile, simple.rank - 2)) return null;
return output_info;
}

fn orthogonalTensorSpinorTerminalStep(self: *Impl, step: coupling.CouplingStep)
    ↪ ?OrthogonalTensorFormTerminalInfo {
const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
switch (simple.family) {
    .b, .d => {},
    else => return null,
}

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (left_info.tower_power != 1) return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
const output_info = ordinaryTensorFormInfo(simple, output) orelse return null;
if (output_info.total_power != left_info.forms.total_power) return null;
if (!formProfileShiftedUpOnce(left_info.forms.profile, output_info.profile)) return
    ↪ null;
return .{
    .input = left_info,
    .output = output_info,
};
}

fn orthogonalTensorSpinorOppositeTerminalStep(self: *Impl, step: coupling.CouplingStep)
    ↪ ?OrthogonalTensorFormTerminalInfo {
const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
if (simple.family != .d) return null;

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;

```

```

const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (left_info.tower_power != 1) return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
const output_info = ordinaryTensorFormInfo(simple, output) orelse return null;
if (output_info.total_power != left_info.forms.total_power) return null;
if (!formProfileShiftedByOnce(left_info.forms.profile, output_info.profile, 2)) return
  ↪ null;
return .{
  .input = left_info,
  .output = output_info,
};
}

fn orthogonalTensorSpinorOppositeFormAddTerminalStep(self: *Impl, step:
  ↪ coupling.CouplingStep) ?OrthogonalTensorFormTerminalInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
  if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
  const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
  if (simple.family != .d) return null;

  const left = self.representations.irrepDynkin(step.left) orelse return null;
  const right = self.representations.irrepDynkin(step.right) orelse return null;
  const output = self.representations.irrepDynkin(step.output) orelse return null;
  const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
  if (left_info.tower_power != 1) return null;
  if (!isSpinorDynkin(simple, right)) return null;
  if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
  const output_info = ordinaryTensorFormInfo(simple, output) orelse return null;
  if (output_info.total_power != left_info.forms.total_power + 1) return null;
  if (!formProfileAddedOnce(left_info.forms.profile, output_info.profile)) return null;
  return .{
    .input = left_info,
    .output = output_info,
  };
}

fn orthogonalTensorSpinorOppositeFormAddShiftTerminalStep(self: *Impl, step:
  ↪ coupling.CouplingStep) ?OrthogonalTensorFormTerminalInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
  if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
  const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
  if (simple.family != .d) return null;

  const left = self.representations.irrepDynkin(step.left) orelse return null;
  const right = self.representations.irrepDynkin(step.right) orelse return null;
  const output = self.representations.irrepDynkin(step.output) orelse return null;
  const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
  if (left_info.tower_power != 1) return null;
  if (!isSpinorDynkin(simple, right)) return null;
  if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
  const output_info = ordinaryTensorFormInfo(simple, output) orelse return null;
  if (output_info.total_power != left_info.forms.total_power + 1) return null;

```



```

    if (!formProfileShiftedUpThenAddedOnce(left_info.forms.profile, output_info.profile))
    ↪ return null;
    return .{
        .input = left_info,
        .output = output_info,
    };
}

fn orthogonalTensorSpinorOppositeRankSplitTerminalStep(self: *Impl, step:
↪ coupling.CouplingStep) ?OrthogonalTensorFormTerminalInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
    if (left_info.tower_power != 1) return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
    const output_info = ordinaryTensorFormInfo(simple, output) orelse return null;
    if (output_info.total_power != left_info.forms.total_power + 1) return null;
    if (!formProfileSplitToNeighborsOnce(left_info.forms.profile, output_info.profile))
    ↪ return null;
    return .{
        .input = left_info,
        .output = output_info,
    };
}

fn orthogonalTensorSpinorOppositeShiftTerminalStep(self: *Impl, step:
↪ coupling.CouplingStep) ?OrthogonalTensorFormTerminalInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
    if (left_info.tower_power != 1) return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
    const output_info = ordinaryTensorFormInfo(simple, output) orelse return null;
    if (output_info.total_power != left_info.forms.total_power) return null;
    if (!formProfileShiftedAllUpOnce(left_info.forms.profile, output_info.profile)) return
    ↪ null;
    return .{
        .input = left_info,
        .output = output_info,
    };
}

```

```

}

fn orthogonalTensorSpinorRankWrapTerminalStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorFormTerminalInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
  ↳ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
  ↳ null;
  if (left_algebra.value != right_algebra.value or left_algebra.value !=
  ↳ output_algebra.value) return null;
  const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
  if (simple.family != .d) return null;

  const left = self.representations.irrepDynkin(step.left) orelse return null;
  const right = self.representations.irrepDynkin(step.right) orelse return null;
  const output = self.representations.irrepDynkin(step.output) orelse return null;
  const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
  if (left_info.tower_power != 1) return null;
  if (!isSpinorDynkin(simple, right)) return null;
  if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
  const output_info = ordinaryTensorFormInfo(simple, output) orelse return null;
  if (output_info.total_power != left_info.forms.total_power) return null;
  if (!formProfileMovedOnce(left_info.forms.profile, output_info.profile, 0, simple.rank
  ↳ - 2)) return null;
  return .{
    .input = left_info,
    .output = output_info,
  };
}

fn orthogonalTensorSpinorRankSplitTerminalStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorFormTerminalInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
  ↳ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
  ↳ null;
  if (left_algebra.value != right_algebra.value or left_algebra.value !=
  ↳ output_algebra.value) return null;
  const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
  if (simple.family != .d) return null;

  const left = self.representations.irrepDynkin(step.left) orelse return null;
  const right = self.representations.irrepDynkin(step.right) orelse return null;
  const output = self.representations.irrepDynkin(step.output) orelse return null;
  const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
  if (left_info.tower_power != 1) return null;
  if (!isSpinorDynkin(simple, right)) return null;
  if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
  const output_info = ordinaryTensorFormInfo(simple, output) orelse return null;
  if (output_info.total_power != left_info.forms.total_power + 1) return null;
  if (!formProfileSplitDownToRankWrapOnce(left_info.forms.profile, output_info.profile,
  ↳ simple.rank - 2)) return null;
  return .{
    .input = left_info,
    .output = output_info,
  };
}

fn orthogonalTensorSpinorFormTerminalStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorFormTerminalInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;

```

```

const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
↳ null;
const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ null;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
↳ output_algebra.value) return null;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
switch (simple.family) {
    .b, .d => {},
    else => return null,
}

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (left_info.tower_power != 1) return null;
if (!isSpinorDynkin(simple, right)) return null;
const output_info = ordinaryTensorFormInfo(simple, output) orelse return null;
if (output_info.total_power != left_info.forms.total_power) return null;
if (output_info.profile != left_info.forms.profile) return null;
return .{
    .input = left_info,
    .output = output_info,
};
}

fn orthogonalTensorSpinorFormAddTerminalStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorFormTerminalInfo {
const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
↳ null;
const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ null;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
↳ output_algebra.value) return null;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
switch (simple.family) {
    .b, .d => {},
    else => return null,
}

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (left_info.tower_power != 1) return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
const output_info = ordinaryTensorFormInfo(simple, output) orelse return null;
if (output_info.total_power != left_info.forms.total_power + 1) return null;
if (!formProfileAddedOnce(left_info.forms.profile, output_info.profile)) return null;
return .{
    .input = left_info,
    .output = output_info,
};
}

fn orthogonalTensorSpinorFormPowerTerminalStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorFormTerminalInfo {
const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
↳ null;

```

```

const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ null;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
↳ output_algebra.value) return null;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
switch (simple.family) {
    .b, .d => {},
    else => return null,
}

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (left_info.tower_power != 1) return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
const output_info = ordinaryTensorFormInfo(simple, output) orelse return null;
if (output_info.total_power != left_info.forms.total_power + 1) return null;
if (!formProfileIncrementedExistingOnce(left_info.forms.profile, output_info.profile))
↳ return null;
return .{
    .input = left_info,
    .output = output_info,
};
}

fn orthogonalTensorSpinorFormTowerStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorSpinorTransitionInfo {
const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
↳ null;
const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ null;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
↳ output_algebra.value) return null;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
if (simple.family != .d) return null;

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
if (output_info.chirality != left_info.chirality) return null;
if (output_info.tower_power != left_info.tower_power + 1) return null;
if (output_info.forms.total_power + 1 != left_info.forms.total_power) return null;
if (!formProfileRemovedOnce(left_info.forms.profile, output_info.forms.profile))
↳ return null;
return .{
    .input = left_info,
    .output = output_info,
};
}

fn orthogonalTensorSpinorFormTowerTerminalStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorSpinorTowerTerminalInfo {
const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
↳ null;
const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ null;

```

```

    if (left_algebra.value != right_algebra.value or left_algebra.value !=
        ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
    const output_tower = spinorTowerInfo(simple, output) orelse return null;
    if (output_tower.chirality != left_info.chirality) return null;
    if (output_tower.power != left_info.tower_power + 1) return null;
    if (!formProfileRemovedOnce(left_info.forms.profile, 0)) return null;
    return .{
        .input = left_info,
        .output = output_tower,
    };
}

fn orthogonalTensorSpinorFormTowerRemoveShiftStep(self: *Impl, step:
    ↪ coupling.CouplingStep) ?OrthogonalTensorSpinorTransitionInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
        ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
        ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
        ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    switch (simple.family) {
        .b, .d => {},
        else => return null,
    }

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
    const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
    if (output_info.chirality != left_info.chirality) return null;
    if (output_info.tower_power != left_info.tower_power + 1) return null;
    if (output_info.forms.total_power + 1 != left_info.forms.total_power) return null;
    if (!formProfileRemovedThenShiftedUpOnce(left_info.forms.profile,
        ↪ output_info.forms.profile)) return null;
    return .{
        .input = left_info,
        .output = output_info,
    };
}

fn orthogonalTensorSpinorFormTowerShiftDownStep(self: *Impl, step: coupling.CouplingStep)
    ↪ ?OrthogonalTensorSpinorTransitionInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
        ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
        ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
        ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;

```

```

switch (simple.family) {
  .b, .d => {},
  else => return null,
}

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
if (output_info.chirality != left_info.chirality) return null;
if (output_info.tower_power != left_info.tower_power + 1) return null;
if (output_info.forms.total_power != left_info.forms.total_power) return null;
if (!formProfileMovedDownOnce(left_info.forms.profile, output_info.forms.profile, 2))
  ↪ return null;
return .{
  .input = left_info,
  .output = output_info,
};
}

fn orthogonalTensorSpinorFormTowerAllShiftDownStep(self: *Impl, step:
  ↪ coupling.CouplingStep) ?OrthogonalTensorSpinorTransitionInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
  ↪ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
  ↪ null;
  if (left_algebra.value != right_algebra.value or left_algebra.value !=
  ↪ output_algebra.value) return null;
  const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
  switch (simple.family) {
    .b, .d => {},
    else => return null,
  }

  const left = self.representations.irrepDynkin(step.left) orelse return null;
  const right = self.representations.irrepDynkin(step.right) orelse return null;
  const output = self.representations.irrepDynkin(step.output) orelse return null;
  const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
  if (!isSpinorDynkin(simple, right)) return null;
  if (rendering.gammaChiralityTag(simple, right) != left_info.chirality) return null;
  const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
  if (output_info.chirality != left_info.chirality) return null;
  if (output_info.tower_power != left_info.tower_power + 1) return null;
  if (output_info.forms.total_power != left_info.forms.total_power) return null;
  if (!formProfileShiftedAllDownOnce(left_info.forms.profile,
  ↪ output_info.forms.profile)) return null;
  return .{
    .input = left_info,
    .output = output_info,
  };
}

fn orthogonalTensorSpinorOppositeFormTowerStep(self: *Impl, step: coupling.CouplingStep)
  ↪ ?OrthogonalTensorSpinorTransitionInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
  ↪ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
  ↪ null;

```



```

    if (left_algebra.value != right_algebra.value or left_algebra.value !=
        ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
    const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
    if (output_info.chirality != left_info.chirality) return null;
    if (output_info.tower_power != left_info.tower_power + 1) return null;
    if (output_info.forms.total_power + 1 != left_info.forms.total_power) return null;
    if (!formProfileRemovedOnce(left_info.forms.profile, output_info.forms.profile))
        ↪ return null;
    return .{
        .input = left_info,
        .output = output_info,
    };
}

fn orthogonalTensorSpinorOppositeFormTowerTerminalStep(self: *Impl, step:
    ↪ coupling.CouplingStep) ?OrthogonalTensorSpinorTowerTerminalInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
        ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
        ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
        ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
    if (!isSpinorDynkin(simple, right)) return null;
    if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
    if (orthogonalFormInfo(simple, output) != null) return null;
    const output_tower = spinorTowerInfo(simple, output) orelse return null;
    if (output_tower.chirality != left_info.chirality) return null;
    if (output_tower.power != left_info.tower_power + left_info.forms.total_power) return
        ↪ null;
    if (!formProfileRemovedOnce(left_info.forms.profile, 0)) return null;
    return .{
        .input = left_info,
        .output = output_tower,
    };
}

fn orthogonalTensorSpinorOppositeFormTowerMergeStep(self: *Impl, step:
    ↪ coupling.CouplingStep) ?OrthogonalTensorSpinorTransitionInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
        ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
        ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
        ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

```

```

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
if (output_info.chirality != left_info.chirality) return null;
if (output_info.tower_power != left_info.tower_power + 1) return null;
if (output_info.forms.total_power + 1 != left_info.forms.total_power) return null;
if (!formProfileMergedPairToMiddleOnce(left_info.forms.profile,
    ↪ output_info.forms.profile)) return null;
return .{
    .input = left_info,
    .output = output_info,
};
}

fn orthogonalTensorSpinorOppositeFormTowerRemoveShiftStep(self: *Impl, step:
    ↪ coupling.CouplingStep) ?OrthogonalTensorSpinorTransitionInfo {
const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
if (simple.family != .d) return null;

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;
const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
if (output_info.chirality != left_info.chirality) return null;
if (output_info.tower_power != left_info.tower_power + 1) return null;
if (output_info.forms.total_power + 1 != left_info.forms.total_power) return null;
if (!formProfileRemovedThenShiftedUpOnce(left_info.forms.profile,
    ↪ output_info.forms.profile)) return null;
return .{
    .input = left_info,
    .output = output_info,
};
}

fn orthogonalTensorSpinorOppositeFormTowerShiftDownStep(self: *Impl, step:
    ↪ coupling.CouplingStep) ?OrthogonalTensorSpinorTransitionInfo {
const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
if (simple.family != .d) return null;

const left = self.representations.irrepDynkin(step.left) orelse return null;
const right = self.representations.irrepDynkin(step.right) orelse return null;
const output = self.representations.irrepDynkin(step.output) orelse return null;

```



```

const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
if (!isSpinorDynkin(simple, right)) return null;
if (rendering.gammaChiralityTag(simple, right) == left_info.chirality) return null;
const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
if (output_info.chirality != left_info.chirality) return null;
if (output_info.tower_power != left_info.tower_power + 1) return null;
if (output_info.forms.total_power != left_info.forms.total_power) return null;
if (!formProfileMovedDownAnyOnce(left_info.forms.profile, output_info.forms.profile))
  ↪ return null;
return .{
  .input = left_info,
  .output = output_info,
};
}

fn orthogonalTensorFormSpinorStep(self: *Impl, step: coupling.CouplingStep)
  ↪ ?OrthogonalTensorFormSpinorInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
  ↪ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
  ↪ null;
  if (left_algebra.value != right_algebra.value or left_algebra.value !=
  ↪ output_algebra.value) return null;
  const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
  switch (simple.family) {
    .b, .d => {},
    else => return null,
  }

  const left = self.representations.irrepDynkin(step.left) orelse return null;
  const right = self.representations.irrepDynkin(step.right) orelse return null;
  const output = self.representations.irrepDynkin(step.output) orelse return null;
  const left_info = ordinaryTensorFormInfo(simple, left) orelse return null;
  if (!isSpinorDynkin(simple, right)) return null;
  const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
  if (output_info.tower_power != 1) return null;
  if (output_info.forms.total_power + 1 != left_info.total_power) return null;
  if (!formProfileRemovedOnce(left_info.profile, output_info.forms.profile)) return
  ↪ null;
  return .{
    .input = left_info,
    .output = output_info,
  };
}

fn orthogonalTensorFormSpinorPreserveStep(self: *Impl, step: coupling.CouplingStep)
  ↪ ?OrthogonalTensorFormSpinorInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
  ↪ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
  ↪ null;
  if (left_algebra.value != right_algebra.value or left_algebra.value !=
  ↪ output_algebra.value) return null;
  const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
  switch (simple.family) {
    .b, .d => {},
    else => return null,
  }

  const left = self.representations.irrepDynkin(step.left) orelse return null;
  const right = self.representations.irrepDynkin(step.right) orelse return null;
  const output = self.representations.irrepDynkin(step.output) orelse return null;

```

```

const left_info = ordinaryTensorFormInfo(simple, left) orelse return null;
if (!isSpinorDynkin(simple, right)) return null;
const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
if (output_info.tower_power != 1) return null;
if (output_info.forms.profile != left_info.profile) return null;
return .{
    .input = left_info,
    .output = output_info,
};
}

fn orthogonalTensorFormSpinorShiftStep(self: *Impl, step: coupling.CouplingStep)
↪ ?OrthogonalTensorFormSpinorInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    switch (simple.family) {
        .b, .d => {},
        else => return null,
    }

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = ordinaryTensorFormInfo(simple, left) orelse return null;
    if (!isSpinorDynkin(simple, right)) return null;
    const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
    if (output_info.tower_power != 1) return null;
    if (output_info.forms.total_power + 1 != left_info.total_power) return null;
    if (!formProfileRemovedThenShiftedUpOnce(left_info.profile,
    ↪ output_info.forms.profile)) return null;
    return .{
        .input = left_info,
        .output = output_info,
    };
}

fn orthogonalTensorFormSpinorRemoveShiftDownStep(self: *Impl, step: coupling.CouplingStep)
↪ ?OrthogonalTensorFormSpinorInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    switch (simple.family) {
        .b, .d => {},
        else => return null,
    }

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = ordinaryTensorFormInfo(simple, left) orelse return null;
    if (!isSpinorDynkin(simple, right)) return null;
    const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
    if (output_info.tower_power != 1) return null;

```

```

    if (output_info.forms.total_power + 1 != left_info.total_power) return null;
    if (!formProfileRemovedThenMovedDownOnce(left_info.profile,
    ↪ output_info.forms.profile)) return null;
    return .{
        .input = left_info,
        .output = output_info,
    };
}

fn orthogonalTensorFormSpinorShiftDownStep(self: *Impl, step: coupling.CouplingStep)
↪ ?OrthogonalTensorFormSpinorInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = ordinaryTensorFormInfo(simple, left) orelse return null;
    if (!isSpinorDynkin(simple, right)) return null;
    const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
    if (output_info.tower_power != 1) return null;
    if (output_info.forms.total_power != left_info.total_power) return null;
    if (!formProfileMovedDownOnce(left_info.profile, output_info.forms.profile, 1)) return
    ↪ null;
    return .{
        .input = left_info,
        .output = output_info,
    };
}

fn orthogonalTensorFormSpinorShiftDownTwoStep(self: *Impl, step: coupling.CouplingStep)
↪ ?OrthogonalTensorFormSpinorInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
    ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = ordinaryTensorFormInfo(simple, left) orelse return null;
    if (!isSpinorDynkin(simple, right)) return null;
    const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
    if (output_info.tower_power != 1) return null;
    if (output_info.forms.total_power != left_info.total_power) return null;
    if (!formProfileMovedDownOnce(left_info.profile, output_info.forms.profile, 2)) return
    ↪ null;
    return .{
        .input = left_info,
        .output = output_info,
    };
}

```

```

fn orthogonalTensorFormSpinorMixedShiftDownStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorFormSpinorInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
  ↳ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
  ↳ null;
  if (left_algebra.value != right_algebra.value or left_algebra.value !=
  ↳ output_algebra.value) return null;
  const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
  if (simple.family != .d) return null;

  const left = self.representations.irrepDynkin(step.left) orelse return null;
  const right = self.representations.irrepDynkin(step.right) orelse return null;
  const output = self.representations.irrepDynkin(step.output) orelse return null;
  const left_info = ordinaryTensorFormInfo(simple, left) orelse return null;
  if (!isSpinorDynkin(simple, right)) return null;
  const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
  if (output_info.tower_power != 1) return null;
  if (output_info.forms.total_power != left_info.total_power) return null;
  if (!formProfileShiftedAllDownWithOneExtraOnce(left_info.profile,
  ↳ output_info.forms.profile)) return null;
  return .{
    .input = left_info,
    .output = output_info,
  };
}

fn orthogonalTensorFormSpinorAllShiftDownStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorFormSpinorInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
  ↳ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
  ↳ null;
  if (left_algebra.value != right_algebra.value or left_algebra.value !=
  ↳ output_algebra.value) return null;
  const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
  if (simple.family != .d) return null;

  const left = self.representations.irrepDynkin(step.left) orelse return null;
  const right = self.representations.irrepDynkin(step.right) orelse return null;
  const output = self.representations.irrepDynkin(step.output) orelse return null;
  const left_info = ordinaryTensorFormInfo(simple, left) orelse return null;
  if (!isSpinorDynkin(simple, right)) return null;
  const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
  if (output_info.tower_power != 1) return null;
  if (output_info.forms.total_power != left_info.total_power) return null;
  if (!formProfileShiftedAllDownOnce(left_info.profile, output_info.forms.profile))
  ↳ return null;
  return .{
    .input = left_info,
    .output = output_info,
  };
}

fn orthogonalTensorFormSpinorShiftDownAnyStep(self: *Impl, step: coupling.CouplingStep)
↳ ?OrthogonalTensorFormSpinorInfo {
  const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
  const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
  ↳ null;
  const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
  ↳ null;

```

```

    if (left_algebra.value != right_algebra.value or left_algebra.value !=
        ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    switch (simple.family) {
        .b, .d => {},
        else => return null,
    }

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = ordinaryTensorFormInfo(simple, left) orelse return null;
    if (!isSpinorDynkin(simple, right)) return null;
    const output_info = orthogonalTensorSpinorInfo(simple, output) orelse return null;
    if (output_info.tower_power != 1) return null;
    if (output_info.forms.total_power != left_info.total_power) return null;
    if (!formProfileMovedDownAnyOnce(left_info.profile, output_info.forms.profile)) return
        ↪ null;
    return .{
        .input = left_info,
        .output = output_info,
    };
}

fn orthogonalTensorSpinorMiddleFormStep(self: *Impl, step: coupling.CouplingStep)
    ↪ ?OrthogonalTensorMiddleFormTerminalInfo {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return null;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
        ↪ null;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
        ↪ null;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
        ↪ output_algebra.value) return null;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return null;
    if (simple.family != .d) return null;

    const left = self.representations.irrepDynkin(step.left) orelse return null;
    const right = self.representations.irrepDynkin(step.right) orelse return null;
    const output = self.representations.irrepDynkin(step.output) orelse return null;
    const left_info = orthogonalTensorSpinorInfo(simple, left) orelse return null;
    if (left_info.tower_power != 1 or left_info.forms.total_power != 1) return null;
    if (!isSpinorDynkin(simple, right)) return null;
    const output_info = orthogonalFormInfo(simple, output) orelse return null;
    if (output_info.duality == .none) return null;
    if (output_info.rank != rendering.orthogonalDimension(simple) / 2) return null;
    return .{
        .input = left_info,
        .output = output_info,
    };
}

fn isOrthogonalDualIrrepSingletStep(self: *Impl, step: coupling.CouplingStep, irrep:
    ↪ store.IrrepHandle) bool {
    const metadata = self.representations.irrepMetadata(irrep) orelse return false;
    const dual = metadata.dual orelse return false;
    const output = self.representations.irrepDynkin(step.output) orelse return false;
    return step.left.value == irrep.value and step.right.value == dual.value and
        ↪ isZeroDynkin(output);
}

fn isOrthogonalFormPairSingletStep(self: *Impl, step: coupling.CouplingStep) bool {
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return false;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
        ↪ false;

```

```

const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ false;
if (left_algebra.value != right_algebra.value or left_algebra.value !=
↳ output_algebra.value) return false;
const simple = self.representations.simpleAlgebra(left_algebra) orelse return false;
switch (simple.family) {
    .b, .d => {},
    else => return false,
}
const left = self.representations.irrepDynkin(step.left) orelse return false;
if (orthogonalFormInfo(simple, left) == null) return false;
return self.isOrthogonalDualIrrepSingletStep(step, step.left);
}

fn isCartanProductStep(self: *Impl, step: coupling.CouplingStep) bool {
    if (step.multiplicity_copy != 0) return false;
    const left_algebra = self.representations.irrepAlgebra(step.left) orelse return false;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
↳ false;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
↳ false;
    if (left_algebra.value != right_algebra.value or left_algebra.value !=
↳ output_algebra.value) return false;
    const simple = self.representations.simpleAlgebra(left_algebra) orelse return false;
    switch (simple.family) {
        .b, .d => {},
        else => return false,
    }

    const left = self.representations.irrepDynkin(step.left) orelse return false;
    const right = self.representations.irrepDynkin(step.right) orelse return false;
    const output = self.representations.irrepDynkin(step.output) orelse return false;
    if (left.len != right.len or left.len != output.len) return false;
    if (isZeroDynkin(left) or isZeroDynkin(right)) return false;
    if (!isSpinorTowerDynkin(simple, left) or !isSpinorTowerDynkin(simple, right) or
↳ !isSpinorTowerDynkin(simple, output)) return false;
    for (output, 0..) |entry, index| {
        if (entry != left[index] + right[index]) return false;
    }
    return true;
}

fn isA2FundamentalCubicStep(self: *Impl, first: coupling.CouplingStep, second:
↳ coupling.CouplingStep) bool {
    return self.isA2FundamentalPairToAntiFundamentalStep(first) and
        second.left.value == first.output.value and
        self.isA2AntiFundamentalFundamentalSingletStep(second);
}

fn isA2FundamentalPairToAntiFundamentalStep(self: *Impl, step: coupling.CouplingStep) bool
↳ {
    const simple = self.stepSimpleAlgebra(step) orelse return false;
    if (simple.family != .a or simple.rank != 2) return false;
    const left = self.representations.irrepDynkin(step.left) orelse return false;
    const right = self.representations.irrepDynkin(step.right) orelse return false;
    const output = self.representations.irrepDynkin(step.output) orelse return false;
    return isA2FundamentalDynkin(left) and isA2FundamentalDynkin(right) and
↳ isA2AntiFundamentalDynkin(output);
}

fn isA2AntiFundamentalFundamentalSingletStep(self: *Impl, step: coupling.CouplingStep)
↳ bool {
    const simple = self.stepSimpleAlgebra(step) orelse return false;
    if (simple.family != .a or simple.rank != 2) return false;

```



```

    const left = self.representations.irrepDynkin(step.left) orelse return false;
    const right = self.representations.irrepDynkin(step.right) orelse return false;
    const output = self.representations.irrepDynkin(step.output) orelse return false;
    return isA2AntiFundamentalDynkin(left) and isA2FundamentalDynkin(right) and
    ↪ isZeroDynkin(output);
}

fn isE6FundamentalCubicStep(self: *Impl, first: coupling.CouplingStep, second:
↪ coupling.CouplingStep) bool {
    return self.isE6FundamentalPairToDualFundamentalStep(first) and
        second.left.value == first.output.value and
        self.isE6DualFundamentalFundamentalSingletStep(second);
}

fn isE6FundamentalPairToDualFundamentalStep(self: *Impl, step: coupling.CouplingStep) bool
↪ {
    const simple = self.stepSimpleAlgebra(step) orelse return false;
    if (simple.family != .e6 or simple.rank != 6) return false;
    const left = self.representations.irrepDynkin(step.left) orelse return false;
    const right = self.representations.irrepDynkin(step.right) orelse return false;
    const output = self.representations.irrepDynkin(step.output) orelse return false;
    return isE6FundamentalDynkin(left) and isE6FundamentalDynkin(right) and
    ↪ isE6DualFundamentalDynkin(output);
}

fn isE6DualFundamentalFundamentalSingletStep(self: *Impl, step: coupling.CouplingStep)
↪ bool {
    const simple = self.stepSimpleAlgebra(step) orelse return false;
    if (simple.family != .e6 or simple.rank != 6) return false;
    const left = self.representations.irrepDynkin(step.left) orelse return false;
    const right = self.representations.irrepDynkin(step.right) orelse return false;
    const output = self.representations.irrepDynkin(step.output) orelse return false;
    return isE6DualFundamentalDynkin(left) and isE6FundamentalDynkin(right) and
    ↪ isZeroDynkin(output);
}

fn isSimpleAdjointCubicStep(self: *Impl, first: coupling.CouplingStep, second:
↪ coupling.CouplingStep) bool {
    return self.isSimpleAdjointAdjointStep(first) and
        second.left.value == first.output.value and
        self.isSimpleAdjointPairSingletStep(second);
}

fn isSimpleAdjointAdjointStep(self: *Impl, step: coupling.CouplingStep) bool {
    if (step.multiplicity_copy != 0) return false;
    const algebra = self.representations.irrepAlgebra(step.left) orelse return false;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ false;
    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ false;
    if (algebra.value != right_algebra.value or algebra.value != output_algebra.value)
    ↪ return false;
    const simple = self.representations.simpleAlgebra(algebra) orelse return false;
    return self.isSimpleAdjointIrrep(simple, step.left) and
        self.isSimpleAdjointIrrep(simple, step.right) and
        self.isSimpleAdjointIrrep(simple, step.output);
}

fn isSimpleAdjointPairSingletStep(self: *Impl, step: coupling.CouplingStep) bool {
    if (step.multiplicity_copy != 0) return false;
    const algebra = self.representations.irrepAlgebra(step.left) orelse return false;
    const right_algebra = self.representations.irrepAlgebra(step.right) orelse return
    ↪ false;

```

```

    const output_algebra = self.representations.irrepAlgebra(step.output) orelse return
    ↪ false;
    if (algebra.value != right_algebra.value or algebra.value != output_algebra.value)
    ↪ return false;
    const simple = self.representations.simpleAlgebra(algebra) orelse return false;
    const output = self.representations.irrepDynkin(step.output) orelse return false;
    return self.isSimpleAdjointIrrep(simple, step.left) and
           self.isSimpleAdjointIrrep(simple, step.right) and
           isZeroDynkin(output);
}

fn isSimpleAdjointIrrep(self: *Impl, simple: symmetry.SimpleLieAlgebra, irrep:
    ↪ store.IrrepHandle) bool {
    const label = self.representations.irrepDynkin(irrep) orelse return false;
    if (isZeroDynkin(label)) return false;
    const metadata = self.representations.irrepMetadata(irrep) orelse return false;
    const dimension = metadata.dimension orelse return false;
    return dimension == simpleAdjointDimension(simple);
}

fn appendNamedProjectorAtoms(self: *Impl, atoms: *std.ArrayList(rendering.SymbolicAtom),
    ↪ steps: []const coupling.CouplingStep) !u32 {
    var appended: u32 = 0;
    for (steps) |step| {
        const span = try self.projectors.ensureNamedExpansion(step.projector);
        if (span.len != 1) return error.InvalidProjectorExpansion;
        const term = self.projectors.expansionTerm(span, 0) orelse return
        ↪ error.InvalidProjectorExpansion;
        try atoms.appendSlice(self.allocator, term.atoms);
        appended += 1;
    }
    return appended;
}

fn streamLocalFormulaPathTerms(self: *Impl, steps: []const coupling.CouplingStep, options:
    ↪ rendering.RenderOptions, filter: rendering.ExpansionFilter, sink: anytype, audit:
    ↪ *rendering.ExpansionAudit, scratch: *ExpansionScratch) !bool {
    if (steps.len == 0) return false;
    if (steps.len > max_local_formula_steps) return
    ↪ error.ProjectorExpansionNotImplemented;

    var term_counts = [_]u8{0} ** max_local_formula_steps;
    var term_indices = [_]u8{0} ** max_local_formula_steps;
    var formula_kinds = [_]projector.ProjectorFormulaKind{.named_only} **
    ↪ max_local_formula_steps;
    if (!try self.prepareLocalFormulaTermFrontier(steps, term_counts[0..steps.len],
    ↪ formula_kinds[0..steps.len])) return false;

    while (true) {
        scratch.local_path_atoms.clearRetainingCapacity();
        const coefficient = try
        ↪ self.appendLocalFormulaPathTermAtoms(&scratch.local_path_atoms, steps,
        ↪ formula_kinds[0..steps.len], term_indices[0..steps.len]);
        try self.emitLocalFormulaTerm(&scratch.local_term_atoms,
        ↪ scratch.local_path_atoms.items, coefficient, options, filter, sink, audit,
        ↪ &scratch.lowered_atoms);
        if (!incrementLocalFormulaTermIndices(term_indices[0..steps.len],
        ↪ term_counts[0..steps.len])) break;
    }
    return true;
}

fn prepareLocalFormulaTermFrontier(self: *Impl, steps: []const coupling.CouplingStep,
    ↪ term_counts: []u8, formula_kinds: []projector.ProjectorFormulaKind) !bool {

```



```

    if (steps.len == 0 or steps.len != term_counts.len or steps.len != formula_kinds.len)
    ↪ return false;
    for (steps, 0..) |step, step_index| {
        const formula_kind = self.localProductFormulaKind(step);
        if (formula_kind == .named_only) return false;
        const cached_kind = try self.projectors.ensureFormulaProgram(step.projector);
        if (cached_kind != formula_kind) return error.InvalidProjectorExpansion;
        const count = try self.localFormulaStepTermCount(step, cached_kind);
        if (count == 0) return false;
        term_counts[step_index] = count;
        formula_kinds[step_index] = cached_kind;
    }
    return true;
}

fn localFormulaStepTermCount(self: *Impl, step: coupling.CouplingStep, formula_kind:
↪ projector.ProjectorFormulaKind) !?u8 {
    if (try self.localFormulaProjectionStepTermCount(step, formula_kind)) |count| return
    ↪ count;
    return switch (formula_kind) {
        .backend_specific_structure, .named_only => 0,
        else => 1,
    };
}

fn localFormulaProjectionStepTermCount(self: *Impl, step: coupling.CouplingStep,
↪ formula_kind: projector.ProjectorFormulaKind) !?u8 {
    if (formula_kind == .orthogonal_structural_projection) {
        const count = try self.localStructuralTermCount(step, 0, 1, 2);
        if (count == 0) return error.UnsupportedStructuralProjectorTerm;
        return count;
    }
    var atoms: std.ArrayList(rendering.SymbolicAtom) = .empty;
    defer atoms.deinit(self.allocator);
    if (!try self.appendLocalFormulaStepAtomForKind(&atoms, step, 0, 1, 2, formula_kind))
    ↪ return null;
    if (atoms.items.len != 1) return null;
    return switch (atoms.items[0]) {
        .tensor_spinor_projection => |projection| blk: {
            const spec = tensorSpinorProjectionSpec(projection);
            const count = try self.tensor_spinor_projection_programs.termCount(spec);
            if (count == 0) return error.UnsupportedTensorSpinorProjectionTerm;
            break :blk count;
        },
        .tensor_form_projection => |projection| blk: {
            const count = try
            ↪ self.tensor_form_projection_programs.termCount(tensorFormProjectionSpec(projection));
            if (count == 0) return error.UnsupportedTensorFormProjectionTerm;
            break :blk count;
        },
        else => null,
    };
}

fn appendLocalFormulaPathTermAtoms(self: *Impl, atoms:
↪ *std.ArrayList(rendering.SymbolicAtom), steps: []const coupling.CouplingStep,
↪ formula_kinds: []const projector.ProjectorFormulaKind, term_indices: []const u8)
↪ !rendering.RationalId {
    if (steps.len != formula_kinds.len or steps.len != term_indices.len) return
    ↪ error.InvalidProjectorExpansion;
    const scratch_base: u32 = @intCast(steps.len + 1);
    var coefficient = rendering.rationalOne();
    var previous_formula_atom: ?rendering.SymbolicAtom = null;
    for (steps, 0..) |step, step_index| {

```

```

const left_index: rendering.IndexRef = if (step_index == 0) 0 else scratch_base +
↳ @as(u32, @intCast(step_index - 1));
const right_index: rendering.IndexRef = @intCast(step_index + 1);
const output_index: rendering.IndexRef = scratch_base + @as(u32,
↳ @intCast(step_index));
const atom_index = atoms.items.len;
const step_coefficient = try self.appendLocalFormulaStepTermAtoms(atoms, step,
↳ left_index, right_index, output_index, formula_kinds[step_index],
↳ term_indices[step_index]);
coefficient = try multiplyRenderingRationals(coefficient, step_coefficient);
if (atoms.items.len == atom_index) return error.InvalidProjectorExpansion;
const current_formula_atom = atoms.items[atom_index];
if (previous_formula_atom) |previous| {
    try appendCliffordProductAtoms(self.allocator, atoms, previous,
↳ current_formula_atom);
}
previous_formula_atom = current_formula_atom;
}
return coefficient;
}

fn appendLocalFormulaStepTermAtoms(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef,
↳ formula_kind: projector.ProjectorFormulaKind, term_index: u8) !rendering.RationalId {
    if (try self.appendLocalFormulaProjectionStepTerm(atoms, step, left_index,
↳ right_index, output_index, formula_kind, term_index)) |coefficient| return
↳ coefficient;
    if (term_index != 0) return error.ProjectorConstructorTermOutOfBounds;
    if (!try self.appendLocalFormulaStepAtomForKind(atoms, step, left_index, right_index,
↳ output_index, formula_kind)) return error.ProjectorExpansionNotImplemented;
    return rendering.rationalOne();
}

fn appendLocalFormulaProjectionStepTerm(self: *Impl, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef,
↳ formula_kind: projector.ProjectorFormulaKind, term_index: u8) !?rendering.RationalId {
    if (formula_kind == .orthogonal_structural_projection) {
        if (try self.localStructuralTermCount(step, left_index, right_index, output_index)
↳ == 0) return error.UnsupportedStructuralProjectorTerm;
        return try self.appendLocalStructuralTerm(atoms, step, left_index, right_index,
↳ output_index, term_index);
    }
    var compact_atoms: std.ArrayList(rendering.SymbolicAtom) = .empty;
    defer compact_atoms.deinit(self.allocator);
    if (!try self.appendLocalFormulaStepAtomForKind(&compact_atoms, step, left_index,
↳ right_index, output_index, formula_kind)) return null;
    if (compact_atoms.items.len != 1) return null;
    return switch (compact_atoms.items[0]) {
        .tensor_spinor_projection => |projection| blk: {
            const spec = tensorSpinorProjectionSpec(projection);
            if (try self.tensor_spinor_projection_programs.termCount(spec) == 0) return
↳ error.UnsupportedTensorSpinorProjectionTerm;
            break :blk try self.tensor_spinor_projection_programs.appendTerm(atoms, spec,
↳ term_index);
        },
        .tensor_form_projection => |projection| blk: {
            const spec = tensorFormProjectionSpec(projection);
            if (try self.tensor_form_projection_programs.termCount(spec) == 0) return
↳ error.UnsupportedTensorFormProjectionTerm;
            break :blk try self.tensor_form_projection_programs.appendTerm(atoms, spec,
↳ term_index);
        },
    },
}

```

```

        else => null,
    };
}

fn localStructuralTermCount(self: *Impl, step: coupling.CouplingStep, left_index:
↳ rendering.IndexRef, right_index: rendering.IndexRef, output_index: rendering.IndexRef)
↳ !u8 {
    const structural = self.orthogonalStructuralStep(step, left_index, right_index,
↳ output_index) orelse return 0;
    return
↳ self.structural_projection_programs.termCount(structuralProjectorSpecFromOrthogonalStep(structural))
}

fn appendLocalStructuralTerm(self: *Impl, atoms: *std.ArrayList(rendering.SymbolicAtom),
↳ step: coupling.CouplingStep, left_index: rendering.IndexRef, right_index:
↳ rendering.IndexRef, output_index: rendering.IndexRef, term_index: u8)
↳ !rendering.RationalId {
    const structural = self.orthogonalStructuralStep(step, left_index, right_index,
↳ output_index) orelse return error.ProjectorExpansionNotImplemented;
    return self.structural_projection_programs.appendTerm(atoms,
↳ structuralProjectorSpecFromOrthogonalStep(structural), term_index);
}

fn emitLocalFormulaTerm(self: *Impl, atoms: *std.ArrayList(rendering.SymbolicAtom),
↳ formula_atoms: []const rendering.SymbolicAtom, coefficient: rendering.RationalId,
↳ options: rendering.RenderOptions, filter: rendering.ExpansionFilter, sink: anytype,
↳ audit: *rendering.ExpansionAudit, lowered_atoms:
↳ *std.ArrayList(rendering.SymbolicAtom)) !void {
    atoms.clearRetainingCapacity();
    try atoms.appendSlice(self.allocator, formula_atoms);
    const term: rendering.SymbolicTerm = .{
        .coefficient = coefficient,
        .atoms = atoms.items,
    };
    const decision = try filter.decide(term);
    audit.recordDecision(decision);
    switch (decision) {
        .reject => {},
        .accept, .undecided => try emitRenderedTerm(self.allocator, options, sink, term,
↳ audit, lowered_atoms),
    }
}

fn streamBoundaryFormulaTerms(self: *Impl, request: coupling.InvariantBasisRequest, steps:
↳ []const coupling.CouplingStep, options: rendering.RenderOptions, filter:
↳ rendering.ExpansionFilter, sink: anytype, audit: *rendering.ExpansionAudit, scratch:
↳ *ExpansionScratch) !void {
    var formula_count: u32 = 0;
    var formula: BoundaryFormula = undefined;
    for (request.external_legs, 0..) |leg, leg_index| {
        const handle = switch (leg.realization) {
            .handle => |handle| handle,
            .primitive_irrep, .registered_name => continue,
        };
        if (try self.orthogonalVectorSpinorBoundaryFormula(handle, leg, request,
↳ leg_index)) |found| {
            formula = found;
            formula_count += 1;
        } else {
            const spec = self.realizations.realizationSpecFor(handle) orelse return
↳ error.UnknownRealization;
            if (spec.ambient.len != 0) return error.ProjectorExpansionNotImplemented;
        }
    }
}

```

```

if (formula_count != 1) return error.ProjectorExpansionNotImplemented;
const cached_kind = try self.projectors.ensureFormulaProgram(formula.operator_id);
if (cached_kind != .orthogonal_gamma_traceless) return
↳ error.InvalidProjectorExpansion;

const spec: projector_constructor.VectorSpinorTracelessSpec = .{
    .operator_id = formula.operator_id,
    .vector = formula.vector,
    .spinor = formula.spinor,
    .orthogonal_dimension = formula.orthogonal_dimension,
    .chirality = formula.chirality,
};

if (steps.len == 0) {
    try self.streamBoundaryFormulaTermsWithPath(spec, &.{}, rendering.rationalOne(),
    ↳ options, filter, sink, audit, scratch);
    return;
}

scratch.boundary_path_atoms.clearRetainingCapacity();
if (try self.appendSpecialExplicitPathAtoms(&scratch.boundary_path_atoms, steps)) {
    try self.streamBoundaryFormulaTermsWithPath(spec,
    ↳ scratch.boundary_path_atoms.items, rendering.rationalOne(), options, filter,
    ↳ sink, audit, scratch);
    return;
}

if (steps.len > max_local_formula_steps) return
↳ error.ProjectorExpansionNotImplemented;
var term_counts = [_]u8{0} ** max_local_formula_steps;
var term_indices = [_]u8{0} ** max_local_formula_steps;
var formula_kinds = [_]projector.ProjectorFormulaKind{.named_only} **
↳ max_local_formula_steps;
if (!try self.prepareLocalFormulaTermFrontier(steps, term_counts[0..steps.len],
↳ formula_kinds[0..steps.len])) return error.ProjectorExpansionNotImplemented;

while (true) {
    scratch.boundary_path_atoms.clearRetainingCapacity();
    const path_coefficient = try
    ↳ self.appendLocalFormulaPathTermAtoms(&scratch.boundary_path_atoms, steps,
    ↳ formula_kinds[0..steps.len], term_indices[0..steps.len]);
    try self.streamBoundaryFormulaTermsWithPath(spec,
    ↳ scratch.boundary_path_atoms.items, path_coefficient, options, filter, sink,
    ↳ audit, scratch);
    if (!incrementLocalFormulaTermIndices(term_indices[0..steps.len],
    ↳ term_counts[0..steps.len])) break;
}

}

fn streamBoundaryFormulaTermsWithPath(self: *Impl, spec:
↳ projector_constructor.VectorSpinorTracelessSpec, path_atoms: []const
↳ rendering.SymbolicAtom, path_coefficient: rendering.RationalId, options:
↳ rendering.RenderOptions, filter: rendering.ExpansionFilter, sink: anytype, audit:
↳ *rendering.ExpansionAudit, scratch: *ExpansionScratch) !void {
    var term_index: u8 = 0;
    while (term_index < projector_constructor.vectorSpinorTracelessTermCount(spec)) :
    ↳ (term_index += 1) {
        scratch.boundary_formula_atoms.clearRetainingCapacity();
        const formula_coefficient = try
        ↳ projector_constructor.appendVectorSpinorTracelessTerm(self.allocator,
        ↳ &scratch.boundary_formula_atoms, spec, term_index);
        const coefficient = try multiplyRenderingRationals(path_coefficient,
        ↳ formula_coefficient);

```

```

        try self.emitBoundaryFormulaTerm(&scratch.boundary_term_atoms, path_atoms,
        ↪ scratch.boundary_formula_atoms.items, coefficient, options, filter, sink,
        ↪ audit, &scratch.lowered_atoms);
    }
}

fn emitBoundaryFormulaTerm(self: *Impl, atoms: *std.ArrayList(rendering.SymbolicAtom),
↪ path_atoms: []const rendering.SymbolicAtom, formula_atoms: []const
↪ rendering.SymbolicAtom, coefficient: rendering.RationalIid, options:
↪ rendering.RenderOptions, filter: rendering.ExpansionFilter, sink: anytype, audit:
↪ *rendering.ExpansionAudit, lowered_atoms: *std.ArrayList(rendering.SymbolicAtom))
↪ !void {
    atoms.clearRetainingCapacity();
    try atoms.appendSlice(self.allocator, formula_atoms);
    try atoms.appendSlice(self.allocator, path_atoms);
    const term: rendering.SymbolicTerm = .{
        .coefficient = coefficient,
        .atoms = atoms.items,
    };
    const decision = try filter.decide(term);
    audit.recordDecision(decision);
    switch (decision) {
        .reject => {},
        .accept, .undecided => {
            try emitRenderedTerm(self.allocator, options, sink, term, audit,
            ↪ lowered_atoms);
        },
    }
}

fn hasOrthogonalVectorSpinorBoundaryFormula(self: *Impl, handle:
↪ realization.RealizationHandle) !bool {
    const spec = self.realizations.realizationSpecFor(handle) orelse return
    ↪ error.UnknownRealization;
    return self.isOrthogonalVectorSpinorBoundarySpec(spec);
}

fn orthogonalVectorSpinorBoundaryFormulaAudit(self: *Impl, handle:
↪ realization.RealizationHandle) !projector.ProjectorFormulaAudit {
    const spec = self.realizations.realizationSpecFor(handle) orelse return
    ↪ error.UnknownRealization;
    if (!self.isOrthogonalVectorSpinorBoundarySpec(spec)) return .{};
    const target_algebra = self.representations.irrepAlgebra(spec.channel.irrep) orelse
    ↪ return error.UnknownIrrep;
    const simple = self.representations.simpleAlgebra(target_algebra) orelse return
    ↪ error.UnsupportedAlgebraFamily;
    return orthogonalVectorSpinorFormulaAudit(rendering.orthogonalDimension(simple));
}

fn orthogonalVectorSpinorBoundaryFormula(self: *Impl, handle:
↪ realization.RealizationHandle, leg: realization.ExternalLeg, request:
↪ coupling.InvariantBasisRequest, leg_index: usize) !?BoundaryFormula {
    const spec = self.realizations.realizationSpecFor(handle) orelse return
    ↪ error.UnknownRealization;
    if (!self.isOrthogonalVectorSpinorBoundarySpec(spec)) return null;
    const target_algebra = self.representations.irrepAlgebra(spec.channel.irrep) orelse
    ↪ return error.UnknownIrrep;
    const simple = self.representations.simpleAlgebra(target_algebra) orelse return
    ↪ error.UnsupportedAlgebraFamily;
    const spinor = try self.boundarySpinorFactor(spec);
    const spinor_label = self.representations.irrepDynkin(spinor) orelse return
    ↪ error.UnknownIrrep;
    const first_index = externalIndexOffset(request, leg_index);

```

```

const vector_slot = findIndexKind(leg.indices, .vector) orelse return
↳ error.InvalidBoundaryFormulaIndices;
const spinor_slot = findSpinorIndex(leg.indices) orelse return
↳ error.InvalidBoundaryFormulaIndices;
return .{
    .operator_id = try self.boundaryRealizationProjector(handle),
    .vector = first_index + @as(u32, @intCast(vector_slot)),
    .spinor = first_index + @as(u32, @intCast(spinor_slot)),
    .orthogonal_dimension = rendering.orthogonalDimension(simple),
    .chirality = @intFromEnum(rendering.gammaChiralityTag(simple, spinor_label)),
};
}

fn isOrthogonalVectorSpinorBoundarySpec(self: *Impl, spec: realization.RealizationSpec)
↳ bool {
    if (spec.ambient.len != 2) return false;
    const target_algebra = self.representations.irrepAlgebra(spec.channel.irrep) orelse
↳ return false;
    const simple = self.representations.simpleAlgebra(target_algebra) orelse return false;
    switch (simple.family) {
        .b, .d => {},
        else => return false,
    }
    const target = self.representations.irrepDynkin(spec.channel.irrep) orelse return
↳ false;
    if (!isVectorSpinorDynkin(simple, target)) return false;
    const spinor = self.boundarySpinorFactor(spec) catch return false;
    const vector = self.boundaryVectorFactor(spec) catch return false;
    const spinor_label = self.representations.irrepDynkin(spinor) orelse return false;
    const vector_label = self.representations.irrepDynkin(vector) orelse return false;
    return isSpinorDynkin(simple, spinor_label) and isVectorDynkin(vector_label);
}

fn boundarySpinorFactor(self: *Impl, spec: realization.RealizationSpec) !store.IrrepHandle
↳ {
    for (spec.ambient) |factor| {
        const algebra = self.representations.irrepAlgebra(factor) orelse return
↳ error.UnknownIrrep;
        const simple = self.representations.simpleAlgebra(algebra) orelse return
↳ error.UnsupportedAlgebraFamily;
        const label = self.representations.irrepDynkin(factor) orelse return
↳ error.UnknownIrrep;
        if (isSpinorDynkin(simple, label)) return factor;
    }
    return error.InvalidBoundaryFormulaChannel;
}

fn boundaryVectorFactor(self: *Impl, spec: realization.RealizationSpec) !store.IrrepHandle
↳ {
    for (spec.ambient) |factor| {
        const label = self.representations.irrepDynkin(factor) orelse return
↳ error.UnknownIrrep;
        if (isVectorDynkin(label)) return factor;
    }
    return error.InvalidBoundaryFormulaChannel;
}

fn externalLegIrrep(self: *Impl, leg: realization.ExternalLeg) !store.IrrepHandle {
    return switch (leg.realization) {
        .primitive_irrep => |irrep| irrep,
        .handle => |handle| self.realizations.targetIrrep(handle) orelse return
↳ error.UnknownRealization,
        .registered_name => return error.RegisteredRealizationNotResolved,
    };
};

```

```

}

fn targetIrrep(self: *Impl, algebra: store.AlgebraHandle, target: coupling.TargetIrrep)
↳ !store.IrrepHandle {
    return switch (target) {
        .irrep => |irrep| irrep,
        .singlet => singlet: {
            const simple = self.representations.simpleAlgebra(algebra) orelse return
            ↳ error.UnsupportedAlgebraFamily;
            const label = try self.allocator.alloc(i16, simple.rank);
            defer self.allocator.free(label);
            @memset(label, 0);
            break :singlet try self.representations.internIrrep(algebra, .{ .dynkin =
            ↳ label });
        },
    };
}

};

fn continuationKey(next_index: usize, irrep: store.IrrepHandle) u64 {
    return (@as(u64, @intCast(next_index)) << 32) | @as(u64, irrep.value);
}

const PartialPathNode = struct {
    output: store.IrrepHandle,
    parent: u32,
    has_parent: bool,
    step: coupling.CouplingStep,
};

const OrthogonalFormInfo = struct {
    rank: u8,
    duality: rendering.DualityTag = .none,
};

const SpinorTowerInfo = struct {
    power: u16,
    chirality: rendering.GammaChirality = .none,
};

const OrthogonalFormSetInfo = struct {
    first_rank: u8,
    count: u8,
    total_power: u8,
    mask: u64,
    profile: u128,
    duality: rendering.DualityTag = .none,
};

const OrthogonalTensorSpinorInfo = struct {
    forms: OrthogonalFormSetInfo,
    tower_power: u16,
    chirality: rendering.GammaChirality = .none,
};

const OrthogonalTensorSpinorTransitionInfo = struct {
    input: OrthogonalTensorSpinorInfo,
    output: OrthogonalTensorSpinorInfo,
};

const OrthogonalTensorSpinorTowerTerminalInfo = struct {
    input: OrthogonalTensorSpinorInfo,
    output: SpinorTowerInfo,
};

```

```

const OrthogonalTensorFormTerminalInfo = struct {
    input: OrthogonalTensorSpinorInfo,
    output: OrthogonalFormSetInfo,
};

const OrthogonalTensorMiddleFormTerminalInfo = struct {
    input: OrthogonalTensorSpinorInfo,
    output: OrthogonalFormInfo,
};

const OrthogonalTensorFormSpinorInfo = struct {
    input: OrthogonalFormSetInfo,
    output: OrthogonalTensorSpinorInfo,
};

const max_orthogonal_young_rows = 8;

const OrthogonalIrrepKind = enum {
    scalar,
    form_profile,
    vector_young_shape,
    spinor_tower,
    tensor_spinor,
    unsupported,
};

const OrthogonalIrrepDescriptor = struct {
    kind: OrthogonalIrrepKind,
    dimension: u16,
    form_profile: u128 = 0,
    form_mask: u64 = 0,
    form_count: u8 = 0,
    form_total_power: u8 = 0,
    form_rank: u8 = 0,
    form_duality: rendering.DualityTag = .none,
    young_row_count: u8 = 0,
    young_rows: [max_orthogonal_young_rows]u8 = [_]u8{0} ** max_orthogonal_young_rows,
    has_spinor: bool = false,
    chirality: u8 = 0,
    tower_power: u16 = 0,
};

const OrthogonalEndpoint = struct {
    irrep: OrthogonalIrrepDescriptor,
    index: rendering.IndexRef,
};

const OrthogonalStructuralStep = struct {
    operator_id: u32,
    dimension: u16,
    left: OrthogonalEndpoint,
    right: OrthogonalEndpoint,
    output: OrthogonalEndpoint,
    multiplicity_copy: u16,
};

const BoundaryFormula = struct {
    operator_id: u32,
    vector: rendering.IndexRef,
    spinor: rendering.IndexRef,
    orthogonal_dimension: u16,
    chirality: u8,
};

```



```

const max_local_formula_steps = 32;

const ExpansionScratch = struct {
    atoms: std.ArrayList(rendering.SymbolicAtom) = .empty,
    local_path_atoms: std.ArrayList(rendering.SymbolicAtom) = .empty,
    local_term_atoms: std.ArrayList(rendering.SymbolicAtom) = .empty,
    boundary_path_atoms: std.ArrayList(rendering.SymbolicAtom) = .empty,
    boundary_formula_atoms: std.ArrayList(rendering.SymbolicAtom) = .empty,
    boundary_term_atoms: std.ArrayList(rendering.SymbolicAtom) = .empty,
    lowered_atoms: std.ArrayList(rendering.SymbolicAtom) = .empty,

    fn deinit(self: *ExpansionScratch, allocator: std.mem.Allocator) void {
        self.lowered_atoms.deinit(allocator);
        self.boundary_term_atoms.deinit(allocator);
        self.boundary_formula_atoms.deinit(allocator);
        self.boundary_path_atoms.deinit(allocator);
        self.local_term_atoms.deinit(allocator);
        self.local_path_atoms.deinit(allocator);
        self.atoms.deinit(allocator);
        self.* = .{};
    }
};

const PathEnumeration = struct {
    allocator: std.mem.Allocator,
    paths: std.ArrayList(coupling.CouplingPath) = .empty,
    steps: std.ArrayList(coupling.CouplingStep) = .empty,

    fn init(allocator: std.mem.Allocator) !PathEnumeration {
        return .{ .allocator = allocator };
    }

    fn deinit(self: *PathEnumeration) void {
        self.steps.deinit(self.allocator);
        self.paths.deinit(self.allocator);
        self.* = undefined;
    }
};

const BackendRegistry = struct {
    records: [6]backend.Record,

    fn init(generic: *generic_backend.Backend) BackendRegistry {
        return .{ .records = .{
            generic.record(0, .a),
            generic.record(1, .b),
            generic.record(2, .d),
            generic.record(3, .e6),
            generic.record(4, .e7),
            generic.record(5, .e8),
        } };
    }

    fn select(self: *const BackendRegistry, simple: symmetry.SimpleLieAlgebra) ?backend.Record
    ↪ {
        for (self.records) |record| {
            if (record.family == simple.family) return record;
        }
        return null;
    }
};

fn EvaluationSinkAdapter(comptime Sink: type) type {

```

```

return struct {
    inner: Sink,

    pub fn emitTerm(self: *@This(), term: rendering.SymbolicTerm) !void {
        try rendering.emitEvaluationTerm(self.inner, .{
            .coefficient = term.coefficient,
            .atoms = term.atoms,
        });
    }
};
}

fn emitRenderedTerm(allocator: std.mem.Allocator, options: rendering.RenderOptions, sink:
↳ anytype, term: rendering.SymbolicTerm, audit: *rendering.ExpansionAudit, lowered_atoms:
↳ *std.ArrayList(rendering.SymbolicAtom)) !void {
    var emitted_term = term;
    if (options.lower_clifford_product_atoms or options.gamma_only) {
        lowered_atoms.clearRetainingCapacity();
        audit.clifford_product_factors += try
↳ rendering.appendLoweredCliffordProductAtoms(allocator, lowered_atoms, term);
        emitted_term = .{
            .coefficient = term.coefficient,
            .atoms = lowered_atoms.items,
        };
    }
    if (options.gamma_only) {
        var lowering_audit: rendering.AtomLoweringAudit = .{};
        try lowering_audit.recordTerm(emitted_term);
        if (!lowering_audit.gammaOnly()) return error.NonGammaOnlyRenderTerm;
    }

    try rendering.emitTerm(sink, emitted_term);
    audit.emitted += 1;
    if (!options.stream_clifford_product_factors or options.lower_clifford_product_atoms)
↳ return;
    if (comptime sinkSupportsCliffordProductFactors(@TypeOf(sink))) {
        audit.clifford_product_factors += try rendering.streamCliffordProductFactors(term,
↳ sink);
    } else {
        return error.UnsupportedCliffordProductFactorSink;
    }
}

fn incrementLocalFormulaTermIndices(indices: []u8, counts: []const u8) bool {
    if (indices.len != counts.len) return false;
    var index = indices.len;
    while (index > 0) {
        index -= 1;
        indices[index] += 1;
        if (indices[index] < counts[index]) return true;
        indices[index] = 0;
    }
    return false;
}

fn tensorSpinorProjectionSpec(projection: rendering.TensorSpinorProjection)
↳ projector_constructor.TensorSpinorProjectionSpec {
    return .{
        .operator_id = projection.operator_id,
        .left = projection.left,
        .right = projection.right,
        .output = projection.output,
        .orthogonal_dimension = projection.orthogonal_dimension,
        .left_has_spinor = projection.left_has_spinor,
    }
}

```

```

        .right_has_spinor = projection.right_has_spinor,
        .output_has_spinor = projection.output_has_spinor,
        .right_chirality = projection.right_chirality,
        .input_form_profile = projection.input_form_profile,
        .input_form_count = projection.input_form_count,
        .input_tower_power = projection.input_tower_power,
        .form_rank = projection.form_rank,
        .form_count = projection.form_count,
        .form_mask = projection.form_mask,
        .form_profile = projection.form_profile,
        .tower_power = projection.tower_power,
        .chirality = projection.chirality,
        .duality = projection.duality,
    };
}

fn tensorFormProjectionSpec(projection: rendering.TensorFormProjection)
↳ projector_constructor.TensorFormProjectionSpec {
    return .{
        .operator_id = projection.operator_id,
        .left = projection.left,
        .right = projection.right,
        .output = projection.output,
        .orthogonal_dimension = projection.orthogonal_dimension,
        .input_form_profile = projection.input_form_profile,
        .input_form_mask = projection.input_form_mask,
        .output_form_profile = projection.output_form_profile,
        .output_form_count = projection.output_form_count,
        .output_form_rank = projection.output_form_rank,
        .output_duality = projection.output_duality,
        .right_chirality = projection.right_chirality,
        .chirality = projection.chirality,
    };
}

fn structuralProjectorSpecFromOrthogonalStep(step: OrthogonalStructuralStep)
↳ projector_constructor.StructuralProjectorSpec {
    return .{
        .operator_id = step.operator_id,
        .orthogonal_dimension = step.dimension,
        .left = structuralEndpointFromOrthogonalEndpoint(step.left),
        .right = structuralEndpointFromOrthogonalEndpoint(step.right),
        .output = structuralEndpointFromOrthogonalEndpoint(step.output),
    };
}

fn structuralEndpointFromOrthogonalEndpoint(endpoint: OrthogonalEndpoint)
↳ projector_constructor.StructuralEndpoint {
    const irrep = endpoint.irrep;
    return .{
        .index = endpoint.index,
        .form_profile = irrep.form_profile,
        .form_mask = irrep.form_mask,
        .form_count = irrep.form_count,
        .form_rank = irrep.form_rank,
        .form_duality = irrep.form_duality,
        .tower_power = irrep.tower_power,
        .chirality = irrep.chirality,
        .has_spinor = irrep.has_spinor,
    };
}

fn multiplyRenderingRationals(left: rendering.RationalId, right: rendering.RationalId)
↳ !rendering.RationalId {

```

```

    if (rendering.rationalEq(left, rendering.rationalOne())) return right;
    if (rendering.rationalEq(right, rendering.rationalOne())) return left;
    var left_numerator: i128 = left.numerator;
    var right_numerator: i128 = right.numerator;
    var left_denominator: u128 = left.denominator;
    var right_denominator: u128 = right.denominator;
    if (left_numerator == 0 or right_numerator == 0) return rendering.rationalFromSmall(0, 1);

    const left_cross = gcdU128(absI128(left_numerator), right_denominator);
    left_numerator = @divExact(left_numerator, @as(i128, @intCast(left_cross)));
    right_denominator = @divExact(right_denominator, left_cross);
    const right_cross = gcdU128(absI128(right_numerator), left_denominator);
    right_numerator = @divExact(right_numerator, @as(i128, @intCast(right_cross)));
    left_denominator = @divExact(left_denominator, right_cross);

    return .{
        .numerator = try std.math.mul(i128, left_numerator, right_numerator),
        .denominator = try std.math.mul(u128, left_denominator, right_denominator),
    };
}

fn absI128(value: i128) u128 {
    if (value == std.math.minInt(i128)) return @as(u128, 1) << 127;
    return if (value < 0) @intCast(-value) else @intCast(value);
}

fn gcdU128(left: u128, right: u128) u128 {
    var a = left;
    var b = right;
    while (b != 0) {
        const next = a % b;
        a = b;
        b = next;
    }
    return if (a == 0) 1 else a;
}

fn sinkSupportsCliffordProductFactors(comptime Sink: type) bool {
    const target = switch (@typeInfo(Sink)) {
        .pointer => |pointer| pointer.child,
        else => Sink,
    };
    return @hasDecl(target, "emitCliffordProductFactor");
}

```

```

const CountingSink = struct {
    term_count: u32 = 0,
    atom_count: u32 = 0,
    metric_pair_count: u32 = 0,
    gamma_matrix_count: u32 = 0,
    gamma_form_count: u32 = 0,
    gamma_action_count: u32 = 0,
    exterior_gamma_action_count: u32 = 0,
    clifford_product_count: u32 = 0,
    clifford_product_factor_count: u32 = 0,
    clifford_product_metric_factor_count: u32 = 0,
    clifford_product_output_vector_factor_count: u32 = 0,
    gamma_trace_count: u32 = 0,
    spinor_pair_count: u32 = 0,
    vector_spinor_identity_count: u32 = 0,
    identity_route_count: u32 = 0,
    spinor_index_delta_count: u32 = 0,
    spinor_symmetrized_product_count: u32 = 0,
    spinor_tower_contract_count: u32 = 0,
    spinor_tower_contract_adjoint_count: u32 = 0,
    product_identity_count: u32 = 0,
    epsilon_count: u32 = 0,
    generalized_delta_count: u32 = 0,
    cartan_product_count: u32 = 0,
    tensor_spinor_projection_count: u32 = 0,
    tensor_form_projection_count: u32 = 0,
    tensor_form_gamma_wedge_count: u32 = 0,
    tensor_form_gamma_map_count: u32 = 0,
    tensor_form_gamma_map_adjoint_count: u32 = 0,
    tensor_form_spinor_pair_count: u32 = 0,
    structure_constant_count: u32 = 0,
    tensor_spinor_profile_audit: projector_constructor.StructuralProjectorCoverageAudit = .{},
    gamma_operator_count: u32 = 0,
    projector_operator_count: u32 = 0,
    first_gamma_operator_id: ?u32 = null,
    first_gamma_dimension: ?u16 = null,
    first_gamma_rank: ?u8 = null,
    first_gamma_chirality: ?u8 = null,
    first_gamma_form_operator_id: ?u32 = null,
    first_gamma_form_dimension: ?u16 = null,
    first_gamma_form_rank: ?u8 = null,
    first_gamma_form_chirality: ?u8 = null,
    first_gamma_form_duality: ?rendering.DualityTag = null,
    first_exterior_gamma_action_operator_id: ?u32 = null,
    first_exterior_gamma_action_dimension: ?u16 = null,
    first_exterior_gamma_action_chirality: ?u8 = null,
    first_clifford_product_left_rank: ?u8 = null,
    first_clifford_product_right_rank: ?u8 = null,
    first_clifford_product_output_rank: ?u8 = null,
    first_clifford_product_contractions: ?u8 = null,
    first_clifford_product_metric_count: ?u16 = null,
    first_clifford_product_free_vector_count: ?u16 = null,
    first_clifford_product_coefficient: ?i64 = null,
    first_clifford_product_factor_atom_index: ?u32 = null,
    first_clifford_product_factor_index: ?u16 = null,

```

```

first_clifford_product_factor_left_operator_id: ?u32 = null,
first_clifford_product_factor_right_operator_id: ?u32 = null,
first_clifford_product_factor_dimension: ?u16 = null,
first_clifford_product_factor_output_rank: ?u8 = null,
first_clifford_product_factor_coefficient: ?i64 = null,
first_gamma_trace_operator_id: ?u32 = null,
first_gamma_trace_dimension: ?u16 = null,
first_gamma_trace_chirality: ?u8 = null,
first_gamma_trace_coefficient: ?rendering.RationalValue = null,
first_spinor_pair_operator_id: ?u32 = null,
first_spinor_pair_dimension: ?u16 = null,
first_spinor_pair_left_chirality: ?u8 = null,
first_spinor_pair_right_chirality: ?u8 = null,
first_identity_route_operator_id: ?u32 = null,
first_spinor_index_delta_operator_id: ?u32 = null,
first_spinor_symmetrized_product_operator_id: ?u32 = null,
first_spinor_symmetrized_product_dimension: ?u16 = null,
first_spinor_symmetrized_product_left_power: ?u16 = null,
first_spinor_symmetrized_product_right_power: ?u16 = null,
first_spinor_symmetrized_product_output_power: ?u16 = null,
first_spinor_symmetrized_product_chirality: ?u8 = null,
first_spinor_tower_contract_operator_id: ?u32 = null,
first_spinor_tower_contract_dimension: ?u16 = null,
first_spinor_tower_contract_input_power: ?u16 = null,
first_spinor_tower_contract_output_power: ?u16 = null,
first_spinor_tower_contract_chirality: ?u8 = null,
first_spinor_tower_contract_form_rank: ?u8 = null,
first_product_identity_operator_id: ?u32 = null,
first_cartan_product_operator_id: ?u32 = null,
first_tensor_spinor_projection_operator_id: ?u32 = null,
first_tensor_spinor_projection_dimension: ?u16 = null,
first_tensor_spinor_projection_input_profile: ?u128 = null,
first_tensor_spinor_projection_input_count: ?u8 = null,
first_tensor_spinor_projection_form_rank: ?u8 = null,
first_tensor_spinor_projection_form_count: ?u8 = null,
first_tensor_spinor_projection_form_mask: ?u64 = null,
first_tensor_spinor_projection_form_profile: ?u128 = null,
first_tensor_spinor_projection_tower_power: ?u16 = null,
first_tensor_spinor_projection_chirality: ?u8 = null,
first_tensor_form_projection_operator_id: ?u32 = null,
first_tensor_form_projection_dimension: ?u16 = null,
first_tensor_form_projection_input_mask: ?u64 = null,
first_tensor_form_projection_output_profile: ?u128 = null,
first_tensor_form_projection_output_count: ?u8 = null,
first_tensor_form_projection_output_rank: ?u8 = null,
first_tensor_form_projection_output_duality: ?rendering.DualityTag = null,
first_tensor_form_gamma_wedge_operator_id: ?u32 = null,
first_tensor_form_gamma_wedge_dimension: ?u16 = null,
first_tensor_form_gamma_wedge_input_mask: ?u64 = null,
first_tensor_form_gamma_wedge_input_profile: ?u128 = null,
first_tensor_form_gamma_wedge_input_count: ?u8 = null,
first_tensor_form_gamma_wedge_output_profile: ?u128 = null,
first_tensor_form_gamma_wedge_output_count: ?u8 = null,
first_tensor_form_gamma_wedge_inserted_rank: ?u8 = null,
first_tensor_form_gamma_wedge_chirality: ?u8 = null,
first_tensor_form_gamma_map_operator_id: ?u32 = null,
first_tensor_form_gamma_map_dimension: ?u16 = null,
first_tensor_form_gamma_map_source_rank: ?u8 = null,
first_tensor_form_gamma_map_lower_rank: ?u8 = null,
first_tensor_form_gamma_map_upper_rank: ?u8 = null,
first_tensor_form_gamma_map_chirality: ?u8 = null,
first_tensor_form_spinor_pair_operator_id: ?u32 = null,
first_tensor_form_spinor_pair_dimension: ?u16 = null,
first_tensor_form_spinor_pair_input_mask: ?u64 = null,

```

```

first_tensor_form_spinor_pair_input_profile: ?u128 = null,
first_tensor_form_spinor_pair_input_count: ?u8 = null,
first_tensor_form_spinor_pair_output_profile: ?u128 = null,
first_tensor_form_spinor_pair_output_count: ?u8 = null,
first_tensor_form_spinor_pair_left_chirality: ?u8 = null,
first_tensor_form_spinor_pair_right_chirality: ?u8 = null,
first_structure_constant_operator_id: ?u32 = null,
first_structure_constant_family: ?symmetry.LieFamily = null,
first_structure_constant_rank: ?u8 = null,
first_projector_operator_id: ?u32 = null,

pub fn emitTerm(self: *CountingSink, term: rendering.SymbolicTerm) !void {
    self.term_count += 1;
    self.atom_count += @intCast(term.atoms.len);
    for (term.atoms) |atom| {
        switch (atom) {
            .metric_pair => self.metric_pair_count += 1,
            .generalized_delta => self.generalized_delta_count += 1,
            .gamma_matrix => |gamma| {
                self.gamma_matrix_count += 1;
                if (self.first_gamma_operator_id == null) self.first_gamma_operator_id =
                    ↪ gamma.operator_id;
                if (self.first_gamma_dimension == null) self.first_gamma_dimension =
                    ↪ gamma.orthogonal_dimension;
                if (self.first_gamma_rank == null) self.first_gamma_rank = gamma.rank;
                if (self.first_gamma_chirality == null) self.first_gamma_chirality =
                    ↪ gamma.chirality;
            },
            .gamma_form => |gamma| {
                self.gamma_form_count += 1;
                if (self.first_gamma_form_operator_id == null)
                    ↪ self.first_gamma_form_operator_id = gamma.operator_id;
                if (self.first_gamma_form_dimension == null)
                    ↪ self.first_gamma_form_dimension = gamma.orthogonal_dimension;
                if (self.first_gamma_form_rank == null) self.first_gamma_form_rank =
                    ↪ gamma.rank;
                if (self.first_gamma_form_chirality == null)
                    ↪ self.first_gamma_form_chirality = gamma.chirality;
                if (self.first_gamma_form_duality == null) self.first_gamma_form_duality =
                    ↪ gamma.duality;
            },
            .gamma_action => self.gamma_action_count += 1,
            .exterior_gamma_action => |action| {
                self.exterior_gamma_action_count += 1;
                if (self.first_exterior_gamma_action_operator_id == null)
                    ↪ self.first_exterior_gamma_action_operator_id = action.operator_id;
                if (self.first_exterior_gamma_action_dimension == null)
                    ↪ self.first_exterior_gamma_action_dimension =
                        ↪ action.orthogonal_dimension;
                if (self.first_exterior_gamma_action_chirality == null)
                    ↪ self.first_exterior_gamma_action_chirality = action.chirality;
            },
            .clifford_product => |product| {
                self.clifford_product_count += 1;
                if (self.first_clifford_product_left_rank == null)
                    ↪ self.first_clifford_product_left_rank = product.left_rank;
                if (self.first_clifford_product_right_rank == null)
                    ↪ self.first_clifford_product_right_rank = product.right_rank;
                if (self.first_clifford_product_output_rank == null)
                    ↪ self.first_clifford_product_output_rank = product.output_rank;
                if (self.first_clifford_product_contractions == null)
                    ↪ self.first_clifford_product_contractions = product.contractions;
                if (self.first_clifford_product_metric_count == null)
                    ↪ self.first_clifford_product_metric_count = product.metric_count;
            }
        }
    }
}

```

```

        if (self.first_clifford_product_free_vector_count == null)
        ↪ self.first_clifford_product_free_vector_count =
        ↪ product.free_vector_count;
        if (self.first_clifford_product_coefficient == null)
        ↪ self.first_clifford_product_coefficient = product.coefficient;
    },
    .clifford_product_factor => |record| self.countCliffordProductFactor(record),
    .gamma_trace => |trace| {
        self.gamma_trace_count += 1;
        if (self.first_gamma_trace_operator_id == null)
        ↪ self.first_gamma_trace_operator_id = trace.operator_id;
        if (self.first_gamma_trace_dimension == null)
        ↪ self.first_gamma_trace_dimension = trace.orthogonal_dimension;
        if (self.first_gamma_trace_chirality == null)
        ↪ self.first_gamma_trace_chirality = trace.chirality;
        if (self.first_gamma_trace_coefficient == null)
        ↪ self.first_gamma_trace_coefficient =
        ↪ rendering.rationalValue(term.coefficient);
    },
    .spinor_pair => |pair| {
        self.spinor_pair_count += 1;
        if (self.first_spinor_pair_operator_id == null)
        ↪ self.first_spinor_pair_operator_id = pair.operator_id;
        if (self.first_spinor_pair_dimension == null)
        ↪ self.first_spinor_pair_dimension = pair.orthogonal_dimension;
        if (self.first_spinor_pair_left_chirality == null)
        ↪ self.first_spinor_pair_left_chirality = pair.chirality_left;
        if (self.first_spinor_pair_right_chirality == null)
        ↪ self.first_spinor_pair_right_chirality = pair.chirality_right;
    },
    .vector_spinor_identity => self.vector_spinor_identity_count += 1,
    .identity_route => |identity| {
        self.identity_route_count += 1;
        if (self.first_identity_route_operator_id == null)
        ↪ self.first_identity_route_operator_id = identity.operator_id;
    },
    .spinor_index_delta => |delta| {
        self.spinor_index_delta_count += 1;
        if (self.first_spinor_index_delta_operator_id == null)
        ↪ self.first_spinor_index_delta_operator_id = delta.operator_id;
    },
    .spinor_symmetrized_product => |product| {
        self.spinor_symmetrized_product_count += 1;
        if (self.first_spinor_symmetrized_product_operator_id == null)
        ↪ self.first_spinor_symmetrized_product_operator_id =
        ↪ product.operator_id;
        if (self.first_spinor_symmetrized_product_dimension == null)
        ↪ self.first_spinor_symmetrized_product_dimension =
        ↪ product.orthogonal_dimension;
        if (self.first_spinor_symmetrized_product_left_power == null)
        ↪ self.first_spinor_symmetrized_product_left_power = product.left_power;
        if (self.first_spinor_symmetrized_product_right_power == null)
        ↪ self.first_spinor_symmetrized_product_right_power =
        ↪ product.right_power;
        if (self.first_spinor_symmetrized_product_output_power == null)
        ↪ self.first_spinor_symmetrized_product_output_power =
        ↪ product.output_power;
        if (self.first_spinor_symmetrized_product_chirality == null)
        ↪ self.first_spinor_symmetrized_product_chirality = product.chirality;
    },
    .product_identity => |identity| {
        self.product_identity_count += 1;
        if (self.first_product_identity_operator_id == null)
        ↪ self.first_product_identity_operator_id = identity.operator_id;
    }

```



```

},
.epsilon => self.epsilon_count += 1,
.structure_constant => |constant| {
  self.structure_constant_count += 1;
  if (self.first_structure_constant_operator_id == null)
    ↪ self.first_structure_constant_operator_id = constant.operator_id;
  if (self.first_structure_constant_family == null)
    ↪ self.first_structure_constant_family = constant.family;
  if (self.first_structure_constant_rank == null)
    ↪ self.first_structure_constant_rank = constant.rank;
},
.cartan_product => |cartan| {
  self.cartan_product_count += 1;
  if (self.first_cartan_product_operator_id == null)
    ↪ self.first_cartan_product_operator_id = cartan.operator_id;
},
.spinor_tower_contract => |contract| {
  self.spinor_tower_contract_count += 1;
  if (self.first_spinor_tower_contract_operator_id == null)
    ↪ self.first_spinor_tower_contract_operator_id = contract.operator_id;
  if (self.first_spinor_tower_contract_dimension == null)
    ↪ self.first_spinor_tower_contract_dimension =
    ↪ contract.orthogonal_dimension;
  if (self.first_spinor_tower_contract_input_power == null)
    ↪ self.first_spinor_tower_contract_input_power = contract.input_power;
  if (self.first_spinor_tower_contract_output_power == null)
    ↪ self.first_spinor_tower_contract_output_power = contract.output_power;
  if (self.first_spinor_tower_contract_chirality == null)
    ↪ self.first_spinor_tower_contract_chirality = contract.chirality;
  if (self.first_spinor_tower_contract_form_rank == null)
    ↪ self.first_spinor_tower_contract_form_rank = contract.form_rank;
},
.spinor_tower_contract_adjoint => {
  self.spinor_tower_contract_adjoint_count += 1;
},
.tensor_spinor_projection => |projection| {
  self.tensor_spinor_projection_count += 1;
  self.tensor_spinor_profile_audit.recordTensorSpinorAtom(projection);
  if (self.first_tensor_spinor_projection_operator_id == null)
    ↪ self.first_tensor_spinor_projection_operator_id =
    ↪ projection.operator_id;
  if (self.first_tensor_spinor_projection_dimension == null)
    ↪ self.first_tensor_spinor_projection_dimension =
    ↪ projection.orthogonal_dimension;
  if (self.first_tensor_spinor_projection_input_profile == null)
    ↪ self.first_tensor_spinor_projection_input_profile =
    ↪ projection.input_form_profile;
  if (self.first_tensor_spinor_projection_input_count == null)
    ↪ self.first_tensor_spinor_projection_input_count =
    ↪ projection.input_form_count;
  if (self.first_tensor_spinor_projection_form_rank == null)
    ↪ self.first_tensor_spinor_projection_form_rank = projection.form_rank;
  if (self.first_tensor_spinor_projection_form_count == null)
    ↪ self.first_tensor_spinor_projection_form_count =
    ↪ projection.form_count;
  if (self.first_tensor_spinor_projection_form_mask == null)
    ↪ self.first_tensor_spinor_projection_form_mask = projection.form_mask;
  if (self.first_tensor_spinor_projection_form_profile == null)
    ↪ self.first_tensor_spinor_projection_form_profile =
    ↪ projection.form_profile;
  if (self.first_tensor_spinor_projection_tower_power == null)
    ↪ self.first_tensor_spinor_projection_tower_power =
    ↪ projection.tower_power;

```

```

        if (self.first_tensor_spinor_projection_chirality == null)
        ↪ self.first_tensor_spinor_projection_chirality = projection.chirality;
    },
    .tensor_form_projection => |projection| {
        self.tensor_form_projection_count += 1;
        if (self.first_tensor_form_projection_operator_id == null)
        ↪ self.first_tensor_form_projection_operator_id =
        ↪ projection.operator_id;
        if (self.first_tensor_form_projection_dimension == null)
        ↪ self.first_tensor_form_projection_dimension =
        ↪ projection.orthogonal_dimension;
        if (self.first_tensor_form_projection_input_mask == null)
        ↪ self.first_tensor_form_projection_input_mask =
        ↪ projection.input_form_mask;
        if (self.first_tensor_form_projection_output_profile == null)
        ↪ self.first_tensor_form_projection_output_profile =
        ↪ projection.output_form_profile;
        if (self.first_tensor_form_projection_output_count == null)
        ↪ self.first_tensor_form_projection_output_count =
        ↪ projection.output_form_count;
        if (self.first_tensor_form_projection_output_rank == null)
        ↪ self.first_tensor_form_projection_output_rank =
        ↪ projection.output_form_rank;
        if (self.first_tensor_form_projection_output_duality == null)
        ↪ self.first_tensor_form_projection_output_duality =
        ↪ projection.output_duality;
    },
    .tensor_form_gamma_wedge => |projection| {
        self.tensor_form_gamma_wedge_count += 1;
        if (self.first_tensor_form_gamma_wedge_operator_id == null)
        ↪ self.first_tensor_form_gamma_wedge_operator_id =
        ↪ projection.operator_id;
        if (self.first_tensor_form_gamma_wedge_dimension == null)
        ↪ self.first_tensor_form_gamma_wedge_dimension =
        ↪ projection.orthogonal_dimension;
        if (self.first_tensor_form_gamma_wedge_input_mask == null)
        ↪ self.first_tensor_form_gamma_wedge_input_mask =
        ↪ projection.input_form_mask;
        if (self.first_tensor_form_gamma_wedge_input_profile == null)
        ↪ self.first_tensor_form_gamma_wedge_input_profile =
        ↪ projection.input_form_profile;
        if (self.first_tensor_form_gamma_wedge_input_count == null)
        ↪ self.first_tensor_form_gamma_wedge_input_count =
        ↪ projection.input_form_count;
        if (self.first_tensor_form_gamma_wedge_output_profile == null)
        ↪ self.first_tensor_form_gamma_wedge_output_profile =
        ↪ projection.output_form_profile;
        if (self.first_tensor_form_gamma_wedge_output_count == null)
        ↪ self.first_tensor_form_gamma_wedge_output_count =
        ↪ projection.output_form_count;
        if (self.first_tensor_form_gamma_wedge_inserted_rank == null)
        ↪ self.first_tensor_form_gamma_wedge_inserted_rank =
        ↪ projection.inserted_rank;
        if (self.first_tensor_form_gamma_wedge_chirality == null)
        ↪ self.first_tensor_form_gamma_wedge_chirality = projection.chirality;
    },
    .tensor_form_gamma_map => |projection| {
        self.tensor_form_gamma_map_count += 1;
        if (self.first_tensor_form_gamma_map_operator_id == null)
        ↪ self.first_tensor_form_gamma_map_operator_id = projection.operator_id;
        if (self.first_tensor_form_gamma_map_dimension == null)
        ↪ self.first_tensor_form_gamma_map_dimension =
        ↪ projection.orthogonal_dimension;
    }

```

```

        if (self.first_tensor_form_gamma_map_source_rank == null)
        ↪ self.first_tensor_form_gamma_map_source_rank = projection.source_rank;
        if (self.first_tensor_form_gamma_map_lower_rank == null)
        ↪ self.first_tensor_form_gamma_map_lower_rank = projection.lower_rank;
        if (self.first_tensor_form_gamma_map_upper_rank == null)
        ↪ self.first_tensor_form_gamma_map_upper_rank = projection.upper_rank;
        if (self.first_tensor_form_gamma_map_chirality == null)
        ↪ self.first_tensor_form_gamma_map_chirality = projection.chirality;
    },
    .tensor_form_gamma_map_adjoint => {
        self.tensor_form_gamma_map_adjoint_count += 1;
    },
    .tensor_form_spinor_pair => |projection| {
        self.tensor_form_spinor_pair_count += 1;
        if (self.first_tensor_form_spinor_pair_operator_id == null)
        ↪ self.first_tensor_form_spinor_pair_operator_id =
        ↪ projection.operator_id;
        if (self.first_tensor_form_spinor_pair_dimension == null)
        ↪ self.first_tensor_form_spinor_pair_dimension =
        ↪ projection.orthogonal_dimension;
        if (self.first_tensor_form_spinor_pair_input_mask == null)
        ↪ self.first_tensor_form_spinor_pair_input_mask =
        ↪ projection.input_form_mask;
        if (self.first_tensor_form_spinor_pair_input_profile == null)
        ↪ self.first_tensor_form_spinor_pair_input_profile =
        ↪ projection.input_form_profile;
        if (self.first_tensor_form_spinor_pair_input_count == null)
        ↪ self.first_tensor_form_spinor_pair_input_count =
        ↪ projection.input_form_count;
        if (self.first_tensor_form_spinor_pair_output_profile == null)
        ↪ self.first_tensor_form_spinor_pair_output_profile =
        ↪ projection.output_form_profile;
        if (self.first_tensor_form_spinor_pair_output_count == null)
        ↪ self.first_tensor_form_spinor_pair_output_count =
        ↪ projection.output_form_count;
        if (self.first_tensor_form_spinor_pair_left_chirality == null)
        ↪ self.first_tensor_form_spinor_pair_left_chirality =
        ↪ projection.left_chirality;
        if (self.first_tensor_form_spinor_pair_right_chirality == null)
        ↪ self.first_tensor_form_spinor_pair_right_chirality =
        ↪ projection.right_chirality;
    },
    .named_operator => |operator| switch (operator.kind) {
        .gamma => self.gamma_operator_count += 1,
        .projector => {
            self.projector_operator_count += 1;
            if (self.first_projector_operator_id == null)
            ↪ self.first_projector_operator_id = operator.operator_id;
        },
        else => {},
    },
    else => {},
}
}
}

pub fn emitCliffordProductFactor(self: *CountingSink, record:
    ↪ rendering.CliffordProductFactorRecord) !void {
    self.countCliffordProductFactor(record);
}

fn countCliffordProductFactor(self: *CountingSink, record:
    ↪ rendering.CliffordProductFactorRecord) void {
    self.clifford_product_factor_count += 1;
}

```

```

        if (self.first_clifford_product_factor_atom_index == null)
            ↪ self.first_clifford_product_factor_atom_index = record.atom_index;
        if (self.first_clifford_product_factor_index == null)
            ↪ self.first_clifford_product_factor_index = record.factor_index;
        if (self.first_clifford_product_factor_left_operator_id == null)
            ↪ self.first_clifford_product_factor_left_operator_id = record.left_operator_id;
        if (self.first_clifford_product_factor_right_operator_id == null)
            ↪ self.first_clifford_product_factor_right_operator_id = record.right_operator_id;
        if (self.first_clifford_product_factor_dimension == null)
            ↪ self.first_clifford_product_factor_dimension = record.orthogonal_dimension;
        if (self.first_clifford_product_factor_output_rank == null)
            ↪ self.first_clifford_product_factor_output_rank = record.output_rank;
        if (self.first_clifford_product_factor_coefficient == null)
            ↪ self.first_clifford_product_factor_coefficient = record.coefficient;
        switch (record.factor) {
            .metric => self.clifford_product_metric_factor_count += 1,
            .output_vector => self.clifford_product_output_vector_factor_count += 1,
        }
    }
};

const EvaluationCountingSink = struct {
    term_count: u32 = 0,
    atom_count: u32 = 0,
    metric_pair_count: u32 = 0,
    gamma_matrix_count: u32 = 0,
    clifford_product_factor_count: u32 = 0,
    compact_formula_count: u32 = 0,
    first_coefficient: ?rendering.RationalValue = null,

    pub fn emitEvaluationTerm(self: *EvaluationCountingSink, term: rendering.EvaluationTerm)
        ↪ !void {
        self.term_count += 1;
        self.atom_count += @intCast(term.atoms.len);
        if (self.first_coefficient == null) self.first_coefficient =
            ↪ rendering.rationalValue(term.coefficient);
        for (term.atoms) |atom| {
            switch (atom) {
                .metric_pair => self.metric_pair_count += 1,
                .gamma_matrix => self.gamma_matrix_count += 1,
                .clifford_product_factor => self.clifford_product_factor_count += 1,
                .named_operator,
                .clifford_product,
                .product_identity,
                .cartan_product,
                .tensor_spinor_projection,
                .tensor_form_projection,
                .gamma_trace,
                .vector_spinor_identity,
                => self.compact_formula_count += 1,
                else => {},
            }
        }
    }
};

fn isVectorDynkin(label: []const i16) bool {
    if (label.len == 0 or label[0] != 1) return false;
    for (label[1..]) |entry| {
        if (entry != 0) return false;
    }
    return true;
}

```

```

fn isSpinorDynkin(simple: symmetry.SimpleLieAlgebra, label: []const i16) bool {
    if (label.len != simple.rank) return false;
    switch (simple.family) {
        .b => {
            if (label.len == 0 or label[label.len - 1] != 1) return false;
            for (label[0 .. label.len - 1]) |entry| {
                if (entry != 0) return false;
            }
            return true;
        },
        .d => {
            if (label.len < 2) return false;
            const left = label.len - 2;
            const right = label.len - 1;
            if (!((label[left] == 1 and label[right] == 0) or (label[left] == 0 and
↵ label[right] == 1))) return false;
            for (label[0..left]) |entry| {
                if (entry != 0) return false;
            }
            return true;
        },
        else => return false,
    }
}

fn isVectorSpinorDynkin(simple: symmetry.SimpleLieAlgebra, label: []const i16) bool {
    if (label.len != simple.rank or label.len < 2) return false;
    if (label[0] != 1) return false;
    switch (simple.family) {
        .b => {
            if (label[label.len - 1] != 1) return false;
            for (label[1 .. label.len - 1]) |entry| {
                if (entry != 0) return false;
            }
            return true;
        },
        .d => {
            const left = label.len - 2;
            const right = label.len - 1;
            if (!((label[left] == 1 and label[right] == 0) or (label[left] == 0 and
↵ label[right] == 1))) return false;
            for (label[1..left]) |entry| {
                if (entry != 0) return false;
            }
            return true;
        },
        else => return false,
    }
}

fn isA2FundamentalDynkin(label: []const i16) bool {
    return label.len == 2 and label[0] == 1 and label[1] == 0;
}

fn isA2AntiFundamentalDynkin(label: []const i16) bool {
    return label.len == 2 and label[0] == 0 and label[1] == 1;
}

fn isE6FundamentalDynkin(label: []const i16) bool {
    return label.len == 6 and label[0] == 1 and label[1] == 0 and label[2] == 0 and label[3]
↵ == 0 and label[4] == 0 and label[5] == 0;
}

fn isE6DualFundamentalDynkin(label: []const i16) bool {

```

```

    return label.len == 6 and label[0] == 0 and label[1] == 0 and label[2] == 0 and label[3]
    ↪ == 0 and label[4] == 1 and label[5] == 0;
}

fn simpleAdjointDimension(simple: symmetry.SimpleLieAlgebra) u128 {
    const rank: u128 = simple.rank;
    return switch (simple.family) {
        .a => rank * (rank + 2),
        .b => rank * (2 * rank + 1),
        .d => rank * (2 * rank - 1),
        .e6 => 78,
        .e7 => 133,
        .e8 => 248,
        else => 0,
    };
}

fn orthogonalVectorSpinorFormulaAudit(orthogonal_dimension: u16)
↪ projector.ProjectorFormulaAudit {
    const trace_projector_denominator = orthogonal_dimension;
    const gamma_trace_square_factor = orthogonal_dimension;
    const normalized = trace_projector_denominator != 0;
    const gamma_trace_law = normalized and gamma_trace_square_factor ==
    ↪ trace_projector_denominator;
    return .{
        .normalized = normalized,
        .idempotent = gamma_trace_law,
        .channel_separated = gamma_trace_law,
        .gamma_traceless = gamma_trace_law,
    };
}

fn orthogonalSpinorBilinearFormulaAudit(simple: symmetry.SimpleLieAlgebra, left: []const i16,
↪ right: []const i16, rank: u8, duality: rendering.DualityTag)
↪ projector.ProjectorFormulaAudit {
    if (!isSpinorDynkin(simple, left) or !isSpinorDynkin(simple, right)) return .{};
    const orthogonal_dimension = rendering.orthogonalDimension(simple);
    const normalized = orthogonalGammaGradeNormNonZero(simple, orthogonal_dimension, rank,
    ↪ duality) and orthogonalSpinorBilinearChiralityValid(simple, left, right, rank);
    return .{
        .normalized = normalized,
        .idempotent = normalized,
        .channel_separated = normalized,
    };
}

fn orthogonalGammaGradeNormNonZero(simple: symmetry.SimpleLieAlgebra, orthogonal_dimension:
↪ u16, rank: u8, duality: rendering.DualityTag) bool {
    if (orthogonal_dimension == 0 or rank > orthogonal_dimension) return false;
    switch (duality) {
        .none => return true,
        .self_dual, .anti_self_dual => return simple.family == .d and @as(u16, rank) * 2 ==
    ↪ orthogonal_dimension,
    }
}

fn orthogonalSpinorBilinearChiralityValid(simple: symmetry.SimpleLieAlgebra, left: []const
↪ i16, right: []const i16, rank: u8) bool {
    const left_chirality = rendering.gammaChiralityTag(simple, left);
    const right_chirality = rendering.gammaChiralityTag(simple, right);
    return switch (simple.family) {
        .b => left_chirality == .none and right_chirality == .none,
        .d => if (rank % 2 == 0) left_chirality != right_chirality else left_chirality ==
    ↪ right_chirality and left_chirality != .none,
    }
}

```

```

        else => false,
    };
}

fn oppositeGammaChirality(chirality: rendering.GammaChirality) rendering.GammaChirality {
    return switch (chirality) {
        .left => .right,
        .right => .left,
        .none => .none,
    };
}

fn isGramFormulaKind(kind: projector.ProjectorFormulaKind) bool {
    const value = @intFromEnum(kind);
    return kind == .orthogonal_structural_projection or
        value >=
            ↪ @intFromEnum(projector.ProjectorFormulaKind.orthogonal_spinor_tower_form_channel)
            ↪ and
        value <=
            ↪ @intFromEnum(projector.ProjectorFormulaKind.orthogonal_tensor_spinor_middle_form_channel);
}

fn exactPrimitiveFormulaAudit() projector.ProjectorFormulaAudit {
    return .{
        .normalized = true,
        .idempotent = true,
        .channel_separated = true,
    };
}

fn exactGramProjectorFormulaAudit() projector.ProjectorFormulaAudit {
    return .{
        .normalized = true,
        .idempotent = true,
        .channel_separated = true,
    };
}

fn orthogonalIrrepDescriptor(simple: symmetry.SimpleLieAlgebra, label: []const i16)
    ↪ OrthogonalIrrepDescriptor {
    if (label.len != simple.rank) return unsupportedOrthogonalDescriptor();
    switch (simple.family) {
        .b, .d => {},
        else => return unsupportedOrthogonalDescriptor(),
    }

    const dimension = rendering.orthogonalDimension(simple);
    if (isZeroDynkin(label)) return .{ .kind = .scalar, .dimension = dimension };
    if (orthogonalFormInfo(simple, label)) |form| return descriptorFromSingleForm(dimension,
        ↪ form);
    if (orthogonalTensorSpinorInfo(simple, label)) |info| return
        ↪ descriptorFromTensorSpinor(dimension, info);
    if (spinorTowerInfo(simple, label)) |tower| return descriptorFromSpinorTower(dimension,
        ↪ tower);
    if (ordinaryTensorFormInfo(simple, label)) |forms| return descriptorFromFormSet(dimension,
        ↪ forms);
    return unsupportedOrthogonalDescriptor();
}

fn unsupportedOrthogonalDescriptor() OrthogonalIrrepDescriptor {
    return .{ .kind = .unsupported, .dimension = 0 };
}

```

```

fn descriptorFromSingleForm(dimension: u16, form: OrthogonalFormInfo)
↳ OrthogonalIrrepDescriptor {
    if (form.rank == 0 or form.rank > 32) return unsupportedOrthogonalDescriptor();
    const rank_index: u7 = @intCast(form.rank - 1);
    return .{
        .kind = .form_profile,
        .dimension = dimension,
        .form_profile = @as(u128, 1) << @intCast(rank_index * 4),
        .form_mask = @as(u64, 1) << @intCast(rank_index),
        .form_count = 1,
        .form_total_power = 1,
        .form_rank = form.rank,
        .form_duality = form.duality,
    };
}

fn descriptorFromFormSet(dimension: u16, forms: OrthogonalFormSetInfo)
↳ OrthogonalIrrepDescriptor {
    return .{
        .kind = .form_profile,
        .dimension = dimension,
        .form_profile = forms.profile,
        .form_mask = forms.mask,
        .form_count = forms.count,
        .form_total_power = forms.total_power,
        .form_rank = forms.first_rank,
        .form_duality = forms.duality,
    };
}

fn descriptorFromSpinorTower(dimension: u16, tower: SpinorTowerInfo) OrthogonalIrrepDescriptor
↳ {
    return .{
        .kind = .spinor_tower,
        .dimension = dimension,
        .has_spinor = true,
        .chirality = @intFromEnum(tower.chirality),
        .tower_power = tower.power,
    };
}

fn descriptorFromTensorSpinor(dimension: u16, info: OrthogonalTensorSpinorInfo)
↳ OrthogonalIrrepDescriptor {
    return .{
        .kind = .tensor_spinor,
        .dimension = dimension,
        .form_profile = info.forms.profile,
        .form_mask = info.forms.mask,
        .form_count = info.forms.count,
        .form_total_power = info.forms.total_power,
        .form_rank = info.forms.first_rank,
        .form_duality = info.forms.duality,
        .has_spinor = true,
        .chirality = @intFromEnum(info.chirality),
        .tower_power = info.tower_power,
    };
}

fn orthogonalFormInfo(simple: symmetry.SimpleLieAlgebra, label: []const i16)
↳ ?OrthogonalFormInfo {
    if (label.len != simple.rank) return null;
    switch (simple.family) {
        .b => {
            if (label.len < 2) return null;

```



```

        for (label, 0..) |entry, index| {
            if (entry == 0) continue;
            if (entry != 1 or index + 1 >= label.len) return null;
            return .{ .rank = @intCast(index + 1) };
        }
        return null;
    },
    .d => {
        if (label.len < 3) return null;
        for (label[0 .. label.len - 2], 0..) |entry, index| {
            if (entry == 0) continue;
            if (entry != 1) return null;
            for (label[index + 1 ..]) |tail| {
                if (tail != 0) return null;
            }
            return .{ .rank = @intCast(index + 1) };
        }

        const left = label.len - 2;
        const right = label.len - 1;
        if (label[left] == 2 and label[right] == 0) return .{
            .rank = @intCast(label.len),
            .duality = .anti_self_dual,
        };
        if (label[left] == 0 and label[right] == 2) return .{
            .rank = @intCast(label.len),
            .duality = .self_dual,
        };
        if (label[left] == 1 and label[right] == 1) return .{
            .rank = @intCast(label.len - 1),
        };
        return null;
    },
    else => return null,
}
}

fn orthogonalTensorSpinorInfo(simple: symmetry.SimpleLieAlgebra, label: []const i16)
→ ?OrthogonalTensorSpinorInfo {
    if (label.len != simple.rank) return null;
    const tower = outputSpinorTowerInfo(simple, label) orelse return null;
    const forms = outputFormSetInfo(simple, label) orelse return null;
    return .{
        .forms = forms,
        .tower_power = tower.power,
        .chirality = tower.chirality,
    };
}

fn outputFormSetInfo(simple: symmetry.SimpleLieAlgebra, label: []const i16)
→ ?OrthogonalFormSetInfo {
    if (label.len != simple.rank) return null;
    const form_end = switch (simple.family) {
        .b => if (label.len == 0) return null else label.len - 1,
        .d => if (label.len < 2) return null else label.len - 2,
        else => return null,
    };
    var found: OrthogonalFormSetInfo = .{
        .first_rank = 0,
        .count = 0,
        .total_power = 0,
        .mask = 0,
        .profile = 0,
    };
}

```

```

    for (label[0..form_end], 0..) |entry, index| {
      if (entry == 0) continue;
      if (entry < 0 or entry > 15 or index >= 64) return null;
      const rank: u8 = @intCast(index + 1);
      if (found.count == 0) found.first_rank = rank;
      found.count += 1;
      found.total_power += @intCast(entry);
      found.mask |= @as(u64, 1) << @intCast(index);
      found.profile |= @as(u128, @intCast(entry)) << @intCast(index * 4);
    }
    if (simple.family == .d) {
      const left = label.len - 2;
      const right = label.len - 1;
      if (label[left] < 0 or label[right] < 0) return null;
      const common = @min(label[left], label[right]);
      if (common > 0) {
        const rank_index = label.len - 2;
        if (common > 15 or rank_index >= 64) return null;
        if (found.count == 0) found.first_rank = @intCast(rank_index + 1);
        found.count += 1;
        found.total_power += @intCast(common);
        found.mask |= @as(u64, 1) << @intCast(rank_index);
        found.profile |= @as(u128, @intCast(common)) << @intCast(rank_index * 4);
      }
    }
    return if (found.count == 0) null else found;
  }
}

fn ordinaryTensorFormInfo(simple: symmetry.SimpleLieAlgebra, label: []const i16)
→ ?OrthogonalFormSetInfo {
  if (label.len != simple.rank) return null;
  switch (simple.family) {
    .b => {
      if (label.len == 0 or label[label.len - 1] != 0) return null;
    },
    .d => {
      if (label.len < 2) return null;
      const left = label.len - 2;
      const right = label.len - 1;
      if (label[left] != label[right]) return null;
    },
    else => return null,
  }
  return outputFormSetInfo(simple, label);
}

fn outputSpinorTowerInfo(simple: symmetry.SimpleLieAlgebra, label: []const i16)
→ ?SpinorTowerInfo {
  if (label.len != simple.rank) return null;
  return switch (simple.family) {
    .b => {
      if (label.len == 0 or label[label.len - 1] <= 0) return null;
      return .{
        .power = @intCast(label[label.len - 1]),
        .chirality = .none,
      };
    },
    .d => {
      if (label.len < 2) return null;
      const left = label.len - 2;
      const right = label.len - 1;
      if (label[left] < 0 or label[right] < 0) return null;
      const common = @min(label[left], label[right]);
      const left_power = label[left] - common;

```

```

        const right_power = label[right] - common;
        if (left_power > 0 and right_power == 0) return .{
            .power = @intCast(left_power),
            .chirality = .left,
        };
        if (left_power == 0 and right_power > 0) return .{
            .power = @intCast(right_power),
            .chirality = .right,
        };
        return null;
    },
    else => null,
};
}

fn isSpinorTowerDynkin(simple: symmetry.SimpleLieAlgebra, label: []const i16) bool {
    const tower = spinorTowerInfo(simple, label) orelse return false;
    return tower.power > 0;
}

fn spinorTowerInfo(simple: symmetry.SimpleLieAlgebra, label: []const i16) ?SpinorTowerInfo {
    if (label.len != simple.rank) return null;
    const form_end = switch (simple.family) {
        .b => if (label.len == 0) return null else label.len - 1,
        .d => if (label.len < 2) return null else label.len - 2,
        else => return null,
    };
    for (label[0..form_end]) |entry| {
        if (entry != 0) return null;
    }
    if (simple.family == .d and @min(label[label.len - 2], label[label.len - 1]) != 0) return
    ↪ null;
    return outputSpinorTowerInfo(simple, label);
}

fn formMaskShiftedUpOnce(left: u64, output: u64) bool {
    if (left == 0 or output == 0) return false;
    const removed = left & ~output;
    const added = output & ~left;
    if (@popCount(removed) != 1 or @popCount(added) != 1) return false;
    return added == removed << 1;
}

fn formProfileShiftedUpOnce(left: u128, output: u128) bool {
    if (left == 0 or output == 0) return false;
    for (0..31) |rank_index| {
        const count = formProfileCount(left, rank_index);
        if (count == 0) continue;
        var candidate = left;
        candidate -= @as(u128, 1) << @intCast(rank_index * 4);
        candidate += @as(u128, 1) << @intCast((rank_index + 1) * 4);
        if (candidate == output) return true;
    }
    return false;
}

fn formProfileShiftedByOnce(left: u128, output: u128, delta: usize) bool {
    if (left == 0 or output == 0 or delta == 0) return false;
    for (0..32) |rank_index| {
        const output_index = rank_index + delta;
        if (output_index >= 32) break;
        const count = formProfileCount(left, rank_index);
        if (count == 0) continue;
        var candidate = left;

```

```

        candidate -= @as(u128, 1) << @intCast(rank_index * 4);
        candidate += @as(u128, 1) << @intCast(output_index * 4);
        if (candidate == output) return true;
    }
    return false;
}

fn formProfileShiftedAllUpOnce(left: u128, output: u128) bool {
    if (left == 0 or output == 0 or formProfileCount(left, 31) != 0) return false;
    var candidate: u128 = 0;
    for (0..31) |rank_index| {
        const count = formProfileCount(left, rank_index);
        if (count == 0) continue;
        candidate += @as(u128, count) << @intCast((rank_index + 1) * 4);
    }
    return candidate == output;
}

fn formProfileShiftedAllDownOnce(left: u128, output: u128) bool {
    if (left == 0 or output == 0 or formProfileCount(left, 0) != 0) return false;
    var candidate: u128 = 0;
    for (1..32) |rank_index| {
        const count = formProfileCount(left, rank_index);
        if (count == 0) continue;
        candidate += @as(u128, count) << @intCast((rank_index - 1) * 4);
    }
    return candidate == output;
}

fn formProfileShiftedAllDownWithOneExtraOnce(left: u128, output: u128) bool {
    if (left == 0 or output == 0 or formProfileCount(left, 0) != 0) return false;
    for (2..32) |extra_index| {
        const count = formProfileCount(left, extra_index);
        if (count == 0) continue;
        var candidate: u128 = 0;
        for (1..32) |rank_index| {
            const current = formProfileCount(left, rank_index);
            if (current == 0) continue;
            if (rank_index == extra_index) {
                candidate += @as(u128, 1) << @intCast((rank_index - 2) * 4);
                if (current > 1) candidate += @as(u128, current - 1) << @intCast((rank_index -
                    ↪ 1) * 4);
            } else {
                candidate += @as(u128, current) << @intCast((rank_index - 1) * 4);
            }
        }
        if (candidate == output) return true;
    }
    return false;
}

fn formProfileIncrementedExistingOnce(left: u128, output: u128) bool {
    if (left == 0 or output == 0) return false;
    for (0..32) |rank_index| {
        const count = formProfileCount(left, rank_index);
        if (count == 0 or count >= 15) continue;
        const candidate = left + (@as(u128, 1) << @intCast(rank_index * 4));
        if (candidate == output) return true;
    }
    return false;
}

fn formProfileIncrementedRank(left: u128, output: u128) ?u8 {
    if (left == 0 or output == 0) return null;

```

```

    for (0..32) |rank_index| {
        const count = formProfileCount(left, rank_index);
        if (count == 0 or count >= 15) continue;
        const candidate = left + (@as(u128, 1) << @intCast(rank_index * 4));
        if (candidate == output) return @intCast(rank_index + 1);
    }
    return null;
}

fn formProfileAddedOnce(left: u128, output: u128) bool {
    if (output == 0) return false;
    for (0..32) |rank_index| {
        const count = formProfileCount(left, rank_index);
        if (count >= 15) continue;
        const candidate = left + (@as(u128, 1) << @intCast(rank_index * 4));
        if (candidate == output) return true;
    }
    return false;
}

fn formProfileAddedRank(left: u128, output: u128) ?u8 {
    if (output == 0) return null;
    for (0..32) |rank_index| {
        const count = formProfileCount(left, rank_index);
        if (count >= 15) continue;
        const candidate = left + (@as(u128, 1) << @intCast(rank_index * 4));
        if (candidate == output) return @intCast(rank_index + 1);
    }
    return null;
}

fn formProfileShiftedUpThenAddedOnce(left: u128, output: u128) bool {
    if (left == 0 or output == 0) return false;
    for (0..31) |shift_index| {
        const count = formProfileCount(left, shift_index);
        if (count == 0) continue;
        var shifted = left;
        shifted -= @as(u128, 1) << @intCast(shift_index * 4);
        shifted += @as(u128, 1) << @intCast((shift_index + 1) * 4);
        if (formProfileAddedOnce(shifted, output)) return true;
    }
    return false;
}

fn formProfileMovedOnce(left: u128, output: u128, from_index: usize, to_index: usize) bool {
    if (left == 0 or output == 0 or from_index >= 32 or to_index >= 32 or from_index ==
        ⇨ to_index) return false;
    const count = formProfileCount(left, from_index);
    if (count == 0) return false;
    var candidate = left;
    candidate -= @as(u128, 1) << @intCast(from_index * 4);
    candidate += @as(u128, 1) << @intCast(to_index * 4);
    return candidate == output;
}

fn formProfileMovedDownOnce(left: u128, output: u128, delta: usize) bool {
    if (left == 0 or output == 0 or delta == 0) return false;
    for (delta..32) |rank_index| {
        const count = formProfileCount(left, rank_index);
        if (count == 0) continue;
        var candidate = left;
        candidate -= @as(u128, 1) << @intCast(rank_index * 4);
        candidate += @as(u128, 1) << @intCast((rank_index - delta) * 4);
        if (candidate == output) return true;
    }
}

```

```

    }
    return false;
}

fn formProfileMovedDownAnyOnce(left: u128, output: u128) bool {
    if (left == 0 or output == 0) return false;
    for (1..32) |delta| {
        if (formProfileMovedDownOnce(left, output, delta)) return true;
    }
    return false;
}

fn formProfileMergedPairToMiddleOnce(left: u128, output: u128) bool {
    if (left == 0 or output == 0) return false;
    for (0..30) |rank_index| {
        if (formProfileCount(left, rank_index) == 0 or formProfileCount(left, rank_index + 2)
            == 0) continue;
        var candidate = left;
        candidate -= @as(u128, 1) << @intCast(rank_index * 4);
        candidate -= @as(u128, 1) << @intCast((rank_index + 2) * 4);
        candidate += @as(u128, 1) << @intCast((rank_index + 1) * 4);
        if (candidate == output) return true;
    }
    return false;
}

fn formProfileSplitDownToRankWrapOnce(left: u128, output: u128, wrap_index: usize) bool {
    if (left == 0 or output == 0 or wrap_index >= 32) return false;
    for (1..32) |rank_index| {
        if (rank_index == wrap_index + 1) break;
        const count = formProfileCount(left, rank_index);
        if (count == 0) continue;
        var candidate = left;
        candidate -= @as(u128, 1) << @intCast(rank_index * 4);
        candidate += @as(u128, 1) << @intCast((rank_index - 1) * 4);
        candidate += @as(u128, 1) << @intCast(wrap_index * 4);
        if (candidate == output) return true;
    }
    return false;
}

fn formProfileSplitToNeighborsOnce(left: u128, output: u128) bool {
    if (left == 0 or output == 0) return false;
    for (1..31) |rank_index| {
        const count = formProfileCount(left, rank_index);
        if (count == 0) continue;
        var candidate = left;
        candidate -= @as(u128, 1) << @intCast(rank_index * 4);
        candidate += @as(u128, 1) << @intCast((rank_index - 1) * 4);
        candidate += @as(u128, 1) << @intCast((rank_index + 1) * 4);
        if (candidate == output) return true;
    }
    return false;
}

fn formProfileRemovedOnce(left: u128, output: u128) bool {
    if (left == 0) return false;
    for (0..32) |rank_index| {
        const count = formProfileCount(left, rank_index);
        if (count == 0) continue;
        const candidate = left - (@as(u128, 1) << @intCast(rank_index * 4));
        if (candidate == output) return true;
    }
    return false;
}

```

```

}

fn formProfileRemovedAtRank(left: u128, output: u128, rank_index: usize) bool {
    if (left == 0 or rank_index >= 32) return false;
    const count = formProfileCount(left, rank_index);
    if (count == 0) return false;
    return left - (@as(u128, 1) << @intCast(rank_index * 4)) == output;
}

fn formProfileRemovedThenShiftedUpOnce(left: u128, output: u128) bool {
    if (left == 0) return false;
    for (0..32) |rank_index| {
        const count = formProfileCount(left, rank_index);
        if (count == 0) continue;
        const removed = left - (@as(u128, 1) << @intCast(rank_index * 4));
        if (formProfileShiftedUpOnce(removed, output)) return true;
    }
    return false;
}

fn formProfileRemovedThenMovedDownOnce(left: u128, output: u128) bool {
    if (left == 0) return false;
    for (0..32) |rank_index| {
        const count = formProfileCount(left, rank_index);
        if (count == 0) continue;
        const removed = left - (@as(u128, 1) << @intCast(rank_index * 4));
        if (formProfileMovedDownOnce(removed, output, 1)) return true;
    }
    return false;
}

fn formProfileCount(profile: u128, rank_index: usize) u8 {
    if (rank_index >= 32) return 0;
    return @intCast((profile >> @intCast(rank_index * 4)) & 0xf);
}

fn formProfileFromMask(mask: u64) u128 {
    var profile: u128 = 0;
    var rank_index: usize = 0;
    while (rank_index < 32) : (rank_index += 1) {
        if (((mask >> @intCast(rank_index)) & 1) == 0) continue;
        profile += @as(u128, 1) << @intCast(rank_index * 4);
    }
    return profile;
}

fn formProfilesIntersect(left: u128, right: u128) bool {
    var rank_index: usize = 0;
    while (rank_index < 32) : (rank_index += 1) {
        if (formProfileCount(left, rank_index) != 0 and formProfileCount(right, rank_index) !=
            ↪ 0) return true;
    }
    return false;
}

fn appendCliffordProductAtoms(allocator: std.mem.Allocator, atoms:
    ↪ *std.ArrayList(rendering.SymbolicAtom), left_atom: rendering.SymbolicAtom, right_atom:
    ↪ rendering.SymbolicAtom) !void {
    const left = gammaProductLeftBlock(left_atom) orelse return;
    const right = gammaProductRightAction(right_atom) orelse return;
    if (left.block != right.block) return;
    if (left.orthogonal_dimension != right.orthogonal_dimension) return
        ↪ error.InvalidGammaProductTerm;
}

```

```

const expansion = try rendering.gammaProductExpansion(left.orthogonal_dimension,
↳ left.rank, right.rank, left.duality, right.duality);
for (expansion.slice()) |term| {
const plan = try rendering.gammaProductContractionPlan(left.rank, right.rank, term);
try atoms.append(allocator, .{ .clifford_product = .{
    .left_operator_id = left.operator_id,
    .right_operator_id = right.operator_id,
    .left_block = left.block,
    .right_block = right.block,
    .orthogonal_dimension = left.orthogonal_dimension,
    .left_rank = left.rank,
    .right_rank = right.rank,
    .output_rank = term.rank,
    .contractions = term.contractions,
    .metric_count = plan.metric_len,
    .free_vector_count = plan.free_len,
    .coefficient = term.coefficient,
    .left_duality = left.duality,
    .right_duality = right.duality,
    .output_duality = term.duality,
} });
}
}

const GammaProductInput = struct {
    operator_id: u32,
    block: rendering.IndexBlockId,
    orthogonal_dimension: u16,
    rank: u8,
    duality: rendering.DualityTag = .none,
};

fn gammaProductLeftBlock(atom: rendering.SymbolicAtom) ?GammaProductInput {
return switch (atom) {
    .gamma_matrix => |gamma| .{
        .operator_id = gamma.operator_id,
        .block = gamma.vector,
        .orthogonal_dimension = gamma.orthogonal_dimension,
        .rank = gamma.rank,
        .duality = gamma.duality,
    },
    .gamma_form => |gamma| .{
        .operator_id = gamma.operator_id,
        .block = gamma.form,
        .orthogonal_dimension = gamma.orthogonal_dimension,
        .rank = gamma.rank,
        .duality = gamma.duality,
    },
    else => null,
};
}

fn gammaProductRightAction(atom: rendering.SymbolicAtom) ?GammaProductInput {
return switch (atom) {
    .gamma_action => |gamma| .{
        .operator_id = gamma.operator_id,
        .block = gamma.form,
        .orthogonal_dimension = gamma.orthogonal_dimension,
        .rank = gamma.rank,
        .duality = gamma.duality,
    },
    else => null,
};
}

```



```

fn isZeroDynkin(label: []const i16) bool {
    for (label) |entry| {
        if (entry != 0) return false;
    }
    return true;
}

fn externalIndexOffset(request: coupling.InvariantBasisRequest, leg_index: usize) u32 {
    var offset: u32 = 0;
    for (request.external_legs[0..leg_index]) |leg| {
        offset += @intCast(leg.indices.len);
    }
    return offset;
}

fn tensorSpinorSpinorIndex(index: rendering.IndexRef) rendering.IndexRef {
    return 0x80000000 | index;
}

const TensorFormProjectionGammaFallback = struct {
    operator_id: u32,
    left: rendering.IndexRef,
    right: rendering.IndexRef,
    output: rendering.IndexRef,
    orthogonal_dimension: u16,
    input_form_profile: u128,
    input_form_mask: u64,
    output_form_profile: u128,
    output_form_count: u8,
    output_form_rank: u8,
    output_duality: rendering.DualityTag,
    right_chirality: u8,
    chirality: u8,
};

fn appendTensorFormProjectionGammaFallback(cache:
↳ *projector_constructor.TensorFormProjectionProgramCache, atoms:
↳ *std.ArrayList(rendering.SymbolicAtom), program: TensorFormProjectionGammaFallback) !void
↳ {
    _ = try cache.appendExpression(atoms, .{
        .operator_id = program.operator_id,
        .left = program.left,
        .right = program.right,
        .output = program.output,
        .orthogonal_dimension = program.orthogonal_dimension,
        .input_form_profile = program.input_form_profile,
        .input_form_mask = program.input_form_mask,
        .output_form_profile = program.output_form_profile,
        .output_form_count = program.output_form_count,
        .output_form_rank = program.output_form_rank,
        .output_duality = program.output_duality,
        .right_chirality = program.right_chirality,
        .chirality = program.chirality,
    });
}

fn findIndexKind(indices: []const realization.NamedIndex, kind: realization.IndexKind) ?usize
↳ {
    for (indices, 0..) |index, offset| {
        if (index.kind == kind) return offset;
    }
    return null;
}

```

```

fn findSpinorIndex(indices: []const realization.NamedIndex) ?usize {
    for (indices, 0..) |index, offset| {
        switch (index.kind) {
            .spinor, .conjugate_spinor => return offset,
            else => {},
        }
    }
    return null;
}

fn expectSpin10GammaForm(ctx: *Context, so10: store.AlgebraHandle, spinor: store.IrrepHandle,
    ↪ spinor_kind: realization.IndexKind, form_label: []const i16, expected_rank: u8,
    ↪ expected_chirality: u8, expected_duality: rendering.DualityTag) !void {
    const testing = std.testing;

    const form = try ctx.registerIrrep(so10, .{ .dynkin = form_label });
    const external_form = try ctx.dualIrrep(form);
    const spinor_leg = realization.ExternalLeg.primitive(spinor, &.{
        .init(spinor_kind, "a"),
    });
    const form_leg = realization.ExternalLeg.primitive(external_form, &.{
        .init(form, "M"),
    });
    const basis = try ctx.invariantBasis(.{
        .algebra = so10,
        .external_legs = &.{ spinor_leg, spinor_leg, form_leg },
    });

    try testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?);
    var sink: CountingSink = .{};
    const audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors =
    ↪ .expanded_terms }, rendering.ExpansionFilter.acceptAll(), &sink);
    try testing.expectEqual(@as(u32, 1), sink.term_count);
    try testing.expectEqual(@as(u32, 2), sink.atom_count);
    try testing.expectEqual(@as(u32, 1), sink.gamma_form_count);
    try testing.expectEqual(@as(u32, 1), sink.generalized_delta_count);
    try testing.expectEqual(@as(u16, 10), sink.first_gamma_form_dimension.?);
    try testing.expectEqual(expected_rank, sink.first_gamma_form_rank.?);
    try testing.expectEqual(expected_chirality, sink.first_gamma_form_chirality.?);
    try testing.expectEqual(expected_duality, sink.first_gamma_form_duality.?);
    try testing.expectEqual(@as(u64, 1), audit.accepted);
    try testing.expectEqual(@as(u64, 1), audit.emitted);

    const descriptor = ctx.projectorDescriptor(sink.first_gamma_form_operator_id.?).?;
    switch (descriptor.role) {
        .product_channel => |channel| {
            try testing.expectEqual(spinor.value, channel.left.value);
            try testing.expectEqual(spinor.value, channel.right.value);
            try testing.expectEqual(form.value, channel.output.value);
            try testing.expectEqual(@as(u16, 0), channel.multiplicity_copy);
        },
        else => return error.ExpectedGammaFormProductChannel,
    }
    try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
    ↪ descriptor.derivation.status);
    try testing.expectEqual(projector.ProjectorFormulaKind.orthogonal_spinor_form_channel,
    ↪ descriptor.derivation.formula_kind);
    try testing.expect(descriptor.derivation.formula_audit.verified());

    const paths = ctx.impl().couplings.basisPaths(basis).?;
    const steps = ctx.impl().couplings.pathSteps(paths[0]);
    try testing.expectEqual(@as(usize, 2), steps.len);

```

```

const form_pair_derivation =
  ↪ ctx.impl().projectors.projectorDerivation(steps[1].projector).?;
try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
  ↪ form_pair_derivation.status);
try testing.expectEqual(projector.ProjectorFormulaKind.orthogonal_form_pair,
  ↪ form_pair_derivation.formula_kind);
try testing.expect(form_pair_derivation.formula_audit.verified());

var gamma_filter_sink: CountingSink = .{};
const gamma_filter_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors
  ↪ = .expanded_terms }, rendering.ExpansionFilter.withoutOperatorKind(.gamma),
  ↪ &gamma_filter_sink);
try testing.expectEqual(@as(u32, 0), gamma_filter_sink.term_count);
try testing.expectEqual(@as(u64, 1), gamma_filter_audit.rejected);
try testing.expectEqual(@as(u64, 0), gamma_filter_audit.emitted);
}

fn expectSpin10TensorSpinorProjection(ctx: *Context, so10: store.AlgebraHandle, tower:
  ↪ store.IrrepHandle, spinor: store.IrrepHandle, output_label: []const i16, expected_rank:
  ↪ u8, expected_form_count: u8, expected_form_mask: u64, expected_tower_power: u16,
  ↪ expected_chirality: u8, expected_formula_kind: projector.ProjectorFormulaKind) !void {
  const testing = std.testing;
  _ = expected_rank;
  _ = expected_form_mask;
  _ = expected_formula_kind;

  const output = try ctx.registerIrrep(so10, .{ .dynkin = output_label });
  const tower_leg = realization.ExternalLeg.primitive(tower, &.{
    .init(.custom, "T"),
  });
  const spinor_leg = realization.ExternalLeg.primitive(spinor, &.{
    .init(.spinor, "a"),
  });
  const basis = try ctx.invariantBasis(.{
    .algebra = so10,
    .external_legs = &.{ tower_leg, spinor_leg },
    .target = .{ .irrep = output },
  });

  try testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?);
  var sink: CountingSink = .{};
  const audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors =
    ↪ .expanded_terms }, rendering.ExpansionFilter.acceptAll(), &sink);
  try testing.expect(sink.term_count > 0);
  const solved_spinor_contract = sink.spinor_tower_contract_count > 0;
  const solved_gamma_wedge_shift = sink.tensor_form_gamma_wedge_count > 0;
  const solved_gamma_map = sink.tensor_form_gamma_map_count > 0 or
    ↪ sink.tensor_form_gamma_map_adjoint_count > 0;
  const solved_form_spinor_pair = sink.tensor_form_spinor_pair_count > 0;
  const solved_gamma_action = sink.gamma_action_count > 0;
  const solved_exterior_gamma_action = sink.exterior_gamma_action_count > 0;
  const solved_structural = solved_spinor_contract or solved_gamma_wedge_shift or
    ↪ solved_gamma_map or solved_form_spinor_pair or solved_gamma_action or
    ↪ solved_exterior_gamma_action;
  if (solved_spinor_contract and solved_gamma_wedge_shift) {
    try testing.expectEqual(@as(u32, 0), sink.tensor_spinor_projection_count);
    try testing.expect(sink.tensor_form_gamma_wedge_count >= 2);
    try testing.expectEqual(sink.spinor_tower_contract_count,
      ↪ sink.spinor_tower_contract_adjoint_count);
    try testing.expect(sink.spinor_tower_contract_count >= 1);
    try testing.expect(sink.atom_count >= sink.spinor_tower_contract_count +
      ↪ sink.spinor_tower_contract_adjoint_count + sink.tensor_form_gamma_wedge_count +
      ↪ sink.generalized_delta_count);
    try testing.expectEqual(@as(u16, 10), sink.first_spinor_tower_contract_dimension.?);
  }
}

```

```

    try testing.expectEqual(@as(u16, 10), sink.first_tensor_form_gamma_wedge_dimension.?);
    try testing.expect(sink.first_spinor_tower_contract_chirality.? != 0);
    try testing.expect(sink.first_tensor_form_gamma_wedge_chirality.? != 0);
} else if (solved_spinor_contract) {
    try testing.expectEqual(@as(u32, 2) + sink.generalized_delta_count, sink.atom_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_spinor_projection_count);
    try testing.expectEqual(@as(u32, 1), sink.spinor_tower_contract_count);
    try testing.expectEqual(@as(u32, 1), sink.spinor_tower_contract_adjoint_count);
    try testing.expectEqual(@as(u32, 0), sink.generalized_delta_count % 2);
    try testing.expect(sink.generalized_delta_count <= @as(u32, expected_form_count) * 2);
    try testing.expectEqual(@as(u16, 10), sink.first_spinor_tower_contract_dimension.?);
    const contract_input = sink.first_spinor_tower_contract_input_power.?;
    const contract_output = sink.first_spinor_tower_contract_output_power.?;
    try testing.expect(contract_input == contract_output + 1);
    try testing.expect(contract_input == expected_tower_power or contract_output ==
        ↪ expected_tower_power);
    try testing.expect(sink.first_spinor_tower_contract_form_rank.? <= 10);
    try testing.expect(sink.first_spinor_tower_contract_chirality.? != 0);
} else if (solved_gamma_wedge_shift) {
    try testing.expectEqual(@as(u32, 2), sink.atom_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_spinor_projection_count);
    try testing.expectEqual(@as(u32, 2), sink.tensor_form_gamma_wedge_count);
    try testing.expectEqual(@as(u16, 10), sink.first_tensor_form_gamma_wedge_dimension.?);
    try testing.expect(sink.first_tensor_form_gamma_wedge_input_profile.? != 0);
    try testing.expect(sink.first_tensor_form_gamma_wedge_output_profile.? != 0);
    try testing.expect(sink.first_tensor_form_gamma_wedge_inserted_rank.? != 0);
    try testing.expectEqual(expected_chirality,
        ↪ sink.first_tensor_form_gamma_wedge_chirality.?);
} else {
    try testing.expect(sink.atom_count > 0);
    try testing.expect(solved_structural);
    try testing.expectEqual(@as(u32, 0), sink.tensor_spinor_projection_count);
    if (solved_gamma_map) {
        try testing.expectEqual(@as(u16, 10),
            ↪ sink.first_tensor_form_gamma_map_dimension.?);
        try testing.expect(sink.first_tensor_form_gamma_map_source_rank.? <= 10);
        try testing.expect(sink.first_tensor_form_gamma_map_chirality.? != 0);
    }
    if (solved_form_spinor_pair) {
        try testing.expectEqual(@as(u16, 10),
            ↪ sink.first_tensor_form_spinor_pair_dimension.?);
        try testing.expect(sink.first_tensor_form_spinor_pair_output_count.? <=
            ↪ expected_form_count);
    }
}
try testing.expectEqual(@as(u64, sink.term_count), audit.accepted);
try testing.expectEqual(@as(u64, sink.term_count), audit.emitted);

const paths = ctx.impl().couplings.basisPaths(basis).?;
const steps = ctx.impl().couplings.pathSteps(paths[0]);
try testing.expect(steps.len > 0);
const descriptor = ctx.projectorDescriptor(steps[0].projector).?;
switch (descriptor.role) {
    .product_channel => |channel| {
        try testing.expectEqual(tower.value, channel.left.value);
        try testing.expectEqual(spinor.value, channel.right.value);
        try testing.expectEqual(output.value, channel.output.value);
        try testing.expectEqual(@as(u16, 0), channel.multiplicity_copy);
    },
    else => return error.ExpectedTensorSpinorProductChannel,
}
try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
    ↪ descriptor.derivation.status);

```

```

try testing.expectEqual(projector.ProjectorFormulaKind.orthogonal_structural_projection,
↳ descriptor.derivation.formula_kind);
try testing.expect(descriptor.derivation.formula_audit.verified());

var projector_filter_sink: CountingSink = .{};
const projector_filter_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{
↳ .projectors = .expanded_terms },
↳ rendering.ExpansionFilter.withoutOperatorKind(.projector), &projector_filter_sink);
if (solved_structural) {
    try testing.expect(projector_filter_sink.term_count > 0);
    try testing.expectEqual(@as(u32, 0),
↳ projector_filter_sink.tensor_spinor_projection_count);
    try testing.expectEqual(@as(u64, 0), projector_filter_audit.rejected);
    try testing.expectEqual(@as(u64, projector_filter_sink.term_count),
↳ projector_filter_audit.emitted);
} else {
    try testing.expectEqual(@as(u32, 0), projector_filter_sink.term_count);
    try testing.expectEqual(@as(u64, 1), projector_filter_audit.rejected);
    try testing.expectEqual(@as(u64, 0), projector_filter_audit.emitted);
}
}

fn expectSpin10TensorFormProjection(ctx: *Context, so10: store.AlgebraHandle, left:
↳ store.IrrepHandle, spinor: store.IrrepHandle, output_label: []const i16,
↳ expected_input_mask: u64, expected_output_profile: u128, expected_output_count: u8,
↳ expected_formula_kind: projector.ProjectorFormulaKind) !void {
    const testing = std.testing;

    const output = try ctx.registerIrrep(so10, .{ .dynkin = output_label });
    const left_leg = realization.ExternalLeg.primitive(left, &.{
        .init(.custom, "T"),
    });
    const spinor_leg = realization.ExternalLeg.primitive(spinor, &.{
        .init(.spinor, "a"),
    });
    const basis = try ctx.invariantBasis(.{
        .algebra = so10,
        .external_legs = &.{ left_leg, spinor_leg },
        .target = .{ .irrep = output },
    });

    try testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?);
    var sink: CountingSink = .{};
    const audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors =
↳ .expanded_terms }, rendering.ExpansionFilter.acceptAll(), &sink);
    try testing.expectEqual(@as(u32, 1), sink.term_count);
    const gamma_terminal_delta_count: u32 = if
↳ (isTensorFormGammaPrimitiveFormulaKind(expected_formula_kind) and
↳ formProfilesIntersect(formProfileFromMask(expected_input_mask),
↳ expected_output_profile)) 2 else 0;
    const gamma_map_delta_count: u32 = if
↳ (isTensorFormGammaMapPrimitiveFormulaKind(expected_formula_kind) and
↳ formProfilesIntersect(formProfileFromMask(expected_input_mask),
↳ expected_output_profile)) 2 else 0;
    try testing.expect(sink.atom_count > 0);
    const operator_id = if (expected_formula_kind ==
↳ .orthogonal_tensor_spinor_terminal_channel) terminal: {
        try testing.expectEqual(@as(u32, 0), sink.tensor_form_projection_count);
        try testing.expectEqual(@as(u32, 0), sink.tensor_form_gamma_wedge_count);
        try testing.expectEqual(@as(u32, 0), sink.tensor_form_spinor_pair_count);
        try testing.expectEqual(@as(u32, 1), sink.gamma_form_count);
        try testing.expectEqual(@as(u32, 1), sink.gamma_action_count);
        try testing.expectEqual(gamma_terminal_delta_count, sink.generalized_delta_count);
        try testing.expectEqual(@as(u16, 10), sink.first_gamma_form_dimension.?).

```

```

    try testing.expectEqual(@as(u8, 1), sink.first_gamma_form_rank.?);
    try testing.expect(sink.first_gamma_form_chirality.? != 0);
    break :terminal sink.first_gamma_form_operator_id.?.
} else if (expected_formula_kind == .orthogonal_tensor_spinor_opposite_terminal_channel)
↪ opposite_terminal: {
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_projection_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_gamma_wedge_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_spinor_pair_count);
    try testing.expectEqual(@as(u32, 1), sink.gamma_form_count);
    try testing.expectEqual(@as(u32, 1), sink.gamma_action_count);
    try testing.expectEqual(gamma_terminal_delta_count, sink.generalized_delta_count);
    try testing.expectEqual(@as(u16, 10), sink.first_gamma_form_dimension.?);
    try testing.expectEqual(@as(u8, 2), sink.first_gamma_form_rank.?);
    try testing.expect(sink.first_gamma_form_chirality.? != 0);
    break :opposite_terminal sink.first_gamma_form_operator_id.?.
} else if (expected_formula_kind ==
↪ .orthogonal_tensor_spinor_opposite_form_add_terminal_channel)
↪ opposite_form_add_terminal: {
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_projection_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_gamma_wedge_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_spinor_pair_count);
    try testing.expectEqual(@as(u32, 1), sink.gamma_form_count);
    try testing.expectEqual(@as(u32, 1), sink.gamma_action_count);
    try testing.expectEqual(gamma_terminal_delta_count, sink.generalized_delta_count);
    try testing.expectEqual(@as(u16, 10), sink.first_gamma_form_dimension.?);
    try testing.expect(sink.first_gamma_form_rank.? != 0);
    try testing.expect(sink.first_gamma_form_chirality.? != 0);
    break :opposite_form_add_terminal sink.first_gamma_form_operator_id.?.
} else if (expected_formula_kind == .orthogonal_tensor_spinor_form_add_terminal_channel)
↪ form_add_terminal: {
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_projection_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_gamma_wedge_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_spinor_pair_count);
    try testing.expectEqual(@as(u32, 1), sink.gamma_form_count);
    try testing.expectEqual(@as(u32, 1), sink.gamma_action_count);
    try testing.expectEqual(gamma_terminal_delta_count, sink.generalized_delta_count);
    try testing.expectEqual(@as(u16, 10), sink.first_gamma_form_dimension.?);
    try testing.expect(sink.first_gamma_form_rank.? != 0);
    try testing.expect(sink.first_gamma_form_chirality.? != 0);
    break :form_add_terminal sink.first_gamma_form_operator_id.?.
} else if (expected_formula_kind == .orthogonal_tensor_spinor_form_power_terminal_channel)
↪ form_power_terminal: {
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_projection_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_gamma_wedge_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_spinor_pair_count);
    try testing.expectEqual(@as(u32, 1), sink.gamma_form_count);
    try testing.expectEqual(@as(u32, 1), sink.gamma_action_count);
    try testing.expectEqual(gamma_terminal_delta_count, sink.generalized_delta_count);
    try testing.expectEqual(@as(u16, 10), sink.first_gamma_form_dimension.?);
    try testing.expect(sink.first_gamma_form_rank.? != 0);
    try testing.expect(sink.first_gamma_form_chirality.? != 0);
    break :form_power_terminal sink.first_gamma_form_operator_id.?.
} else if (isTensorFormGammaPrimitiveFormulaKind(expected_formula_kind)) gamma_terminal: {
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_projection_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_gamma_wedge_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_spinor_pair_count);
    try testing.expectEqual(@as(u32, 1), sink.gamma_form_count);
    try testing.expectEqual(@as(u32, 1), sink.gamma_action_count);
    try testing.expectEqual(gamma_terminal_delta_count, sink.generalized_delta_count);
    try testing.expectEqual(@as(u16, 10), sink.first_gamma_form_dimension.?);
    try testing.expect(sink.first_gamma_form_rank.? != 0);
    try testing.expect(sink.first_gamma_form_chirality.? != 0);
    break :gamma_terminal sink.first_gamma_form_operator_id.?.

```



```

} else if (expected_formula_kind == .orthogonal_tensor_spinor_form_terminal_channel)
  ↪ form_terminal: {
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_projection_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_gamma_wedge_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_spinor_pair_count);
    try testing.expectEqual(@as(u32, 1), sink.spinor_pair_count);
    try testing.expectEqual(@as(u32, 1), sink.generalized_delta_count);
    try testing.expectEqual(@as(u16, 10), sink.first_spinor_pair_dimension.?);
    try testing.expect(sink.first_spinor_pair_left_chirality.? != 0);
    try testing.expect(sink.first_spinor_pair_right_chirality.? != 0);
    break :form_terminal sink.first_spinor_pair_operator_id.?;
} else if (isTensorFormGammaMapPrimitiveFormulaKind(expected_formula_kind)) gamma_map: {
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_projection_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_gamma_wedge_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_spinor_pair_count);
    try testing.expectEqual(gamma_map_delta_count, sink.generalized_delta_count);
    if (sink.tensor_form_gamma_map_count != 0) {
      try testing.expectEqual(@as(u32, 1), sink.tensor_form_gamma_map_count);
      try testing.expectEqual(@as(u32, 1), sink.tensor_form_gamma_map_adjoint_count);
      try testing.expectEqual(@as(u16, 10),
        ↪ sink.first_tensor_form_gamma_map_dimension.?);
      try testing.expect(sink.first_tensor_form_gamma_map_source_rank.? != 0);
      try testing.expect((expected_input_mask & (@as(u64, 1) <<
        ↪ @intCast(sink.first_tensor_form_gamma_map_source_rank.? - 1))) != 0);
      try testing.expect(formProfileCount(expected_output_profile,
        ↪ sink.first_tensor_form_gamma_map_lower_rank.? - 1) != 0);
      try testing.expect(formProfileCount(expected_output_profile,
        ↪ sink.first_tensor_form_gamma_map_upper_rank.? - 1) != 0);
      try testing.expect(sink.first_tensor_form_gamma_map_chirality.? != 0);
      break :gamma_map sink.first_tensor_form_gamma_map_operator_id.?;
    }
    if (sink.exterior_gamma_action_count != 0) {
      try testing.expectEqual(@as(u16, 10),
        ↪ sink.first_exterior_gamma_action_dimension.?);
      try testing.expect(sink.first_exterior_gamma_action_chirality.? != 0);
      break :gamma_map sink.first_exterior_gamma_action_operator_id.?;
    }
    try testing.expect(sink.gamma_form_count > 0 and sink.gamma_action_count > 0);
    try testing.expectEqual(@as(u16, 10), sink.first_gamma_form_dimension.?);
    try testing.expect(sink.first_gamma_form_chirality.? != 0);
    break :gamma_map sink.first_gamma_form_operator_id.?;
} else compact: {
    try testing.expectEqual(@as(u32, 1), sink.tensor_form_projection_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_gamma_wedge_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_spinor_pair_count);
    try testing.expectEqual(@as(u16, 10), sink.first_tensor_form_projection_dimension.?);
    try testing.expectEqual(expected_input_mask,
      ↪ sink.first_tensor_form_projection_input_mask.?);
    try testing.expectEqual(expected_output_profile,
      ↪ sink.first_tensor_form_projection_output_profile.?);
    try testing.expectEqual(expected_output_count,
      ↪ sink.first_tensor_form_projection_output_count.?);
    break :compact sink.first_tensor_form_projection_operator_id.?;
};
try testing.expectEqual(@as(u64, 1), audit.accepted);
try testing.expectEqual(@as(u64, 1), audit.emitted);

const descriptor = ctx.projectorDescriptor(operator_id).?;
switch (descriptor.role) {
  .product_channel => |channel| {
    try testing.expectEqual(left.value, channel.left.value);
    try testing.expectEqual(spinor.value, channel.right.value);
    try testing.expectEqual(output.value, channel.output.value);
    try testing.expectEqual(@as(u16, 0), channel.multiplicity_copy);

```

```

    },
    else => return error.ExpectedTensorFormProductChannel,
}
try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
    ↪ descriptor.derivation.status);
try testing.expectEqual(expected_formula_kind, descriptor.derivation.formula_kind);
try testing.expect(descriptor.derivation.formula_audit.verified());

var projector_filter_sink: CountingSink = .{};
const projector_filter_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{
    ↪ .projectors = .expanded_terms },
    ↪ rendering.ExpansionFilter.withoutOperatorKind(.projector), &projector_filter_sink);
if (isTensorFormGammaPrimitiveFormulaKind(expected_formula_kind)) {
    try testing.expectEqual(@as(u32, 1), projector_filter_sink.term_count);
    try testing.expectEqual(@as(u32, 1), projector_filter_sink.gamma_form_count);
    try testing.expectEqual(@as(u32, 1), projector_filter_sink.gamma_action_count);
    try testing.expectEqual(gamma_terminal_delta_count,
        ↪ projector_filter_sink.generalized_delta_count);
    try testing.expectEqual(@as(u64, 0), projector_filter_audit.rejected);
    try testing.expectEqual(@as(u64, 1), projector_filter_audit.emitted);
} else if (expected_formula_kind == .orthogonal_tensor_spinor_form_terminal_channel) {
    try testing.expectEqual(@as(u32, 1), projector_filter_sink.term_count);
    try testing.expectEqual(@as(u32, 1), projector_filter_sink.spinor_pair_count);
    try testing.expectEqual(@as(u32, 1), projector_filter_sink.generalized_delta_count);
    try testing.expectEqual(@as(u64, 0), projector_filter_audit.rejected);
    try testing.expectEqual(@as(u64, 1), projector_filter_audit.emitted);
} else if (isTensorFormGammaMapPrimitiveFormulaKind(expected_formula_kind)) {
    try testing.expectEqual(@as(u32, 1), projector_filter_sink.term_count);
    try testing.expectEqual(@as(u32, 0),
        ↪ projector_filter_sink.tensor_form_projection_count);
    try testing.expect(projector_filter_sink.tensor_form_gamma_map_count != 0 or
        projector_filter_sink.exterior_gamma_action_count != 0 or
        projector_filter_sink.gamma_form_count != 0);
    try testing.expectEqual(gamma_map_delta_count,
        ↪ projector_filter_sink.generalized_delta_count);
    try testing.expectEqual(@as(u64, 0), projector_filter_audit.rejected);
    try testing.expectEqual(@as(u64, 1), projector_filter_audit.emitted);
} else {
    try testing.expectEqual(@as(u32, 0), projector_filter_sink.term_count);
    try testing.expectEqual(@as(u64, 1), projector_filter_audit.rejected);
    try testing.expectEqual(@as(u64, 0), projector_filter_audit.emitted);
}
}

fn isTensorFormGammaPrimitiveFormulaKind(kind: projector.ProjectorFormulaKind) bool {
    return kind == .orthogonal_tensor_spinor_terminal_channel or
        kind == .orthogonal_tensor_spinor_opposite_terminal_channel or
        kind == .orthogonal_tensor_spinor_opposite_form_add_terminal_channel or
        kind == .orthogonal_tensor_spinor_form_add_terminal_channel or
        kind == .orthogonal_tensor_spinor_form_power_terminal_channel or
        kind == .orthogonal_tensor_spinor_opposite_shift_terminal_channel or
        kind == .orthogonal_tensor_spinor_rank_wrap_terminal_channel;
}

fn isTensorFormGammaMapPrimitiveFormulaKind(kind: projector.ProjectorFormulaKind) bool {
    return kind == .orthogonal_tensor_spinor_opposite_rank_split_terminal_channel or
        kind == .orthogonal_tensor_spinor_rank_split_terminal_channel or
        kind == .orthogonal_tensor_spinor_opposite_form_add_shift_terminal_channel;
}

fn expectSpin10TensorMiddleFormProjection(ctx: *Context, so10: store.AlgebraHandle, left:
    ↪ store.IrrepHandle, spinor: store.IrrepHandle, output_label: []const i16,
    ↪ expected_input_mask: u64, expected_output_rank: u8, expected_duality:
    ↪ rendering.DualityTag, expected_formula_kind: projector.ProjectorFormulaKind) !void {

```



```

const testing = std.testing;

const output = try ctx.registerIrrep(so10, .{ .dynkin = output_label });
const left_leg = realization.ExternalLeg.primitive(left, &.{
    .init(.custom, "T"),
});
const spinor_leg = realization.ExternalLeg.primitive(spinor, &.{
    .init(.spinor, "a"),
});
const basis = try ctx.invariantBasis(.{
    .algebra = so10,
    .external_legs = &.{ left_leg, spinor_leg },
    .target = .{ .irrep = output },
});

try testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?);
var sink: CountingSink = .{};
const audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors =
    ↪ .expanded_terms }, rendering.ExpansionFilter.acceptAll(), &sink);
try testing.expectEqual(@as(u32, 1), sink.term_count);
const structural_middle =
    expected_formula_kind == .orthogonal_spinor_tower_middle_form_channel or
    expected_formula_kind == .orthogonal_tensor_spinor_middle_form_channel;
try testing.expect(sink.atom_count > 0);
if (structural_middle) {
    const input_rank: u8 = if (expected_input_mask == 0) 0 else
        ↪ @intCast(@ctz(expected_input_mask) + 1);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_projection_count);
    if (sink.tensor_form_gamma_wedge_count != 0) {
        try testing.expectEqual(@as(u16, 10),
            ↪ sink.first_tensor_form_gamma_wedge_dimension.?);
        try testing.expectEqual(expected_input_mask,
            ↪ sink.first_tensor_form_gamma_wedge_input_mask.?);
        try testing.expectEqual(@as(u128, 0),
            ↪ sink.first_tensor_form_gamma_wedge_output_profile.?);
        try testing.expectEqual(@as(u8, 1),
            ↪ sink.first_tensor_form_gamma_wedge_output_count.?);
        try testing.expectEqual(expected_output_rank - input_rank,
            ↪ sink.first_tensor_form_gamma_wedge_inserted_rank.?);
    } else if (sink.exterior_gamma_action_count != 0) {
        try testing.expectEqual(@as(u16, 10),
            ↪ sink.first_exterior_gamma_action_dimension.?);
    } else {
        try testing.expect(sink.gamma_form_count > 0 and sink.gamma_action_count > 0);
        try testing.expectEqual(@as(u16, 10), sink.first_gamma_form_dimension.?);
        try testing.expectEqual(expected_duality, sink.first_gamma_form_duality.?);
    }
} else {
    try testing.expectEqual(@as(u32, 1), sink.tensor_form_projection_count);
    try testing.expectEqual(@as(u32, 0), sink.tensor_form_gamma_wedge_count);
    try testing.expectEqual(@as(u16, 10), sink.first_tensor_form_projection_dimension.?);
    try testing.expectEqual(expected_input_mask,
        ↪ sink.first_tensor_form_projection_input_mask.?);
    try testing.expectEqual(@as(u128, 0),
        ↪ sink.first_tensor_form_projection_output_profile.?);
    try testing.expectEqual(@as(u8, 1), sink.first_tensor_form_projection_output_count.?);
    try testing.expectEqual(expected_output_rank,
        ↪ sink.first_tensor_form_projection_output_rank.?);
    try testing.expectEqual(expected_duality,
        ↪ sink.first_tensor_form_projection_output_duality.?);
}
try testing.expectEqual(@as(u64, 1), audit.accepted);
try testing.expectEqual(@as(u64, 1), audit.emitted);

```

```

const operator_id = if (!structural_middle)
  ↪ sink.first_tensor_form_projection_operator_id.? else if
  ↪ (sink.tensor_form_gamma_wedge_count != 0)
  ↪ sink.first_tensor_form_gamma_wedge_operator_id.? else if
  ↪ (sink.exterior_gamma_action_count != 0) sink.first_exterior_gamma_action_operator_id.?
  ↪ else sink.first_gamma_form_operator_id.?.
const descriptor = ctx.projectorDescriptor(operator_id).?.
switch (descriptor.role) {
  .product_channel => |channel| {
    try testing.expectEqual(left.value, channel.left.value);
    try testing.expectEqual(spinor.value, channel.right.value);
    try testing.expectEqual(output.value, channel.output.value);
    try testing.expectEqual(@as(u16, 0), channel.multiplicity_copy);
  },
  else => return error.ExpectedTensorMiddleFormProductChannel,
}
try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
  ↪ descriptor.derivation.status);
try testing.expectEqual(expected_formula_kind, descriptor.derivation.formula_kind);
try testing.expect(descriptor.derivation.formula_audit.verified());
}

fn expectDualPairInvariantCount(ctx: *Context, simple: symmetry.SimpleLieAlgebra, label:
  ↪ []const i16) !void {
  const algebra = try ctx.registerAlgebra(.{ .simple = simple });
  const irrep = try ctx.registerIrrep(algebra, .{ .dynkin = label });
  const dual = try ctx.dualIrrep(irrep);
  const left = realization.ExternalLeg.primitive(irrep, &.{
    .init(.custom, "left"),
  });
  const right = realization.ExternalLeg.primitive(dual, &.{
    .init(.custom, "right"),
  });
  const basis = try ctx.invariantBasis(.{
    .algebra = algebra,
    .external_legs = &.{ left, right },
  });

  try std.testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?.);
  try std.testing.expectEqual(@as(u128, 1), ctx.basisAudit(basis).?.path_count);
}

fn expectRepeatedLegInvariantCount(ctx: *Context, simple: symmetry.SimpleLieAlgebra, label:
  ↪ []const i16, comptime leg_count: usize, expected_count: u128) !void {
  const algebra = try ctx.registerAlgebra(.{ .simple = simple });
  const irrep = try ctx.registerIrrep(algebra, .{ .dynkin = label });
  const leg = realization.ExternalLeg.primitive(irrep, &.{
    .init(.custom, "x"),
  });
  const legs = [_]realization.ExternalLeg{leg} ** leg_count;
  const basis = try ctx.invariantBasis(.{
    .algebra = algebra,
    .external_legs = legs[0..],
  });

  try std.testing.expectEqual(expected_count, ctx.basisInvariantCount(basis).?.);
  try std.testing.expectEqual(expected_count, ctx.basisAudit(basis).?.path_count);
}

```

5 Tensor Backend

Backends are internal family-specific providers. The public workflow should not depend on an ADE, gamma-matrix, Young-symmetrizer, or exceptional-algebra implementation detail. A backend

validates support, supplies representation metadata, decomposes binary products, constructs local projector ids, and streams concrete symbolic terms when rendering is requested.

```

const decomposition = @import("decomposition.zig");
const projector = @import("projector.zig");
const rendering = @import("rendering.zig");
const root_data = @import("root-data.zig");
const store = @import("representation-store.zig");
const symmetry = @import("symmetry.zig");

/// BackendId names one registered Lie-family backend.
pub const BackendId = u16;

/// PrimitiveInvariantSpan names backend-owned primitive invariant descriptors.
pub const PrimitiveInvariantSpan = struct {
    offset: u32,
    len: u32,
};

/// RenderConvention records the concrete rendering convention passed to a backend.
pub const RenderConvention = struct {
    options: rendering.RenderOptions,
};

/// WeightSystemHandle names one backend-cached weight system.
pub const WeightSystemHandle = struct {
    value: u32,

    /// init constructs a weight-system handle.
    pub fn init(value: u32) WeightSystemHandle {
        return .{ .value = value };
    }
};

/// WeightSystemAudit stores mandatory compact weight-generation counters.
pub const WeightSystemAudit = struct {
    weight_count: u32 = 0,
    total_multiplicity: u128 = 0,
};

/// Capabilities is the internal contract every Lie-family backend must satisfy.
pub const Capabilities = struct {
    validate_algebra: *const fn (*anyopaque, symmetry.SimpleLieAlgebra) anyerror!void,
    irrep_metadata: *const fn (*anyopaque, store.AlgebraHandle, store.IrrepHandle)
        ↪ anyerror!store.IrrepMetadata,
    weight_system: *const fn (*anyopaque, *store.Store, store.AlgebraHandle,
        ↪ store.IrrepHandle) anyerror!WeightSystemHandle,
    decompose_product: *const fn (*anyopaque, *store.Store, *decomposition.Store,
        ↪ decomposition.ProductKey) anyerror!decomposition.ProductDecompositionHandle,
    local_projector: *const fn (*anyopaque, projector.ProjectorKey)
        ↪ anyerror!projector.ProjectorId,
    primitive_invariants: *const fn (*anyopaque, []const store.IrrepHandle)
        ↪ anyerror!PrimitiveInvariantSpan,
    render_projector: *const fn (*anyopaque, projector.ProjectorId, RenderConvention,
        ↪ *anyopaque) anyerror!void,
};

/// Record stores one backend state pointer and capability table.
pub const Record = struct {
    id: BackendId,
    state: *anyopaque,
    family: symmetry.LieFamily,
    root_conventions: root_data.AlgebraConventions,
    capabilities: Capabilities,
};

```

```
};
```

6 Tensor Representation Store

This module owns the abstract representation data. A Lie algebra and an irrep are interned once, then every downstream system carries small typed handles. That keeps tensor generation independent of how a representation is eventually rendered as indices, matrices, or symbolic projectors.

```
const std = @import("std");
const root_data = @import("root-data.zig");
const symmetry = @import("symmetry.zig");
```

Handles are public because other modules need stable references. The records behind them stay private.

```
/// AlgebraHandle names a registered Lie algebra in a tensor context.
pub const AlgebraHandle = struct {
    value: u32,

    /// init constructs an algebra handle from a store index.
    pub fn init(value: u32) AlgebraHandle {
        return .{ .value = value };
    }
};

/// IrrepHandle names an abstract irreducible representation.
pub const IrrepHandle = struct {
    value: u32,

    /// init constructs an irrep handle from a store index.
    pub fn init(value: u32) IrrepHandle {
        return .{ .value = value };
    }
};

/// ChannelHandle names one irrep copy inside a local product.
pub const ChannelHandle = struct {
    value: u32,

    /// init constructs a channel handle from a store index.
    pub fn init(value: u32) ChannelHandle {
        return .{ .value = value };
    }
};
```

Specs are caller-owned construction data. Interning clones only the slices we must retain.

```
/// AlgebraSpec describes a requested algebra before it is interned.
pub const AlgebraSpec = union(enum) {
    u1,
    simple: symmetry.SimpleLieAlgebra,
};

/// IrrepSpec describes a requested representation before it is interned.
pub const IrrepSpec = union(enum) {
    u1_charge: symmetry.U1ChargeRational,
    dynkin: []const i16,
};

/// IrrepMetadata stores derived representation data.
pub const IrrepMetadata = struct {
```

```

dimension: ?u128 = null,
dual: ?IrrepHandle = null,
quadratic_casimir: ?root_data.Rational = null,
};

```

The store is intentionally simple: linear lookup first, then append. The first implementation has few registered algebras and irreps compared with the later coupling search, so this avoids prematurely adding hash table machinery. If profiles show this store in the hot path, the record layout can stay fixed while we add an index.

```

/// Store owns interned algebra and irrep records.
pub const Store = struct {
    allocator: std.mem.Allocator,
    conventions: root_data.AlgebraConventions,
    algebras: std.ArrayList(AlgebraRecord) = .empty,
    irreps: std.ArrayList(IrrepRecord) = .empty,

    /// init constructs an empty representation store.
    pub fn init(allocator: std.mem.Allocator, conventions: root_data.AlgebraConventions) Store
    ↪ {
        return .{ .allocator = allocator, .conventions = conventions };
    }

    /// deinit releases all representation-store memory.
    pub fn deinit(self: *Store) void {
        for (self.irreps.items) |record| {
            switch (record.label) {
                .dynkin => |label| self.allocator.free(label),
                .u1_charge => {},
            }
        }
        self.irreps.deinit(self.allocator);
        self.algebras.deinit(self.allocator);
        self.* = Store.init(self.allocator, self.conventions);
    }

    /// internAlgebra returns the stable handle for an algebra spec.
    pub fn internAlgebra(self: *Store, spec: AlgebraSpec) !AlgebraHandle {
        try validateAlgebraSpec(spec);
        for (self.algebras.items, 0..) |record, index| {
            if (algebraSpecEq(record.spec, spec)) {
                return AlgebraHandle.init(@intCast(index));
            }
        }

        try self.algebras.append(self.allocator, .{
            .spec = spec,
            .root_data = try makeRootDatum(spec, self.conventions),
        });
        return AlgebraHandle.init(@intCast(self.algebras.items.len - 1));
    }

    /// containsAlgebra returns whether an algebra handle belongs to this store.
    pub fn containsAlgebra(self: Store, handle: AlgebraHandle) bool {
        return self.algebraRecord(handle) != null;
    }

    /// internIrrep returns the stable handle for an irrep spec in an algebra.
    pub fn internIrrep(self: *Store, algebra: AlgebraHandle, spec: IrrepSpec)
    ↪ anyerror!IrrepHandle {
        const algebra_record = self.algebraRecord(algebra) orelse return error.UnknownAlgebra;
        try validateIrrepSpec(algebra_record.spec, spec);
    }

```

```

    for (self.irreps.items, 0..) |record, index| {
        if (handleEq(record.algebra, algebra) and irrepsLabelSpecEq(record.label, spec))
            ↪ {
                return IrrepHandle.init(@intCast(index));
            }
    }

    const label = try cloneIrrepSpec(self.allocator, spec);
    errdefer freeIrrepLabel(self.allocator, label);

    try self.irreps.append(self.allocator, .{
        .algebra = algebra,
        .label = label,
        .metadata = .{},
    });
    errdefer _ = self.irreps.pop();
    const handle = IrrepHandle.init(@intCast(self.irreps.items.len - 1));
    try self.fillIrrepMetadata(handle);
    return handle;
}

/// dualIrrep returns the handle for the dual irrep, interning it if requested.
pub fn dualIrrep(self: *Store, handle: IrrepHandle) anyerror!IrrepHandle {
    const index: usize = @intCast(handle.value);
    if (index >= self.irreps.items.len) return error.UnknownIrrep;
    if (self.irreps.items[index].metadata.dual) |dual| return dual;

    const record = self.irreps.items[index];
    const dual = switch (record.label) {
        .u1_charge => |charge| try self.internIrrep(record.algebra, .{ .u1_charge = .{
            .numerator = -charge.numerator,
            .denominator = charge.denominator,
        } }},
        .dynkin => |label| dual: {
            const algebra = self.algebraRecord(record.algebra) orelse return
                ↪ error.UnknownAlgebra;
            const dual_label = try root_data.dualDynkin(self.allocator,
                ↪ algebra.spec.simple, label);
            defer self.allocator.free(dual_label);
            break :dual try self.internIrrep(record.algebra, .{ .dynkin = dual_label });
        },
    };

    self.irreps.items[index].metadata.dual = dual;
    const dual_index: usize = @intCast(dual.value);
    if (dual_index < self.irreps.items.len) self.irreps.items[dual_index].metadata.dual =
        ↪ handle;
    return dual;
}

/// containsIrrep returns whether an irrep handle belongs to this store.
pub fn containsIrrep(self: Store, handle: IrrepHandle) bool {
    return self.irrepRecord(handle) != null;
}

/// irrepMetadata returns derived metadata for an irrep handle.
pub fn irrepMetadata(self: Store, handle: IrrepHandle) ?IrrepMetadata {
    const record = self.irrepRecord(handle) orelse return null;
    return record.metadata;
}

/// irrepAlgebra returns the algebra that owns an irrep handle.
pub fn irrepAlgebra(self: Store, handle: IrrepHandle) ?AlgebraHandle {
    const record = self.irrepRecord(handle) orelse return null;

```

```

        return record.algebra;
    }

    /// simpleAlgebra returns the simple algebra behind an internal handle.
    pub fn simpleAlgebra(self: Store, handle: AlgebraHandle) ?Symmetry.SimpleLieAlgebra {
        const record = self.algebraRecord(handle) orelse return null;
        return switch (record.spec) {
            .simple => |simple| simple,
            .u1 => null,
        };
    }

    /// irrepDynkin returns the borrowed Dynkin label behind an irrep handle.
    pub fn irrepDynkin(self: Store, handle: IrrepHandle) ?[]const i16 {
        const record = self.irrepRecord(handle) orelse return null;
        return switch (record.label) {
            .dynkin => |label| label,
            .u1_charge => null,
        };
    }

    fn algebraRecord(self: Store, handle: AlgebraHandle) ?AlgebraRecord {
        const index: usize = @intCast(handle.value);
        if (index >= self.algebras.items.len) return null;
        return self.algebras.items[index];
    }

    fn irrepRecord(self: Store, handle: IrrepHandle) ?IrrepRecord {
        const index: usize = @intCast(handle.value);
        if (index >= self.irreps.items.len) return null;
        return self.irreps.items[index];
    }

    fn fillIrrepMetadata(self: *Store, handle: IrrepHandle) anyerror!void {
        const index: usize = @intCast(handle.value);
        const record = self.irreps.items[index];
        const algebra = self.algebraRecord(record.algebra) orelse return error.UnknownAlgebra;

        switch (record.label) {
            .u1_charge => |charge| {
                self.irreps.items[index].metadata.dimension = 1;
                self.irreps.items[index].metadata.quadratic_casimir = try
                    ↪ u1QuadraticCasimir(charge);
                if (charge.numerator == 0) self.irreps.items[index].metadata.dual = handle;
            },
            .dynkin => |label| {
                const simple = algebra.spec.simple;
                self.irreps.items[index].metadata.dimension = try
                    ↪ root_data.dimension(self.allocator, simple, label);
                self.irreps.items[index].metadata.quadratic_casimir = try
                    ↪ root_data.quadraticCasimir(self.allocator, simple, label,
                    ↪ self.conventions);
                const dual_label = try root_data.dualDynkin(self.allocator, simple, label);
                defer self.allocator.free(dual_label);
                if (std.mem.eql(i16, label, dual_label)) {
                    self.irreps.items[index].metadata.dual = handle;
                } else if (self.findIrrep(record.algebra, .{ .dynkin = dual_label })) |dual| {
                    self.irreps.items[index].metadata.dual = dual;
                }
            },
        }
    }

    fn findIrrep(self: Store, algebra: AlgebraHandle, spec: IrrepSpec) ?IrrepHandle {

```

```

        for (self.irreps.items, 0..) |record, index| {
            if (handleEqL(record.algebra, algebra) and irreplabelSpecEqL(record.label, spec))
                ↪ {
                    return IrrepHandle.init(@intCast(index));
                }
        }
        return null;
    }
};

```

Private records carry derived metadata, root data, and future product-channel occurrences. They are not exported because callers should not depend on the storage layout.

```

const AlgebraRecord = struct {
    spec: AlgebraSpec,
    root_data: ?root_data.RootDatum,
};

const IrrepRecord = struct {
    algebra: AlgebraHandle,
    label: IrrepLabel,
    metadata: IrrepMetadata,
};

const IrrepLabel = union(enum) {
    u1_charge: symmetry.U1ChargeRational,
    dynkin: []const i16,
};

const ChannelRecord = struct {
    irrep: IrrepHandle,
    multiplicity_copy: u16,
};

const OccurrenceKind = enum {
    external_leg,
    product_node,
    realization,
};

const OccurrenceRecord = struct {
    channel: ChannelHandle,
    kind: OccurrenceKind,
    source_id: u32,
    realization: ?u32 = null,
};

```

Interning equality is structural and allocation-free. Validation happens before interning: a simple-algebra irrep must have one non-negative Dynkin label per rank, and a U(1) irrep must be a rational charge. Unknown dimension and dual data is represented by `null`, never by a fake handle.

```

fn handleEqL(left: AlgebraHandle, right: AlgebraHandle) bool {
    return left.value == right.value;
}

fn validateAlgebraSpec(spec: AlgebraSpec) !void {
    switch (spec) {
        .u1 => {},
        .simple => |simple| try root_data.validateSimple(simple),
    }
}

```



```

fn makeRootDatum(spec: AlgebraSpec, conventions: root_data.AlgebraConventions)
↳ !?root_data.RootDatum {
    return switch (spec) {
        .u1 => null,
        .simple => |simple| try root_data.RootDatum.init(simple, conventions),
    };
}

fn algebraSpecEq1(left: AlgebraSpec, right: AlgebraSpec) bool {
    return switch (left) {
        .u1 => switch (right) {
            .u1 => true,
            .simple => false,
        },
        .simple => |left_simple| switch (right) {
            .u1 => false,
            .simple => |right_simple| left_simple.family == right_simple.family and
↳ left_simple.rank == right_simple.rank,
        },
    };
}

fn validateIrrepSpec(algebra: AlgebraSpec, spec: IrrepSpec) !void {
    switch (algebra) {
        .u1 => switch (spec) {
            .u1_charge => |charge| {
                if (charge.denominator == 0) return error.InvalidU1Charge;
            },
            .dynkin => return error.InvalidIrrepForAlgebra,
        },
        .simple => |simple| switch (spec) {
            .u1_charge => return error.InvalidIrrepForAlgebra,
            .dynkin => |label| {
                if (label.len != simple.rank) return error.InvalidDynkinRank;
                for (label) |entry| {
                    if (entry < 0) return error.InvalidDynkinLabel;
                }
            },
        },
    }
}

fn u1QuadraticCasimir(charge: symmetry.U1ChargeRational) !root_data.Rational {
    const numerator = @as(i128, charge.numerator) * @as(i128, charge.numerator);
    const denominator = @as(i128, charge.denominator) * @as(i128, charge.denominator);
    return root_data.Rational.init(numerator, denominator);
}

fn irrepreLabelSpecEq1(left: IrrepLabel, right: IrrepSpec) bool {
    return switch (left) {
        .u1_charge => |left_charge| switch (right) {
            .u1_charge => |right_charge| left_charge.numerator == right_charge.numerator and
↳ left_charge.denominator == right_charge.denominator,
            .dynkin => false,
        },
        .dynkin => |left_dynkin| switch (right) {
            .u1_charge => false,
            .dynkin => |right_dynkin| std.mem.eql(i16, left_dynkin, right_dynkin),
        },
    };
}

fn cloneIrrepSpec(allocator: std.mem.Allocator, spec: IrrepSpec) !IrrepLabel {
    return switch (spec) {

```

```

        .u1_charge => |charge| .{ .u1_charge = charge },
        .dynkin => |label| .{ .dynkin = try allocator.dupe(i16, label) },
    };
}

fn freeIrrepLabel(allocator: std.mem.Allocator, label: IrrepLabel) void {
    switch (label) {
        .dynkin => |dynkin| allocator.free(dynkin),
        .u1_charge => {},
    }
}

```

7 Tensor Decomposition

A tensor-product decomposition is the compact statement

$$R \otimes S = \bigoplus_T m_T T.$$

This module stores that statement as flat product terms. It does not compute the decomposition; the family backend will do that and intern the result here.

```

const std = @import("std");
const coupling = @import("coupling.zig");
const store = @import("representation-store.zig");

/// ProductDecompositionHandle names a cached binary decomposition.
pub const ProductDecompositionHandle = struct {
    value: u32,

    /// init constructs a product-decomposition handle.
    pub fn init(value: u32) ProductDecompositionHandle {
        return .{ .value = value };
    }
};

/// ProductKey identifies a cached tensor-product decomposition.
pub const ProductKey = struct {
    left: store.IrrepHandle,
    right: store.IrrepHandle,
};

/// ProductTerm stores one channel and its multiplicity in a product.
pub const ProductTerm = struct {
    irrep: store.IrrepHandle,
    multiplicity: u16,
};

/// ProductTermSpan stores a flat range of product terms.
pub const ProductTermSpan = struct {
    offset: u32,
    len: u32,
};

/// Store owns cached binary tensor-product decomposition records.
pub const Store = struct {
    allocator: std.mem.Allocator,
    decompositions: std.ArrayList(ProductDecompositionHandle) = .empty,
    terms: std.ArrayList(ProductTerm) = .empty,
    product_index: std.AutoHashMap(u64, ProductDecompositionHandle),

    /// init constructs an empty decomposition store.
    pub fn init(allocator: std.mem.Allocator) Store {
        return .{

```

```

        .allocator = allocator,
        .product_index = std.AutoHashMap(u64, ProductDecompositionHandle).init(allocator),
    };
}

/// deinit releases decomposition-store memory.
pub fn deinit(self: *Store) void {
    self.product_index.deinit();
    self.terms.deinit(self.allocator);
    self.decompositions.deinit(self.allocator);
    self.* = Store.init(self.allocator);
}

/// findProduct returns a cached product handle if present.
pub fn findProduct(self: Store, key: ProductKey) ?ProductDecompositionHandle {
    return self.product_index.get(productKeyBits(key));
}

/// internProduct stores a binary product decomposition.
pub fn internProduct(self: *Store, key: ProductKey, terms: []const ProductTerm)
↳ !ProductDecompositionHandle {
    if (self.findProduct(key)) |handle| return handle;

    const offset: u32 = @intCast(self.terms.items.len);
    try self.terms.appendSlice(self.allocator, terms);
    errdefer self.terms.shrinkRetainingCapacity(offset);

    const handle =
    ↳ ProductDecompositionHandle.init(@intCast(self.decompositions.items.len));
    try self.decompositions.append(self.allocator, .{
        .key = key,
        .terms = .{ .offset = offset, .len = @intCast(terms.len) },
    });
    errdefer _ = self.decompositions.pop();

    try self.product_index.put(productKeyBits(key), handle);
    return handle;
}

/// productTerms returns the cached term span for a product handle.
pub fn productTerms(self: Store, handle: ProductDecompositionHandle) ?[]const ProductTerm
↳ {
    const index: usize = @intCast(handle.value);
    if (index >= self.decompositions.items.len) return null;
    const span = self.decompositions.items[index].terms;
    const start: usize = @intCast(span.offset);
    const end = start + span.len;
    return self.terms.items[start..end];
}

};

/// ProductDecomposition stores a flat range of product terms.
const ProductDecomposition = struct {
    key: ProductKey,
    terms: ProductTermSpan,
};

/// SingletCountRequest records a generic invariant-counting problem.
const SingletCountRequest = struct {
    algebra: store.AlgebraHandle,
    factors: []const store.IrrepHandle,
    target: coupling.TargetIrrep = .singlet,
};

```

```
fn productKeyBits(key: ProductKey) u64 {
    return (@as(u64, key.left.value) << 32) | @as(u64, key.right.value);
}
```

8 Tensor Realization

Realizations map abstract irreps to concrete index layouts. This is where a field acquires caller-visible slots such as vector, spinor, fundamental, or adjoint indices. The coupling search still reasons about the target irrep, so projected realizations like an irrep inside $V \otimes S$ do not force every later step to carry the full ambient product.

```
const std = @import("std");
const store = @import("representation-store.zig");
```

Public handles identify registered concrete realizations and external legs.

```
/// RealizationHandle names a concrete index realization of an irrep.
pub const RealizationHandle = struct {
    value: u32,

    /// init constructs a realization handle from a store index.
    pub fn init(value: u32) RealizationHandle {
        return .{ .value = value };
    }
};

/// ExternalLegHandle names one external leg in an invariant request.
pub const ExternalLegHandle = struct {
    value: u32,

    /// init constructs an external-leg handle from a store index.
    pub fn init(value: u32) ExternalLegHandle {
        return .{ .value = value };
    }
};
```

Construction records are public because CFT code must be able to describe its free indices. The arrays are caller-owned until they are registered in the context.

```
/// RealizationRef identifies a primitive or registered external tensor shape.
pub const RealizationRef = union(enum) {
    primitive_irrep: store.IrrepHandle,
    handle: RealizationHandle,
    registered_name: []const u8,
};

/// IndexKind classifies the visible slot type of an external index.
pub const IndexKind = enum {
    vector,
    spinor,
    conjugate_spinor,
    fundamental,
    anti_fundamental,
    adjoint,
    form,
    custom,
};

/// NamedIndex records one caller-supplied free index name.
pub const NamedIndex = struct {
    kind: IndexKind,
    name: []const u8,
```

```

    /// init constructs one caller-named index slot.
    pub fn init(kind: IndexKind, name: []const u8) NamedIndex {
        return .{ .kind = kind, .name = name };
    }
};

/// RealizationChannel selects one irrep copy inside a realization product.
pub const RealizationChannel = struct {
    irrep: store.IrrepHandle,
    copy: u16 = 0,

    /// init constructs a selected irrep copy.
    pub fn init(irrep: store.IrrepHandle, copy: u16) RealizationChannel {
        return .{ .irrep = irrep, .copy = copy };
    }

    /// first selects the first copy of an irrep.
    pub fn first(irrep: store.IrrepHandle) RealizationChannel {
        return .{ .irrep = irrep };
    }
};

/// ExternalLeg describes a public invariant leg with named free indices.
pub const ExternalLeg = struct {
    realization: RealizationRef,
    indices: []const NamedIndex,
    label: ?[]const u8 = null,

    /// primitive constructs a leg whose realization is the irrep itself.
    pub fn primitive(irrep: store.IrrepHandle, indices: []const NamedIndex) ExternalLeg {
        return .{
            .realization = .{ .primitive_irrep = irrep },
            .indices = indices,
        };
    }

    /// realized constructs a leg from a registered realization handle.
    pub fn realized(realization: RealizationHandle, indices: []const NamedIndex) ExternalLeg {
        return .{
            .realization = .{ .handle = realization },
            .indices = indices,
        };
    }

    /// registered constructs a leg from a preset realization name.
    pub fn registered(name: []const u8, indices: []const NamedIndex) ExternalLeg {
        return .{
            .realization = .{ .registered_name = name },
            .indices = indices,
        };
    }

    /// named returns the leg with a caller-facing label attached.
    pub fn named(self: ExternalLeg, label: []const u8) ExternalLeg {
        var out = self;
        out.label = label;
        return out;
    }
};

/// RealizationSpec describes a primitive or projected external index model.
pub const RealizationSpec = struct {
    channel: RealizationChannel,

```

```

ambient: []const store.IrrepHandle = &.{},
indices: []const NamedIndex,
name: ?[]const u8 = null,

/// primitive constructs a direct index realization of an irrep.
pub fn primitive(target: store.IrrepHandle, indices: []const NamedIndex) RealizationSpec {
    return .{
        .channel = RealizationChannel.first(target),
        .indices = indices,
    };
}

/// projected constructs an irrep realization inside an ambient product.
pub fn projected(target: store.IrrepHandle, ambient: []const store.IrrepHandle, indices:
↳ []const NamedIndex, copy: u16) RealizationSpec {
    return RealizationSpec.projectedChannel(RealizationChannel.init(target, copy),
↳ ambient, indices);
}

/// projectedChannel constructs a realization from an explicit channel.
pub fn projectedChannel(channel: RealizationChannel, ambient: []const store.IrrepHandle,
↳ indices: []const NamedIndex) RealizationSpec {
    return .{
        .channel = channel,
        .ambient = ambient,
        .indices = indices,
    };
}

/// named returns the realization spec with a preset-facing name attached.
pub fn named(self: RealizationSpec, name: []const u8) RealizationSpec {
    var out = self;
    out.name = name;
    return out;
}

/// targetIrrep returns the abstract irrep realized by this index model.
pub fn targetIrrep(self: RealizationSpec) store.IrrepHandle {
    return self.channel.irrep;
}
};

```

The store owns cloned realization specs and interns repeated requests.

```

/// Store owns interned concrete realization records.
pub const Store = struct {
    allocator: std.mem.Allocator,
    specs: std.ArrayList(RealizationSpec) = .empty,

    /// init constructs an empty realization store.
    pub fn init(allocator: std.mem.Allocator) Store {
        return .{ .allocator = allocator };
    }

    /// deinit releases all realization-store memory.
    pub fn deinit(self: *Store) void {
        for (self.specs.items) |spec| {
            freeRealizationSpec(self.allocator, spec);
        }
        self.specs.deinit(self.allocator);
        self.* = Store.init(self.allocator);
    }

    /// internRealization returns the stable handle for a realization spec.

```

```

pub fn internRealization(self: *Store, spec: RealizationSpec) !RealizationHandle {
    for (self.specs.items, 0..) |stored, index| {
        if (realizationSpecEqL(stored, spec)) {
            return RealizationHandle.init(@intCast(index));
        }
    }

    const owned = try cloneRealizationSpec(self.allocator, spec);
    errdefer freeRealizationSpec(self.allocator, owned);
    try self.specs.append(self.allocator, owned);
    return RealizationHandle.init(@intCast(self.specs.items.len - 1));
}

/// containsRealization returns whether a realization handle belongs to this store.
pub fn containsRealization(self: Store, handle: RealizationHandle) bool {
    return self.realizationSpec(handle) != null;
}

/// targetIrrep returns the irrep realized by a stored realization handle.
pub fn targetIrrep(self: Store, handle: RealizationHandle) ?store.IrrepHandle {
    const spec = self.realizationSpec(handle) orelse return null;
    return spec.targetIrrep();
}

/// realizationSpecFor returns the stored realization specification for a handle.
pub fn realizationSpecFor(self: Store, handle: RealizationHandle) ?RealizationSpec {
    return self.realizationSpec(handle);
}

fn realizationSpec(self: Store, handle: RealizationHandle) ?RealizationSpec {
    const index: usize = @intCast(handle.value);
    if (index >= self.specs.items.len) return null;
    return self.specs.items[index];
}
};

```

These private records describe the future projector derivation path for a realization. The selected channel is a single value, so future embedding conventions have one natural place to attach.

```

/// IndexSlotSpec declares one slot in a realization layout.
const IndexSlotSpec = struct {
    kind: IndexKind,
    default_name: ?[]const u8 = null,
};

/// RealizationRequest declares an irrep as a projected ambient tensor shape.
const RealizationRequest = struct {
    channel: RealizationChannel,
    ambient: []const store.IrrepHandle,
    index_layout: []const IndexSlotSpec,
};

/// RealizationKind distinguishes primitive from projected realizations.
const RealizationKind = enum {
    primitive,
    projected_channel,
};

/// RealizationRecord stores one concrete index layout for an irrep.
const RealizationRecord = struct {
    channel: RealizationChannel,
    kind: RealizationKind,
    ambient_offset: u32,
};

```

```

    ambient_len: u16,
    index_layout_offset: u32,
    index_layout_len: u16,
    projector: ?RealizationProjectorId = null,
};

/// RealizationProjectorId names a derived boundary realization projector.
const RealizationProjectorId = u32;

/// AmbientFactor records one irrep in an ambient realization product.
const AmbientFactor = struct {
    irrep: store.IrrepHandle,
};

/// RealizationRule records the construction request for one realization.
const RealizationRule = struct {
    request: RealizationRequest,
    projector: ?RealizationProjectorId = null,
};

```

Structural equality and clone/free helpers are private implementation details. They keep public construction records lightweight while making registered realizations stable after caller buffers go out of scope.

```

fn realizationSpecEq(left: RealizationSpec, right: RealizationSpec) bool {
    return realizationChannelEq(left.channel, right.channel) and
        optionalStringEq(left.name, right.name) and
        irrepSliceEq(left.ambient, right.ambient) and
        namedIndexSliceEq(left.indices, right.indices);
}

fn realizationChannelEq(left: RealizationChannel, right: RealizationChannel) bool {
    return left.irrep.value == right.irrep.value and left.copy == right.copy;
}

fn irrepSliceEq(left: []const store.IrrepHandle, right: []const store.IrrepHandle) bool {
    if (left.len != right.len) return false;
    for (left, right) |left_item, right_item| {
        if (left_item.value != right_item.value) return false;
    }
    return true;
}

fn namedIndexSliceEq(left: []const NamedIndex, right: []const NamedIndex) bool {
    if (left.len != right.len) return false;
    for (left, right) |left_item, right_item| {
        if (left_item.kind != right_item.kind) return false;
        if (!std.mem.eql(u8, left_item.name, right_item.name)) return false;
    }
    return true;
}

fn optionalStringEq(left: ?[]const u8, right: ?[]const u8) bool {
    if (left == null or right == null) return left == null and right == null;
    return std.mem.eql(u8, left.?, right.?);
}

fn cloneRealizationSpec(allocator: std.mem.Allocator, spec: RealizationSpec) !RealizationSpec
↪ {
    const ambient = try allocator.dupe(store.IrrepHandle, spec.ambient);
    errdefer allocator.free(ambient);

    const indices = try cloneNamedIndices(allocator, spec.indices);
    errdefer freeNamedIndices(allocator, indices);
}

```



```

const name = if (spec.name) |value| try allocator.dupe(u8, value) else null;
errdefer if (name) |value| allocator.free(value);

return .{
    .channel = spec.channel,
    .ambient = ambient,
    .indices = indices,
    .name = name,
};
}

fn freeRealizationSpec(allocator: std.mem.Allocator, spec: RealizationSpec) void {
    allocator.free(spec.ambient);
    freeNamedIndices(allocator, spec.indices);
    if (spec.name) |name| allocator.free(name);
}

fn cloneNamedIndices(allocator: std.mem.Allocator, indices: []const NamedIndex) ![]NamedIndex
↪ {
    const owned = try allocator.alloc(NamedIndex, indices.len);
    var initialized: usize = 0;
    errdefer {
        for (owned[0..initialized]) |previous| allocator.free(previous.name);
        allocator.free(owned);
    }

    for (indices, owned[0..]) |source, *target| {
        const name = try allocator.dupe(u8, source.name);
        target.* = .{ .kind = source.kind, .name = name };
        initialized += 1;
    }

    return owned;
}

fn freeNamedIndices(allocator: std.mem.Allocator, indices: []const NamedIndex) void {
    for (indices) |index| {
        allocator.free(index.name);
    }
    allocator.free(indices);
}

```

9 Tensor Projector

A projector is an idempotent intertwiner. It can select a channel in $R \otimes S$, impose a boundary realization such as a gamma-traceless subspace, or change between invariant-basis conventions. This module stores projector identities and derivation labels; it does not construct formulas.

```

const std = @import("std");
const realization = @import("realization.zig");
const rendering = @import("rendering.zig");
const store = @import("representation-store.zig");

/// ProjectorId names one local projector or Clebsch-Gordan kernel.
pub const ProjectorId = u32;

/// ProjectorConvention records configurable projector-rendering choices.
pub const ProjectorConvention = struct {
    render_mode: rendering.ProjectorRenderMode = .named,
    signature: rendering.SignatureConvention = .complex,
    normalization_id: u32 = 0,
};

```

```

/// ProjectorRole records what mathematical object a projector selects.
pub const ProjectorRole = union(enum) {
    product_channel: ProductChannelKey,
    boundary_realization: realization.RealizationHandle,
    basis_change: BasisChangeKey,
};

/// ProductChannelKey identifies one target channel inside a binary product.
pub const ProductChannelKey = struct {
    left: store.IrrepHandle,
    right: store.IrrepHandle,
    output: store.IrrepHandle,
    multiplicity_copy: u16,
};

/// ProjectorExpansionSpan names cached local expansion terms.
pub const ProjectorExpansionSpan = struct {
    offset: u32,
    len: u32,
};

/// ProjectorExpansionCounters stores cache-hit evidence for local expansions.
pub const ProjectorExpansionCounters = struct {
    named_hits: u64 = 0,
    named_misses: u64 = 0,
    formula_hits: u64 = 0,
    formula_misses: u64 = 0,
};

/// ProjectorDescriptor exposes read-only metadata for a projector id.
pub const ProjectorDescriptor = struct {
    role: ProjectorRole,
    convention: ProjectorConvention,
    derivation: ProjectorDerivation,
};

/// BasisChangeKey identifies a change of invariant-basis convention.
pub const BasisChangeKey = struct {
    source_basis: u32,
    target_basis: u32,
};

/// Store owns projector identities and compact emitted-term spans.
pub const Store = struct {
    allocator: std.mem.Allocator,
    records: std.ArrayList(ProjectorRecord) = .empty,
    terms: std.ArrayList(ProjectorExpansionTermRecord) = .empty,
    atoms: std.ArrayList(rendering.SymbolicAtom) = .empty,
    counters: ProjectorExpansionCounters = .{},

    /// init constructs an empty projector store.
    pub fn init(allocator: std.mem.Allocator) Store {
        return .{ .allocator = allocator };
    }

    /// deinit releases projector-store memory.
    pub fn deinit(self: *Store) void {
        self.atoms.deinit(self.allocator);
        self.terms.deinit(self.allocator);
        self.records.deinit(self.allocator);
        self.* = Store.init(self.allocator);
    }
};

```

```

/// internProjector returns a stable id for a projector identity.
pub fn internProjector(self: *Store, key: ProjectorKey, derivation: ProjectorDerivation)
↳ !ProjectorId {
    for (self.records.items, 0..) |record, index| {
        if (projectorKeyEq(record.key, key)) return @intCast(index);
    }
    try self.records.append(self allocator, .{
        .key = key,
        .derivation = derivation,
        .term_offset = 0,
        .term_len = 0,
        .formula_cached = false,
    });
    return @intCast(self.records.items.len - 1);
}

/// projectorDerivation returns compact derivation metadata for an id.
pub fn projectorDerivation(self: Store, id: ProjectorId) ?ProjectorDerivation {
    const index: usize = @intCast(id);
    if (index >= self.records.items.len) return null;
    return self.records.items[index].derivation;
}

/// projectorDescriptor returns read-only metadata for a projector id.
pub fn projectorDescriptor(self: Store, id: ProjectorId) ?ProjectorDescriptor {
    const index: usize = @intCast(id);
    if (index >= self.records.items.len) return null;
    const record = self.records.items[index];
    return .{
        .role = record.key.role,
        .convention = record.key.convention,
        .derivation = record.derivation,
    };
}

/// expansionStatus returns the current expansion state for an id.
pub fn expansionStatus(self: Store, id: ProjectorId) ?ExpansionStatus {
    return if (self.projectorDerivation(id)) |derivation| derivation.status else null;
}

/// canExpandToTerms reports whether an id has a route to streamed terms.
pub fn canExpandToTerms(self: Store, id: ProjectorId) bool {
    return self.expansionStatus(id) == .expandable_terms;
}

/// canRenderMode reports whether a projector can satisfy a render mode.
pub fn canRenderMode(self: Store, id: ProjectorId, mode: rendering.ProjectorRenderMode)
↳ bool {
    const status = self.expansionStatus(id) orelse return false;
    return expansionStatusCovers(status, mode);
}

/// ensureNamedExpansion returns a cached one-term named expansion.
pub fn ensureNamedExpansion(self: *Store, id: ProjectorId) !ProjectorExpansionSpan {
    const index: usize = @intCast(id);
    if (index >= self.records.items.len) return error.UnknownProjector;
    if (!self.canRenderMode(id, .named)) return error.ProjectorExpansionNotImplemented;

    const record = &self.records.items[index];
    if (record.term_len != 0) {
        self.counters.named_hits += 1;
        return .{ .offset = record.term_offset, .len = record.term_len };
    }
    self.counters.named_misses += 1;
}

```

```

const atom_offset: u32 = @intCast(self.atoms.items.len);
try self.atoms.append(self.allocator, .{ .named_operator = .{
    .kind = .projector,
    .operator_id = id,
    .first_index = 0,
    .index_len = 0,
} });
errdefer _ = self.atoms.pop();

const term_offset: u32 = @intCast(self.terms.items.len);
try self.terms.append(self.allocator, .{
    .coefficient = rendering.rationalOne(),
    .atom_offset = atom_offset,
    .atom_len = 1,
});
errdefer _ = self.terms.pop();

record.term_offset = term_offset;
record.term_len = 1;
return .{ .offset = term_offset, .len = 1 };
}

/// ensureFormulaProgram records the cached executable formula route for an id.
pub fn ensureFormulaProgram(self: *Store, id: ProjectorId) !ProjectorFormulaKind {
    const index: usize = @intCast(id);
    if (index >= self.records.items.len) return error.UnknownProjector;
    if (!self.canRenderMode(id, .expanded_terms)) return
        ↪ error.ProjectorExpansionNotImplemented;

    const record = &self.records.items[index];
    if (record.derivation.formula_kind == .named_only) return
        ↪ error.ProjectorExpansionNotImplemented;
    if (record.formula_cached) {
        self.counters.formula_hits += 1;
    } else {
        self.counters.formula_misses += 1;
        record.formula_cached = true;
    }
    return record.derivation.formula_kind;
}

/// expansionTerm returns one borrowed cached expansion term.
pub fn expansionTerm(self: Store, span: ProjectorExpansionSpan, term_index: u32)
    ↪ ?rendering.SymbolicTerm {
    if (term_index >= span.len) return null;
    const index: usize = @intCast(span.offset + term_index);
    if (index >= self.terms.items.len) return null;
    const record = self.terms.items[index];
    const start: usize = @intCast(record.atom_offset);
    const end = start + record.atom_len;
    if (end > self.atoms.items.len) return null;
    return .{
        .coefficient = record.coefficient,
        .atoms = self.atoms.items[start..end],
    };
}

/// expansionCounters returns cache-hit counters for local expansion data.
pub fn expansionCounters(self: Store) ProjectorExpansionCounters {
    return self.counters;
}
};

```

```

/// ProjectorKey identifies one projector in one convention.
pub const ProjectorKey = struct {
    role: ProjectorRole,
    convention: ProjectorConvention,
};

/// ProjectorKind records how a local projector was derived.
pub const ProjectorKind = enum {
    identity,
    quadratic_casimir,
    higher_casimir_signature,
    local_idempotent_split,
    highest_weight_solver,
    backend_specific_verified,
};

/// ExpansionSource records the stored route used for future expansion.
pub const ExpansionSource = enum {
    none,
    casimir_polynomial,
    idempotent_split,
    highest_weight_solver,
    backend_specific_verified,
};

/// ExpansionStatus records how far a projector can currently be expanded.
pub const ExpansionStatus = enum {
    not_requested,
    expandable_named,
    expandable_blocks,
    expandable_terms,
    blocked_missing_formula,
    blocked_unverified_identity,
};

/// ProjectorFormulaKind records the executable local formula route.
pub const ProjectorFormulaKind = enum {
    named_only,
    identity,
    orthogonal_vector_metric,
    orthogonal_spinor_pair,
    orthogonal_gamma_trace,
    orthogonal_gamma_traceless,
    orthogonal_spinor_form_channel,
    orthogonal_form_pair,
    orthogonal_form_spinor_channel,
    cartan_product_channel,
    orthogonal_structural_projection,
    orthogonal_spinor_tower_form_channel,
    orthogonal_spinor_tower_middle_form_channel,
    orthogonal_spinor_tower_opposite_form_channel,
    orthogonal_spinor_tower_opposite_lower_channel,
    orthogonal_tensor_spinor_tower_channel,
    orthogonal_tensor_spinor_form_power_channel,
    orthogonal_tensor_spinor_tower_raise_channel,
    orthogonal_tensor_spinor_opposite_tower_lower_channel,
    orthogonal_tensor_spinor_opposite_shift_channel,
    orthogonal_tensor_spinor_opposite_all_shift_channel,
    orthogonal_tensor_spinor_opposite_form_add_channel,
    orthogonal_tensor_spinor_opposite_rank_split_channel,
    orthogonal_tensor_spinor_shift_channel,
    orthogonal_tensor_spinor_rank_wrap_shift_channel,
    orthogonal_tensor_spinor_rank_split_channel,
    orthogonal_tensor_spinor_terminal_channel,

```

```

    orthogonal_tensor_spinor_opposite_terminal_channel,
    orthogonal_tensor_spinor_opposite_form_add_terminal_channel,
    orthogonal_tensor_spinor_opposite_form_add_shift_terminal_channel,
    orthogonal_tensor_spinor_opposite_rank_split_terminal_channel,
    orthogonal_tensor_spinor_opposite_shift_terminal_channel,
    orthogonal_tensor_spinor_rank_wrap_terminal_channel,
    orthogonal_tensor_spinor_rank_split_terminal_channel,
    orthogonal_tensor_spinor_form_terminal_channel,
    orthogonal_tensor_spinor_form_add_terminal_channel,
    orthogonal_tensor_spinor_form_power_terminal_channel,
    orthogonal_tensor_spinor_form_tower_channel,
    orthogonal_tensor_spinor_form_tower_terminal_channel,
    orthogonal_tensor_spinor_form_tower_remove_shift_channel,
    orthogonal_tensor_spinor_form_tower_shift_down_channel,
    orthogonal_tensor_spinor_form_tower_all_shift_down_channel,
    orthogonal_tensor_spinor_opposite_form_tower_channel,
    orthogonal_tensor_spinor_opposite_form_tower_terminal_channel,
    orthogonal_tensor_spinor_opposite_form_tower_merge_channel,
    orthogonal_tensor_spinor_opposite_form_tower_remove_shift_channel,
    orthogonal_tensor_spinor_opposite_form_tower_shift_down_channel,
    orthogonal_tensor_form_spinor_channel,
    orthogonal_tensor_form_spinor_preserve_channel,
    orthogonal_tensor_form_spinor_shift_channel,
    orthogonal_tensor_form_spinor_remove_shift_down_channel,
    orthogonal_tensor_form_spinor_shift_down_channel,
    orthogonal_tensor_form_spinor_shift_down_two_channel,
    orthogonal_tensor_form_spinor_mixed_shift_down_channel,
    orthogonal_tensor_form_spinor_all_shift_down_channel,
    orthogonal_tensor_form_spinor_shift_down_any_channel,
    orthogonal_tensor_spinor_middle_form_channel,
    backend_specific_structure,
};

/// ProjectorFormulaAudit records symbolic checks passed by a formula route.
pub const ProjectorFormulaAudit = struct {
    normalized: bool = false,
    idempotent: bool = false,
    channel_separated: bool = false,
    gamma_traceless: bool = false,

    /// verified reports whether all required projector-law checks passed.
    pub fn verified(self: ProjectorFormulaAudit) bool {
        return self.normalized and self.idempotent and self.channel_separated;
    }
};

/// ProjectorDerivation stores the construction route and expansion state.
pub const ProjectorDerivation = struct {
    kind: ProjectorKind,
    source: ExpansionSource,
    status: ExpansionStatus,
    formula_kind: ProjectorFormulaKind = .named_only,
    formula_audit: ProjectorFormulaAudit = .{},
};

/// expansionStatusCovers reports whether stored expansion data is sufficient.
pub fn expansionStatusCovers(status: ExpansionStatus, mode: rendering.ProjectorRenderMode)
    → bool {
    return switch (mode) {
        .named => status == .expandable_named or status == .expandable_blocks or status ==
            → .expandable_terms,
        .expanded_blocks => status == .expandable_blocks or status == .expandable_terms,
        .expanded_terms => status == .expandable_terms,
    };
};

```

```

}

/// ProjectorRecord stores one local kernel and its derivation kind.
const ProjectorRecord = struct {
    key: ProjectorKey,
    derivation: ProjectorDerivation,
    term_offset: u32,
    term_len: u32,
    formula_cached: bool,
};

const ProjectorExpansionTermRecord = struct {
    coefficient: rendering.RationalId,
    atom_offset: u32,
    atom_len: u16,
};

/// ProjectorAudit stores local splitting diagnostics for feasibility studies.
const ProjectorAudit = struct {
    product: ProductChannelKey,
    channel_count: u32,
    quadratic_collision_count: u32,
    true_multiplicity_count: u32,
    residual_block_max: u32,
};

fn projectorKeyEq(left: ProjectorKey, right: ProjectorKey) bool {
    return projectorRoleEq(left.role, right.role) and
        left.convention.render_mode == right.convention.render_mode and
        left.convention.signature == right.convention.signature and
        left.convention.normalization_id == right.convention.normalization_id;
}

fn projectorRoleEq(left: ProjectorRole, right: ProjectorRole) bool {
    return switch (left) {
        .product_channel => |left_key| switch (right) {
            .product_channel => |right_key| left_key.left.value == right_key.left.value and
                left_key.right.value == right_key.right.value and
                left_key.output.value == right_key.output.value and
                left_key.multiplicity_copy == right_key.multiplicity_copy,
            else => false,
        },
        .boundary_realization => |left_handle| switch (right) {
            .boundary_realization => |right_handle| left_handle.value == right_handle.value,
            else => false,
        },
        .basis_change => |left_key| switch (right) {
            .basis_change => |right_key| left_key.source_basis == right_key.source_basis and
                left_key.target_basis == right_key.target_basis,
            else => false,
        },
    };
}

```

10 Tensor Coupling

Coupling owns invariant-basis requests and, later, the compact paths that span those bases. It does not own dense tensor values. A basis element should be a small recipe: which external legs are coupled, which intermediate channels are used, and which local projector kernels render the final invariant if a caller asks for explicit components.

```

const std = @import("std");
const projector = @import("projector.zig");
const realization = @import("realization.zig");
const store = @import("representation-store.zig");

```

Basis and invariant handles are public ids. The path records behind them stay private because we want freedom to change the solver layout.

```

/// BasisHandle names a generated invariant basis.
pub const BasisHandle = struct {
    value: u32,

    /// init constructs a basis handle from a store index.
    pub fn init(value: u32) BasisHandle {
        return .{ .value = value };
    }
};

/// InvariantHandle names one invariant inside a basis.
pub const InvariantHandle = struct {
    value: u32,

    /// init constructs an invariant handle from a store index.
    pub fn init(value: u32) InvariantHandle {
        return .{ .value = value };
    }
};

```

The public request is deliberately small: algebra, external legs, target, and basis policy. The default target is the singlet because that is the CFT workflow for invariant tensor structures.

```

/// TargetIrrep selects the target representation, usually the singlet.
pub const TargetIrrep = union(enum) {
    singlet,
    irrep: store.IrrepHandle,
};

/// TreePolicy controls the coupling tree used for a basis.
pub const TreePolicy = union(enum) {
    auto,
    fixed: []const u16,
};

/// BasisPolicy controls the public basis convention.
pub const BasisPolicy = enum {
    default,
    coupling_paths,
    channel_adapted,
};

/// InvariantBasisRequest is the CFT-facing request for invariant tensors.
pub const InvariantBasisRequest = struct {
    algebra: store.AlgebraHandle,
    external_legs: []const realization.ExternalLeg,
    target: TargetIrrep = .singlet,
    tree_policy: TreePolicy = .auto,
    basis_policy: BasisPolicy = .default,
};

/// TensorExpansionTerm stores a CFT coefficient attached to one invariant id.
pub const TensorExpansionTerm = struct {
    rational_function_id: u32,
    invariant: InvariantHandle,
};

```



```

    scalar_id: u32,
};

/// CouplingStep stores one local channel choice in the canonical tree.
pub const CouplingStep = struct {
    left: store.IrrepHandle,
    right: store.IrrepHandle,
    output: store.IrrepHandle,
    multiplicity_copy: u16,
    projector: projector.ProjectorId,
};

/// CouplingPath stores a flat span of local coupling steps.
pub const CouplingPath = struct {
    step_offset: u32,
    step_len: u16,
};

/// CouplingPathSpan stores paths belonging to one basis handle.
pub const CouplingPathSpan = struct {
    offset: u32,
    len: u32,
};

/// BasisAudit stores compact evidence for one generated path basis.
pub const BasisAudit = struct {
    path_count: u128 = 0,
    stored_path_count: u32 = 0,
    step_count: u32 = 0,
    /// distinct_projector_count counts local coupling-step projectors.
    distinct_projector_count: u32 = 0,
    boundary_projector_count: u32 = 0,
    total_projector_count: u32 = 0,
    max_path_step_len: u16 = 0,
};

```

The store interns basis requests. This is not invariant generation yet; it is the stable ownership layer the generator will consume. The important invariant is that caller-owned slices can disappear after registration.

```

/// Store owns interned invariant-basis requests.
pub const Store = struct {
    allocator: std.mem.Allocator,
    requests: std.ArrayList(InvariantBasisRequest) = .empty,
    path_counts: std.ArrayList(u128) = .empty,
    basis_audits: std.ArrayList(BasisAudit) = .empty,
    basis_paths: std.ArrayList(CouplingPathSpan) = .empty,
    paths: std.ArrayList(CouplingPath) = .empty,
    steps: std.ArrayList(CouplingStep) = .empty,

    /// init constructs an empty coupling store.
    pub fn init(allocator: std.mem.Allocator) Store {
        return .{ .allocator = allocator };
    }

    /// deinit releases all coupling-store memory.
    pub fn deinit(self: *Store) void {
        for (self.requests.items) |request| {
            freeRequest(self.allocator, request);
        }
        self.steps.deinit(self.allocator);
        self.paths.deinit(self.allocator);
        self.basis_paths.deinit(self.allocator);
        self.basis_audits.deinit(self.allocator);
    }
};

```

```

        self.path_counts.deinit(self.allocator);
        self.requests.deinit(self.allocator);
        self.* = Store.init(self.allocator);
    }

    /// internBasisRequest returns a stable basis handle for a request.
    pub fn internBasisRequest(self: *Store, request: InvariantBasisRequest) !BasisHandle {
        return self.internBasisRequestWithCount(request, 0);
    }

    /// internBasisRequestWithCount stores a basis request and its path-count audit.
    pub fn internBasisRequestWithCount(self: *Store, request: InvariantBasisRequest,
        ↪ path_count: u128) !BasisHandle {
        return self.internBasisRequestWithPaths(request, path_count, &.{}, &.{}, 0);
    }

    /// internBasisRequestWithPaths stores a basis request and compact path records.
    pub fn internBasisRequestWithPaths(self: *Store, request: InvariantBasisRequest,
        ↪ path_count: u128, paths: []const CouplingPath, steps: []const CouplingStep,
        ↪ boundary_projector_count: u32) !BasisHandle {
        for (self.requests.items, 0..) |stored, index| {
            if (requestEq(stored, request)) {
                self.path_counts.items[index] = path_count;
                self.basis_audits.items[index] = try makeBasisAudit(self.allocator,
                    ↪ path_count, paths, steps, boundary_projector_count);
                return BasisHandle.init(@intCast(index));
            }
        }

        const audit = try makeBasisAudit(self.allocator, path_count, paths, steps,
            ↪ boundary_projector_count);
        const owned = try cloneRequest(self.allocator, request);
        errdefer freeRequest(self.allocator, owned);
        try self.requests.append(self.allocator, owned);
        errdefer _ = self.requests.pop();
        try self.path_counts.append(self.allocator, path_count);
        errdefer _ = self.path_counts.pop();
        try self.basis_audits.append(self.allocator, audit);
        errdefer _ = self.basis_audits.pop();

        const step_offset: u32 = @intCast(self.steps.items.len);
        try self.steps.appendSlice(self.allocator, steps);
        errdefer self.steps.shrinkRetainingCapacity(step_offset);

        const path_offset: u32 = @intCast(self.paths.items.len);
        for (paths) |path| {
            try self.paths.append(self.allocator, .{
                .step_offset = step_offset + path.step_offset,
                .step_len = path.step_len,
            });
        }
        errdefer self.paths.shrinkRetainingCapacity(path_offset);

        try self.basis_paths.append(self.allocator, .{
            .offset = path_offset,
            .len = @intCast(paths.len),
        });
        errdefer _ = self.basis_paths.pop();
        return BasisHandle.init(@intCast(self.requests.items.len - 1));
    }

    /// basisPathCount returns the audited path count for a basis handle.
    pub fn basisPathCount(self: Store, handle: BasisHandle) ?u128 {
        const index: usize = @intCast(handle.value);

```

```

        if (index >= self.path_counts.items.len) return null;
        return self.path_counts.items[index];
    }

    /// basisAudit returns compact generation counters for a basis handle.
    pub fn basisAudit(self: Store, handle: BasisHandle) ?BasisAudit {
        const index: usize = @intCast(handle.value);
        if (index >= self.basis_audits.items.len) return null;
        return self.basis_audits.items[index];
    }

    /// basisRequest returns the stored request for a basis handle.
    pub fn basisRequest(self: Store, handle: BasisHandle) ?InvariantBasisRequest {
        const index: usize = @intCast(handle.value);
        if (index >= self.requests.items.len) return null;
        return self.requests.items[index];
    }

    /// basisPathSpan returns the stored path span for a basis handle.
    pub fn basisPathSpan(self: Store, handle: BasisHandle) ?CouplingPathSpan {
        const index: usize = @intCast(handle.value);
        if (index >= self.basis_paths.items.len) return null;
        return self.basis_paths.items[index];
    }

    /// basisPaths returns the compact paths for a basis handle.
    pub fn basisPaths(self: Store, handle: BasisHandle) ?[]const CouplingPath {
        const span = self.basisPathSpan(handle) orelse return null;
        const start: usize = @intCast(span.offset);
        const end = start + span.len;
        return self.paths.items[start..end];
    }

    /// pathSteps returns the local steps for one compact path.
    pub fn pathSteps(self: Store, path: CouplingPath) []const CouplingStep {
        const start: usize = @intCast(path.step_offset);
        const end = start + path.step_len;
        return self.steps.items[start..end];
    }
};

```

/ BasisRecord stores one generated invariant basis. const BasisRecord = struct { request: InvariantBasisRequest, path_{offset}: u32, path_{len}: u32, };

/ ReachabilityRecord stores pruning data for one coupling tree node. const ReachabilityRecord = struct { node: u32, irrep_{offset}: u32, irrep_{len}: u32, }; #+end_{src}

Request equality is structural. This makes repeated high-level calls cheap and deterministic before the expensive solver exists.

```

fn requestEqL(left: InvariantBasisRequest, right: InvariantBasisRequest) bool {
    return left.algebra.value == right.algebra.value and
        targetEqL(left.target, right.target) and
        treePolicyEqL(left.tree_policy, right.tree_policy) and
        left.basis_policy == right.basis_policy and
        externalLegsEqL(left.external_legs, right.external_legs);
}

fn targetEqL(left: TargetIrrep, right: TargetIrrep) bool {
    return switch (left) {
        .singlet => switch (right) {
            .singlet => true,
            .irrep => false,
        },
        .irrep => |left_irrep| switch (right) {

```

```

        .singlet => false,
        .irrep => |right_irrep| left_irrep.value == right_irrep.value,
    },
};
}

fn treePolicyEq(left: TreePolicy, right: TreePolicy) bool {
    return switch (left) {
        .auto => switch (right) {
            .auto => true,
            .fixed => false,
        },
        .fixed => |left_fixed| switch (right) {
            .auto => false,
            .fixed => |right_fixed| std.mem.eql(u16, left_fixed, right_fixed),
        },
    };
}

fn externalLegsEq(left: []const realization.ExternalLeg, right: []const
→ realization.ExternalLeg) bool {
    if (left.len != right.len) return false;
    for (left, right) |left_leg, right_leg| {
        if (!externalLegEq(left_leg, right_leg)) return false;
    }
    return true;
}

fn externalLegEq(left: realization.ExternalLeg, right: realization.ExternalLeg) bool {
    return realizationRefEq(left.realization, right.realization) and
        namedIndicesEq(left.indices, right.indices) and
        optionalStringEq(left.label, right.label);
}

fn realizationRefEq(left: realization.RealizationRef, right: realization.RealizationRef) bool
→ {
    return switch (left) {
        .primitive_irrep => |left_irrep| switch (right) {
            .primitive_irrep => |right_irrep| left_irrep.value == right_irrep.value,
            else => false,
        },
        .handle => |left_handle| switch (right) {
            .handle => |right_handle| left_handle.value == right_handle.value,
            else => false,
        },
        .registered_name => |left_name| switch (right) {
            .registered_name => |right_name| std.mem.eql(u8, left_name, right_name),
            else => false,
        },
    };
}

fn namedIndicesEq(left: []const realization.NamedIndex, right: []const
→ realization.NamedIndex) bool {
    if (left.len != right.len) return false;
    for (left, right) |left_index, right_index| {
        if (left_index.kind != right_index.kind) return false;
        if (!std.mem.eql(u8, left_index.name, right_index.name)) return false;
    }
    return true;
}

fn optionalStringEq(left: ?[]const u8, right: ?[]const u8) bool {
    if (left == null or right == null) return left == null and right == null;

```

```

    return std.mem.eql(u8, left.?, right.?);
}

fn makeBasisAudit(allocator: std.mem.Allocator, path_count: u128, paths: []const CouplingPath,
↳ steps: []const CouplingStep, boundary_projector_count: u32) !BasisAudit {
    var projectors = std.AutoHashMap(projector.ProjectorId, void).init(allocator);
    defer projectors.deinit();
    for (steps) |step| {
        try projectors.put(step.projector, {});
    }

    var max_path_step_len: u16 = 0;
    for (paths) |path| {
        if (path.step_len > max_path_step_len) max_path_step_len = path.step_len;
    }

    return .{
        .path_count = path_count,
        .stored_path_count = @intCast(paths.len),
        .step_count = @intCast(steps.len),
        .distinct_projector_count = @intCast(projectors.count()),
        .boundary_projector_count = boundary_projector_count,
        .total_projector_count = @as(u32, @intCast(projectors.count())) +
↳ boundary_projector_count,
        .max_path_step_len = max_path_step_len,
    };
}

```

The clone/free helpers keep ownership local to this module. Later, generated path data can be added beside requests without changing the public request shape.

```

fn cloneRequest(allocator: std.mem.Allocator, request: InvariantBasisRequest)
↳ !InvariantBasisRequest {
    const external_legs = try cloneExternalLegs(allocator, request.external_legs);
    errdefer freeExternalLegs(allocator, external_legs);

    return .{
        .algebra = request.algebra,
        .external_legs = external_legs,
        .target = request.target,
        .tree_policy = try cloneTreePolicy(allocator, request.tree_policy),
        .basis_policy = request.basis_policy,
    };
}

fn freeRequest(allocator: std.mem.Allocator, request: InvariantBasisRequest) void {
    freeExternalLegs(allocator, request.external_legs);
    switch (request.tree_policy) {
        .fixed => |fixed| allocator.free(fixed),
        .auto => {},
    }
}

fn cloneTreePolicy(allocator: std.mem.Allocator, policy: TreePolicy) !TreePolicy {
    return switch (policy) {
        .auto => .auto,
        .fixed => |fixed| .{ .fixed = try allocator.dupe(u16, fixed) },
    };
}

fn cloneExternalLegs(allocator: std.mem.Allocator, legs: []const realization.ExternalLeg)
↳ ![]realization.ExternalLeg {
    const owned = try allocator.alloc(realization.ExternalLeg, legs.len);
    var initialized: usize = 0;

```

```

    errdefer {
        for (owned[0..initialized]) |leg| freeExternalLeg(allocator, leg);
        allocator.free(owned);
    }

    for (legs, owned[0..]) |source, *target| {
        target.* = try cloneExternalLeg(allocator, source);
        initialized += 1;
    }
    return owned;
}

fn freeExternalLegs(allocator: std.mem.Allocator, legs: []const realization.ExternalLeg) void
↪ {
    for (legs) |leg| freeExternalLeg(allocator, leg);
    allocator.free(legs);
}

fn cloneExternalLeg(allocator: std.mem.Allocator, leg: realization.ExternalLeg)
↪ !realization.ExternalLeg {
    const indices = try cloneNamedIndices(allocator, leg.indices);
    errdefer freeNamedIndices(allocator, indices);

    const label = if (leg.label) |value| try allocator.dupe(u8, value) else null;
    errdefer if (label) |value| allocator.free(value);

    const ref = try cloneRealizationRef(allocator, leg.realization);
    errdefer freeRealizationRef(allocator, ref);

    return .{ .realization = ref, .indices = indices, .label = label };
}

fn freeExternalLeg(allocator: std.mem.Allocator, leg: realization.ExternalLeg) void {
    freeRealizationRef(allocator, leg.realization);
    freeNamedIndices(allocator, leg.indices);
    if (leg.label) |label| allocator.free(label);
}

fn cloneRealizationRef(allocator: std.mem.Allocator, ref: realization.RealizationRef)
↪ !realization.RealizationRef {
    return switch (ref) {
        .primitive_irrep => |irrep| .{ .primitive_irrep = irrep },
        .handle => |handle| .{ .handle = handle },
        .registered_name => |name| .{ .registered_name = try allocator.dupe(u8, name) },
    };
}

fn freeRealizationRef(allocator: std.mem.Allocator, ref: realization.RealizationRef) void {
    switch (ref) {
        .registered_name => |name| allocator.free(name),
        .primitive_irrep, .handle => {},
    }
}

fn cloneNamedIndices(allocator: std.mem.Allocator, indices: []const realization.NamedIndex)
↪ ![]realization.NamedIndex {
    const owned = try allocator.alloc(realization.NamedIndex, indices.len);
    var initialized: usize = 0;
    errdefer {
        for (owned[0..initialized]) |index| allocator.free(index.name);
        allocator.free(owned);
    }

    for (indices, owned[0..]) |source, *target| {

```

```

        target.* = .{
            .kind = source.kind,
            .name = try allocator.dupe(u8, source.name),
        };
        initialized += 1;
    }
    return owned;
}

fn freeNamedIndices(allocator: std.mem.Allocator, indices: []const realization.NamedIndex)
↪ void {
    for (indices) |index| allocator.free(index.name);
    allocator.free(indices);
}

```

11 Tensor Rendering

Rendering turns compact invariant ids into concrete notation. Signature is a rendering/evaluation convention, not a different abstract representation. The symbolic output is streamed one term at a time so a large expanded projector does not become one large allocation.

```

const std = @import("std");
const coupling = @import("coupling.zig");
const symmetry = @import("symmetry.zig");

/// SignatureConvention selects the metric convention for rendering/evaluation.
pub const SignatureConvention = enum {
    complex,
    euclidean,
    lorentzian_mostly_plus,
    lorentzian_mostly_minus,
};

/// ProjectorRenderMode controls how much derived projector data is expanded.
pub const ProjectorRenderMode = enum {
    named,
    expanded_blocks,
    expanded_terms,
};

/// RenderFormat selects the output syntax level.
pub const RenderFormat = enum {
    symbolic_terms,
    text,
};

/// RenderOptions controls streamed invariant rendering.
pub const RenderOptions = struct {
    format: RenderFormat = .symbolic_terms,
    projectors: ProjectorRenderMode = .named,
    signature: SignatureConvention = .complex,
    preserve_caller_index_names: bool = true,
    stream_clifford_product_factors: bool = false,
    lower_clifford_product_atoms: bool = false,
    gamma_only: bool = false,
};

/// FilterDecision is the tri-state result for a streamed expansion prefix.
pub const FilterDecision = enum {
    accept,
    reject,
    undecided,
};

```

```

/// ExpansionAudit stores compact counters from streamed expansion.
pub const ExpansionAudit = struct {
    invariants: u64 = 0,
    accepted: u64 = 0,
    rejected: u64 = 0,
    undecided: u64 = 0,
    emitted: u64 = 0,
    boundary_projector_atoms: u64 = 0,
    local_projector_atoms: u64 = 0,
    clifford_product_factors: u64 = 0,
    max_frontier: u32 = 0,

    /// recordDecision updates filter counters for one term or prefix.
    pub fn recordDecision(self: *ExpansionAudit, decision: FilterDecision) void {
        switch (decision) {
            .accept => self.accepted += 1,
            .reject => self.rejected += 1,
            .undecided => self.undecided += 1,
        }
    }

    /// merge adds counters from another streamed expansion.
    pub fn merge(self: *ExpansionAudit, other: ExpansionAudit) void {
        self.invariants += other.invariants;
        self.accepted += other.accepted;
        self.rejected += other.rejected;
        self.undecided += other.undecided;
        self.emitted += other.emitted;
        self.boundary_projector_atoms += other.boundary_projector_atoms;
        self.local_projector_atoms += other.local_projector_atoms;
        self.clifford_product_factors += other.clifford_product_factors;
        self.max_frontier = @max(self.max_frontier, other.max_frontier);
    }
};

/// ExpansionFilter decides whether streamed terms should continue or emit.
pub const ExpansionFilter = struct {
    state: ?*anyopaque = null,
    decide_fn: *const fn (?*anyopaque, SymbolicTerm) anyerror!FilterDecision =
        ↳ acceptAllDecision,

    /// acceptAll constructs a filter that emits every completed term.
    pub fn acceptAll() ExpansionFilter {
        return .{ .decide_fn = acceptAllDecision };
    }

    /// rejectAll constructs a filter that rejects every term or prefix.
    pub fn rejectAll() ExpansionFilter {
        return .{ .decide_fn = rejectAllDecision };
    }

    /// rejectsAll reports whether every term can be rejected before expansion.
    pub fn rejectsAll(self: ExpansionFilter) bool {
        return self.state == null and self.decide_fn == rejectAllDecision;
    }

    /// withoutOperatorKind rejects terms containing one named operator kind.
    pub fn withoutOperatorKind(kind: NamedOperatorKind) ExpansionFilter {
        return .{
            .state = @ptrFromInt(@as(usize, @intFromEnum(kind)) + 1),
            .decide_fn = withoutOperatorKindDecision,
        };
    }
};

```



```

    /// decide applies the filter to a borrowed symbolic term.
    pub fn decide(self: ExpansionFilter, term: SymbolicTerm) !FilterDecision {
        return self.decide_fn(self.state, term);
    }
};

/// EvalOptions controls component evaluation of one invariant.
pub const EvalOptions = struct {
    signature: SignatureConvention = .complex,
};

/// EvaluationAudit stores counters from streamed symbolic evaluation.
pub const EvaluationAudit = struct {
    terms: u64 = 0,
    rejected: u64 = 0,
    lowered_clifford_factors: u64 = 0,
};

/// EvaluationTerm stores one lowered symbolic term for evaluation consumers.
pub const EvaluationTerm = struct {
    coefficient: RationalId,
    /// atoms is borrowed and valid only until the sink returns.
    atoms: []const SymbolicAtom,
};

/// IndexRef names one symbolic index in a rendered term.
pub const IndexRef = u32;

/// IndexBlockId names a compact antisymmetric or ordered index block.
pub const IndexBlockId = u32;

/// RationalId stores one exact rational coefficient by value.
pub const RationalId = struct {
    numerator: i128,
    denominator: u128,
};

/// RationalValue is a decoded exact rational coefficient.
pub const RationalValue = RationalId;

/// GammaBlock stores one internal antisymmetrized gamma factor.
pub const GammaBlock = struct {
    spinor_left: IndexRef,
    spinor_right: IndexRef,
    vector_block: IndexBlockId,
    rank: u8,
    chirality: u8,
    duality: DualityTag = .none,
};

/// CliffordProductTerm stores one grade term in a gamma-block product.
pub const CliffordProductTerm = struct {
    rank: u8,
    contractions: u8,
    coefficient: i64,
    duality: DualityTag = .none,
};

/// CliffordVectorSource identifies which input gamma block owns a vector.
pub const CliffordVectorSource = enum {
    left,
    right,
};

```

```

/// CliffordMetricContraction stores one metric between input gamma slots.
pub const CliffordMetricContraction = struct {
    left_offset: u8,
    right_offset: u8,
};

/// CliffordFreeVector stores one uncontracted vector slot in output order.
pub const CliffordFreeVector = struct {
    source: CliffordVectorSource,
    offset: u8,
};

/// CliffordProductMetricFactor names one metric in a reduced gamma product.
pub const CliffordProductMetricFactor = struct {
    left_block: IndexBlockId,
    right_block: IndexBlockId,
    left_offset: u8,
    right_offset: u8,
};

/// CliffordProductFreeVectorFactor maps one input slot into the output gamma.
pub const CliffordProductFreeVectorFactor = struct {
    source: CliffordVectorSource,
    input_block: IndexBlockId,
    input_offset: u8,
    output_offset: u8,
};

/// CliffordProductFactor stores one explicit factor of a reduced product atom.
pub const CliffordProductFactor = union(enum) {
    metric: CliffordProductMetricFactor,
    output_vector: CliffordProductFreeVectorFactor,
};

/// CliffordProductFactorRecord stores one streamed factor with product context.
pub const CliffordProductFactorRecord = struct {
    atom_index: u32,
    factor_index: u16,
    left_operator_id: u32,
    right_operator_id: u32,
    orthogonal_dimension: u16,
    output_rank: u8,
    coefficient: i64,
    output_duality: DualityTag = .none,
    factor: CliffordProductFactor,
};

/// CliffordProductFactorScan summarizes and validates lowered reducer factors.
pub const CliffordProductFactorScan = struct {
    product_count: u32 = 0,
    factor_count: u32 = 0,
    metric_count: u32 = 0,
    output_vector_count: u32 = 0,
    first_atom_index: ?u32 = null,
    last_atom_index: ?u32 = null,
    first_orthogonal_dimension: ?u16 = null,
    first_output_rank: ?u8 = null,
    first_coefficient: ?i64 = null,

    /// merge adds another factor scan into this accumulator.
    pub fn merge(self: *CliffordProductFactorScan, other: CliffordProductFactorScan) !void {
        if (other.product_count == 0) return;
        self.product_count = try addScanCount(self.product_count, other.product_count);
    }
};

```

```

        self.factor_count = try addScanCount(self.factor_count, other.factor_count);
        self.metric_count = try addScanCount(self.metric_count, other.metric_count);
        self.output_vector_count = try addScanCount(self.output_vector_count,
        ↪ other.output_vector_count);
        if (self.first_atom_index == null) {
            self.first_atom_index = other.first_atom_index;
            self.first_orthogonal_dimension = other.first_orthogonal_dimension;
            self.first_output_rank = other.first_output_rank;
            self.first_coefficient = other.first_coefficient;
        }
        self.last_atom_index = other.last_atom_index;
    }

    /// recordTerm validates one term and accumulates its factor scan.
    pub fn recordTerm(self: *CliffordProductFactorScan, term: SymbolicTerm) !void {
        try self.merge(try scanCliffordProductFactors(term));
    }
};

/// CliffordProductExpansion stores the small finite grade expansion.
pub const CliffordProductExpansion = struct {
    terms: [256]CliffordProductTerm = undefined,
    len: u16 = 0,

    fn append(self: *CliffordProductExpansion, term: CliffordProductTerm) !void {
        if (self.len >= self.terms.len) return error.GammaProductTooLarge;
        self.terms[self.len] = term;
        self.len += 1;
    }

    /// slice returns the initialized product terms.
    pub fn slice(self: *const CliffordProductExpansion) []const CliffordProductTerm {
        return self.terms[0..self.len];
    }
};

/// CliffordContractionPlan stores metric pairs and residual gamma slots.
pub const CliffordContractionPlan = struct {
    metrics: [256]CliffordMetricContraction = undefined,
    metric_len: u16 = 0,
    free_vectors: [256]CliffordFreeVector = undefined,
    free_len: u16 = 0,

    fn appendMetric(self: *CliffordContractionPlan, metric: CliffordMetricContraction) !void {
        if (self.metric_len >= self.metrics.len) return error.GammaProductTooLarge;
        self.metrics[self.metric_len] = metric;
        self.metric_len += 1;
    }

    fn appendFreeVector(self: *CliffordContractionPlan, vector: CliffordFreeVector) !void {
        if (self.free_len >= self.free_vectors.len) return error.GammaProductTooLarge;
        self.free_vectors[self.free_len] = vector;
        self.free_len += 1;
    }

    /// metricSlice returns the initialized cross-block metric contractions.
    pub fn metricSlice(self: *const CliffordContractionPlan) []const CliffordMetricContraction
    ↪ {
        return self.metrics[0..self.metric_len];
    }

    /// freeVectorSlice returns residual gamma vector slots in output order.
    pub fn freeVectorSlice(self: *const CliffordContractionPlan) []const CliffordFreeVector {
        return self.free_vectors[0..self.free_len];
    }
};

```

```

    }
};

/// GammaMatrix stores one explicit spinor-vector Clifford atom.
pub const GammaMatrix = struct {
    operator_id: u32,
    spinor_left: IndexRef,
    spinor_right: IndexRef,
    vector: IndexRef,
    orthogonal_dimension: u16,
    rank: u8 = 1,
    chirality: u8 = 0,
    duality: DualityTag = .none,
};

/// GammaForm stores one explicit spinor bilinear coupled to a form irrep.
pub const GammaForm = struct {
    operator_id: u32,
    spinor_left: IndexRef,
    spinor_right: IndexRef,
    form: IndexRef,
    orthogonal_dimension: u16,
    rank: u8,
    chirality: u8 = 0,
    duality: DualityTag = .none,
};

/// GammaAction stores an antisymmetric gamma form acting on a spinor.
pub const GammaAction = struct {
    operator_id: u32,
    form: IndexRef,
    spinor_input: IndexRef,
    spinor_output: IndexRef,
    orthogonal_dimension: u16,
    rank: u8,
    chirality: u8 = 0,
    duality: DualityTag = .none,
};

/// CliffordProduct stores one reduced gamma-product term.
pub const CliffordProduct = struct {
    left_operator_id: u32,
    right_operator_id: u32,
    left_block: IndexBlockId,
    right_block: IndexBlockId,
    orthogonal_dimension: u16,
    left_rank: u8,
    right_rank: u8,
    output_rank: u8,
    contractions: u8,
    metric_count: u16,
    free_vector_count: u16,
    coefficient: i64,
    left_duality: DualityTag = .none,
    right_duality: DualityTag = .none,
    output_duality: DualityTag = .none,
};

/// CartanProduct stores a compact highest-weight product projection.
pub const CartanProduct = struct {
    operator_id: u32,
    left: IndexRef,
    right: IndexRef,
    output: IndexRef,

```

```

};

/// IdentityRoute stores the explicit identity product with a singlet factor.
pub const IdentityRoute = struct {
    operator_id: u32,
    left: IndexRef,
    right: IndexRef,
    output: IndexRef,
};

/// SpinorSymmetricProduct stores the spinor-tower Cartan product formula.
pub const SpinorSymmetricProduct = struct {
    operator_id: u32,
    left: IndexRef,
    right: IndexRef,
    output: IndexRef,
    orthogonal_dimension: u16,
    left_power: u16,
    right_power: u16,
    output_power: u16,
    chirality: u8 = 0,
};

/// SpinorTowerContract stores one contraction lowering a spinor tower by one spinor.
pub const SpinorTowerContract = struct {
    operator_id: u32,
    input_tower: IndexRef,
    spinor: IndexRef,
    output_tower: IndexRef,
    output_form: IndexBlockId = 0,
    orthogonal_dimension: u16,
    input_power: u16,
    output_power: u16,
    form_rank: u8 = 0,
    chirality: u8 = 0,
};

/// SpinorTowerContractAdjoint stores the adjoint of a spinor-tower contraction.
pub const SpinorTowerContractAdjoint = struct {
    operator_id: u32,
    input_tower: IndexRef,
    spinor: IndexRef,
    output_tower: IndexRef,
    input_form: IndexBlockId = 0,
    orthogonal_dimension: u16,
    input_power: u16,
    output_power: u16,
    form_rank: u8 = 0,
    chirality: u8 = 0,
};

/// SpinorIndexDelta stores one primitive spinor Kronecker-delta route.
pub const SpinorIndexDelta = struct {
    operator_id: u32,
    source_tower: IndexRef,
    output_tower: IndexRef,
    source_slot: u16,
    output_slot: u16,
    chirality: u8 = 0,
};

/// TensorSpinorProjection stores a compact form-valued spinor-tower channel.
pub const TensorSpinorProjection = struct {
    operator_id: u32,

```

```

    left: IndexRef,
    right: IndexRef,
    output: IndexRef,
    orthogonal_dimension: u16,
    left_has_spinor: bool = true,
    right_has_spinor: bool = true,
    output_has_spinor: bool = true,
    right_chirality: u8 = 255,
    input_form_profile: u128 = 0,
    input_form_count: u8 = 0,
    input_tower_power: u16 = 0,
    form_rank: u8,
    form_count: u8 = 1,
    form_mask: u64 = 0,
    form_profile: u128 = 0,
    tower_power: u16,
    chirality: u8 = 0,
    duality: DualityTag = .none,
};

/// TensorFormProjection stores a compact tensor-form terminal channel.
pub const TensorFormProjection = struct {
    operator_id: u32,
    left: IndexRef,
    right: IndexRef,
    output: IndexRef,
    orthogonal_dimension: u16,
    input_form_profile: u128 = 0,
    input_form_mask: u64,
    output_form_profile: u128,
    output_form_count: u8,
    output_form_rank: u8 = 0,
    output_duality: DualityTag = .none,
    right_chirality: u8 = 0,
    chirality: u8 = 0,
};

/// ExteriorGammaActionAtom stores the primitive exterior gamma contraction.
pub const ExteriorGammaActionAtom = struct {
    operator_id: u32,
    input_form: IndexBlockId,
    gamma_form: IndexBlockId,
    output_form: IndexBlockId,
    spinor_input: IndexRef,
    spinor_output: IndexRef,
    orthogonal_dimension: u16,
    input_rank: u8,
    gamma_rank: u8,
    output_rank: u8,
    contraction_count: u8,
    chirality: u8 = 0,
    duality: DualityTag = .none,
};

/// FormRankSplitDelta stores the primitive generalized delta for a rank split.
pub const FormRankSplitDelta = struct {
    source_form: IndexBlockId,
    lower_form: IndexBlockId,
    upper_form: IndexBlockId,
    source_rank: u8,
    lower_rank: u8,
    upper_rank: u8,
};

```

```

/// TensorFormGammaWedge stores a terminal tensor-spinor gamma-wedge formula.
pub const TensorFormGammaWedge = struct {
    operator_id: u32,
    left: IndexRef,
    right: IndexRef,
    output: IndexRef,
    orthogonal_dimension: u16,
    input_form_mask: u64,
    input_form_profile: u128,
    input_form_count: u8,
    output_form_profile: u128,
    output_form_count: u8,
    inserted_rank: u8,
    action_input_rank: u8 = 0,
    action_output_rank: u8 = 0,
    action_contraction_count: u8 = 0,
    chirality: u8 = 0,
    duality: DualityTag = .none,
};

/// TensorFormGammaMap stores one explicit gamma map between form profiles.
pub const TensorFormGammaMap = struct {
    operator_id: u32,
    spinor_left: IndexRef,
    spinor_right: IndexRef,
    input_form: IndexBlockId,
    output_lower_form: IndexBlockId,
    output_upper_form: IndexBlockId,
    orthogonal_dimension: u16,
    source_rank: u8,
    lower_rank: u8,
    upper_rank: u8,
    chirality: u8 = 0,
};

/// TensorFormGammaMapAdjoint stores the adjoint of a gamma rank-split map.
pub const TensorFormGammaMapAdjoint = struct {
    operator_id: u32,
    spinor_left: IndexRef,
    spinor_right: IndexRef,
    input_lower_form: IndexBlockId,
    input_upper_form: IndexBlockId,
    output_form: IndexBlockId,
    orthogonal_dimension: u16,
    source_rank: u8,
    lower_rank: u8,
    upper_rank: u8,
    chirality: u8 = 0,
};

/// TensorFormSpinorPair stores a form-preserving terminal spinor pairing formula.
pub const TensorFormSpinorPair = struct {
    operator_id: u32,
    left: IndexRef,
    right: IndexRef,
    output: IndexRef,
    orthogonal_dimension: u16,
    input_form_mask: u64,
    input_form_profile: u128,
    input_form_count: u8,
    output_form_profile: u128,
    output_form_count: u8,
    left_chirality: u8 = 0,
    right_chirality: u8 = 0,
};

```

```

};

/// ProductIdentity stores the identity product with a singlet factor.
pub const ProductIdentity = struct {
    operator_id: u32,
    left: IndexRef,
    right: IndexRef,
    output: IndexRef,
};

/// StructureConstant stores one backend-specific invariant tensor atom.
pub const StructureConstant = struct {
    operator_id: u32,
    first: IndexRef,
    second: IndexRef,
    third: IndexRef,
    family: symmetry.LieFamily,
    rank: u8,
};

/// SpinorPair stores one explicit invariant spinor bilinear atom.
pub const SpinorPair = struct {
    operator_id: u32,
    spinor_left: IndexRef,
    spinor_right: IndexRef,
    orthogonal_dimension: u16,
    chirality_left: u8 = 0,
    chirality_right: u8 = 0,
};

/// VectorSpinorIdentity stores the identity on a vector-spinor slot.
pub const VectorSpinorIdentity = struct {
    operator_id: u32,
    vector: IndexRef,
    spinor: IndexRef,
    orthogonal_dimension: u16,
    chirality: u8 = 0,
};

/// GammaTrace stores the gamma-trace part of a vector-spinor projector.
pub const GammaTrace = struct {
    operator_id: u32,
    vector: IndexRef,
    spinor: IndexRef,
    orthogonal_dimension: u16,
    chirality: u8 = 0,
};

/// DualityTag records middle-form self-duality data.
pub const DualityTag = enum {
    none,
    self_dual,
    anti_self_dual,
};

/// GammaChirality classifies chiral spinor conventions for gamma renderers.
pub const GammaChirality = enum(u8) {
    none = 0,
    left = 1,
    right = 2,
};

/// gammaChiralityTag returns a compact chirality tag for a spinor Dynkin label.

```



```

pub fn gammaChiralityTag(simple: symmetry.SimpleLieAlgebra, label: []const i16) GammaChirality
↪ {
    if (!isSpinorDynkin(simple, label)) return .none;
    return switch (simple.family) {
        .b => .none,
        .d => if (label[label.len - 2] == 1) .left else .right,
        else => .none,
    };
}

/// orthogonalDimension returns the vector dimension for B/D algebras.
pub fn orthogonalDimension(simple: symmetry.SimpleLieAlgebra) u16 {
    return switch (simple.family) {
        .b => @as(u16, simple.rank) * 2 + 1,
        .d => @as(u16, simple.rank) * 2,
        else => 0,
    };
}

/// gammaDualityValid checks whether a gamma block duality tag matches its grade.
pub fn gammaDualityValid(orthogonal_dimension: u16, rank: u8, duality: DualityTag) bool {
    if (orthogonal_dimension == 0 or rank > orthogonal_dimension) return false;
    return switch (duality) {
        .none => true,
        .self_dual, .anti_self_dual => @as(u16, rank) * 2 == orthogonal_dimension,
    };
}

/// gammaProductExpansion returns the grade expansion of two gamma blocks.
pub fn gammaProductExpansion(orthogonal_dimension: u16, left_rank: u8, right_rank: u8,
↪ left_duality: DualityTag, right_duality: DualityTag) !CliffordProductExpansion {
    if (!gammaDualityValid(orthogonal_dimension, left_rank, left_duality)) return
    ↪ error.InvalidGammaRank;
    if (!gammaDualityValid(orthogonal_dimension, right_rank, right_duality)) return
    ↪ error.InvalidGammaRank;

    var expansion: CliffordProductExpansion = .{};
    const max_contractions = @min(left_rank, right_rank);
    for (0..@as(usize, max_contractions) + 1) |contractions| {
        const contraction_count: u8 = @intCast(contractions);
        const rank = left_rank + right_rank - 2 * contraction_count;
        if (rank > orthogonal_dimension) continue;
        const coefficient = try gammaProductCoefficient(left_rank, right_rank,
    ↪ contraction_count);
        try expansion.append(.{
            .rank = rank,
            .contractions = contraction_count,
            .coefficient = coefficient,
            .duality = gammaProductOutputDuality(orthogonal_dimension, rank, left_duality,
    ↪ right_duality),
        });
    }
    return expansion;
}

/// gammaProductContractionPlan returns metrics and free slots for one product term.
pub fn gammaProductContractionPlan(left_rank: u8, right_rank: u8, term: CliffordProductTerm)
↪ !CliffordContractionPlan {
    if (term.contractions > @min(left_rank, right_rank)) return error.InvalidGammaProductTerm;
    if (term.rank != left_rank + right_rank - 2 * term.contractions) return
    ↪ error.InvalidGammaProductTerm;

    var plan: CliffordContractionPlan = .{};
    const left_free_count = left_rank - term.contractions;

```

```

    for (0..left_free_count) |offset| {
        try plan.appendFreeVector(.{ .source = .left, .offset = @intCast(offset) });
    }
    for (0..term.contractions) |offset| {
        try plan.appendMetric(.{
            .left_offset = @intCast(left_free_count + offset),
            .right_offset = @intCast(offset),
        });
    }
    for (term.contractions..right_rank) |offset| {
        try plan.appendFreeVector(.{ .source = .right, .offset = @intCast(offset) });
    }
    return plan;
}

/// cliffordProductTerm decodes the grade term stored in a streamed atom.
pub fn cliffordProductTerm(product: CliffordProduct) CliffordProductTerm {
    return .{
        .rank = product.output_rank,
        .contractions = product.contractions,
        .coefficient = product.coefficient,
        .duality = product.output_duality,
    };
}

/// cliffordProductContractionPlan reconstructs the metric/free-slot plan.
pub fn cliffordProductContractionPlan(product: CliffordProduct) !CliffordContractionPlan {
    if (!gammaDualityValid(product.orthogonal_dimension, product.left_rank,
        ⇨ product.left_duality)) return error.InvalidGammaProductTerm;
    if (!gammaDualityValid(product.orthogonal_dimension, product.right_rank,
        ⇨ product.right_duality)) return error.InvalidGammaProductTerm;
    if (!gammaDualityValid(product.orthogonal_dimension, product.output_rank,
        ⇨ product.output_duality)) return error.InvalidGammaProductTerm;
    const term = cliffordProductTerm(product);
    const plan = try gammaProductContractionPlan(product.left_rank, product.right_rank, term);
    if (plan.metric_len != product.metric_count or plan.free_len != product.free_vector_count)
        ⇨ return error.InvalidGammaProductTerm;
    return plan;
}

/// cliffordProductFactorCount returns the number of explicit factors in an atom.
pub fn cliffordProductFactorCount(product: CliffordProduct) !u16 {
    const plan = try cliffordProductContractionPlan(product);
    return plan.metric_len + plan.free_len;
}

/// cliffordProductFactorAt returns one metric or residual vector factor.
pub fn cliffordProductFactorAt(product: CliffordProduct, index: u16) !CliffordProductFactor {
    const plan = try cliffordProductContractionPlan(product);
    if (index < plan.metric_len) {
        const metric = plan.metricSlice()[index];
        return .{ .metric = .{
            .left_block = product.left_block,
            .right_block = product.right_block,
            .left_offset = metric.left_offset,
            .right_offset = metric.right_offset,
        } };
    }
}

const free_index = index - plan.metric_len;
if (free_index >= plan.free_len) return error.CliffordProductFactorOutOfBounds;
const vector = plan.freeVectorSlice()[free_index];
return .{ .output_vector = .{
    .source = vector.source,

```

```

        .input_block = if (vector.source == .left) product.left_block else
        ↪ product.right_block,
        .input_offset = vector.offset,
        .output_offset = @intCast(free_index),
    } }];
}

/// streamCliffordProductFactors emits factors from compact or lowered reducers.
pub fn streamCliffordProductFactors(term: SymbolicTerm, sink: anytype) !u32 {
    var emitted: u32 = 0;
    for (term.atoms, 0..) |atom, atom_index| {
        const product = switch (atom) {
            .clifford_product => |product| product,
            .clifford_product_factor => |record| {
                try sink.emitCliffordProductFactor(record);
                emitted += 1;
                continue;
            },
            else => continue,
        };
        const plan = try cliffordProductContractionPlan(product);
        var factor_index: u16 = 0;
        for (plan.metricSlice()) |metric| {
            try sink.emitCliffordProductFactor(.{
                .atom_index = @intCast(atom_index),
                .factor_index = factor_index,
                .left_operator_id = product.left_operator_id,
                .right_operator_id = product.right_operator_id,
                .orthogonal_dimension = product.orthogonal_dimension,
                .output_rank = product.output_rank,
                .coefficient = product.coefficient,
                .output_duality = product.output_duality,
                .factor = .{ .metric = .{
                    .left_block = product.left_block,
                    .right_block = product.right_block,
                    .left_offset = metric.left_offset,
                    .right_offset = metric.right_offset,
                } },
            });
            factor_index += 1;
            emitted += 1;
        }
        for (plan.freeVectorSlice(), 0..) |vector, free_index| {
            try sink.emitCliffordProductFactor(.{
                .atom_index = @intCast(atom_index),
                .factor_index = factor_index,
                .left_operator_id = product.left_operator_id,
                .right_operator_id = product.right_operator_id,
                .orthogonal_dimension = product.orthogonal_dimension,
                .output_rank = product.output_rank,
                .coefficient = product.coefficient,
                .output_duality = product.output_duality,
                .factor = .{ .output_vector = .{
                    .source = vector.source,
                    .input_block = if (vector.source == .left) product.left_block else
                    ↪ product.right_block,
                    .input_offset = vector.offset,
                    .output_offset = @intCast(free_index),
                } },
            });
            factor_index += 1;
            emitted += 1;
        }
    }
}

```

```

    return emitted;
}

/// scanCliffordProductFactors validates and counts compact or lowered factors.
pub fn scanCliffordProductFactors(term: SymbolicTerm) !CliffordProductFactorScan {
    var scan: CliffordProductFactorScan = .{};
    var current_lowered_atom: ?u32 = null;
    var next_lowered_factor_index: u16 = 0;
    var current_left_operator_id: u32 = 0;
    var current_right_operator_id: u32 = 0;
    var current_dimension: u16 = 0;
    var current_output_rank: u8 = 0;
    var current_coefficient: i64 = 0;
    var current_duality: DualityTag = .none;

    for (term.atoms, 0..) |atom, atom_index| {
        switch (atom) {
            .clifford_product => |product| {
                const plan = try cliffordProductContractionPlan(product);
                try recordCliffordProductScanHeader(&scan, @intCast(atom_index),
                    ↪ product.orthogonal_dimension, product.output_rank, product.coefficient);
                scan.factor_count += plan.metric_len + plan.free_len;
                scan.metric_count += plan.metric_len;
                scan.output_vector_count += plan.free_len;
            },
            .clifford_product_factor => |record| {
                if (current_lowered_atom) |current_atom| {
                    if (record.atom_index == current_atom) {
                        if (record.factor_index != next_lowered_factor_index) return
                            ↪ error.InvalidCliffordProductFactorSequence;
                        if (record.left_operator_id != current_left_operator_id or
                            record.right_operator_id != current_right_operator_id or
                            record.orthogonal_dimension != current_dimension or
                            record.output_rank != current_output_rank or
                            record.coefficient != current_coefficient or
                            record.output_duality != current_duality)
                        {
                            return error.InvalidCliffordProductFactorSequence;
                        }
                    } else {
                        if (record.atom_index <= current_atom) return
                            ↪ error.InvalidCliffordProductFactorSequence;
                        if (record.factor_index != 0) return
                            ↪ error.InvalidCliffordProductFactorSequence;
                        try recordCliffordProductScanHeader(&scan, record.atom_index,
                            ↪ record.orthogonal_dimension, record.output_rank,
                            ↪ record.coefficient);
                        current_lowered_atom = record.atom_index;
                        current_left_operator_id = record.left_operator_id;
                        current_right_operator_id = record.right_operator_id;
                        current_dimension = record.orthogonal_dimension;
                        current_output_rank = record.output_rank;
                        current_coefficient = record.coefficient;
                        current_duality = record.output_duality;
                    }
                } else {
                    if (record.factor_index != 0) return
                        ↪ error.InvalidCliffordProductFactorSequence;
                    try recordCliffordProductScanHeader(&scan, record.atom_index,
                        ↪ record.orthogonal_dimension, record.output_rank, record.coefficient);
                    current_lowered_atom = record.atom_index;
                    current_left_operator_id = record.left_operator_id;
                    current_right_operator_id = record.right_operator_id;
                    current_dimension = record.orthogonal_dimension;
                }
            }
        }
    }
}

```

```

        current_output_rank = record.output_rank;
        current_coefficient = record.coefficient;
        current_duality = record.output_duality;
    }

    next_lowered_factor_index = record.factor_index + 1;
    scan.factor_count += 1;
    switch (record.factor) {
        .metric => scan.metric_count += 1,
        .output_vector => scan.output_vector_count += 1,
    }
},
    else => {},
}
}
return scan;
}

fn recordCliffordProductScanHeader(scan: *CliffordProductFactorScan, atom_index: u32,
↳ dimension: u16, output_rank: u8, coefficient: i64) !void {
    if (scan.product_count == std.math.maxInt(u32)) return error.CliffordProductScanTooLarge;
    scan.product_count += 1;
    if (scan.first_atom_index == null) {
        scan.first_atom_index = atom_index;
        scan.first_orthogonal_dimension = dimension;
        scan.first_output_rank = output_rank;
        scan.first_coefficient = coefficient;
    }
    scan.last_atom_index = atom_index;
}

fn addScanCount(left: u32, right: u32) !u32 {
    return std.math.add(u32, left, right) catch error.CliffordProductScanTooLarge;
}

fn addAuditAtomCount(left: u64, right: usize) !u64 {
    return std.math.add(u64, left, @intCast(right)) catch error.CliffordProductScanTooLarge;
}

fn addAuditCount(left: u64, right: u64) !u64 {
    return std.math.add(u64, left, right) catch error.CliffordProductScanTooLarge;
}

/// appendLoweredCliffordProductAtoms replaces reducer atoms with factor atoms.
pub fn appendLoweredCliffordProductAtoms(allocator: std.mem.Allocator, lowered_atoms:
↳ *std.ArrayList(SymbolicAtom), term: SymbolicTerm) !u32 {
    var lowered_factor_count: u32 = 0;
    for (term.atoms, 0..) |atom, atom_index| {
        const product = switch (atom) {
            .clifford_product => |product| product,
            else => {
                try lowered_atoms.append(allocator, atom);
                continue;
            },
        };
        const plan = try cliffordProductContractionPlan(product);
        var factor_index: u16 = 0;
        for (plan.metricSlice()) |metric| {
            try lowered_atoms.append(allocator, .{ .clifford_product_factor = .{
                .atom_index = @intCast(atom_index),
                .factor_index = factor_index,
                .left_operator_id = product.left_operator_id,
                .right_operator_id = product.right_operator_id,
                .orthogonal_dimension = product.orthogonal_dimension,
            }

```

```

        .output_rank = product.output_rank,
        .coefficient = product.coefficient,
        .output_duality = product.output_duality,
        .factor = .{ .metric = .{
            .left_block = product.left_block,
            .right_block = product.right_block,
            .left_offset = metric.left_offset,
            .right_offset = metric.right_offset,
        } },
    } });
    factor_index += 1;
    lowered_factor_count += 1;
}
for (plan.freeVectorSlice(), 0..) |vector, free_index| {
    try lowered_atoms.append(allocator, .{ .clifford_product_factor = .{
        .atom_index = @intCast(atom_index),
        .factor_index = factor_index,
        .left_operator_id = product.left_operator_id,
        .right_operator_id = product.right_operator_id,
        .orthogonal_dimension = product.orthogonal_dimension,
        .output_rank = product.output_rank,
        .coefficient = product.coefficient,
        .output_duality = product.output_duality,
        .factor = .{ .output_vector = .{
            .source = vector.source,
            .input_block = if (vector.source == .left) product.left_block else
                ↪ product.right_block,
            .input_offset = vector.offset,
            .output_offset = @intCast(free_index),
        } },
    } });
    factor_index += 1;
    lowered_factor_count += 1;
}
}
return lowered_factor_count;
}

/// NamedOperatorKind classifies compact backend-rendered operators.
pub const NamedOperatorKind = enum {
    projector,
    gamma,
    metric,
    epsilon,
    structure_constant,
    custom,
};

/// SymbolicAtom stores one symbolic invariant contraction atom.
pub const SymbolicAtom = union(enum) {
    metric_pair: struct { left: IndexRef, right: IndexRef },
    gamma_matrix: GammaMatrix,
    gamma_form: GammaForm,
    gamma_action: GammaAction,
    clifford_product: CliffordProduct,
    clifford_product_factor: CliffordProductFactorRecord,
    identity_route: IdentityRoute,
    spinor_symmetrized_product: SpinorSymmetricProduct,
    spinor_index_delta: SpinorIndexDelta,
    spinor_tower_contract: SpinorTowerContract,
    spinor_tower_contract_adjoint: SpinorTowerContractAdjoint,
    product_identity: ProductIdentity,
    cartan_product: CartanProduct,
    tensor_spinor_projection: TensorSpinorProjection,

```

```

    tensor_form_projection: TensorFormProjection,
    exterior_gamma_action: ExteriorGammaActionAtom,
    form_rank_split_delta: FormRankSplitDelta,
    tensor_form_gamma_wedge: TensorFormGammaWedge,
    tensor_form_gamma_map: TensorFormGammaMap,
    tensor_form_gamma_map_adjoint: TensorFormGammaMapAdjoint,
    tensor_form_spinor_pair: TensorFormSpinorPair,
    gamma_trace: GammaTrace,
    spinor_pair: SpinorPair,
    vector_spinor_identity: VectorSpinorIdentity,
    structure_constant: StructureConstant,
    generalized_delta: struct { upper: IndexBlockId, lower: IndexBlockId },
    epsilon: struct { block: IndexBlockId },
    hodge_star: struct { input: IndexBlockId, output: IndexBlockId },
    antisymmetrizer: struct { block: IndexBlockId },
    named_operator: struct { kind: NamedOperatorKind, operator_id: u32, first_index: u32,
        ↪ index_len: u16 },
};

/// SymbolicAtomTag names one symbolic atom variant.
pub const SymbolicAtomTag = std.meta.Tag(SymbolicAtom);

/// SymbolicTerm stores one streamed coefficient times symbolic atoms.
pub const SymbolicTerm = struct {
    coefficient: RationalId,
    /// atoms is borrowed and valid only until the sink returns.
    atoms: []const SymbolicAtom,
};

/// AtomLoweringAudit records how far streamed atoms have been lowered.
pub const AtomLoweringAudit = struct {
    term_count: u64 = 0,
    atom_count: u64 = 0,
    primitive_atoms: u64 = 0,
    compact_formula_atoms: u64 = 0,
    named_operator_atoms: u64 = 0,
    named_projector_atoms: u64 = 0,
    clifford_product_atoms: u64 = 0,
    clifford_product_factor_atoms: u64 = 0,
    product_identity_atoms: u64 = 0,
    cartan_product_atoms: u64 = 0,
    tensor_spinor_projection_atoms: u64 = 0,
    tensor_form_projection_atoms: u64 = 0,
    structural_atoms: u64 = 0,
    spinor_symmetrized_product_atoms: u64 = 0,
    spinor_tower_contract_atoms: u64 = 0,
    spinor_tower_contract_adjoint_atoms: u64 = 0,
    tensor_form_gamma_wedge_atoms: u64 = 0,
    tensor_form_gamma_map_atoms: u64 = 0,
    tensor_form_gamma_map_adjoint_atoms: u64 = 0,
    gamma_trace_atoms: u64 = 0,
    vector_spinor_identity_atoms: u64 = 0,
    first_compact_kind: ?SymbolicAtomTag = null,
    first_structural_kind: ?SymbolicAtomTag = null,

    /// recordTerm classifies one borrowed rendered term.
    pub fn recordTerm(self: *AtomLoweringAudit, term: SymbolicTerm) !void {
        self.term_count = try addAuditCount(self.term_count, 1);
        self.atom_count = try addAuditAtomCount(self.atom_count, term.atoms.len);
        for (term.atoms) |atom| {
            try self.recordAtom(atom);
        }
    }
};

```

```

fn recordAtom(self: *AtomLoweringAudit, atom: SymbolicAtom) !void {
  switch (atom) {
    .metric_pair,
    .gamma_matrix,
    .gamma_form,
    .gamma_action,
    .spinor_pair,
    .structure_constant,
    .generalized_delta,
    .epsilon,
    .hodge_star,
    .antisymmetrizer,
    .identity_route,
    .spinor_index_delta,
    .exterior_gamma_action,
    .form_rank_split_delta,
    .tensor_form_spinor_pair,
    => self.primitive_atoms = try addAuditCount(self.primitive_atoms, 1),
    .spinor_symmetrized_product => try self.recordStructural(atom,
      ↪ &self.spinor_symmetrized_product_atoms),
    .spinor_tower_contract => try self.recordStructural(atom,
      ↪ &self.spinor_tower_contract_atoms),
    .spinor_tower_contract_adjoint => try self.recordStructural(atom,
      ↪ &self.spinor_tower_contract_adjoint_atoms),
    .tensor_form_gamma_wedge => try self.recordStructural(atom,
      ↪ &self.tensor_form_gamma_wedge_atoms),
    .tensor_form_gamma_map => try self.recordStructural(atom,
      ↪ &self.tensor_form_gamma_map_atoms),
    .tensor_form_gamma_map_adjoint => try self.recordStructural(atom,
      ↪ &self.tensor_form_gamma_map_adjoint_atoms),
    .clifford_product_factor => {
      self.primitive_atoms = try addAuditCount(self.primitive_atoms, 1);
      self.clifford_product_factor_atoms = try
        ↪ addAuditCount(self.clifford_product_factor_atoms, 1);
    },
    .clifford_product => {
      self.clifford_product_atoms = try addAuditCount(self.clifford_product_atoms,
        ↪ 1);
      try self.recordCompact(atom);
    },
    .product_identity => {
      self.product_identity_atoms = try addAuditCount(self.product_identity_atoms,
        ↪ 1);
      try self.recordCompact(atom);
    },
    .cartan_product => {
      self.cartan_product_atoms = try addAuditCount(self.cartan_product_atoms, 1);
      try self.recordCompact(atom);
    },
    .tensor_spinor_projection => {
      self.tensor_spinor_projection_atoms = try
        ↪ addAuditCount(self.tensor_spinor_projection_atoms, 1);
      try self.recordCompact(atom);
    },
    .tensor_form_projection => {
      self.tensor_form_projection_atoms = try
        ↪ addAuditCount(self.tensor_form_projection_atoms, 1);
      try self.recordCompact(atom);
    },
    .gamma_trace => {
      self.gamma_trace_atoms = try addAuditCount(self.gamma_trace_atoms, 1);
      try self.recordCompact(atom);
    },
    .vector_spinor_identity => {

```



```

        self.vector_spinor_identity_atoms = try
        ↪ addAuditCount(self.vector_spinor_identity_atoms, 1);
        try self.recordCompact(atom);
    },
    .named_operator => |operator| {
        self.named_operator_atoms = try addAuditCount(self.named_operator_atoms, 1);
        if (operator.kind == .projector) self.named_projector_atoms = try
        ↪ addAuditCount(self.named_projector_atoms, 1);
        try self.recordCompact(atom);
    },
}
}

fn recordCompact(self: *AtomLoweringAudit, atom: SymbolicAtom) !void {
    self.compact_formula_atoms = try addAuditCount(self.compact_formula_atoms, 1);
    if (self.first_compact_kind == null) self.first_compact_kind =
    ↪ std.meta.activeTag(atom);
}

fn recordStructural(self: *AtomLoweringAudit, atom: SymbolicAtom, counter: *u64) !void {
    counter.* = try addAuditCount(counter.*, 1);
    self.structural_atoms = try addAuditCount(self.structural_atoms, 1);
    if (self.first_structural_kind == null) self.first_structural_kind =
    ↪ std.meta.activeTag(atom);
    try self.recordCompact(atom);
}

/// gammaOnly returns true when no compact or structural atom remains.
pub fn gammaOnly(self: AtomLoweringAudit) bool {
    return self.compact_formula_atoms == 0 and
        self.structural_atoms == 0 and
        self.named_projector_atoms == 0 and
        self.tensor_spinor_projection_atoms == 0 and
        self.tensor_form_projection_atoms == 0 and
        self.clifford_product_atoms == 0;
}

};

/// GammaOnlyAudit records compact and structural blockers for gamma-only terms.
pub const GammaOnlyAudit = AtomLoweringAudit;

/// AtomLoweringAuditSink accumulates atom-lowering counters from terms.
pub const AtomLoweringAuditSink = struct {
    audit: AtomLoweringAudit = .{},

    /// emitTerm records one rendered symbolic term.
    pub fn emitTerm(self: *AtomLoweringAuditSink, term: SymbolicTerm) !void {
        try self.audit.recordTerm(term);
    }
};

/// CliffordProductFactorAuditSink accumulates factor scans from rendered terms.
pub const CliffordProductFactorAuditSink = struct {
    term_count: u32 = 0,
    atom_count: u64 = 0,
    scan: CliffordProductFactorScan = .{},

    /// emitTerm records one rendered symbolic term.
    pub fn emitTerm(self: *CliffordProductFactorAuditSink, term: SymbolicTerm) !void {
        self.term_count = try addScanCount(self.term_count, 1);
        self.atom_count = try addAuditAtomCount(self.atom_count, term.atoms.len);
        try self.scan.recordTerm(term);
    }
};

```

```

/// emitTerm sends one symbolic term to a caller-provided sink.
pub fn emitTerm(sink: anytype, term: SymbolicTerm) !void {
    try sink.emitTerm(term);
}

/// emitEvaluationTerm sends one evaluation term to a caller-provided sink.
pub fn emitEvaluationTerm(sink: anytype, term: EvaluationTerm) !void {
    try sink.emitEvaluationTerm(term);
}

/// rationalOne returns the coefficient id for one.
pub fn rationalOne() RationalId {
    return .{ .numerator = 1, .denominator = 1 };
}

/// rationalEq1 compares two exact rational coefficients.
pub fn rationalEq1(left: RationalId, right: RationalId) bool {
    return left.numerator == right.numerator and left.denominator == right.denominator;
}

/// rationalFromSmall normalizes an exact rational coefficient.
pub fn rationalFromSmall(numerator: i64, denominator: u64) !RationalId {
    if (denominator == 0) return error.InvalidRationalDenominator;
    if (numerator == 0) return .{ .numerator = 0, .denominator = 1 };
    const divisor = gcdU128(absI128(numerator), denominator);
    return .{
        .numerator = @divExact(@as(i128, numerator), @as(i128, @intCast(divisor))),
        .denominator = @divExact(@as(u128, denominator), divisor),
    };
}

/// rationalValue decodes a coefficient id produced by rationalFromSmall.
pub fn rationalValue(id: RationalId) RationalValue {
    return id;
}

fn absI128(value: i128) u128 {
    if (value == std.math.minInt(i128)) return @as(u128, 1) << 127;
    return if (value < 0) @intCast(-value) else @intCast(value);
}

fn gcdU128(left: u128, right: u128) u128 {
    var a = left;
    var b = right;
    while (b != 0) {
        const remainder = a % b;
        a = b;
        b = remainder;
    }
    return if (a == 0) 1 else a;
}

fn gammaProductCoefficient(left_rank: u8, right_rank: u8, contractions: u8) !i64 {
    var coefficient: i64 = 1;
    coefficient = try checkedMulI64(coefficient, try binomialI64(left_rank, contractions));
    coefficient = try checkedMulI64(coefficient, try binomialI64(right_rank, contractions));
    coefficient = try checkedMulI64(coefficient, try factorialI64(contractions));
    const contraction_pairs = if (contractions < 2) 0 else @as(u16, contractions) * (@as(u16,
    ↪ contractions) - 1) / 2;
    if (contraction_pairs % 2 == 1) coefficient = -coefficient;
    return coefficient;
}

```

```

fn gammaProductOutputDuality(orthogonal_dimension: u16, rank: u8, left_duality: DualityTag,
↪ right_duality: DualityTag) DualityTag {
    if (@as(u16, rank) * 2 != orthogonal_dimension) return .none;
    if (left_duality != .none) return left_duality;
    if (right_duality != .none) return right_duality;
    return .none;
}

fn binomialI64(n: u8, k: u8) !i64 {
    if (k > n) return 0;
    const choose = @min(k, n - k);
    var result: i64 = 1;
    var divisor: u8 = 1;
    while (divisor <= choose) : (divisor += 1) {
        const numerator = n - choose + divisor;
        result = try checkedMulI64(result, numerator);
        result = @divExact(result, divisor);
    }
    return result;
}

fn factorialI64(n: u8) !i64 {
    if (n < 2) return 1;
    var result: i64 = 1;
    for (2..@as(usize, n) + 1) |factor| {
        result = try checkedMulI64(result, @intCast(factor));
    }
    return result;
}

fn checkedMulI64(left: i64, right: i64) !i64 {
    return std.math.mul(i64, left, right) catch error.GammaProductCoefficientOverflow;
}

/// RenderedTermSpan stores a compact range of already emitted terms.
const RenderedTermSpan = struct {
    offset: u32,
    len: u32,
};

/// ReductionTerm stores one coefficient of an invariant in a target basis.
const ReductionTerm = struct {
    invariant: coupling.InvariantHandle,
    coefficient: RationalId,
};

fn acceptAllDecision(_: ?*anyopaque, _: SymbolicTerm) anyerror!FilterDecision {
    return .accept;
}

fn rejectAllDecision(_: ?*anyopaque, _: SymbolicTerm) anyerror!FilterDecision {
    return .reject;
}

fn isSpinorDynkin(simple: symmetry.SimpleLieAlgebra, label: []const i16) bool {
    if (label.len != simple.rank) return false;
    switch (simple.family) {
        .b => {
            if (label.len == 0 or label[label.len - 1] != 1) return false;
            for (label[0 .. label.len - 1]) |entry| {
                if (entry != 0) return false;
            }
            return true;
        },
    },
}

```

```

.d => {
  if (label.len < 2) return false;
  const left = label.len - 2;
  const right = label.len - 1;
  if (!((label[left] == 1 and label[right] == 0) or (label[left] == 0 and
  ↪ label[right] == 1))) return false;
  for (label[0..left]) |entry| {
    if (entry != 0) return false;
  }
  return true;
},
else => return false,
}
}

fn withoutOperatorKindDecision(state: ?*anyopaque, term: SymbolicTerm) anyerror!FilterDecision
↪ {
  const rejected_kind: NamedOperatorKind = @enumFromInt(@intFromPtr(state.?) - 1);
  for (term.atoms) |atom| {
    switch (atom) {
      .gamma_matrix, .gamma_form, .gamma_action, .exterior_gamma_action,
      ↪ .clifford_product, .clifford_product_factor, .gamma_trace,
      ↪ .tensor_form_gamma_wedge, .tensor_form_gamma_map,
      ↪ .tensor_form_gamma_map_adjoint => if (rejected_kind == .gamma) return .reject,
      .metric_pair, .spinor_pair, .tensor_form_spinor_pair => if (rejected_kind ==
      ↪ .metric) return .reject,
      .epsilon => if (rejected_kind == .epsilon) return .reject,
      .structure_constant => if (rejected_kind == .structure_constant) return .reject,
      .product_identity, .cartan_product, .tensor_spinor_projection,
      ↪ .tensor_form_projection => if (rejected_kind == .projector) return .reject,
      .named_operator => |operator| if (operator.kind == rejected_kind) return .reject,
      else => {},
    }
  }
  return .undecided;
}

```

On-Shell-Code

This code consumes CFT-Code and handles on-shell string theory computations. That is, given a set of vertex Operators, it computes their string amplitudes from their Correlators, via prescriptions appropriate for the given string theory. We shall support all five superstrings: heterotic $SO(32)$, $E_8 \times E_8$, Type IIA, Type IIB and Type I, and bosonic string theory. In both open, closed and open-closed sectors. For free fields, we can derive the open, open-closed sector data from that of the closed sector given a boundary condition on the fundamental fields (doubling trick). Note that each requires us to define the appropriate ghost system Theory sectors. The superstring theories each come with the a distinct notion of a GSO projection.

SFT-Code

This module defines string field theory (SFT) brackets and vertices for open, closed and open-closed string fields theories (for all five superstrings and the bosonic string). As such, it applies conformal transforms defined by consistent local coordinate data on Operators and follows up by computing their OPE (in case of brackets) or Correlators in the case of vertices, integrated over for now abstract chains in the moduli space, with appropriate PCO insertions in the superstring case.

NLSM-Code

This is an overview of NLSM-Code.

- R4 Coefficient Technology
- NLSM API Requirements
- NLSM Backend And Lisp DSL
- NLSM Feature Request For CFT-Code And Tensor-Code

1 R4 Coefficient Technology

This note records the literature and implementation path for using NLSM-Code and the current Correlators machinery to compute the R^4 coefficients from the Polyakov path integral.

1.1 Literature Anchor

The local stringbook [2] uses the generalized Polyakov action with G , B , and Φ , expands about a background map in Riemann normal coordinates, and regularizes the worldsheet theory by dimensional regularization in $d = 2 - \epsilon$. The appendix fixes the renormalization scheme: bare couplings are written as $\mu^{-\epsilon}$ times finite couplings plus minimal-subtraction poles, and the beta function is determined by the $1/\epsilon$ counterterm. The Weyl anomaly is expressed through hatted beta functions, differing from ordinary beta functions by total derivative terms and the classical dilaton contribution.

The first old-paper layer is:

- Friedan [3] gives the general covariant renormalization framework: normal coordinates, metric coupling as a point in the space of target metrics, and beta functions as geometric tensors.
- Callan-Friedan-Martinec-Perry [4] gives the direct string-background statement: worldsheet conformal invariance produces the spacetime equations of motion.
- Tseytlin's anomaly and Weyl-invariance papers [5] [6] are the scheme references for the stringbook-style hatted Weyl-anomaly coefficients, G, B, Φ , and the minimal-subtraction organization.
- Metsaev-Tseytlin [7] checks two-loop equivalence including dilaton and antisymmetric tensor dependence.

For R^4 , the relevant perturbative fact is that type-II corrections begin at order α'^3 . In the worldsheet supersymmetric NLSM this is a four-loop divergence; on Ricci-flat Kahler targets the lower-loop terms vanish but the four-loop term does not.

- Gross-Witten [8] fixes the tree-level R^4 correction from four-graviton amplitudes.
- Grisaru-van de Ven-Zanon [9] gives the explicit $N = 1$ four-loop divergence structures.
- Grisaru-van de Ven-Zanon [10] shows that Ricci-flat Kahler targets are not finite at four loops.
- Grisaru-van de Ven-Zanon [11] gives the compact four-loop beta-function result for $N = 1, 2$ NLSMs.

- Park-Zanon [12] bridges the four-loop NLSM beta function to the on-shell α'^3 low-energy effective action.

For compact targets and later checks:

- Antoniadis-Ferrara-Minasian-Narain [13] relates R^4 couplings to Calabi-Yau reductions and Euler-number contributions.
- Liu-Minasian [14] is a useful modern check on $H^2 R^3$ and compactification tests on K3 and CY3.
- Grimm-Mayer-Weissenbacher [15] is a compactification-level reference for reducing known eight-derivative terms on CY3.
- Candelas-de la Ossa-Green-Parkes [16] is the first one-modulus CY3 target: the quintic has $h^{1,1} = 1$, so the local curvature expansion can be fitted against one Kahler volume variable before moving to multi-modulus examples.

1.2 Computation Target

The first target is not the full ten-dimensional action. It is a local counterterm calculation that produces the MS-scheme beta-function tensor at order α'^3 and identifies the invariant it comes from, modulo field redefinitions. The amplitude literature fixes normalization and the preferred string-frame invariant basis.

The output should therefore be:

- a stream of divergent local terms, grouped by loop order and tensor invariant;
- the $1/\epsilon$ counterterm $K_{\mu\nu}^{(1)}$ or the Kahler-potential counterterm in the $N = (2, 2)$ case;
- the beta function obtained from the stringbook MS rule [2];
- the hatted Weyl-anomaly coefficient after adding W_μ, L_μ, Φ terms;
- a field-redefinition map to the amplitude-normalized R^4 basis.

1.3 Required Technology

1. Regulated Polyakov path integral

Implement a scheme descriptor for the stringbook regularization [2]: $d = 2 - \epsilon$, MS poles, $\mu^{-\epsilon}$ coupling scaling, and the optional B -field epsilon-tensor prescription. The scheme object must own loop-integral families and the pole-extraction rule; the correlator should only emit local kernel factors.

2. Riemann-normal-coordinate expansion

Build an NLSM preset that lowers background fields to polynomial fluctuation vertices. The vertex stream should contain only the data used by the next stage: fluctuation fields, worldsheet derivatives, target tensors, and covariant derivatives of curvature, H , and Φ . Do not construct a giant expanded action term. Emit vertices lazily by order in ξ , derivative count, and loop budget.

3. Worldsheet field content

Start with the bosonic G, B, Φ NLSM to reproduce one- and two-loop checks. Add the $N = (1, 1)$ and $N = (2, 2)$ supersymmetric versions before attacking R^4 , because the type-II R^4 term is the four-loop supersymmetric sigma-model effect, and CY3/K3 require the complex/Kahler presentation.

4. Current-correlator integration

Reuse the existing streaming Wick architecture. The NLSM preset lowers RNC vertices to normal-ordered polynomial profiles in fluctuation fields. The generic correlator only contracts free fluctuation propagators and streams coefficient factors. Tensor factors and loop kernels stay opaque, just as current free-boson profile factors do.

5. Divergence extraction

Add a local counterterm collector after correlator evaluation. It should receive streamed diagrams, apply the selected dimensionally regulated kernel identities, discard finite parts unless requested, and canonicalize the remaining local tensor expressions.

6. Tensor canonicalization

The tensor layer must know the Riemann symmetries, Bianchi identities, covariant derivative commutators, Kahler index restrictions, and integration by parts. It also needs a field-redefinition quotient, because beta functions and spacetime actions are scheme-dependent.

7. Beta and Weyl-anomaly assembly

Use the stringbook relation [2] between $K^{(1)}$, beta functions, and hatted Weyl-anomaly coefficients. This should be a deterministic post-processing step over counterterm records, not embedded in diagram evaluation.

8. Normalization and invariant matching

Match against known low-loop beta functions first, then against the Gross-Witten four-graviton R^4 coefficient [8]. The beta-function engine should report its own MS-scheme tensor; the string-effective-action layer translates it to $t_8 t_8 R^4$, epsilon-epsilon terms, and compactification conventions.

1.4 Data Flow

The efficient data flow is:

1. background descriptor $\rightarrow$ RNC vertex iterator;
2. vertex tuple iterator $\rightarrow$ Wick/correlator stream;
3. correlator stream $\rightarrow$ regulated loop-kernel pole extractor;
4. pole terms $\rightarrow$ tensor canonicalizer;
5. counterterms $\rightarrow$ beta functions;
6. beta functions $\rightarrow$ Weyl-anomaly coefficients and action matching.

Every stage should be independently streamable. The only large combinatorics are vertex tuple enumeration and Wick branching; both must be chunkable by loop order, field content, and target-tensor signature.

1.5 Generalization To CY3 And K3

Nothing in the first implementation should assume a flat ten-dimensional target or a generic real Riemannian target only. The geometry layer should support multiple target presentations:

- real Riemannian tensors for universal R^4 checks;
- complex Kahler tensors for $N = (2, 2)$ CY3 computations;
- hyperkahler/K3 identities for $N = (4, 4)$ checks;
- product targets, so external spacetime and internal compact directions can be separated without changing the correlator.

The NLSM preset should therefore expose target geometry as data: dimension, holonomy constraints, complex structure, Kahler form, curvature families, and allowed index sectors. The same correlator and divergence extractor then work for R^4 , CY3, and K3; only the vertex generator and tensor quotient change.

For compact targets, target data must also include cohomology. The CY3 descriptor needs Hodge numbers, harmonic-form roles, intersection numbers, Chern-class pairings, and a volume model. The K3 descriptor can ship with the known Hodge diamond and hyperkahler form roles. These data do not affect the local Wick contractions. They affect the later projection of local curvature invariants onto moduli-space coordinates and large-volume observables.

The intended CY3 data flow is:

1. compute local curvature counterterms with the same NLSM kernels;
2. evaluate those local invariants on a supplied numerical Ricci-flat metric;
3. integrate/project them against the allowed harmonic forms and cycle data;
4. fit the result as an expansion in Kahler moduli, cycle volumes, or the overall volume.

The quintic should be the first integration target because $h^{1,1} = 1$. There is a single Kahler generator J , $\int J^3 = 5$, and therefore a one-variable large-volume fit for any scalar observable that survives the local R^4 computation [16].

1.6 Milestones

1. Reproduce the stringbook one-loop G, B, Φ beta functions in the MS scheme [2].
2. Reproduce the known two-loop metric and torsion checks, including the scheme dependence of the B -field epsilon prescription.
3. Implement $N = (1, 1)$ supersymmetric RNC vertices and verify the expected lower-loop cancellations.
4. Implement $N = (2, 2)$ Kahler superspace/component vertices and reproduce the four-loop Ricci-flat Kahler non-finiteness result.
5. Match the resulting order- α'^3 beta-function tensor to the R^4 invariant basis and calibrate normalization against the amplitude result.
6. Add K3 and CY3 cohomology descriptors, including the known K3 Hodge diamond.
7. Run the same machinery with CY3 and K3 tensor quotients, producing local correction terms ready for compactification integrals.
8. Use the quintic as the first numerical large-volume fit target [16].

2 NLSM API Requirements

This note records the exact API surface required by the R4 Coefficient Technology pipeline after inspecting the current CFT and tensor modules.

2.1 Existing Surface We Can Reuse

The usable runtime boundary is CFT Kernel `Call.MultiOp`:

```
pub const MultiOp = struct {
    operators: []const LocalOp,
    labels: *const LabelStore,
};

pub const LocalOp = struct {
    insertion: operators.OperatorInsertion,
    kind: operators.OperatorKindId,
    labels: theory.Id.LabelSpan,
    normal_order_group: u16 = 0,
};

pub const LabelValue = union(enum) {
    integer: i64,
    rational: RationalLabel,
    tensor: tensor.TensorExprId,
    symbol: u32,
};
```

The current Preset Shared Runtime already gives preset authors:

- opaque local builder tokens;
- coordinate, index, momentum, profile, target-point, profile-coefficient, tensor-projector, boundary-stack handles;
- profile presentations `position_space`, `fourier`, and `polynomial_rnc`;
- Wick rule terms containing scalar, coordinate, tensor, action, and residual factors;
- named Green kernels and differentiated coordinate kernels;
- zero-mode profile factors for Fourier, polynomial/RNC, and residual Gaussian presentations;
- product and boundary config merging for ordinary CFT/BCFT factors.

For NLSM, this means the free fluctuation correlator should not be a new generic correlator. The RNC path should lower vertices to explicit field occurrences with attached tensor/coefficient data. The elementary Wick pair is then $\xi\xi$, $\psi\psi$, or the corresponding free fluctuation pair, and the pair operation consumes both occurrences.

2.2 Immediate Gaps

The current API is insufficient in four exact places.

1. Tensor factors are flat-space oriented.

Current Wick tensor factors cover metric, momentum-index, momentum-pair, and projector variants. RNC and R^4 need curvature tensors, covariant derivatives, Kahler projections, holomorphic/antiholomorphic index families, epsilon tensors, and field-redefinition markers.

CY3/K3 calculations also need target-topology data that is not a local tensor factor: Hodge numbers, harmonic-form roles, intersection numbers, Chern-class pairings, and volume coordinates. This data is consumed by projection and large-volume fitting after local tensor canonicalization.

2. The coefficient layer has no local counterterm or pole object.

`Coefficient` can carry coordinate kernels, but the dimensional regularization step needs a stream of local pole terms: loop-integral family, $1/\epsilon^n$ pole, local derivative order, and target tensor expression.

3. Tensor-code is representation-theory oriented, not differential geometry oriented.

It can produce invariant bases for Lie-algebra representations, but it does not canonicalize Riemann tensors, Bianchi identities, Kahler restrictions, covariant-derivative commutators, integration by parts, or field redefinitions.

4. The current public preset root has no NLSM namespace.

`cft-code.zig` exports `FreeBoson`, `Bc`, `FreeFermion`, `EtaXi`, `product`, and `boundary`. NLSM should be exported as a new compact root namespace rather than as public shared-runtime internals.

2.3 Required Public NLSM API

The public root should follow the existing preset style:

```
pub const NLSM = preset_impl.NLSM;
```

The public constructors should be minimal:

```
pub const NLSM = struct {
    /// bosonic builds the regulated bosonic G,B,Phi sigma-model preset.
    pub const bosonic = nlsm.bosonic;

    /// supersymmetric builds N=(1,1), N=(2,2), or N=(4,4) sigma-model data.
    pub const supersymmetric = nlsm.supersymmetric;

    /// geometry groups target-geometry descriptor constructors.
    pub const geometry = nlsm.geometry;

    /// scheme groups regulator and subtraction-scheme constructors.
    pub const scheme = nlsm.scheme;

    /// counterterms exposes local pole extraction and beta-function assembly.
    pub const counterterms = nlsm.counterterms;
};
```

2.4 Geometry API

Geometry must be data, not behavior hidden inside the correlator. The NLSM vertex generator receives only a compact geometry descriptor.

```
pub const TargetGeometry = opaque {};
pub const GeometryHandle = enum(u32) { _ };
pub const TargetIndex = enum(u32) { _ };
pub const TensorAtom = enum(u32) { _ };

pub const GeometryKind = enum {
    real_riemannian,
```

```

    kahler,
    calabi_yau,
    hyperkahler,
    product,
};

pub const Holonomy = enum {
    generic,
    su,
    sp,
};

pub const GeometryConfig = struct {
    kind: GeometryKind,
    real_dimension: u16,
    complex_dimension: ?u16 = null,
    holonomy: Holonomy = .generic,
    topology: ?TopologyHandle = null,
};

pub const TopologyHandle = enum(u32) { _ };
pub const HodgeDiamondId = enum(u32) { _ };
pub const HarmonicFormId = enum(u32) { _ };

pub const HodgeNumber = struct {
    p: u8,
    q: u8,
    value: u16,
};

pub const HarmonicForm = struct {
    degree: u8,
    hodge_p: u8,
    hodge_q: u8,
    role: enum {
        kahler_modulus,
        complex_structure_modulus,
        holomorphic_volume,
        hyperkahler_triplet,
        other,
    },
};

pub const IntersectionNumber = struct {
    forms: []const HarmonicFormId,
    coefficient: RationalId,
};

pub const VolumeModel = union(enum) {
    none,
    cubic_kahler: struct { kahler_forms: []const HarmonicFormId },
    overall_volume: struct { symbol: u32 },
};

pub const LargeVolumeFitRequest = struct {
    target: GeometryHandle,
    observables: []const LocalCountertermKind,
    variables: []const HarmonicFormId,
    metric_samples: []const NumericalMetricSampleId,
};

pub const NumericalMetricSampleId = enum(u32) { _ };

pub fn realRiemannian(comptime dimension: u16) type;

```

```
pub fn kahler(comptime complex_dimension: u16) type;
pub fn calabiYau(comptime complex_dimension: u16) type;
pub fn k3() type;
pub fn quintic() type;
pub fn product(comptime factors: anytype) type;
```

Each generated geometry type should expose stable tensor atoms:

```
pub const tensor = struct {
    pub const metric: TensorAtom;
    pub const inverse_metric: TensorAtom;
    pub const riemann: TensorAtom;
    pub const covariant_derivative_riemann: TensorAtom;
    pub const h_flux: TensorAtom;
    pub const dilaton_derivative: TensorAtom;
    pub const kahler_form: ?TensorAtom;
    pub const complex_structure: ?TensorAtom;
    pub const holomorphic_volume_form: ?TensorAtom;
};
```

The essential point is that R^4 , CY3, and K3 all differ at the tensor quotient layer. They should not require different correlator code.

For compact targets they also differ at the topology/projection layer. The local counterterm kernel should not know Hodge diamonds or intersection numbers. Those data are attached to the target descriptor and used by the large-volume fit layer.

2.5 Scheme API

The regulator must model the stringbook scheme explicitly [2]:

```
pub const DimRegMS = struct {
    epsilon_symbol: u32,
    mu_symbol: u32,
    dimension: struct {
        base: u16 = 2,
        epsilon_sign: enum { minus, plus } = .minus,
    } = .{},
    subtract: enum { minimal, modified_minimal } = .minimal,
    b_field_epsilon: BFieldEpsilonScheme = .none,
};

pub const BFieldEpsilonScheme = union(enum) {
    none,
    frame_epsilon: struct { f_epsilon_symbol: u32 },
};

pub fn stringbookMS() DimRegMS;
```

This descriptor is consumed by loop-pole extraction and beta assembly. Wick rules should only emit named Green-kernel factors; they should not decide how dimensional regularization extracts poles.

2.6 RNC Vertex API

RNC expansion should be a streaming vertex generator. It should never build a full expanded action as one symbolic expression.

```
pub const RncRequest = struct {
    max_xi_order: u8,
```

```

    max_loop_order: u8,
    include_b_field: bool = false,
    include_dilaton: bool = false,
    worldsheet: WorldsheetModel,
};

pub const WorldsheetModel = enum {
    bosonic,
    n1_1,
    n2_2,
    n4_4,
};

pub const VertexId = u32;
pub const VertexField = struct {
    kind: FieldKind,
    target_index: TargetIndex,
    worldsheet_derivatives: WorldsheetDerivatives,
};

pub const FieldKind = enum {
    xi,
    psi,
    psit,
    auxiliary,
};

pub const WorldsheetDerivatives = packed struct {
    holomorphic: u4 = 0,
    antiholomorphic: u4 = 0,
};

pub const VertexTensor = struct {
    atom: TensorAtom,
    indices: []const TargetIndex,
    covariant_derivatives: u8 = 0,
};

pub const VertexSink = struct {
    pub fn beginVertex(id: VertexId, order: u8, scalar: anytype) !void;
    pub fn emitField(field: VertexField) !void;
    pub fn emitTensor(tensor: VertexTensor) !void;
    pub fn endVertex() !void;
};

pub fn streamRncVertices(
    comptime Geometry: type,
    scheme: nlscheme.DimRegMS,
    request: RncRequest,
    sink: anytype,
) !void;

```

This API gives the later diagram enumerator compact vertices with exactly the data it needs: fields to contract and target tensors to keep.

2.7 NLSM Diagram/Contraction API

The RNC calculation should not represent one interaction vertex as one composite local operator and then ask Wick contractions to return residual remainders. The vertex iterator already knows the fields inside a vertex, so the diagram engine should track field occurrences directly. A primitive pair contraction consumes both endpoints and emits only coefficient, coordinate, and tensor factors.

```

pub const VertexOccurrence = struct {
    vertex: VertexId,
    field_index: u16,
    field: VertexField,
};

pub const ContractionPair = struct {
    left: VertexOccurrence,
    right: VertexOccurrence,
};

pub const ContractionTerm = struct {
    scalar: ScalarFactorId,
    coordinate: coefficient.CoordinateFactor,
    tensor: GeometryTensorExprId,
};

pub fn emitContraction(
    scheme: nlscheme.DimRegMS,
    pair: ContractionPair,
    sink: anytype,
) !void;

```

Uncontracted occurrences at the end of a vacuum counterterm branch are not Wick residuals; they are an invalid branch unless the request explicitly asks for an insertion or external composite operator. Background tensors attached to vertices are carried by the branch accumulator, not by residual local operators.

The existing `wickCorrelator` residual API remains relevant to current free-boson profile/exponential presets. It is not a prerequisite for the RNC/NLSM R^4 pipeline.

2.8 Counterterm API

Counterterm extraction sits after correlator streaming. It consumes branches, applies scheme-dependent local pole identities, and emits compact local tensor terms.

```

pub const CountertermRequest = struct {
    loop_order: u8,
    pole_order: u8 = 1,
    local_operator: LocalCountertermKind,
    tensor_basis: TensorCanonicalization,
};

pub const LocalCountertermKind = enum {
    metric_beta,
    b_field_beta,
    dilaton_beta,
    kahler_potential_beta,
    scalar_effective_action,
};

pub const PoleTerm = struct {
    pole_order: u8,
    loop_order: u8,
    scalar: ScalarFactorId,
    tensor: GeometryTensorExprId,
    derivative_order: u8,
};

pub fn streamPoleTerms(
    scheme: nlscheme.DimRegMS,
    branches: anytype,

```

```

    request: CountertermRequest,
    sink: anytype,
) !void;

```

This must be stream-first. The four-loop $N = (2, 2)$ calculation will be too large to materialize as a complete symbolic expression.

2.9 Geometry Tensor Canonicalization API

Do not route Riemann canonicalization through the existing Lie-algebra invariant-basis API. Add a geometry tensor store with a small public boundary:

```

pub const GeometryTensorExprId = u32;

pub const TensorCanonicalization = struct {
    presentation: enum {
        real_riemannian,
        kahler,
        hyperkahler,
        product,
    },
    quotient: []const TensorRelationSet,
    field_redefinitions: FieldRedefinitionPolicy = .track,
};

pub const TensorRelationSet = enum {
    metric_compatibility,
    riemann_symmetry,
    first_bianchi,
    second_bianchi,
    kahler_type,
    kahler_bianchi,
    hyperkahler_identities,
    integration_by_parts,
    equations_of_motion,
};

pub const FieldRedefinitionPolicy = enum {
    keep_all,
    quotient,
    track,
};

pub fn canonicalizeGeometryTensor(
    geometry: GeometryHandle,
    expr: GeometryTensorExprId,
    options: TensorCanonicalization,
    sink: anytype,
) !void;

```

The existing Tensor-Code remains useful for spinor, gamma-matrix, and invariant tensor structures. Differential-geometry tensors need their own canonicalizer.

2.10 Beta Function API

The beta assembly should be deterministic post-processing over pole terms:

```

pub const BetaRequest = struct {
    scheme: nlscheme.DimRegMS,
    target: LocalCountertermKind,

```

```

include_hatted_weyl_terms: bool = true,
field_frame: FieldFrame = .minimal_subtraction,
};

pub const FieldFrame = enum {
    minimal_subtraction,
    string_frame,
    einstein_frame,
    amplitude_matched,
};

pub const BetaTerm = struct {
    target: LocalCountertermKind,
    alpha_prime_order: u8,
    tensor: GeometryTensorExprId,
    field_redefinition_class: u32,
};

pub fn assembleBetaFunctions(
    poles: anytype,
    request: BetaRequest,
    sink: anytype,
) !void;

```

This API encodes the stringbook rule [2] that the MS beta function is determined by the $1/\epsilon$ counterterm, while hatted Weyl coefficients include total-derivative and dilaton terms.

2.11 Minimum Implementation Order

1. Add NLSM root export and empty constructor namespace.
2. Add geometry descriptors and tensor atoms for real, Kahler, CY3, and K3 targets.
3. Add `streamRncVertices` for bosonic G -only vertices through quartic order in ξ .
4. Add a counterterm sink that can recognize one-loop poles and reproduce the stringbook $R_{\mu\nu}$ beta function [2].
5. Add the contraction-only RNC diagram enumerator over vertex occurrences.
6. Add supersymmetric field kinds and reproduce lower-loop cancellations.
7. Add geometry tensor canonicalization for Kahler and hyperkahler quotients.
8. Run the four-loop $N = (2, 2)$ Ricci-flat Kahler target calculation and match the R^4 coefficient modulo field redefinitions.

3 NLSM Backend And Lisp DSL

This note reviews and replaces the first backend sketch. The corrected design is a compact calculation DSL backed by generated, streaming Zig kernels. The physics anchor is the stringbook dimensional-regularization/minimal-subtraction scheme for the Polyakov path integral, together with the older sigma-model literature on background fields, Weyl anomaly coefficients, and four-loop supersymmetric NLSM beta functions [2] [3] [4] [5] [6] [7] [11] [10] [9] [8].

3.1 Reference Map

Each paper below must translate to a concrete table, kernel, or check. If a reference does not constrain code, it does not belong in the implementation note.

- Stringbook [2] gives the working scheme: the Polyakov path integral with G, B, Φ , dimensional regularization in $2-\epsilon$, minimal subtraction, $G^\epsilon = \mu^{-\epsilon}(G + \sum \epsilon^{-n} K^{(n)})$, beta functions from the simple pole $K^{(1)}$, the B -field epsilon-frame ambiguity, and hatted Weyl coefficients with W_μ, L_μ, Φ total derivative terms. Code translation: `SchemeRow(:stringbook-ms)`, `PoleRow`, `BetaAssembler`, `WeylAnomalyRow`, and a `BFieldEpsilonPolicy`.
- Friedan [3] is the background-field/RNC renormalization anchor: NLSM couplings are geometric tensors, counterterms are local tensor functionals, and normal-coordinate algorithms organize the perturbation series covariantly. Code translation: `RncVertexPlan` streams vertices from geometry descriptors; `GeometryTensorWord` stores counterterms as local tensor words; `CouplingSpaceOp` records metric variations used by beta assembly.
- Callan-Friedan-Martinec-Perry [4] explains why the beta functions of G, B, Φ are the spacetime equations of motion for string backgrounds. Code translation: jobs target physical rows such as `:metric-beta`, `:b-beta`, `:dilaton-beta`, and `:effective-action` instead of generic correlator output.
- Tseytlin's conformal-anomaly paper [5] fixes the distinction between RG beta functions and Weyl-anomaly coefficients in curved backgrounds. Code translation: keep `BetaRow` separate from `WeylAnomalyRow`; do not bake W_μ, L_μ , or dilaton improvements into the contraction kernel.
- Tseytlin's Weyl-invariance paper [6] emphasizes scheme dependence and field redefinitions in the map from sigma-model beta functions to string equations. Code translation: `BasisPolicy` must include `:field-redefinitions` and `:eom` quotients, and every emitted row carries a scheme id.
- Metsaev-Tseytlin [7] compares two-loop sigma-model Weyl invariance with string effective equations including B and Φ , and makes field-frame dependence explicit. Code translation: `ActionMatcher` consumes canonical beta rows plus a `FieldFrame` and emits effective-action tensor rows; $H = dB$ and dilaton derivative atoms are part of the geometry descriptor.
- Grisaru-van de Ven-Zanon PLB [11] gives the compact four-loop superspace beta-function result for $N = 1, 2$ NLSMs. Code translation: this is the first high-loop regression target for `define-nlsm-job` with `:n1-1` and `:n2-2` worldsheet presets.
- Grisaru-van de Ven-Zanon on Ricci-flat Kahler targets [10] shows that Ricci-flat Kahler manifolds are not finite at four loops. Code translation: `TargetPreset(:cy3)` must impose Kahler type and Ricci-flatness but still allow the four-loop local tensor to survive in the `:kahler-beta` job.
- Grisaru-van de Ven-Zanon on four-loop divergences [9] supplies explicit divergence structures and diagrammatic checks. Code translation: `inspect-nlsm` must expose vertices, primitive diagram classes, pole signatures, and uncanonicalized tensor words for term-by-term comparison.
- Gross-Witten [8] fixes the amplitude-normalized R^4 correction and its implication that generic Ricci-flat targets are corrected while special Kahler holonomy changes the answer. Code translation: `ActionMatcher(:r4)` maps beta rows to the R^4 action basis and provides the normalization check for type-II jobs.
- Liu-Minasian [17] gives duality and B -field tests for higher-derivative couplings. Code translation: `TargetPreset(:k3)` and `TargetPreset(:cy3)` should share the same R^4 kernel but use different quotient policies and compactification checks.

- Policastro-Tsimpis [18] clarifies purified R^4 invariants and the removal of ambiguous Ricci/EOM terms. Code translation: `BasisPolicy(:r4)` has an on-shell/purified mode that quotients Ricci, scalar-curvature, integration-by-parts, and field-redefinition terms before comparing to amplitudes.
- Candelas-de la Ossa [19] and Candelas-de la Ossa-Green-Parkes [16] fix the CY moduli and quintic test case. Code translation: compact target descriptors need `CohomologyRow`, `IntersectionRow`, and `LargeVolumeFitPlan`. The quintic preset has $h^{1,1} = 1$, so it is the first numerical metric target: fit local curvature integrals against one Kahler volume parameter before moving to multi-modulus CY3 examples.

3.2 Review Findings

The previous proposal had the right backend direction but the wrong authoring surface.

- Good: generated Zig fixtures, streaming kernels, descriptor tables, and a contraction-only RNC path.
- Good: CY3 and K3 should alter target descriptors and tensor quotients, not replace the correlator engine.
- Bad: the Lisp examples exposed too much per-model boilerplate: explicit propagator kernels, repeated index declarations, manual background legs, and full RNC source declarations.
- Bad: public Zig functions such as `streamVertices` and `streamDiagrams` are useful internally, but too wide as the main user-facing API.

The user wants to say what calculation is being done. Zig wants a compact static job plan with interned ids, flat arrays, and no symbolic tree materialization.

3.3 Correct Backend

The backend is a generated kernel plan:

```
Lisp job form
-> normalized NLSM descriptor
-> generated Zig fixture tables
-> generated dispatch by job id
-> streaming kernels
-> pole, beta, or action rows
```

The performance-critical kernels are:

1. `vertex_plan`: expands standard RNC vertex families by order and field content.
2. `diagram_plan`: enumerates vertex multisets and quantum occurrence pairings.
3. `contract`: consumes fluctuation pairs such as $\xi\xi$, $\psi\psi$, or mixed supersymmetric free pairs.
4. `pole`: maps coordinate-kernel signatures to local $1/\epsilon^n$ pole rows in the selected scheme.
5. `canon`: canonicalizes geometry tensor words in the requested quotient.
6. `assemble`: converts simple poles to beta-function or local-action rows.

The RNC Wick operation has no residual. A branch contains quantum occurrences to be paired, background legs to keep, tensor atoms to multiply, and compact coordinate/scalar ids. Pairing a quantum occurrence consumes it. Remaining background legs are the local counterterm operator.

3.4 Data Shape

The generated descriptor should use flat tables and integer ids.

```
pub const JobRow = struct {
    model: ModelId,
    scheme: SchemeId,
    target_op: TargetOp,
    max_loops: u8,
    alpha_prime_order: u8,
    vertex_policy: VertexPolicyId,
    basis: BasisPolicyId,
    emit: EmitFlags,
};

pub const ModelRow = struct {
    target: TargetId,
    worldsheet: WorldsheetId,
    field_set: FieldSetId,
    default_scheme: SchemeId,
    topology: TopologyId,
};

pub const TopologyRow = struct {
    hodge: HodgeDiamondId,
    first_form: u32,
    form_len: u16,
    first_intersection: u32,
    intersection_len: u16,
    volume_model: VolumeModelId,
};

pub const FormRow = struct {
    degree: u8,
    hodge_p: u8,
    hodge_q: u8,
    multiplicity: u16,
    role: FormRole,
};

pub const IntersectionRow = struct {
    first_form: u32,
    form_len: u8,
    coefficient: RationalId,
};

pub const VertexRow = struct {
    first_quantum: u32,
    quantum_len: u16,
    first_background: u32,
    background_len: u16,
    first_tensor: u32,
    tensor_len: u16,
    first_scalar: u32,
    scalar_len: u16,
    alpha_prime_power: i16,
    symmetry_denominator: u32,
};
```

The hot branch frame should be mutable and small:

```
pub const BranchState = struct {
    live_quantum: OccurrenceBitSet,
    pairs: PairStack,
```

```

scalar: ScalarAccum,
coordinate: CoordinateAccum,
tensor: TensorWordAccum,
background: BackgroundAccum,
};

```

No kernel should allocate an expanded action, a full diagram forest, or a full symbolic sum. Every stage streams rows to the next stage or to an inspection sink.

3.5 Public Zig API

The public API should be job-oriented. Fine-grained streams stay internal until there is a concrete external caller.

```

pub const NLSM = struct {
    /// run streams the requested stage for a generated NLSM job.
    pub fn run(job: JobId, stage: Stage, sink: anytype) !void;

    /// inspect streams selected intermediate rows for debugging or paper checks.
    pub fn inspect(job: JobId, request: InspectRequest, sink: anytype) !void;
};

```

The generated ABI can mirror this:

```

nlsml_run(job_id, stage_id, sink)
nlsml_inspect(job_id, selector_id, sink)

```

Everything else should remain private until another module needs it.

3.6 Compact Lisp DSL

The normal user should write one compact job form. Defaults come from named target/worldsheet presets.

```

(define-nlsm-job bosonic-one-loop-g
  (target :riemannian :dim d)
  (worldsheet :bosonic)
  (scheme :stringbook-ms)
  (compute :metric-beta :loops 1)
  (basis :riemannian)
  (emit :poles :beta))

```

The R^4 CY3 calculation should be similarly short:

```

(define-nlsm-job cy3-r4
  (target :cy3 :dim 3
    :cohomology (hodge ((0 0 1)
      (1 1 h11)
      (2 1 h21) (1 2 h21)
      (2 2 h11)
      (3 0 1) (0 3 1)
      (3 3 1)))
    :forms ((J a :h11 :kahler)
      (chi alpha :h21 :complex)
      (Omega :h30 :holomorphic-volume)))
  (worldsheet :n2-2)
  (scheme :stringbook-ms)
  (compute :kahler-beta :loops 4 :alpha-prime 3)
  (basis :r4 :quotient (:kahler :ricci-flat :ibp :field-redefinitions))
  (emit :poles :beta :action))

```

K3 changes only the target preset and quotient:

```
(define-nlsm-job k3-r4-check
  (target :k3
    :cohomology (hodge ((0 0 1)
                        (1 0 0) (0 1 0)
                        (2 0 1)
                        (1 1 20)
                        (0 2 1)
                        (2 1 0) (1 2 0)
                        (2 2 1)))
    :forms ((omega A :h11 :kahler)
            (Omega :h20 :holomorphic-two-form)))
  (worldsheet :n4-4)
  (scheme :stringbook-ms)
  (compute :metric-beta :loops 4 :alpha-prime 3)
  (basis :r4 :quotient (:hyperkahler :ricci-flat :ibp :field-redefinitions))
  (emit :poles :beta))
```

The DSL compiler expands these forms to:

- standard fields for the selected worldsheet model;
- standard propagators for the selected scheme;
- standard RNC vertex families through the requested loop order;
- default background operator legs for the requested counterterm;
- target-specific tensor canonicalization relations.

The user should not have to declare ξ , $G_{\mu\nu}$, the propagator, or $\partial X \bar{\partial} X$ for common metric and Kahler beta calculations.

For a concrete one-modulus starting point, the quintic should be a named target preset:

```
(define-nlsm-job quintic-large-volume-r4
  (target :quintic
    :cohomology (hodge ((0 0 1)
                        (1 1 1)
                        (2 1 101) (1 2 101)
                        (2 2 1)
                        (3 0 1) (0 3 1)
                        (3 3 1)))
    :forms ((J :h11 :kahler)
            (chi alpha :h21 :complex)
            (Omega :h30 :holomorphic-volume))
    :intersection ((J J J) 5)
    :c2 ((J) 50)
    :volume (cubic :kahler J))
  (worldsheet :n2-2)
  (scheme :stringbook-ms)
  (compute :kahler-beta :loops 4 :alpha-prime 3)
  (fit :large-volume :variables (vol J) :observables (:curvature-integrals))
  (emit :beta :action :fit-data))
```

This separates two tasks. The local NLSM kernel computes a curvature expansion. Once a numerical Ricci-flat metric is supplied, the fitting layer integrates emitted local invariants against harmonic-form and cycle data, then fits the result as an expansion in Kahler parameters or the overall volume.

The target form declarations are intentionally topology-level data. They tell the backend which moduli exist and which projections are legal. They do not change the local $\xi\xi$ Wick kernel.

3.7 Reusable Models

If several jobs share one model, define the model once:

```
(define-nlsm-model type-ii-cy3
  (target :cy3 :dim 3 :cohomology cy3-cohomology)
  (worldsheet :n2-2)
  (scheme :stringbook-ms))

(define-nlsm-job cy3-r4
  (model type-ii-cy3)
  (compute :kahler-beta :loops 4 :alpha-prime 3)
  (basis :r4 :quotient (:kahler :ricci-flat :ibp :field-redefinitions))
  (emit :beta :action))
```

This is the authoring path we should optimize.

3.8 Inspection DSL

Manual detail belongs in inspection forms, not in normal jobs.

```
(inspect-nlsm cy3-r4
  (vertices :order 4)
  (diagrams :topology :primitive)
  (poles :tensor (r4 :kahler)))
```

For term-by-term checks against old papers, allow a low-level vertex fixture:

```
(define-nlsm-fixture metric-rnc-2
  (model bosonic-riemannian)
  (vertex
    (:quantum (xi rho) (xi sigma))
    (:background (dX mu) (dbarX nu))
    (:tensor (riemann mu rho sigma nu))
    (:symmetry 3)))
```

This fixture path is for debugging the generator. It is not the main calculation language.

3.9 Descriptor Defaults

Named presets should carry the boilerplate:

- `:stringbook-ms`: $d = 2 - \epsilon$, minimal subtraction, simple-pole beta extraction, hatted Weyl anomaly option, and the stringbook B -field epsilon-frame convention.
- `:riemannian`: real vector indices, Levi-Civita connection, Riemann symmetries, Bianchi relations, metric compatibility.
- `:cy3`: complex dimension 3, Kahler index type, Ricci flatness, $SU(3)$ holonomy, holomorphic volume form when requested, and user-specified Hodge numbers/forms/intersections.
- `:k3`: complex dimension 2, hyperkahler quotient, Ricci flatness, $Sp(1)$ holonomy, and the known K3 Hodge diamond.
- `:quintic`: CY3 preset with $h^{1,1} = 1$, $h^{2,1} = 101$, one Kahler generator J , $\int J^3 = 5$, and a one-variable large-volume fit.
- `:bosonic`, `:n1-1`, `:n2-2`, `:n4-4`: standard fluctuation field sets, free contractions, signs, and supersymmetry selection rules.

The old NLSM and string effective-action literature makes two design constraints unavoidable: scheme data must be explicit, and field redefinition/equation-of-motion equivalence must be part of the basis policy [6] [7] [11] [8].

3.10 What Zig Must Make Fast

The fast path is not Lisp macro expansion. It is the generated Zig job:

- enumerate only vertex multisets compatible with loop order and external operator;
- pair only compatible quantum occurrence types;
- represent tensor products as interned tensor words with small ids;
- cache canonical tensor words by target quotient;
- reduce coordinate kernels by signature tables, not expression matching;
- stream pole rows immediately;
- assemble beta/action rows after canonicalization.
- project local curvature invariants onto harmonic-form and intersection data only after the local tensor result has been streamed and canonicalized.

The key cache keys are:

```
const VertexCacheKey = struct { model: ModelId, loops: u8, op: TargetOp };
const DiagramCacheKey = struct { job: JobId, signature: OccupancySignature };
const PoleCacheKey = struct { scheme: SchemeId, coordinate: CoordinateSignature };
const CanonCacheKey = struct { target: TargetId, basis: BasisPolicyId, word: TensorWordId };
const FitCacheKey = struct { target: TargetId, observable: ObservableId, volume: VolumeModelId
↪ };
```

This layout leaves room for parallelization by vertex multiset, primitive topology, coordinate signature, or canonicalization bucket.

3.11 Implementation Order

1. Build the descriptor compiler for `define-nlsm-model` and `define-nlsm-job`.
2. Generate `JobRow`, `ModelRow`, and preset target/worldsheet tables.
3. Implement the bosonic one-loop `:metric-beta` job and reproduce the stringbook $R_{\mu\nu}$ result in the MS scheme.
4. Add `inspect-nlsm` for vertices, diagrams, and poles.
5. Add `:n1-1` and `:n2-2` field presets and reproduce the four-loop Ricci-flat Kahler non-finiteness checks [11] [10] [9].
6. Add CY3/K3 topology descriptors: Hodge diamonds, harmonic-form roles, intersection data, and volume models. Use K3's known Hodge diamond and the one-modulus quintic as the first CY3 target [16].
7. Add CY3/K3 tensor quotients and match the R^4 action basis against amplitude and duality checks [8] [17] [18].

4 NLSM Feature Request For CFT-Code And Tensor-Code

This note records the feature requests that the NLSM R^4 , CY3, and K3 calculations impose primarily on CFT Kernel, with a possible later extraction path into Tensor-Code if the geometry reducer proves reusable. The main requirement is to keep the fast free-field contraction machinery from Free Boson Preset and Free Fermion Preset, while adding enough type information that Kahler and Calabi-Yau targets can be expressed without ad hoc conventions in user code.

4.1 Boundary Between Modules

The right split is:

- **cft-code** owns local field occurrences, normal-order groups, coordinate kernels, and Wick traversal;
- **nlsm-code** owns RNC or Kahler-normal-coordinate vertex generation, the first local tensor canonicalizer, regularization bookkeeping, and beta-function projection;
- **tensor-code** is only a later home for the geometry reducer if the same tensor identities are needed outside NLSM.

cft-code should not know curvature identities, Bianchi identities, or Hodge data. It only needs enough structure to reject impossible contractions early and to pass typed tensor labels to the sink. The first target-space tensor reducer should live directly in **nlsm-code** as a streamed sink over Wick branches. **tensor-code** should only absorb parts of that reducer later if a second client appears.

4.2 CFT-Code Requests

At the level of **cft-code**, the right abstraction is not "geometry", but a small extension of the existing preset-handle and Wick-rule vocabulary.

4.2.1 Typed Index Sorts

The current preset shared runtime has one generic `Handle.Index`. For NLSM on real, Kahler, CY3, and K3 targets, this is too weak. We need a compact index sort tag attached to each target-space index handle.

```
pub const IndexSort = enum(u8) {
    real_tangent,
    holomorphic_tangent,
    antiholomorphic_tangent,
    real_cotangent,
    holomorphic_cotangent,
    antiholomorphic_cotangent,
};

pub const TargetIndex = packed struct(u32) {
    raw: u24,
    sort: IndexSort,
};
```

This should stay a handle, not a full symbolic object. The only job of the sort tag inside **cft-code** is to make primitive contractions and primitive tensor factors type-safe.

The rule for choosing the sort is:

- use `real_tangent` for generic Riemannian RNC;
- use `holomorphic_tangent` and `antiholomorphic_tangent` for Kahler, CY3, and K3 fluctuation fields;
- add cotangent sorts only when the local NLSM tensor reducer needs explicit variance and it is cheaper to carry that variance directly than to reconstruct it later.

For the first implementation, tangent sorts are enough if metric insertions record which slots are raised or lowered.

4.2.2 Primitive Tensor-Factor Kinds

The current primitive tensor factors are flat-space oriented. NLSM needs a few more primitive factor families, but still at the level of handles:

```
pub const TensorFactor = union(enum) {
    none,
    metric: struct { left: LabelRef, right: LabelRef },
    hermitian_metric: struct { holo: LabelRef, antiholo: LabelRef },
    inverse_metric: struct { left: LabelRef, right: LabelRef },
    kahler_form: struct { holo: LabelRef, antiholo: LabelRef },
    complex_structure: struct { left: LabelRef, right: LabelRef },
    tensor_atom_slot: struct {
        atom: u32,
        slots: []const LabelRef,
    },
};
```

The important addition is `tensor_atom_slot`: a compact way for a preset or NLSM vertex lowerer to say "this Wick branch contributes one external tensor atom with these free target indices." That is a better abstraction than teaching `cft-code` what a Riemann tensor is.

4.2.3 Index-Compatible Contractions

Primitive contraction rules should be able to reject a branch before sink emission when index sorts are incompatible.

For Kahler targets:

- $\xi^i \xi^j$ is forbidden;
- $\xi^{\bar{i}} \xi^{\bar{j}}$ is forbidden;
- $\xi^i \xi^{\bar{j}}$ is allowed;
- $\psi^i \psi^j$ and $\psi^{\bar{i}} \psi^{\bar{j}}$ depend on the chosen chiral field family, but the sort check is still local.

This should be implemented as metadata on the primitive pair rules, not as a dynamic geometry system inside the correlator.

```
pub const PairIndexConstraint = enum(u8) {
    none,
    same_sort,
    conjugate_complex_sort,
};
```

4.2.4 Undifferentiated Free-Boson Rules

For bosonic RNC we need the free-boson preset to expose the missing primitive pairings involving the explicit X field:

- $X \ X;$
- $dX \ X;$
- $dX_t \ X.$

These are required because the RNC vertices contain undifferentiated fluctuations ξ , not only derivatives or plane waves.

4.2.5 External Tensor Labels

The current `kernel.Call.LabelValue` already supports `tensor: =tensor.TensorExprId`. That is the right hook. The missing step is a stable way for preset authors or NLSM lowering code to assign typed index slots to that tensor expression so that the sink knows how the free indices are ordered.

That should be one compact side table owned by `nlsm-code` or the preset runtime, not another symbolic layer inside `cft-code`.

4.3 NLSM Geometry Reducer Requests

At the first implementation stage, the right abstraction is not a new full differential-geometry CAS and not an immediate `tensor-code` extension. It is a small target-geometry reducer in `nlsm-code` that consumes the `cft-code` stream and canonicalizes only the local tensor monomials needed by the sigma-model computation.

4.3.1 Index-Slot Schema

Each tensor atom should declare the sort of each slot.

```
pub const TensorSlotSort = enum(u8) {
    real_tangent,
    holomorphic_tangent,
    antiholomorphic_tangent,
    real_cotangent,
    holomorphic_cotangent,
    antiholomorphic_cotangent,
};

pub const TensorAtomDecl = struct {
    name: []const u8,
    slot_sorts: []const TensorSlotSort,
    symmetry: SlotSymmetryId,
};
```

Examples:

- $g_{i\bar{j}}$ has slots (`holomorphic_cotangent`, `antiholomorphic_cotangent`);
- $R_{i\bar{j}k\bar{l}}$ has four cotangent slots alternating holomorphic and antiholomorphic types;
- Ω_{ijk} on a CY3 has three holomorphic cotangent slots;
- the K3 holomorphic two-form has two holomorphic cotangent slots.

This is the main place where index types should be chosen properly: by tensor slot declaration in the NLSM reducer, not by user naming convention.

4.3.2 Geometry Flavor

The NLSM tensor reducer should be parameterized by a compact geometry flavor:

```
pub const GeometryFlavor = enum(u8) {
    generic_riemannian,
    kahler,
    calabi_yau,
    hyperkahler,
};
```

This controls which local identities are legal:

- generic Riemannian: metric symmetry, Riemann antisymmetry, Bianchi;
- Kahler: only mixed metric components, only $R_{i\bar{j}k\bar{l}}$ -type curvature components survive;
- Calabi-Yau: Kahler plus Ricci-flat and $c_1 = 0$ -compatible reductions;
- hyperkahler: Calabi-Yau in complex dimension two plus the stronger K3 structure where needed.

This does not require topology or Hodge data. It is local differential geometry only.

4.3.3 Minimal Canonicalization Surface

The first NLSM reducer should stop at:

- slot-sort checking;
- dummy-index renaming within fixed sorts;
- metric and Hermitian-metric contraction;
- Kahler vanishing rules from slot sorts;
- Riemann and covariant-derivative symmetry canonicalization;
- Ricci and scalar-curvature detection.

It should not try to solve large-volume fitting, Hodge decomposition, or cycle intersection logic. Those belong in `nlsm-code` after local tensor terms have already been produced.

The intended data flow is:

```
cft-code Wick stream
-> nlsm-code tensor-term sink
-> local canonicalizer / hash-cons table
-> coefficient accumulation
-> pole extraction
-> beta-function projection
```

4.4 Recommended Public Shape

The new public shape should remain minimal.

For `cft-code`:

```
pub const Handle = struct {
    pub const Coord = enum(u32) { _ };
    pub const BoundaryCoord = enum(u32) { _ };
    pub const Index = enum(u32) { _ };
    pub const TypedIndex = TargetIndex;
};
```

New operator builders for NLSM-facing presets should accept `TypedIndex`. Old presets can keep using `Index` and default to `real_tangent` lowering.

For `nlsm-code`:

```
pub const GeometryFlavor = geometry.GeometryFlavor;
pub const TensorAtomDecl = geometry.TensorAtomDecl;
pub const TensorSlotSort = geometry.TensorSlotSort;
```

Keep these as declarations and handles. Do not expose internal rewrite engines, caches, or canonicalization tables.

If a later extraction into `tensor-code` becomes justified, the public shape above is already the right minimal interface to move.

4.5 Non-Goals

This feature request deliberately does not ask `cft-code` to own:

- Kahler-normal-coordinate expansion;
- Polyakov action expansion;
- loop-integral evaluation;
- dimensional-regularization poles;
- beta-function assembly;
- CY3 or K3 cohomology descriptors.

Those belong to NLSM API Requirements and the NLSM-specific backend. The role of `cft-code` is to make typed free-field contractions cheap. The first role of `nlsm-code` is to make the resulting local tensor expressions canonical.

This feature request also does not ask `tensor-code` to immediately absorb the geometry reducer. That move should wait until the reducer is implemented and we know which pieces are genuinely reusable.

4.6 Immediate Implementation Order

1. Add `X X`, `dX X`, and `dXt X` to the free-boson preset.
2. Add a typed-index handle in the preset shared runtime with a real default.
3. Add pair-rule index-compatibility metadata to the Wick descriptor path.
4. Add tensor-atom slot declarations and slot-sort checking in `nlsm-code`.
5. Add Kahler and Calabi-Yau geometry flavors to the NLSM tensor reducer.
6. Lower NLSM RNC vertices onto the free-field backend.
7. Revisit whether any reducer pieces belong in `tensor-code`.

Pure-Spinor-Code

This module computes OPE and Correlators in the CFT defined by the appropriately regularized pure spinor NLSM on $PSU(2, 2|4)$.

Tests

A String-Code

A.1 API

```
test "top-level API exposes tensor and cft boundaries" {
  const testing = @import("std").testing;
  try testing.expect(@hasDecl(@This(), "tensor"));
  try testing.expect(@hasDecl(@This(), "cft"));
  try testing.expect(@hasDecl(cft, "FreeBoson"));
  try testing.expect(@hasDecl(cft, "Bc"));
  try testing.expect(@hasDecl(cft, "FreeFermion"));
  try testing.expect(@hasDecl(cft, "EtaXi"));
  try testing.expect(@hasDecl(cft, "product"));
  try testing.expect(@hasDecl(cft, "boundary"));
  try testing.expect(@hasDecl(cft.FreeBoson, "make"));
  try testing.expect(@hasDecl(cft.FreeBoson, "boundary"));
  try testing.expect(@hasDecl(cft.Bc, "sphere"));
  try testing.expect(@hasDecl(cft.Bc, "diskBoundary"));
  try testing.expect(@hasDecl(cft.FreeFermion, "sphere"));
  try testing.expect(@hasDecl(cft.EtaXi, "sphere"));
  try testing.expect(!@hasDecl(cft.FreeBoson, "Config"));
  try testing.expect(!@hasDecl(cft.FreeBoson, "BoundaryConfig"));
  try testing.expect(!@hasDecl(cft.Bc, "SphereConfig"));
  try testing.expect(!@hasDecl(cft.Bc, "DiskConfig"));
  try testing.expect(!@hasDecl(cft.FreeFermion, "SphereConfig"));
  try testing.expect(!@hasDecl(cft.EtaXi, "SphereConfig"));
  try testing.expect(!@hasDecl(cft, "Handle"));
  try testing.expect(!@hasDecl(cft, "scalars"));
  try testing.expect(!@hasDecl(cft, "freeBoson"));
  try testing.expect(!@hasDecl(cft, "bcSphere"));
  try testing.expect(!@hasDecl(cft, "freeFermionSphere"));
  try testing.expect(!@hasDecl(cft, "etaXiSphere"));
  try testing.expect(!@hasDecl(cft, "freeBosonBoundary"));
  try testing.expect(!@hasDecl(cft, "bcDiskBoundary"));
  try testing.expect(!@hasDecl(cft, "presets"));
  try testing.expect(!@hasDecl(cft, "kernel"));
  try testing.expect(!@hasDecl(cft, "theory"));
  try testing.expect(!@hasDecl(cft, "expressions"));
}

test "tensor context interns projected realization requests without expansion" {
  const testing = @import("std").testing;

  var ctx = try tensor.Context.init(testing.allocator);
  defer ctx.deinit();

  const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
  const so10_again = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
  try testing.expectEqual(so10.value, so10_again.value);

  const vector = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 0 } });
  const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
  const vector_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 1 } });
}
```

```

const vector_spinor_again = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 1 }
↪ });
try testing.expectEqual(vector_spinor.value, vector_spinor_again.value);
try testing.expectEqual(@as(u128, 10), ctx.irrepMetadata(vector)?.dimension.?);
try testing.expectEqual(@as(u128, 16), ctx.irrepMetadata(spinor)?.dimension.?);
try testing.expectEqual(@as(u128, 144), ctx.irrepMetadata(vector_spinor)?.dimension.?);
const conjugate_spinor = try ctx.dualIrrep(spinor);
try testing.expect(conjugate_spinor.value != spinor.value);
try testing.expectEqual(spinor.value, (try ctx.dualIrrep(conjugate_spinor)).value);
try testing.expectError(error.InvalidDynkinRank, ctx.registerIrrep(so10, .{ .dynkin = &.{
↪ 1, 0 } }));

const su2 = try ctx.registerAlgebra(.{ .simple = .{ .family = .a, .rank = 1 } });
const fundamental = try ctx.registerIrrep(su2, .{ .dynkin = &.{1} });
try testing.expectError(error.MixedRealizationAlgebras,
↪ ctx.registerRealization(tensor.RealizationSpec.projected(vector_spinor,
↪ &.{fundamental}, &.{
    .init(.vector, "m"),
    .init(.spinor, "a"),
}, 0)));

const channel = tensor.RealizationChannel.first(vector_spinor);
const realization = try
↪ ctx.registerRealization(tensor.RealizationSpec.projectedChannel(channel, &.{ vector,
↪ spinor }, &.{
    .init(.vector, "m"),
    .init(.spinor, "a"),
}).named("S010.vector_spinor"));
const realization_again = try
↪ ctx.registerRealization(tensor.RealizationSpec.projectedChannel(channel, &.{ vector,
↪ spinor }, &.{
    .init(.vector, "m"),
    .init(.spinor, "a"),
}).named("S010.vector_spinor"));
try testing.expectEqual(realization.value, realization_again.value);

const leg = tensor.ExternalLeg.realized(realization, &.{
    .init(.vector, "m1"),
    .init(.spinor, "a1"),
});
const basis = try ctx.invariantBasis(.{
    .algebra = so10,
    .external_legs = &.{ leg, leg, leg, leg, leg, leg },
});
const basis_again = try ctx.invariantBasis(.{
    .algebra = so10,
    .external_legs = &.{ leg, leg, leg, leg, leg, leg },
});
try testing.expectEqual(basis.value, basis_again.value);

try testing.expectError(error.MixedInvariantAlgebras, ctx.invariantBasis(.{
    .algebra = so10,
    .external_legs = &.{tensor.ExternalLeg.primitive(fundamental, &.{
        .init(.fundamental, "i"),
    })),
}));

const so9 = try ctx.registerAlgebra(.{ .simple = .{ .family = .b, .rank = 4 } });
const so9_vector = try ctx.registerIrrep(so9, .{ .dynkin = &.{ 1, 0, 0, 0 } });
const so9_spinor = try ctx.registerIrrep(so9, .{ .dynkin = &.{ 0, 0, 0, 1 } });
try testing.expectEqual(@as(u128, 9), ctx.irrepMetadata(so9_vector)?.dimension.?);
try testing.expectEqual(@as(u128, 16), ctx.irrepMetadata(so9_spinor)?.dimension.?);

const e6 = try ctx.registerAlgebra(.{ .simple = .{ .family = .e6, .rank = 6 } });

```

```

const e7 = try ctx.registerAlgebra(.{ .simple = .{ .family = .e7, .rank = 7 } });
const e8 = try ctx.registerAlgebra(.{ .simple = .{ .family = .e8, .rank = 8 } });
const e6_27 = try ctx.registerIrrep(e6, .{ .dynkin = &.{ 1, 0, 0, 0, 0, 0 } });
const e7_56 = try ctx.registerIrrep(e7, .{ .dynkin = &.{ 0, 0, 0, 0, 0, 1, 0 } });
const e8_248 = try ctx.registerIrrep(e8, .{ .dynkin = &.{ 0, 0, 0, 0, 0, 0, 1, 0 } });
try testing.expectEqual(@as(u128, 27), ctx.irrepMetadata(e6_27)?.dimension.?);
try testing.expectEqual(@as(u128, 56), ctx.irrepMetadata(e7_56)?.dimension.?);
try testing.expectEqual(@as(u128, 248), ctx.irrepMetadata(e8_248)?.dimension.?);
}

```

B CFT-Code

B.1 API

```

test "root CFT API exposes selected constructors without implementation namespaces" {
  const testing = @import("std").testing;
  try testing.expect(@hasDecl(@This(), "FreeBoson"));
  try testing.expect(@hasDecl(@This(), "Bc"));
  try testing.expect(@hasDecl(@This(), "FreeFermion"));
  try testing.expect(@hasDecl(@This(), "EtaXi"));
  try testing.expect(@hasDecl(@This(), "product"));
  try testing.expect(@hasDecl(@This(), "boundary"));
  try testing.expect(@hasDecl(FreeBoson, "make"));
  try testing.expect(@hasDecl(FreeBoson, "boundary"));
  try testing.expect(@hasDecl(Bc, "sphere"));
  try testing.expect(@hasDecl(Bc, "diskBoundary"));
  try testing.expect(@hasDecl(FreeFermion, "sphere"));
  try testing.expect(@hasDecl(EtaXi, "sphere"));
  try testing.expect(!@hasDecl(FreeBoson, "Config"));
  try testing.expect(!@hasDecl(FreeBoson, "BoundaryConfig"));
  try testing.expect(!@hasDecl(Bc, "SphereConfig"));
  try testing.expect(!@hasDecl(Bc, "DiskConfig"));
  try testing.expect(!@hasDecl(FreeFermion, "SphereConfig"));
  try testing.expect(!@hasDecl(EtaXi, "SphereConfig"));
  try testing.expect(!@hasDecl(@This(), "Handle"));
  try testing.expect(!@hasDecl(@This(), "scalars"));
  try testing.expect(!@hasDecl(@This(), "freeBoson"));
  try testing.expect(!@hasDecl(@This(), "bcSphere"));
  try testing.expect(!@hasDecl(@This(), "freeFermionSphere"));
  try testing.expect(!@hasDecl(@This(), "etaXiSphere"));
  try testing.expect(!@hasDecl(@This(), "freeBosonBoundary"));
  try testing.expect(!@hasDecl(@This(), "bcDiskBoundary"));
  try testing.expect(!@hasDecl(@This(), "presets"));
  try testing.expect(!@hasDecl(@This(), "kernel"));
  try testing.expect(!@hasDecl(@This(), "theory"));
  try testing.expect(!@hasDecl(@This(), "expressions"));
  try testing.expect(!@hasDecl(@This(), "Generated"));
}

```

B.2 CFT Kernel

B.2.1 Generated Kernels

The first test protects the runtime call shape: a `MultiOp` carries the local operators and the label store needed to interpret them, without receiving a separate context.

```

test "MultiOp carries local operators and labels" {
  const testing = @import("std").testing;

  const labels = Call.LabelStore{

```

```

        .values = &.{ .{ .integer = 7 } },
    };
    const ops = [_]Call.LocalOp{
        .{
            .insertion = .{ .single = .{ .position = 1, .derivatives = 0 } },
            .kind = 2,
            .labels = 0,
        },
    };
    const input = Call.MultiOp{
        .operators = &ops,
        .labels = &labels,
    };

    try testing.expectEqual(@as(usize, 1), input.operators.len);
    try testing.expectEqual(@as(operators.OperatorKindId, 2), input.operators[0].kind);
    switch (input.labels.values[0]) {
        .integer => |value| try testing.expectEqual(@as(i64, 7), value),
        else => return error.UnexpectedLabelKind,
    }
}

```

The second test checks the kernel/theory boundary that matters now: a BCFT generated kernel extends the body schema without mutating the bulk generated kernel.

```

test "BCFT kernel extends bulk schema without mutating bulk schema" {
    const testing = @import("std").testing;

    const bulk_spec = theory.Spec.CFT{
        .sectors = &.{},
        .bulk_operator_kinds = &.{},
        .conformal_structure = 0,
        .conformal_structures = &.{},
        .quantum_names = &.{},
        .label_schemas = &.{},
        .conventions = &.{},
        .weight_rules = &.{},
        .quantum_rules = &.{},
        .statistics_rules = &.{},
        .strategy_support_rules = &.{},
        .wick_rule_sets = &.{},
        .pairing_data = &.{},
        .zero_mode_rule_sets = &.{},
        .correlator_configs = &.{},
        .local_ope_configs = &.{},
        .cocycle_tables = &.{},
        .presentations = &.{},
        .finite_projections = &.{},
        .lattices = &.{},
        .mode_algebras = &.{},
        .mode_action_rules = &.{},
    };

    const bcft_spec = theory.Spec.BCFT{
        .name = "empty boundary extension",
        .bulk = 0,
        .boundary_conditions = &.{},
        .boundary_stacks = &.{},
        .boundary_stress_tensor = .{ .kind = 0, .labels = 0 },
        .boundary_spin = &.{},
        .boundary_operator_kinds = &.{ 1, 2 },
        .boundary_changing_kinds = &.{},
        .wick_rule_sets = &.{},
        .pairing_data = &.{},
    };
}

```



```

        .zero_mode_rule_sets = &.{},
        .correlator_configs = &.{},
        .local_ope_configs = &.{},
    };

    const Bulk = CFTKernel(bulk_spec);
    const BoundaryKernel = BCFTKernel(Bulk, bcft_spec);

    try testing.expectEqual(@as(usize, 0), Bulk.Bodies.operator_schema.boundary_kind_ids.len);
    try testing.expectEqual(@as(usize, 2),
        ↳ BoundaryKernel.Bodies.operator_schema.boundary_kind_ids.len);
    try testing.expectEqual(@as(operators.OperatorKindId, 1),
        ↳ BoundaryKernel.Bodies.operator_schema.boundary_kind_ids[0]);
}

```

B.3 Presets

B.3.1 Preset Families

The tests here are intentionally behavioral. They check concepts that should survive later implementation work: composition exposes only selected namespaces, rule slices compose from the supplied factors, labels remain out-of-line, and the current correlator boundary streams primitive Wick terms rather than allocating a final expression.

```

test "presets compose operator builders without exposing theory tables" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 26 });
    const BosonicSphere = product(.{ X, bcSphere(.{}) });
    const XBoundary = freeBosonBoundary(.{
        .neumann = X.target.subspace(&.{ 0, 1, 2, 3 }),
        .dirichlet = X.target.complement(&.{ 0, 1, 2, 3 }),
        .dirichlet_position = X.target.point("x0"),
        .chan_paton = X.target.boundaryStack("stack"),
    });
    const GhostBoundary = bcDiskBoundary(.{});
    const BosonicDisk = boundary(BosonicSphere, .{ XBoundary, GhostBoundary });

    var local = try BosonicDisk.local(testing.allocator);
    defer local.deinit();

    const k = try local.momentum("k");
    const y = try local.boundaryCoord("y");
    const x = try @TypeOf(XBoundary).op.expXBoundary(&local, k, y);
    const c = try @TypeOf(GhostBoundary).op.cBoundary(&local, 0, y);
    const ops = try local.ops(.{ x, c });

    try testing.expect(!@hasDecl(@TypeOf(BosonicDisk.config.disk), "wick_rules"));
    try testing.expect(!@hasDecl(@TypeOf(BosonicDisk.config.disk), "pair_lookup"));
    try testing.expect(!@hasDecl(@TypeOf(BosonicDisk.config.disk), "zero_modes"));
    try testing.expect(!@hasDecl(@TypeOf(BosonicDisk.config.disk), "config_entries"));
    try testing.expect(!@hasDecl(@TypeOf(X.config.sphere), "wick_rules"));
    const LocalType = @typeInfo(@TypeOf(local)).pointer.child;
    try testing.expect(@typeInfo(LocalType) == .@"opaque");
    try testing.expect(!@hasDecl(LocalType, "symbol"));
    try testing.expect(!@hasDecl(LocalType, "op"));
    try testing.expect(!@hasDecl(LocalType, "add"));
    try testing.expect(!@hasDecl(LocalType, "normalOrdered"));
    try testing.expect(!@hasDecl(LocalType, "finish"));
    const OpsType = @typeInfo(@TypeOf(ops)).pointer.child;
    try testing.expect(@typeInfo(OpsType) == .@"opaque");
    try testing.expect(!@hasDecl(OpsType, "operators"));
}

```

```

    try testing.expect(!@hasDecl(OpsType, "labels"));
}

test "preset composition plumbing stays inside shared lowering" {
    const testing = @import("std").testing;

    try testing.expect(!@hasDecl(shared, "ConfigEntry"));
    try testing.expect(!@hasDecl(shared, "correlatorSliceCount"));
    try testing.expect(!@hasDecl(shared, "appendCorrelatorSlice"));
    try testing.expect(!@hasDecl(shared, "boundaryExtensionSliceCount"));
    try testing.expect(!@hasDecl(shared, "appendBoundaryExtensionSlice"));
    try testing.expect(!@hasDecl(shared, "mergeSphereConfigs"));
    try testing.expect(!@hasDecl(shared, "mergeBoundaryConfig"));
    try testing.expect(@hasDecl(shared, "declare"));
    try testing.expect(@hasDecl(shared.declare, "mergeSphereConfigs"));
    try testing.expect(@hasDecl(shared.declare, "mergeBoundaryConfig"));
    try testing.expect(!@hasDecl(shared, "Spec"));
    try testing.expect(!@hasDecl(shared, "wick"));
    try testing.expect(!@hasDecl(shared, "zero_mode"));
    try testing.expect(!@hasDecl(shared, "scalars"));
    try testing.expect(@hasDecl(shared.declare, "Spec"));
    try testing.expect(@hasDecl(shared.declare, "wick"));
    try testing.expect(@hasDecl(shared.declare, "zero_mode"));
    try testing.expect(@hasDecl(shared.declare, "scalars"));
}

test "preset rules and namespaces compose from supplied factors" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });
    const Ghost = bcSphere(.{});
    const MatterOnly = product(.{X});
    const GhostOnly = product(.{Ghost});
    const MatterGhost = product(.{ X, Ghost });

    try testing.expect(@hasDecl(MatterOnly.config, "sphere"));
    try testing.expect(@hasDecl(GhostOnly.config, "sphere"));
    try testing.expect(@hasDecl(MatterGhost.config, "sphere"));
    try testing.expect(!@hasDecl(MatterGhost.op, "free_boson"));
    try testing.expect(!@hasDecl(MatterGhost.op, "bc"));

    const XBoundary = freeBosonBoundary(.{
        .neumann = X.target.subspace(&.{ 0, 1, 2, 3 }),
        .dirichlet = X.target.complement(&.{ 0, 1, 2, 3 }),
        .dirichlet_position = X.target.point("x0"),
    });
    const GhostBoundary = bcDiskBoundary(.{});
    const DiskMatter = boundary(MatterGhost, .{XBoundary});
    const DiskMatterGhost = boundary(MatterGhost, .{ XBoundary, GhostBoundary });

    try testing.expect(@hasDecl(DiskMatter.op, "bulk"));
    try testing.expect(@hasDecl(DiskMatterGhost.op, "bulk"));
    try testing.expect(!@hasDecl(@TypeOf(DiskMatter.config.disk), "wick_rules"));
    try testing.expect(!@hasDecl(@TypeOf(DiskMatterGhost.config.disk), "wick_rules"));
}

test "local operator tokens are owned by their builder" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });

    var first = try X.local(testing allocator);
    defer first.deinit();
    var second = try X.local(testing allocator);

```

```

defer second.deinit();

const mu = try first.index("mu");
const z = try first.coord("z");
const dx = try X.op.dX(&first, mu, 0, z);

try testing.expectError(error.InvalidLocalOperator, second.ops(.{dx}));
}

test "preset compile-time flags change generated public behavior" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });
    const HolomorphicBc = bcSphere(.{ .include_antiholomorphic_copy = false });
    const MatterGhost = product(.{ X, HolomorphicBc });
    const BoundaryOnlyGhost = boundary(HolomorphicBc, .{
        bcDiskBoundary(.{ .include_mixed_bulk_boundary = false }),
    });

    try testing.expect(@hasDecl(HolomorphicBc.op, "b"));
    try testing.expect(!@hasDecl(HolomorphicBc.op, "bt"));
    try testing.expect(@hasDecl(MatterGhost.config, "sphere"));
    try testing.expect(@hasDecl(BoundaryOnlyGhost.op, "bulk"));
    try testing.expect(!@hasDecl(@TypeOf(BoundaryOnlyGhost.config.disk), "wick_rules"));

    const FullGhostDisk = boundary(HolomorphicBc, .{bcDiskBoundary(.{ })});

    var local = try FullGhostDisk.local(testing.allocator);
    defer local.deinit();

    const z = try local.coord("z");
    const y = try local.boundaryCoord("y");
    const ops = try local.ops(.{
        try HolomorphicBc.op.b(&local, 0, z),
        try @TypeOf(bcDiskBoundary(.{ })).op.cBoundary(&local, 0, y),
    });

    const Sink = struct {
        pub const emitWickScalar = noopWickScalar;
        pub const emitWickCoordinate = noopWickCoordinate;
        pub const emitWickTensor = noopWickTensor;
        pub const emitWickAction = noopWickAction;
        pub const emitWickTermEnd = noopWickTermEnd;
        pub const emitZeroModeFactor = noopZeroModeFactor;
        pub const emitZeroModeBaseEnd = noopZeroModeBaseEnd;

        wick_terms: usize = 0,

        /// emitWickTermStart counts one streamed primitive contraction.
        pub fn emitWickTermStart(self: *@This(), _: anytype) !void {
            self.wick_terms += 1;
        }
    };

    var full_sink = Sink{};
    try FullGhostDisk.correlator(&FullGhostDisk.config.disk, ops, &full_sink);
    try testing.expectEqual(@as(usize, 1), full_sink.wick_terms);

    var boundary_only_sink = Sink{};
    try BoundaryOnlyGhost.correlator(&BoundaryOnlyGhost.config.disk, ops,
        ↪ &boundary_only_sink);
    try testing.expectEqual(@as(usize, 0), boundary_only_sink.wick_terms);
}

```

```

test "free fermion sphere uses generic fermionic Wick signs" {
    const testing = @import("std").testing;

    const Psi = freeFermionSphere(.{ .dimension = 10 });

    var local = try Psi.local(testing.allocator);
    defer local.deinit();

    const mu = try local.index("mu");
    const nu = try local.index("nu");
    const rho = try local.index("rho");
    const sigma = try local.index("sigma");
    const z1 = try local.coord("z1");
    const z2 = try local.coord("z2");
    const z3 = try local.coord("z3");
    const z4 = try local.coord("z4");
    const ops = try local.ops(.{
        try Psi.op.psi(&local, mu, 0, z1),
        try Psi.op.psi(&local, nu, 0, z2),
        try Psi.op.psi(&local, rho, 0, z3),
        try Psi.op.psi(&local, sigma, 0, z4),
    });

    const Sink = struct {
        pub const emitWickScalar = noopWickScalar;
        pub const emitWickCoordinate = noopWickCoordinate;
        pub const emitWickTensor = noopWickTensor;
        pub const emitWickAction = noopWickAction;
        pub const emitWickTermEnd = noopWickTermEnd;
        pub const emitZeroModeFactor = noopZeroModeFactor;
        pub const emitZeroModeBaseEnd = noopZeroModeBaseEnd;

        branch_count: usize = 0,
        primitive_count: usize = 0,
        negative_branch_count: usize = 0,
        pending_negative: bool = false,

        /// emitWickBranchSign records signs supplied by the generic branch walker.
        pub fn emitWickBranchSign(self: *@This(), sign: i8) !void {
            if (sign < 0) self.pending_negative = true;
        }

        /// emitWickTermStart records the first primitive term of each Wick branch.
        pub fn emitWickTermStart(self: *@This(), _: anytype) !void {
            self.primitive_count += 1;
            if ((self.primitive_count % 2) == 1) {
                self.branch_count += 1;
                if (self.pending_negative) {
                    self.negative_branch_count += 1;
                    self.pending_negative = false;
                }
            }
        }
    };

    var sink = Sink{};
    try Psi.correlator(&Psi.config.sphere, ops, &sink);
    try testing.expectEqual(@as(usize, 3), sink.branch_count);
    try testing.expectEqual(@as(usize, 6), sink.primitive_count);
    try testing.expectEqual(@as(usize, 1), sink.negative_branch_count);
    try testing.expect(!@hasDecl(@TypeOf(Psi.config.sphere), "fermion_kinds"));
    try testing.expect(!@hasDecl(@TypeOf(Psi.config.sphere), "pair_lookup"));
}

```

```

test "eta-xi sphere saturates one xi zero mode" {
    const testing = @import("std").testing;

    const EtaXiPreset = etaXiSphere(.{ .include_antiholomorphic_copy = false });

    var local = try EtaXiPreset.local(testing allocator);
    defer local.deinit();

    const z = try local.coord("z");
    const xi = try EtaXiPreset.op.xi(&local, 0, z);

    const SaturatedSink = struct {
        pub const emitWickTermStart = noopWickTermStart;
        pub const emitWickScalar = noopWickScalar;
        pub const emitWickCoordinate = noopWickCoordinate;
        pub const emitWickTensor = noopWickTensor;
        pub const emitWickAction = noopWickAction;
        pub const emitWickTermEnd = noopWickTermEnd;

        zero_mode_count: usize = 0,
        base_count: usize = 0,
        coordinate: ?shared.Handle.Coord = null,

        /// emitZeroModeFactor records the single xi constant-mode factor.
        pub fn emitZeroModeFactor(self: *@This(), factor: anytype) !void {
            switch (factor) {
                .eta_xi_zero_mode => |zero| {
                    try testing.expectEqual(.sphere_holomorphic, zero.support);
                    try testing.expectEqual(@as(u8, 0), zero.xi.derivative_order);
                    self.zero_mode_count += 1;
                    self.coordinate = @as(shared.Handle.Coord,
                        ↪ @enumFromInt(zero.xi.coordinate));
                },
                else => return error.UnexpectedZeroModeFactor,
            }
        }

        /// emitZeroModeBaseEnd records a completed zero-mode base case.
        pub fn emitZeroModeBaseEnd(self: *@This()) !void {
            self.base_count += 1;
        }
    };

    var saturated_sink = SaturatedSink{};
    const saturated_ops = try local.ops(.{xi});
    try EtaXiPreset.correlator(&EtaXiPreset.config.sphere, saturated_ops, &saturated_sink);
    try testing.expectEqual(@as(usize, 1), saturated_sink.zero_mode_count);
    try testing.expectEqual(@as(usize, 1), saturated_sink.base_count);
    try testing.expectEqual(z, saturated_sink.coordinate.?);

    var derivative_local = try EtaXiPreset.local(testing allocator);
    defer derivative_local.deinit();
    const zd = try derivative_local.coord("z");
    const xi_derivative = try EtaXiPreset.op.xi(&derivative_local, 1, zd);
    var rejected_sink = SaturatedSink{};
    const rejected_ops = try derivative_local.ops(.{xi_derivative});
    try EtaXiPreset.correlator(&EtaXiPreset.config.sphere, rejected_ops, &rejected_sink);
    try testing.expectEqual(@as(usize, 0), rejected_sink.zero_mode_count);
    try testing.expectEqual(@as(usize, 0), rejected_sink.base_count);
    try testing.expect(!@hasDecl(@TypeOf(EtaXiPreset.config.sphere), "zero_modes"));
    try testing.expect(!@hasDecl(@TypeOf(EtaXiPreset.config.sphere), "fermion_kinds"));
}

test "eta-xi torus lowers prime-form log-derivative Wick and xi zero mode" {

```

```

const testing = @import("std").testing;

const EtaXiPreset = etaXiSphere(.{ .include_antiholomorphic_copy = false });

var local = try EtaXiPreset.local(testing.allocator);
defer local.deinit();

const z_eta = try local.coord("z_eta");
const z_xi_a = try local.coord("z_xi_a");
const z_xi_b = try local.coord("z_xi_b");
const ops = try local.ops(.{
    try EtaXiPreset.op.eta(&local, 0, z_eta),
    try EtaXiPreset.op.xi(&local, 0, z_xi_a),
    try EtaXiPreset.op.xi(&local, 0, z_xi_b),
});

const Sink = struct {
    pub const emitWickScalar = noopWickScalar;
    pub const emitWickTensor = noopWickTensor;
    pub const emitWickAction = noopWickAction;
    pub const emitWickTermEnd = noopWickTermEnd;

    term_count: usize = 0,
    kernel_count: usize = 0,
    zero_mode_count: usize = 0,
    base_count: usize = 0,

    /// emitWickTermStart records one eta-xi torus primitive contraction.
    pub fn emitWickTermStart(self: *@This(), _: anytype) !void {
        self.term_count += 1;
    }

    /// emitWickCoordinate requires the elliptic prime-form log derivative.
    pub fn emitWickCoordinate(self: *@This(), factor: anytype) !void {
        switch (factor.kernel) {
            .green_kernel => |kernel| {
                try testing.expectEqualStrings("elliptic_prime_form_log_derivative",
                    ↪ kernel.name);
                try testing.expectEqual(@as(u8, 0), kernel.left_derivatives);
                try testing.expectEqual(@as(u8, 0), kernel.right_derivatives);
                self.kernel_count += 1;
            },
            else => return error.UnexpectedTorusKernel,
        }
    }

    /// emitZeroModeFactor requires the torus holomorphic xi zero mode.
    pub fn emitZeroModeFactor(self: *@This(), factor: anytype) !void {
        switch (factor) {
            .eta_xi_zero_mode => |zero| {
                try testing.expectEqual(.torus_holomorphic, zero.support);
                try testing.expectEqual(@as(u8, 0), zero.xi.derivative_order);
                self.zero_mode_count += 1;
            },
            else => return error.UnexpectedZeroModeFactor,
        }
    }

    /// emitZeroModeBaseEnd records one successful torus base case.
    pub fn emitZeroModeBaseEnd(self: *@This()) !void {
        self.base_count += 1;
    }
};

```

```

var sink = Sink{};
try EtaXiPreset.correlator(&EtaXiPreset.config.torus, ops, &sink);
try testing.expectEqual(@as(usize, 2), sink.term_count);
try testing.expectEqual(@as(usize, 2), sink.kernel_count);
try testing.expectEqual(@as(usize, 2), sink.zero_mode_count);
try testing.expectEqual(@as(usize, 2), sink.base_count);
try testing.expect(!@hasDecl(@TypeOf(EtaXiPreset.config.torus), "wick_rules"));
try testing.expect(!@hasDecl(@TypeOf(EtaXiPreset.config.torus), "zero_modes"));
}

test "compact text sink inspects a small correlator without custom callbacks" {
const testing = @import("std").testing;
const Buffer = struct {
    bytes: [512]u8 = undefined,
    len: usize = 0,

    pub fn writeAll(self: *@This(), data: []const u8) !void {
        if (self.len + data.len > self.bytes.len) return error.NoSpaceLeft;
        @memcpy(self.bytes[self.len..][0..data.len], data);
        self.len += data.len;
    }

    fn slice(self: *const @This()) []const u8 {
        return self.bytes[0..self.len];
    }
};

const X = freeBoson(.{ .dimension = 10 });
var local = try X.local(testing.allocator);
defer local.deinit();

const mu = try local.index("mu");
const nu = try local.index("nu");
const z = try local.coord("z");
const w = try local.coord("w");
const ops = try local.ops(.{
    try X.op.dX(&local, mu, 0, z),
    try X.op.dX(&local, nu, 0, w),
});

var buffer = Buffer{};
var text = X.text.compact(&buffer, .small);
try X.correlator(&X.config.sphere, ops, &text);
try testing.expect(@import("std").mem.indexOf(u8, buffer.slice(), "eta()") != null);
}

test "bc zero modes consume c jets by top-form saturation" {
const testing = @import("std").testing;

const Ghost = bcSphere(.{ .include_antiholomorphic_copy = false });

var local = try Ghost.local(testing.allocator);
defer local.deinit();

const z1 = try local.coord("z1");
const z2 = try local.coord("z2");
const z3 = try local.coord("z3");
const ops = try local.ops(.{
    try Ghost.op.c(&local, 0, z1),
    try Ghost.op.c(&local, 1, z2),
    try Ghost.op.c(&local, 2, z3),
});

const Sink = struct {

```



```

pub const emitWickTermStart = noopWickTermStart;
pub const emitWickScalar = noopWickScalar;
pub const emitWickCoordinate = noopWickCoordinate;
pub const emitWickTensor = noopWickTensor;
pub const emitWickAction = noopWickAction;
pub const emitWickTermEnd = noopWickTermEnd;

factor_count: usize = 0,
base_end_count: usize = 0,
derivative_sum: u16 = 0,

/// emitZeroModeFactor records the top-form factor emitted by the base case.
pub fn emitZeroModeFactor(self: *@This(), factor: anytype) !void {
    self.factor_count += 1;
    switch (factor) {
        .bc_top_form => |top| {
            if (top.support != .sphere_holomorphic) return error.UnexpectedBcSupport;
            self.derivative_sum = @as(u16, top.jets[0].derivative_order) +
                ↳ top.jets[1].derivative_order + top.jets[2].derivative_order;
        },
        else => return error.UnexpectedZeroModeFactor,
    }
}

/// emitZeroModeBaseEnd records that every residual insertion was consumed.
pub fn emitZeroModeBaseEnd(self: *@This()) !void {
    self.base_end_count += 1;
}

};

var sink = Sink{};
try Ghost.correlator(&Ghost.config.sphere, ops, &sink);
try testing.expectEqual(@as(usize, 1), sink.factor_count);
try testing.expectEqual(@as(usize, 1), sink.base_end_count);
try testing.expectEqual(@as(u16, 3), sink.derivative_sum);
}

test "disk bc zero modes use one doubled chiral top form" {
    const testing = @import("std").testing;

    const Ghost = bcSphere(.{});
    const GhostBoundary = bcDiskBoundary(.{});
    const Disk = boundary(Ghost, .{GhostBoundary});

    var local = try Disk.local(testing.allocator);
    defer local.deinit();

    const z = try local.coord("z");
    const zbar = try local.coord("zbar");
    const y = try local.boundaryCoord("y");
    const ops = try local.ops.{
        try Disk.op.bulk.c(&local, 0, z),
        try Disk.op.bulk.ct(&local, 0, zbar),
        try @TypeOf(GhostBoundary).op.cBoundary(&local, 0, y),
    });

    const Sink = struct {
        pub const emitWickTermStart = noopWickTermStart;
        pub const emitWickScalar = noopWickScalar;
        pub const emitWickCoordinate = noopWickCoordinate;
        pub const emitWickTensor = noopWickTensor;
        pub const emitWickAction = noopWickAction;
        pub const emitWickTermEnd = noopWickTermEnd;
    };

```



```

    saw_disk_doubled: bool = false,

    /// emitZeroModeFactor records whether the doubled disk top form was emitted.
    pub fn emitZeroModeFactor(self: *@This(), factor: anytype) !void {
        switch (factor) {
            .bc_top_form => |top| self.saw_disk_doubled = top.support == .disk_doubled,
            else => return error.UnexpectedZeroModeFactor,
        }
    }

    /// emitZeroModeBaseEnd accepts the successful zero-mode base case.
    pub fn emitZeroModeBaseEnd(self: *@This()) !void {
        _ = self;
    }
};

var sink = Sink{};
try Disk.correlator(&Disk.config.disk, ops, &sink);
try testing.expect(sink.saw_disk_doubled);
}

test "free boson zero modes emit momentum delta and profile presentations" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });

    var local = try X.local(testing.allocator);
    defer local.deinit();

    const k = try local.momentum("k");
    const z = try local.coord("z");
    const zbar = try local.coord("zbar");
    const f0 = try local.targetFunction("f");
    const fhat = try local.fourierTransform("fhat");
    const coeff = try local.profileCoefficient("a");
    const point = try local.targetPoint("x0");
    const position_profile = X.profile.positionSpace(f0);
    const fourier_profile = X.profile.fourier(fhat);
    const polynomial_profile = X.profile.polynomialRnc(point, .{.coeff, 2 });
    const ops = try local.ops(.{
        try X.op.expX(&local, k, z, zbar),
        try X.op.profile(&local, position_profile, z, zbar),
        try X.op.profile(&local, fourier_profile, z, zbar),
        try X.op.profile(&local, polynomial_profile, z, zbar),
    });

    const Sink = struct {
        pub const emitWickTermStart = noopWickTermStart;
        pub const emitWickScalar = noopWickScalar;
        pub const emitWickCoordinate = noopWickCoordinate;
        pub const emitWickTensor = noopWickTensor;
        pub const emitWickAction = noopWickAction;
        pub const emitWickTermEnd = noopWickTermEnd;

        delta_count: usize = 0,
        gaussian_count: usize = 0,
        fourier_count: usize = 0,
        polynomial_count: usize = 0,
        base_end_count: usize = 0,
        two_pi_power: u16 = 0,

        /// emitZeroModeFactor records each free-boson zero-mode factor by presentation.
        pub fn emitZeroModeFactor(self: *@This(), factor: anytype) !void {
            switch (factor) {

```

```

        .momentum_delta => |delta| {
            self.delta_count += 1;
            self.two_pi_power = delta.two_pi_power;
            if (delta.projector != null or delta.momenta.len != 1) return
                ↪ error.UnexpectedMomentumDelta;
        },
        .profile_gaussian_differential => self.gaussian_count += 1,
        .profile_fourier_integral => self.fourier_count += 1,
        .profile_polynomial_integral => self.polynomial_count += 1,
        else => return error.UnexpectedZeroModeFactor,
    }
}

/// emitZeroModeBaseEnd records that all profile and plane-wave residuals were
↪ consumed.
pub fn emitZeroModeBaseEnd(self: *@This()) !void {
    self.base_end_count += 1;
}

};

var sink = Sink{};
try X.correlator(&X.config.sphere, ops, &sink);
try testing.expectEqual(@as(usize, 1), sink.delta_count);
try testing.expectEqual(@as(u16, 10), sink.two_pi_power);
try testing.expectEqual(@as(usize, 1), sink.gaussian_count);
try testing.expectEqual(@as(usize, 1), sink.fourier_count);
try testing.expectEqual(@as(usize, 1), sink.polynomial_count);
try testing.expectEqual(@as(usize, 1), sink.base_end_count);
}

test "disk free boson zero modes emit Neumann delta and Dirichlet phase" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });
    const XBoundary = freeBosonBoundary(.{
        .neumann = X.target.subspace(&.{ 0, 1, 2, 3 }),
        .dirichlet = X.target.complement(&.{ 0, 1, 2, 3 }),
        .dirichlet_position = X.target.point("x0"),
    });
    const Disk = boundary(product(.{X}), .{XBoundary});

    var local = try Disk.local(testing.allocator);
    defer local.deinit();

    const k = try local.momentum("k");
    const y = try local.boundaryCoord("y");
    const ops = try local.ops(.{
        try @TypeOf(XBoundary).op.expXBoundary(&local, k, y),
    });

    const Sink = struct {
        pub const emitWickTermStart = noopWickTermStart;
        pub const emitWickScalar = noopWickScalar;
        pub const emitWickCoordinate = noopWickCoordinate;
        pub const emitWickTensor = noopWickTensor;
        pub const emitWickAction = noopWickAction;
        pub const emitWickTermEnd = noopWickTermEnd;

        delta_count: usize = 0,
        phase_count: usize = 0,
        base_end_count: usize = 0,
        two_pi_power: u16 = 0,
        saw_disk_scalar: bool = false,
        saw_projector: bool = false,
    };

```

```

saw_position: bool = false,

/// emitZeroModeFactor records Neumann momentum conservation and Dirichlet phase data.
pub fn emitZeroModeFactor(self: *@This(), factor: anytype) !void {
  switch (factor) {
    .momentum_delta => |delta| {
      self.delta_count += 1;
      self.two_pi_power = delta.two_pi_power;
      self.saw_projector = delta.projector != null;
      switch (delta.scalar) {
        .monomial => |monomial| self.saw_disk_scalar = monomial.atom != null
          ↪ and monomial.atom_power == 1,
        else => return error.UnexpectedDiskNormalization,
      }
      if (delta.momenta.len != 1) return error.UnexpectedMomentumDelta;
    },
    .dirichlet_phase => |phase| {
      self.phase_count += 1;
      self.saw_position = phase.momenta.len == 1;
      switch (phase.position) {
        .target_point => {},
        else => return error.UnexpectedDirichletPosition,
      }
    },
    else => return error.UnexpectedZeroModeFactor,
  }
}

/// emitZeroModeBaseEnd records that the disk constant mode consumed the insertion.
pub fn emitZeroModeBaseEnd(self: *@This()) !void {
  self.base_end_count += 1;
}

};

var sink = Sink{};
try Disk.correlator(&Disk.config.disk, ops, &sink);
try testing.expectEqual(@as(usize, 1), sink.delta_count);
try testing.expectEqual(@as(usize, 1), sink.phase_count);
try testing.expectEqual(@as(ui6, 4), sink.two_pi_power);
try testing.expect(sink.saw_disk_scalar);
try testing.expect(sink.saw_projector);
try testing.expect(sink.saw_position);
try testing.expectEqual(@as(usize, 1), sink.base_end_count);
}

test "unconsumed free boson derivative kills the zero-mode base case" {
  const testing = @import("std").testing;

  const X = freeBoson(.{ .dimension = 10 });

  var local = try X.local(testing.allocator);
  defer local.deinit();

  const mu = try local.index("mu");
  const z = try local.coord("z");
  const ops = try local.ops(.{
    try X.op.dX(&local, mu, 0, z),
  });

  const Sink = struct {
    pub const emitWickTermStart = noopWickTermStart;
    pub const emitWickScalar = noopWickScalar;
    pub const emitWickCoordinate = noopWickCoordinate;
    pub const emitWickTensor = noopWickTensor;
  };
}

```

```

pub const emitWickAction = noopWickAction;
pub const emitWickTermEnd = noopWickTermEnd;

base_end_count: usize = 0,

/// emitZeroModeFactor fails if an unconsumed derivative field emits a zero-mode
↪ factor.
pub fn emitZeroModeFactor(_: *@This(), _: anytype) !void {
    return error.UnexpectedZeroModeFactor;
}

/// emitZeroModeBaseEnd fails because the derivative residual should not be consumed.
pub fn emitZeroModeBaseEnd(self: *@This()) !void {
    self.base_end_count += 1;
}
};

var sink = Sink{};
try X.correlator(&X.config.sphere, ops, &sink);
try testing.expectEqual(@as(usize, 0), sink.base_end_count);
}

test "pair Wick streams preset scalar atoms through correlator" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });
    const preset_alpha = scalars.atom("free_boson", "alpha_prime");

    var local = try X.local(testing.allocator);
    defer local.deinit();

    const mu = try local.index("mu");
    const nu = try local.index("nu");
    const z = try local.coord("z");
    const w = try local.coord("w");
    const ops = try local.ops(.{
        try X.op.dX(&local, mu, 0, z),
        try X.op.dX(&local, nu, 0, w),
    });

    switch (scalars.rational(2, 4)) {
        .rational => |value| {
            try testing.expectEqual(@as(i64, 1), value.numerator);
            try testing.expectEqual(@as(i64, 2), value.denominator);
        },
        else => return error.UnexpectedScalarShape,
    }

    const Sink = struct {
        expected_alpha: @TypeOf(preset_alpha),
        saw_scalar: bool = false,
        saw_coordinate: bool = false,

        /// emitWickTermStart accepts the primitive free-boson contraction.
        pub fn emitWickTermStart(_: *@This(), _: anytype) !void {}

        /// emitWickScalar checks the configured alpha-prime atom.
        pub fn emitWickScalar(self: *@This(), factor: anytype) !void {
            switch (factor) {
                .value => |scalar| switch (scalar) {
                    .monomial => |monomial| {
                        if (monomial.rational.numerator != -1 or monomial.rational.denominator
↪ != 2) return error.UnexpectedScalarFactor;

```

```

        if (monomial.imaginary_power != 0 or monomial.atom_power != 1) return
        ↪ error.UnexpectedScalarFactor;
        try testing.expectEqual(self.expected_alpha, monomial.atom.?);
        self.saw_scalar = true;
    },
    else => return error.UnexpectedScalarShape,
},
else => return error.UnexpectedScalarFactor,
}
}

/// emitWickCoordinate checks the differentiated two-point pole.
pub fn emitWickCoordinate(self: *@This(), factor: anytype) !void {
    switch (factor.kernel) {
        .difference_power => |power| {
            if (power.exponent != -2) return error.UnexpectedCoordinateFactor;
            self.saw_coordinate = true;
        },
        else => return error.UnexpectedCoordinateFactor,
    }
}

/// emitWickTensor accepts the contracted target-space tensor.
pub fn emitWickTensor(_: *@This(), _: anytype) !void {}

/// emitWickAction rejects unexpected action factors.
pub fn emitWickAction(_: *@This(), _: anytype) !void {
    return error.UnexpectedAction;
}

/// emitWickTermEnd accepts the completed primitive contraction.
pub fn emitWickTermEnd(_: *@This()) !void {}

/// emitZeroModeFactor accepts the empty free-boson constant mode.
pub fn emitZeroModeFactor(_: *@This(), _: anytype) !void {}

/// emitZeroModeBaseEnd accepts the full contraction base case.
pub fn emitZeroModeBaseEnd(_: *@This()) !void {}
};

var sink = Sink{ .expected_alpha = preset_alpha };
try X.correlator(&X.config.sphere, ops, &sink);
try testing.expect(sink.saw_scalar);
try testing.expect(sink.saw_coordinate);
}

test "pair Wick resolves derivative action on a vector profile" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });

    var local = try X.local(testing.allocator);
    defer local.deinit();

    const mu = try local.index("mu");
    const nu = try local.index("nu");
    const z = try local.coord("z");
    const w = try local.coord("w");
    const wbar = try local.coord("wbar");
    const f0 = try local.targetFunction("f");
    const f = X.profile.positionSpace(f0);

    const dx = try X.op.dX(&local, mu, 0, z);
    const profile = try X.op.profileVector(&local, f, nu, w, wbar);

```

```

const ops = try local.ops(.{ dx, profile });

const Sink = struct {
    term_count: usize = 0,
    scalar_count: usize = 0,
    coordinate_count: usize = 0,
    tensor_count: usize = 0,
    action_count: usize = 0,
    saw_scalar: bool = false,
    saw_coordinate: bool = false,
    saw_tensor_none: bool = false,
    saw_action: bool = false,
    expected_z: u32,
    expected_w: u32,
    expected_profile: u32,
    expected_index: u32,

    /// emitWickTermStart records the single primitive profile Wick term.
    pub fn emitWickTermStart(self: *@This(), event: anytype) !void {
        if (event.left_index != 0 or event.right_index != 1 or event.term_index != 0)
            ↪ return error.UnexpectedWickTermEvent;
        self.term_count += 1;
    }

    /// emitWickScalar checks the expected free-boson profile scalar.
    pub fn emitWickScalar(self: *@This(), factor: anytype) !void {
        self.scalar_count += 1;
        switch (factor) {
            .value => |scalar| switch (scalar) {
                .monomial => |monomial| {
                    if (monomial.rational.numerator != -1 or monomial.rational.denominator
                        ↪ != 2 or monomial.atom == null) return
                        ↪ error.UnexpectedScalarFactor;
                    self.saw_scalar = true;
                },
                else => return error.UnexpectedScalarFactor,
            },
            else => return error.UnexpectedScalarFactor,
        }
    }

    /// emitWickCoordinate checks the resolved profile pole.
    pub fn emitWickCoordinate(self: *@This(), factor: anytype) !void {
        self.coordinate_count += 1;
        if (factor.scalar.numerator != 1 or factor.scalar.denominator != 1) return
            ↪ error.UnexpectedCoordinateScalar;
        switch (factor.kernel) {
            .difference_power => |power| {
                if (power.coordinate.left != self.expected_z or power.coordinate.right !=
                    ↪ self.expected_w or power.exponent != -1) return
                    ↪ error.UnexpectedCoordinateFactor;
                self.saw_coordinate = true;
            },
            else => return error.UnexpectedCoordinateFactor,
        }
    }

    /// emitWickTensor checks that no tensor numerator is emitted.
    pub fn emitWickTensor(self: *@This(), factor: anytype) !void {
        self.tensor_count += 1;
        switch (factor) {
            .none => self.saw_tensor_none = true,
            else => return error.UnexpectedTensorFactor,
        }
    }
}

```

```

    }

    /// emitWickAction checks the profile-derivative action.
    pub fn emitWickAction(self: *@This(), factor: anytype) !void {
        self.action_count += 1;
        switch (factor) {
            .profile_derivative => |action| {
                switch (action.profile) {
                    .symbol => |value| if (value != self.expected_profile) return
                    ⇨ error.UnexpectedProfileLabel,
                    else => return error.UnexpectedProfileLabel,
                }
                switch (action.index) {
                    .symbol => |value| if (value != self.expected_index) return
                    ⇨ error.UnexpectedIndexLabel,
                    else => return error.UnexpectedIndexLabel,
                }
                self.saw_action = true;
            },
            else => return error.UnexpectedProfileAction,
        }
    }

    /// emitWickTermEnd accepts the completed primitive term.
    pub fn emitWickTermEnd(self: *@This()) !void {
        _ = self;
    }

    /// emitZeroModeFactor accepts the surviving profile zero-mode factor.
    pub fn emitZeroModeFactor(_: *@This(), _: anytype) !void {}

    /// emitZeroModeBaseEnd accepts the successful residual base case.
    pub fn emitZeroModeBaseEnd(_: *@This()) !void {}
};

var sink = Sink{
    .expected_z = @intFromEnum(z),
    .expected_w = @intFromEnum(w),
    .expected_profile = @intFromEnum(f),
    .expected_index = @intFromEnum(mu),
};

try X.correlator(&X.config.sphere, ops, &sink);
try testing.expectEqual(@as(usize, 1), sink.term_count);
try testing.expectEqual(@as(usize, 1), sink.scalar_count);
try testing.expectEqual(@as(usize, 1), sink.coordinate_count);
try testing.expectEqual(@as(usize, 1), sink.tensor_count);
try testing.expectEqual(@as(usize, 1), sink.action_count);
try testing.expect(sink.saw_scalar);
try testing.expect(sink.saw_coordinate);
try testing.expect(sink.saw_tensor_none);
try testing.expect(sink.saw_action);

const coeff = try local.profileCoefficient("a");
const x0 = try local.targetPoint("x0");
const polynomial_linear = X.profile.polynomialRnc(x0, .{.coeff, 1 });
const polynomial_quadratic = X.profile.polynomialRnc(x0, .{.coeff, 2 });
try testing.expect(@intFromEnum(polynomial_linear) != @intFromEnum(polynomial_quadratic));
}

test "pair Wick resolves config-backed boundary projectors" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });
    const XBoundary = freeBosonBoundary(.{

```

```

        .neumann = X.target.subspace(&.{ 0, 1, 2, 3 }},
        .dirichlet = X.target.complement(&.{ 0, 1, 2, 3 }},
        .dirichlet_position = X.target.point("x0"),
    });
    const Disk = boundary(product(.{X}), .{XBoundary});
    const neumann = X.target.subspace(&.{ 0, 1, 2, 3 });

    var local = try Disk.local(testing.allocator);
    defer local.deinit();

    const mu = try local.index("mu");
    const nu = try local.index("nu");
    const y = try local.boundaryCoord("y");
    const y2 = try local.boundaryCoord("y2");
    const left = try @TypeOf(XBoundary).op.dXBoundary(&local, mu, 0, y);
    const right = try @TypeOf(XBoundary).op.dXBoundary(&local, nu, 1, y2);
    const ops = try local.ops(.{ left, right });

    const Sink = struct {
        term_count: usize = 0,
        scalar_count: usize = 0,
        coordinate_count: usize = 0,
        tensor_count: usize = 0,
        action_count: usize = 0,
        saw_scalar: bool = false,
        saw_coordinate: bool = false,
        saw_projector: bool = false,
        expected_y: u32,
        expected_y2: u32,
        expected_neumann: u32,

        /// emitWickTermStart records the single boundary Wick term.
        pub fn emitWickTermStart(self: *@This(), event: anytype) !void {
            if (event.left_index != 0 or event.right_index != 1 or event.term_index != 0)
                ↪ return error.UnexpectedWickTermEvent;
            self.term_count += 1;
        }

        /// emitWickScalar checks the boundary free-boson scalar factor.
        pub fn emitWickScalar(self: *@This(), factor: anytype) !void {
            self.scalar_count += 1;
            switch (factor) {
                .value => |scalar| switch (scalar) {
                    .monomial => |monomial| {
                        if (monomial.rational.numerator != -1 or monomial.rational.denominator
                            ↪ != 1 or monomial.atom == null) return
                            ↪ error.UnexpectedScalarFactor;
                        self.saw_scalar = true;
                    },
                    else => return error.UnexpectedScalarFactor,
                },
                else => return error.UnexpectedScalarFactor,
            }
        }

        /// emitWickCoordinate checks the differentiated boundary pole.
        pub fn emitWickCoordinate(self: *@This(), factor: anytype) !void {
            self.coordinate_count += 1;
            if (factor.scalar.numerator != 2 or factor.scalar.denominator != 1) return
                ↪ error.UnexpectedCoordinateScalar;
            switch (factor.kernel) {
                .difference_power => |power| {

```



```

        if (power.coordinate.left != self.expected_y or power.coordinate.right !=
            self.expected_y2 or power.exponent != -3) return
        error.UnexpectedCoordinateFactor;
        self.saw_coordinate = true;
    },
    else => return error.UnexpectedCoordinateFactor,
}
}

/// emitWickTensor checks that the Neumann projector resolves from config.
pub fn emitWickTensor(self: *@This(), factor: anytype) !void {
    self.tensor_count += 1;
    switch (factor) {
        .projector_metric => |projector_metric| switch (projector_metric.projector) {
            .config => |value| switch (value) {
                .tensor_projector => |projector| {
                    if (@intFromEnum(projector) != self.expected_neumann) return
                        error.UnexpectedProjectorConfig;
                    self.saw_projector = true;
                },
                else => return error.UnexpectedProjectorConfig,
            },
            else => return error.UnexpectedProjectorSource,
        },
        else => return error.UnexpectedTensorFactor,
    }
}

/// emitWickAction counts unexpected profile actions.
pub fn emitWickAction(self: *@This(), factor: anytype) !void {
    _ = factor;
    self.action_count += 1;
}

/// emitWickTermEnd accepts the completed boundary term.
pub fn emitWickTermEnd(self: *@This()) !void {
    _ = self;
}

/// emitZeroModeFactor accepts disk empty-base normalization factors.
pub fn emitZeroModeFactor(_: *@This(), _: anytype) !void {}

/// emitZeroModeBaseEnd accepts the successful full contraction base case.
pub fn emitZeroModeBaseEnd(_: *@This()) !void {}
};

var sink = Sink{
    .expected_y = @intFromEnum(y),
    .expected_y2 = @intFromEnum(y2),
    .expected_neumann = @intFromEnum(neumann),
};
try Disk.correlator(&Disk.config.disk, ops, &sink);
try testing.expectEqual(@as(usize, 1), sink.term_count);
try testing.expectEqual(@as(usize, 1), sink.scalar_count);
try testing.expectEqual(@as(usize, 1), sink.coordinate_count);
try testing.expectEqual(@as(usize, 1), sink.tensor_count);
try testing.expectEqual(@as(usize, 0), sink.action_count);
try testing.expect(sink.saw_scalar);
try testing.expect(sink.saw_coordinate);
try testing.expect(sink.saw_projector);
}

```

```

test "recursive correlator enumerates free-boson pairings without dangling branches" {
  const testing = @import("std").testing;

  const X = freeBoson(.{ .dimension = 10 });

  var local = try X.local(testing.allocator);
  defer local.deinit();

  const mu = try local.index("mu");
  const z1 = try local.coord("z1");
  const z2 = try local.coord("z2");
  const z3 = try local.coord("z3");
  const z4 = try local.coord("z4");
  const ops = try local.ops(.{
    try X.op.dX(&local, mu, 0, z1),
    try X.op.dX(&local, mu, 0, z2),
    try X.op.dX(&local, mu, 0, z3),
    try X.op.dX(&local, mu, 0, z4),
  });

  const Sink = struct {
    pub const emitWickScalar = noopWickScalar;
    pub const emitWickCoordinate = noopWickCoordinate;
    pub const emitWickTensor = noopWickTensor;
    pub const emitWickTermEnd = noopWickTermEnd;

    wick_terms: usize = 0,
    zero_modes: usize = 0,
    base_cases: usize = 0,

    /// emitWickTermStart counts one primitive contraction in an accepted branch.
    pub fn emitWickTermStart(self: *@This(), _: anytype) !void {
      self.wick_terms += 1;
    }

    /// emitWickAction rejects unexpected action factors.
    pub fn emitWickAction(_: *@This(), _: anytype) !void {
      return error.UnexpectedAction;
    }

    /// emitZeroModeFactor counts the free-boson constant-mode factor.
    pub fn emitZeroModeFactor(self: *@This(), factor: anytype) !void {
      switch (factor) {
        .momentum_delta => self.zero_modes += 1,
        else => return error.UnexpectedZeroModeFactor,
      }
    }

    /// emitZeroModeBaseEnd counts one accepted full Wick branch.
    pub fn emitZeroModeBaseEnd(self: *@This()) !void {
      self.base_cases += 1;
    }
  };

  var sink = Sink{};
  try X.correlator(&X.config.sphere, ops, &sink);
  try testing.expectEqual(@as(usize, 6), sink.wick_terms);
  try testing.expectEqual(@as(usize, 3), sink.zero_modes);
  try testing.expectEqual(@as(usize, 3), sink.base_cases);
}

test "recursive correlator leaves bc top-form residuals after all b contractions" {
  const testing = @import("std").testing;

```

```

const Ghost = bcSphere(.{ .include_antiholomorphic_copy = false });

var local = try Ghost.local(testing.allocator);
defer local.deinit();

const z1 = try local.coord("z1");
const z2 = try local.coord("z2");
const z3 = try local.coord("z3");
const z4 = try local.coord("z4");
const z5 = try local.coord("z5");
const z6 = try local.coord("z6");
const z7 = try local.coord("z7");
const ops = try local.ops(.{
    try Ghost.op.b(&local, 0, z1),
    try Ghost.op.b(&local, 0, z2),
    try Ghost.op.c(&local, 0, z3),
    try Ghost.op.c(&local, 0, z4),
    try Ghost.op.c(&local, 0, z5),
    try Ghost.op.c(&local, 0, z6),
    try Ghost.op.c(&local, 0, z7),
});

const Sink = struct {
    pub const emitWickScalar = noopWickScalar;
    pub const emitWickCoordinate = noopWickCoordinate;
    pub const emitWickTensor = noopWickTensor;
    pub const emitWickTermEnd = noopWickTermEnd;

    wick_terms: usize = 0,
    base_cases: usize = 0,

    /// emitWickTermStart counts one b-c contraction in an accepted branch.
    pub fn emitWickTermStart(self: *@This(), _: anytype) !void {
        self.wick_terms += 1;
    }

    /// emitWickAction rejects unexpected action factors.
    pub fn emitWickAction(_: *@This(), _: anytype) !void {
        return error.UnexpectedAction;
    }

    /// emitZeroModeFactor accepts the surviving c-zero-mode top form.
    pub fn emitZeroModeFactor(_: *@This(), factor: anytype) !void {
        switch (factor) {
            .bc_top_form => {},
            else => return error.UnexpectedZeroModeFactor,
        }
    }

    /// emitZeroModeBaseEnd counts one accepted full Wick branch.
    pub fn emitZeroModeBaseEnd(self: *@This()) !void {
        self.base_cases += 1;
    }
};

var sink = Sink{};
try Ghost.correlator(&Ghost.config.sphere, ops, &sink);
try testing.expectEqual(@as(usize, 40), sink.wick_terms);
try testing.expectEqual(@as(usize, 20), sink.base_cases);
}

test "recursive correlator emits fermion signs for bc pair crossings" {
    const testing = @import("std").testing;

```

```

const Ghost = bcSphere(.{ .include_antiholomorphic_copy = false });

var local = try Ghost.local(testing.allocator);
defer local.deinit();

const z1 = try local.coord("z1");
const z2 = try local.coord("z2");
const z3 = try local.coord("z3");
const z4 = try local.coord("z4");
const z5 = try local.coord("z5");
const z6 = try local.coord("z6");
const z7 = try local.coord("z7");
const ops = try local.ops(.{
    try Ghost.op.b(&local, 0, z1),
    try Ghost.op.b(&local, 0, z2),
    try Ghost.op.c(&local, 0, z3),
    try Ghost.op.c(&local, 0, z4),
    try Ghost.op.c(&local, 0, z5),
    try Ghost.op.c(&local, 0, z6),
    try Ghost.op.c(&local, 0, z7),
});

const Sink = struct {
    pub const emitWickScalar = noopWickScalar;
    pub const emitWickCoordinate = noopWickCoordinate;
    pub const emitWickTensor = noopWickTensor;
    pub const emitWickTermEnd = noopWickTermEnd;

    negative_signs: usize = 0,
    base_cases: usize = 0,

    /// emitWickBranchSign records negative fermion branch signs.
    pub fn emitWickBranchSign(self: *@This(), sign: i8) !void {
        if (sign != -1) return error.UnexpectedBranchSign;
        self.negative_signs += 1;
    }

    /// emitWickTermStart accepts a primitive contraction.
    pub fn emitWickTermStart(_: *@This(), _: anytype) !void {}

    /// emitWickAction rejects unexpected action factors.
    pub fn emitWickAction(_: *@This(), _: anytype) !void {
        return error.UnexpectedAction;
    }

    /// emitZeroModeFactor accepts the surviving c-zero-mode top form.
    pub fn emitZeroModeFactor(_: *@This(), factor: anytype) !void {
        switch (factor) {
            .bc_top_form => {},
            else => return error.UnexpectedZeroModeFactor,
        }
    }

    /// emitZeroModeBaseEnd counts one accepted full Wick branch.
    pub fn emitZeroModeBaseEnd(self: *@This()) !void {
        self.base_cases += 1;
    }
};

var sink = Sink{};
try Ghost.correlator(&Ghost.config.sphere, ops, &sink);
try testing.expectEqual(@as(usize, 10), sink.negative_signs);
try testing.expectEqual(@as(usize, 20), sink.base_cases);
}

```

B.3.2 Free Boson Preset

Boundary Extension

```
test "free-boson schema builders return opaque local tokens" {
    const testing = std.testing;
    const X = freeBoson(.{ .dimension = 10 });

    var local = try X.local(testing.allocator);
    defer local.deinit();

    const mu = try local.index("mu");
    const k = try local.momentum("k");
    const z = try local.coord("z");
    const zbar = try local.coord("zbar");
    const dx = try X.op.dX(&local, mu, 2, z);
    const exp = try X.op.expX(&local, k, z, zbar);
    const ops = try local.ops(.{ dx, exp });

    try testing.expect(@typeName(@typeName(@TypeOf(dx)).pointer.child) == .@"opaque");
    try testing.expect(@typeName(@typeName(@TypeOf(exp)).pointer.child) == .@"opaque");
    try testing.expect(@typeName(@typeName(@TypeOf(ops)).pointer.child) == .@"opaque");
}

test "free-boson boundary schema builders return opaque local tokens" {
    const testing = std.testing;

    var local = try Local.init(testing.allocator);
    defer local.deinit();

    const mu = try local.index("mu");
    const k = try local.momentum("k");
    const y = try local.boundaryCoord("y");
    const dx = try FreeBosonBoundaryOp.dXBoundary(&local, mu, 3, y);
    const exp = try FreeBosonBoundaryOp.expXBoundary(&local, k, y);
    const ops = try local.ops(.{ dx, exp });

    try testing.expect(@typeName(@typeName(@TypeOf(dx)).pointer.child) == .@"opaque");
    try testing.expect(@typeName(@typeName(@TypeOf(exp)).pointer.child) == .@"opaque");
    try testing.expect(@typeName(@typeName(@TypeOf(ops)).pointer.child) == .@"opaque");
}
```

B.3.3 bc Preset

Disk Boundary

```
test "bc schema builders return opaque local tokens" {
    const testing = std.testing;
    const Ghost = bcSphere(.{ .include_antiholomorphic_copy = false });

    var local = try Ghost.local(testing.allocator);
    defer local.deinit();

    const z1 = try local.coord("z1");
    const z2 = try local.coord("z2");
    const b = try Ghost.op.b(&local, 1, z1);
    const c = try Ghost.op.c(&local, 2, z2);
    const ops = try local.ops(.{ b, c });

    try testing.expect(@typeName(@typeName(@TypeOf(b)).pointer.child) == .@"opaque");
    try testing.expect(@typeName(@typeName(@TypeOf(c)).pointer.child) == .@"opaque");
    try testing.expect(@typeName(@typeName(@TypeOf(ops)).pointer.child) == .@"opaque");
}
```

```

test "bc boundary schema builders return opaque local tokens" {
    const testing = std.testing;

    var local = try Local.init(testing.allocator);
    defer local.deinit();

    const y = try local.boundaryCoord("y");
    const b = try BcDiskBoundaryOp.bBoundary(&local, 1, y);
    const c = try BcDiskBoundaryOp.cBoundary(&local, 2, y);
    const ops = try local.ops(.{ b, c });

    try testing.expect(@typeName(@typeName(@TypeOf(b)).pointer.child) == .@"opaque");
    try testing.expect(@typeName(@typeName(@TypeOf(c)).pointer.child) == .@"opaque");
    try testing.expect(@typeName(@typeName(@TypeOf(ops)).pointer.child) == .@"opaque");
}

```

B.4 Lisp DSL

B.4.1 Generated Fixture ABI

Tests

```

test "generated C ABI counts and streams free-fermion events" {
    try selfTest();
}

```

C Tensor-Code

C.1 API

```

test "tensor root exposes invariant basis audit" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const su2 = try ctx.registerAlgebra(.{ .simple = .{ .family = .a, .rank = 1 } });
    const fundamental = try ctx.registerIrrep(su2, .{ .dynkin = &.{1} });
    const leg = ExternalLeg.primitive(fundamental, &.{
        .init(.fundamental, "i"),
    });
    const basis = try ctx.invariantBasis(.{
        .algebra = su2,
        .external_legs = &.{ leg, leg, leg, leg },
    });

    const audit: BasisAudit = ctx.basisAudit(basis).?;
    try testing.expectEqual(@as(u128, 2), ctx.basisInvariantCount(basis).?);
    try testing.expectEqual(@as(u128, 2), audit.path_count);
    try testing.expectEqual(@as(u32, 2), audit.stored_path_count);
}

test "tensor root exposes ADE formula coverage and basis streaming" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const su3 = try ctx.registerAlgebra(.{ .simple = .{ .family = .a, .rank = 2 } });

```

```

const fundamental = try ctx.registerIrrep(su3, .{ .dynkin = &.{ 1, 0 } });
const leg = ExternalLeg.primitive(fundamental, &.{
    .init(.fundamental, "i"),
});
const basis = try ctx.invariantBasis(.{
    .algebra = su3,
    .external_legs = &.{ leg, leg, leg },
});

const coverage: FormulaCoverageAudit = try ctx.basisFormulaCoverageAudit(basis);
try testing.expectEqual(@as(u32), 1, coverage.path_count);
try testing.expectEqual(@as(u64), 2, coverage.local_step_count);
try testing.expectEqual(@as(u64), 2, coverage.expandable_step_count);
try testing.expectEqual(@as(u64), 0, coverage.missing_step_count);
try testing.expectEqual(@as(u32), 1, coverage.fully_expandable_path_count);

var sink: RootCountingSink = .{};
const audit = try ctx.renderBasisFiltered(basis, .{ .projectors = .expanded_terms },
    ↪ ExpansionFilter.acceptAll(), &sink);
try testing.expectEqual(@as(u64), 1, audit.invariants);
try testing.expectEqual(@as(u64), 1, audit.emitted);
try testing.expectEqual(@as(u32), 1, sink.term_count);
try testing.expectEqual(@as(u32), 1, sink.epsilon_count);
try testing.expectEqual(@as(u32), 0, sink.projector_operator_count);

const e6 = try ctx.registerAlgebra(.{ .simple = .{ .family = .e6, .rank = 6 } });
const e6_fundamental = try ctx.registerIrrep(e6, .{ .dynkin = &.{ 1, 0, 0, 0, 0, 0 } });
const e6_leg = ExternalLeg.primitive(e6_fundamental, &.{
    .init(.fundamental, "i"),
});
const e6_basis = try ctx.invariantBasis(.{
    .algebra = e6,
    .external_legs = &.{ e6_leg, e6_leg, e6_leg },
});
const e6_coverage: FormulaCoverageAudit = try ctx.basisFormulaCoverageAudit(e6_basis);
try testing.expectEqual(@as(u32), 1, e6_coverage.path_count);
try testing.expectEqual(@as(u64), 2, e6_coverage.expandable_step_count);
try testing.expectEqual(@as(u64), 0, e6_coverage.missing_step_count);

var e6_sink: RootCountingSink = .{};
const e6_audit = try ctx.renderBasisFiltered(e6_basis, .{ .projectors = .expanded_terms },
    ↪ ExpansionFilter.acceptAll(), &e6_sink);
try testing.expectEqual(@as(u64), 1, e6_audit.invariants);
try testing.expectEqual(@as(u64), 1, e6_audit.emitted);
try testing.expectEqual(@as(u32), 1, e6_sink.term_count);
try testing.expectEqual(@as(u32), 1, e6_sink.structure_constant_count);
try testing.expectEqual(@as(u32), 0, e6_sink.projector_operator_count);

const e8 = try ctx.registerAlgebra(.{ .simple = .{ .family = .e8, .rank = 8 } });
const e8_adjoint = try ctx.registerIrrep(e8, .{ .dynkin = &.{ 0, 0, 0, 0, 0, 0, 1, 0 } });
const e8_leg = ExternalLeg.primitive(e8_adjoint, &.{
    .init(.adjoint, "a"),
});
const e8_basis = try ctx.invariantBasis(.{
    .algebra = e8,
    .external_legs = &.{ e8_leg, e8_leg, e8_leg },
});
const e8_coverage: FormulaCoverageAudit = try ctx.basisFormulaCoverageAudit(e8_basis);
try testing.expectEqual(@as(u32), 1, e8_coverage.path_count);
try testing.expectEqual(@as(u64), 2, e8_coverage.expandable_step_count);
try testing.expectEqual(@as(u64), 0, e8_coverage.missing_step_count);

var e8_sink: RootCountingSink = .{};

```

```

const e8_audit = try ctx.renderBasisFiltered(e8_basis, .{ .projectors = .expanded_terms },
↳ ExpansionFilter.acceptAll(), &e8_sink);
try testing.expectEqual(@as(u64, 1), e8_audit.invariants);
try testing.expectEqual(@as(u64, 1), e8_audit.emitted);
try testing.expectEqual(@as(u32, 1), e8_sink.term_count);
try testing.expectEqual(@as(u32, 1), e8_sink.structure_constant_count);
try testing.expectEqual(@as(u32, 0), e8_sink.projector_operator_count);
}

```

C.2 Tensor Context

C.2.1 Tests

```

test "context audits SU2 four-doublet invariant path count" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const su2 = try ctx.registerAlgebra(.{ .simple = .{ .family = .a, .rank = 1 } });
    const fundamental = try ctx.registerIrrep(su2, .{ .dynkin = &.{1} });
    const leg = realization.ExternalLeg.primitive(fundamental, &.{
        .init(.fundamental, "i"),
    });

    const basis = try ctx.invariantBasis(.{
        .algebra = su2,
        .external_legs = &.{ leg, leg, leg, leg },
    });

    try testing.expectEqual(@as(u128, 2), ctx.impl().couplings.basisPathCount(basis).?);
    const audit = ctx.impl().couplings.basisAudit(basis).?;
    try testing.expectEqual(@as(u128, 2), audit.path_count);
    try testing.expectEqual(@as(u32, 2), audit.stored_path_count);
    try testing.expectEqual(@as(u32, 6), audit.step_count);
    try testing.expectEqual(@as(u32, 4), audit.distinct_projector_count);
    try testing.expectEqual(@as(u16, 3), audit.max_path_step_len);
    const paths = ctx.impl().couplings.basisPaths(basis).?;
    try testing.expectEqual(@as(usize, 2), paths.len);
    for (paths) |path| {
        const steps = ctx.impl().couplings.pathSteps(path);
        try testing.expectEqual(@as(usize, 3), steps.len);
        try testing.expectEqual(fundamental.value, steps[0].left.value);
        try testing.expectEqual(fundamental.value, steps[0].right.value);
        try testing.expectEqual(@as(u16, 0), steps[0].multiplicity_copy);
        for (steps) |step| {
            const derivation = ctx.impl().projectors.projectorDerivation(step.projector).?;
            if (derivation.formula_kind == .identity) {
                try testing.expectEqual(projector.ProjectorKind.backend_specific_verified,
↳ derivation.kind);
                try testing.expectEqual(projector.ExpansionSource.backend_specific_verified,
↳ derivation.source);
                try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
↳ derivation.status);
                try testing.expect(derivation.formula_audit.verified());
            } else {
                try testing.expectEqual(projector.ProjectorKind.highest_weight_solver,
↳ derivation.kind);
                try testing.expectEqual(projector.ExpansionSource.highest_weight_solver,
↳ derivation.source);
                try testing.expectEqual(projector.ExpansionStatus.expandable_named,
↳ derivation.status);
            }
        }
    }
}

```



```

    }
  }
}

test "context streams empty invariant through expansion filters" {
  const testing = std.testing;

  var ctx = try Context.init(testing.allocator);
  defer ctx.deinit();

  const su2 = try ctx.registerAlgebra(.{ .simple = .{ .family = .a, .rank = 1 } });
  const basis = try ctx.invariantBasis(.{
    .algebra = su2,
    .external_legs = &.{},
  });

  var accept_sink: CountingSink = .{};
  const accept_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{,
    ↪ rendering.ExpansionFilter.acceptAll(), &accept_sink);
  try testing.expectEqual(@as(u32), 1, accept_sink.term_count);
  try testing.expectEqual(@as(u32), 0, accept_sink.atom_count);
  try testing.expectEqual(@as(u64), 1, accept_audit.accepted);
  try testing.expectEqual(@as(u64), 1, accept_audit.emitted);

  var reject_sink: CountingSink = .{};
  const reject_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{,
    ↪ rendering.ExpansionFilter.rejectAll(), &reject_sink);
  try testing.expectEqual(@as(u32), 0, reject_sink.term_count);
  try testing.expectEqual(@as(u64), 1, reject_audit.rejected);
  try testing.expectEqual(@as(u64), 0, reject_audit.emitted);

  var projector_filter_sink: CountingSink = .{};
  const projector_filter_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{,
    ↪ rendering.ExpansionFilter.withoutOperatorKind(.projector), &projector_filter_sink);
  try testing.expectEqual(@as(u32), 1, projector_filter_sink.term_count);
  try testing.expectEqual(@as(u64), 1, projector_filter_audit.undecided);
  try testing.expectEqual(@as(u64), 1, projector_filter_audit.emitted);
}

test "context streams non-empty invariant as named projector chain" {
  const testing = std.testing;

  var ctx = try Context.init(testing.allocator);
  defer ctx.deinit();

  const su2 = try ctx.registerAlgebra(.{ .simple = .{ .family = .a, .rank = 1 } });
  const fundamental = try ctx.registerIrrep(su2, .{ .dynkin = &.{1} });
  const leg = realization.ExternalLeg.primitive(fundamental, &.{
    .init(.fundamental, "i"),
  });
  const basis = try ctx.invariantBasis(.{
    .algebra = su2,
    .external_legs = &.{ leg, leg },
  });

  var sink: CountingSink = .{};
  const audit = try ctx.renderInvariantFiltered(basis, .init(0), .{,
    ↪ rendering.ExpansionFilter.acceptAll(), &sink);
  try testing.expect(sink.term_count > 0);
  try testing.expectEqual(@as(u32), 1, sink.atom_count);
  try testing.expectEqual(@as(u32), 1, sink.projector_operator_count);
  const descriptor = ctx.projectorDescriptor(sink.first_projector_operator_id.?).?;
  switch (descriptor.role) {
    .product_channel => |channel| {

```

```

        try testing.expectEqual(fundamental.value, channel.left.value);
        try testing.expectEqual(fundamental.value, channel.right.value);
        try testing.expectEqual(@as(u16, 0), channel.multiplicity_copy);
    },
    else => return error.ExpectedProductChannelProjector,
}
try testing.expectEqual(projector.ProjectorKind.highest_weight_solver,
    ↪ descriptor.derivation.kind);
try testing.expectEqual(@as(u64, sink.term_count), audit.accepted);
try testing.expectEqual(@as(u64, sink.term_count), audit.emitted);

const first_counters = ctx.impl().projectors.expansionCounters();
try testing.expectEqual(@as(u64, 0), first_counters.named_hits);
try testing.expectEqual(@as(u64, 1), first_counters.named_misses);

var second_sink: CountingSink = .{};
_ = try ctx.renderInvariantFiltered(basis, .init(0), .{};
    ↪ rendering.ExpansionFilter.acceptAll(), &second_sink);
const second_counters = ctx.impl().projectors.expansionCounters();
try testing.expectEqual(@as(u64, 1), second_counters.named_hits);
try testing.expectEqual(@as(u64, 1), second_counters.named_misses);

var projector_filter_sink: CountingSink = .{};
const projector_filter_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{};
    ↪ rendering.ExpansionFilter.withoutOperatorKind(.projector), &projector_filter_sink);
try testing.expectEqual(@as(u32, 0), projector_filter_sink.term_count);
try testing.expectEqual(@as(u64, 1), projector_filter_audit.rejected);
try testing.expectEqual(@as(u64, 0), projector_filter_audit.emitted);
}

test "context rejects render signature mismatches before projector cache use" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const su2 = try ctx.registerAlgebra(.{ .simple = .{ .family = .a, .rank = 1 } });
    const fundamental = try ctx.registerIrrep(su2, .{ .dynkin = &.{1} });
    const leg = realization.ExternalLeg.primitive(fundamental, &.{
        .init(.fundamental, "i"),
    });
    const basis = try ctx.invariantBasis(.{
        .algebra = su2,
        .external_legs = &.{ leg, leg },
    });

    var sink: CountingSink = .{};
    try testing.expectError(error.UnsupportedRenderSignature, ctx.renderInvariant(basis,
        ↪ .init(0), .{ .signature = .euclidean }, &sink));
    try testing.expectEqual(@as(u32, 0), sink.term_count);

    var basis_sink: CountingSink = .{};
    try testing.expectError(error.UnsupportedRenderSignature, ctx.renderBasis(basis, .{
        ↪ .signature = .euclidean }, &basis_sink));
    try testing.expectEqual(@as(u32, 0), basis_sink.term_count);

    const counters = ctx.impl().projectors.expansionCounters();
    try testing.expectEqual(@as(u64, 0), counters.named_hits);
    try testing.expectEqual(@as(u64, 0), counters.named_misses);
    try testing.expectEqual(@as(u64, 0), counters.formula_hits);
    try testing.expectEqual(@as(u64, 0), counters.formula_misses);
}

test "context streams projected realization boundary projector" {

```

```

const testing = std.testing;

var ctx = try Context.init(testing.allocator);
defer ctx.deinit();

const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
const vector = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 0 } });
const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
const vector_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 1 } });
const dual_vector_spinor = try ctx.dualIrrep(vector_spinor);
const projected = try
↳ ctx.registerRealization(realization.RealizationSpec.projected(vector_spinor, &.{
↳ vector, spinor }, &.{
    .init(.vector, "m"),
    .init(.spinor, "a"),
}, 0));
const projected_leg = realization.ExternalLeg.realized(projected, &.{
    .init(.vector, "m"),
    .init(.spinor, "a"),
});
const dual_leg = realization.ExternalLeg.primitive(dual_vector_spinor, &.{
    .init(.conjugate_spinor, "b"),
    .init(.vector, "n"),
});
const basis = try ctx.invariantBasis(.{
    .algebra = so10,
    .external_legs = &.{ projected_leg, dual_leg },
});

try testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?);
const basis_audit = ctx.basisAudit(basis).?;
try testing.expectEqual(@as(u32, 1), basis_audit.distinct_projector_count);
try testing.expectEqual(@as(u32, 1), basis_audit.boundary_projector_count);
try testing.expectEqual(@as(u32, 2), basis_audit.total_projector_count);
var sink: CountingSink = .{};
const audit = try ctx.renderInvariantFiltered(basis, .init(0), .{,
↳ rendering.ExpansionFilter.acceptAll(), &sink);
try testing.expect(sink.term_count > 0);
try testing.expectEqual(@as(u32, 2), sink.projector_operator_count);
try testing.expectEqual(@as(u64, 1), audit.accepted);
try testing.expectEqual(@as(u64, 1), audit.emitted);

const descriptor = ctx.projectorDescriptor(sink.first_projector_operator_id.?).?;
switch (descriptor.role) {
    .boundary_realization => |handle| try testing.expectEqual(projected.value,
↳ handle.value),
    else => return error.ExpectedBoundaryRealizationProjector,
}

var expanded_sink: CountingSink = .{};
try testing.expectError(error.ProjectorExpansionNotImplemented, ctx.renderInvariant(basis,
↳ .init(0), .{ .projectors = .expanded_terms }, &expanded_sink));
try testing.expectEqual(@as(u32, 0), expanded_sink.term_count);

var reject_sink: CountingSink = .{};
const reject_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors =
↳ .expanded_terms }, rendering.ExpansionFilter.rejectAll(), &reject_sink);
try testing.expectEqual(@as(u32, 0), reject_sink.term_count);
try testing.expectEqual(@as(u64, 1), reject_audit.rejected);
try testing.expectEqual(@as(u64, 0), reject_audit.emitted);
}

test "context expands Spin10 vector-spinor boundary projector formula" {
    const testing = std.testing;

```

```

var ctx = try Context.init(testing.allocator);
defer ctx.deinit();

const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
const vector = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 0 } });
const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
const vector_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 1 } });
const projected = try
  ↪ ctx.registerRealization(realization.RealizationSpec.projected(vector_spinor, &.{
  ↪ vector, spinor }, &.{
    .init(.vector, "m"),
    .init(.spinor, "a"),
  }, 0));
const projected_leg = realization.ExternalLeg.realized(projected, &.{
  .init(.vector, "m"),
  .init(.spinor, "a"),
});
const basis = try ctx.invariantBasis(.{
  .algebra = so10,
  .external_legs = &.{projected_leg},
  .target = .{ .irrep = vector_spinor },
});

try testing.expectEqual(@as(u128), 1, ctx.basisInvariantCount(basis).?);
const basis_audit = ctx.basisAudit(basis).?;
try testing.expectEqual(@as(u32), 0, basis_audit.distinct_projector_count);
try testing.expectEqual(@as(u32), 1, basis_audit.boundary_projector_count);
try testing.expectEqual(@as(u32), 1, basis_audit.total_projector_count);
const coverage = try ctx.impl().basisFormulaCoverageAudit(basis);
try testing.expectEqual(@as(u32), 1, coverage.path_count);
try testing.expectEqual(@as(u64), 0, coverage.local_step_count);
try testing.expectEqual(@as(u32), 1, coverage.boundary_projector_count);
try testing.expectEqual(@as(u64), 1, coverage.boundary_step_count);
try testing.expectEqual(@as(u64), 1, coverage.expandable_boundary_step_count);
try testing.expectEqual(@as(u64), 0, coverage.missing_boundary_step_count);
try testing.expectEqual(@as(u32), 1, coverage.distinct_boundary_projector_count);
try testing.expectEqual(@as(u32), 1, coverage.fully_expandable_path_count);
try testing.expect(coverage.first_missing_projector == null);

var sink: CountingSink = .{};
const audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors =
  ↪ .expanded_terms }, rendering.ExpansionFilter.acceptAll(), &sink);
try testing.expectEqual(@as(u32), 2, sink.term_count);
try testing.expectEqual(@as(u32), 2, sink.atom_count);
try testing.expectEqual(@as(u32), 1, sink.vector_spinor_identity_count);
try testing.expectEqual(@as(u32), 1, sink.gamma_trace_count);
try testing.expectEqual(@as(u16), 10, sink.first_gamma_trace_dimension.?.?);
try testing.expectEqual(@as(u8), 2, sink.first_gamma_trace_chirality.?.?);
try testing.expectEqual(@as(i32), -1, sink.first_gamma_trace_coefficient.?.numerator);
try testing.expectEqual(@as(u32), 10, sink.first_gamma_trace_coefficient.?.denominator);
try testing.expectEqual(@as(u64), 1, audit.boundary_projector_atoms);
try testing.expectEqual(@as(u64), 0, audit.local_projector_atoms);
try testing.expectEqual(@as(u64), 2, audit.emitted);

const descriptor = ctx.projectorDescriptor(sink.first_gamma_trace_operator_id.?.?);
try testing.expectEqual(projector.ProjectorKind.backend_specific_verified,
  ↪ descriptor.derivation.kind);
try testing.expectEqual(projector.ExpansionSource.backend_specific_verified,
  ↪ descriptor.derivation.source);
try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
  ↪ descriptor.derivation.status);
try testing.expectEqual(projector.ProjectorFormulaKind.orthogonal_gamma_traceless,
  ↪ descriptor.derivation.formula_kind);

```

```

try testing.expect(descriptor.derivation.formula_audit.normalized);
try testing.expect(descriptor.derivation.formula_audit.idempotent);
try testing.expect(descriptor.derivation.formula_audit.channel_separated);
try testing.expect(descriptor.derivation.formula_audit.gamma_traceless);

var reject_sink: CountingSink = .{};
const reject_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors =
  ↪ .expanded_terms }, rendering.ExpansionFilter.rejectAll(), &reject_sink);
try testing.expectEqual(@as(u32, 0), reject_sink.term_count);
try testing.expectEqual(@as(u64, 1), reject_audit.rejected);
try testing.expectEqual(@as(u64, 0), reject_audit.emitted);

var gamma_filter_sink: CountingSink = .{};
const gamma_filter_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors
  ↪ = .expanded_terms }, rendering.ExpansionFilter.withoutOperatorKind(.gamma),
  ↪ &gamma_filter_sink);
try testing.expectEqual(@as(u32, 1), gamma_filter_sink.term_count);
try testing.expectEqual(@as(u32, 1), gamma_filter_sink.vector_spinor_identity_count);
try testing.expectEqual(@as(u32, 0), gamma_filter_sink.gamma_trace_count);
try testing.expectEqual(@as(u64, 1), gamma_filter_audit.rejected);
try testing.expectEqual(@as(u64, 1), gamma_filter_audit.emitted);
}

test "context audits vector-spinor projector law from orthogonal dimension" {
  const testing = std.testing;

  const spin10 = orthogonalVectorSpinorFormulaAudit(10);
  try testing.expect(spin10.normalized);
  try testing.expect(spin10.idempotent);
  try testing.expect(spin10.channel_separated);
  try testing.expect(spin10.gamma_traceless);

  const rejected = orthogonalVectorSpinorFormulaAudit(0);
  try testing.expect(!rejected.normalized);
  try testing.expect(!rejected.idempotent);
  try testing.expect(!rejected.channel_separated);
  try testing.expect(!rejected.gamma_traceless);
}

test "context audits spinor-square gamma projector law symbolically" {
  const testing = std.testing;

  const so10: symmetry.SimpleLieAlgebra = .{ .family = .d, .rank = 5 };
  const spinor_right = [_]i16{ 0, 0, 0, 0, 1 };
  const spinor_left = [_]i16{ 0, 0, 0, 1, 0 };

  const vector_channel = orthogonalSpinorBilinearFormulaAudit(so10, spinor_right[0..],
    ↪ spinor_right[0..], 1, .none);
  try testing.expect(vector_channel.normalized);
  try testing.expect(vector_channel.idempotent);
  try testing.expect(vector_channel.channel_separated);

  const wrong_chirality = orthogonalSpinorBilinearFormulaAudit(so10, spinor_right[0..],
    ↪ spinor_left[0..], 1, .none);
  try testing.expect(!wrong_chirality.normalized);
  try testing.expect(!wrong_chirality.idempotent);
  try testing.expect(!wrong_chirality.channel_separated);

  const bad_duality = orthogonalSpinorBilinearFormulaAudit(so10, spinor_right[0..],
    ↪ spinor_right[0..], 3, .self_dual);
  try testing.expect(!bad_duality.normalized);
  try testing.expect(!bad_duality.idempotent);
  try testing.expect(!bad_duality.channel_separated);
}

```

```

test "context formula coverage audits missing boundary projector formulas" {
  const testing = std.testing;

  var ctx = try Context.init(testing.allocator);
  defer ctx.deinit();

  const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
  const vector = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 0 } });
  const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
  const projected = try
    ↪ ctx.registerRealization(realization.RealizationSpec.projected(vector, &.{ spinor,
    ↪ spinor }, &.{
      .init(.spinor, "a"),
      .init(.spinor, "b"),
    }, 0));
  const projected_leg = realization.ExternalLeg.realized(projected, &.{
    .init(.spinor, "a"),
    .init(.spinor, "b"),
  });
  const basis = try ctx.invariantBasis(.{
    .algebra = so10,
    .external_legs = &.{projected_leg},
    .target = .{ .irrep = vector },
  });

  const coverage = try ctx.impl().basisFormulaCoverageAudit(basis);
  try testing.expectEqual(@as(u32, 1), coverage.path_count);
  try testing.expectEqual(@as(u64, 0), coverage.local_step_count);
  try testing.expectEqual(@as(u64, 0), coverage.missing_step_count);
  try testing.expectEqual(@as(u32, 1), coverage.boundary_projector_count);
  try testing.expectEqual(@as(u64, 1), coverage.boundary_step_count);
  try testing.expectEqual(@as(u64, 0), coverage.expandable_boundary_step_count);
  try testing.expectEqual(@as(u64, 1), coverage.missing_boundary_step_count);
  try testing.expectEqual(@as(u32, 0), coverage.fully_expandable_path_count);
  try testing.expectEqual(projected.value, coverage.first_missing_boundary.?.value);

  const descriptor = ctx.projectorDescriptor(coverage.first_missing_projector.?.?);
  switch (descriptor.role) {
    .boundary_realization => |handle| try testing.expectEqual(projected.value,
    ↪ handle.value),
    else => return error.ExpectedBoundaryRealizationProjector,
  }
  try testing.expectEqual(projector.ExpansionStatus.expandable_named,
    ↪ descriptor.derivation.status);
  try testing.expectEqual(projector.ProjectorFormulaKind.named_only,
    ↪ descriptor.derivation.formula_kind);

  var expanded_sink: CountingSink = .{};
  try testing.expectError(error.ProjectorExpansionNotImplemented, ctx.renderInvariant(basis,
    ↪ .init(0), .{ .projectors = .expanded_terms }, &expanded_sink));
  try testing.expectEqual(@as(u32, 0), expanded_sink.term_count);
}

test "context streams Spin10 vector square as metric formula" {
  const testing = std.testing;

  var ctx = try Context.init(testing.allocator);
  defer ctx.deinit();

  const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
  const vector = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 0 } });
  const leg = realization.ExternalLeg.primitive(vector, &.{
    .init(.vector, "m"),

```



```

});
const basis = try ctx.invariantBasis({
  .algebra = so10,
  .external_legs = &.{ leg, leg },
});

var sink: CountingSink = .{};
const audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors =
  ⇨ .expanded_terms }, rendering.ExpansionFilter.acceptAll(), &sink);
try testing.expectEqual(@as(u32, 1), sink.term_count);
try testing.expectEqual(@as(u32, 1), sink.atom_count);
try testing.expectEqual(@as(u32, 1), sink.metric_pair_count);
try testing.expectEqual(@as(u32, 0), sink.projector_operator_count);
try testing.expectEqual(@as(u64, 1), audit.accepted);
try testing.expectEqual(@as(u64, 1), audit.emitted);
const paths = ctx.impl().couplings.basisPaths(basis).?;
const steps = ctx.impl().couplings.pathSteps(paths[0]);
const derivation = ctx.impl().projectors.projectorDerivation(steps[0].projector).?;
try testing.expectEqual(projector.ExpansionStatus.expandable_terms, derivation.status);
try testing.expectEqual(projector.ProjectorFormulaKind.orthogonal_vector_metric,
  ⇨ derivation.formula_kind);
try testing.expect(derivation.formula_audit.verified());
const first_counters = ctx.impl().projectors.expansionCounters();
try testing.expectEqual(@as(u64, 0), first_counters.formula_hits);
try testing.expectEqual(@as(u64, 1), first_counters.formula_misses);

var second_sink: CountingSink = .{};
_ = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors = .expanded_terms },
  ⇨ rendering.ExpansionFilter.acceptAll(), &second_sink);
const second_counters = ctx.impl().projectors.expansionCounters();
try testing.expectEqual(@as(u64, 1), second_counters.formula_hits);
try testing.expectEqual(@as(u64, 1), second_counters.formula_misses);

var projector_filter_sink: CountingSink = .{};
const projector_filter_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{
  ⇨ .projectors = .expanded_terms },
  ⇨ rendering.ExpansionFilter.withoutOperatorKind(.projector), &projector_filter_sink);
try testing.expectEqual(@as(u32, 1), projector_filter_sink.term_count);
try testing.expectEqual(@as(u32, 1), projector_filter_sink.metric_pair_count);
try testing.expectEqual(@as(u64, 1), projector_filter_audit.undecided);
try testing.expectEqual(@as(u64, 1), projector_filter_audit.emitted);
}

test "context streams Spin10 spinor conjugate spinor as spinor pairing formula" {
  const testing = std.testing;

  var ctx = try Context.init(testing.allocator);
  defer ctx.deinit();

  const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
  const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
  const conjugate_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 1, 0 } });
  const spinor_leg = realization.ExternalLeg.primitive(spinor, &.{
    .init(.spinor, "a"),
  });
  const conjugate_leg = realization.ExternalLeg.primitive(conjugate_spinor, &.{
    .init(.conjugate_spinor, "b"),
  });
  const basis = try ctx.invariantBasis({
    .algebra = so10,
    .external_legs = &.{ spinor_leg, conjugate_leg },
  });

  try testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?);

```

```

var sink: CountingSink = .{};
const audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors =
  ⇨ .expanded_terms }, rendering.ExpansionFilter.acceptAll(), &sink);
try testing.expectEqual(@as(u32, 1), sink.term_count);
try testing.expectEqual(@as(u32, 1), sink.atom_count);
try testing.expectEqual(@as(u32, 1), sink.spinor_pair_count);
try testing.expectEqual(@as(u16, 10), sink.first_spinor_pair_dimension.?);
try testing.expectEqual(@as(u8, 2), sink.first_spinor_pair_left_chirality.?);
try testing.expectEqual(@as(u8, 1), sink.first_spinor_pair_right_chirality.?);
try testing.expectEqual(@as(u32, 0), sink.projector_operator_count);
try testing.expectEqual(@as(u64, 1), audit.accepted);
try testing.expectEqual(@as(u64, 1), audit.emitted);

const descriptor = ctx.projectorDescriptor(sink.first_spinor_pair_operator_id.?);
switch (descriptor.role) {
  .product_channel => |channel| {
    try testing.expectEqual(spinor.value, channel.left.value);
    try testing.expectEqual(conjugate_spinor.value, channel.right.value);
    try testing.expectEqual(@as(u16, 0), channel.multiplicity_copy);
  },
  else => return error.ExpectedSpinorPairProductChannel,
}
try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
  ⇨ descriptor.derivation.status);
try testing.expectEqual(projector.ProjectorFormulaKind.orthogonal_spinor_pair,
  ⇨ descriptor.derivation.formula_kind);
try testing.expect(descriptor.derivation.formula_audit.verified());

var metric_filter_sink: CountingSink = .{};
const metric_filter_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{
  ⇨ .projectors = .expanded_terms },
  ⇨ rendering.ExpansionFilter.withoutOperatorKind(.metric), &metric_filter_sink);
try testing.expectEqual(@as(u32, 0), metric_filter_sink.term_count);
try testing.expectEqual(@as(u64, 1), metric_filter_audit.rejected);
try testing.expectEqual(@as(u64, 0), metric_filter_audit.emitted);
}

test "context streams Spin10 spinor spinor vector as gamma formula" {
  const testing = std.testing;

  var ctx = try Context.init(testing.allocator);
  defer ctx.deinit();

  const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
  const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
  const vector = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 0 } });
  const spinor_leg = realization.ExternalLeg.primitive(spinor, &{
    .init(.spinor, "a"),
  });
  const vector_leg = realization.ExternalLeg.primitive(vector, &{
    .init(.vector, "m"),
  });
  const basis = try ctx.invariantBasis(.{
    .algebra = so10,
    .external_legs = &{ spinor_leg, spinor_leg, vector_leg },
  });

  try testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?);
  var sink: CountingSink = .{};
  const audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors =
    ⇨ .expanded_terms }, rendering.ExpansionFilter.acceptAll(), &sink);
  try testing.expectEqual(@as(u32, 1), sink.term_count);
  try testing.expectEqual(@as(u32, 2), sink.atom_count);
  try testing.expectEqual(@as(u32, 1), sink.gamma_matrix_count);

```



```

try testing.expectEqual(@as(u32, 1), sink.metric_pair_count);
try testing.expectEqual(@as(u16, 10), sink.first_gamma_dimension.?);
try testing.expectEqual(@as(u8, 1), sink.first_gamma_rank.?);
try testing.expectEqual(@as(u8, 2), sink.first_gamma_chirality.?);
try testing.expectEqual(@as(u32, 0), sink.gamma_operator_count);
try testing.expectEqual(@as(u32, 0), sink.projector_operator_count);
const descriptor = ctx.projectorDescriptor(sink.first_gamma_operator_id.?);
switch (descriptor.role) {
    .product_channel => |channel| {
        try testing.expectEqual(spinor.value, channel.left.value);
        try testing.expectEqual(spinor.value, channel.right.value);
        try testing.expectEqual(vector.value, channel.output.value);
        try testing.expectEqual(@as(u16, 0), channel.multiplicity_copy);
    },
    else => return error.ExpectedGammaProductChannel,
}
try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
    ↪ descriptor.derivation.status);
try testing.expectEqual(projector.ProjectorFormulaKind.orthogonal_spinor_form_channel,
    ↪ descriptor.derivation.formula_kind);
try testing.expect(descriptor.derivation.formula_audit.verified());
try testing.expectEqual(@as(u64, 1), audit.accepted);
try testing.expectEqual(@as(u64, 1), audit.emitted);

var eval_sink: EvaluationCountingSink = .{};
const eval_audit = try ctx.evaluateBasisInvariant(basis, .init(0), .{}, &eval_sink);
try testing.expectEqual(@as(u64, 1), eval_audit.terms);
try testing.expectEqual(@as(u64, 0), eval_audit.rejected);
try testing.expectEqual(@as(u64, 0), eval_audit.lowered_clifford_factors);
try testing.expectEqual(@as(u32, 1), eval_sink.term_count);
try testing.expectEqual(@as(u32, 2), eval_sink.atom_count);
try testing.expectEqual(@as(u32, 1), eval_sink.gamma_matrix_count);
try testing.expectEqual(@as(u32, 1), eval_sink.metric_pair_count);
try testing.expectEqual(@as(u32, 0), eval_sink.compact_formula_count);
try testing.expectEqual(@as(i32, 1), eval_sink.first_coefficient?.numerator);
try testing.expectEqual(@as(u32, 1), eval_sink.first_coefficient?.denominator);
try testing.expectError(error.NotImplemented, ctx.evaluateInvariant(.init(0), .{},
    ↪ &eval_sink));
try testing.expectError(error.UnsupportedEvalSignature, ctx.evaluateBasisInvariant(basis,
    ↪ .init(0), .{ .signature = .euclidean }, &eval_sink));

var gamma_filter_sink: CountingSink = .{};
const gamma_filter_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors
    ↪ = .expanded_terms }, rendering.ExpansionFilter.withoutOperatorKind(.gamma),
    ↪ &gamma_filter_sink);
try testing.expectEqual(@as(u32, 0), gamma_filter_sink.term_count);
try testing.expectEqual(@as(u64, 1), gamma_filter_audit.rejected);
try testing.expectEqual(@as(u64, 0), gamma_filter_audit.emitted);

const coverage = try ctx.impl().basisFormulaCoverageAudit(basis);
try testing.expectEqual(@as(u32, 1), coverage.path_count);
try testing.expectEqual(@as(u64, 2), coverage.local_step_count);
try testing.expectEqual(@as(u64, 2), coverage.expandable_step_count);
try testing.expectEqual(@as(u64, 0), coverage.missing_step_count);
try testing.expectEqual(@as(u32, 1), coverage.fully_expandable_path_count);
}

test "context streams Spin10 conjugate spinor spinor vector as gamma formula" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });

```

```

const conjugate_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 1, 0 } });
const vector = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 0 } });
const spinor_leg = realization.ExternalLeg.primitive(conjugate_spinor, &.{
    .init(.conjugate_spinor, "a"),
});
const vector_leg = realization.ExternalLeg.primitive(vector, &.{
    .init(.vector, "m"),
});
const basis = try ctx.invariantBasis(.{
    .algebra = so10,
    .external_legs = &.{ spinor_leg, spinor_leg, vector_leg },
});

try testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?);
var sink: CountingSink = .{};
_ = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors = .expanded_terms },
    ⇨ rendering.ExpansionFilter.acceptAll(), &sink);
try testing.expectEqual(@as(u32, 1), sink.gamma_matrix_count);
try testing.expectEqual(@as(u16, 10), sink.first_gamma_dimension.?);
try testing.expectEqual(@as(u8, 1), sink.first_gamma_rank.?);
try testing.expectEqual(@as(u8, 1), sink.first_gamma_chirality.?);
}

test "context streams composed gamma product reducer atoms" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
    const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
    const conjugate_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 1, 0 } });
    const spinor_leg = realization.ExternalLeg.primitive(spinor, &.{
        .init(.spinor, "a"),
    });
    const basis = try ctx.invariantBasis(.{
        .algebra = so10,
        .external_legs = &.{ spinor_leg, spinor_leg, spinor_leg },
        .target = .{ .irrep = conjugate_spinor },
    });

    try testing.expectEqual(@as(u128, 2), ctx.basisInvariantCount(basis).?);
    var sink: CountingSink = .{};
    const audit = try ctx.renderBasisFiltered(basis, .{ .projectors = .expanded_terms },
        ⇨ rendering.ExpansionFilter.acceptAll(), &sink);
    try testing.expectEqual(@as(u32, 2), sink.term_count);
    try testing.expect(sink.gamma_matrix_count + sink.gamma_form_count > 0);
    try testing.expect(sink.gamma_action_count > 0);
    try testing.expect(sink.clifford_product_count > 0);
    try testing.expect(sink.first_clifford_product_left_rank.? > 0);
    try testing.expect(sink.first_clifford_product_right_rank.? > 0);
    try testing.expect(sink.first_clifford_product_coefficient.? != 0);
    try testing.expectEqual(@as(u64, 2), audit.emitted);

    var factor_sink: CountingSink = .{};
    const factor_audit = try ctx.renderBasisFiltered(basis, .{
        .projectors = .expanded_terms,
        .stream_clifford_product_factors = true,
    }, rendering.ExpansionFilter.acceptAll(), &factor_sink);
    try testing.expectEqual(@as(u32, 2), factor_sink.term_count);
    try testing.expect(factor_sink.clifford_product_factor_count > 0);
    try testing.expect(factor_sink.clifford_product_metric_factor_count > 0);
    try testing.expect(factor_sink.clifford_product_output_vector_factor_count > 0);
}

```

```

try testing.expectEqual(@as(u64, @intCast(factor_sink.clifford_product_factor_count)),
↳ factor_audit.clifford_product_factors);

var lowered_sink: CountingSink = .{};
const lowered_audit = try ctx.renderBasisFiltered(basis, .{
    .projectors = .expanded_terms,
    .lower_clifford_product_atoms = true,
}, rendering.ExpansionFilter.acceptAll(), &lowered_sink);
try testing.expectEqual(@as(u32, 2), lowered_sink.term_count);
try testing.expectEqual(@as(u32, 0), lowered_sink.clifford_product_count);
try testing.expect(lowered_sink.clifford_product_factor_count > 0);
try testing.expect(lowered_sink.clifford_product_metric_factor_count > 0);
try testing.expect(lowered_sink.clifford_product_output_vector_factor_count > 0);
try testing.expectEqual(lowered_sink.clifford_product_factor_count,
↳ lowered_sink.clifford_product_metric_factor_count +
↳ lowered_sink.clifford_product_output_vector_factor_count);
try testing.expectEqual(@as(u64, @intCast(lowered_sink.clifford_product_factor_count)),
↳ lowered_audit.clifford_product_factors);
try testing.expect(lowered_sink.first_clifford_product_factor_left_operator_id != null);
try testing.expect(lowered_sink.first_clifford_product_factor_right_operator_id != null);
try testing.expectEqual(@as(u16, 10),
↳ lowered_sink.first_clifford_product_factor_dimension.?);
try testing.expect(lowered_sink.first_clifford_product_factor_coefficient.? != 0);

var scan_sink: rendering.CliffordProductFactorAuditSink = .{};
const scan_audit = try ctx.renderBasisFiltered(basis, .{
    .projectors = .expanded_terms,
    .lower_clifford_product_atoms = true,
}, rendering.ExpansionFilter.acceptAll(), &scan_sink);
try testing.expectEqual(@as(u64, 2), scan_audit.emitted);
try testing.expectEqual(@as(u32, 2), scan_sink.term_count);
try testing.expect(scan_sink.scan.product_count > 0);
try testing.expectEqual(lowered_sink.clifford_product_factor_count,
↳ scan_sink.scan.factor_count);
try testing.expectEqual(lowered_sink.clifford_product_metric_factor_count,
↳ scan_sink.scan.metric_count);
try testing.expectEqual(lowered_sink.clifford_product_output_vector_factor_count,
↳ scan_sink.scan.output_vector_count);
try testing.expectEqual(@as(u16, 10), scan_sink.scan.first_orthogonal_dimension.?);
try testing.expect(scan_sink.scan.first_coefficient.? != 0);

var gamma_filter_sink: CountingSink = .{};
const gamma_filter_audit = try ctx.renderBasisFiltered(basis, .{ .projectors =
↳ .expanded_terms }, rendering.ExpansionFilter.withoutOperatorKind(.gamma),
↳ &gamma_filter_sink);
try testing.expectEqual(@as(u32, 0), gamma_filter_sink.term_count);
try testing.expectEqual(@as(u64, 2), gamma_filter_audit.rejected);
try testing.expectEqual(@as(u64, 0), gamma_filter_audit.emitted);
}

test "context streams Spin10 spinor spinor form as gamma-form formula" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
    const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
    const conjugate_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 1, 0 } });
    try expectSpin10GammaForm(&ctx, so10, spinor, .spinor, &.{ 0, 0, 1, 0, 0 }, 3, 2, .none);
    try expectSpin10GammaForm(&ctx, so10, spinor, .spinor, &.{ 0, 0, 0, 0, 2 }, 5, 2,
↳ .self_dual);
    try expectSpin10GammaForm(&ctx, so10, conjugate_spinor, .conjugate_spinor, &.{ 0, 0, 1, 0,
↳ 0 }, 3, 1, .none);

```

```

    try expectSpin10GammaForm(&ctx, so10, conjugate_spinor, .conjugate_spinor, &.{ 0, 0, 0, 2,
    ↪ 0 }, 5, 1, .anti_self_dual);
}

test "context streams A2 fundamental cubic as epsilon formula" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const su3 = try ctx.registerAlgebra(.{ .simple = .{ .family = .a, .rank = 2 } });
    const fundamental = try ctx.registerIrrep(su3, .{ .dynkin = &.{ 1, 0 } });
    const leg = realization.ExternalLeg.primitive(fundamental, &.{
        .init(.fundamental, "i"),
    });
    const basis = try ctx.invariantBasis(.{
        .algebra = su3,
        .external_legs = &.{ leg, leg, leg },
    });

    try testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?);
    var sink: CountingSink = .{};
    const audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors =
    ↪ .expanded_terms }, rendering.ExpansionFilter.acceptAll(), &sink);
    try testing.expectEqual(@as(u32, 1), sink.term_count);
    try testing.expectEqual(@as(u32, 1), sink.atom_count);
    try testing.expectEqual(@as(u32, 1), sink.epsilon_count);
    try testing.expectEqual(@as(u32, 0), sink.projector_operator_count);
    try testing.expectEqual(@as(u64, 1), audit.emitted);

    const paths = ctx.impl().couplings.basisPaths(basis).?;
    const steps = ctx.impl().couplings.pathSteps(paths[0]);
    try testing.expectEqual(@as(usize, 2), steps.len);
    for (steps) |step| {
        const derivation = ctx.impl().projectors.projectorDerivation(step.projector).?;
        try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
        ↪ derivation.status);
        try testing.expectEqual(projector.ProjectorFormulaKind.backend_specific_structure,
        ↪ derivation.formula_kind);
        try testing.expect(derivation.formula_audit.verified());
    }

    var epsilon_filter_sink: CountingSink = .{};
    const epsilon_filter_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{
    ↪ .projectors = .expanded_terms },
    ↪ rendering.ExpansionFilter.withoutOperatorKind(.epsilon), &epsilon_filter_sink);
    try testing.expectEqual(@as(u32, 0), epsilon_filter_sink.term_count);
    try testing.expectEqual(@as(u64, 1), epsilon_filter_audit.rejected);
    try testing.expectEqual(@as(u64, 0), epsilon_filter_audit.emitted);
}

test "context streams E6 fundamental cubic as structure formula" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const e6 = try ctx.registerAlgebra(.{ .simple = .{ .family = .e6, .rank = 6 } });
    const fundamental = try ctx.registerIrrep(e6, .{ .dynkin = &.{ 1, 0, 0, 0, 0, 0 } });
    const leg = realization.ExternalLeg.primitive(fundamental, &.{
        .init(.fundamental, "i"),
    });
    const basis = try ctx.invariantBasis(.{
        .algebra = e6,

```

```

        .external_legs = &.{ leg, leg, leg },
    });

    try testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?);
    const coverage = try ctx.basisFormulaCoverageAudit(basis);
    try testing.expectEqual(@as(u32, 1), coverage.path_count);
    try testing.expectEqual(@as(u64, 2), coverage.local_step_count);
    try testing.expectEqual(@as(u64, 2), coverage.expandable_step_count);
    try testing.expectEqual(@as(u64, 0), coverage.missing_step_count);
    try testing.expectEqual(@as(u32, 1), coverage.fully_expandable_path_count);

    var sink: CountingSink = .{};
    const audit = try ctx.renderBasisFiltered(basis, .{ .projectors = .expanded_terms },
        ↪ rendering.ExpansionFilter.acceptAll(), &sink);
    try testing.expectEqual(@as(u64, 1), audit.invariants);
    try testing.expectEqual(@as(u64, 1), audit.emitted);
    try testing.expectEqual(@as(u32, 1), sink.term_count);
    try testing.expectEqual(@as(u32, 1), sink.atom_count);
    try testing.expectEqual(@as(u32, 1), sink.structure_constant_count);
    try testing.expectEqual(symmetry.LieFamily.e6, sink.first_structure_constant_family.?);
    try testing.expectEqual(@as(u8, 6), sink.first_structure_constant_rank.?);
    try testing.expectEqual(@as(u32, 0), sink.projector_operator_count);

    const paths = ctx.impl().couplings.basisPaths(basis).?;
    const steps = ctx.impl().couplings.pathSteps(paths[0]);
    for (steps |step| {
        const derivation = ctx.impl().projectors.projectorDerivation(step.projector).?;
        try testing.expectEqual(projector.ProjectorKind.backend_specific_verified,
            ↪ derivation.kind);
        try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
            ↪ derivation.status);
        try testing.expectEqual(projector.ProjectorFormulaKind.backend_specific_structure,
            ↪ derivation.formula_kind);
        try testing.expect(derivation.formula_audit.verified());
    })

    var structure_filter_sink: CountingSink = .{};
    const structure_filter_audit = try ctx.renderBasisFiltered(basis, .{ .projectors =
        ↪ .expanded_terms }, rendering.ExpansionFilter.withoutOperatorKind(.structure_constant),
        ↪ &structure_filter_sink);
    try testing.expectEqual(@as(u32, 0), structure_filter_sink.term_count);
    try testing.expectEqual(@as(u64, 1), structure_filter_audit.rejected);
    try testing.expectEqual(@as(u64, 0), structure_filter_audit.emitted);
}

test "context streams E8 adjoint cubic as structure formula" {
    const testing = std.testing;

    var ctx = try Context.init(testing allocator);
    defer ctx.deinit();

    const e8 = try ctx.registerAlgebra(.{ .simple = .{ .family = .e8, .rank = 8 } });
    const adjoint = try ctx.registerIrrep(e8, .{ .dynkin = &.{ 0, 0, 0, 0, 0, 0, 0, 1, 0 } });
    const leg = realization.ExternalLeg.primitive(adjoint, &.{
        .init(.adjoint, "a"),
    });
    const basis = try ctx.invariantBasis(.{
        .algebra = e8,
        .external_legs = &.{ leg, leg, leg },
    });

    try testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?);
    const coverage = try ctx.basisFormulaCoverageAudit(basis);
    try testing.expectEqual(@as(u32, 1), coverage.path_count);

```

```

try testing.expectEqual(@as(u64, 2), coverage.local_step_count);
try testing.expectEqual(@as(u64, 2), coverage.expandable_step_count);
try testing.expectEqual(@as(u64, 0), coverage.missing_step_count);
try testing.expectEqual(@as(u32, 1), coverage.fully_expandable_path_count);

var sink: CountingSink = {};
const audit = try ctx.renderBasisFiltered(basis, {.projectors = .expanded_terms },
    ↪ rendering.ExpansionFilter.acceptAll(), &sink);
try testing.expectEqual(@as(u64, 1), audit.invariants);
try testing.expectEqual(@as(u64, 1), audit.emitted);
try testing.expectEqual(@as(u32, 1), sink.term_count);
try testing.expectEqual(@as(u32, 1), sink.atom_count);
try testing.expectEqual(@as(u32, 1), sink.structure_constant_count);
try testing.expectEqual(symmetry.LieFamily.e8, sink.first_structure_constant_family.?);
try testing.expectEqual(@as(u8, 8), sink.first_structure_constant_rank.?);
try testing.expectEqual(@as(u32, 0), sink.projector_operator_count);

const paths = ctx.impl().couplings.basisPaths(basis).?;
const steps = ctx.impl().couplings.pathSteps(paths[0]);
for (steps) |step| {
    const derivation = ctx.impl().projectors.projectorDerivation(step.projector).?;
    try testing.expectEqual(projector.ProjectorKind.backend_specific_verified,
        ↪ derivation.kind);
    try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
        ↪ derivation.status);
    try testing.expectEqual(projector.ProjectorFormulaKind.backend_specific_structure,
        ↪ derivation.formula_kind);
    try testing.expect(derivation.formula_audit.verified());
}
}

test "context streams singlet product as identity formula" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const so10 = try ctx.registerAlgebra(.{ .simple = {.family = .d, .rank = 5 } });
    const singlet = try ctx.registerIrrep(so10, {.dynkin = &.{ 0, 0, 0, 0, 0 } });
    const spinor = try ctx.registerIrrep(so10, {.dynkin = &.{ 0, 0, 0, 0, 1 } });
    const singlet_leg = realization.ExternalLeg.primitive(singlet, &.{
        .init(.custom, "s"),
    });
    const spinor_leg = realization.ExternalLeg.primitive(spinor, &.{
        .init(.spinor, "a"),
    });
    const basis = try ctx.invariantBasis(.{
        .algebra = so10,
        .external_legs = &.{ singlet_leg, spinor_leg },
        .target = {.irrep = spinor },
    });

    try testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?);
    var sink: CountingSink = {};
    const audit = try ctx.renderInvariantFiltered(basis, .init(0), {.projectors =
        ↪ .expanded_terms }, rendering.ExpansionFilter.acceptAll(), &sink);
    try testing.expectEqual(@as(u32, 1), sink.term_count);
    try testing.expectEqual(@as(u32, 1), sink.atom_count);
    try testing.expectEqual(@as(u32, 1), sink.identity_route_count);
    try testing.expectEqual(@as(u32, 0), sink.product_identity_count);
    try testing.expectEqual(@as(u64, 1), audit.accepted);
    try testing.expectEqual(@as(u64, 1), audit.emitted);

    const descriptor = ctx.projectorDescriptor(sink.first_identity_route_operator_id.?).?;

```



```

switch (descriptor.role) {
    .product_channel => |channel| {
        try testing.expectEqual(singlet.value, channel.left.value);
        try testing.expectEqual(spinor.value, channel.right.value);
        try testing.expectEqual(spinor.value, channel.output.value);
        try testing.expectEqual(@as(u16, 0), channel.multiplicity_copy);
    },
    else => return error.ExpectedIdentityProductChannel,
}
try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
    ↪ descriptor.derivation.status);
try testing.expectEqual(projector.ProjectorFormulaKind.identity,
    ↪ descriptor.derivation.formula_kind);
try testing.expect(descriptor.derivation.formula_audit.verified());

var projector_filter_sink: CountingSink = .{};
const projector_filter_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{
    ↪ .projectors = .expanded_terms },
    ↪ rendering.ExpansionFilter.withoutOperatorKind(.projector), &projector_filter_sink);
try testing.expectEqual(@as(u32, 1), projector_filter_sink.term_count);
try testing.expectEqual(@as(u32, 1), projector_filter_sink.identity_route_count);
try testing.expectEqual(@as(u64, 0), projector_filter_audit.rejected);
try testing.expectEqual(@as(u64, 1), projector_filter_audit.emitted);
}

test "context streams Spin10 spinor tower Cartan product formula" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
    const tower3 = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 3 } });
    const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
    const tower4 = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 4 } });
    const tower_leg = realization.ExternalLeg.primitive(tower3, &.{
        .init(.custom, "T"),
    });
    const spinor_leg = realization.ExternalLeg.primitive(spinor, &.{
        .init(.spinor, "a"),
    });
    const basis = try ctx.invariantBasis(.{
        .algebra = so10,
        .external_legs = &.{ tower_leg, spinor_leg },
        .target = .{ .irrep = tower4 },
    });

    try testing.expectEqual(@as(u128, 1), ctx.basisInvariantCount(basis).?);
    var sink: CountingSink = .{};
    const audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors =
        ↪ .expanded_terms }, rendering.ExpansionFilter.acceptAll(), &sink);
    try testing.expectEqual(@as(u32, 1), sink.term_count);
    try testing.expectEqual(@as(u32, 4), sink.atom_count);
    try testing.expectEqual(@as(u32, 4), sink.spinor_index_delta_count);
    try testing.expectEqual(@as(u32, 0), sink.spinor_symmetrized_product_count);
    try testing.expectEqual(@as(u32, 0), sink.cartan_product_count);
    try testing.expectEqual(@as(u64, 1), audit.accepted);
    try testing.expectEqual(@as(u64, 1), audit.emitted);

    const descriptor = ctx.projectorDescriptor(sink.first_spinor_index_delta_operator_id.?).?;
    switch (descriptor.role) {
        .product_channel => |channel| {
            try testing.expectEqual(tower3.value, channel.left.value);
            try testing.expectEqual(spinor.value, channel.right.value);

```

```

        try testing.expectEqual(tower4.value, channel.output.value);
        try testing.expectEqual(@as(u16, 0), channel.multiplicity_copy);
    },
    else => return error.ExpectedCartanProductChannel,
}
try testing.expectEqual(projector.ExpansionStatus.expandable_terms,
    ↪ descriptor.derivation.status);
try testing.expectEqual(projector.ProjectorFormulaKind.cartan_product_channel,
    ↪ descriptor.derivation.formula_kind);
try testing.expect(descriptor.derivation.formula_audit.verified());

var projector_filter_sink: CountingSink = .{};
const projector_filter_audit = try ctx.renderInvariantFiltered(basis, .init(0), .{
    ↪ .projectors = .expanded_terms },
    ↪ rendering.ExpansionFilter.withoutOperatorKind(.projector), &projector_filter_sink);
try testing.expectEqual(@as(u32, 1), projector_filter_sink.term_count);
try testing.expectEqual(@as(u32, 4), projector_filter_sink.spinor_index_delta_count);
try testing.expectEqual(@as(u64, 0), projector_filter_audit.rejected);
try testing.expectEqual(@as(u64, 1), projector_filter_audit.emitted);

var gamma_only_sink: CountingSink = .{};
try ctx.renderInvariant(basis, .init(0), .{
    .projectors = .expanded_terms,
    .gamma_only = true,
}, &gamma_only_sink);
try testing.expectEqual(@as(u32, 1), gamma_only_sink.term_count);
try testing.expectEqual(@as(u32, 4), gamma_only_sink.spinor_index_delta_count);
}

test "context streams Spin10 spinor tower tensor-spinor formula" {
    var ctx = try Context.init(std.testing.allocator);
    defer ctx.deinit();

    const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
    const tower4 = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 4 } });
    const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });

    const tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 1, 0, 3 } });
    const shifted_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 1, 0, 2 }
    ↪ });
    const terminal_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 1, 1, 0, 1
    ↪ } });
    const opposite_terminal_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0,
    ↪ 1, 1, 0 } });
    const tensor_form = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 2, 0, 0 } });
    const preserve_tensor_form = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 1, 1, 0, 0 }
    ↪ });
    const shifted_tensor_form = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 1, 0, 1, 1 }
    ↪ });
    const shift_down_tensor_form = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 1, 1, 1
    ↪ } });
    const shift_down_any_tensor_form = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0,
    ↪ 1, 1 } });
    const remove_shift_down_tensor_form = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 1,
    ↪ 0, 1, 1 } });
    const rank_form_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 1, 2
    ↪ } });
    const rank_form_tower_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 1, 3
    ↪ } });
    const all_shift_tower_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 1,
    ↪ 0, 1, 2 } });
    const remove_shift_tower_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1,
    ↪ 0, 1, 0, 1 } });

```



```

const wrapped_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 3 }
↳ });
const terminal_wrapped_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 1,
↳ 0, 0, 1 } });
const opposite_vector_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 1, 0
↳ } });
const opposite_rank_split_terminal_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin
↳ = &.{ 0, 1, 0, 1, 0 } });
const opposite_shift_terminal_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{
↳ 1, 0, 1, 1, 0 } });
const split_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 1, 0, 0, 2 }
↳ });
const opposite_add_shift_terminal_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin =
↳ &.{ 0, 1, 1, 1, 0 } });
const rank3_terminal_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 1,
↳ 0, 1 } });
const rank3_tower_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 1, 0,
↳ 2 } });
const opposite_rank4_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0,
↳ 2, 1 } });
const opposite_remove_shift_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{
↳ 1, 0, 0, 2, 1 } });
const opposite_rank3_power_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0,
↳ 0, 2, 1, 0 } });
const opposite_rank3_tower_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0,
↳ 0, 1, 2, 0 } });
const opposite_form_add_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 1,
↳ 0, 2, 0 } });
const opposite_all_shift_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1,
↳ 0, 1, 2, 0 } });
const opposite_shift_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0,
↳ 2, 0 } });
const opposite_merge_tensor_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 1, 0,
↳ 2, 1 } });
const opposite_tower4 = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 4, 0 } });
const opposite_tower3 = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 3, 0 } });

try expectSpin10TensorSpinorProjection(&ctx, so10, tower4, spinor, &.{ 0, 0, 1, 0, 3 }, 3,
↳ 1, 0b100, 3, 2, .orthogonal_spinor_tower_form_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, tower4, spinor, &.{ 1, 0, 0, 0, 3 }, 1,
↳ 1, 0b001, 3, 2, .orthogonal_spinor_tower_form_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, tensor_spinor, spinor, &.{ 1, 0, 1, 0,
↳ 2 }, 1, 2, 0b101, 2, 2, .orthogonal_tensor_spinor_tower_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, shifted_tensor_spinor, spinor, &.{ 0,
↳ 1, 1, 0, 1 }, 2, 2, 0b110, 1, 2, .orthogonal_tensor_spinor_shift_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, tensor_form, spinor, &.{ 0, 0, 1, 1, 0
↳ }, 3, 1, 0b100, 1, 1, .orthogonal_tensor_form_spinor_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, preserve_tensor_form, spinor, &.{ 0, 1,
↳ 1, 0, 1 }, 2, 2, 0b0110, 1, 2, .orthogonal_tensor_form_spinor_preserve_channel);
try expectSpin10TensorFormProjection(&ctx, so10, terminal_tensor_spinor, spinor, &.{ 0, 0,
↳ 2, 0, 0 }, 0b110, (@as(u128, 2) << 8, 2, .orthogonal_tensor_spinor_terminal_channel);
try expectSpin10TensorFormProjection(&ctx, so10, terminal_tensor_spinor, spinor, &.{ 0, 1,
↳ 0, 1, 1 }, 0b110, (@as(u128, 1) << 4) | (@as(u128, 1) << 12), 2,
↳ .orthogonal_tensor_spinor_terminal_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, shifted_tensor_form, spinor, &.{ 0, 0,
↳ 1, 1, 0 }, 3, 1, 0b100, 1, 1, .orthogonal_tensor_form_spinor_shift_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, shift_down_tensor_form, spinor, &.{ 0,
↳ 0, 2, 1, 0 }, 3, 1, 0b100, 1, 1, .orthogonal_tensor_form_spinor_shift_down_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, shift_down_tensor_form, spinor, &.{ 1,
↳ 0, 0, 1, 2 }, 1, 2, 0b1001, 1, 2,
↳ .orthogonal_tensor_form_spinor_shift_down_two_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, shift_down_tensor_form, spinor, &.{ 1,
↳ 0, 1, 1, 0 }, 1, 2, 0b0101, 1, 1,
↳ .orthogonal_tensor_form_spinor_mixed_shift_down_channel);

```

```

try expectSpin10TensorSpinorProjection(&ctx, so10, shift_down_tensor_form, spinor, &{ 0,
↳ 1, 1, 0, 1 }, 2, 2, 0b0110, 1, 2,
↳ .orthogonal_tensor_form_spinor_all_shift_down_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, shift_down_any_tensor_form, spinor, &{
↳ 2, 0, 0, 1, 0 }, 1, 1, 0b0001, 1, 1,
↳ .orthogonal_tensor_form_spinor_shift_down_any_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, remove_shift_down_tensor_form, spinor,
↳ &{ 0, 1, 1, 0, 1 }, 2, 2, 0b0110, 1, 2,
↳ .orthogonal_tensor_form_spinor_remove_shift_down_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, rank_form_tensor_spinor, spinor, &{ 1,
↳ 0, 0, 0, 2 }, 1, 1, 0b001, 2, 2, .orthogonal_tensor_spinor_form_tower_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, rank_form_tower_spinor, spinor, &{ 0,
↳ 0, 0, 0, 3 }, 0, 0, 0, 3, 2, .orthogonal_tensor_spinor_form_tower_terminal_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, all_shift_tower_tensor_spinor, spinor,
↳ &{ 1, 0, 1, 0, 2 }, 1, 2, 0b0101, 2, 2,
↳ .orthogonal_tensor_spinor_form_tower_all_shift_down_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, remove_shift_tower_tensor_spinor,
↳ spinor, &{ 0, 1, 0, 0, 2 }, 2, 1, 0b0010, 2, 2,
↳ .orthogonal_tensor_spinor_form_tower_remove_shift_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, wrapped_tensor_spinor, spinor, &{ 0,
↳ 0, 0, 1, 3 }, 4, 1, 0b1000, 2, 2, .orthogonal_tensor_spinor_rank_wrap_shift_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, wrapped_tensor_spinor, spinor, &{ 2,
↳ 0, 0, 0, 2 }, 1, 1, 0b001, 2, 2, .orthogonal_tensor_spinor_form_power_channel);
try expectSpin10TensorFormProjection(&ctx, so10, terminal_wrapped_tensor_spinor, spinor,
↳ &{ 0, 1, 0, 1, 1 }, 0b0011, (@as(u128, 1) << 4) | (@as(u128, 1) << 12), 2,
↳ .orthogonal_tensor_spinor_rank_wrap_terminal_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, terminal_wrapped_tensor_spinor, spinor,
↳ &{ 1, 0, 0, 0, 2 }, 1, 1, 0b001, 2, 2, .orthogonal_tensor_spinor_form_tower_channel);
try expectSpin10TensorFormProjection(&ctx, so10, opposite_vector_spinor, spinor, &{ 0, 0,
↳ 1, 0, 0 }, 0b0001, @as(u128, 1) << 8, 1,
↳ .orthogonal_tensor_spinor_opposite_terminal_channel);
try expectSpin10TensorFormProjection(&ctx, so10,
↳ opposite_rank_split_terminal_tensor_spinor, spinor, &{ 1, 0, 1, 0, 0 }, 0b0010,
↳ (@as(u128, 1) << 0) | (@as(u128, 1) << 8), 2,
↳ .orthogonal_tensor_spinor_opposite_rank_split_terminal_channel);
try expectSpin10TensorFormProjection(&ctx, so10, opposite_shift_terminal_tensor_spinor,
↳ spinor, &{ 0, 1, 0, 1, 1 }, 0b0101, (@as(u128, 1) << 4) | (@as(u128, 1) << 12), 2,
↳ .orthogonal_tensor_spinor_opposite_shift_terminal_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, split_tensor_spinor, spinor, &{ 1, 0,
↳ 0, 1, 2 }, 1, 2, 0b1001, 1, 2, .orthogonal_tensor_spinor_rank_split_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, split_tensor_spinor, spinor, &{ 0, 0,
↳ 0, 0, 3 }, 0, 0, 0, 3, 2, .orthogonal_tensor_spinor_form_tower_terminal_channel);
try expectSpin10TensorFormProjection(&ctx, so10,
↳ opposite_add_shift_terminal_tensor_spinor, spinor, &{ 1, 1, 0, 1, 1 }, 0b0110,
↳ (@as(u128, 1) << 0) | (@as(u128, 1) << 4) | (@as(u128, 1) << 12), 3,
↳ .orthogonal_tensor_spinor_opposite_form_add_shift_terminal_channel);
try expectSpin10TensorFormProjection(&ctx, so10, rank3_terminal_tensor_spinor, spinor, &{
↳ 0, 0, 2, 0, 0 }, 0b0100, @as(u128, 2) << 8, 2,
↳ .orthogonal_tensor_spinor_form_power_terminal_channel);
try expectSpin10TensorFormProjection(&ctx, so10, rank3_terminal_tensor_spinor, spinor, &{
↳ 0, 1, 0, 1, 1 }, 0b0100, (@as(u128, 1) << 4) | (@as(u128, 1) << 12), 2,
↳ .orthogonal_tensor_spinor_rank_split_terminal_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, rank3_terminal_tensor_spinor, spinor,
↳ &{ 1, 0, 0, 0, 2 }, 1, 1, 0b0001, 2, 2,
↳ .orthogonal_tensor_spinor_form_tower_shift_down_channel);
try expectSpin10TensorFormProjection(&ctx, so10, rank3_terminal_tensor_spinor, spinor, &{
↳ 1, 0, 1, 0, 0 }, 0b0100, (@as(u128, 1) << 0) | (@as(u128, 1) << 8), 2,
↳ .orthogonal_tensor_spinor_form_add_terminal_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, rank3_tower_tensor_spinor, spinor, &{
↳ 0, 0, 1, 0, 3 }, 3, 1, 0b0100, 3, 2, .orthogonal_tensor_spinor_tower_raise_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, opposite_rank4_tensor_spinor, spinor,
↳ &{ 0, 0, 1, 2, 0 }, 3, 1, 0b0100, 2, 1,
↳ .orthogonal_tensor_spinor_opposite_form_tower_shift_down_channel);

```

```

try expectSpin10TensorSpinorProjection(&ctx, so10, opposite_rank4_tensor_spinor, spinor,
↳ &.{ 1, 0, 0, 2, 0 }, 1, 1, 0b0001, 2, 1,
↳ .orthogonal_tensor_spinor_opposite_form_tower_shift_down_channel);
try expectSpin10TensorFormProjection(&ctx, so10, opposite_rank4_tensor_spinor, spinor, &.{
↳ 0, 1, 0, 1, 1 }, 0b1000, (@as(u128, 1) << 4) | (@as(u128, 1) << 12), 2,
↳ .orthogonal_tensor_spinor_opposite_form_add_terminal_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, opposite_remove_shift_tensor_spinor,
↳ spinor, &.{ 0, 1, 0, 2, 0 }, 2, 1, 0b0010, 2, 1,
↳ .orthogonal_tensor_spinor_opposite_form_tower_remove_shift_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, opposite_rank3_power_tensor_spinor,
↳ spinor, &.{ 0, 0, 1, 2, 0 }, 3, 1, 0b0100, 2, 1,
↳ .orthogonal_tensor_spinor_opposite_form_tower_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, opposite_rank3_tower_tensor_spinor,
↳ spinor, &.{ 0, 0, 0, 3, 0 }, 0, 0, 0, 3, 1,
↳ .orthogonal_tensor_spinor_opposite_form_tower_terminal_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, opposite_tower4, spinor, &.{ 0, 0, 0,
↳ 3, 0 }, 0, 0, 0, 3, 1, .orthogonal_spinor_tower_opposite_lower_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, opposite_tower3, spinor, &.{ 0, 0, 0,
↳ 3, 1 }, 4, 1, 0b1000, 2, 1, .orthogonal_spinor_tower_opposite_form_channel);
try expectSpin10TensorMiddleFormProjection(&ctx, so10, opposite_tower3, spinor, &.{ 0, 0,
↳ 0, 2, 0 }, 0, 5, .anti_self_dual, .orthogonal_spinor_tower_middle_form_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, opposite_all_shift_tensor_spinor,
↳ spinor, &.{ 0, 1, 0, 2, 1 }, 2, 2, 0b1010, 1, 1,
↳ .orthogonal_tensor_spinor_opposite_all_shift_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, opposite_form_add_tensor_spinor,
↳ spinor, &.{ 0, 1, 0, 2, 1 }, 2, 2, 0b1010, 1, 1,
↳ .orthogonal_tensor_spinor_opposite_form_add_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, opposite_form_add_tensor_spinor,
↳ spinor, &.{ 1, 0, 1, 1, 0 }, 1, 2, 0b0101, 1, 1,
↳ .orthogonal_tensor_spinor_opposite_rank_split_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, opposite_rank3_tower_tensor_spinor,
↳ spinor, &.{ 0, 0, 1, 1, 0 }, 3, 1, 0b0100, 1, 1,
↳ .orthogonal_tensor_spinor_opposite_tower_lower_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, opposite_merge_tensor_spinor, spinor,
↳ &.{ 0, 0, 1, 2, 0 }, 3, 1, 0b0100, 2, 1,
↳ .orthogonal_tensor_spinor_opposite_form_tower_merge_channel);
try expectSpin10TensorSpinorProjection(&ctx, so10, opposite_shift_tensor_spinor, spinor,
↳ &.{ 0, 0, 1, 1, 0 }, 3, 1, 0b0100, 1, 1,
↳ .orthogonal_tensor_spinor_opposite_shift_channel);
try expectSpin10TensorMiddleFormProjection(&ctx, so10, opposite_terminal_tensor_spinor,
↳ spinor, &.{ 0, 0, 0, 2, 0 }, 0b100, 5, .anti_self_dual,
↳ .orthogonal_tensor_spinor_middle_form_channel);
try expectSpin10TensorFormProjection(&ctx, so10, opposite_terminal_tensor_spinor, spinor,
↳ &.{ 0, 0, 1, 0, 0 }, 0b100, @as(u128, 1) << 8, 1,
↳ .orthogonal_tensor_spinor_form_terminal_channel);
}

test "context refuses expanded render success for missing projector formulas" {
  const testing = std.testing;

  var ctx = try Context.init(testing.allocator);
  defer ctx.deinit();

  const su2 = try ctx.registerAlgebra(.{ .simple = .{ .family = .a, .rank = 1 } });
  const fundamental = try ctx.registerIrrep(su2, .{ .dynkin = &.{1} });
  const leg = realization.ExternalLeg.primitive(fundamental, &.{
    .init(.fundamental, "i"),
  });
  const basis = try ctx.invariantBasis(.{
    .algebra = su2,
    .external_legs = &.{ leg, leg },
  });

  var sink: CountingSink = .{};

```

```

    try testing.expectError(error.ProjectorExpansionNotImplemented, ctx.renderInvariant(basis,
    ↪ .init(0), .{ .projectors = .expanded_terms }, &sink));
    try testing.expectEqual(@as(u32, 0), sink.term_count);
}

test "context reject filter stops non-empty expansion before projector formulas" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const su2 = try ctx.registerAlgebra(.{ .simple = .{ .family = .a, .rank = 1 } });
    const fundamental = try ctx.registerIrrep(su2, .{ .dynkin = &.{1} });
    const leg = realization.ExternalLeg.primitive(fundamental, &.{
        .init(.fundamental, "i"),
    });
    const basis = try ctx.invariantBasis(.{
        .algebra = su2,
        .external_legs = &.{ leg, leg },
    });

    var sink: CountingSink = .{};
    const audit = try ctx.renderInvariantFiltered(basis, .init(0), .{ .projectors =
    ↪ .expanded_terms }, rendering.ExpansionFilter.rejectAll(), &sink);
    try testing.expectEqual(@as(u32, 0), sink.term_count);
    try testing.expectEqual(@as(u64, 1), audit.rejected);
    try testing.expectEqual(@as(u64, 0), audit.emitted);
}

test "context reports unsupported fixed tree instead of empty basis" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const su2 = try ctx.registerAlgebra(.{ .simple = .{ .family = .a, .rank = 1 } });
    const fundamental = try ctx.registerIrrep(su2, .{ .dynkin = &.{1} });
    const leg = realization.ExternalLeg.primitive(fundamental, &.{
        .init(.fundamental, "i"),
    });

    try testing.expectError(error.FixedTreeCountingNotImplemented, ctx.invariantBasis(.{
        .algebra = su2,
        .external_legs = &.{ leg, leg },
        .tree_policy = .{ .fixed = &.{ 0, 1 } },
    }));
}

test "context reports unresolved registered realization instead of empty basis" {
    const testing = std.testing;

    var ctx = try Context.init(testing.allocator);
    defer ctx.deinit();

    const su2 = try ctx.registerAlgebra(.{ .simple = .{ .family = .a, .rank = 1 } });
    const unresolved = realization.ExternalLeg.registered("missing-realization", &.{
        .init(.fundamental, "i"),
    });

    try testing.expectError(error.RegisteredRealizationNotResolved, ctx.invariantBasis(.{
        .algebra = su2,
        .external_legs = &.{ unresolved },
    }));
}

```

```

test "context counts dual-pair invariant across ADE families" {
  const testing = std.testing;

  var ctx = try Context.init(testing.allocator);
  defer ctx.deinit();

  try expectDualPairInvariantCount(&ctx, .{ .family = .a, .rank = 2 }, &.{ 1, 0 });
  try expectDualPairInvariantCount(&ctx, .{ .family = .d, .rank = 5 }, &.{ 0, 0, 0, 0, 1 });
  try expectDualPairInvariantCount(&ctx, .{ .family = .e6, .rank = 6 }, &.{ 1, 0, 0, 0, 0, 0
    ↪ });
  try expectDualPairInvariantCount(&ctx, .{ .family = .e7, .rank = 7 }, &.{ 0, 0, 0, 0, 0,
    ↪ 1, 0 });
  try expectDualPairInvariantCount(&ctx, .{ .family = .e8, .rank = 8 }, &.{ 0, 0, 0, 0, 0,
    ↪ 0, 1, 0 });
}

test "context counts representative multi-leg ADE invariants" {
  const testing = std.testing;

  var ctx = try Context.init(testing.allocator);
  defer ctx.deinit();

  try expectRepeatedLegInvariantCount(&ctx, .{ .family = .a, .rank = 2 }, &.{ 1, 0 }, 3, 1);
  try expectRepeatedLegInvariantCount(&ctx, .{ .family = .d, .rank = 5 }, &.{ 1, 0, 0, 0, 0
    ↪ }, 2, 1);
  try expectRepeatedLegInvariantCount(&ctx, .{ .family = .e6, .rank = 6 }, &.{ 1, 0, 0, 0,
    ↪ 0, 0 }, 3, 1);
  try expectRepeatedLegInvariantCount(&ctx, .{ .family = .e8, .rank = 8 }, &.{ 0, 0, 0, 0,
    ↪ 0, 0, 1, 0 }, 3, 1);
}

test "context benchmarks Spin10 spinor twelfth power invariant paths" {
  const testing = std.testing;

  var ctx = try Context.init(testing.allocator);
  defer ctx.deinit();

  const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
  const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
  const leg = realization.ExternalLeg.primitive(spinor, &.{
    .init(.spinor, "a"),
  });
  const basis = try ctx.invariantBasis(.{
    .algebra = so10,
    .external_legs = &.{ leg, leg, leg, leg, leg, leg, leg, leg, leg, leg, leg, leg },
  });
  const audit = ctx.impl().couplings.basisAudit(basis).?;

  try testing.expectEqual(@as(u128, 73744), audit.path_count);
  try testing.expectEqual(@as(u128, 73744), ctx.basisInvariantCount(basis).?);
  try testing.expectEqual(@as(u128, 73744), ctx.basisAudit(basis).?.path_count);
  try testing.expectEqual(@as(u32, 73744), audit.stored_path_count);
  try testing.expectEqual(@as(u16, 11), audit.max_path_step_len);
  try testing.expect(audit.step_count <= 73744 * 11);
  try testing.expect(audit.distinct_projector_count < audit.step_count);
  const coverage = try ctx.impl().basisFormulaCoverageAudit(basis);
  try testing.expectEqual(@as(u32, 73744), coverage.path_count);
  try testing.expectEqual(coverage.local_step_count, coverage.expandable_step_count +
    ↪ coverage.missing_step_count);
  try testing.expectEqual(@as(u64, 0), coverage.missing_step_count);
  try testing.expectEqual(@as(u32, 73744), coverage.fully_expandable_path_count);
  try testing.expect(coverage.expandable_step_count > 73744);
  try testing.expect(coverage.first_missing_projector == null);
}

```

```

var sink: CountingSink = .{};
const expanded_audit = try ctx.renderBasisFiltered(basis, .{ .projectors = .expanded_terms
↪ }, rendering.ExpansionFilter.acceptAll(), &sink);
try testing.expectEqual(@as(u64, 73744), expanded_audit.invariants);
try testing.expect(sink.term_count >= 73744);
try testing.expectEqual(@as(u64, sink.term_count), expanded_audit.emitted);
try testing.expectEqual(@as(u32, 0), sink.projector_operator_count);
try testing.expectEqual(@as(u32, 0), sink.tensor_spinor_projection_count);
try testing.expectEqual(@as(u32, 0), sink.tensor_form_projection_count);
try testing.expectEqual(@as(u32, 0), sink.cartan_product_count);
}

```

C.3 Tensor Rendering

C.3.1 Tests

```

test "rendering expands gamma product grades without dense tensors" {
    const testing = std.testing;

    const vector_square = try gammaProductExpansion(10, 1, 1, .none, .none);
    try testing.expectEqual(@as(u16, 2), vector_square.len);
    try testing.expectEqual(@as(u8, 2), vector_square.slice()[0].rank);
    try testing.expectEqual(@as(u8, 0), vector_square.slice()[0].contractions);
    try testing.expectEqual(@as(i64, 1), vector_square.slice()[0].coefficient);
    try testing.expectEqual(@as(u8, 0), vector_square.slice()[1].rank);
    try testing.expectEqual(@as(u8, 1), vector_square.slice()[1].contractions);
    try testing.expectEqual(@as(i64, 1), vector_square.slice()[1].coefficient);

    const vector_on_three_form = try gammaProductExpansion(10, 3, 1, .none, .none);
    try testing.expectEqual(@as(u16, 2), vector_on_three_form.len);
    try testing.expectEqual(@as(u8, 4), vector_on_three_form.slice()[0].rank);
    try testing.expectEqual(@as(i64, 1), vector_on_three_form.slice()[0].coefficient);
    try testing.expectEqual(@as(u8, 2), vector_on_three_form.slice()[1].rank);
    try testing.expectEqual(@as(u8, 1), vector_on_three_form.slice()[1].contractions);
    try testing.expectEqual(@as(i64, 3), vector_on_three_form.slice()[1].coefficient);

    const two_form_square = try gammaProductExpansion(10, 2, 2, .none, .none);
    try testing.expectEqual(@as(u16, 3), two_form_square.len);
    try testing.expectEqual(@as(u8, 4), two_form_square.slice()[0].rank);
    try testing.expectEqual(@as(i64, 1), two_form_square.slice()[0].coefficient);
    try testing.expectEqual(@as(u8, 2), two_form_square.slice()[1].rank);
    try testing.expectEqual(@as(u8, 1), two_form_square.slice()[1].contractions);
    try testing.expectEqual(@as(i64, 4), two_form_square.slice()[1].coefficient);
    try testing.expectEqual(@as(u8, 0), two_form_square.slice()[2].rank);
    try testing.expectEqual(@as(u8, 2), two_form_square.slice()[2].contractions);
    try testing.expectEqual(@as(i64, -2), two_form_square.slice()[2].coefficient);
}

test "rendering validates gamma product rank and duality" {
    const testing = std.testing;

    const middle = try gammaProductExpansion(10, 5, 1, .self_dual, .none);
    try testing.expectEqual(@as(u16, 2), middle.len);
    try testing.expectEqual(@as(u8, 4), middle.slice()[1].rank);
    try testing.expectEqual(DualityTag.none, middle.slice()[1].duality);

    try testing.expectError(error.InvalidGammaRank, gammaProductExpansion(10, 3, 1,
↪ .self_dual, .none));
    try testing.expectError(error.InvalidGammaRank, gammaProductExpansion(10, 11, 1, .none,
↪ .none));
}

```



```

}

test "rendering plans gamma product metric contractions" {
    const testing = std.testing;

    const expansion = try gammaProductExpansion(10, 3, 1, .none, .none);
    const contracted = expansion.slice()[1];
    const plan = try gammaProductContractionPlan(3, 1, contracted);
    try testing.expectEqual(@as(u16, 1), plan.metric_len);
    try testing.expectEqual(@as(u8, 2), plan.metricSlice()[0].left_offset);
    try testing.expectEqual(@as(u8, 0), plan.metricSlice()[0].right_offset);
    try testing.expectEqual(@as(u16, 2), plan.free_len);
    try testing.expectEqual(CliffordVectorSource.left, plan.freeVectorSlice()[0].source);
    try testing.expectEqual(@as(u8, 0), plan.freeVectorSlice()[0].offset);
    try testing.expectEqual(CliffordVectorSource.left, plan.freeVectorSlice()[1].source);
    try testing.expectEqual(@as(u8, 1), plan.freeVectorSlice()[1].offset);

    const scalar = (try gammaProductExpansion(10, 1, 1, .none, .none)).slice()[1];
    const scalar_plan = try gammaProductContractionPlan(1, 1, scalar);
    try testing.expectEqual(@as(u16, 1), scalar_plan.metric_len);
    try testing.expectEqual(@as(u16, 0), scalar_plan.free_len);

    try testing.expectError(error.InvalidGammaProductTerm, gammaProductContractionPlan(1, 1,
    ↪ .{
        .rank = 1,
        .contractions = 1,
        .coefficient = 1,
    }));
}

test "rendering decodes streamed clifford product atoms" {
    const testing = std.testing;

    const expansion = try gammaProductExpansion(10, 3, 1, .none, .none);
    const contracted = expansion.slice()[1];
    const plan = try gammaProductContractionPlan(3, 1, contracted);
    const product: CliffordProduct = .{
        .left_operator_id = 1,
        .right_operator_id = 2,
        .left_block = 3,
        .right_block = 3,
        .orthogonal_dimension = 10,
        .left_rank = 3,
        .right_rank = 1,
        .output_rank = contracted.rank,
        .contractions = contracted.contractions,
        .metric_count = plan.metric_len,
        .free_vector_count = plan.free_len,
        .coefficient = contracted.coefficient,
    };

    const decoded = cliffordProductTerm(product);
    try testing.expectEqual(contracted.rank, decoded.rank);
    try testing.expectEqual(contracted.contractions, decoded.contractions);
    try testing.expectEqual(contracted.coefficient, decoded.coefficient);
    const decoded_plan = try cliffordProductContractionPlan(product);
    try testing.expectEqual(plan.metric_len, decoded_plan.metric_len);
    try testing.expectEqual(plan.free_len, decoded_plan.free_len);
    try testing.expectEqual(@as(u16, 3), try cliffordProductFactorCount(product));

    const first_factor = try cliffordProductFactorAt(product, 0);
    try testing.expectEqual(std.meta.Tag(CliffordProductFactor).metric,
    ↪ std.meta.activeTag(first_factor));
    try testing.expectEqual(@as(IndexBlockId, 3), first_factor.metric.left_block);

```

```

try testing.expectEqual(@as(u8, 2), first_factor.metric.left_offset);
try testing.expectEqual(@as(u8, 0), first_factor.metric.right_offset);

const second_factor = try cliffordProductFactorAt(product, 1);
try testing.expectEqual(std.meta.Tag(CliffordProductFactor).output_vector,
↳ std.meta.activeTag(second_factor));
try testing.expectEqual(CliffordVectorSource.left, second_factor.output_vector.source);
try testing.expectEqual(@as(u8, 0), second_factor.output_vector.input_offset);
try testing.expectEqual(@as(u8, 0), second_factor.output_vector.output_offset);

try testing.expectError(error.CliffordProductFactorOutOfBounds,
↳ cliffordProductFactorAt(product, 3));

const FactorSink = struct {
    count: u32 = 0,
    metric_count: u32 = 0,
    output_vector_count: u32 = 0,
    first_atom_index: ?u32 = null,
    first_factor_index: ?u16 = null,
    first_output_rank: ?u8 = null,
    first_coefficient: ?i64 = null,

    fn emitCliffordProductFactor(self: *@This(), record: CliffordProductFactorRecord)
    ↳ !void {
        self.count += 1;
        if (self.first_atom_index == null) self.first_atom_index = record.atom_index;
        if (self.first_factor_index == null) self.first_factor_index =
        ↳ record.factor_index;
        if (self.first_output_rank == null) self.first_output_rank = record.output_rank;
        if (self.first_coefficient == null) self.first_coefficient = record.coefficient;
        switch (record.factor) {
            .metric => self.metric_count += 1,
            .output_vector => self.output_vector_count += 1,
        }
    }
};

const term_atoms = [_]SymbolicAtom{
    .{ .metric_pair = .{ .left = 10, .right = 11 } },
    .{ .clifford_product = product },
};

const term: SymbolicTerm = .{
    .coefficient = rationalOne(),
    .atoms = &term_atoms,
};

var factor_sink: FactorSink = .{};
try testing.expectEqual(@as(u32, 3), try streamCliffordProductFactors(term,
↳ &factor_sink));
try testing.expectEqual(@as(u32, 3), factor_sink.count);
try testing.expectEqual(@as(u32, 1), factor_sink.metric_count);
try testing.expectEqual(@as(u32, 2), factor_sink.output_vector_count);
try testing.expectEqual(@as(u32, 1), factor_sink.first_atom_index.?);
try testing.expectEqual(@as(u16, 0), factor_sink.first_factor_index.?);
try testing.expectEqual(product.output_rank, factor_sink.first_output_rank.?);
try testing.expectEqual(product.coefficient, factor_sink.first_coefficient.?);

var lowered_atoms: std.ArrayList(SymbolicAtom) = .empty;
defer lowered_atoms.deinit(testing.allocator);
try testing.expectEqual(@as(u32, 3), try
↳ appendLoweredCliffordProductAtoms(testing.allocator, &lowered_atoms, term));
try testing.expectEqual(@as(usize, 4), lowered_atoms.items.len);
try testing.expectEqual(std.meta.Tag(SymbolicAtom).metric_pair,
↳ std.meta.activeTag(lowered_atoms.items[0]));
try testing.expectEqual(std.meta.Tag(SymbolicAtom).clifford_product_factor,
↳ std.meta.activeTag(lowered_atoms.items[1]));

```



```

try testing.expectEqual(@as(u32, 1),
    ↪ lowered_atoms.items[1].clifford_product_factor.atom_index);
try testing.expectEqual(@as(u16, 0),
    ↪ lowered_atoms.items[1].clifford_product_factor.factor_index);
try testing.expectEqual(product.left_operator_id,
    ↪ lowered_atoms.items[1].clifford_product_factor.left_operator_id);
try testing.expectEqual(product.right_operator_id,
    ↪ lowered_atoms.items[1].clifford_product_factor.right_operator_id);
try testing.expectEqual(product.orthogonal_dimension,
    ↪ lowered_atoms.items[1].clifford_product_factor.orthogonal_dimension);
try testing.expectEqual(product.output_rank,
    ↪ lowered_atoms.items[1].clifford_product_factor.output_rank);
try testing.expectEqual(product.coefficient,
    ↪ lowered_atoms.items[1].clifford_product_factor.coefficient);
const lowered_term: SymbolicTerm = .{
    .coefficient = rationalOne(),
    .atoms = lowered_atoms.items,
};
var lowered_factor_sink: FactorSink = .{};
try testing.expectEqual(@as(u32, 3), try streamCliffordProductFactors(lowered_term,
    ↪ &lowered_factor_sink));
try testing.expectEqual(@as(u32, 3), lowered_factor_sink.count);
try testing.expectEqual(@as(u32, 1), lowered_factor_sink.metric_count);
try testing.expectEqual(@as(u32, 2), lowered_factor_sink.output_vector_count);
try testing.expectEqual(@as(u32, 1), lowered_factor_sink.first_atom_index.?);
try testing.expectEqual(@as(u16, 0), lowered_factor_sink.first_factor_index.?);
try testing.expectEqual(product.output_rank, lowered_factor_sink.first_output_rank.?);
try testing.expectEqual(product.coefficient, lowered_factor_sink.first_coefficient.?);

const compact_scan = try scanCliffordProductFactors(term);
try testing.expectEqual(@as(u32, 1), compact_scan.product_count);
try testing.expectEqual(@as(u32, 3), compact_scan.factor_count);
try testing.expectEqual(@as(u32, 1), compact_scan.metric_count);
try testing.expectEqual(@as(u32, 2), compact_scan.output_vector_count);
try testing.expectEqual(@as(u32, 1), compact_scan.first_atom_index.?);
try testing.expectEqual(@as(u32, 1), compact_scan.last_atom_index.?);
try testing.expectEqual(product.orthogonal_dimension,
    ↪ compact_scan.first_orthogonal_dimension.?);
try testing.expectEqual(product.output_rank, compact_scan.first_output_rank.?);
try testing.expectEqual(product.coefficient, compact_scan.first_coefficient.?);

const lowered_scan = try scanCliffordProductFactors(lowered_term);
try testing.expectEqual(compact_scan.product_count, lowered_scan.product_count);
try testing.expectEqual(compact_scan.factor_count, lowered_scan.factor_count);
try testing.expectEqual(compact_scan.metric_count, lowered_scan.metric_count);
try testing.expectEqual(compact_scan.output_vector_count,
    ↪ lowered_scan.output_vector_count);
try testing.expectEqual(compact_scan.first_atom_index.?, lowered_scan.first_atom_index.?);
try testing.expectEqual(compact_scan.first_orthogonal_dimension.?,
    ↪ lowered_scan.first_orthogonal_dimension.?);
try testing.expectEqual(compact_scan.first_output_rank.?,
    ↪ lowered_scan.first_output_rank.?);
try testing.expectEqual(compact_scan.first_coefficient.?,
    ↪ lowered_scan.first_coefficient.?);

var accumulated_scan: CliffordProductFactorScan = .{};
try accumulated_scan.recordTerm(term);
try accumulated_scan.recordTerm(lowered_term);
try testing.expectEqual(@as(u32, 2), accumulated_scan.product_count);
try testing.expectEqual(@as(u32, 6), accumulated_scan.factor_count);
try testing.expectEqual(@as(u32, 2), accumulated_scan.metric_count);
try testing.expectEqual(@as(u32, 4), accumulated_scan.output_vector_count);
try testing.expectEqual(compact_scan.first_atom_index.?,
    ↪ accumulated_scan.first_atom_index.?);

```

```

try testing.expectEqual(lowered_scan.last_atom_index.?,
↳ accumulated_scan.last_atom_index.?);
try testing.expectEqual(compact_scan.first_orthogonal_dimension.?,
↳ accumulated_scan.first_orthogonal_dimension.?);
try testing.expectEqual(compact_scan.first_output_rank.?,
↳ accumulated_scan.first_output_rank.?);
try testing.expectEqual(compact_scan.first_coefficient.?,
↳ accumulated_scan.first_coefficient.?);

var audit_sink: CliffordProductFactorAuditSink = .{};
try audit_sink.emitTerm(term);
try audit_sink.emitTerm(lowered_term);
try testing.expectEqual(@as(u32, 2), audit_sink.term_count);
try testing.expectEqual(@as(u64, @intCast(term.atoms.len + lowered_term.atoms.len)),
↳ audit_sink.atom_count);
try testing.expectEqual(accumulated_scan.product_count, audit_sink.scan.product_count);
try testing.expectEqual(accumulated_scan.factor_count, audit_sink.scan.factor_count);
try testing.expectEqual(accumulated_scan.metric_count, audit_sink.scan.metric_count);
try testing.expectEqual(accumulated_scan.output_vector_count,
↳ audit_sink.scan.output_vector_count);

var lowering_sink: AtomLoweringAuditSink = .{};
try lowering_sink.emitTerm(term);
try lowering_sink.emitTerm(lowered_term);
try testing.expectEqual(@as(u64, 2), lowering_sink.audit.term_count);
try testing.expectEqual(@as(u64, 6), lowering_sink.audit.atom_count);
try testing.expectEqual(@as(u64, 5), lowering_sink.audit.primitive_atoms);
try testing.expectEqual(@as(u64, 1), lowering_sink.audit.compact_formula_atoms);
try testing.expectEqual(@as(u64, 1), lowering_sink.audit.clifford_product_atoms);
try testing.expectEqual(@as(u64, 3), lowering_sink.audit.clifford_product_factor_atoms);
try testing.expectEqual(SymbolicAtomTag.clifford_product,
↳ lowering_sink.audit.first_compact_kind.?);
try testing.expect(!lowering_sink.audit.gammaOnly());

var gamma_only_audit: AtomLoweringAudit = .{};
try gamma_only_audit.recordTerm(lowered_term);
try testing.expect(gamma_only_audit.gammaOnly());

const structural_atoms = [_]SymbolicAtom{.{ .spinor_tower_contract = .{
    .operator_id = 7,
    .input_tower = 1,
    .spinor = 2,
    .output_tower = 3,
    .orthogonal_dimension = 10,
    .input_power = 4,
    .output_power = 3,
    .form_rank = 1,
    .chirality = 2,
} }};
var structural_audit: AtomLoweringAudit = .{};
try structural_audit.recordTerm(.{
    .coefficient = rationalOne(),
    .atoms = &structural_atoms,
});
try testing.expect(!structural_audit.gammaOnly());
try testing.expectEqual(@as(u64, 1), structural_audit.structural_atoms);
try testing.expectEqual(@as(u64, 1), structural_audit.spinor_tower_contract_atoms);
try testing.expectEqual(SymbolicAtomTag.spinor_tower_contract,
↳ structural_audit.first_structural_kind.?);

var malformed = lowered_atoms.items[1];
malformed.clifford_product_factor.factor_index = 1;
const malformed_atoms = [_]SymbolicAtom{malformed};

```

```

try testing.expectError(error.InvalidCliffordProductFactorSequence,
↳ scanCliffordProductFactors({
    .coefficient = rationalOne(),
    .atoms = &malformed_atoms,
}));

var inconsistent = product;
inconsistent.metric_count += 1;
try testing.expectError(error.InvalidGammaProductTerm,
↳ cliffordProductContractionPlan(inconsistent));
}

```

References

Bibliography

- [1] Joseph J. Atick and Ashoke Sen. “Covariant One-Loop Fermion Emission Amplitudes in Closed String Theories”. In: *Nuclear Physics B* 293 (1987), pp. 317–347. DOI: 10.1016/0550-3213(87)90074-4.
- [2] Xi Yin. *String Theory*. 2026. URL: <https://github.com/xiyin137/stringbook> (visited on 06/06/2026).
- [3] Daniel Harry Friedan. “Nonlinear Models in $2 + \epsilon$ Dimensions”. In: *Annals of Physics* 163.2 (1985), pp. 318–419. DOI: 10.1016/0003-4916(85)90384-7.
- [4] Curtis G. Callan et al. “Strings in Background Fields”. In: *Nuclear Physics B* 262.4 (1985), pp. 593–609. DOI: 10.1016/0550-3213(85)90506-1.
- [5] Arkady A. Tseytlin. “Conformal Anomaly in Two-Dimensional Sigma Model on Curved Background and Strings”. In: *Physics Letters B* 178 (1986), p. 34. DOI: 10.1016/0370-2693(86)90465-X.
- [6] Arkady A. Tseytlin. “Sigma Model Weyl Invariance Conditions and String Equations of Motion”. In: *Nuclear Physics B* 294 (1987), pp. 383–411. DOI: 10.1016/0550-3213(87)90588-8.
- [7] R. R. Metsaev and Arkady A. Tseytlin. “Order Alpha-Prime Two Loop Equivalence of the String Equations of Motion and the Sigma Model Weyl Invariance Conditions: Dependence on the Dilaton and the Antisymmetric Tensor”. In: *Nuclear Physics B* 293 (1987), pp. 385–419. DOI: 10.1016/0550-3213(87)90077-0.
- [8] David J. Gross and Edward Witten. “Superstring Modifications of Einstein’s Equations”. In: *Nuclear Physics B* 277 (1986), pp. 1–10. DOI: 10.1016/0550-3213(86)90429-3.
- [9] Marcus T. Grisaru, A. E. M. van de Ven, and D. Zanon. “Four Loop Divergences for the $N=1$ Supersymmetric Nonlinear Sigma Model in Two-Dimensions”. In: *Nuclear Physics B* 277 (1986), pp. 409–428. DOI: 10.1016/0550-3213(86)90449-9.
- [10] Marcus T. Grisaru, A. E. M. van de Ven, and D. Zanon. “Two-Dimensional Supersymmetric Sigma Models on Ricci Flat Kahler Manifolds Are Not Finite”. In: *Nuclear Physics B* 277 (1986), pp. 388–408. DOI: 10.1016/0550-3213(86)90448-7.
- [11] Marcus T. Grisaru, A. E. M. van de Ven, and D. Zanon. “Four Loop Beta Function for the $N=1$ and $N=2$ Supersymmetric Nonlinear Sigma Model in Two-Dimensions”. In: *Physics Letters B* 173 (1986), pp. 423–428. DOI: 10.1016/0370-2693(86)90408-9.
- [12] Q. Park and D. Zanon. “More on Sigma-Model Beta Functions and Low-Energy Effective Actions”. In: *Physical Review D* 35 (1987), p. 4038. DOI: 10.1103/PhysRevD.35.4038.

- [13] Ignatios Antoniadis et al. “R4 Couplings in M and Type II Theories on Calabi-Yau Spaces”. In: *Nuclear Physics B* 507 (1997), pp. 571–588. DOI: 10.1016/S0550-3213(97)00572-5. arXiv: hep-th/9707013.
- [14] James T. Liu and Ruben Minasian. “Higher-Derivative Couplings in String Theory: Five-Point Contact Terms”. In: *Nuclear Physics B* 967 (2021), p. 115386. DOI: 10.1016/j.nuclphysb.2021.115386. arXiv: 1912.10974 [hep-th].
- [15] Thomas W. Grimm, Kilian Mayer, and Matthias Weissenbacher. “Higher Derivatives in Type II and M-Theory on Calabi-Yau Threefolds”. In: *Journal of High Energy Physics* 02 (2018), p. 127. DOI: 10.1007/JHEP02(2018)127. arXiv: 1702.08404 [hep-th].
- [16] Philip Candelas et al. “A Pair of Calabi-Yau Manifolds as an Exactly Soluble Superconformal Theory”. In: *Nuclear Physics B* 359 (1991), pp. 21–74. DOI: 10.1016/0550-3213(91)90292-6.
- [17] James T. Liu and Ruben Minasian. “Higher-Derivative Couplings in String Theory: Dualities and the B-Field”. In: *Nuclear Physics B* 874 (2013), pp. 413–470. DOI: 10.1016/j.nuclphysb.2013.06.002. arXiv: 1304.3137 [hep-th].
- [18] Giuseppe Policastro and Dimitrios Tsimpis. “R4, Purified”. In: *Classical and Quantum Gravity* 23 (2006), pp. 4753–4780. DOI: 10.1088/0264-9381/23/14/012. arXiv: hep-th/0603165.
- [19] Philip Candelas and Xenia de la Ossa. “Moduli Space of Calabi-Yau Manifolds”. In: *Nuclear Physics B* 355 (1991), pp. 455–481. DOI: 10.1016/0550-3213(91)90122-E.